Object Oriented Analysis and Design

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

Object Oriented Analysis and Design

Overview of the Course

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

What we Know?

Traditional SDLC

- We have seen in Software Engineering course, the design step falls between understanding your requirements and building the product.

Structured Systems Analysis & Design(SSAD)

Structured Systems Analysis & Design (SSAD) is a framework of activities and tasks that need to be accomplished to develop an information system. This approach is also known as top-down design, modular programming, and stepwise refinement.

What will we study?

What is OOAD?

One approach to help make the design process easier is the object-oriented(OO) approach. This allows for the description of **concepts** in the problem and solution spaces as **objects.**

What is Object Oriented Approach?

- In software development lifecycle we can apply and implement OO concepts by following three steps.

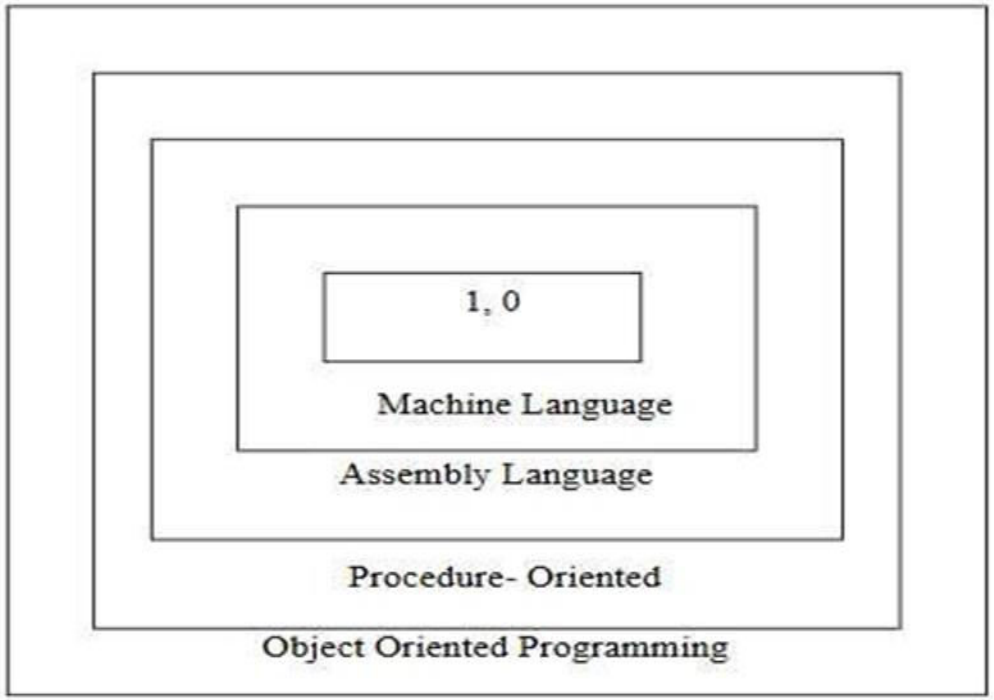
OO Analysis --> OO Design --> OO implementation by using OO languages(Java)

Object Oriented Analysis and Design

Need for Object-oriented Approach

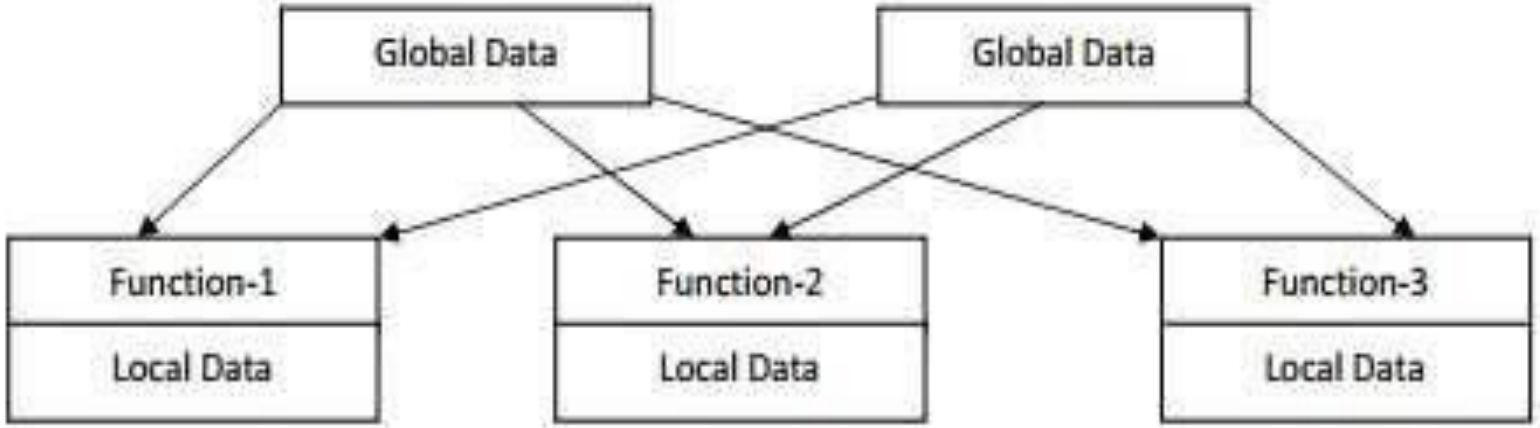
1. **Real-Life Entity Representation:**
   - Directly models real-life entities using objects.
2. **Open Interface Design:**
   - Emphasizes well-defined interfaces for adaptable system architectures.
3. **Improved Software Quality:**
   - Enforces encapsulation and abstraction for cleaner, maintainable code.
4. **Reusability and Extensibility:**
   - Promotes modularity and inheritance, ensuring reusable, extensible systems.
5. **Modules Tolerant of Changes:**
   - Encapsulation allows changes without affecting external interfaces.
6. **Improved Software Productivity:**
   - Facilitates concurrent development for increased productivity.
7. **Decreased Software Costs:**
   - Modular and reusable structure reduces maintenance costs.

Software Evolution concepts: Programming Knowledge



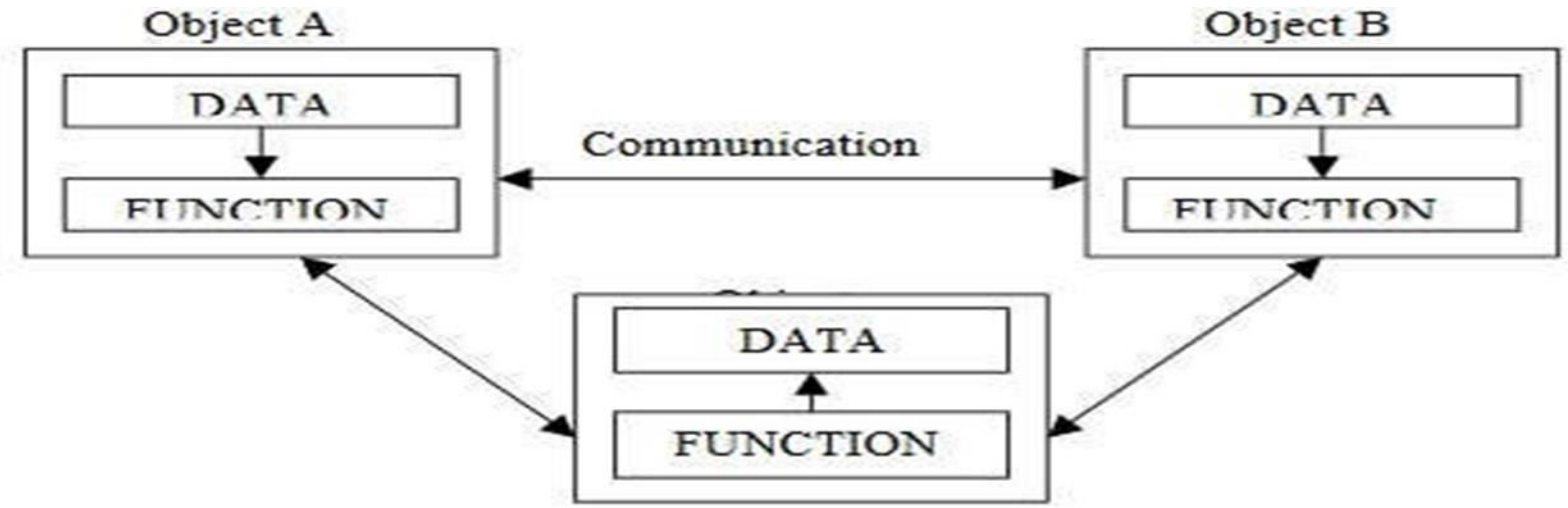
Procedure-oriented programming

- Conventional programming, using high level languages such as COBOL, FORTRAN and C, is commonly known as procedure- oriented programming (POP) .
- In the procedure-oriented approach, the problem is viewed as a sequence of things to be done such as reading, calculating and printing. A number of functions are written to accomplish these tasks .
- The primary focus is on **functions** :



Object-oriented Programming

- Emphasis is on data rather than procedure.
- Programs are divided into what are known as objects.
- Data may be hidden and may not be accessed by external functions.
- Objects may communicate with each other through functions.
- New data and functions can be easily added whenever necessary.
- Follows bottom-up approach in program design



Object-based Programming

Feature	Object-Based Programming(OBP)	Object-Oriented Programming (OOP)
Definition	Focuses on objects, but may not support inheritance and polymorphism.	Emphasizes objects and supports inheritance and polymorphism.
Inheritance	May not support inheritance or supports limited forms.	Supports inheritance, allowing the creation of new classes based on existing ones.
Polymorphism	Limited or no support for polymorphism.	Supports polymorphism, allowing objects of different types to be treated as objects of a common base type.
Encapsulation	Generally supports encapsulation, but may not enforce it strictly.	Enforces encapsulation by bundling data and methods that operate on the data within a single unit (class).
Abstraction	Provides basic abstraction through objects.	Offers higher-level abstraction by allowing the creation of abstract classes and interfaces.
Example Language	JavaScript, early versions of Visual Basic.	Java, C++, Python.

Syllabus

PES UNIVERSITY
CELEBRATING 50 YEARS

Unit 1: **Object Oriented Design and UML Diagrams - Requirements, Modeling and Analysis**

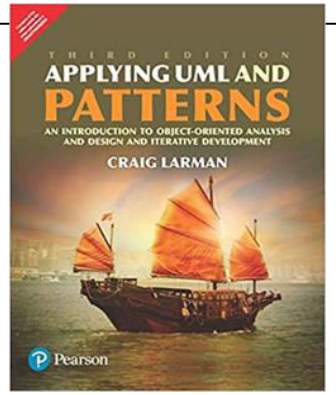
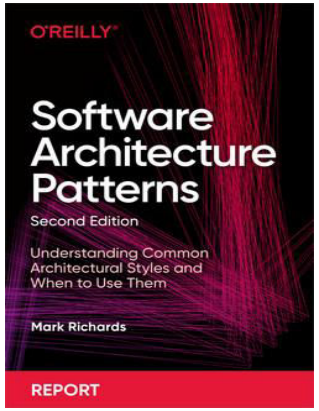
Unit 2: **Object Oriented Programming and Architecture design**

Unit 3: **Design Principles and Patterns**

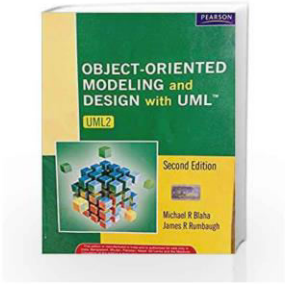
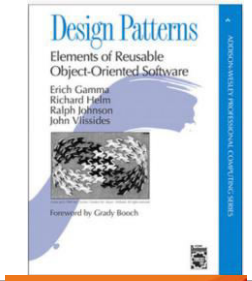
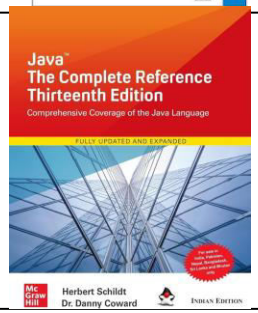
Unit 4: **OO Design Patterns & Anti-Patterns**

Object Oriented Analysis and Design

Text Books

T1: “Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development”, by Craig Larman, 3rd Edition, Pearson 2015.	
T2: “Software Architecture Patterns” by Mark Richards, 2nd Edition, OReilly 2022	

Reference Books

R1: "Object-Oriented Modelling and Design with UML", Michael R Blaha and James R Rumbaugh, 2nd Edition, Pearson 2007.	
R2: "Design Patterns: Elements of Reusable Object-Oriented Software" by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides, 1st Edition, Pearson 2015	
R3: "Java the Complete Reference", Herbert Schildt ,McGraw-Hill ,11th Edition, 2018.	

ISA Evaluation Policy

Theory				
Evaluation	**Count**	**Hours**	**Weight-age**	**Total Weight-age**
CBT	2	-	20	40
Assignment	2	4-6 hrs	10	10
Theory Total (ISA)				**50**
Lab				
Lab Assignment	9	18	10	10
Mini- Project	1	15 hrs (approx)	10	10
Lab Total (FSA)				**20**

ISA – Theory Assignments

Assignment 1 – Banana Problem (Week 4)	Class Modeling	3 marks	Rubrics: - Identification of all the required classes – 2 mark - Partial identification of classes – 1 mark - Identification of all relationships, their types and cardinality – 2 mark - Partial identification of relationships, their types and cardinality – 1 mark
Assignment 2 – Orange Problem (Week 13)	Case Study on Design Principles	7 marks	Rubrics: • Identification of the appropriate Design Principle(s) – 2 mark • Usage of any other design pattern – 1 mark • Class Diagram for the applied Design Pattern – 2 mark • Partially complete class diagram – 1 mark • Implementation using Java – 2 marks • Partial implementation using Java – 1 mark

FSA – Lab Assignments

Lab Assignment contains 3 components:

- Lab Assignments – each evaluated for 10 Marks
  - UML Modeling & Design Pattern Case Study (7 Nos.)
  - Java Programming (2 No.)
- Mini Project (1 No. - 9 Hrs in Lab + 6 Hrs outside Lab)
- Viva – for 10 Marks

Lab-1: Online UML Tools Demo, java Installation
Lab-2 : Case Study on Usecase Diagram
Lab-3 : Case Study on Class Diagram
Lab-4 : Case Study on Activity Diagram Mini Project - Use case Diagram
Lab-5 : Case Study on State Modeling, Mini project - Class Modeling
Lab-6 : Interfaces & Method Overloading
Lab-7 : Inheritance & Abstract Class
Lab - 8 : MVC pattern implementation
Lab-9 - Mini project - Detailed Class Diagram
Lab-10 - Mini project -Activity & State Diagram
Lab-11 – Mini Project – MVC implementation
Lab-12 - Creational Design Pattern
Lab-13 - structural Design pattern
Lab-14 - Mini Project Implementation

FSA – Lab Assignments

Mini Project Evaluation Policy:

1: Presentation of the project will be team-based.

2: Each student will be evaluated individually.

Marks Distribution:

Analysis and Design Models	2 marks
Use of MVC Architecture Pattern	2 mark
Use of Design Principles and Patterns (at least 1each per team member)	3 marks
Presentation/ Demo/ Explaining the code	3 marks
Total	**10 marks**

Note: * - 4 Design patterns and 4 design principles in total for the project for a 4 member team

THANK YOU

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

vinayj@pes.edu

Object Oriented Analysis and Design

Prof. Vinay Joshi
Department of Computer Science and Engineering

Requirement Engineering

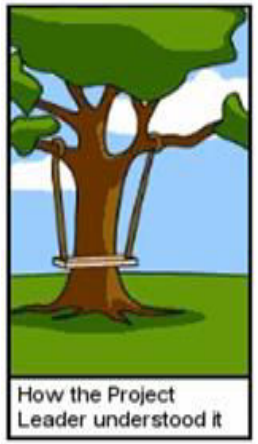
RE is usually the first step in any software intensive development lifecycle irrespective of model

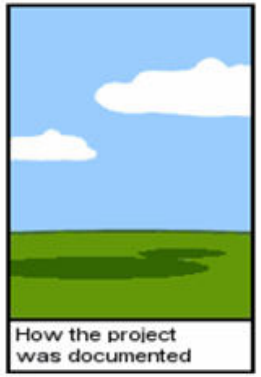
- Usually difficult, error prone and costly
- Critical for successful completion of all downstream activities
- Errors introduced during requirements phase if not handled properly and early will propagate into the subsequent phases
- Requirement errors are expensive to fix at a later stage
- Unnecessary, late or invalid requirements can make the system cumbersome or slip

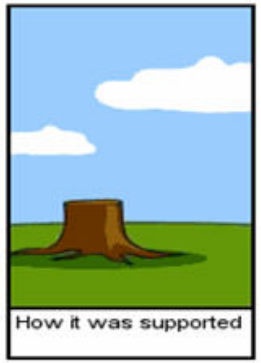
PES UNIVERSITY
CELEBRATING 50 YEARS

Object Oriented Analysis and Design

Requirement Engineering







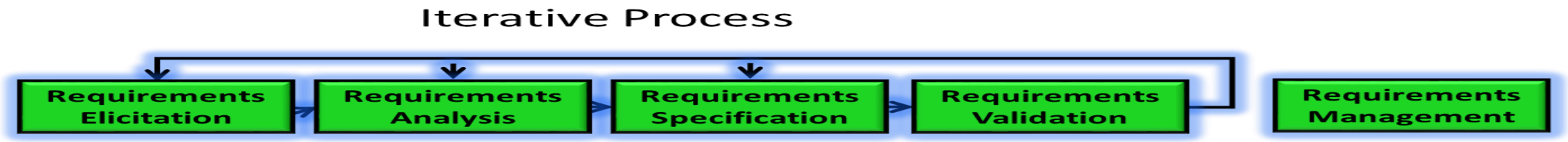
Requirement Engineering

- Requirement is the Property which must be exhibited by software developed/adapted to solve a particular problem
- Requirement should specify the externally visible behavior of **what** and **not how**
- Requirements can be looked at as
  - Individual requirements
  - Set of requirements

Requirement Engineering

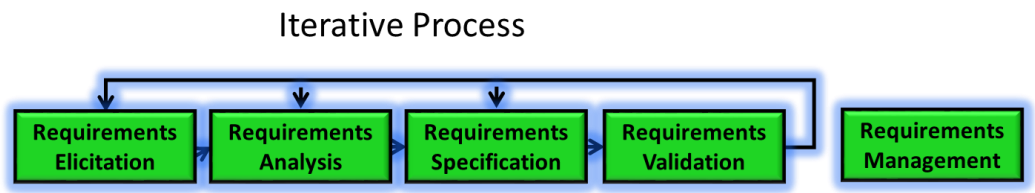
Is a process of working proactively with all stakeholders gathering their needs, articulating their problem, identify and negotiate potential conflicts thereby **establishing a clear scope and boundary for a project.**

It can also be described as a process of ensuring that the stakeholders have been identified and they have been given an **opportunity to explain their problem and needs** and describe what they would like the new system to do.



Object Oriented Analysis and Design

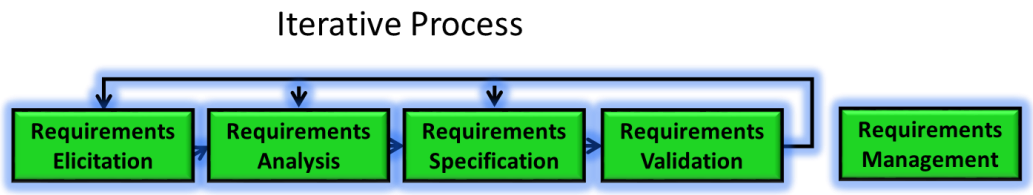
Requirement Analysis Process



1. **Understand the requirements in depth, both from a product and process perspective**
2. **Classify and Organize the requirements into coherent clusters**
   - **Functional, Non-Functional & Domain requirements**
   - **System and User Requirements**
3. **Model the requirements**
4. **Analyze the requirements (if necessary) using fish bone diagram**

Object Oriented Analysis and Design

Requirement Analysis Process



5. Recognize and resolve conflicts (e.g., functionality v. cost v. timeliness)
6. Negotiate Requirements
7. Prioritize the requirements (MoSCoW -Must have, Should have, Could have, Wont have)
8. Identify risks if any
9. Decide on Build or Buy (Commercial Of The Shelf Solution) and refine requirements

Requirement Modelling

A Model is a representation of a system in some form.
A is a Model of B if A can be used to answer questions about B
Part of Requirement Analysis and Specification phases.

Couple of important goals of Modelling

- Providing an Understanding (existing) System
  - Analyzing and Validating the requirements in terms of visible requirements within the problem
- Communicating the requirements in terms of who, what and interpreting it in the same way

- **Discussed different kinds of Models**
  - Structural Models and Behavioral models

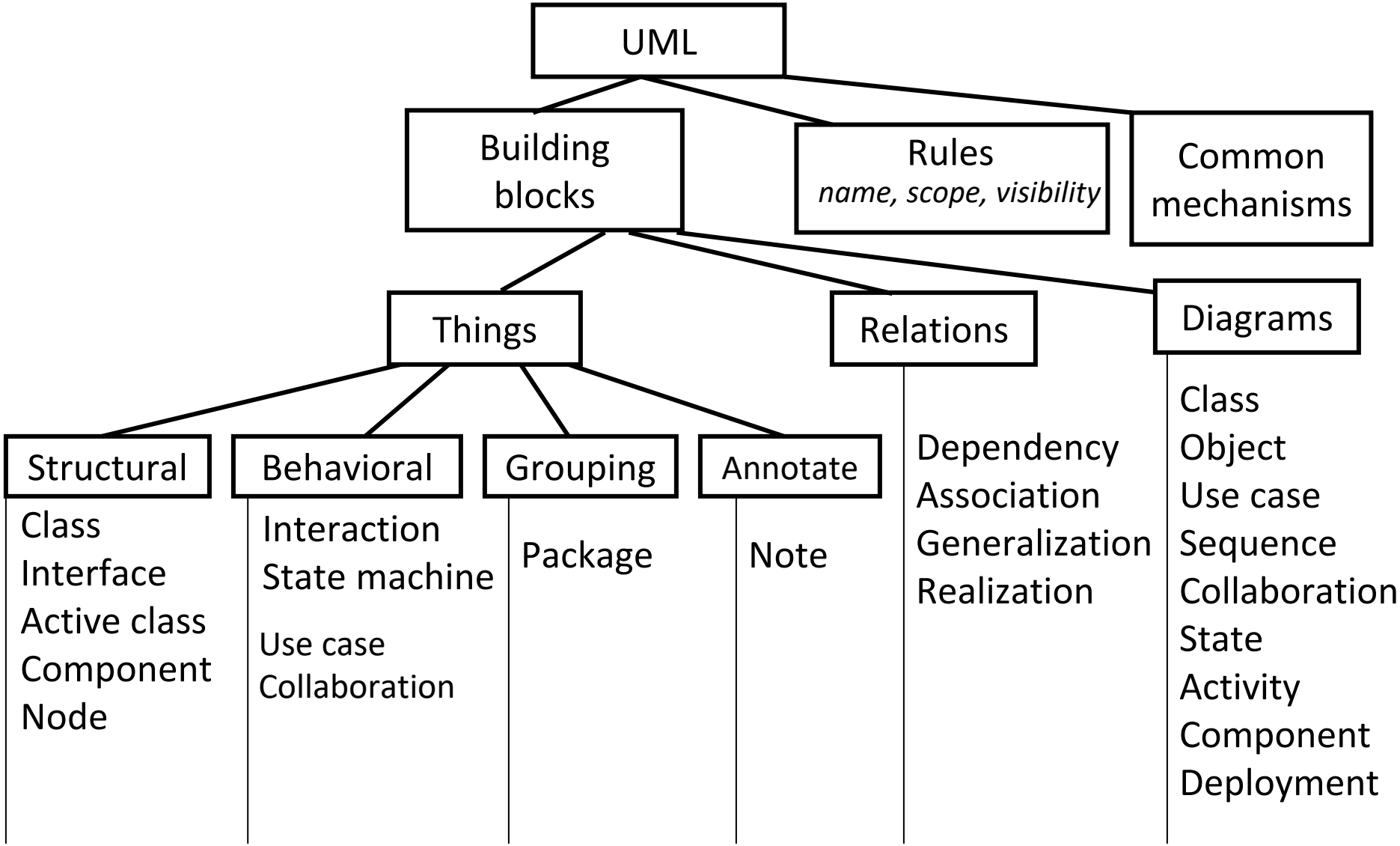
What is UML?

- **Unified Modeling Language**
  - Language to express different types of models
  - Language defines
  - Syntax – Symbols and rules for using them
  - Semantics – What these symbols represent
- **UML can be used to**
  - Visualize (Graphical Notation)
  - Specify (Complete and Unambiguous)
  - Construct (Code Generation)
  - Document (Design, Architecture, etc.)

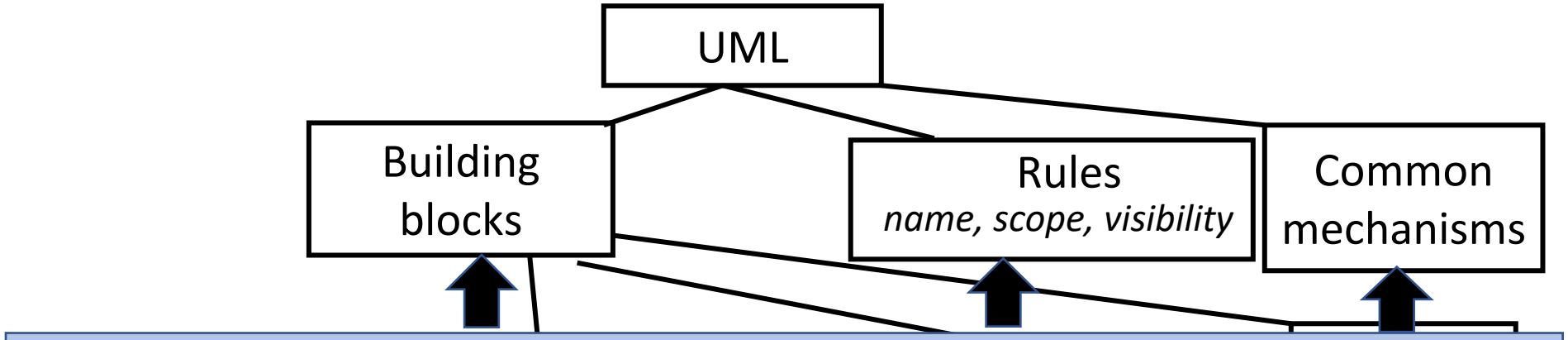
  the artifacts of software-intensive systems

PES UNIVERSITY
CELEBRATING 50 YEARS

Conceptual Models of UML



Conceptual Models of UML

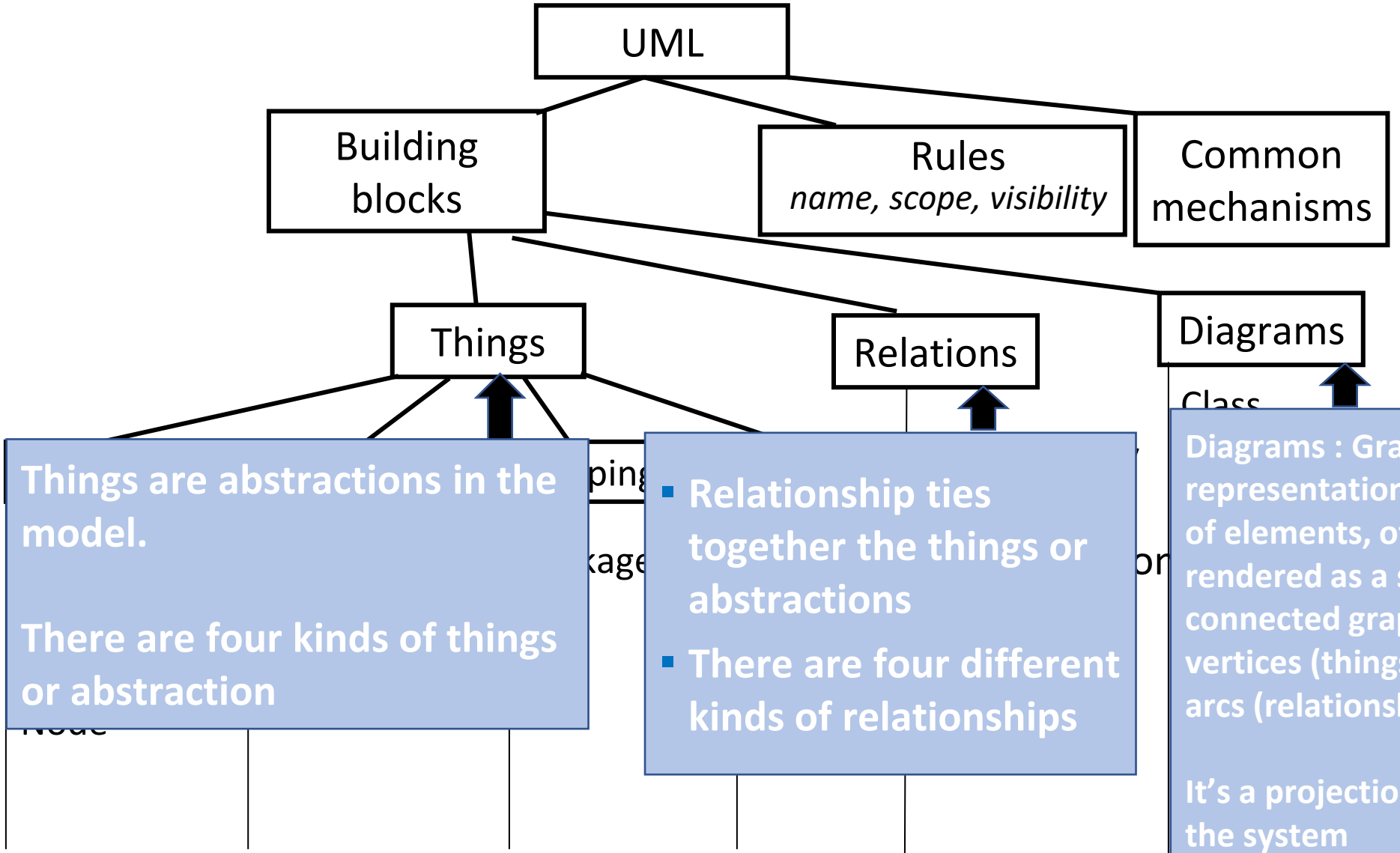


UML is made up of three conceptual elements. They are

- **Building blocks which make up the UML**
- **Rules that dictate how these building blocks can be put together**
- **Common mechanisms that apply through out UML**

Interface Active class Component Node	State machine Use case Collaboration			Realization	Collaboration State Activity Component Deployment

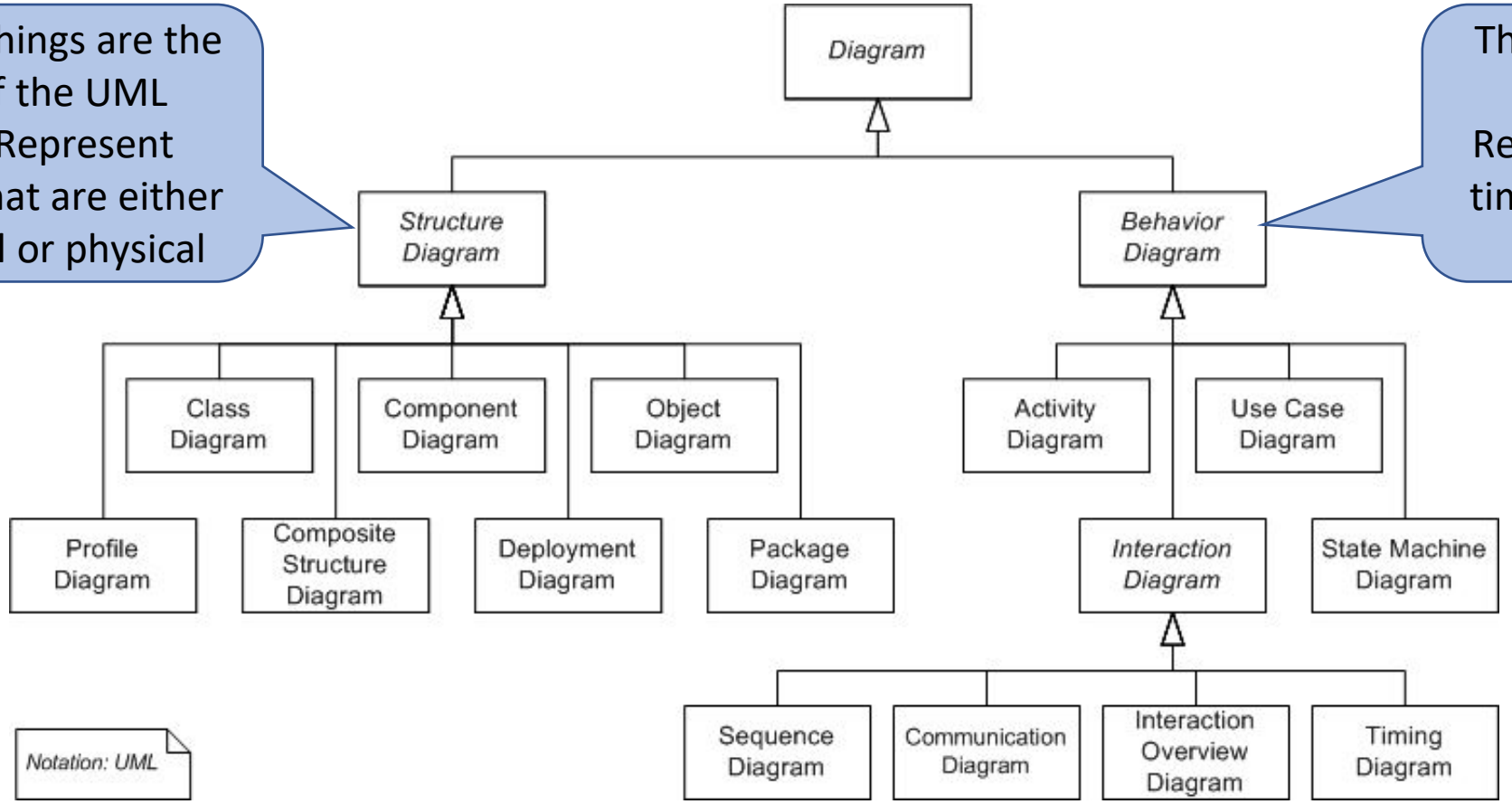
Conceptual Models of UML



Object Oriented Analysis and Design

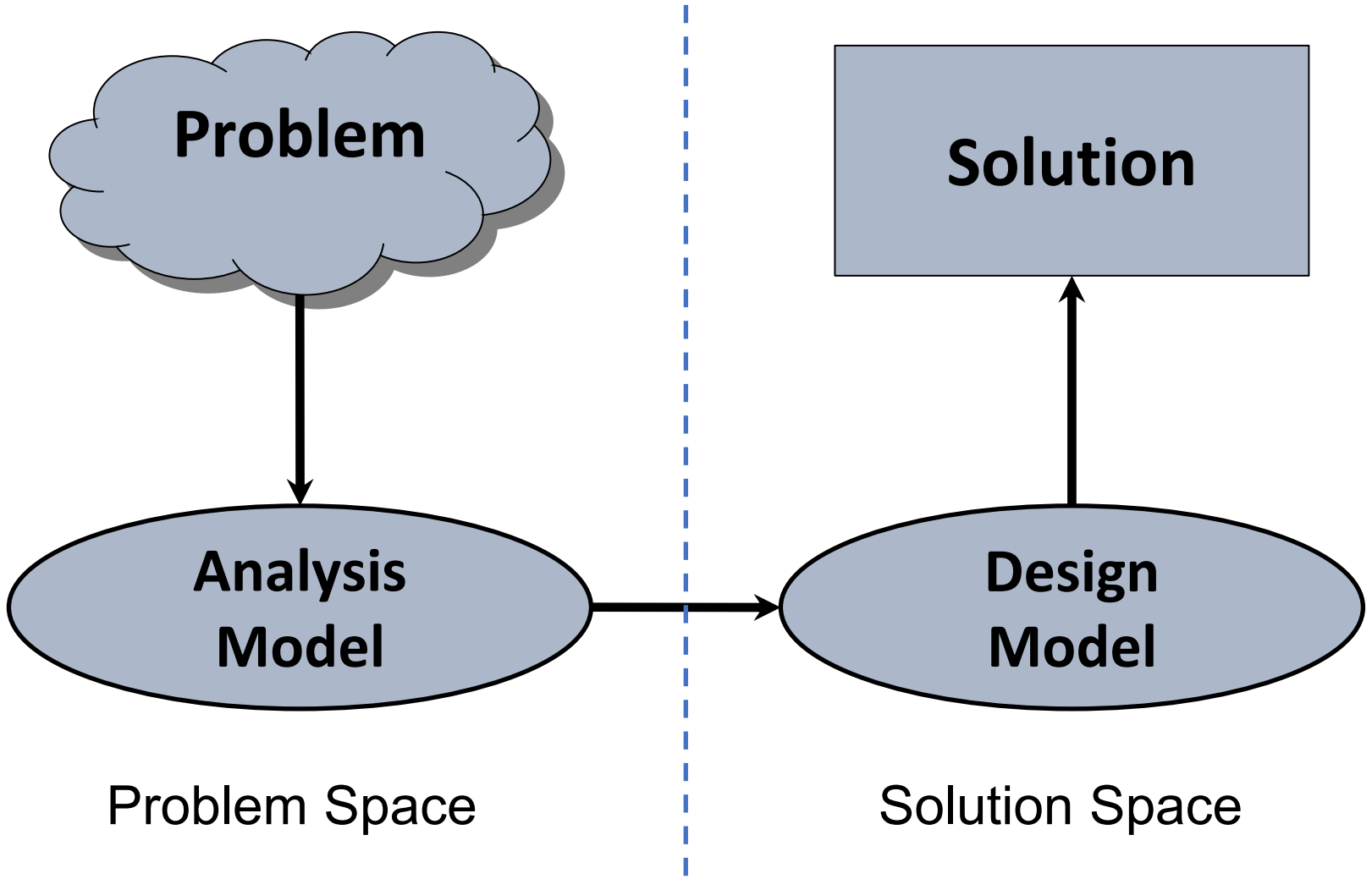
UML Diagrams

Structural things are the nouns of the UML models. Represent elements that are either conceptual or physical

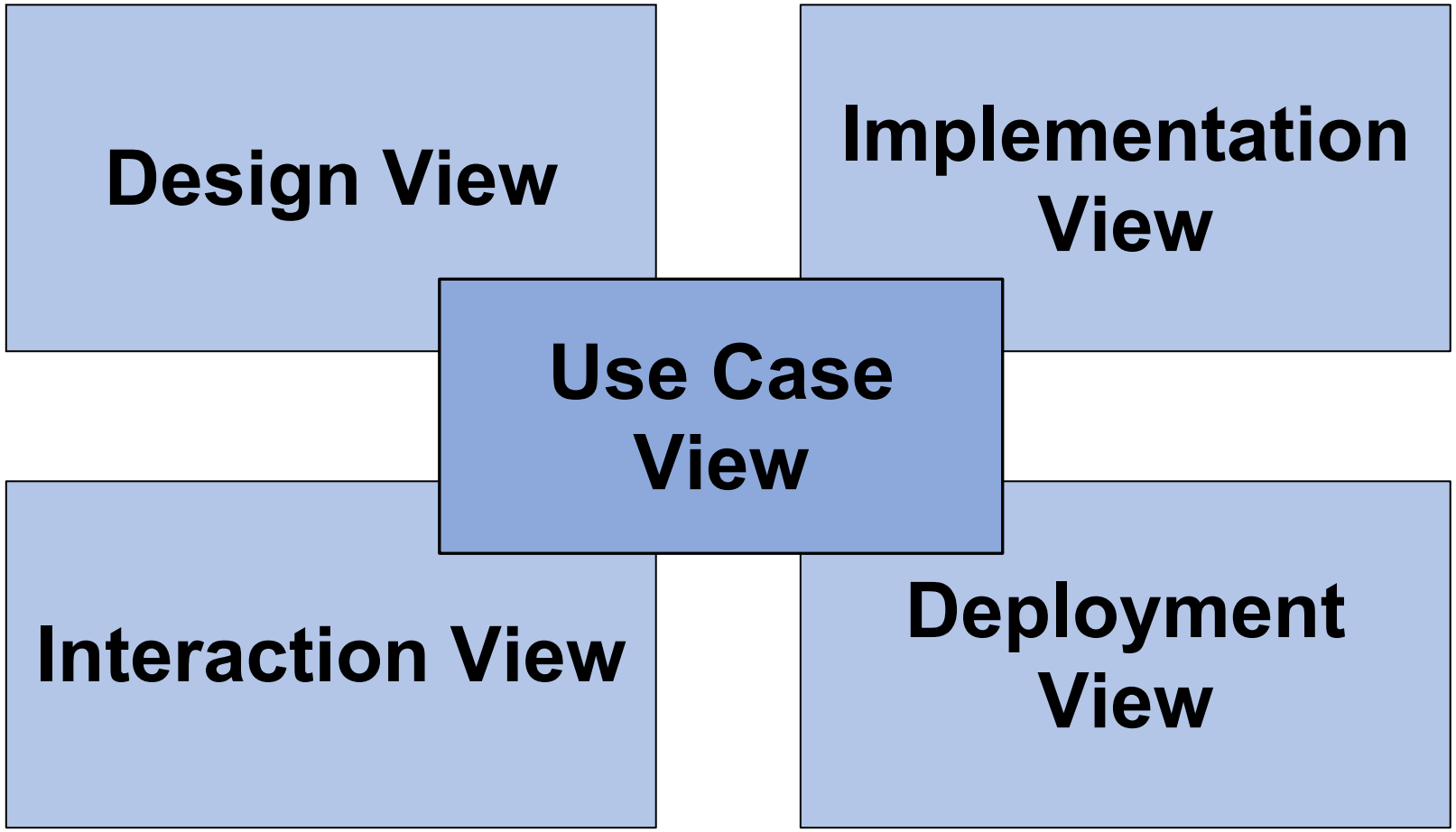


These are dynamic parts of the UML models. Represent behavior over time and space. Typically verbs of the model

Models in Solution Development



Views of a Solution System



Views of a Solution System

- Use Case View
  - Describes behavior of system as seen by end users, analysts, and testers
- Design View
  - Describes Classes, Interfaces, and Collaborations that form the vocabulary of the problem and its solution
- Interaction View
  - Describes flow of control among its various parts
  - Address performance, scalability, and throughput

PES UNIVERSITY
CELEBRATING 50 YEARS

Views of a Solution System

- Implementation View
  - Describes artifacts that are used to assemble and release the physical system
  - Address configuration management of system releases

- Deployment View
  - Address the distribution, delivery, and installation of the parts that make up the physical system

PES UNIVERSITY
CELEBRATING 50 YEARS

THANK YOU

Vinay Joshi
Department of Computer Science and Engineering

vinayj@pes.edu

Object Oriented Analysis and Design

Prof. Vinay Joshi
Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

Object Oriented Analysis and Design

Use case Modeling: Use case Diagrams

Prof. Vinay Joshi

Department of Computer Science and Engineering

Use case Modeling - Agenda

- Introduction To Use case Modelling
- Use case diagrams
- Use case
- Actor
- Use case description
- Example use case diagrams
- Relation between these use case and Actor
- Relation between the use cases
- Use case specification
- Practice: ATM
- Few inputs

PES UNIVERSITY
CELEBRATING 50 YEARS

Use case Modeling: Introduction

- Describes the **interaction of users and the system**
- Describes **what functionality does a system provides to its users**.
- Use case model has **two important elements** - **actors** and **use cases**.

Actor/s: One or set of objects who directly interacts with the system

Every actor has a defined purpose while interacting with the system.

An actor can be a person, device or another system.

Use case: A piece of functionality that a system offers to its users.

Set of all use cases defines the entire functionality of the system.

Also define the error conditions that may occur while interacting with the system

Use case Diagrams

- Incorporates both **actor and use cases and also the relationship**

  between them in the **graphical representation**.

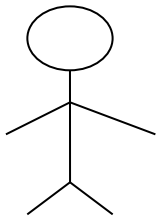
- Used to visualize, specify, construct, and document the (intended) behavior of the system, during requirements capture and analysis.
- **Provide a way for developers, domain experts and end-users to Communicate.**
- **Serve as basis for testing**

Use case

- Use cases **specify the desired behavior**.
- A use case is
  - a description of a set of sequences of actions a system performs to yield an observable result of value to an actor.
  - includes variants
- Name starts with a verb.
- Each sequence represent an interaction of actors with the system.

Actor

- Represents a set of roles that users of use case play when interacting with these use cases.
- Can **be human or automated systems.**
- Actor is someone interacting with use case (system function). **Named by noun.**
- Actors are entities which require help from the system to perform their task or are needed to execute the system's functions.
- Actors are not part of the system.
- An **Actor triggers use case**
- Actor has responsibility toward the system (inputs), and have expectations from the system (outputs).



name

<< actor >>
X Authorization System

How to create Use case diagrams?

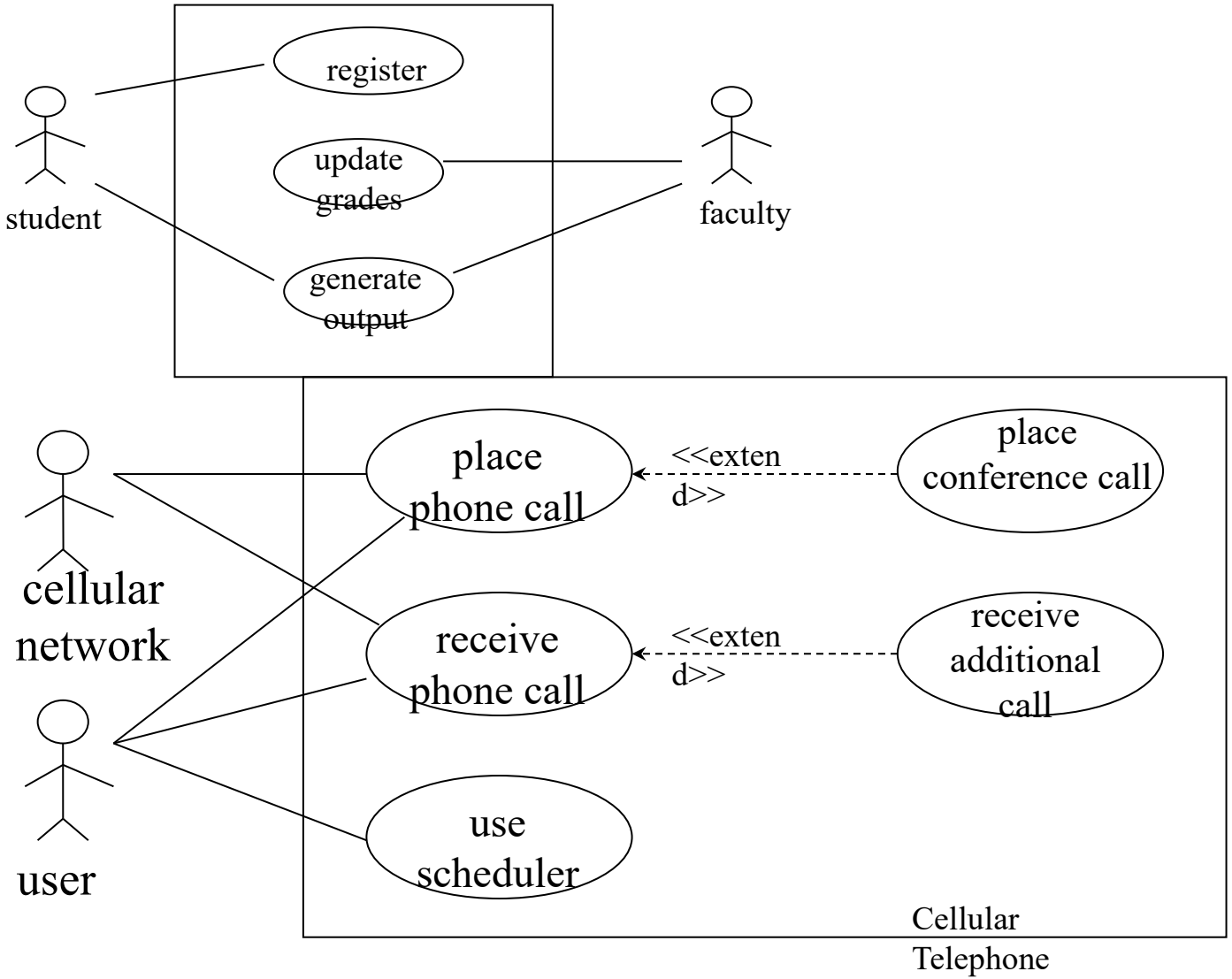
- List main system functions (use cases) in a column
- Draw ovals around the function labels
- Draw system boundary
- Draw actors and connect them with use cases
- Specify include and extend relationships between use cases

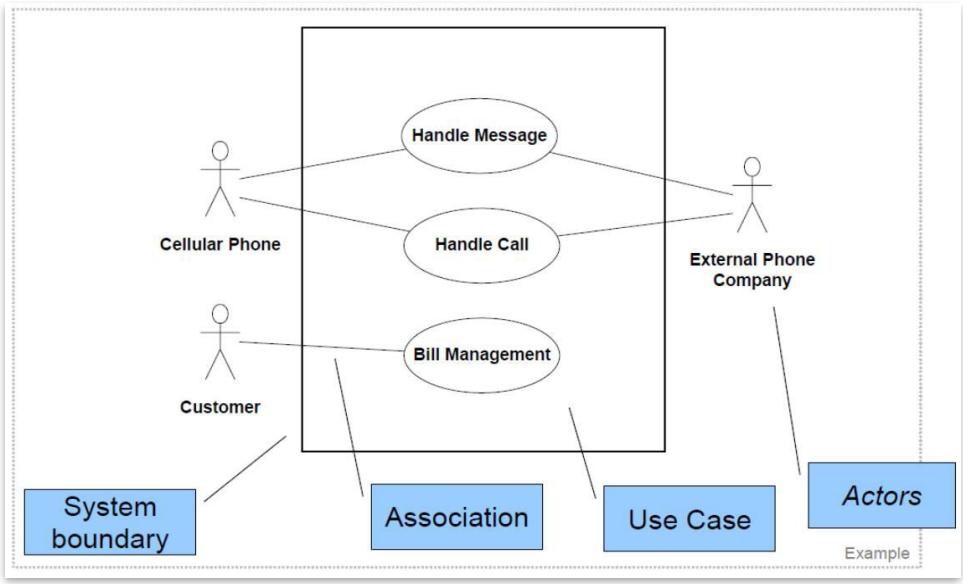
Use case description

Each use case may include all or part of the following

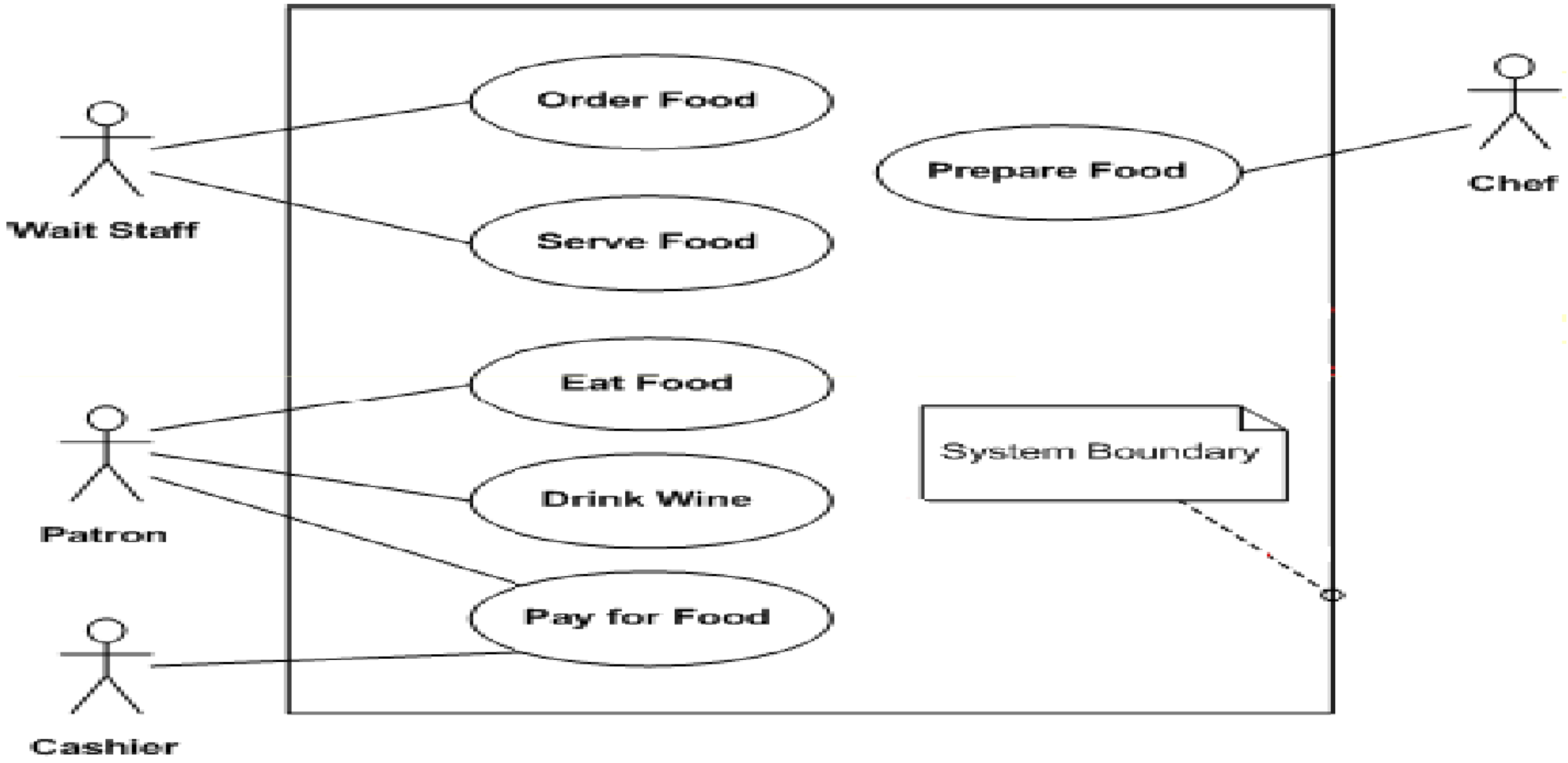
- Title or Reference Name - meaningful name of the UC
- Author/Date - the author and creation date
- Modification/Date - last modification and its date
- Purpose - specifies the goal to be achieved
- Overview - short description of the processes
- Cross References - requirements references
- Actors - agents participating
- Pre Conditions - must be true to allow execution
- Post Conditions - will be set when completes normally
- Normal flow of events - regular flow of activities
- Alternative flow of events - other flow of activities
- Exceptional flow of events - unusual situations
- Implementation issues - foreseen implementation problems

Example use case diagram



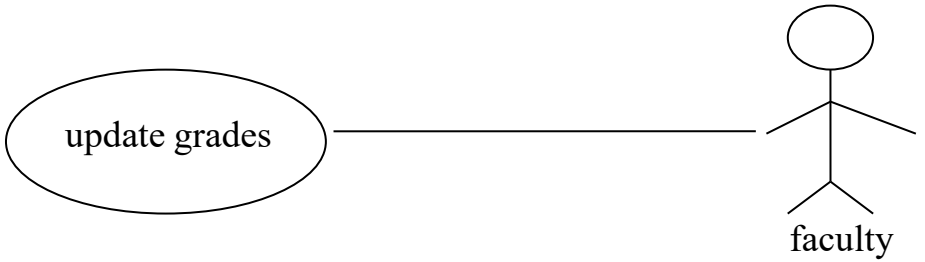


Example use case diagram for a restaurant



Relationships between Use Cases and Actors

- Actors may be connected to use cases by associations, indicating that the actor and the use case communicate with one another using messages.



Relationship between use cases

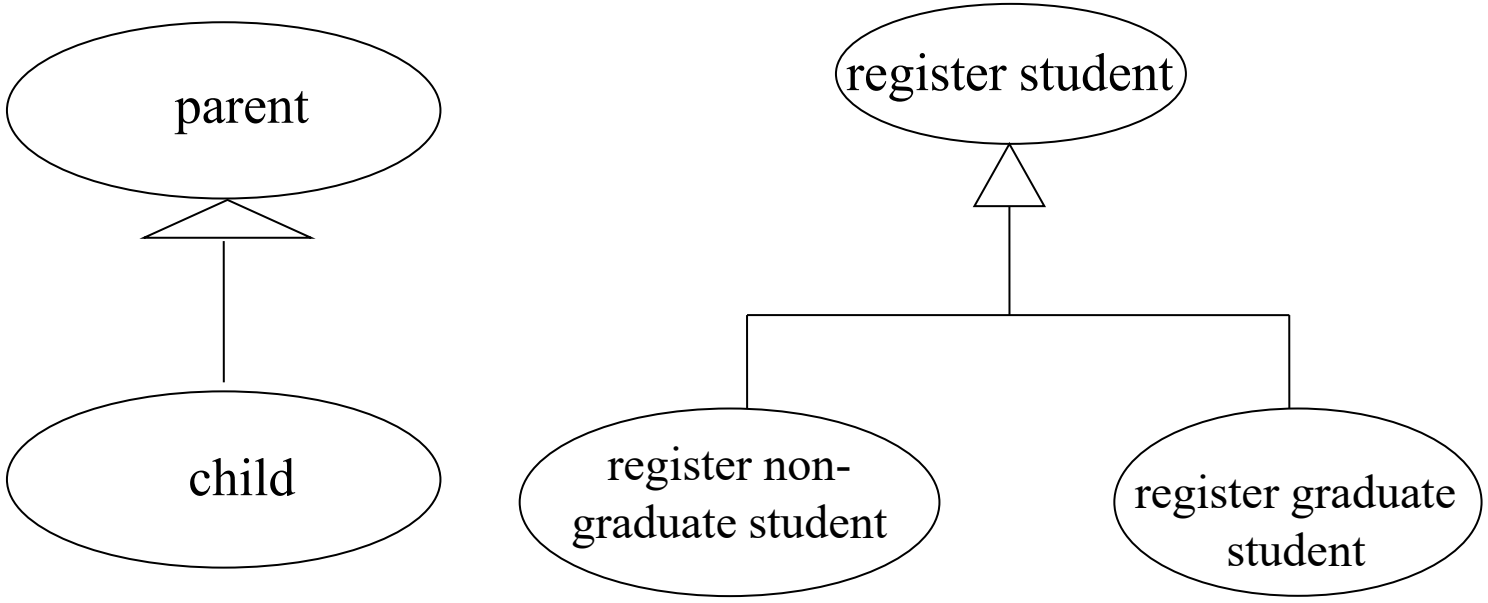
- **Generalization** - Use cases that are specialized versions of other use cases.

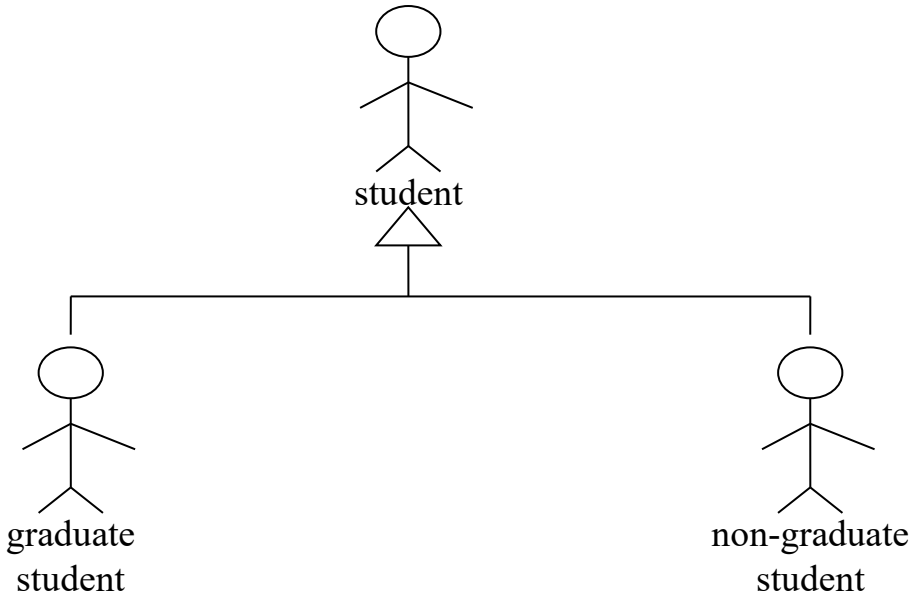
- **Include** - Use cases that are included as parts of other use cases. Enable to factor common behavior.

- **Extend** - Use cases that extend the behavior of other core use cases. Enable to factor variants.

Generalization

- The child use case inherits the behavior and meaning of the parent use case.
- The child may add to or override the behavior of its parent.
- Share the same relationship to the actor

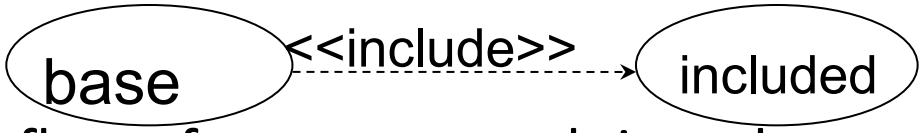




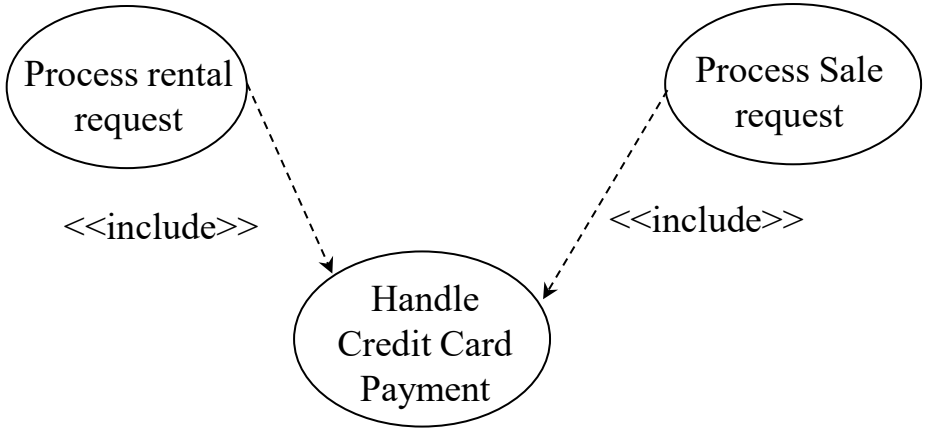
Generalization as a Relationships between Actors

Include

- The base use case explicitly incorporates the behavior of another use case at a location specified in the base.
- The included use case never stands alone. It only occurs as a part of some larger base that includes it.



- Enables to avoid describing the same flow of events several times by putting the common behavior in a use case of its own



Use case specification

- Name (Must start with a verb)
- Summary
- Actors
- Pre-conditions
  - Conditions that must exist *before* the use case is executed
- Description
  - Textual description (may include steps to execute) and typically is the primary functionality
- Exceptions
  - These are paths which will need to handle exceptions which could be all to provide handling of things which are not provide you with a primary functionality including things like power failure
- Alternate Flows
  - Handles the other functionality paths for the summary these could be some in the exceptions too
- Post-conditions
  - Conditions that must exist *after* the use case is executed

Sample use case specification

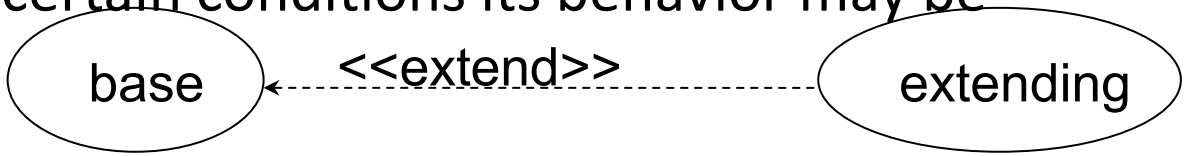
- Name: Transfer Funds
- Summary/Overview : Transfer funds from one account to another
- Actor: Customer
- Pre-conditions: Source account must have sufficient funds
- Description:
  a. Customer identifies the accounts from which and to which funds have to be transferred
  b. Enters the amount to be transferred
  c. Confirm the transaction
- Exceptions
  - Cancel, Insufficient funds, Cannot identify destination account
  - Needs to handle ..say power failure, slow network
- Alternate Flows
  - Handles all the alternate paths (Accounts belonging to the same customer)
- Post-conditions: Funds transferred and account balances updated

Practice: ATM

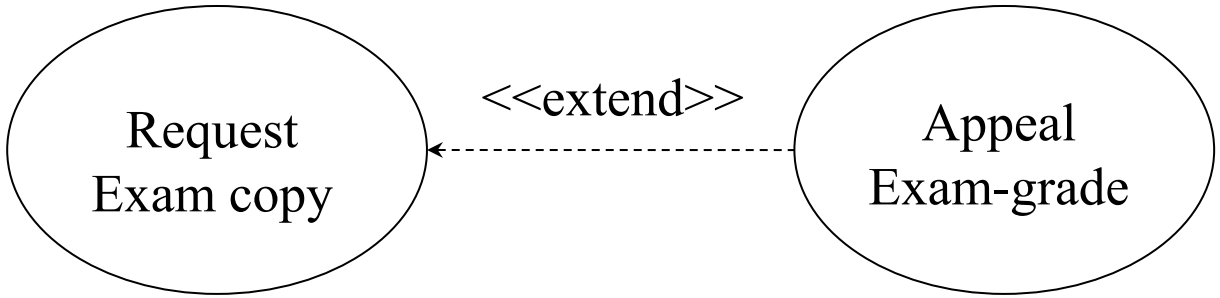
- Use Case: Withdraw money
- Actors: Customer
- Pre Condition:
  - The ATM must be in a state ready to accept transactions
  - The ATM must have at least some cash on hand that it can dispense
  - The ATM must have enough paper to print a receipt for at least one transaction
- Post Condition:
  - The current amount of cash in the user account is the amount before the withdraw minus the withdraw amount
  - A receipt was printed on the withdraw amount
  - The withdraw transaction was audit in the System log file

Extend

- The base use case implicitly incorporates the behavior of another use case at certain points called extension points.
- The base use case may stand alone, but under certain conditions its behavior may be extended by the behavior of another use case.



- Enables to model optional behavior or branching under conditions.



ATM : System and user Actions

Actor Actions	System Actions
1. Begins when a Customer arrives at ATM	
2. Customer inserts a Credit card into ATM	3. System verifies the customer ID and status
5. Customer chooses "Withdraw" operation	4. System asks for an operation type
7. Customer enters the cash amount	6. System asks for the withdrawal amount
	8. System checks if withdrawal amount is legal
	9. System dispenses the cash
	10. System deduces the withdraw amount from account
	11. System prints a receipt
13. Customer takes the cash and the receipt	12. System ejects the cash card

Alternative flow of events:

Step 3: Customer authorization failed. Display an error message, cancel the transaction and eject the card.
Step 8: Customer has insufficient funds in its account. Display an error message, and go to step 6.
Step 8: Customer exceeds its legal amount. Display an error message, and go to step 6.

Exceptional flow of events:

Power failure in the process of the transaction before **step 9**, cancel the transaction and eject the card

Few Inputs

- One method to identify use cases is actor-based:
  - Identify the actors related to a system or organization.
  - For each actor, identify the processes they initiate or participate in.
- A second method to identify use cases is event-based:
  - Identify the external events that a system must respond to.
  - Relate the events to actors and use cases.
- The following questions may be used to help identify the use cases for a system:
  - What are tasks of each actor ?
  - Will any actor create, store, change, remove, or read information in the system ?
  - What use cases will create, store, change, remove, or read this information ?
  - Will any actor need to inform the system about sudden, external changes ?
  - Does any actor need to be informed about certain occurrences in the system ?
  - Can all functional requirements be performed by the use cases ?

PES UNIVERSITY
CELEBRATING 50 YEARS

THANK YOU

Prof. Vinay Joshi

Department of Computer Science and Engineering

vinayj@pes.edu

Object Oriented Analysis and Design

Class Modeling-UML Class Diagrams

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

Class Modelling

- A class model captures the static structure of a system by characterizing the objects in the system, the relationships between the objects and the attributes and operations for each class of objects.
- **Class diagram** is a graphical notation used to construct and visualize object oriented systems.
- **Class diagram can be mapped directly with object oriented languages.**
- Class diagrams capture the static structure of Object-Oriented systems as how they are structured rather than how they behave.
- It supports architectural design.
- They identify what classes are there, how they interrelate and how they interact.
- A UML class diagram is made up of:

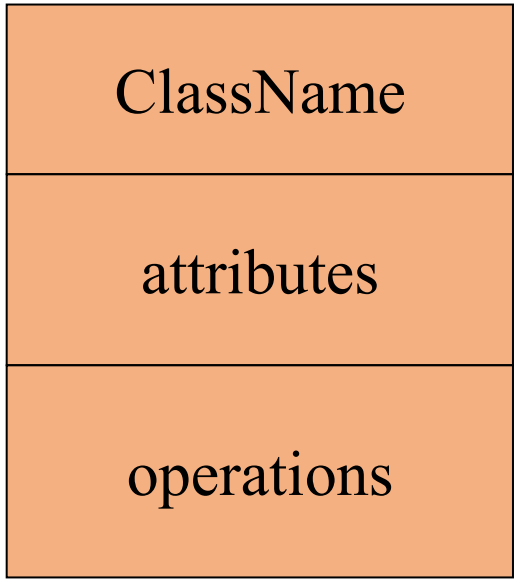
➢ A set of classes and

➢ A set of relationships between classes

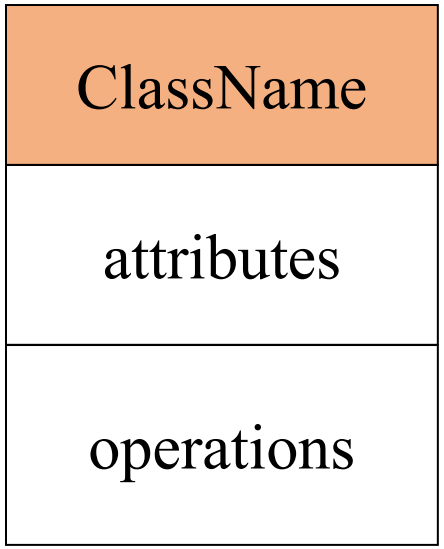
Class

A class is a description of a set of objects that share the same attributes, operations, relationships and semantics.

Graphically, a class is rendered as a rectangle, usually including its name, attributes, and operations in separate, designated compartments.

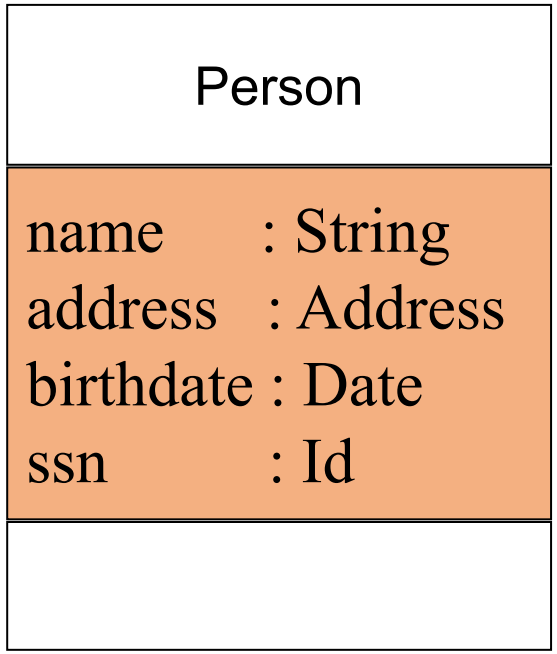


Class



The name of the class is the only required tag in the graphical representation of a class. It always appears in the top-most compartment.

Attributes



Each class can have attributes.

An *attribute* is a named property of a class that describes the object being modeled.

Each attribute has a type.

In the class diagram, attributes appear in the second compartment just below the name-compartment.

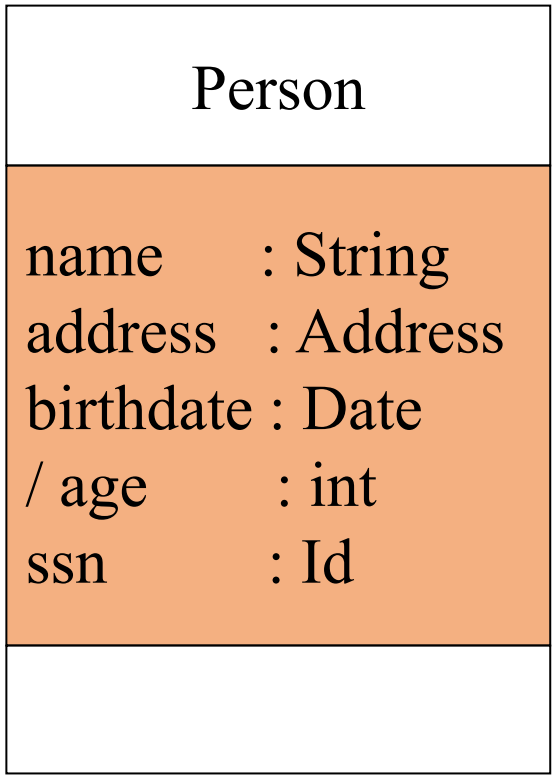
Derived Attribute

Attributes are usually listed in the form:

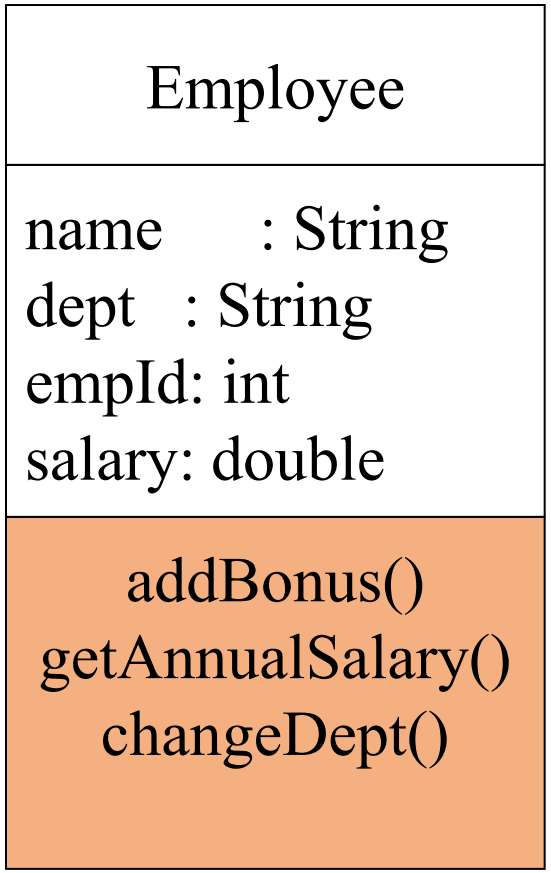
attributeName : Type

A *derived* attribute is one that can be computed from other attributes, but doesn't actually exist.
For example,
a Person's age can be computed from his birth date.
A derived attribute is
designated by a preceding '/' as in:

/ age : int

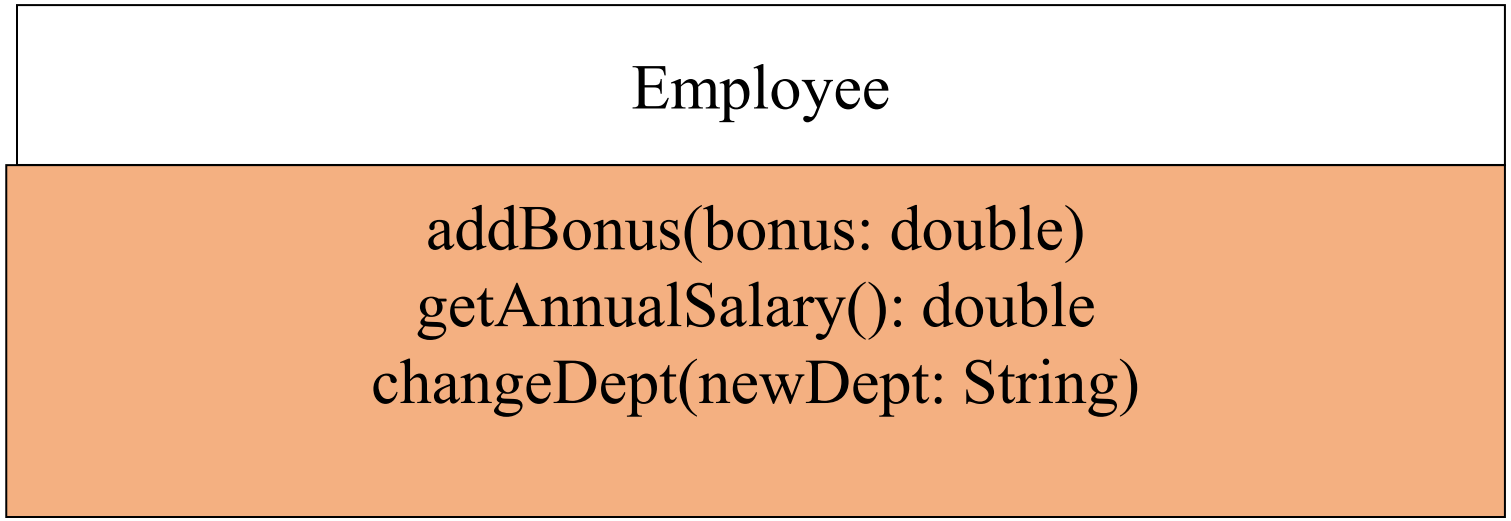


Class Operations



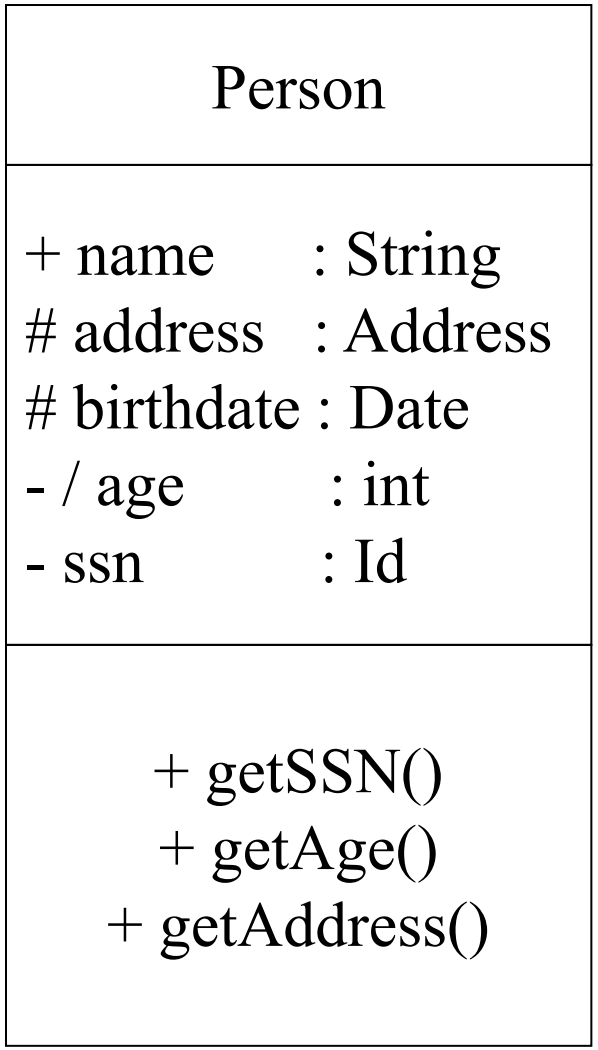
Operations describe the class behavior and appear in the third compartment.

Class operations



You can specify an operation by stating its signature: listing the name, type, and default value of all parameters, and a return type if any.

Visibility



attributes and operations can be declared with different visibility modes:

+ public: any class can use the feature (attribute or operation);

protected: any descendant of the class can use the feature;

- private: only the class itself can use the feature.

Finding Classes

Finding classes in use case, or in text descriptions:
Look for nouns and noun phrases in the description of a use case or a problem statement;
These are only included in the model if they explain the nature or structure of information in the application.

Don't create classes for concepts which:
Are beyond the scope of the system;
Refer to the system as a whole;
Duplicate other classes;
Are too vague or too specific (few instances)

Finding classes in other sources:
Reviewing background information;
Users and other stakeholders;
Analysis patterns;
CRC (Class Responsibility Collaboration) cards.

Approaches for identifying classes

1. Noun phrase approach
2. Common class patterns approach
3. Classes, responsibilities and collaborators (CRC) approach (to be discussed in next lecture)
4. Use case driven approach (discussed in previous session)

1. Noun phrase approach

- Proposed by Rebecca, brian and lauren.
- The requirement document need to be read and noun phrase should be identified.
- Nouns to be considered as classes and verbs to be methods of the classes.
- All plurals changed to singular.
- Nouns are listed and divided into three categories:
  relevant, fuzzy and irrelevant classes
- Irrelevant classes need to be identified .
- Candidate classes can be identified from other two categories.

- Identifying tentative classes:
  - Look for noun and noun phrases in the use cases.
  - Some classes are implicit or taken from general knowledge.
  - Carefully choose and define class names.
- Finding classes is an incremental and iterative process.

Relevant, Fuzzy, and Irrelevant Classes

PES UNIVERSITY
CELEBRATING 50 YEARS

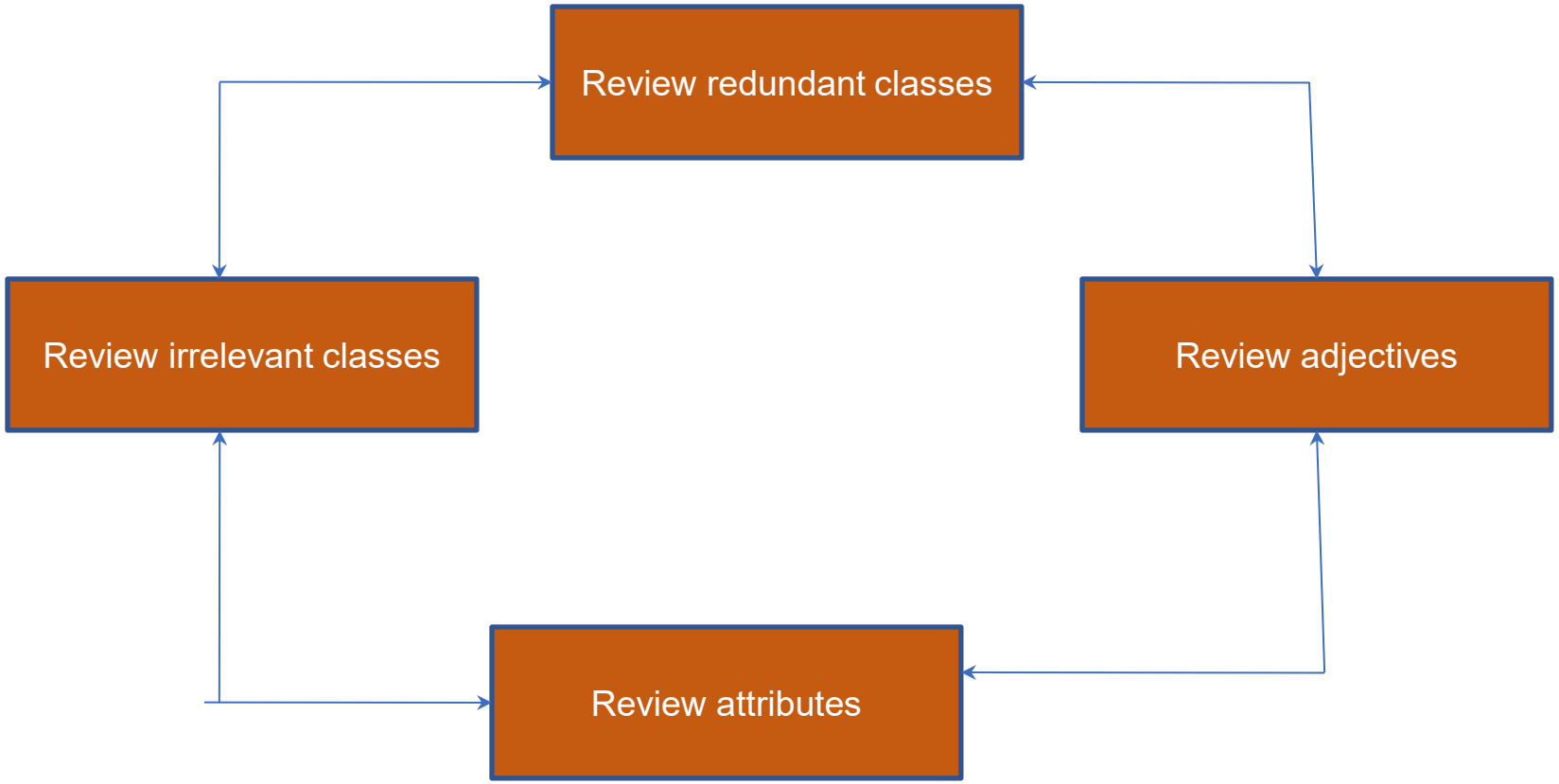
Class Type	Description	Banking System Examples	Action
Relevant Classes	Core domain concepts with clear responsibilities, attributes, and behavior	Customer, Account, Transaction, Loan, Branch	**Keep in the class model**
Fuzzy Classes	Tentative or ambiguous concepts that may later become attributes, roles, or enums	AccountStatus, CustomerType, TransactionCategory, LoanType	**Review and refine** (convert to attribute or enum if needed)
Irrelevant Classes	Concepts outside the problem domain or related to technical/UI implementation	DatabaseConnection, ATMButton, LoginScreen, ReportGenerator	**Remove from the class model**

selecting classes from the relevant and fuzzy categories

- Guidelines to select candidate classes from the relevant and fuzzy categories:
  - Redundant classes: Select a single class when more than one class describes the same information. This helps in building a common vocabulary for the entire system. Choose the term that is used by the user.
  - Attribute classes: Tentative objects that are used only as values should be defined or represented as attributes rather than as classes. For example, client status can be modeled as an attribute instead of a class

- Irrelevant classes: class should be defined clearly with its purpose. If the purpose of statement is not possible for any class, class should be eliminated.

- The process of identifying classes and refining is the iterative process.
- This is not the sequential process. You can move back and forth.


Review redundant classes
Review irrelevant classes
Review adjectives
Review attributes

2. Common class patterns approach

- Researchers like Shlaer and Mellor proposed some patterns for identifying the candidate class and objects
- Concept class:
  - Emphasis principles that are not tangible but used to organize or keep track of business activities or communications.
  - Ex: Account, Customer

- Event class:
  - These are points in time that must be recorded.
  - Things happen, usually to something else at given date and time or as a step in an ordered sequence.
  - Associated with things remembered are attributes such as who,what, when, where, how or why.

Ex: Transaction, Withdrawal, Deposit

- Organization class:
  - it's the collection of people, resources,facilities or groups to which the users belong.
  - Ex: Bank, Branch

- People class:
  - It specifies different roles users play in interacting with the application.

  - *Two types:*
    - the users of the system (operator or clerk) or who interacts with the system.
      Ex. Retail, Corporate, Senior Citizen Customers
    - Who do not use the system but whose information is kept by the system.
      Ex. Beneficiary

- Place class:
  - Places are physical locations that the system must keep information.
  - Ex: buildings, stores and offices
- Tangible things and device class:
  - This includes physical objects or groups of objects that are tangible.
  - The devices with which the application interacts.
  - Ex: car, pressure sensor

References

[1] "**Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development**", by Craig Larman, 3rd Edition, Pearson 2015.

[2] **"Object-Oriented Software Engineering: Practical Software Development using UML and Java"**, Timothy C. Lethbridge, Robert Laganière, Second edition, Mc Gram Hill,2016

[3] **"Object-Oriented Modelling and Design with UML"**, Michael R Blaha and James R Rumbaugh, 2nd Edition, Pearson 2007.

[4] *https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-class-diagram-tutorial/*

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design

Class Modeling: OO-Relationships

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

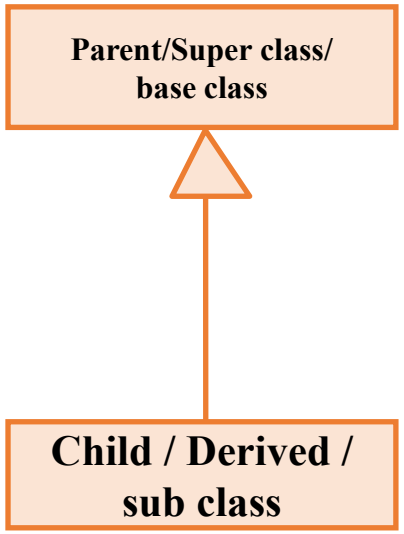
OO-Relationships

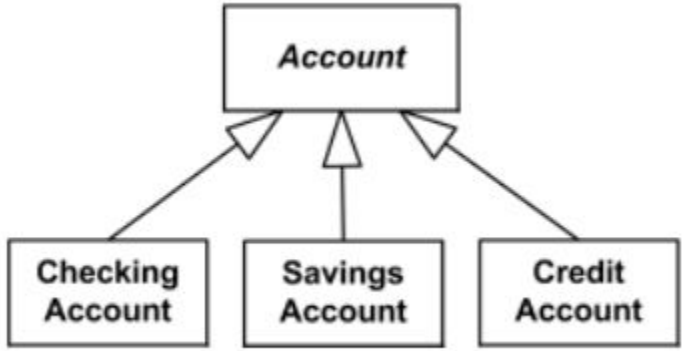
In UML, object interconnections (logical or physical), are modeled as relationships. The type of relationships are listed below:

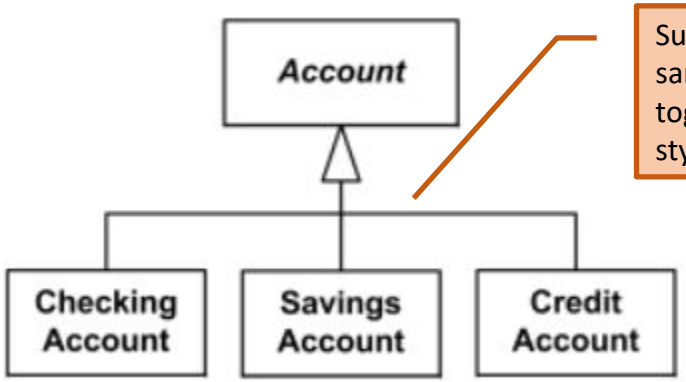
- Generalization- an inheritance relationship
- Abstraction
- Realization - interface implementation
- Dependency
- Association – using relationship
    - Aggregation
    - Composition

Generalization

- Also known as Inheritance.
- It represents “is a” or “is a kind of” relationship since the child class is a type of the parent class.
- It is used to showcase reusable elements in the class diagram.
- The child classes “inherit” the common functionality(attributes and methods) defined in the parent class.
- A generalization is shown as a line with a hollow triangle as an arrowhead.
- The arrowhead points to the super class



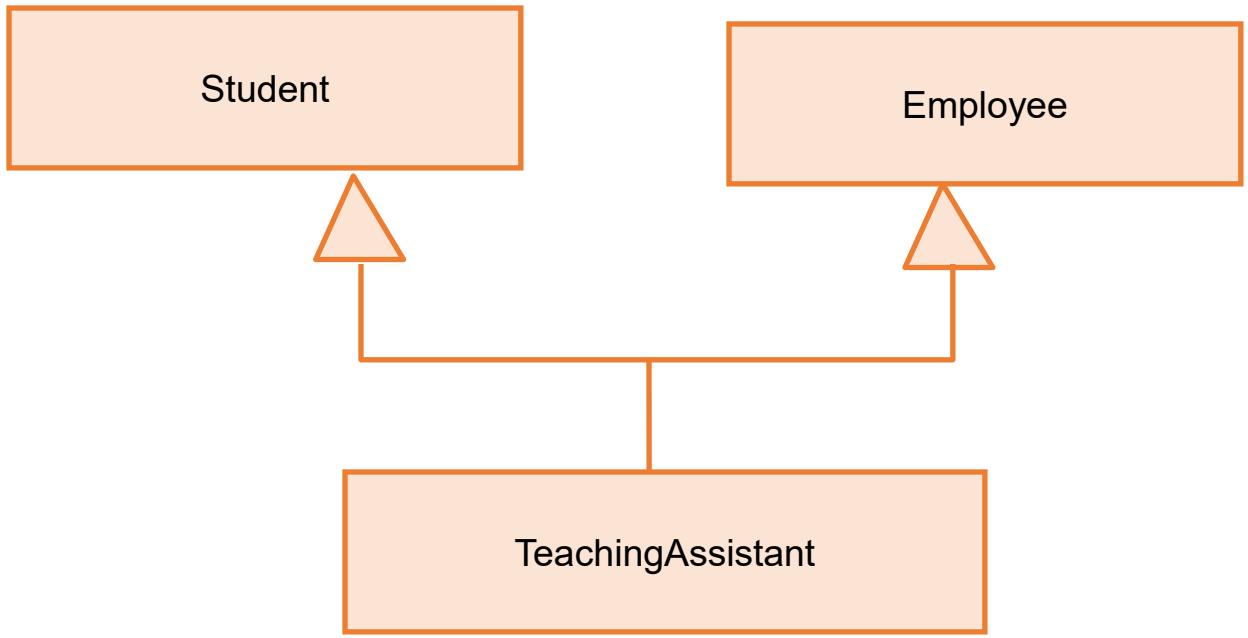




Sub classes that reference the same base can also be connected together in the "shared target style."

Generalization Relationships (Cont'd)

UML permits a class to inherit from multiple super classes, although some programming languages (*e.g.,* Java) do not permit multiple inheritance.

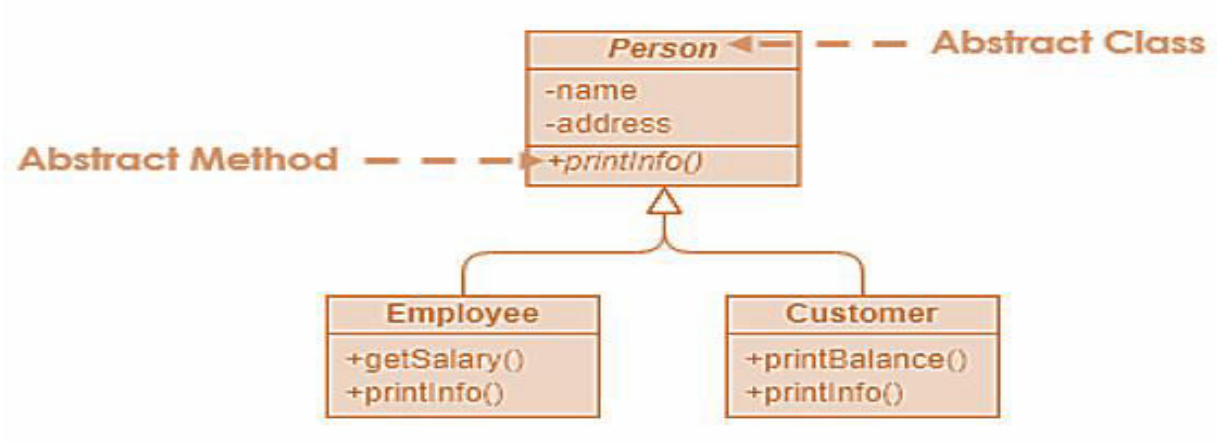


Abstraction

In an inheritance hierarchy, subclasses implement specific details, whereas the parent class defines the framework its subclasses. The parent class also serves as a template for common methods that will be implemented by its subclasses.

The name of an abstract Class is typically shown in italics; alternatively, an abstract Class may be shown using the textual annotation, also called stereotype {abstract} after or below its name.

An abstract method is a method that do not have implementation. In order to create an abstract method, create a operation and make it italic.

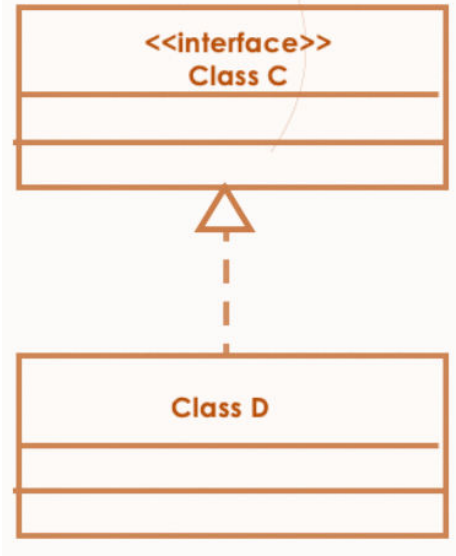


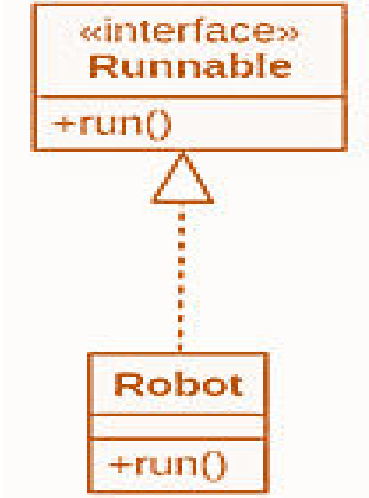
Realization / Interfaces

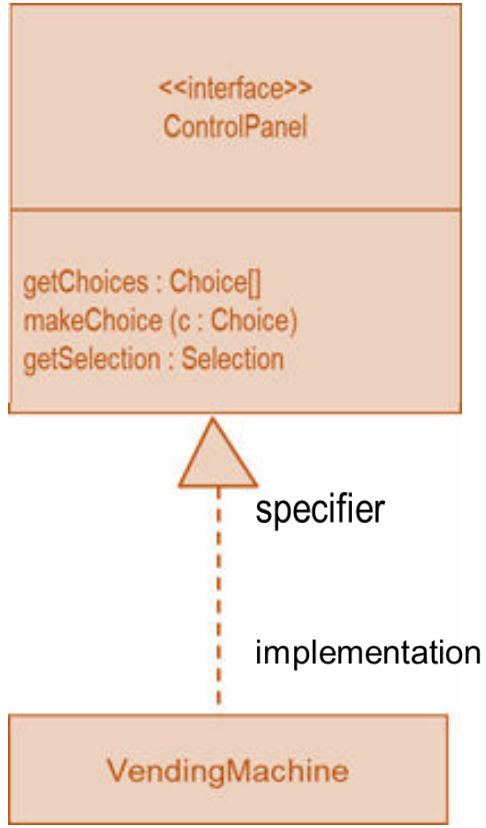
Realization is a specialized abstraction relationship between two things where one thing (an interface) specifies a contract that another thing (a class) guarantees to carry out by implementing the operations specified in that contract.

Interface defines a set of functionalities as a contract and the other entity “realizes” the contract by implementing the functionality defined in the contract.

Realization is shown as a dashed directed line with an open arrowhead pointing to the interface. Stereotype {interface} is generally mentioned before its name

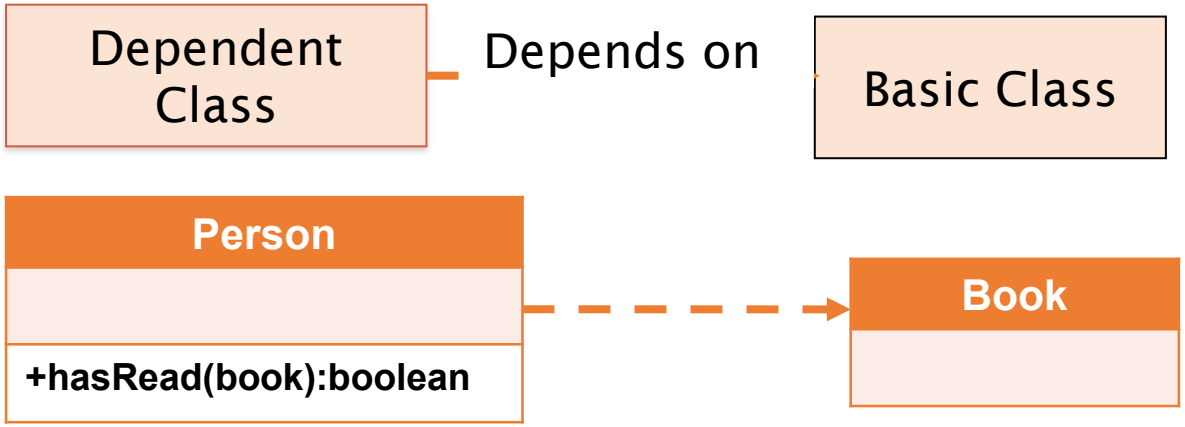






Dependency

- Dependency means "**One class uses the other**"
- A dependency relationship indicates that a **change** in **one class** may **affect** the **dependent class**, but not necessarily the reverse.
- A dependency relationship is often used to show that a **method has object** of a class **as arguments**.



An object of one class might use an object of another class in the code of a method. If the object is not stored in any field, then this is modeled as a dependency relationship. For example, the Person class might have a hasRead method with Book as a parameter that returns true if the person has read the book.

Object Oriented Analysis and Design

Association Relationships

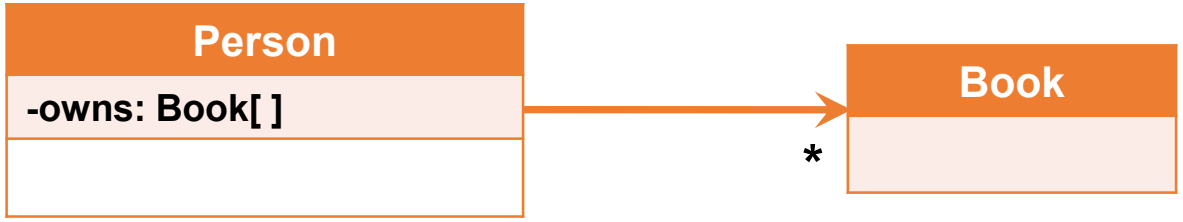
If two classes in a model need to communicate with each other, there must be link between them.
An *association* denotes that link.

We can constrain the association relationship by defining the *navigability* of the association.
Association is of two types.
They are

- Unidirectional Association
- Bidirectional Association

Association Relationships (Cont'd)

- In a unidirectional association, two classes are related, but only one class "knows" that the relationship exists.
- A unidirectional association is drawn as a solid line with an open arrowhead pointing to the known class.
- Uni-directional association includes a role name and a multiplicity value, but only for the known class.
- In this example Orderdetails class only knows that it has a relationship with Item class but Item class does not know about their relationship.



An object might store another object in a field. For example, people own books, which might be modeled by an owns field in Person objects. However, a book might be owned by a very large number of people, so the reverse direction might not be modeled. The * in the figure indicate that a person can own any number of books.

Association Relationships (Cont'd)

We can indicate the *multiplicity* of an association by adding *multiplicity adornments* to the line denoting the association.

The multiplicity of an association is the number of possible instances of the class associated with a single instance of the other end.

Multiplicities are single number or ranges of numbers.

Multiplicities	Meaning
0..1	zero or one instance.
0..* *or* *	no limit on the number of instances (including none).
1	exactly one instance
1..*	at least one instance
n..m	***n*** to ***m*** instances (n and m stand for numbers, e.g. **0..4**, **3..15**)
n	exactly n instance (where n stands for a number, e.g. **3**)

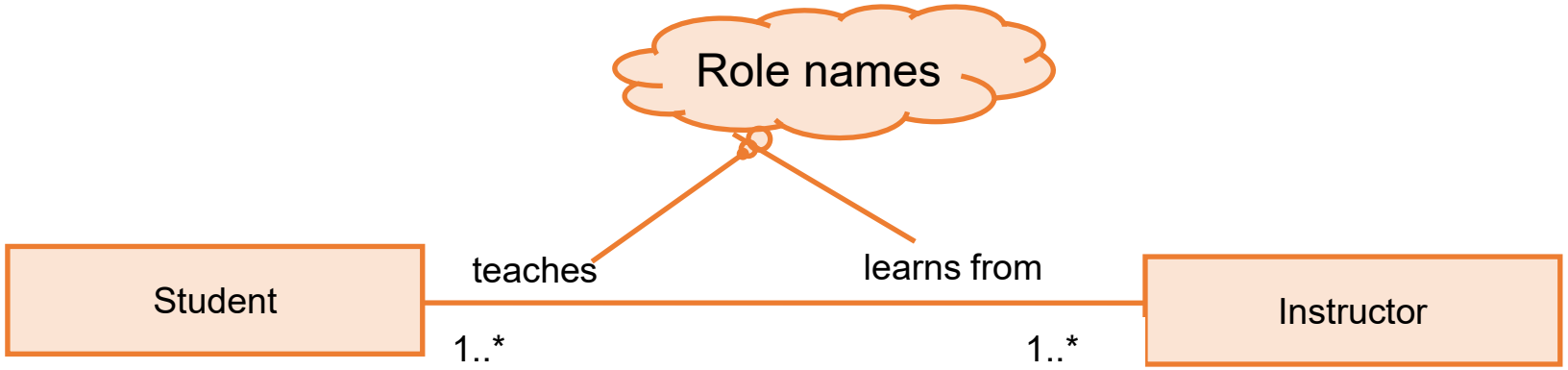
Association Relationships (Cont'd)

The example indicates that a *Student* has one or more *Instructors*:

The example indicates that every *Instructor* has one or more *Students*:

Association Relationships (Cont'd)

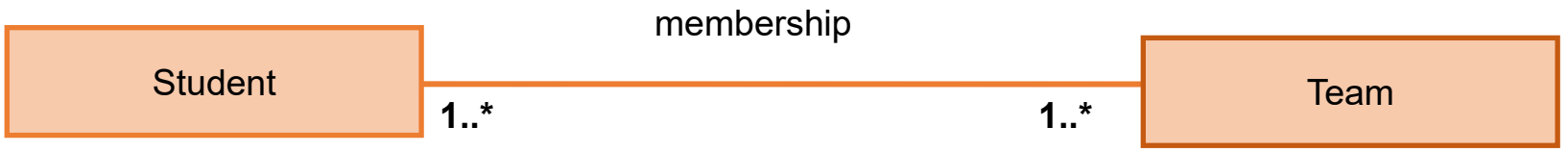
We can also indicate the behavior of an object in an association (*i.e.,* the *role* of an object) using *role names.*



Role names must describe the role played by the class at the opposite end of the association.

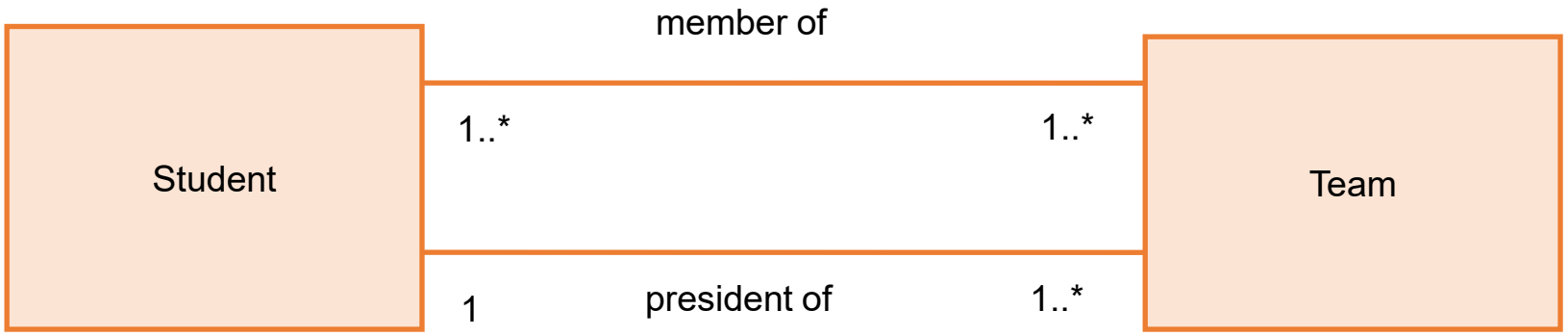
Association Relationships (Cont'd)

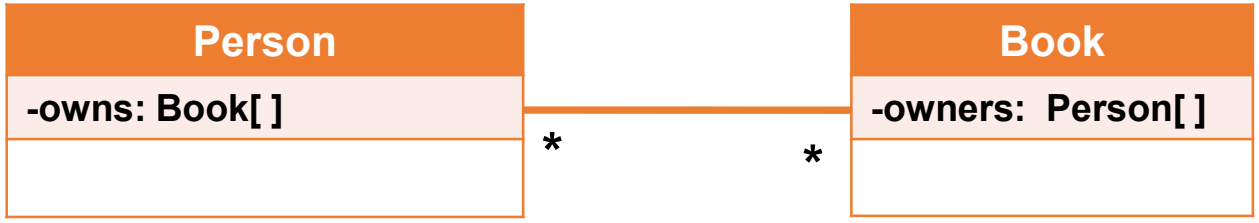
We can also name the association.



Association Relationships (Cont'd)

We can specify dual associations using bidirectional association

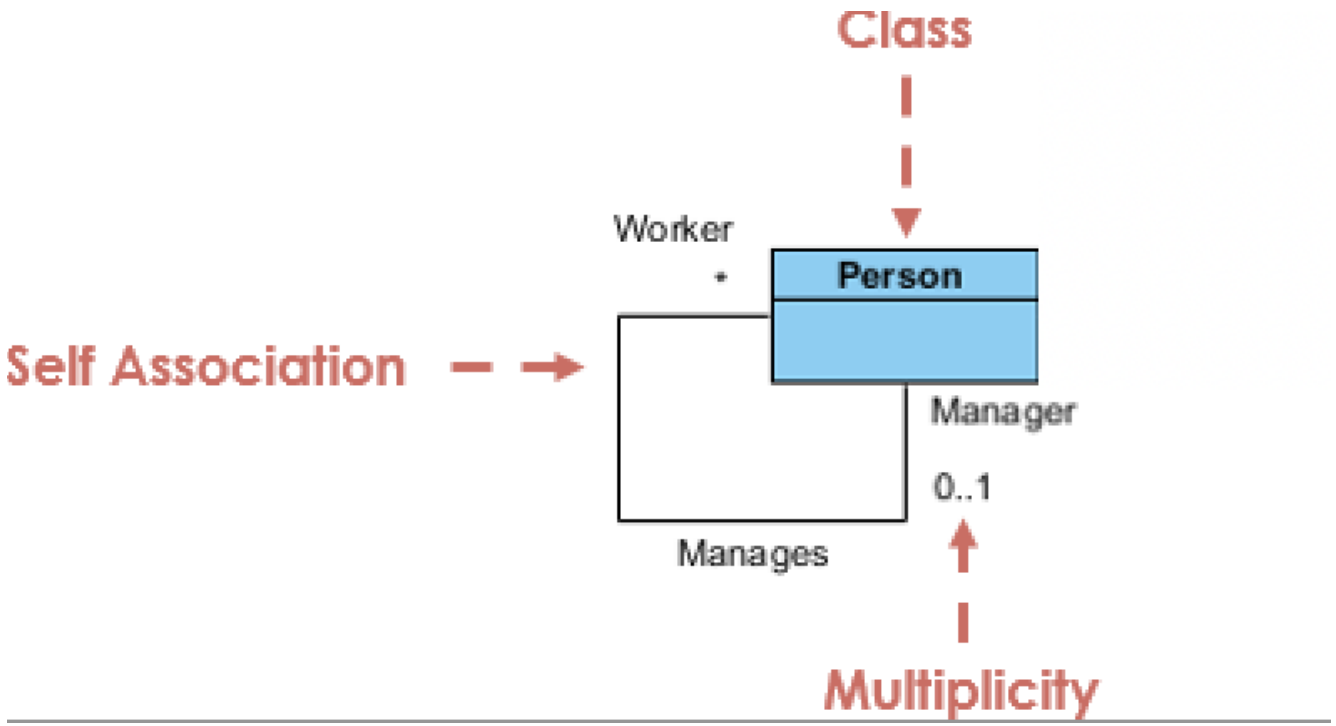




Two objects might store each other in fields. For example, in addition to a Person object listing all the books that the person owns, a Book object might list all the people that own it

Association Relationships (Cont'd)

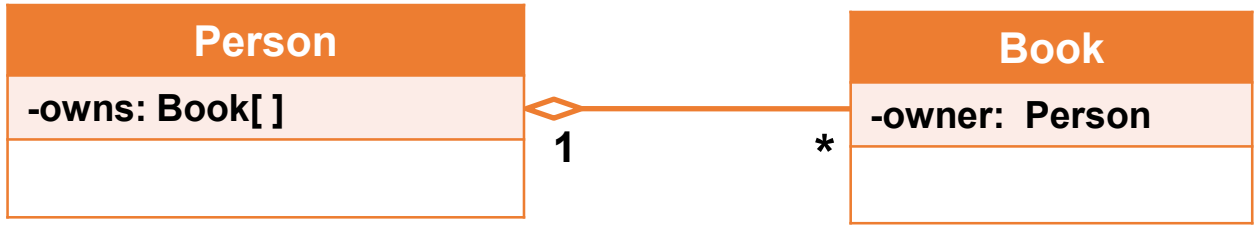
A class can have a *self association*.



Aggregation

We can model objects that contain other objects by way of special associations called *aggregations* and *compositions. It is also known as "has a" relationship.*

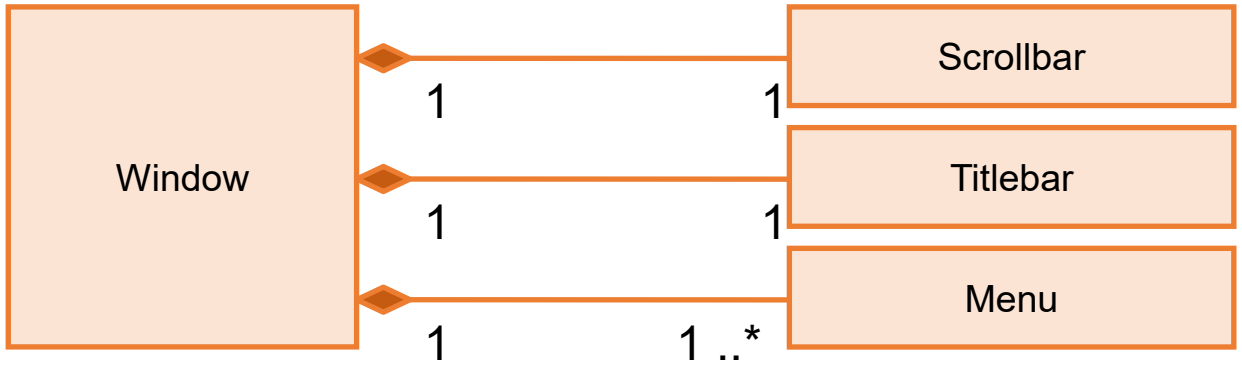
An *aggregation* specifies a whole-part relationship between an aggregate (a whole) and a constituent part, where the part can exist independently from the aggregate. Aggregations are denoted by a hollow-diamond adornment on the association.

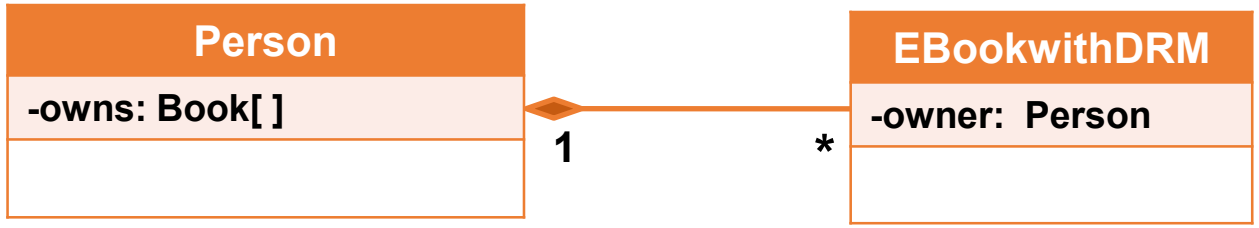


One object A has or owns another object B, and/or B is part of A. For example, suppose there are different Book objects for different physical copies. Then the Person object has/owns the Book object, and, while the book is not really part of the person, the book is part of the person's property. In this case, each book will (usually) have one owner. Of course, a person might own any number of books.

Composition

A *composition* indicates a strong ownership and coincident lifetime of parts by the whole (*i.e.,* they live and die as a whole). Compositions are denoted by a filled-diamond adornment on the association.

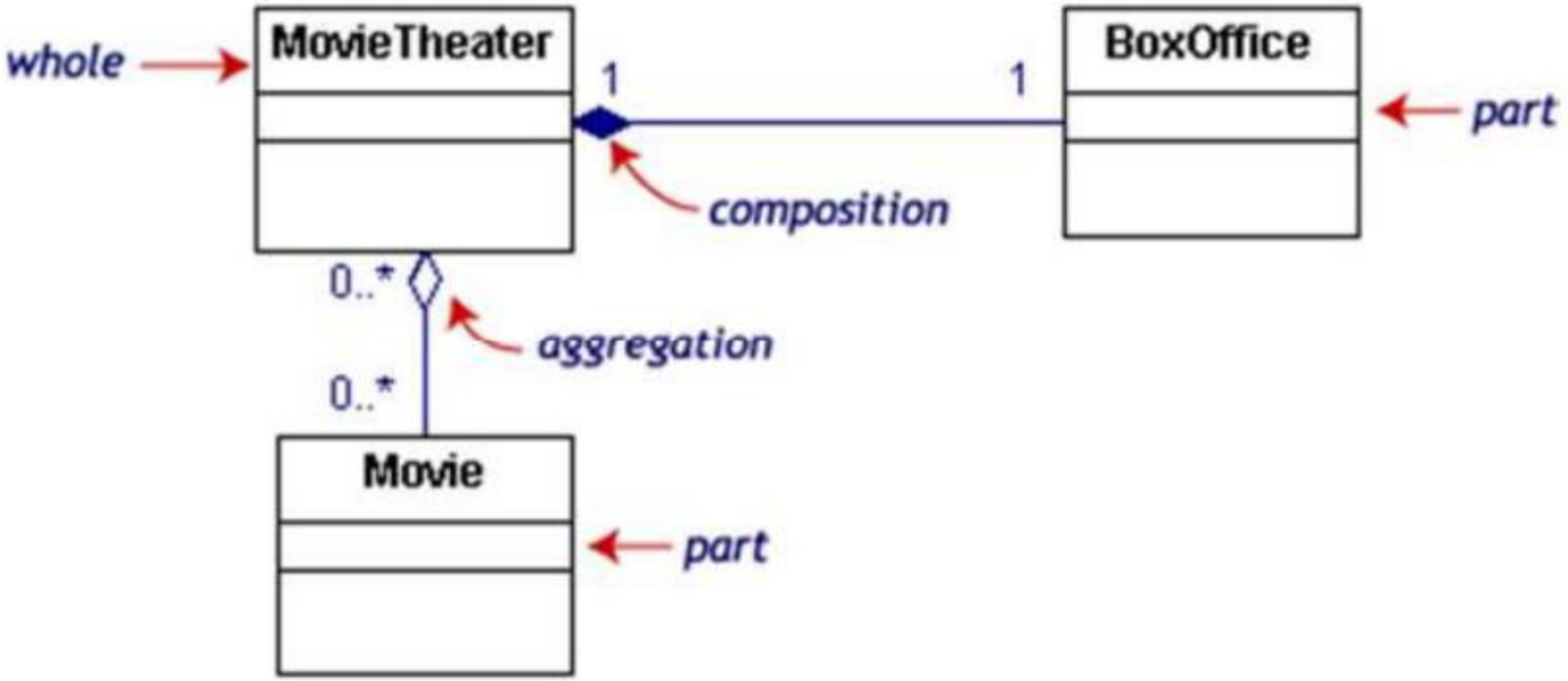




In addition to an aggregation relationship, the lifetimes of the objects might be identical, or near For example, in an idealized world of electronic books with DRM (Digital Rights Management), a person can own an ebook, but cannot sell it. After the person dies, no one else can access the ebook. [This is idealized, but might be considered less than ideal.]

Object Oriented Analysis and Design

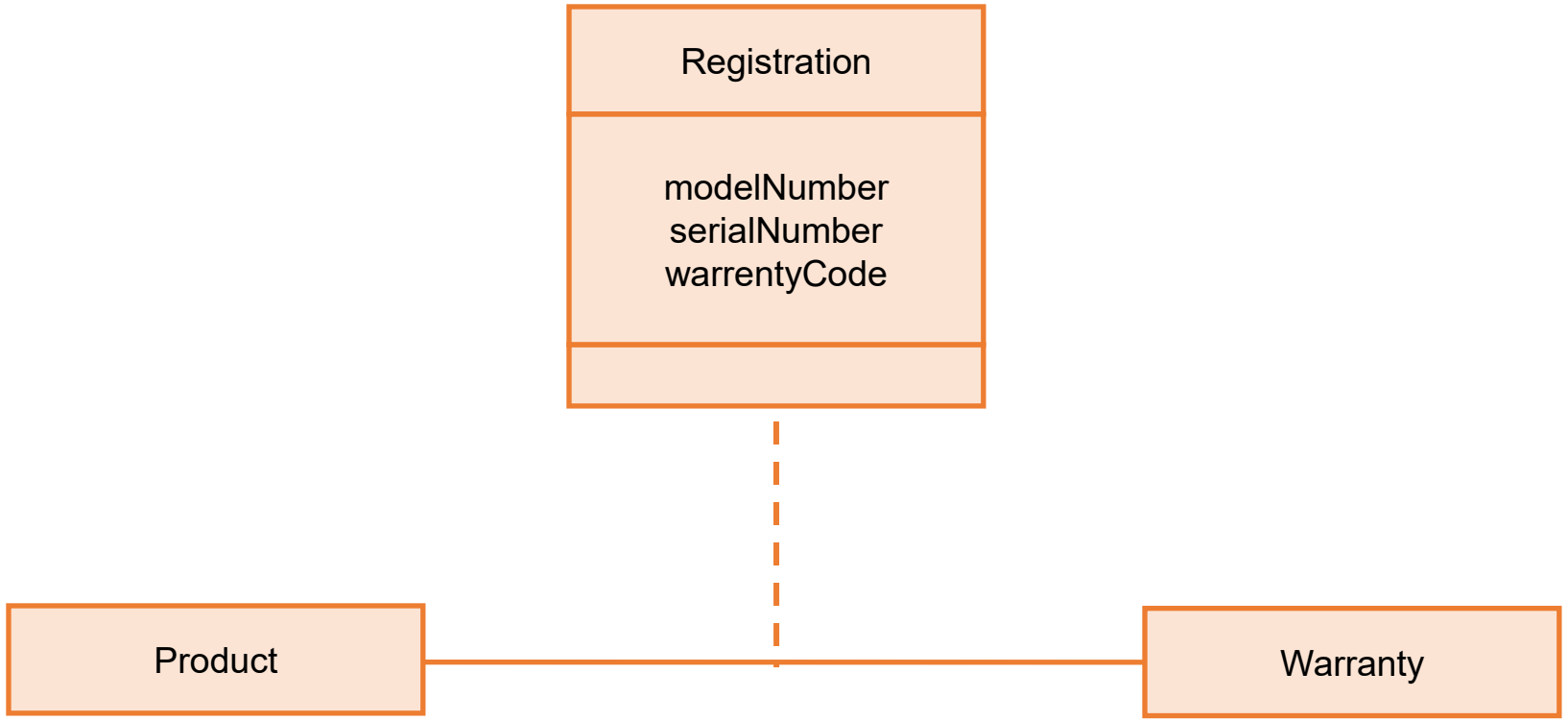
Composition / Aggregation Example



If the movie theater goes away
so does the box office => composition
but movies may still exist => aggregation

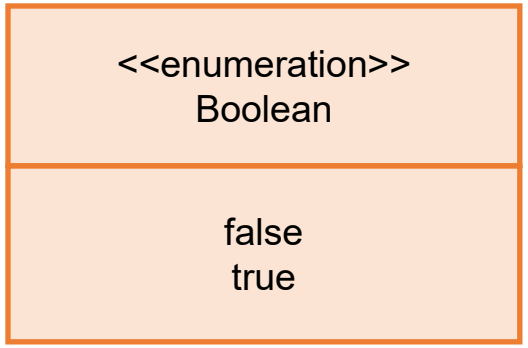
Association Relationships (Cont'd)

Associations can also be objects themselves, called *link classes* or an *association classes*.
An association class is represented like a normal class, but it is linked to an association line with a dotted line.



Enumeration

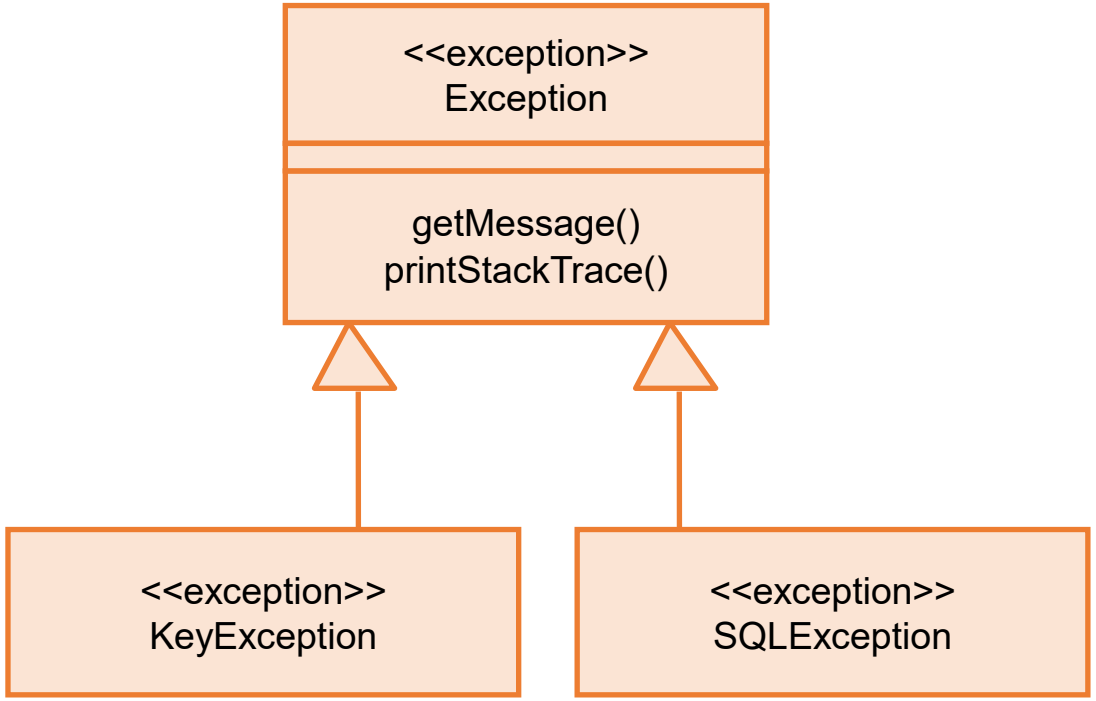
An *enumeration* is a user-defined data type that consists of a name and an ordered list of enumeration literals.



Exceptions

Exceptions can be modeled just like any other class.

Notice the <<exception>> stereotype in the name compartment.



How to make a class diagram

1.Identify all the classes participating in the software solution.

2. Draw them in a class diagram.

3. Identify the attributes.

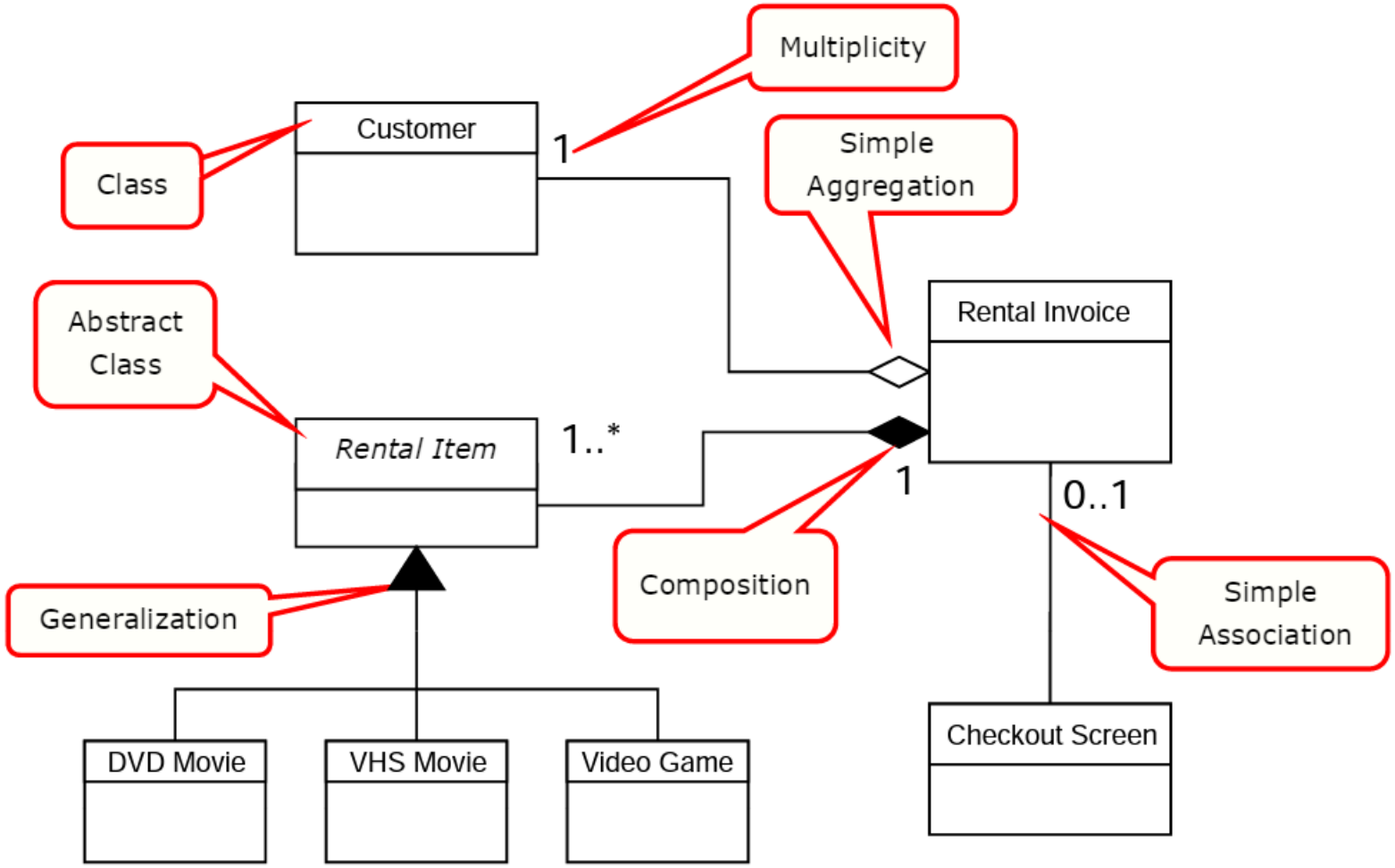
4. Identify the methods.

5. Add associations, generalizations, aggregations and dependencies.

6. Add other details (roles, constraints, ...)

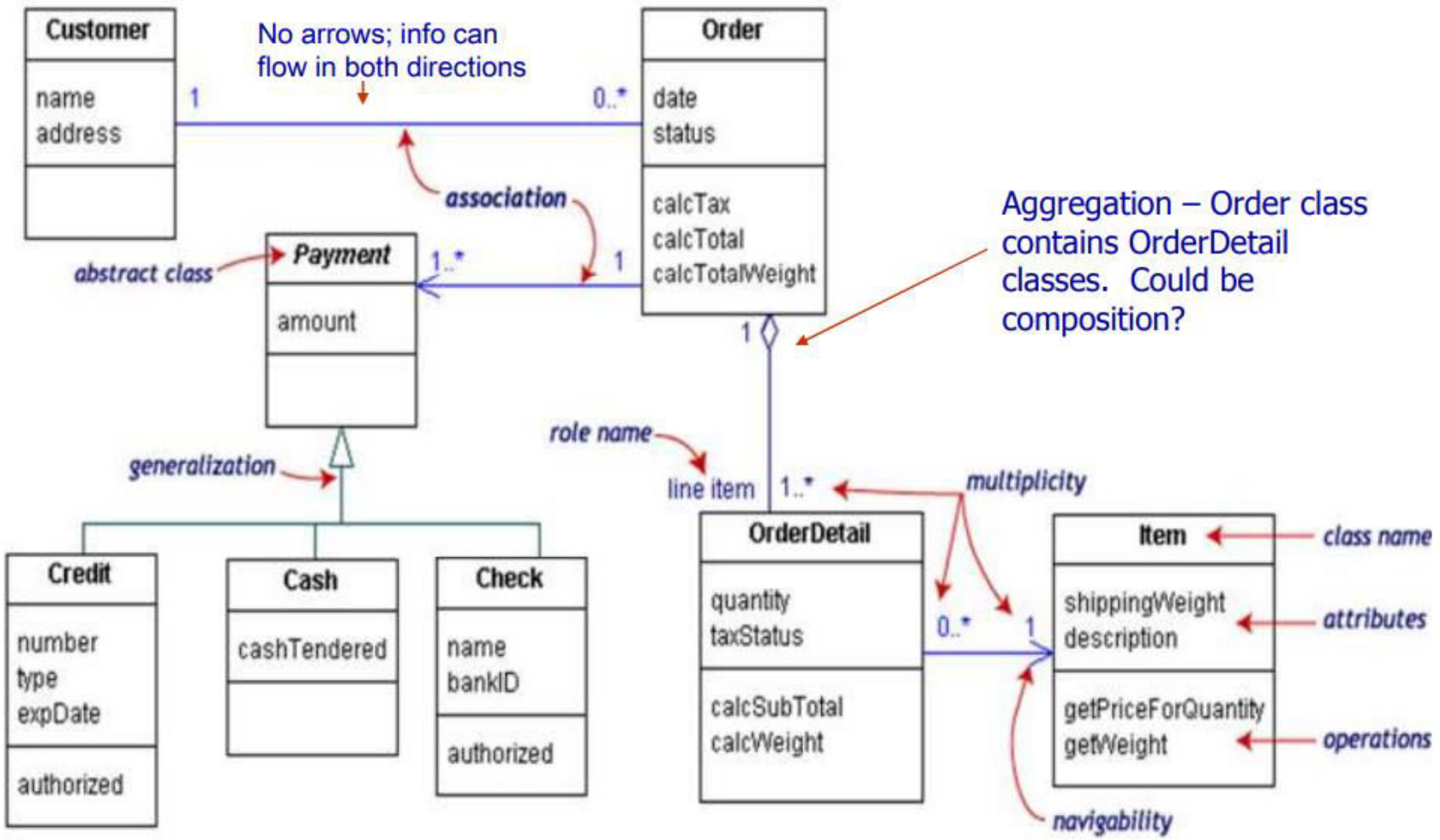
Object Oriented Analysis and Design

Example class diagram –Video Store



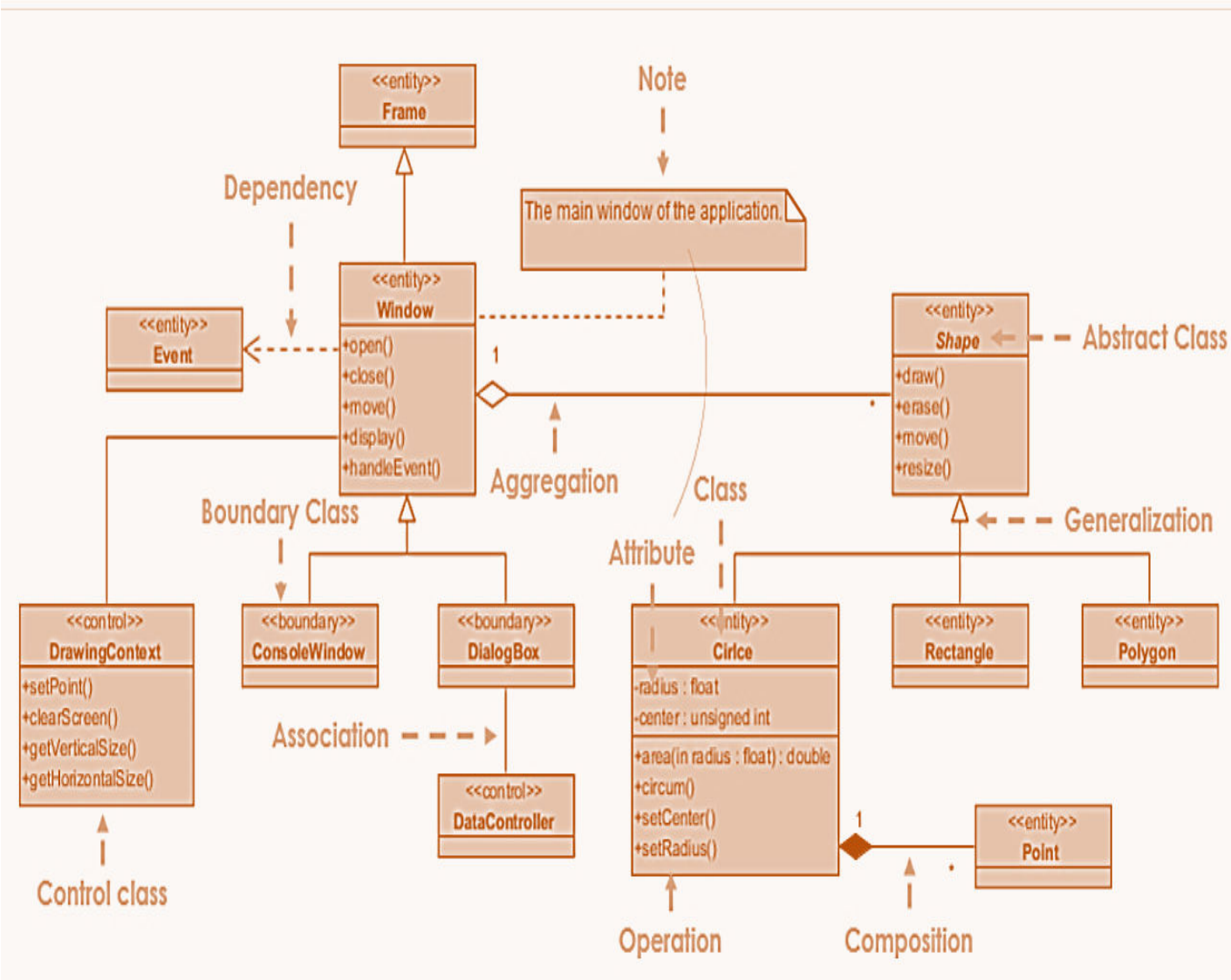
Object Oriented Analysis and Design

Example class diagram –Video Store



Object Oriented Analysis and Design

Example class diagram



Shape is an abstract class. It is shown in Italics.
Shape is a superclass. Circle, Rectangle and Polygon are derived from Shape. This is a generalization / inheritance relationship.
There is an association between DialogBox and DataController.
Shape is part-of Window. This is an aggregation relationship.
Shape can exist without Window.
Point is part-of Circle. This is a composition relationship.
Point cannot exist without a Circle.
Window is dependent on Event. However, Event is not dependent on Window.
The attributes of Circle are radius and center. This is an entity class.
The method names of Circle are area(), circum(), setCenter() and setRadius().
The parameter radius in Circle is an in parameter of type float.
The method area() of class Circle returns a value of type double.
The attributes and method names of Rectangle are hidden.
Some other classes in the diagram also have their attributes

References

[1] "**Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development**", by Craig Larman, 3rd Edition, Pearson 2015.

[2] **"Object-Oriented Software Engineering: Practical Software Development using UML and Java"**, Timothy C. Lethbridge, Robert Laganière, Second edition, Mc Gram Hill,2016

[3] **"Object-Oriented Modelling and Design with UML"**, Michael R Blaha and James R Rumbaugh, 2nd Edition, Pearson 2007.

[4] *https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-class-diagram-tutorial/*

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design

CRC for a case study

Dr. Sudeepa Roy Dey
Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

Object Oriented Analysis and Design

CRC for a case study

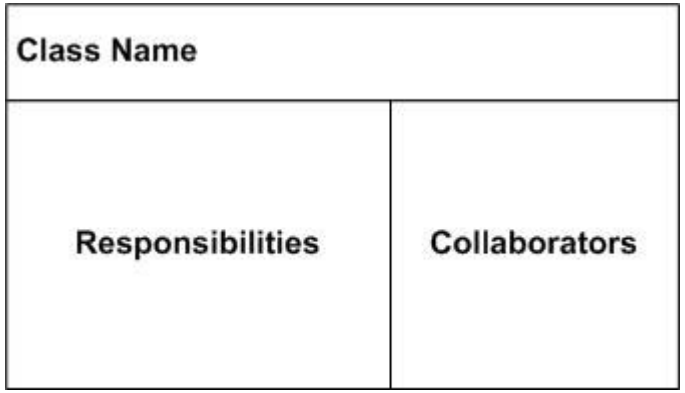
Dr. Sudeepa Roy Dey
Department of Computer Science and Engineering

With grateful thanks for contribution of slides to:
Prof. Ruby D, Asst Professor at the Department of CSE, PESU-EC

Object Oriented Analysis and Design

CRC (Class-Responsibility-Collaborators)

- CRC cards are tool used for brainstorming in OO design and agile methodologies
- CRC cards are created from Index cards
- Each member of the project team write one CRC card for each relevant class/object of their design
- Objects need to interact with other objects (Collaborators) in order to fulfill their responsibilities
- Since the cards are small, prevents to get into details and give too many responsibilities to a class
- They can be easily placed on the table and rearranged to describe and evolve the design



What is a CRC Card?

CRC stands for Class, Responsibility and Collaboration.

- Class
  - An object-oriented class name
  - Include information about super- and sub-classes
- Responsibility
  - What information this class stores
  - What this class does
  - The behavior for which an object is accountable
- Collaboration
  - Relationship to other classes
  - Which other classes this class uses

Class Name	
Responsibilities	Collaborators

Class: Account	
Responsibilities	**Collaborators**
Know balance	
Deposit Funds	Transaction
Withdraw Funds	Transaction, Policy
Standing Instructions	Transaction, StandingInstruction Policy, Account

Ex: CRC card

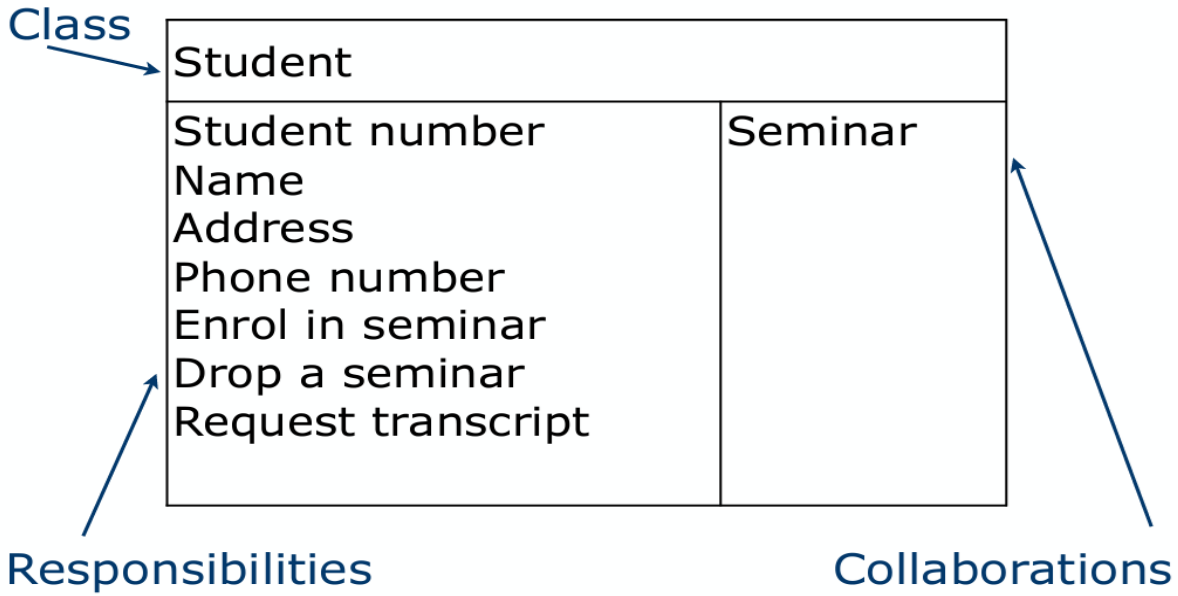
Object Oriented Analysis and Design

Example: Student CRC card.

A responsibility is anything that a class **knows or does.** The things a class knows and does constitute its responsibilities.

For example, students have names, addresses, and phone numbers. These are the things a student knows.

Students also enroll in seminars, drop seminars, and request transcripts. These are the things a student does.



Students only have information about themselves (their names and so forth), and not about seminars. What the student needs to do is collaborate/interact with the card labeled *Seminar* to sign up for a seminar. Therefore, *Seminar* is included in the list of collaborators of *Student*.

Refer to link : https://agilemodeling.com/artifacts/crcmodel.htm for details

CRC Card

- CRC cards are an aid to a **group role-playing activity.**
- Index cards are used in preference to pieces of paper due to their robustness and to the limitations that their size (approx. 15cm x 8cm) imposes on the number of responsibilities and collaborations that can be effectively allocated to each class.
- A class name is entered at the top of each card and responsibilities and collaborations are listed underneath as they become apparent.

- For the sake of clarity, each collaboration is normally listed next to the corresponding responsibility.
- From a UML perspective, use of CRC cards is in analyzing the object interaction that is triggered by a particular use case scenario.

The process of using CRC cards is usually structured as follows.

1.Conduct a session to identify which objects are involved in the use case.
2. Allocate each object to a team member who will play the role of that object.
3. Act out the use case.
4. Identify and record any missing or redundant objects.

References

[1] "**Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development**", by Craig Larman, 3rd Edition, Pearson 2015.

[2] **"Object-Oriented Software Engineering: Practical Software Development using UML and Java"**, Timothy C. Lethbridge, Robert Laganière, Second edition, Mc Gram Hill,2016

[3] **"Object-Oriented Modelling and Design with UML"**, Michael R Blaha and James R Rumbaugh, 2nd Edition, Pearson 2007.

[4] *https://agilemodeling.com/artifacts/crcmodel.htm*

THANK YOU

Dr. Sudeepa Roy Dey

Department of Computer Science and Engineering

sudeepar@pes.edu

Object Oriented Analysis and Design

Prof. Nivedita Kasturi
Department of Computer Science and Engineering

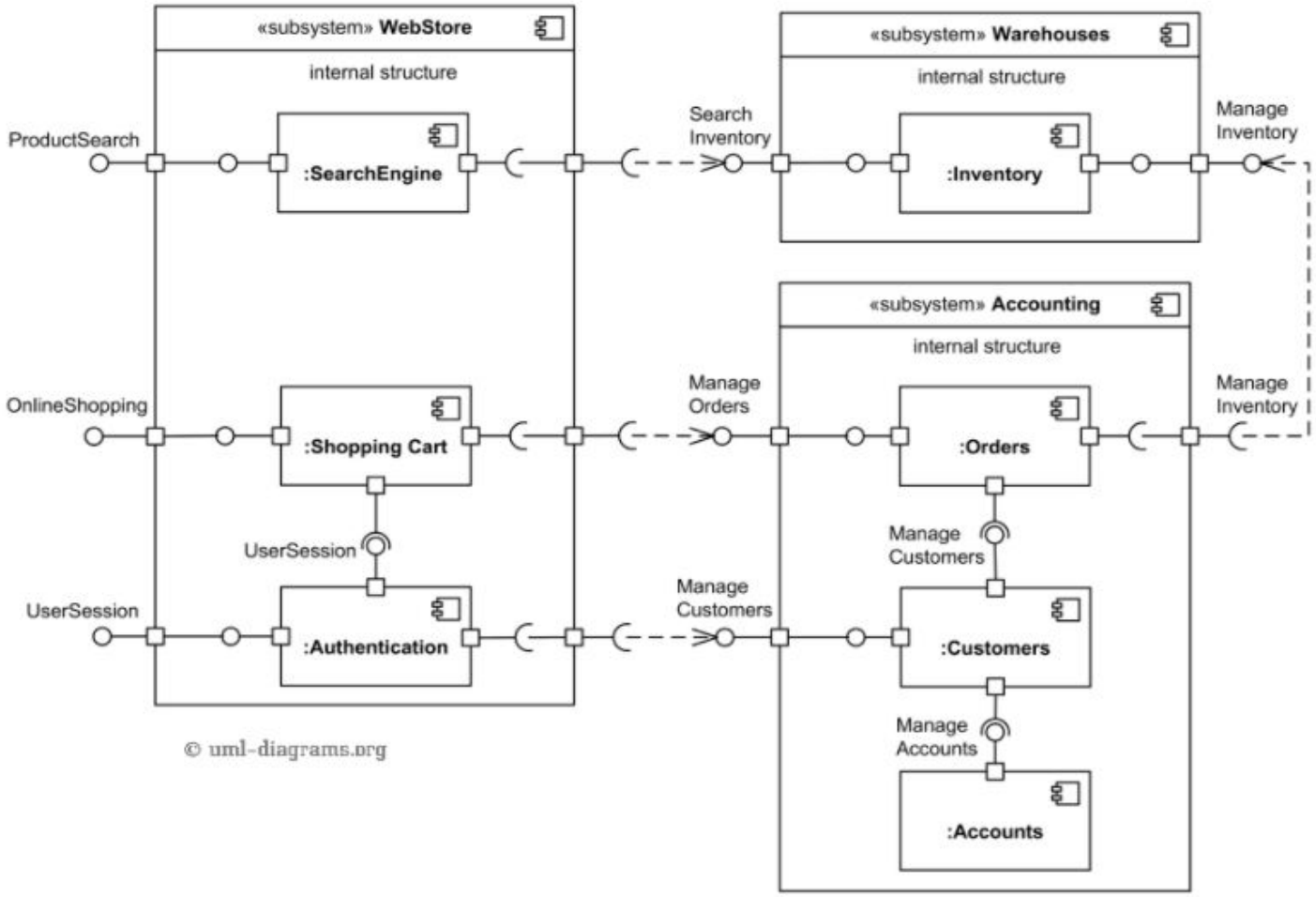
Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

Object Oriented Analysis and Design

Component Model

Prof. Nivedita Kasturi
Department of Computer Science and Engineering

UML Component Diagram



Online shopping UML component diagram example with three related subsystems - WebStore, Warehouses, and Accounting.

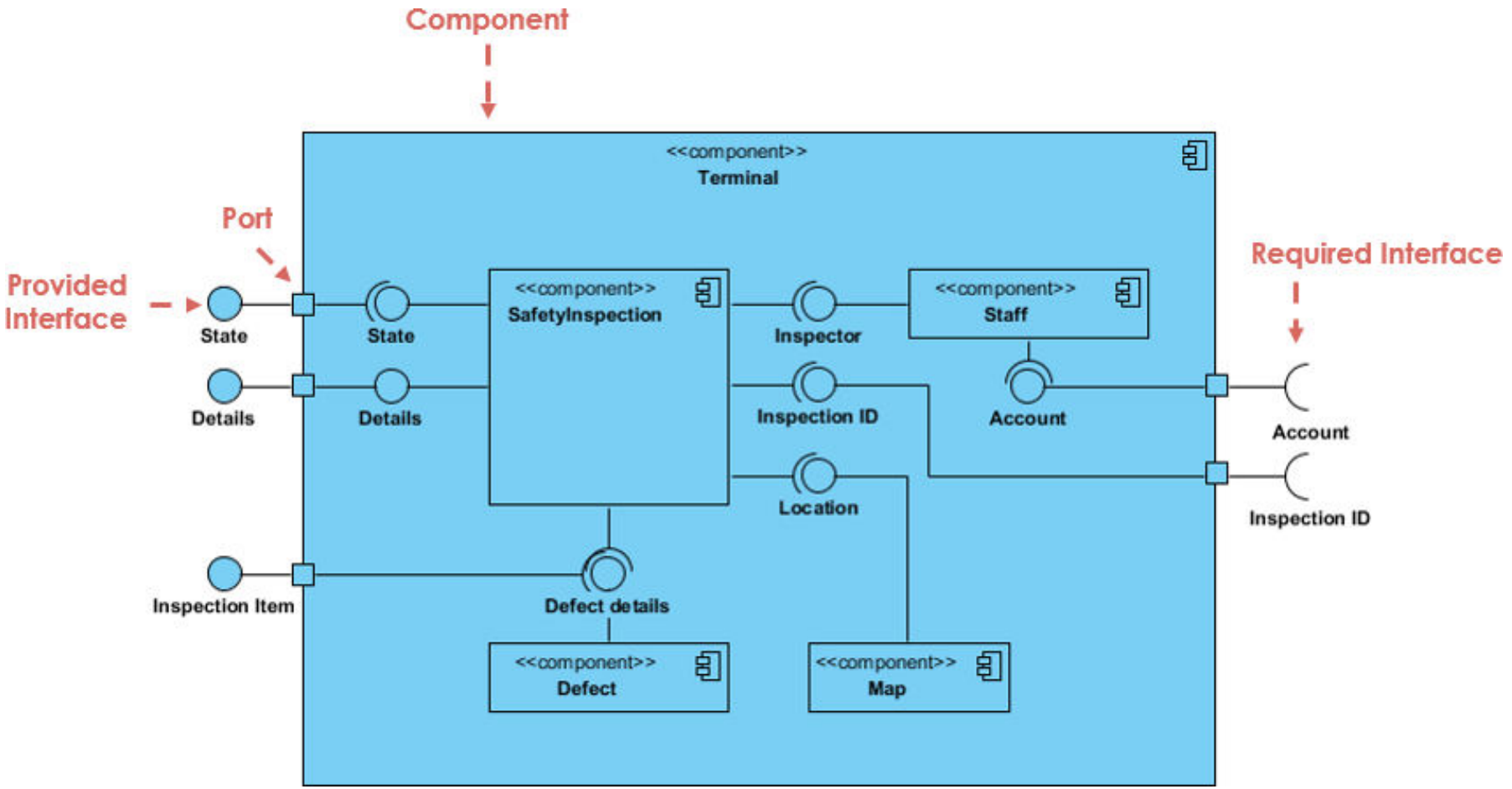
- **Used for modelling large systems into smaller subsystems** which can be easily managed
- Used to represent different components of a system.
- When modelling large OO-systems, break down the system into manageable subsystems.

What is Component in OOAD?

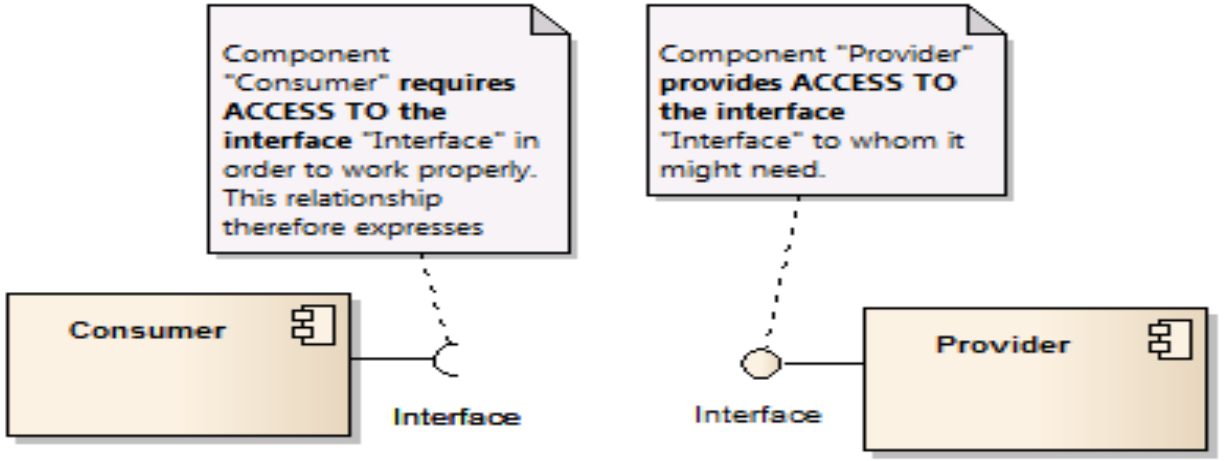
- Component is a *replaceable and executable piece of a system whose implementation details are hidden.*
- Component provides the set of *interfaces that a component realizes* or implements.
- Components require interfaces to carry out a function.
- **It is a modular part of a system that encapsulates its contents.**
- They are the logical elements of a system that plays an essential role during the execution of a system.
- *A component is similar to a **black box** whose external behaviour is defined by a provided interface and required interfaces.*

Component in OOAD?

- A component diagram breaks down the actual system under development into various high levels of functionality.
- *Each component is responsible for one clear aim* within the entire system and only interacts with other essential elements on a need-to-know basis.

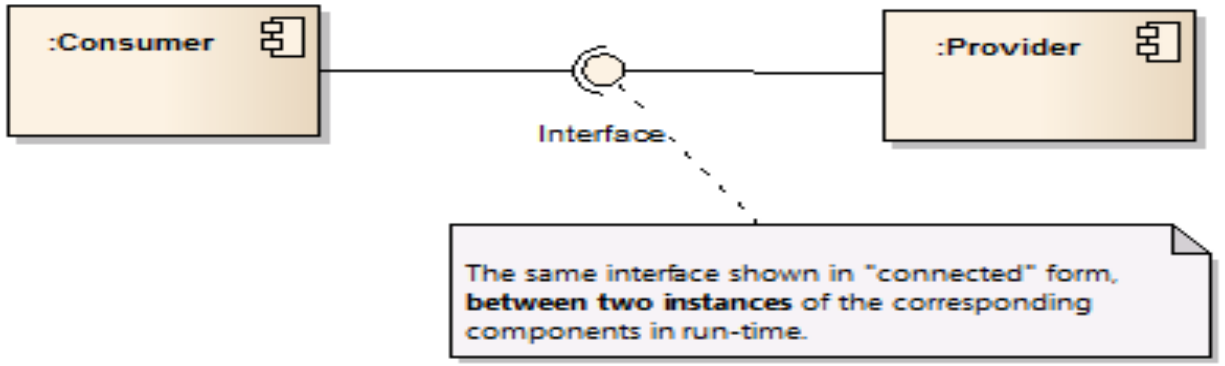


Provided and required interface always refer to the concept of interface, indicating the point of view.
The diagrams sheds some light on the subject.



On the implementation level a provided interface is the interface implemented by a class (in the most common sense, e.g. a class B implements the interface I).

Required interface would be any use of an interface by a component (e.g. if a class A defines a method that has the interface I as a parameter, this means that class A has a required interface I).



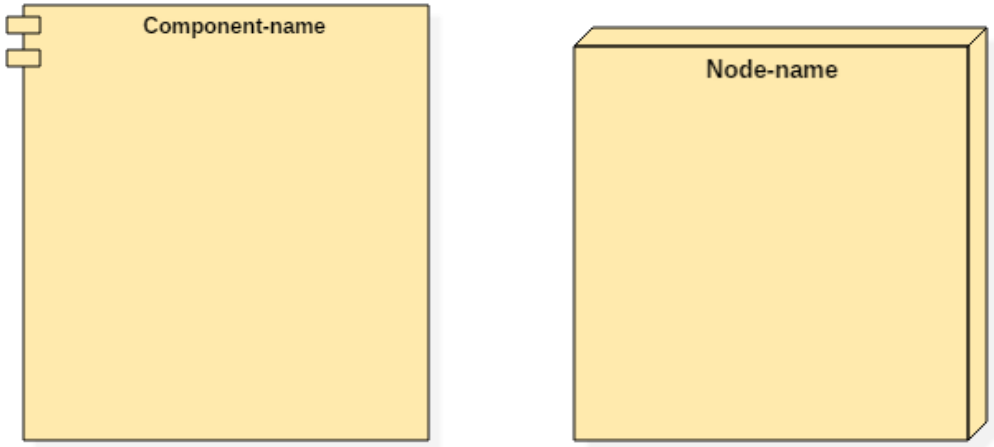
Structure of a UML Component

- A component is represented with classifier rectangle stereotypes as: **<< component >>:**
- *Component details are hidden for the outside world.*
- The name of a component is placed at the center of a rectangle.
- A component icon is displayed at the upper right corner of a rectangle, **which is optional.**

Component Diagram Notations

Component Notation in Component Diagram

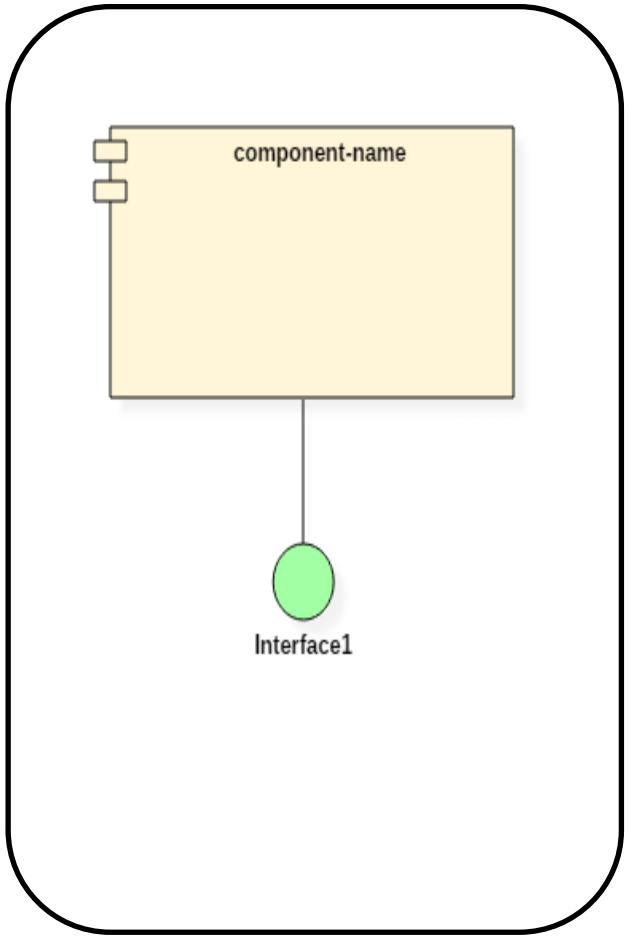
Node Notation in Component Diagram



What are nodes and components?
Components are things that participate in the execution of a system; nodes are things that execute components. Components represent the physical packaging of logical elements; nodes represent the physical deployment of components.

Interface in Component Diagram

- The Interface is a named **set of public features**.
- It separates the *specification of functionality from its implementation by a class diagram* or a subsystem.
- An interface symbol cannot be instantiated.
- It declares a **contract that may be realized** by zero or more classifiers such as a class or a subsystem.
- Anything that realizes an interface accepts the functionalities of the interface and agrees to abide by the contract defined by the interface.
- If the implementation language does not support interfaces, the use of abstract classes, interfaces are named just like classes, in **UpperCamelCase**.



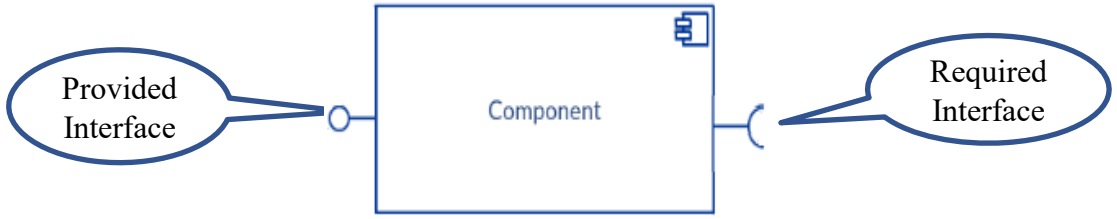
Interface in Component Diagram

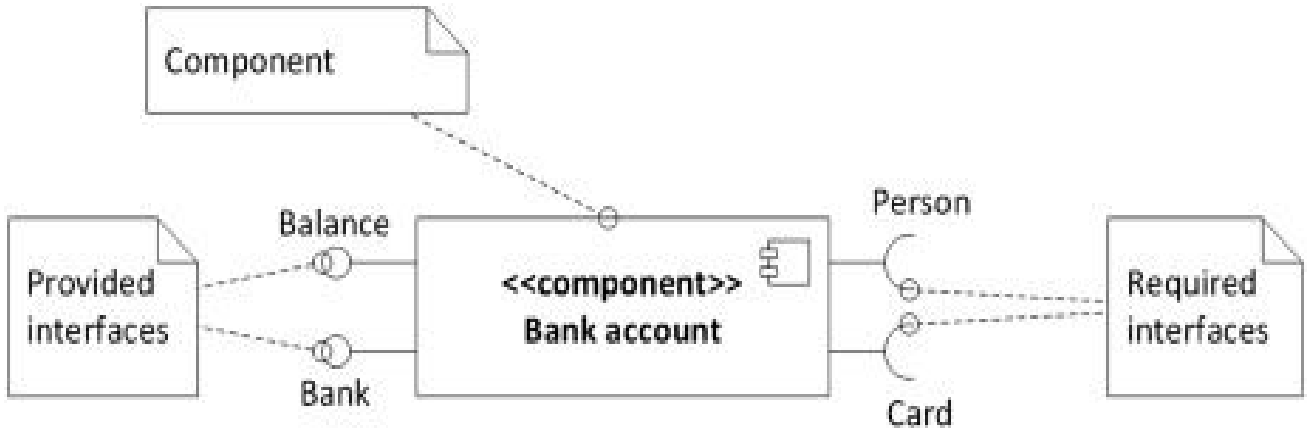
The two types of Interfaces in Component Diagram are **Provided interfaces and Required interfaces**

We can connect provided and required interfaces using assembly connector.

Advantages: It increases the flexibility and extensibility of a class and it decreases the implementation dependencies.

Disadvantages: Extra flexibility leads to complex classes and too many interfaces make systems hard to understand.





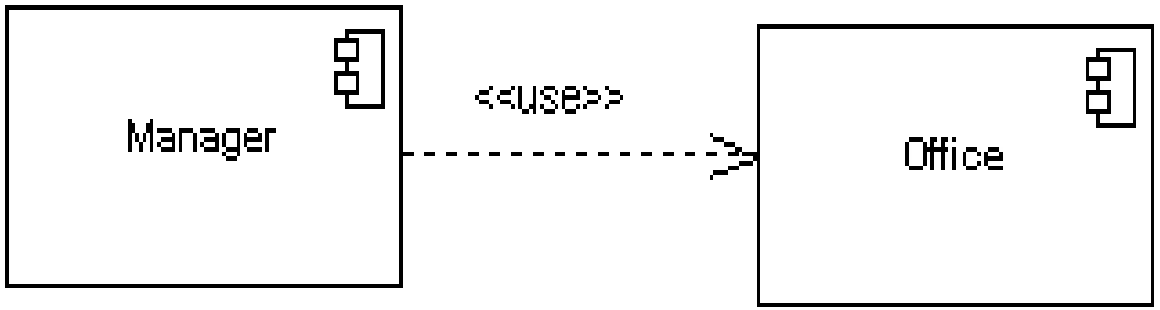
Object Oriented Analysis and Design

Component Diagram – Usage Connector

Components can be **connected by usage dependencies.**

Usage Dependency

- *A usage dependency is relationship which one element requires another element for its full implementation*
- It is a dependency in which the client requires the presence of the supplier
- **It is shown as a dashed arrow with a <<use>> keyword**
- The arrowhead point from the dependent component to the one of which it is dependent on



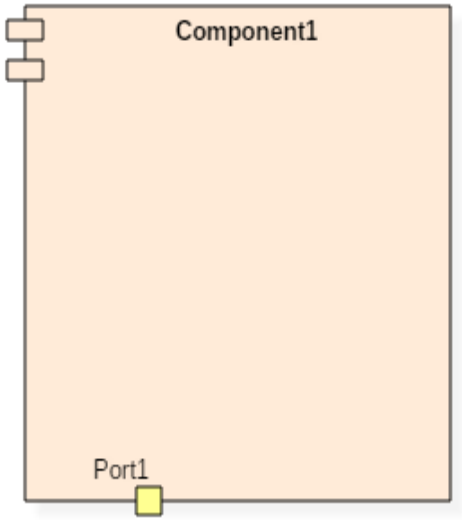
Component Diagram Subsystems

- It is a **component base that acts as a decomposition** unit for larger systems.
- It is a logical construct which is used to break down an extensive system into smaller systems which are known as subsystems.
- This process makes it easy to manage each subsystem efficiently.
- A subsystem cannot be instantiated during runtime, but their contents can be initialized. When subsystems are connected, it creates a single system.

Port in Component Diagram

- **A Port is an interaction point between a classifier and an external environment.**
- It groups semantically cohesive set of provided and required interfaces.
- A port can be used in UML **without specifying the name of the port.**
- A port may have visibility. When a port is drawn over the boundary of a classifier, then it means that the port is **public**. It also means that all the interfaces used are made as public.
- When a port is drawn inside the classifier, then it is either **protected** or **private**.
- A port also has **multiplicity that indicates the number** of instances of the port classifier will have.

A port in UML diagram is denoted as given below:



Here, the port1 is drawn over the boundary, which means it has visibility as public.

How to Draw Component Diagram

A component is nothing but an **executable piece of a system.** Various components together make a single system. Component diagrams are useful during the implementation phase of any system.

Step 1) Before modeling the component diagram, **one must know all the components within the system.** The working of each component should be mentioned. Component diagrams are used to analyze the execution of a system.

Step 2) One should also **explore each component in depth to understand the connection of a component to other** physical artifacts in the system.

Step 3) Identifying the **relationship amongst various artifacts, libraries**, and files are the essential things required during modeling of a component diagram.

Why use Component Diagram?

- These are the **static diagrams** of the unified modelling language.
- **A component diagram is used to represent the structure and organization of components during any instance of time.**
- Component diagrams are used for modelling the subsystems.
- These subsystems collectively represent the entire working view of any system.
- **A single component cannot visualize the whole system**, but the collection of multiple components can.

Why use Component Diagram?

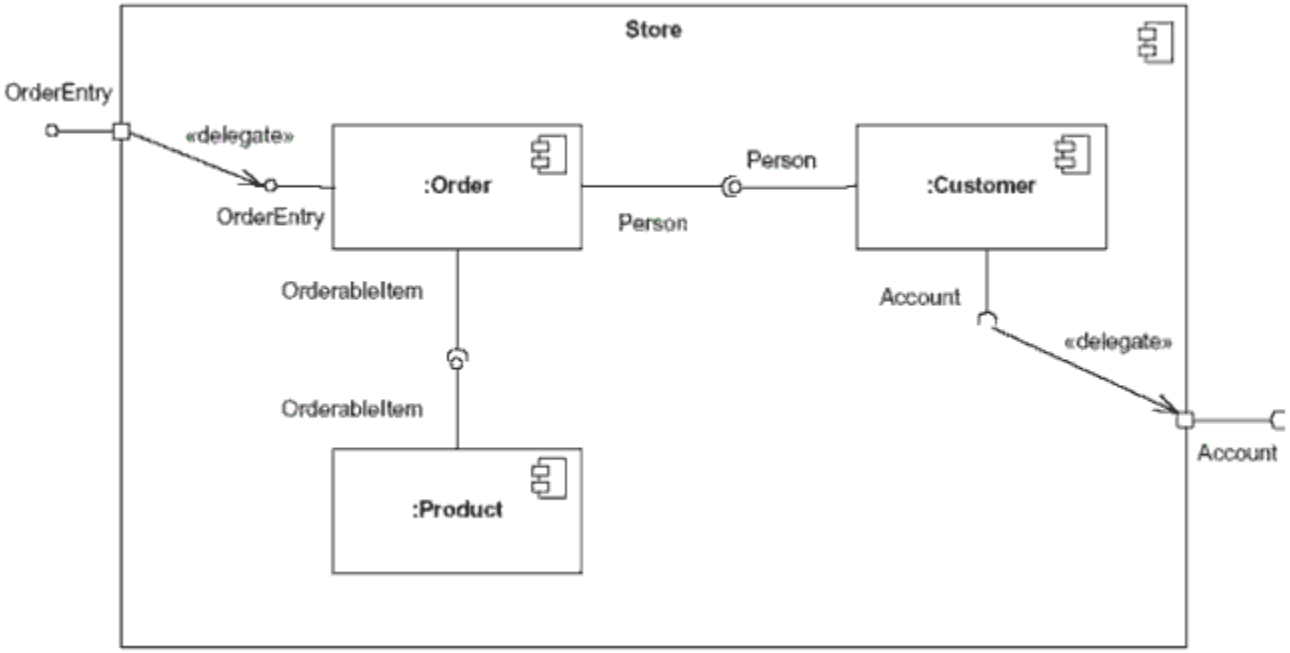
So, Component Diagrams are used for:

- To represent the **components of any system at runtime.**
- It helps during **testing of a system.**
- It visualizes the **connection between various components.**

Object Oriented Analysis and Design

Internal and External view of a Component

Internal View

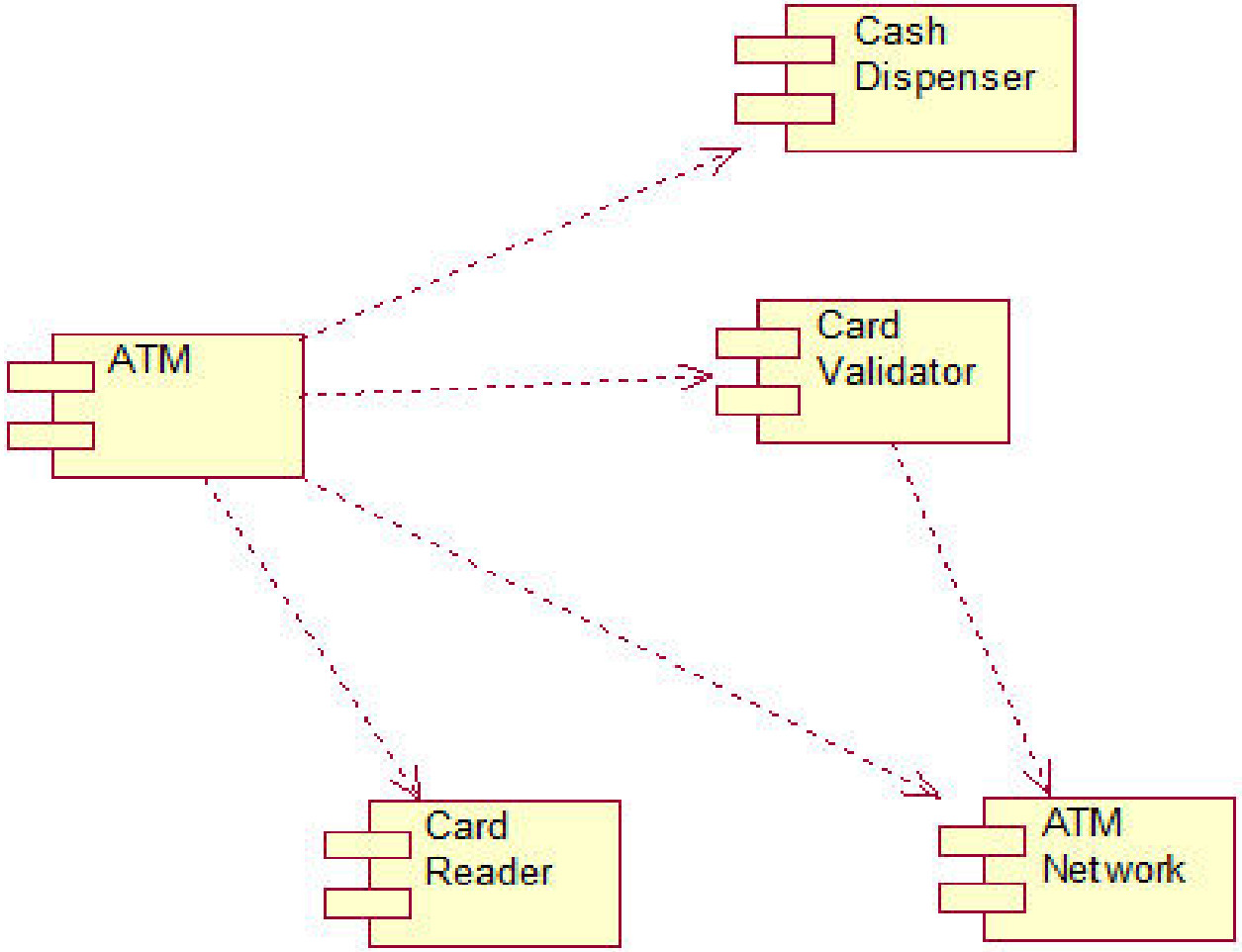


An internal view (or white box view) of a component is where the realizing classes/components are nested within the component shape

External View

An external view (or black box view) shows publicly visible properties and operations

Example – Component Diagram of an ATM



THANK YOU

Prof. Nivedita Kasturi

Department of Computer Science and Engineering

niveditak@pes.edu

Object Oriented Analysis and Design

Prof. Nivedita Kasturi
Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

Object Oriented Analysis and Design

Deployment Model

Prof. Nivedita Kasturi
Department of Computer Science and Engineering

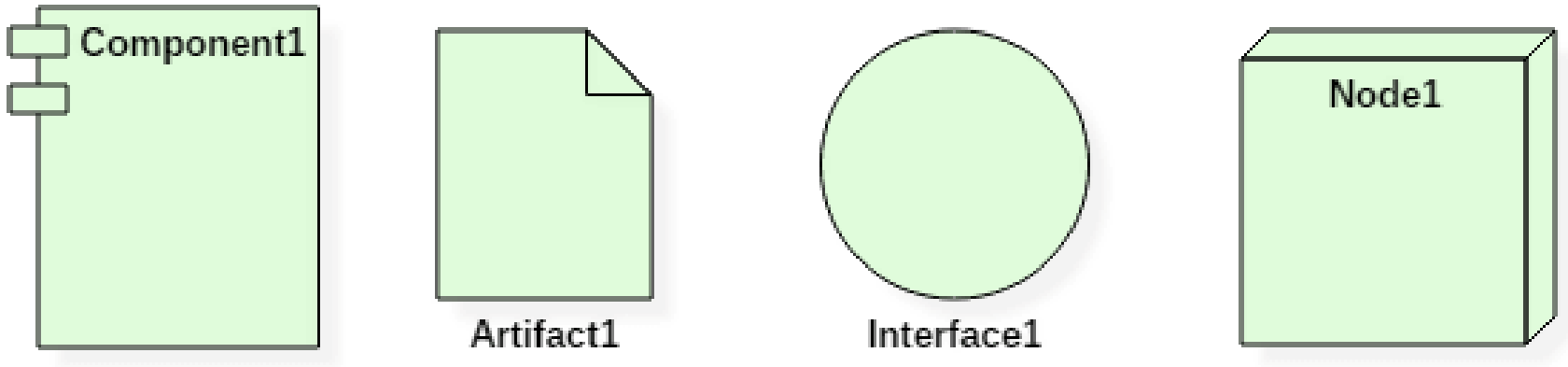
Introduction

- Software components developed as part of the development process would need to be deployed on some set of hardware or software to be executed.
- Deployment diagram maps the software architecture created in design to the physical system architecture that executes it. In distributed systems, **it models the distribution of the software across the physical nodes.**
- **Deployment diagrams shows how the components described in component diagrams are deployed in hardware.**
- Deployment diagrams are used to visualize
  - The topology of the physical components of a system, where the software components are deployed.
  - Describe the hardware components used to deploy software components.
  - Describe the runtime processing nodes (physical hardware) .

Deployment Diagram – Notations

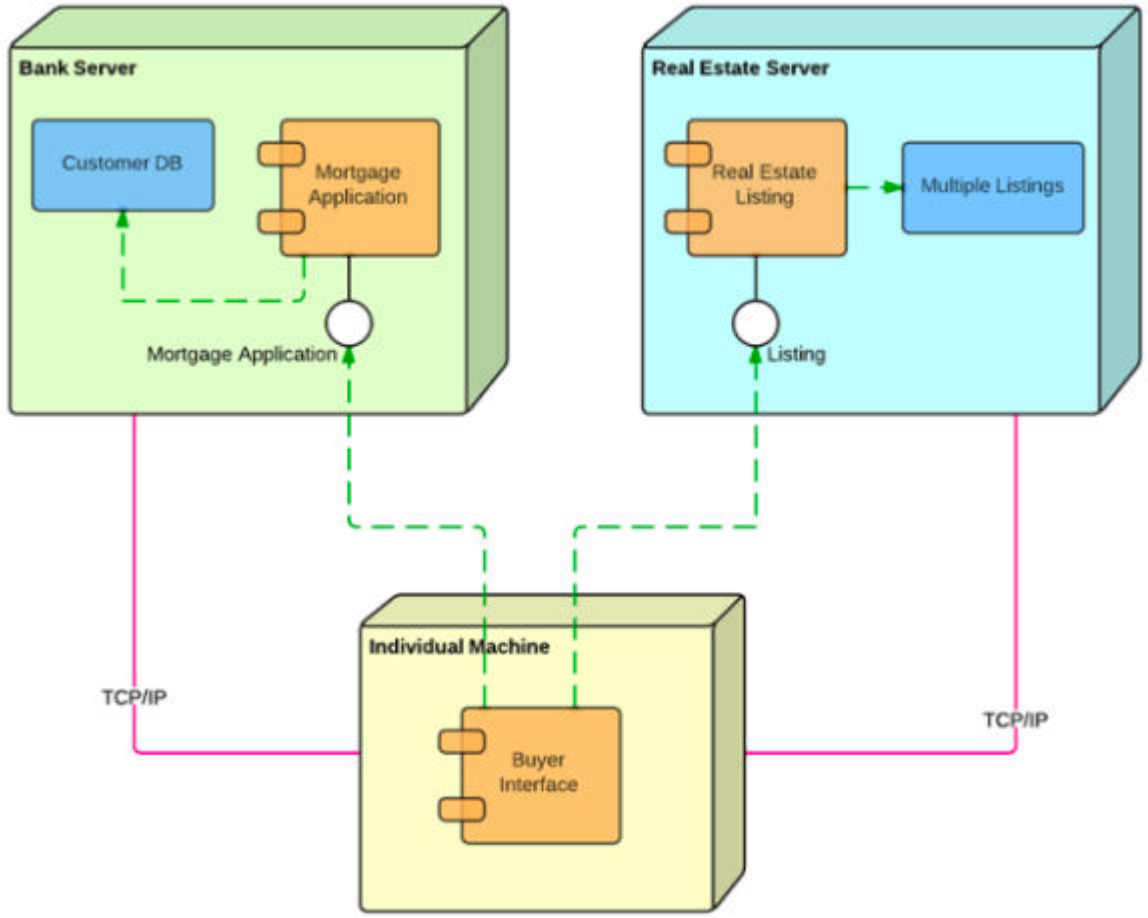
- Deployment diagram, models the **run-time** architecture of a system.
- **It shows the configuration of the hardware elements (nodes) and shows how software elements and artifacts are mapped onto those nodes.**

Some of the deployment diagram symbols and notations



Elements in a Deployment Diagram

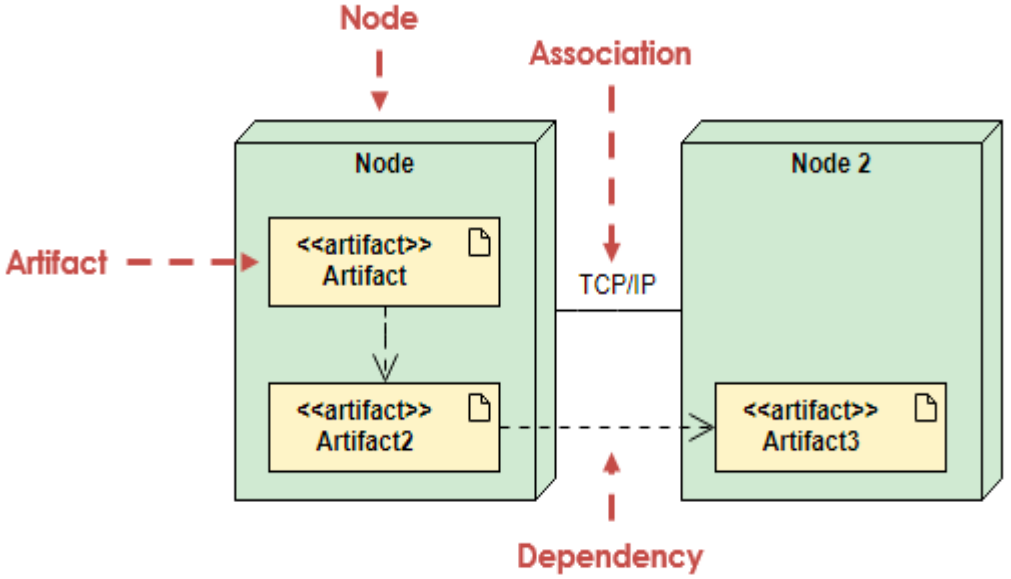
- **Artifact:** A product developed by the software, symbolized by a rectangle with the name and the word "artifact" enclosed by double arrows.
- **Association**: A line that indicates a message or other type of communication between nodes.
- **Component:** A rectangle with two tabs that indicates a software element.
- **Dependency:** A dashed line that ends in an arrow, which indicates that one node or component is dependent on another.
- **Interface:** A circle that indicates a contractual relationship. Those objects that realize the interface must complete some sort of obligation.
- **Node:** A hardware or software object, shown by a three-dimensional box.



Artifact

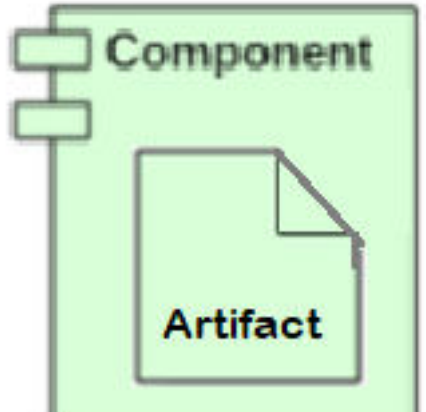
- Artifacts represent concrete elements in the physical world that are the result of a development process. **Examples of artifacts are executable files, libraries, archives, database schemas, configuration files,** etc.

- Artifacts are concrete elements that are caused by a development process. Examples of artifacts are libraries, archives, configuration files, executable files etc.



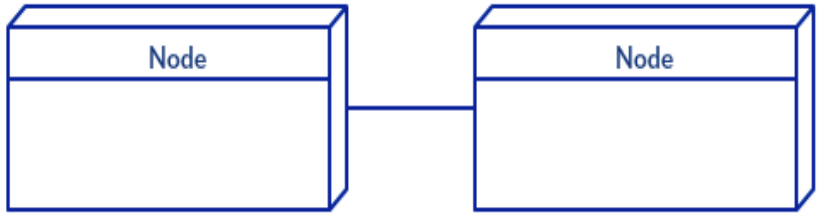
Component with Artifacts

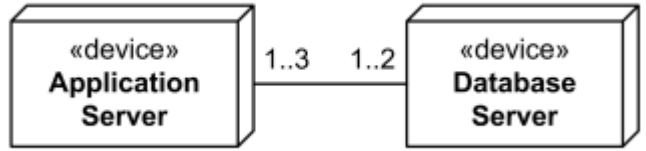
- Component with Artifacts represents the concrete real-world entity related to software development.
- Things that participate in the execution of a system or executable entities that reside in nodes.
- Represent the physical packaging of the logical elements.
- Artifacts are deployed on the nodes.
- Each artifact has a filename in its specification that indicates the physical location of the artifact.



Associations

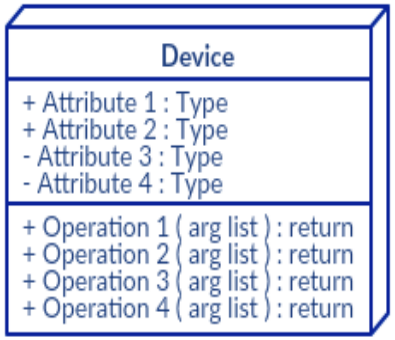
Communication Association

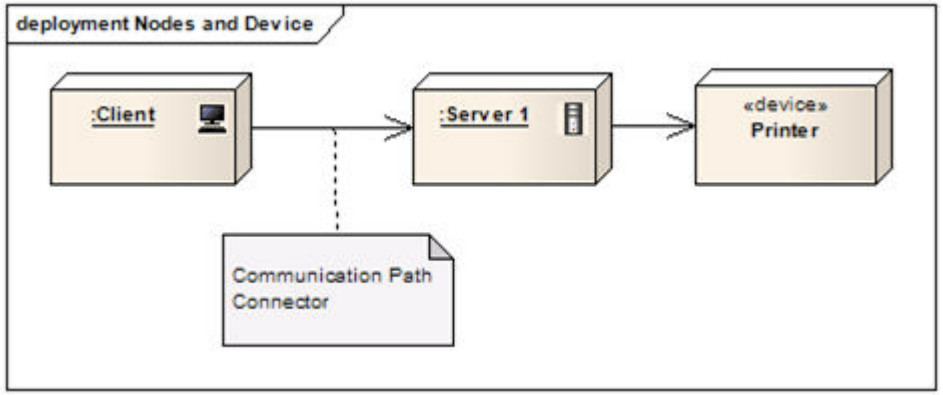




This is represented by a solid line between two nodes. It shows the path of communication between nodes.

Devices

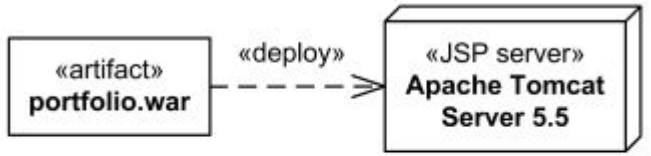




A device is a node that is used to represent a physical computational resource in a system. An example of a device is an application server.

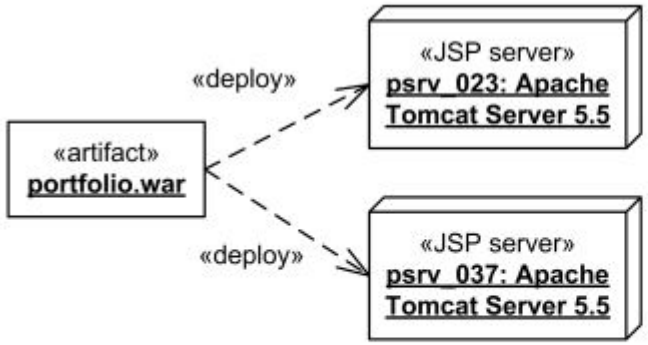
Associations…(cntd.)

- Deployment could be shown as a dependency that is drawn from the artifact (supplier) to the deployment target (client) and is labeled with «deploy».



J2EE web application archive portfolio.war deployed on Apache Tomcat JSP server.

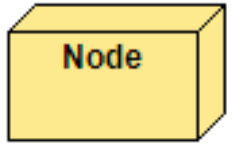
- At the "instance level" instances of artifacts could be deployed to specific instances of the deployment target.

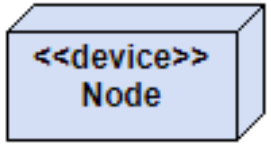


J2EE web application archive portfolio.war deployed on two instances of Apache Tomcat JSP server - psrv_023 and psrv_037.

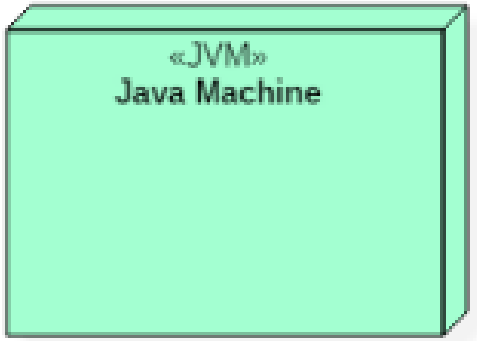
Nodes

- Node : Node is a computational resource upon which component artifacts are deployed for execution.





device node



execution environment node

- There are two types of Nodes:
  - **Device Node**
  - **Execution Environment Node**

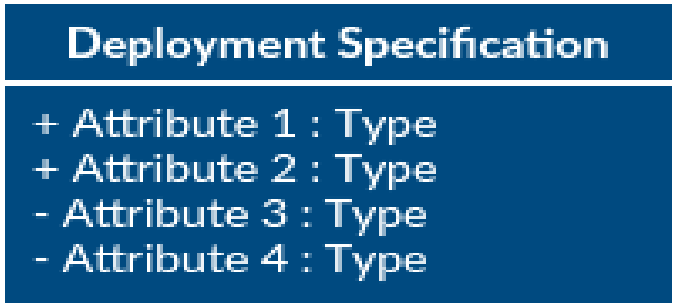
Device nodes are physical computing resources with processing memory and services to execute software, such as typical computers or mobile phones.

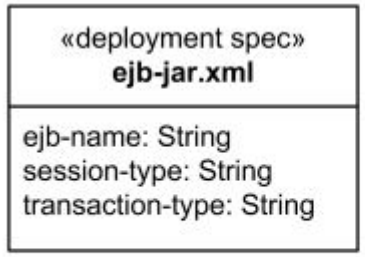
An execution environment node (EEN) is a software computing resource that runs within an outer node and which itself provides a service to host and execute other executable software elements.

Deployment Specifications

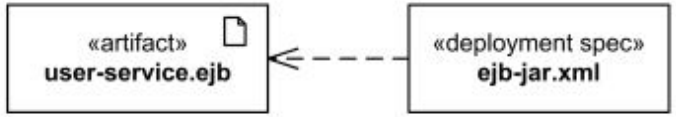
- Deployment specifications is a configuration file, such as a text file or an XML document. It describes how an artifact is deployed on a node.

- A deployment specification at specification level is rendered as a classifier rectangle with optional deployment properties in a compartment.

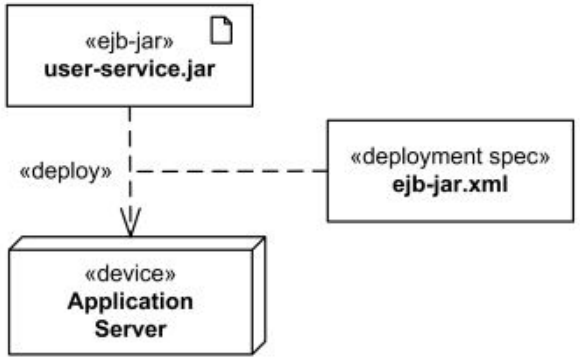




The ejb-jar.xml deployment specification



The ejb-jar.xml deployment specification for user-service.ejb artifact.



The ejb-jar.xml deployment specification attached to deployment.

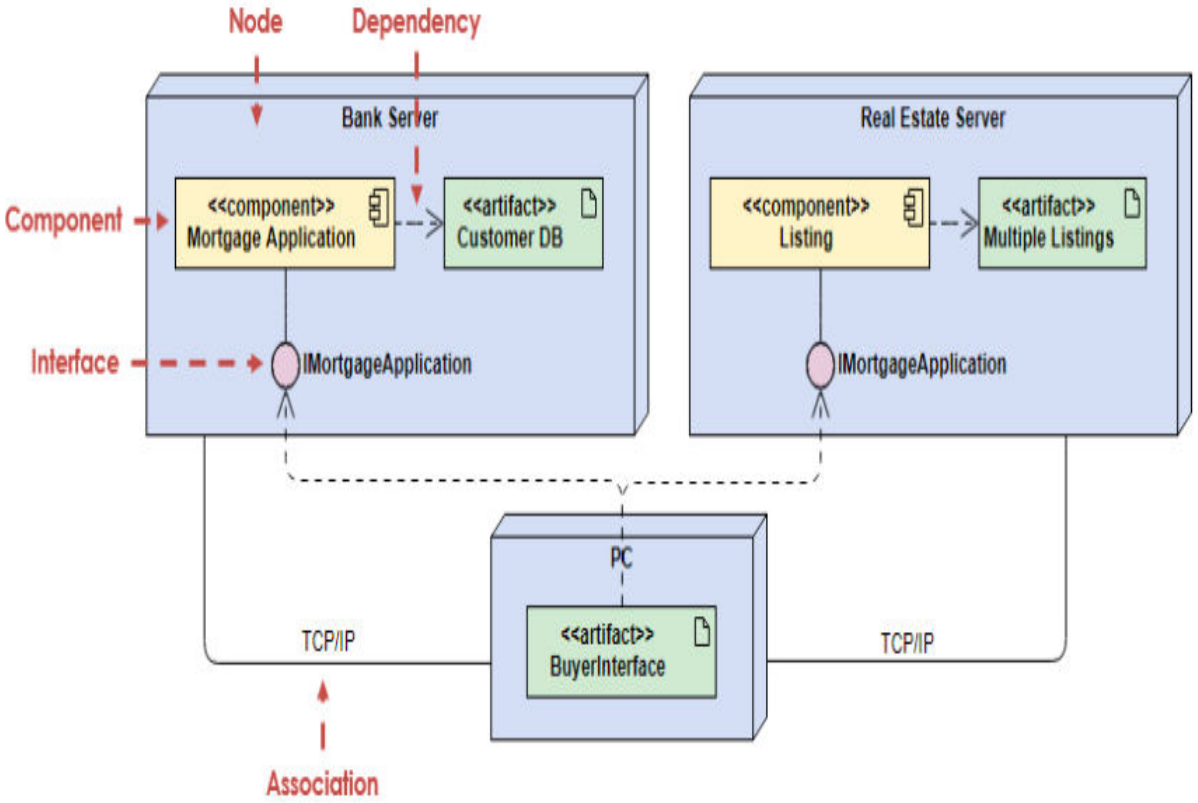
How to Draw deployment Diagram

Step 1: Identify the purpose of your deployment diagram. And to do so, you need to identify the nodes and devices within the system you'll be visualizing with the diagram.

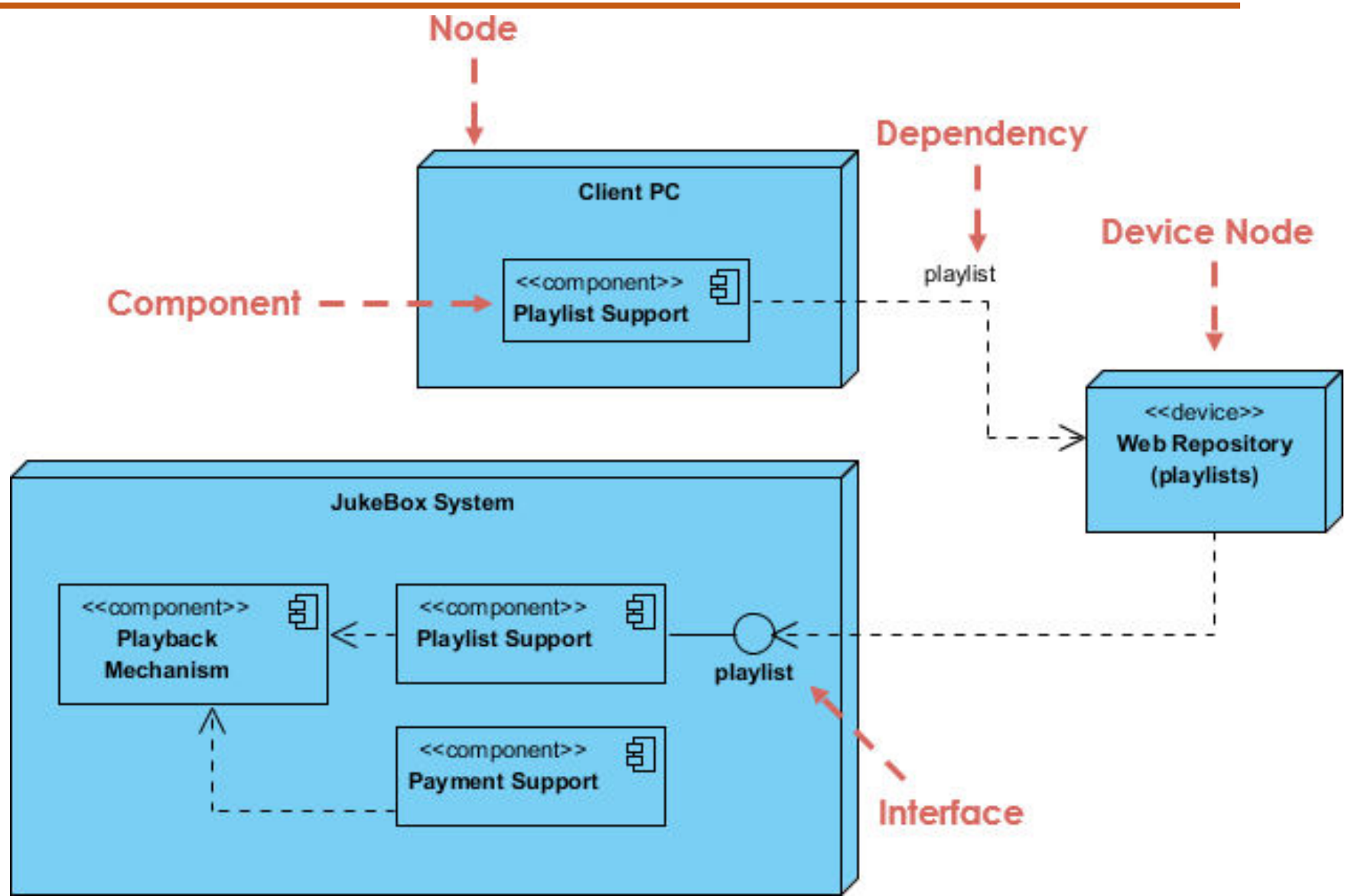
Step 2: Figure out the relationships between the nodes and devices. Once you know how they are connected, proceed to add the communication associations to the diagram.

Step 3: Identify what other elements like components, active objects you need to add to complete the diagram.

Step 4: Add dependencies between components and objects as required.

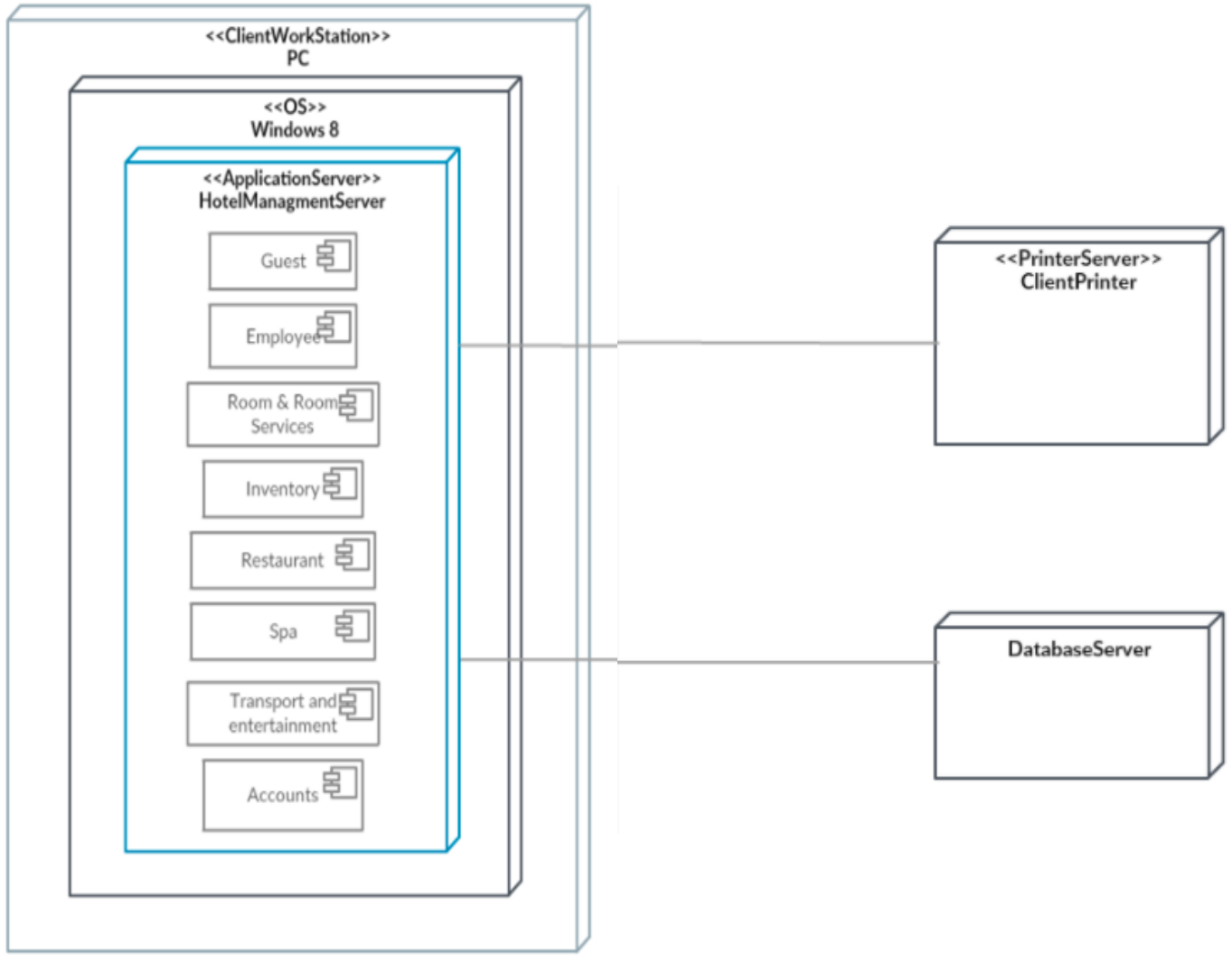


Example – Jukebox

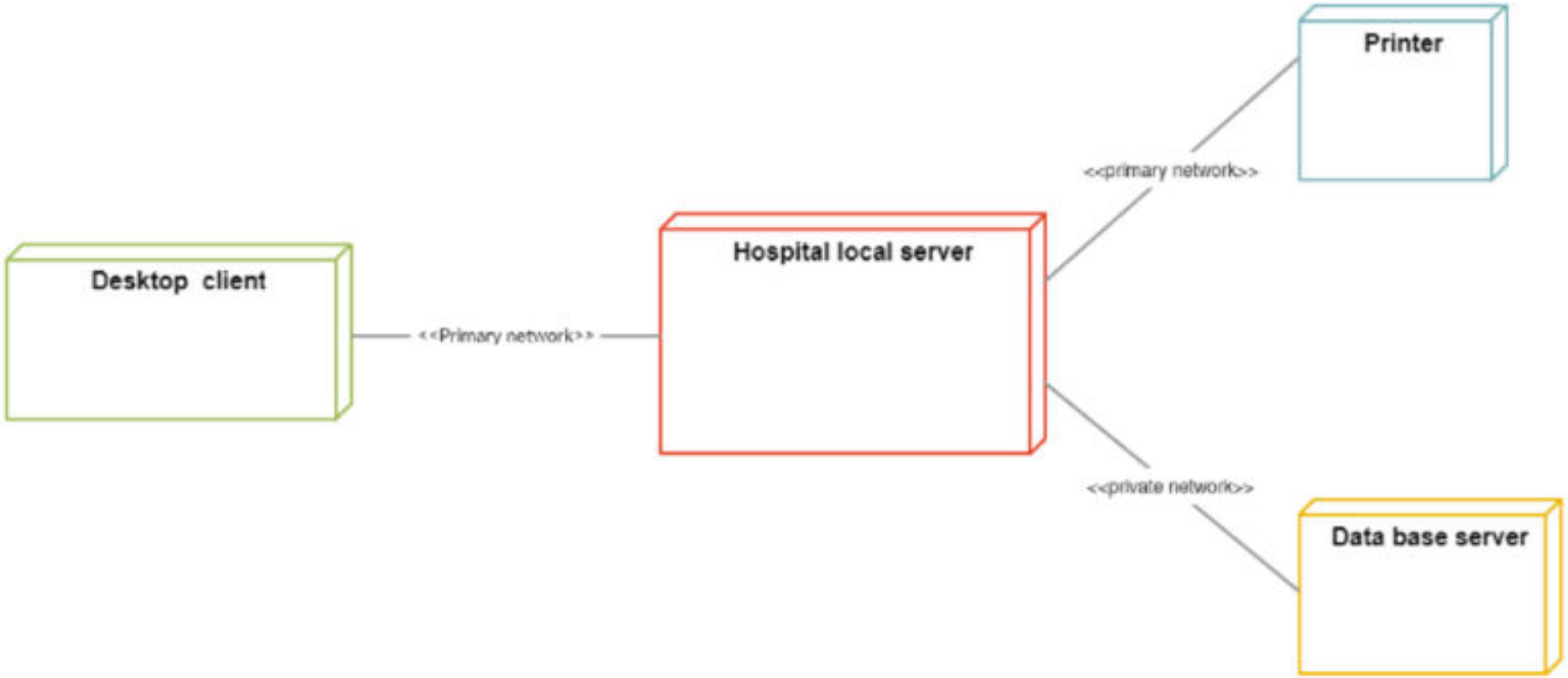


Object Oriented Analysis and Design

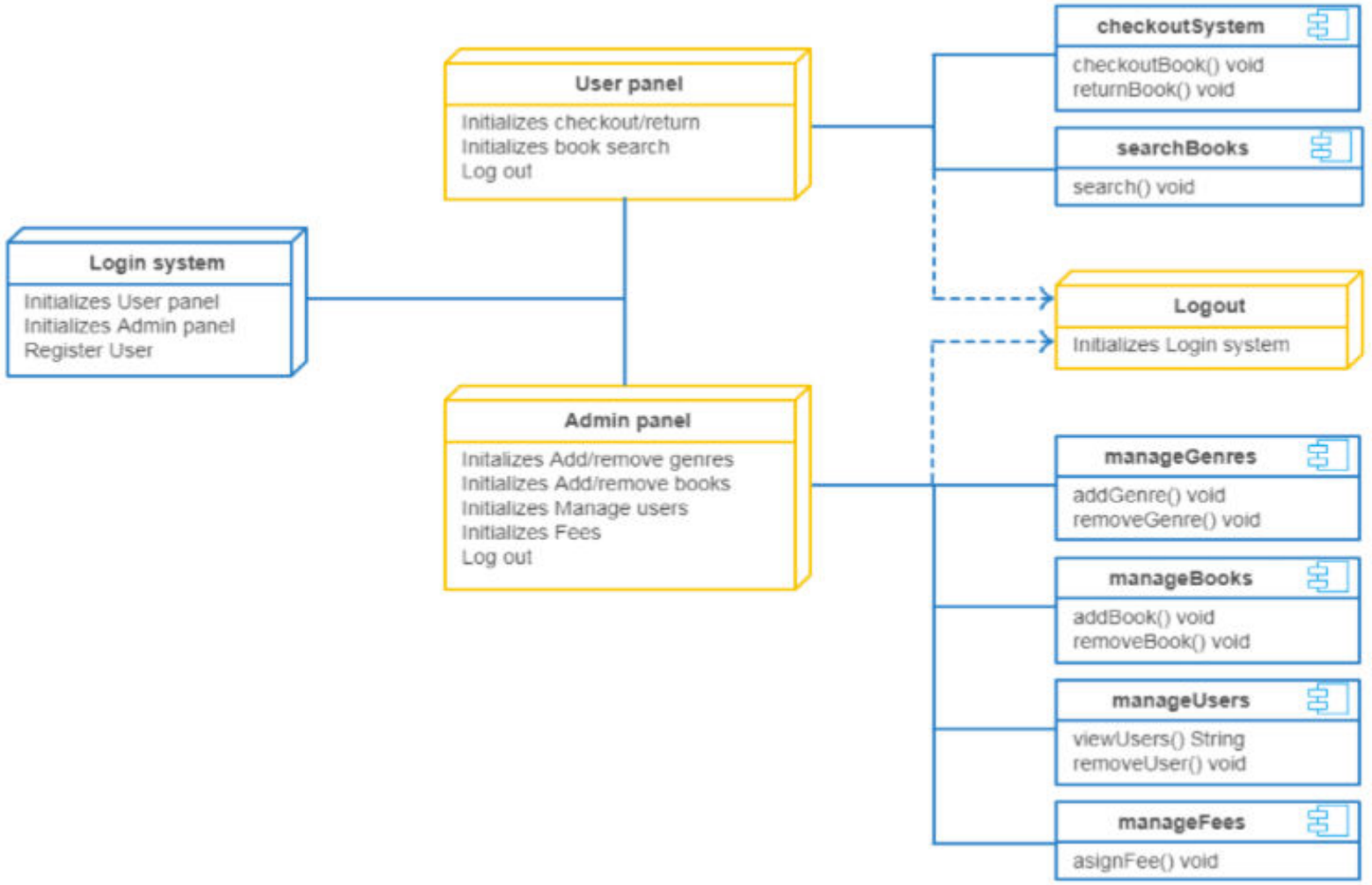
Example – Hotel Management System



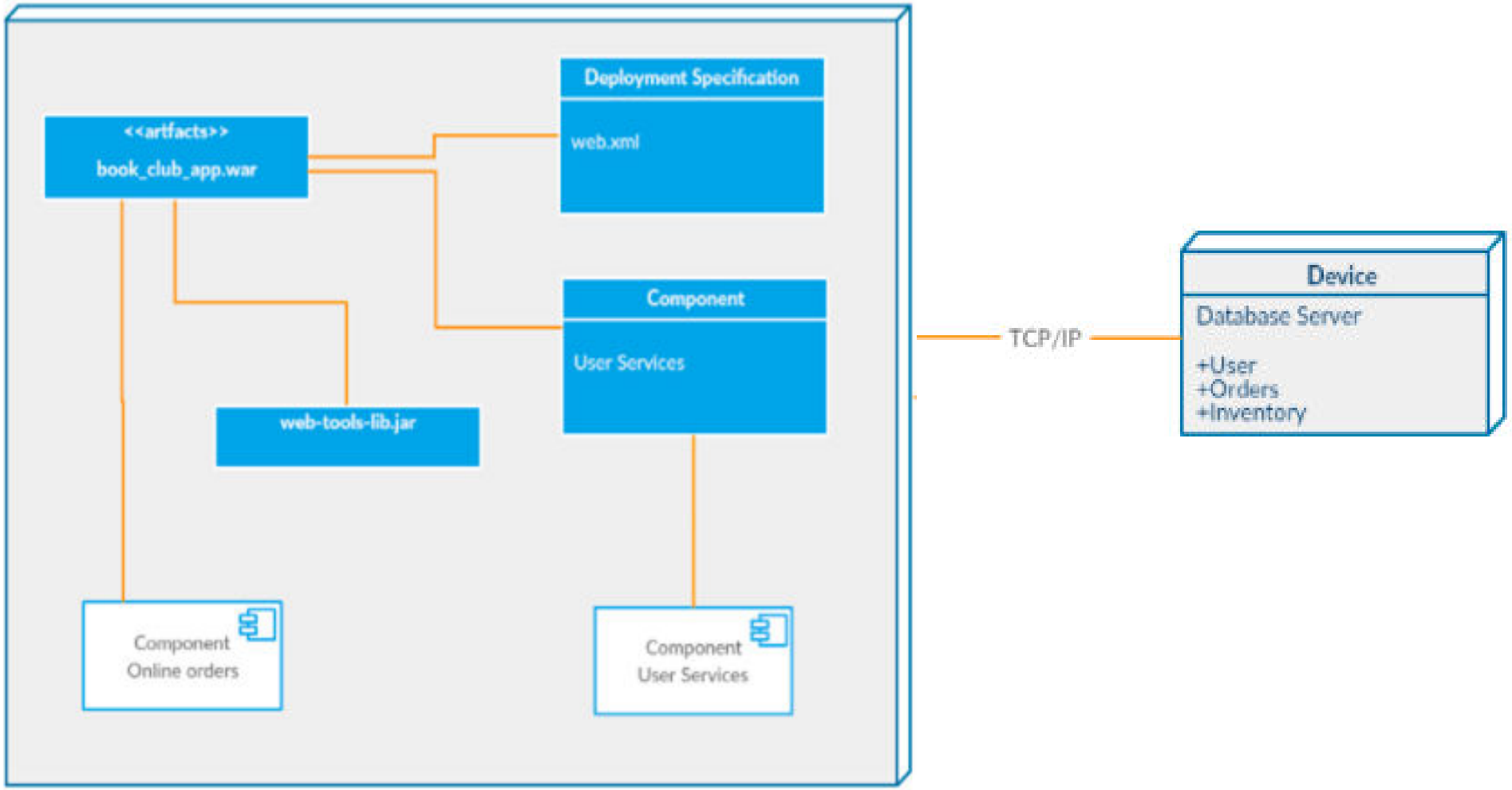
Example – Hospital Management System



Example – Library Management System



Example – Online Shopping System



Example – Modeling a Distributed System

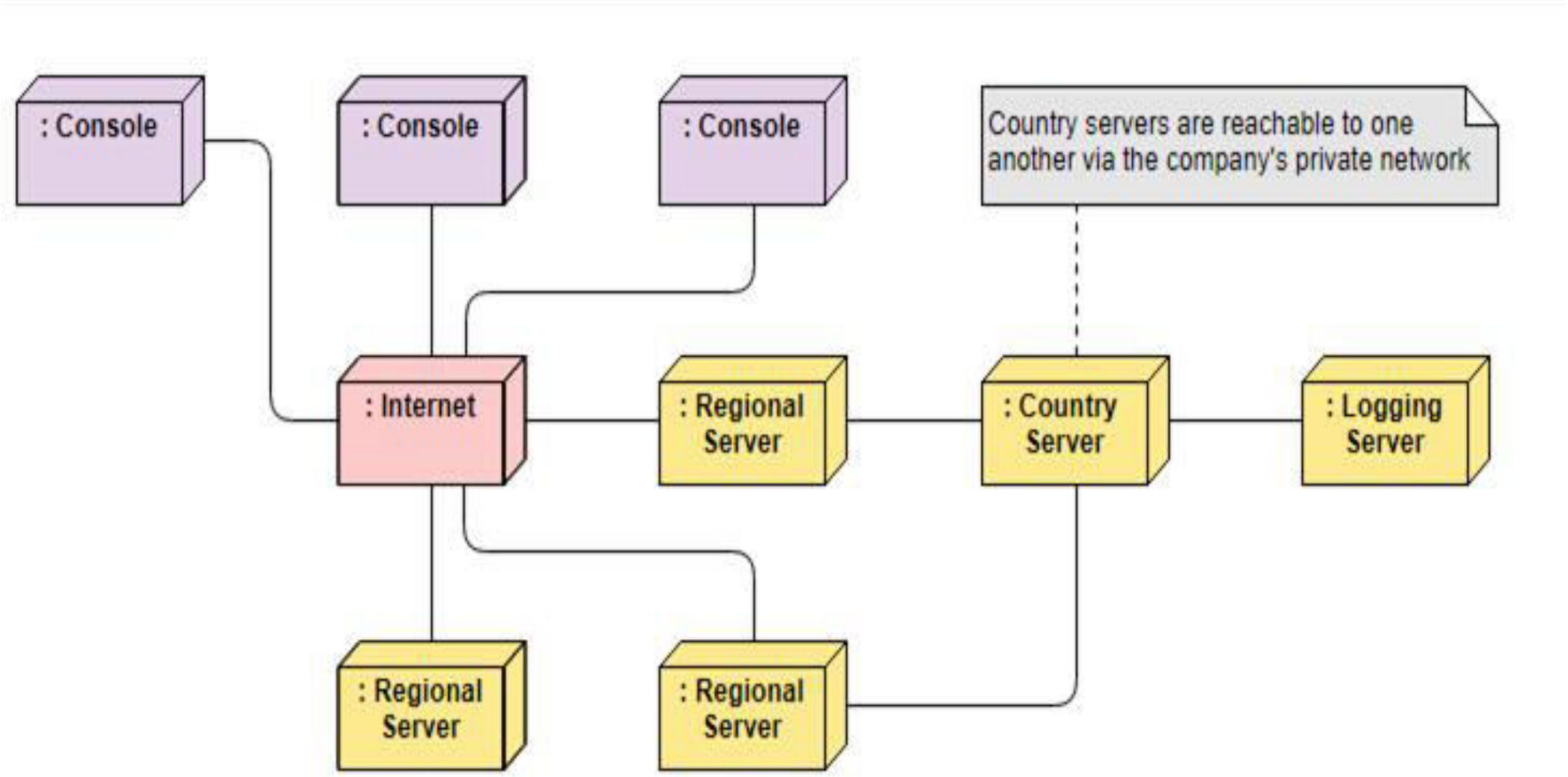
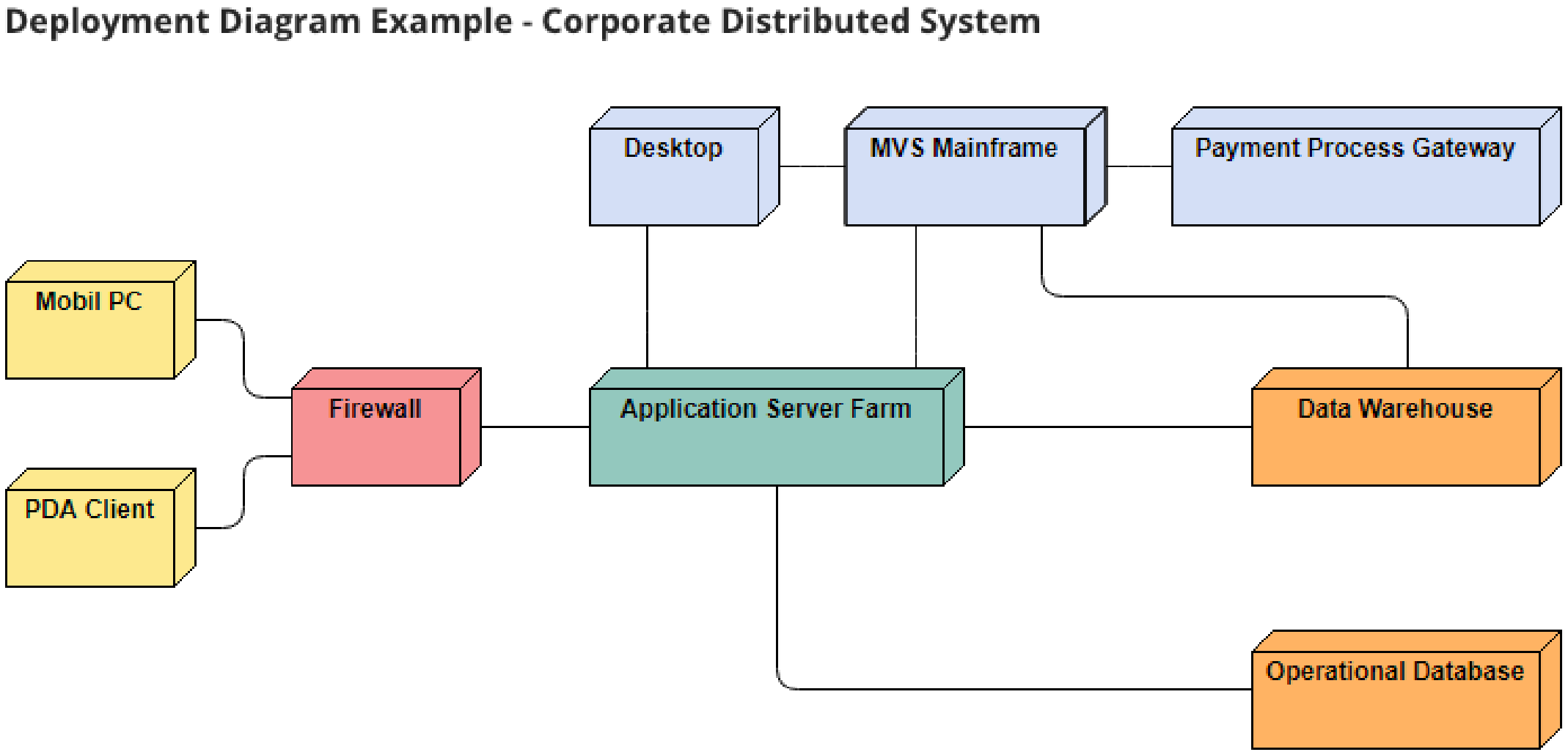


Illustration of a Deployment Diagram



THANK YOU

Prof. Nivedita Kasturi

Department of Computer Science and Engineering

Niveditak@pes.edu

Object Oriented Analysis and Design

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

Object Oriented Analysis and Design

Activity Modelling: UML Activity Diagrams, Modelling and Guidelines

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Interaction Modeling

The interaction model describes how objects interact to produce useful results. It is a holistic view of behavior across many objects

Interactions can be modeled at different levels of abstraction.

- ❑ Use Case Model
- ❑ Sequence Model
- ❑ Activity Model

Interaction Modeling

Use Case Model:

At a high level, use cases describe how a system interacts with outside actors. Each use case represents a piece of functionality that a system provides to its users. Use cases are helpful for capturing informal requirements.

Sequence Model:

Sequence diagrams provide more detail and show the messages exchanged among a set of objects over time. Messages include both asynchronous signals and procedure calls. Sequence diagrams depicts the behavior sequences seen by users of a system.

Interaction Modeling

Activity Model:

- Activity diagrams provide further detail and show the flow of control among the steps of a computation.
- Activity diagrams can show data flows as well as control flows.
- Activity diagrams document the steps necessary to implement an operation or a business process referenced in a sequence diagram.

Activity Diagram

- An activity diagram visually presents a series of actions or flow of control in a system similar to a flowchart or a data flow diagram.
- Activity diagrams are often used in business process modeling. They can also describe the steps in a use case diagram.
- **Activities modeled can be sequential and concurrent.**
- In both cases an activity diagram will have a beginning (an initial state) and an end (a final state).

Activity Diagram: Notations and Symbols

Initial State or Start Point

A small filled circle followed by an arrow represents the initial action state or the start point for any activity diagram. For activity diagram using swimlanes, make sure the start point is placed in the top left corner of the first column.

Start Point/Initial State

Activity Diagram: Notations and Symbols

Activity or Action State

An action state represents the **non-interruptible action** of objects. A rectangle with rounded corners represent an activity or an action state.

Activity

Activity Diagram: Notations and Symbols

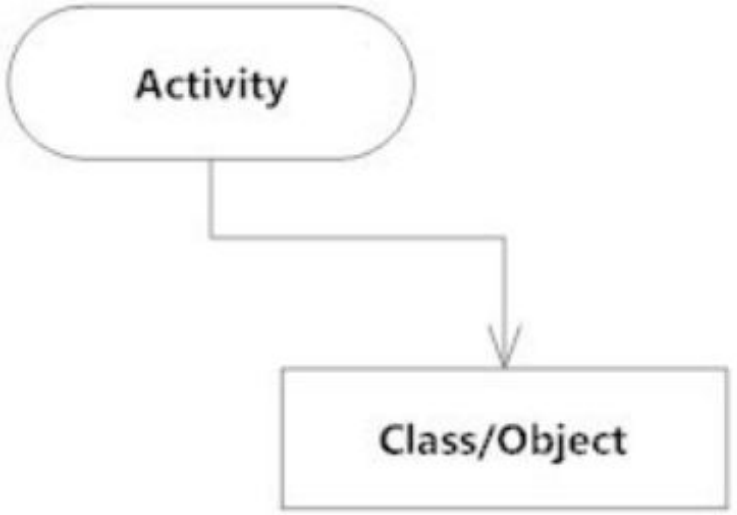
Action Flow:

Action flows, also called edges and paths, illustrate the transitions from one action state to another. They are usually drawn with an arrowed line.

Activity Diagram: Notations and Symbols

Object Flow:

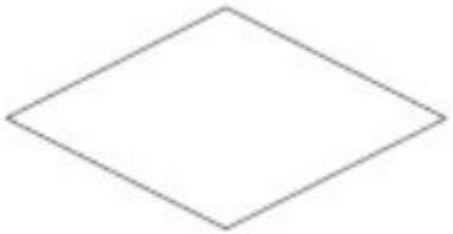
- Object flow refers to the creation and modification of objects by activities.
- An object flow arrow from an action to an object means that the action **creates or influences the object.**
- An object flow arrow from an object to an action indicates that the **action state uses the object.**



Activity Diagram: Notations and Symbols

Decisions and Branching:

A diamond represents a decision with alternate paths. When an activity requires a decision prior to moving on to the next activity, add a diamond between the two activities. The outgoing alternates should be labeled with a condition or guard expression. One of the paths can be labeled as "else“.

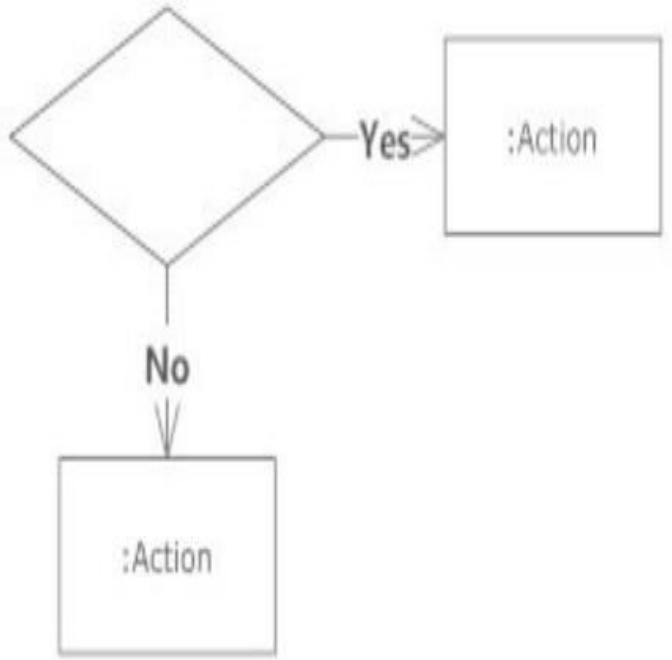


Decision Symbol

Activity Diagram: Notations and Symbols

Guards:

In UML, guards are a statement written next to a decision diamond that must be true before moving next to the next activity.
These are not essential, but are useful when a specific answer, such as "Yes, three labels are printed," is needed before moving forward.

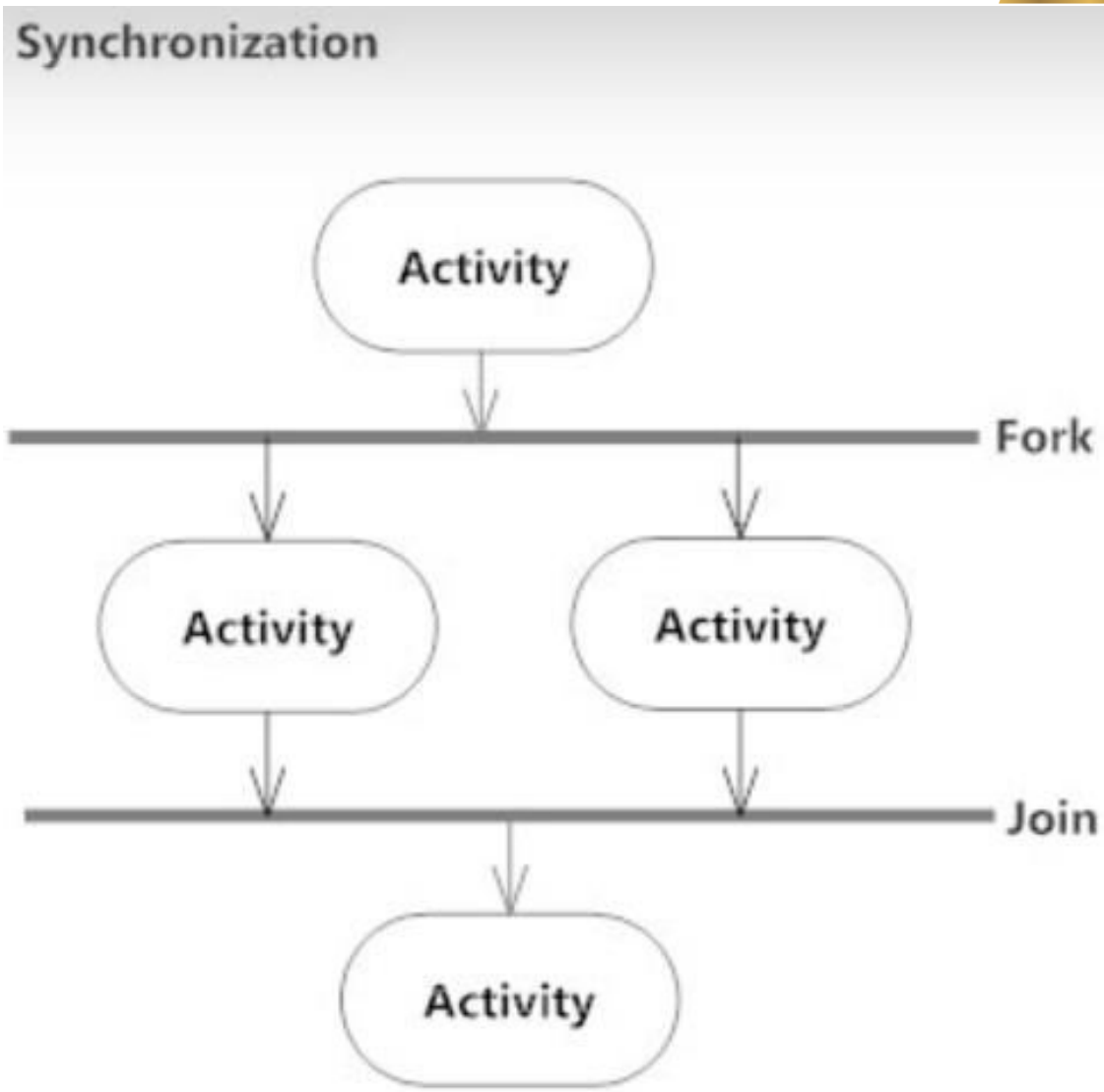


Guard Symbols

Activity Diagram: Notations and Symbols

Synchronization :

- A **fork** is used to split a single incoming flow into multiple concurrent flows. It is represented as a straight, slightly thicker line in an activity diagram.
- A **join** is used to join multiple concurrent flows back into a single outgoing flow.
- A fork and join mode used together are often referred to as synchronization. It helps **to show parallel occurrence of activities.**



Activity Diagram: Notations and Symbols

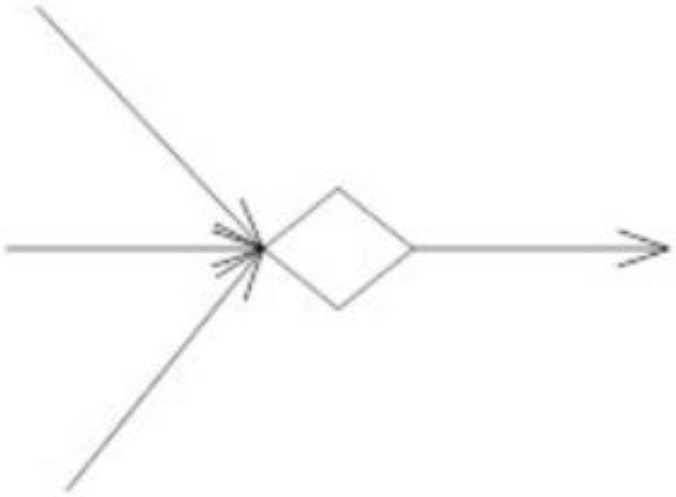
Time Event:

- This refers to an event that stops the flow for a time
- An hourglass depicts it.

Activity Diagram: Notations and Symbols

Merge Event:

A merge event brings together multiple flows that are not concurrent. A merge node combines mutually exclusive alternative flows usually after the decisior



Merge

Activity Diagram: Notations and Symbols

Sent and Received Signals

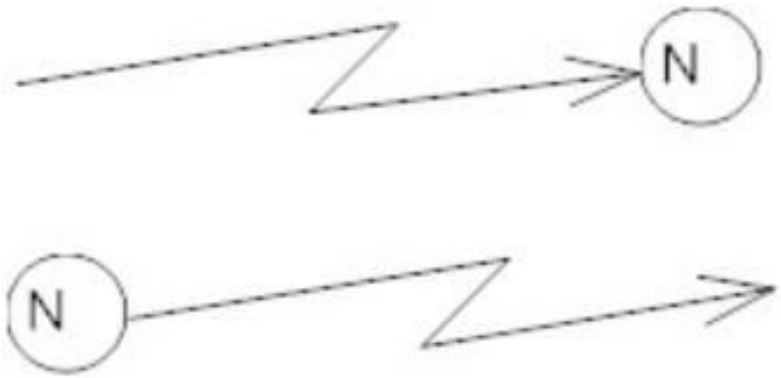
- Signals represent how activities can be modified from outside the system
- They usually appear in pairs of sent and received signals, because the state can't change until a response is received
- For example, an authorization of payment is needed before an order can be completed

Signal sent and received

Activity Diagram: Notations and Symbols

Interrupting Edge:

An event, such as a cancellation, that interrupts the flow denoted with a lightning bolt.



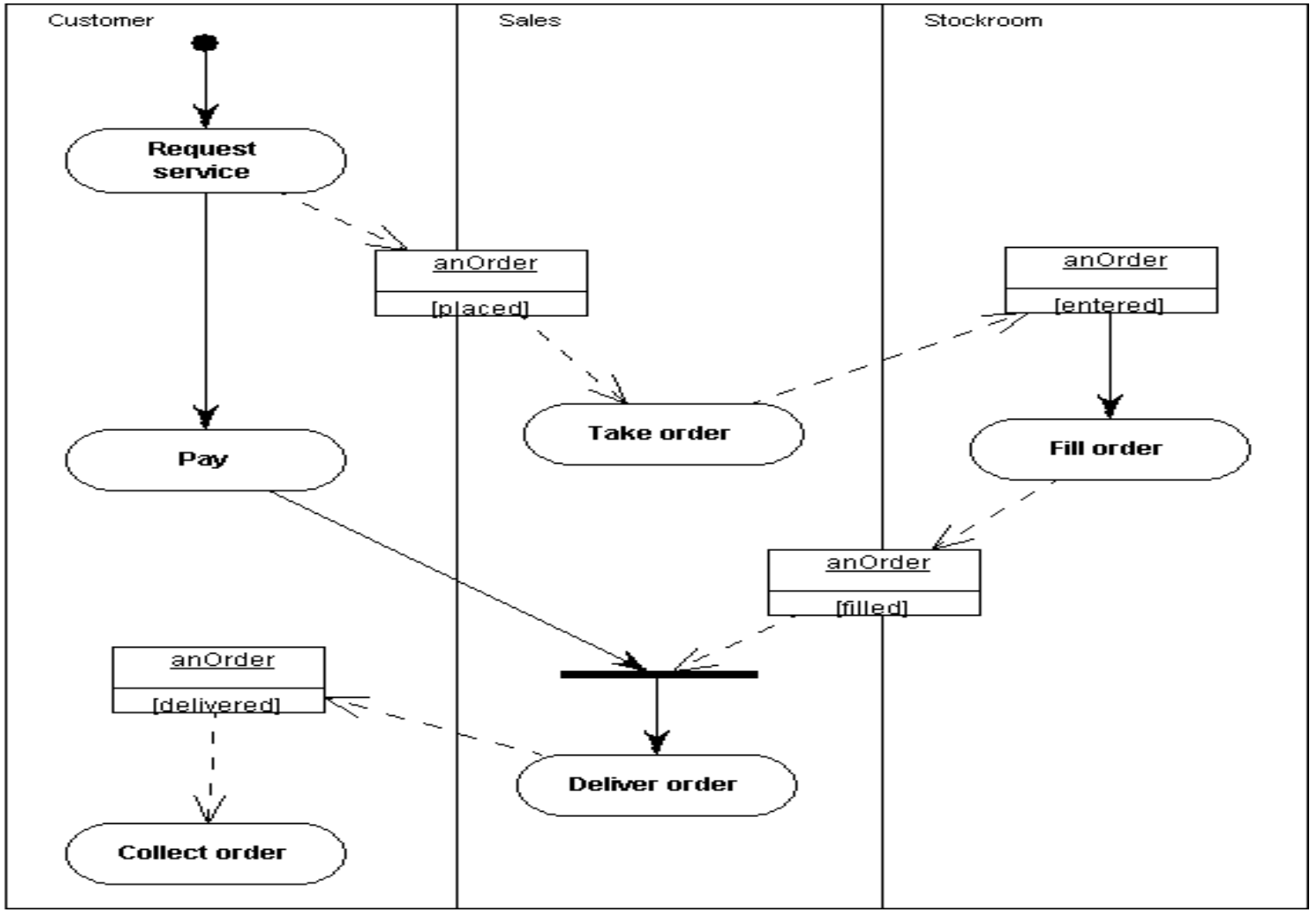
Interrupting Edge Symbols

Activity Diagram: Notations and Symbols

Swimlanes:

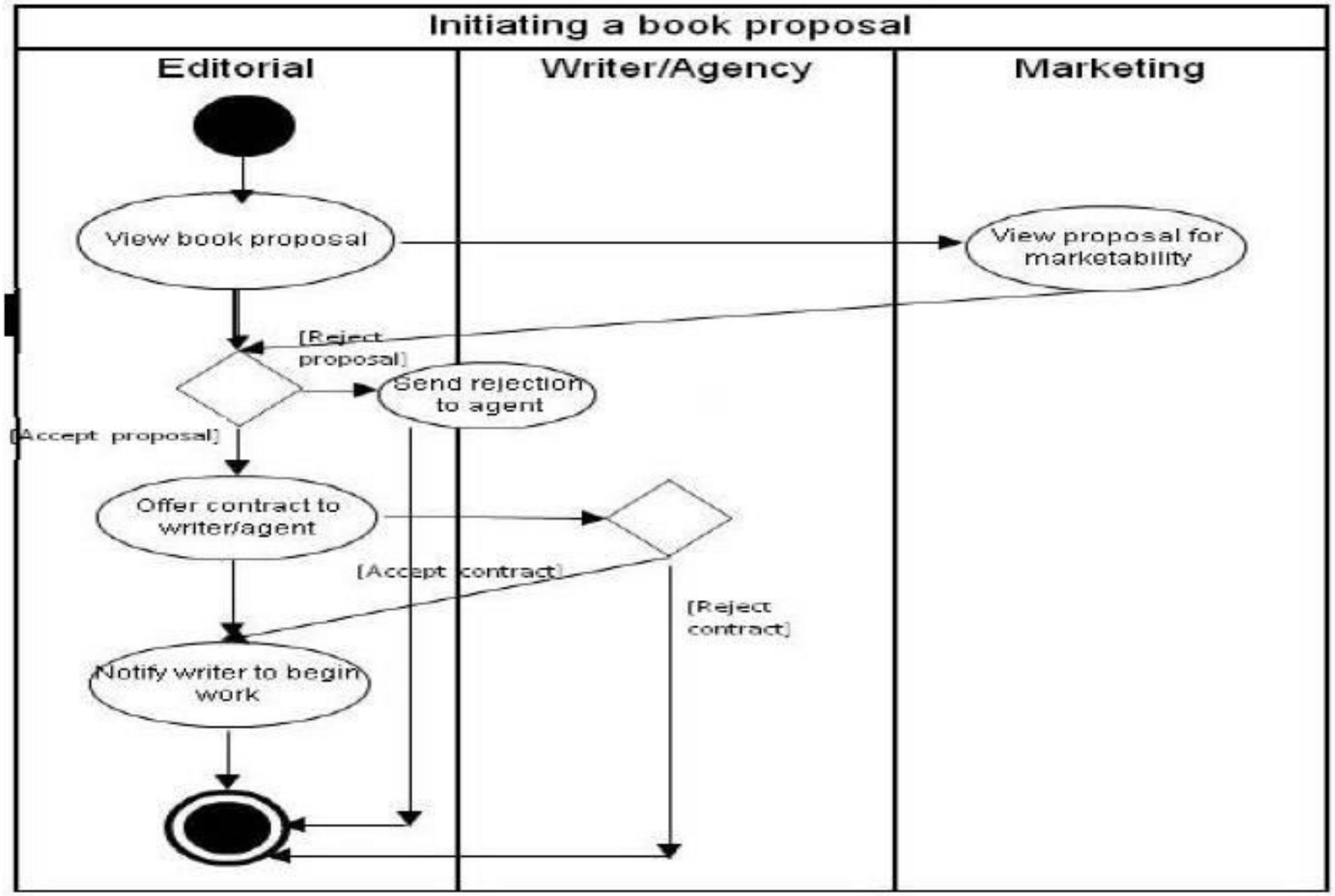
Swimlanes group related activities into one column

A dotted (dashed) arrow represents an object flow, showing the movement of data or objects between activities, not control flow.



Activity Diagram: Notations and Symbols

Swimlanes:



Activity Diagram: Notations and Symbols

Final State or End Point

An arrow pointing to a filled circle nested inside another circle represents the final action state.

End Point Symbol

Steps to Construct an Activity Diagram

Step 1: Figure out the action steps from the use case

Here you need to identify the various activities and actions your business process or system is made up of.

Step 2: Identify the actors who are involved

If you already have figured out who the actors are, then it's easier to discern each action they are responsible for.

Step 3: Find a flow among the activities

Figure out in which order the actions are processed. Mark down the conditions that have to be met in order to carry out certain processes (which actions occur at the same time, whether you need to add any branches in the diagram, do you have to complete some actions before you can proceed to others?).

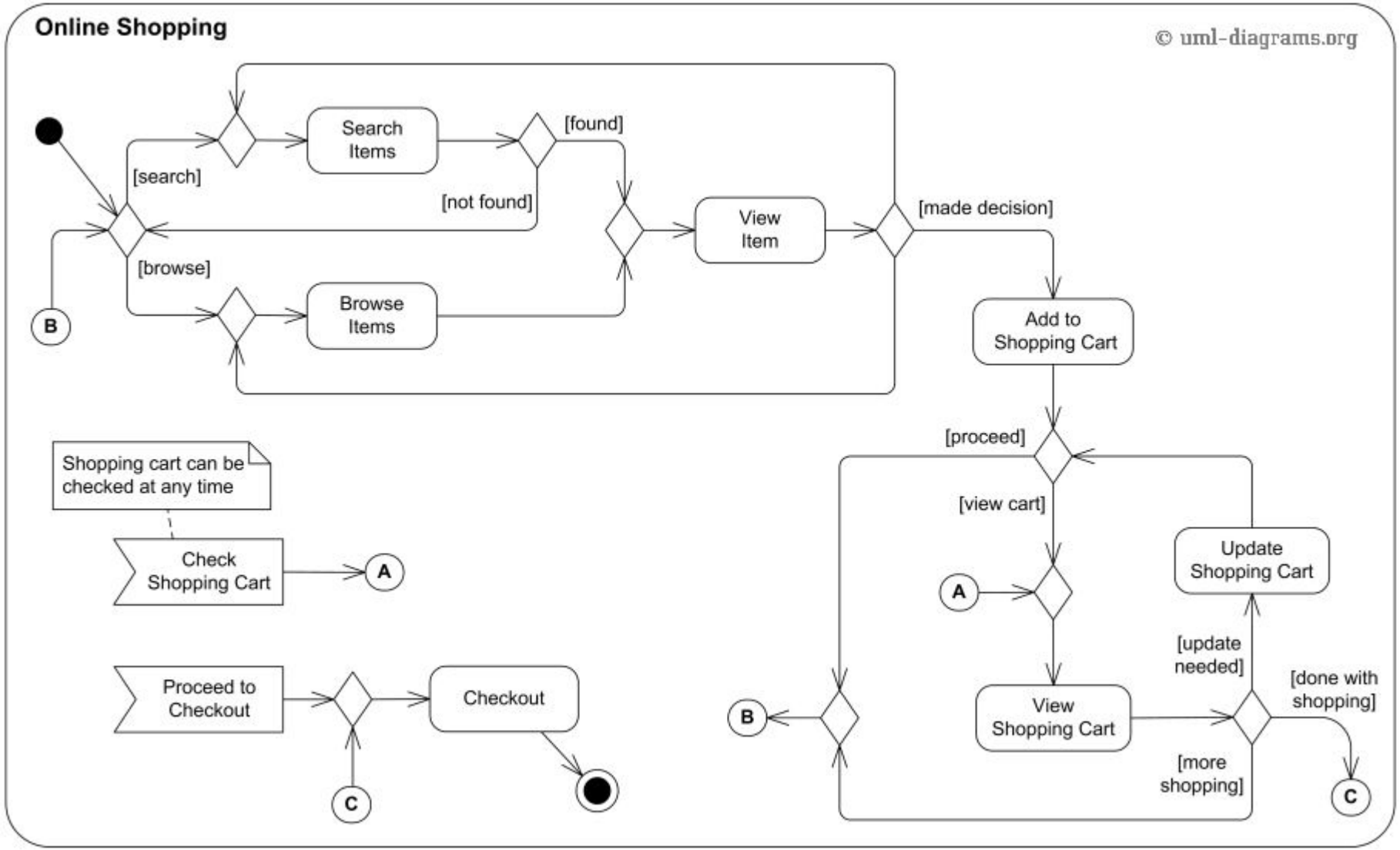
Step 4: Add swimlanes

Figure out who is responsible for each action. And assign them to a swimlanes by grouping each action the objects are responsible for, under them.

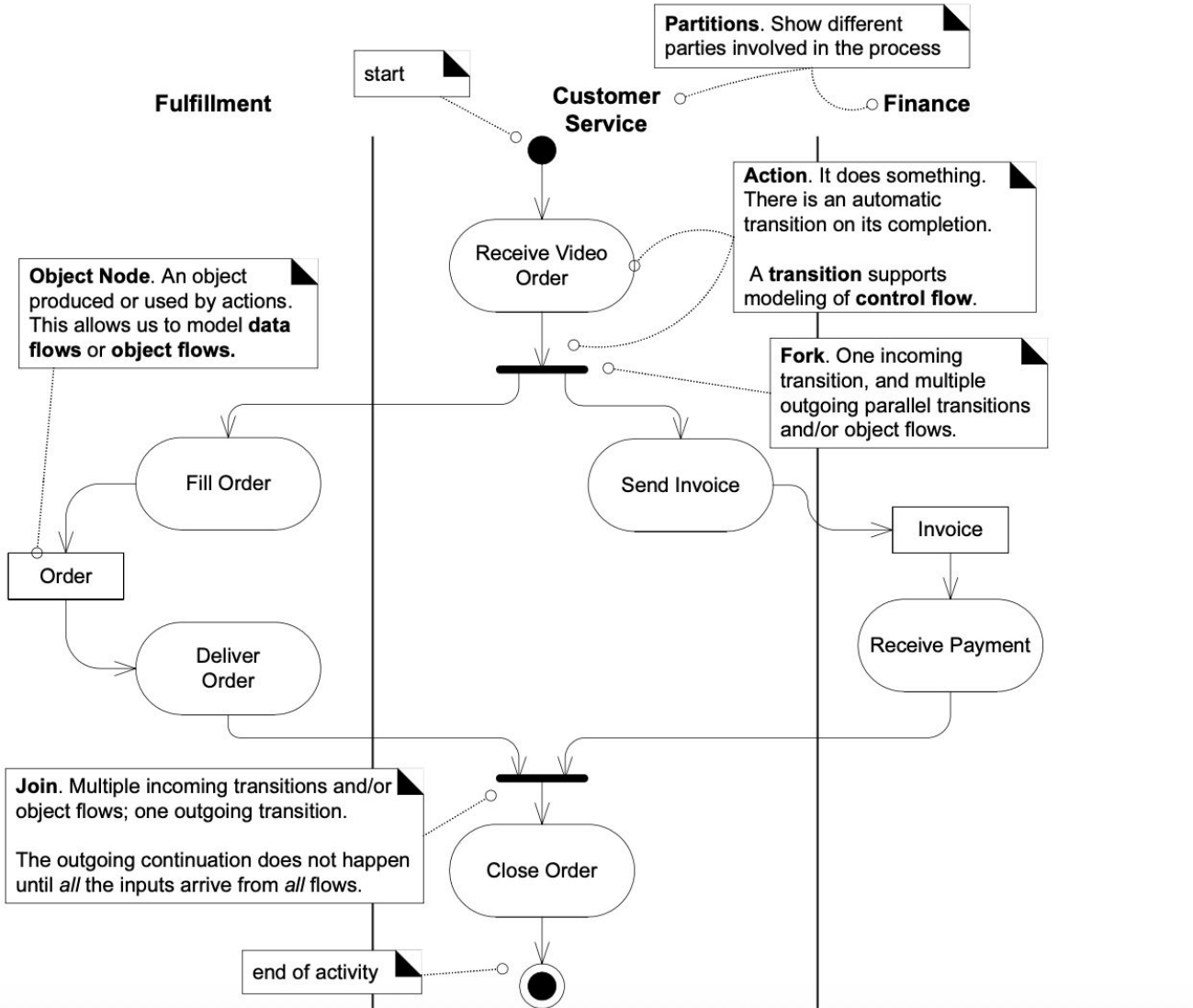
PES UNIVERSITY
CELEBRATING 50 YEARS

Key Rules

- Every use case can be represented by an activity diagram, but activity diagrams should only be created for use cases that involve complex workflows, decisions, or parallel activities. Simple use cases are better documented using textual descriptions or other UML diagrams.
- Use Decision box (diamond) when user/system must choose ONE option (Mutual Exclusion)
- Use Fork (Thick Bar) when ALL actions must happen in parallel (Concurrency)
- Use Merge (Diamond) when flows are mutually exclusive (OR) to proceed
- Use Join when ALL parallel activities must finish (AND) to proceed


Online Shopping
© uml-diagrams.org
Search Items
[found]
[search]
[not found]
View Item
[made decision]
[browse]
Browse Items
B
Add to Shopping Cart
[proceed]
Shopping cart can be checked at any time
[view cart]
Check Shopping Cart
A
Update Shopping Cart
A
[update needed]
Proceed to Checkout
Checkout
B
View Shopping Cart
[done with shopping]
[more shopping]
C
C

Object Oriented Analysis and Design



THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

Object Oriented Analysis and Design

Behaviour Modelling: Sequence Diagram

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgement:
Dr. Kamatchi Priya L
Department of Computer Science and Engineering

PES UNIVERSITY
CELEBRATING 50 YEARS

Behaviour Modelling

- **UML Behavioral Diagrams Overview:**
  Illustrate elements in a system dependent on time, conveying dynamic concepts and their interrelationships.

- **Nature of Elements:**
  Elements in these diagrams resemble verbs in natural language, representing dynamic actions.

- **Temporal Relationships:**
  Relationships among elements typically signify the passage of time.

- **Example Scenario:**
  A behavioral diagram of a vehicle reservation system might contain elements such as Make a Reservation, Rent a Car, and Provide Credit Card Details

PES UNIVERSITY
CELEBRATING 50 YEARS

Sequence Diagram

A Sequence diagram is a structured representation of behavior as a series of sequential steps over time. You can use it to:

- Depict workflow, Message passing and how elements in general cooperate over time to achieve a result
- Capture the flow of information and responsibility throughout the system, early in analysis; Messages between elements eventually become method calls in the Class model
- Make explanatory models for Use Case scenarios; by creating a Sequence diagram with an Actor and elements involved in the Use Case, you can model the sequence of steps the user and the system undertake to complete the required tasks

PES UNIVERSITY
CELEBRATING 50 YEARS

Sequence Diagram Construction

- Horizontal Axis: Objects are arranged horizontally.
- Time Flow: The vertical axis represents time, flowing downwards.
- Sequence: Messages are drawn from sender to receiver, showing the precise order they are sent, from top to bottom.
- Each element has a dashed stem called a Lifeline, where that element exists and potentially takes part in the interactions

Sequence Diagram: Notations and Symbols

Actor:

- An Actor is a user of the system; user can mean a human user, a machine, or even another system or subsystem in the model. Anything that interacts with the system from the outside or system boundary is termed an Actor. Actors are typically associated with Use Cases.

- Actors can use the system through a graphical user interface, through a batch interface or through some other media. An Actor's interaction with a Use Case is documented in a Use Case scenario, which details the functions a system must provide to satisfy the user requirements.

- Actors also represent the role of a user in Sequence diagrams, where you can display them using rectangle notation. Enterprise Architect supports a stereotyped Actor element for business modeling. The business modeling elements also represent Actors as stereotyped Objects.

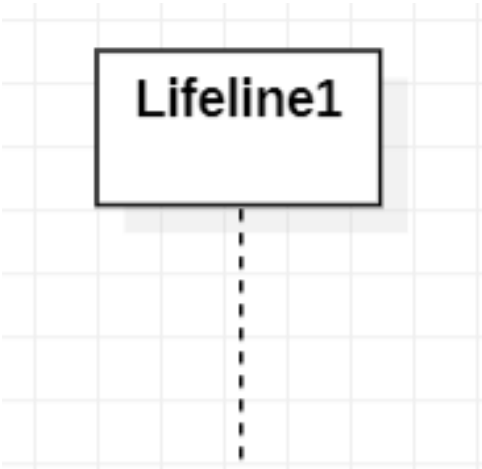
Sequence Diagram: Notations and Symbols

Lifeline:

A Lifeline is an individual participant in an interaction (that is, Lifelines cannot have multiplicity).

A Lifeline represents a distinct connectable element.

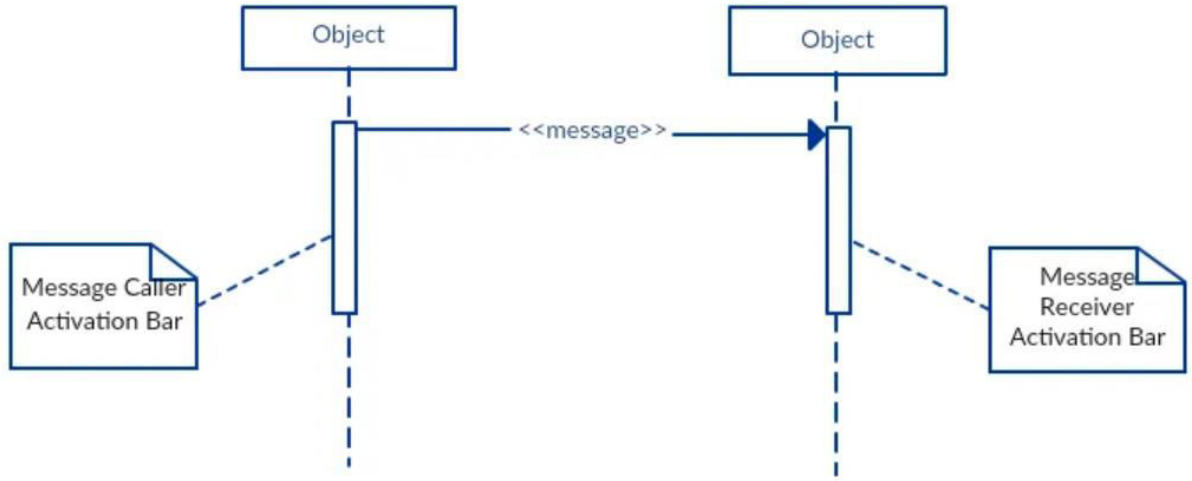
Vertical dashed lines representing individual objects or participants (actors, classes, components) in the interaction.



Sequence Diagram: Notations and Symbols

Activation Bar (Execution Occurrence):

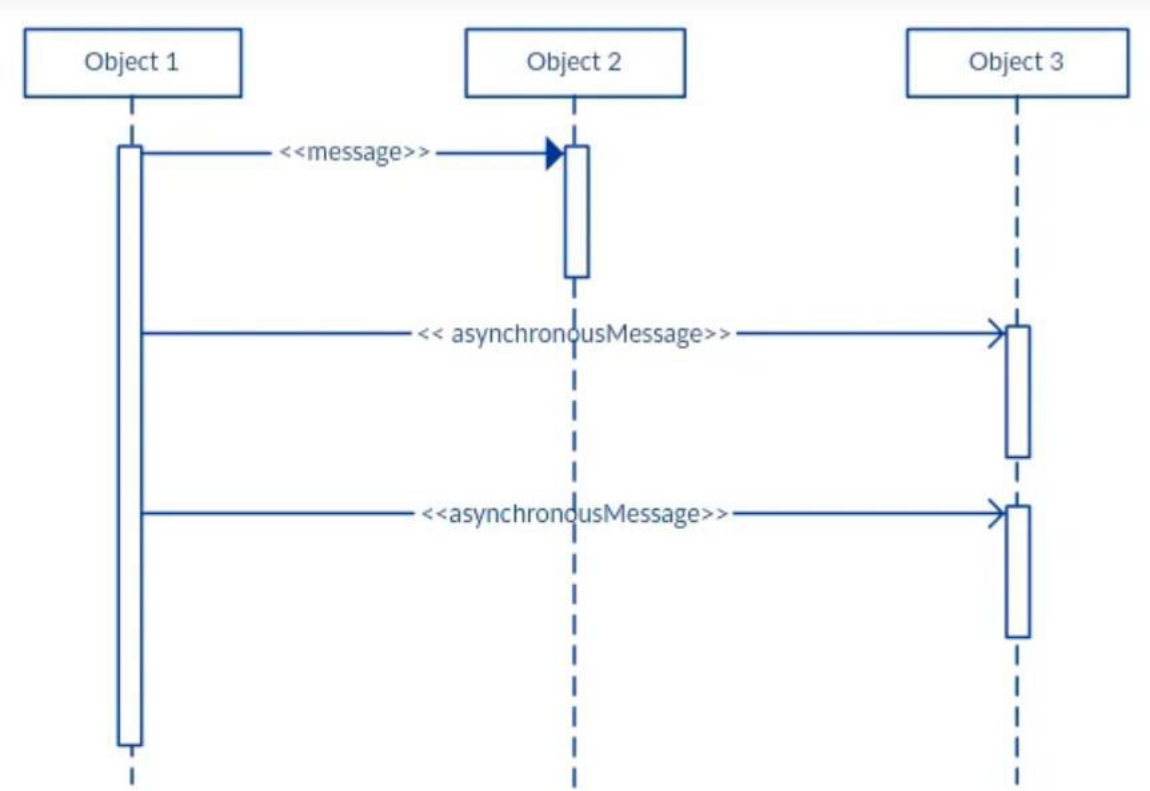
- Indicates the period during which an object is active or performing some action.
- It shows when messages are being sent and received.



Sequence Diagram: Notations and Symbols

Message:

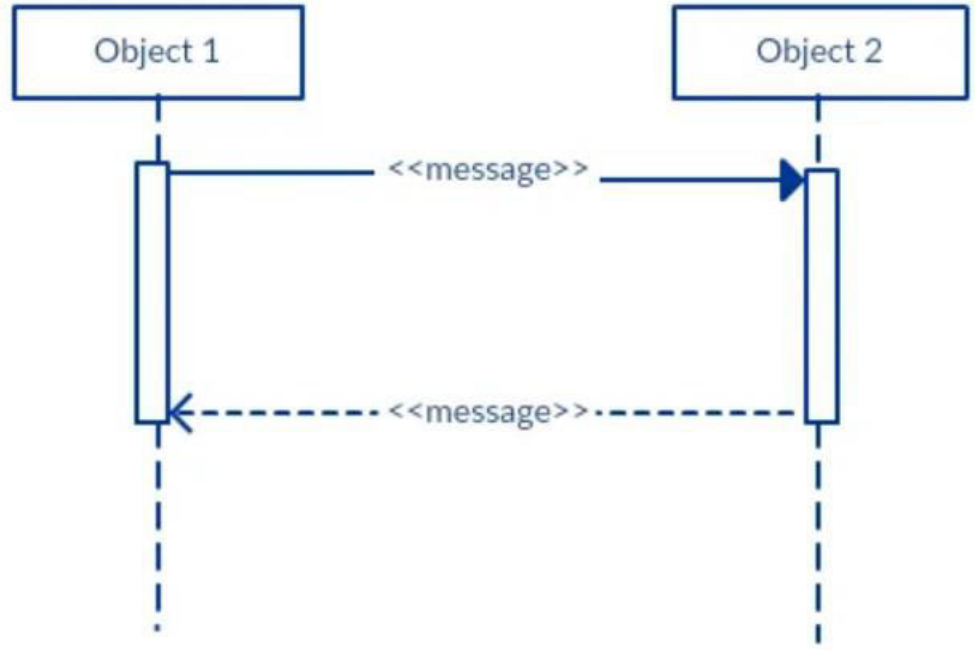
- Represents communication or interaction between objects.
- Messages can be synchronous (solid line with filled arrowhead), asynchronous (solid line with open arrowhead), or a return message (dotted line with open arrowhead).



Sequence Diagram: Notations and Symbols

Message:

- Represents communication or interaction between objects.
- Messages can be synchronous (solid line with filled arrowhead), asynchronous (solid line with open arrowhead), or a return message (dotted line with open arrowhead).



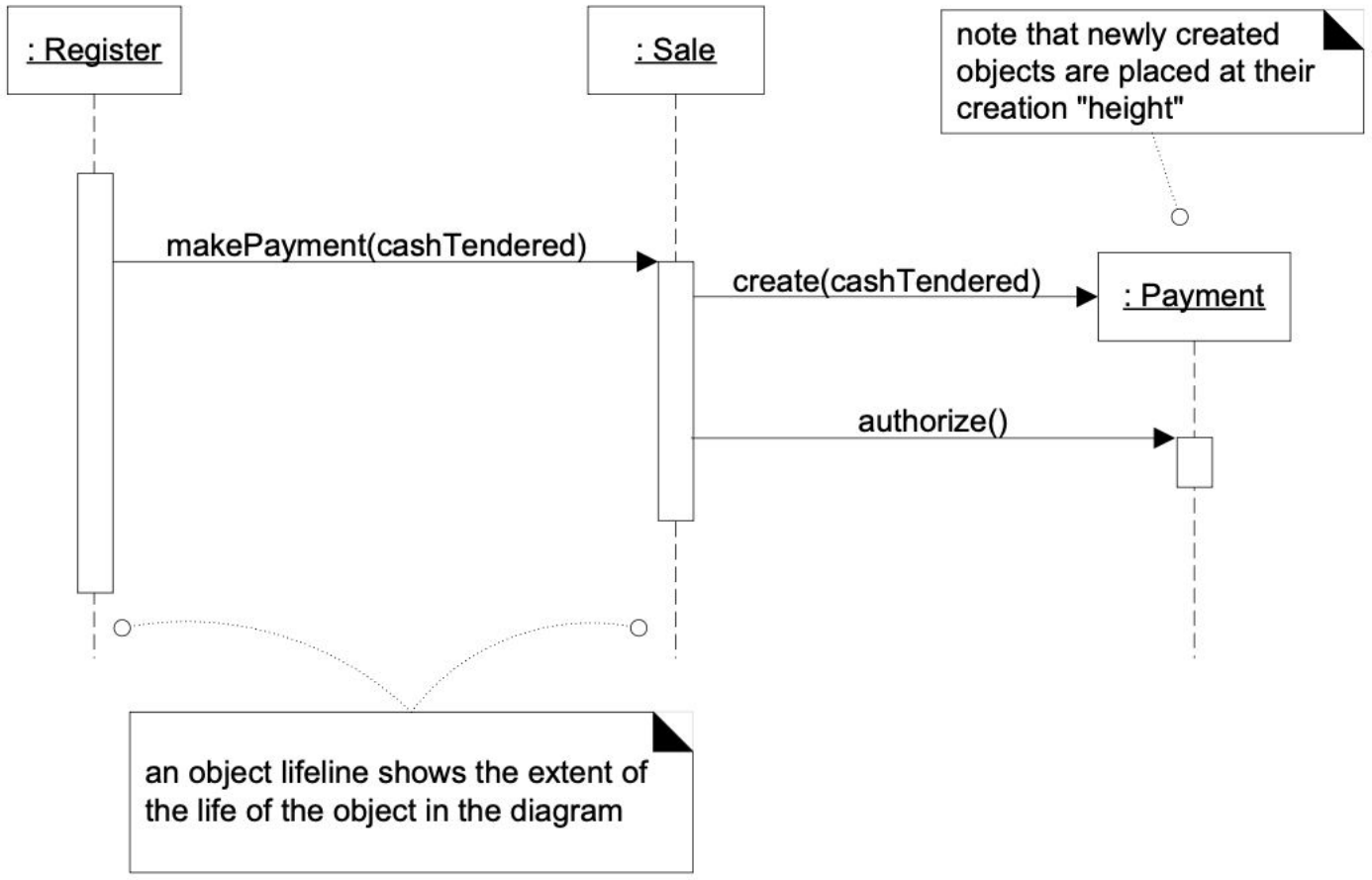
Sequence Diagram: Notations and Symbols

PES UNIVERSITY
CELEBRATING 50 YEARS

Create Message:

- **Symbol:** Solid line with a filled arrow, with optional stereotype «create»
- **Description:** Denotes the creation of a new object. It indicates that an object is being instantiated.

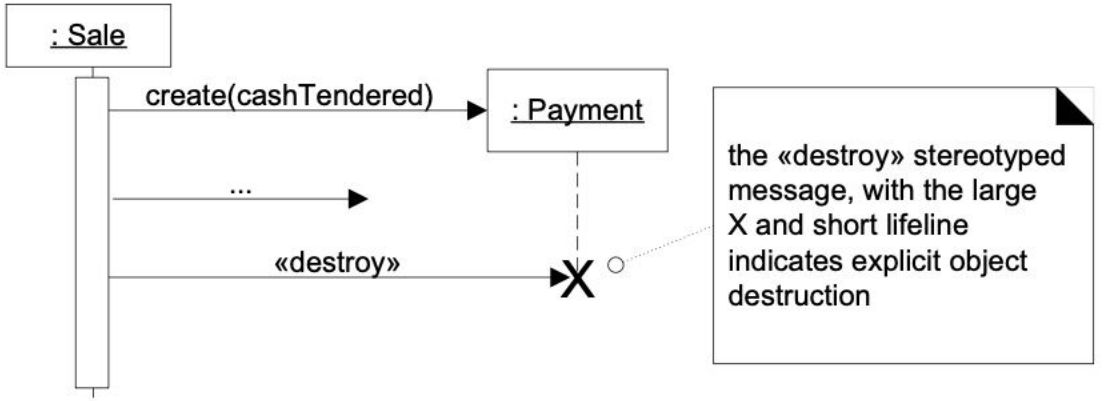
Creation of Instances



Sequence Diagram: Notations and Symbols

Destroy Message:

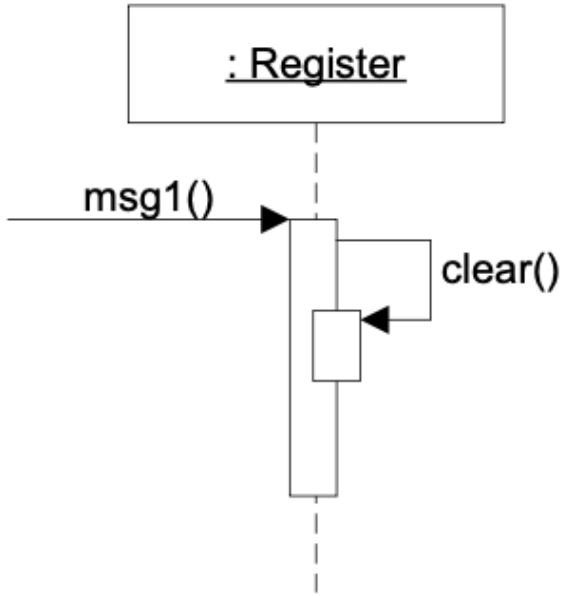
- **Symbol:** X-shaped arrowed line
- **Description:** Represents the termination or destruction of an object. It indicates the end of the object's existence.



Sequence Diagram: Notations and Symbols

Self Message:

- **Symbol:** Loop arrowed line
- **Description:** Denotes a message sent by an object to itself. It is used to represent actions or method calls within the same object.



Sequence Diagram: Notations and Symbols

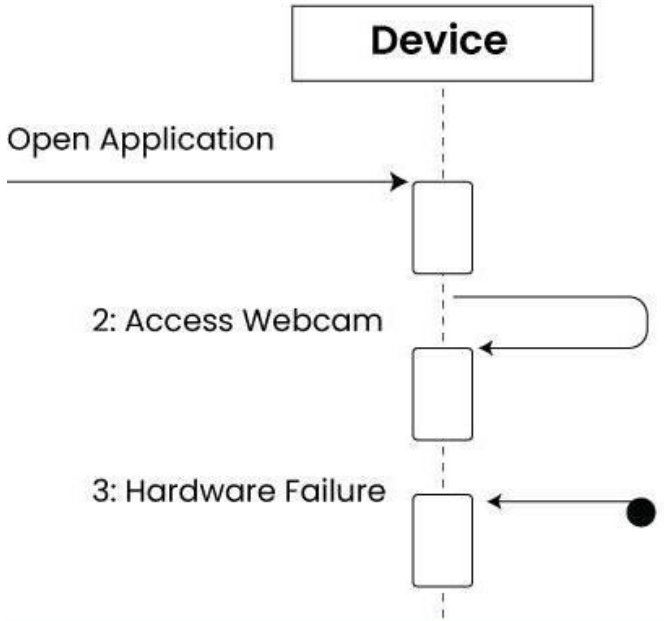
Found Message:

- **Symbol:** Arrowed line with an open circle
- **Description:** to represent a scenario where an unknown source sends the message

Examples - External system trigger

Hardware / timer event

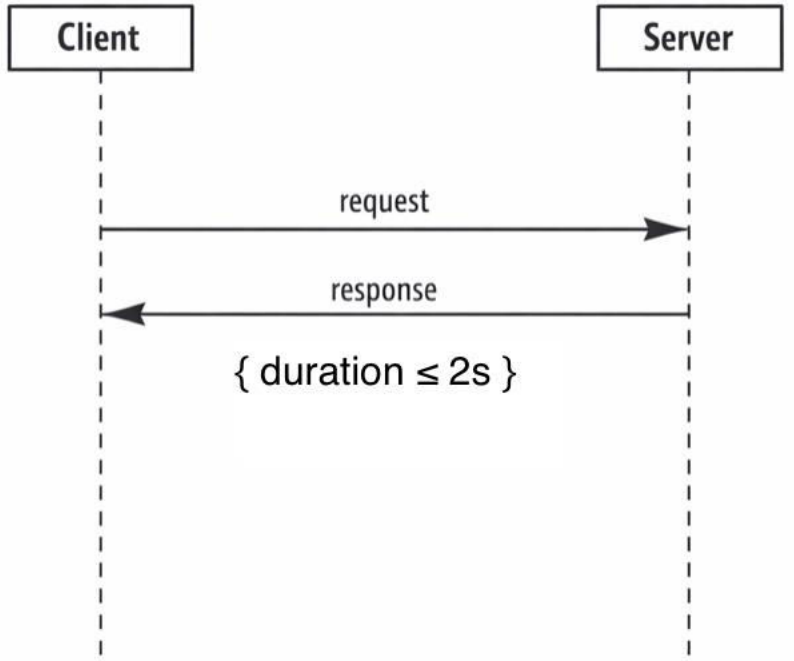
Incoming HTTP request



Sequence Diagram: Notations and Symbols

Duration Constraint:

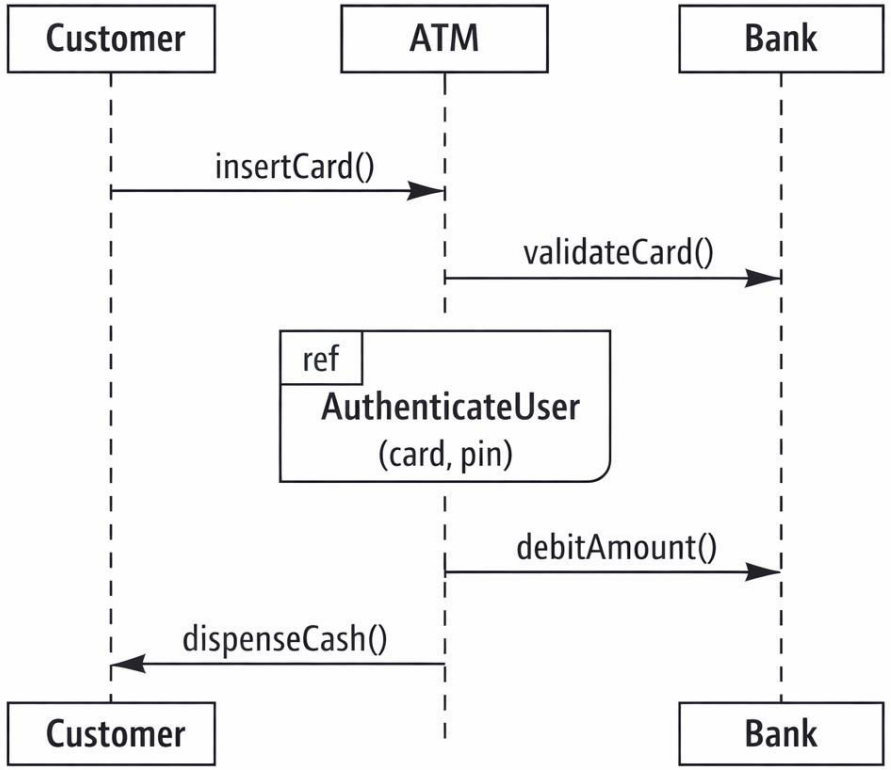
- **Symbol:** Rectangular box with a label
- **Description:** Specifies the minimum and maximum time duration of a message or an entire interaction. It provides additional information about the timing aspects of the system.



Sequence Diagram: Notations and Symbols

Interaction Use (formerly "Ref fragment"):

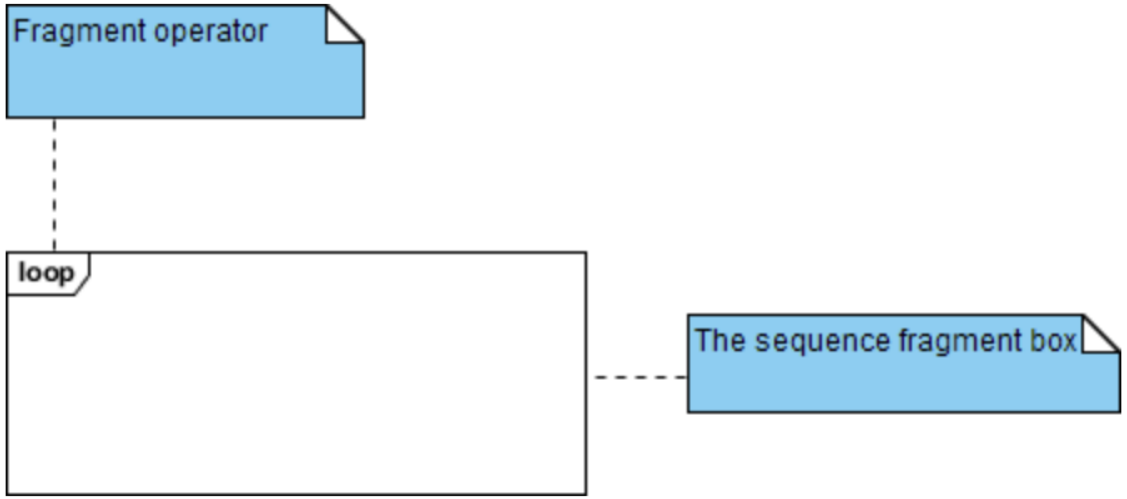
- **Symbol:** Rectangle with a name
- **Description:** Represents a reusable sequence of messages defined in another diagram. It helps in modularizing and simplifying complex sequence diagrams.



Sequence Diagram: Notations and Symbols

Sequence Fragments

- **Symbol:** Rectangle with a name
- **Description:** called a combined fragment which encloses a portion of the interaction within a sequence diagram



Sequence Fragments

Operator	Fragment Type
alt	Alternative multiple fragments: only the one whose condition is true will execute.
opt	Optional: the fragment executes only if the supplied condition is true. Equivalent to an alt only with one trace.
par	Parallel: each fragment is run in parallel.
loop	Loop: the fragment may execute multiple times, and the guard indicates the basis of iteration.
region	Critical region: the fragment can have only one thread executing it at once.
neg	Negative: the fragment shows an invalid interaction.
ref	Reference: refers to an interaction defined on another diagram. The frame is drawn to cover the lifelines involved in the interaction. You can define parameters and a return value.
sd	Sequence diagram: used to surround an entire sequence diagram.

Steps to Construct an Sequence Diagram

Identify the Use Case:

Clearly define the use case or scenario that you want to represent with the sequence diagram. Understand the purpose of the interaction and the involved actors.

Identify Objects/Participants:

Identify the main objects or participants (actors, classes, components) that are part of the interaction. These will be represented by lifelines in the sequence diagram.

Draw Lifelines:

Draw vertical dashed lines to represent the lifelines of each identified object or participant. Lifelines indicate the existence of an object over time.

Arrange Lifelines:

Steps to Construct an Sequence Diagram

PES UNIVERSITY
CELEBRATING 50 YEARS

Arrange Lifelines:

Position the lifelines vertically, ensuring they are spaced appropriately. This layout represents the temporal order of the participating objects.

Determine the Order of Execution:

Identify the order in which the objects will execute their actions. This helps in organizing the sequence of messages in the diagram.

Draw Messages:

Use arrows to draw messages between the lifelines to represent interactions or communications between objects. Different types of messages (synchronous, asynchronous, etc.) can be indicated by varying line styles and arrowheads.

Steps to Construct an Sequence Diagram

Label Messages:

- Label each message with the method or operation being called, along with any parameters or return values. This adds clarity to the diagram and helps in understanding the purpose of each message.

Include Create and Destroy Messages:

- If necessary, include create and destroy messages to represent the instantiation and termination of objects. These are particularly relevant for dynamic interactions.

Add Conditions and Loops (Optional):

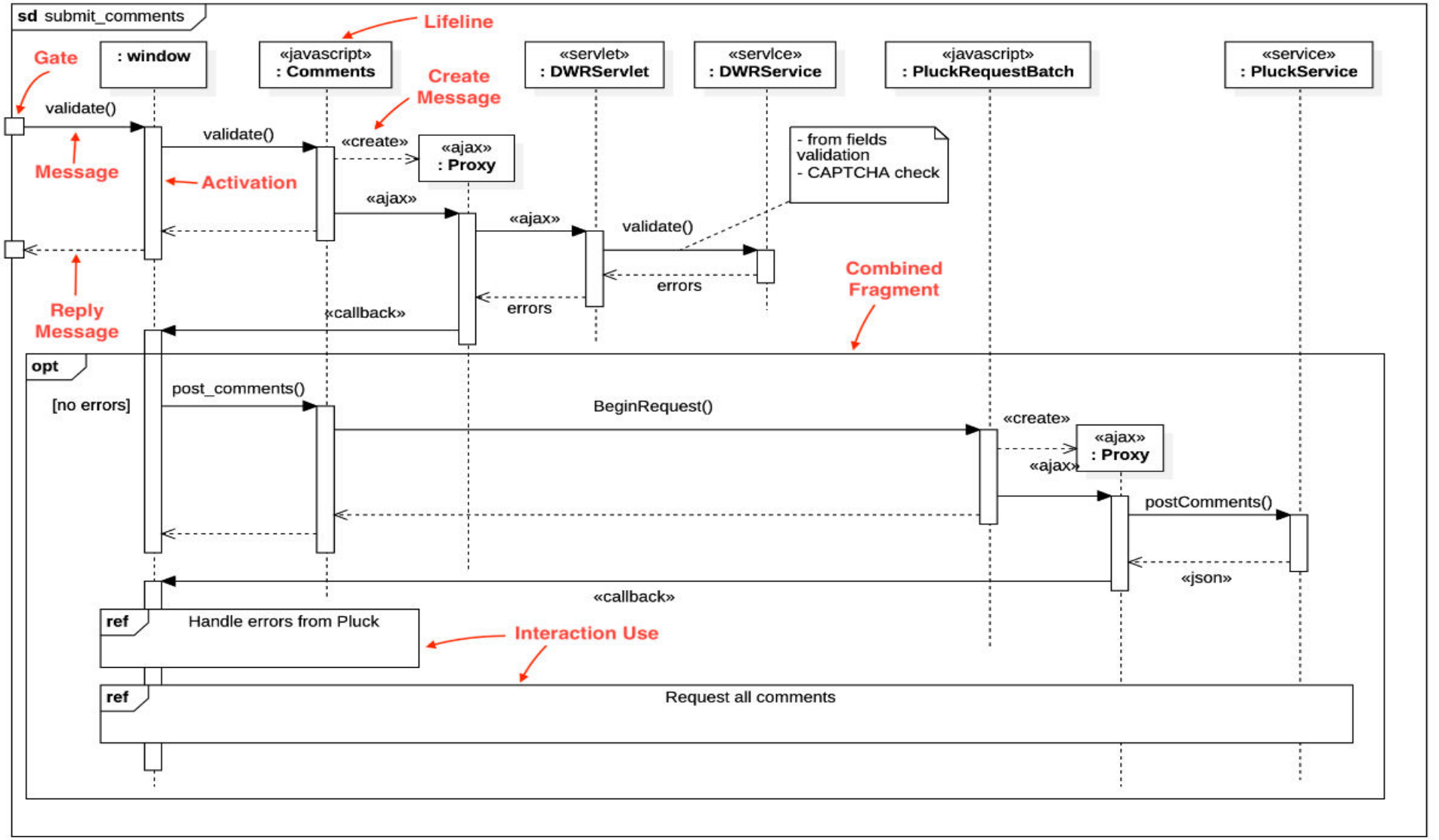
- If the sequence involves conditions or loops, you can use combined fragments or loop/alt/option boxes to indicate these structures within the sequence diagram.
- Include Additional Details:
- Add annotations, notes, or comments to provide additional information or context that might be important for understanding the sequence of events.

Review and Refine:

- Review the sequence diagram to ensure it accurately represents the desired interaction. Refine the diagram as needed to improve clarity and readability.
- Consider Time Constraints (Optional):
- If necessary, include duration constraints to specify the minimum and maximum time duration of certain interactions.
- Validate and Iterate:
- Validate the sequence diagram with stakeholders or team members to ensure it accurately captures the intended behavior. Iterate on the diagram based on feedback received.
- Create Additional Diagrams (Optional):
- If the interaction is complex, consider creating separate sequence diagrams for different aspects of the interaction or breaking it down into smaller, more manageable parts.

Object Oriented Analysis and Design

Messaging System



THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

Object Oriented Analysis and Design

Behavior Modeling:
UML State Diagrams and Models

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgement:
Dr. Kamatchi Priya L
Department of Computer Science and Engineering

Behavior Modeling

- A behavior Modeling is intended to provide clarity, for example, about internal processes, business processes or the interaction of different systems.
- UML Behavioral Diagrams depict the elements of a system that are dependent on time and that convey the dynamic concepts of the system and how they relate to each other.
- The elements in these diagrams resemble the verbs in a natural language and the relationships that connect them typically convey the passage of time.

State Diagram

- State machine diagram typically are used to describe state-dependent behavior for an object.
- An object responds differently to the same event depending on what state it is in.
- The state model is a reductionist view of behaviour that examines each object individually.

State Diagram Vs Flowchart

- **A flowchart illustrates processes that are executed in the system that change the state of objects.**

- A state diagram shows the actual changes in state, not the processes or commands that created those changes.

How to Draw State Diagram

- Before you begin your drawing find the initial and final state of the object in question.
- Next, think of the states the object might undergo. For example, in e-commerce a product will have a release or available state, a sold out state, a restocked state, placed in cart state, a saved on wish list state, a purchased state, and so on.
- Certain transitions will not be applicable when an object is in a particular state, for example a product can not be in a purchased state or a saved in cart state if its previous state is sold out.

State Diagram: Notations and Symbols

States

States represent situations during the life of an object. A state is represented by using a rectangle with rounded corners.

A simple state

A state with internal activities

State Diagram: Notations and Symbols

Transition

A solid arrow represents the path between different states of an object. Label the transition with the event that triggered it and the action that results from it. A state can have a transition that points back to itself.

State Diagram: Notations and Symbols

Initial State

A filled circle followed by an arrow represents the object's initial state.

State Diagram: Notations and Symbols

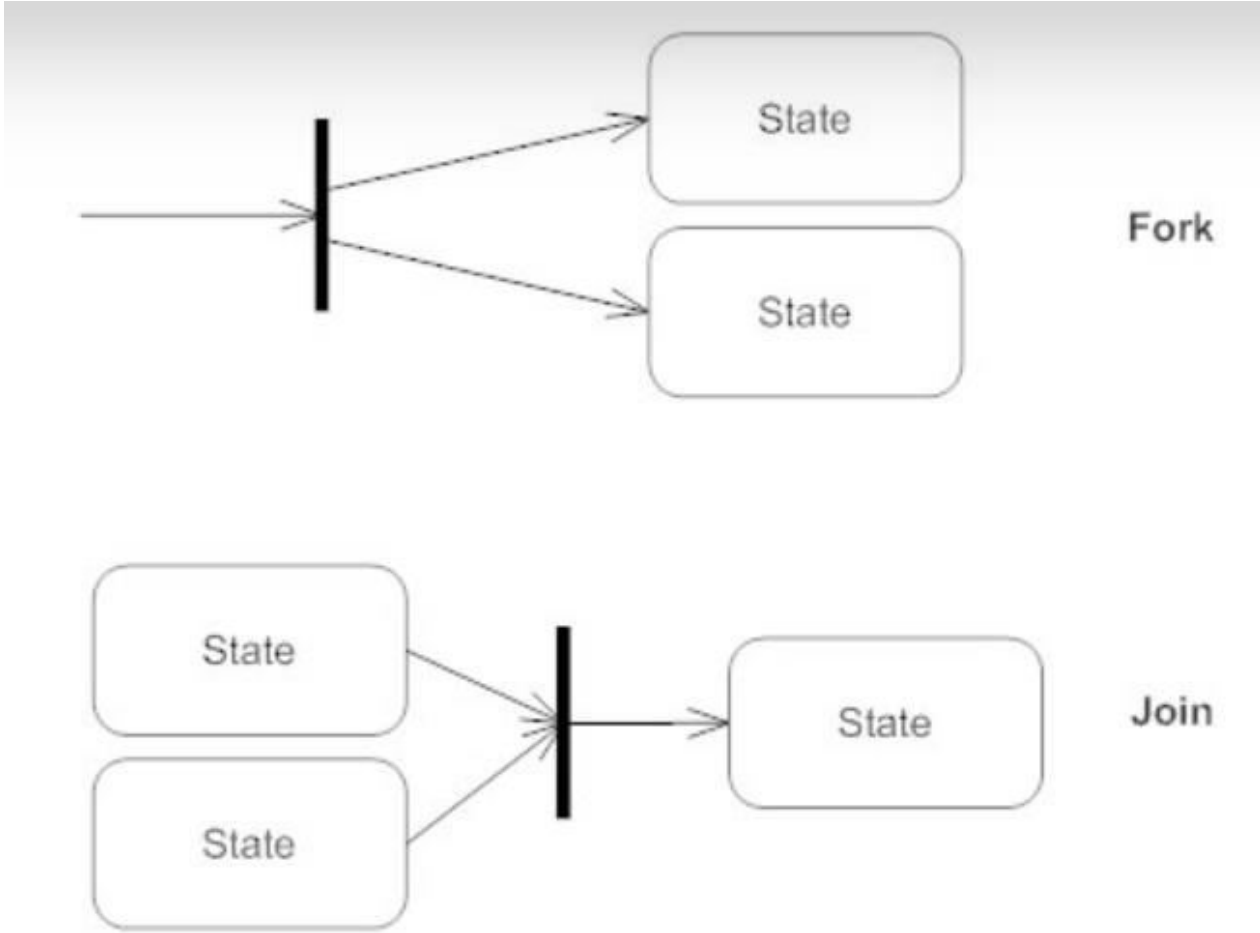
Final State

An arrow pointing to a filled circle nested inside another circle represents the object's final state

State Diagram: Notations and Symbols

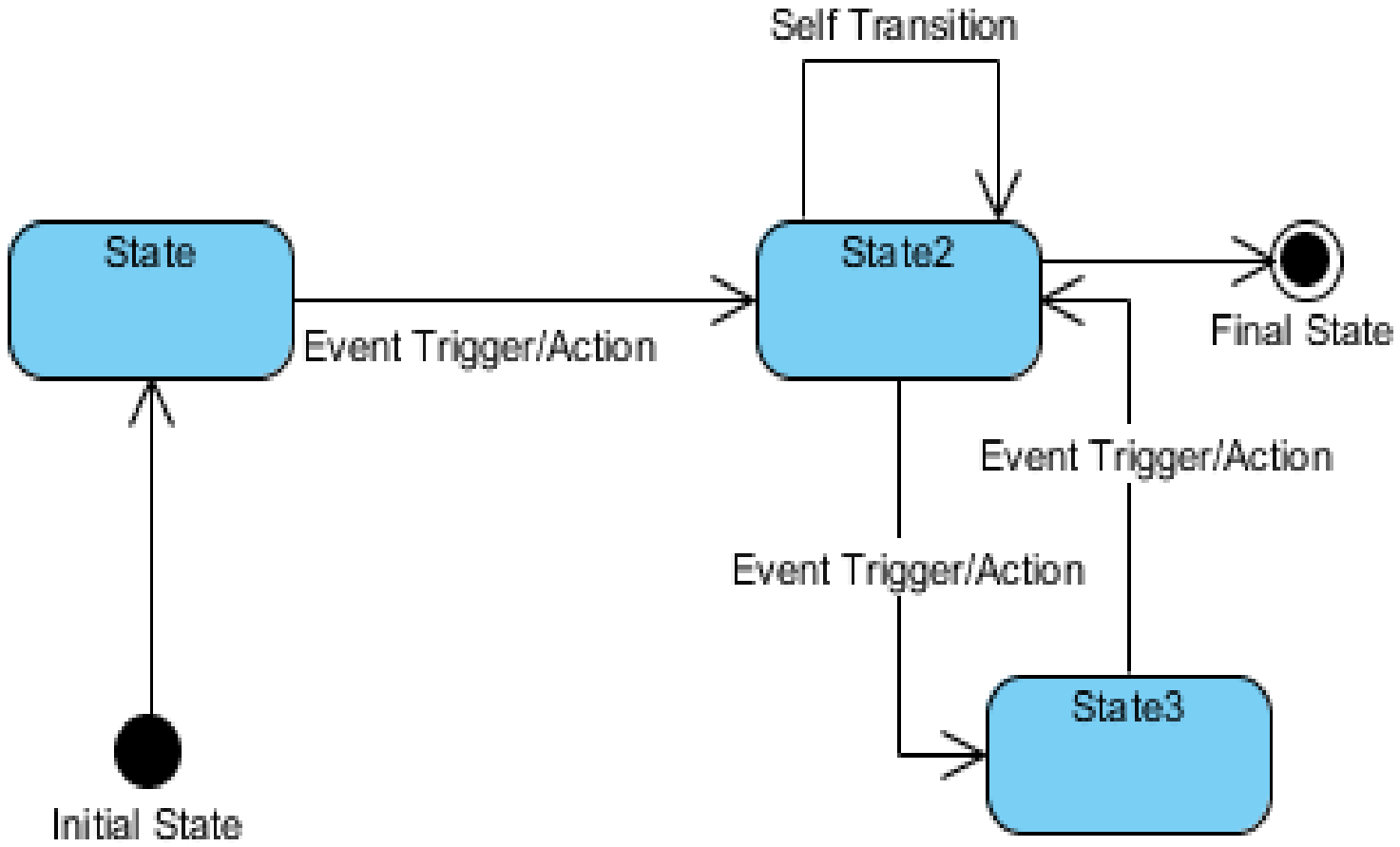
Synchronization and Splitting of Control

- A short heavy bar with two transitions entering it represents a synchronization of control.
- The first bar is often called a fork where a single transition splits into concurrent multiple transitions.
- The second bar is called a join, where the concurrent transitions reduce back to one.



Triggers Event, Action and Guard Condition

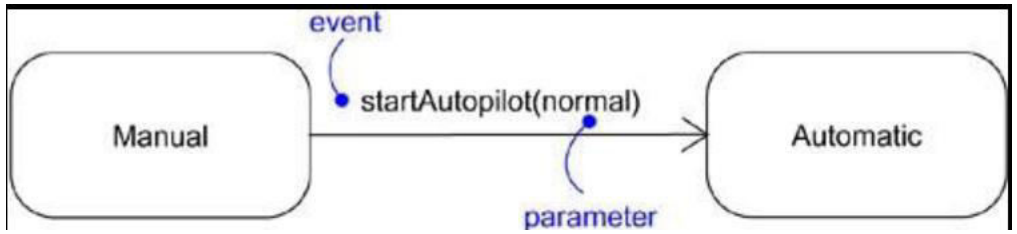
- A trigger is an **event** that initiates a transition from one state to another.
- A **guard condition** is a Boolean condition that must be satisfied for a transition to occur.
- An effect is the **action** or activity that happens when a transition occurs.



Adding triggers to state machine diagrams

Adding call events to state machine diagrams

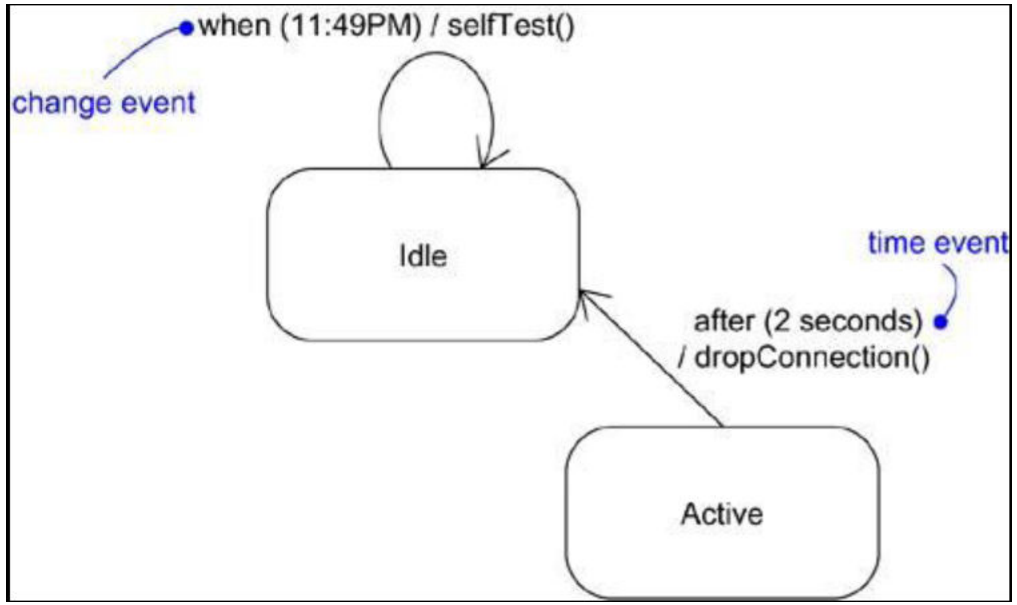
A call event is an event that represents the receipt of a request to invoke an operation. A transition with a call event initiates when the called operation is invoked.



Adding triggers to state machine diagrams

Adding change events to state machine diagrams

A change event is an event that represents a condition. The condition is defined by a Boolean-valued expression that triggers a transition when its value changes from false to true.



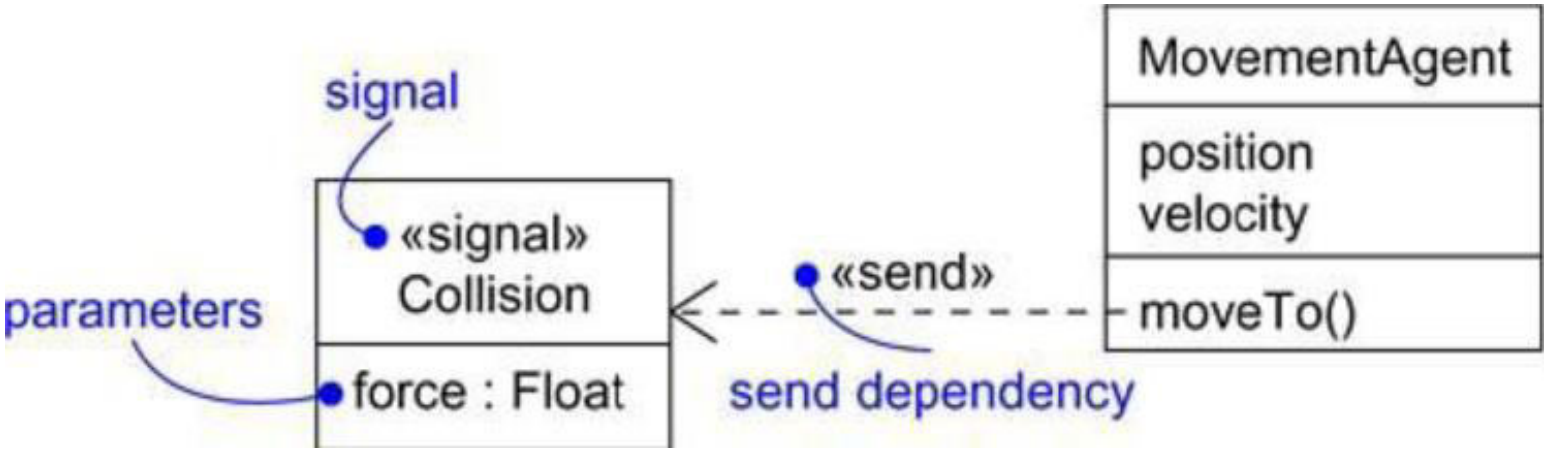
Adding time events to state machine diagrams

A time event is an event that represents the passage of a defined period of time or an absolute time. A transition with a time event trigger initiates when the time value is satisfied.

Adding triggers to state machine diagrams

Adding signal events to state machine diagrams

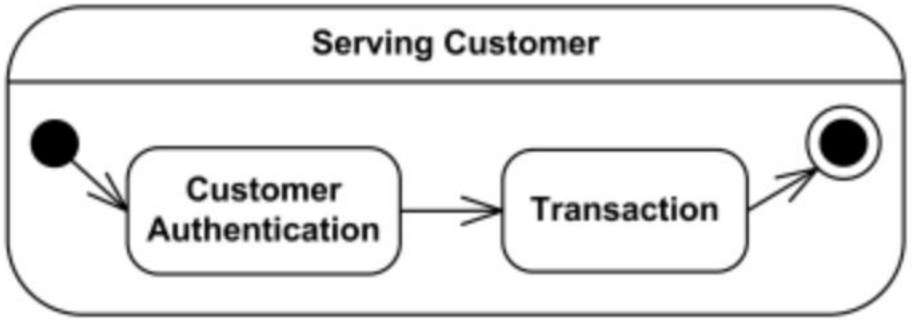
A signal event represents a specific message that initiates a transition when an object receives it.



Composite States

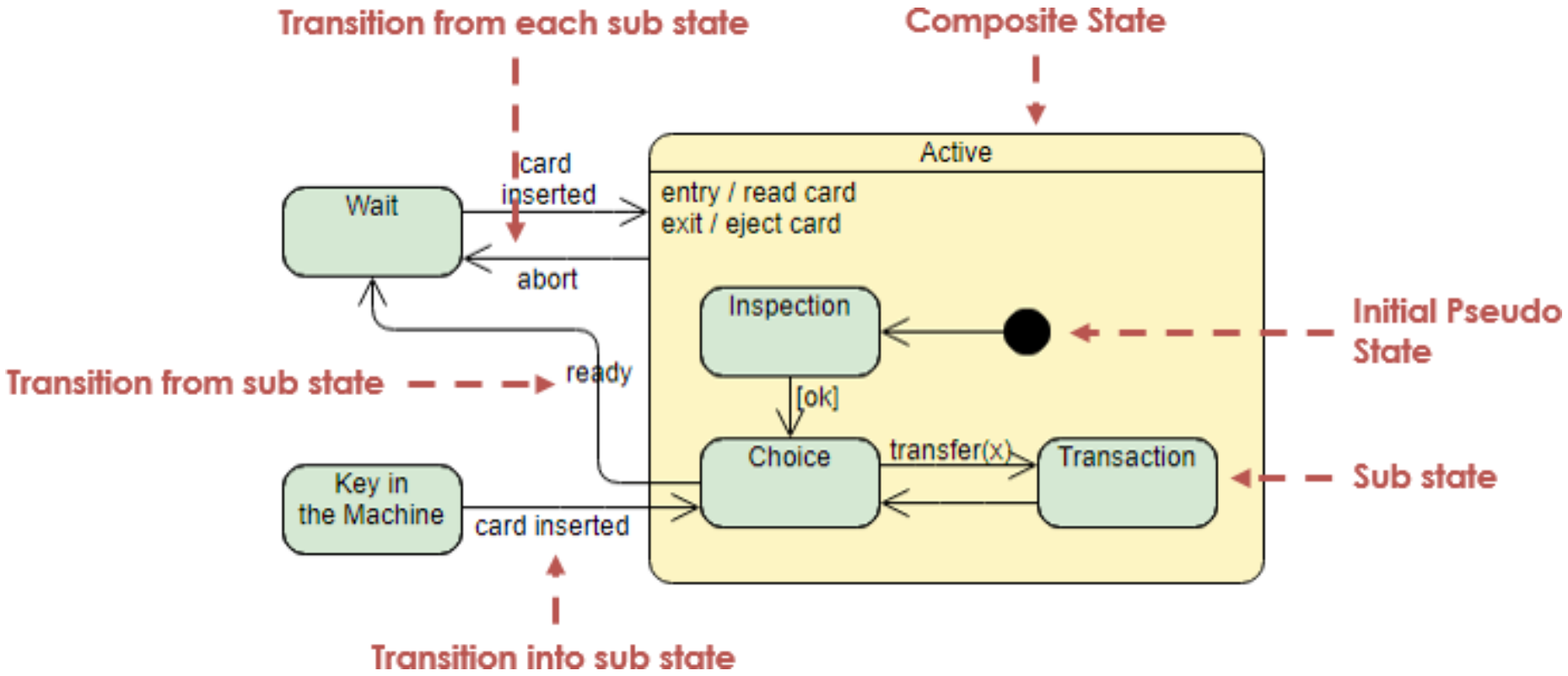
- **Definition**: A composite state in a state diagram is a state that encapsulates and represents a more complex internal structure (state that has substates - nested states)
- **Components**: It consists of substates and transitions, forming a hierarchical structure within the larger state diagram. It contains one or more regions.
- **Representation**: Typically depicted as a box within the state diagram, with the internal structure detailed inside.
- **Entry and Exit Points**: Composite states may have entry, do and exit points, defining specific states to enter or exit when transitioning into or out of the composite state.
- **Transitions:** Transitions within a composite state define how the system moves between its substates or transitions out of the composite state entirely.
- **Clarity and Modularity:** Composite states enhance the clarity and modularity of state diagrams, making it easier to comprehend and manage complex systems.

Composite States



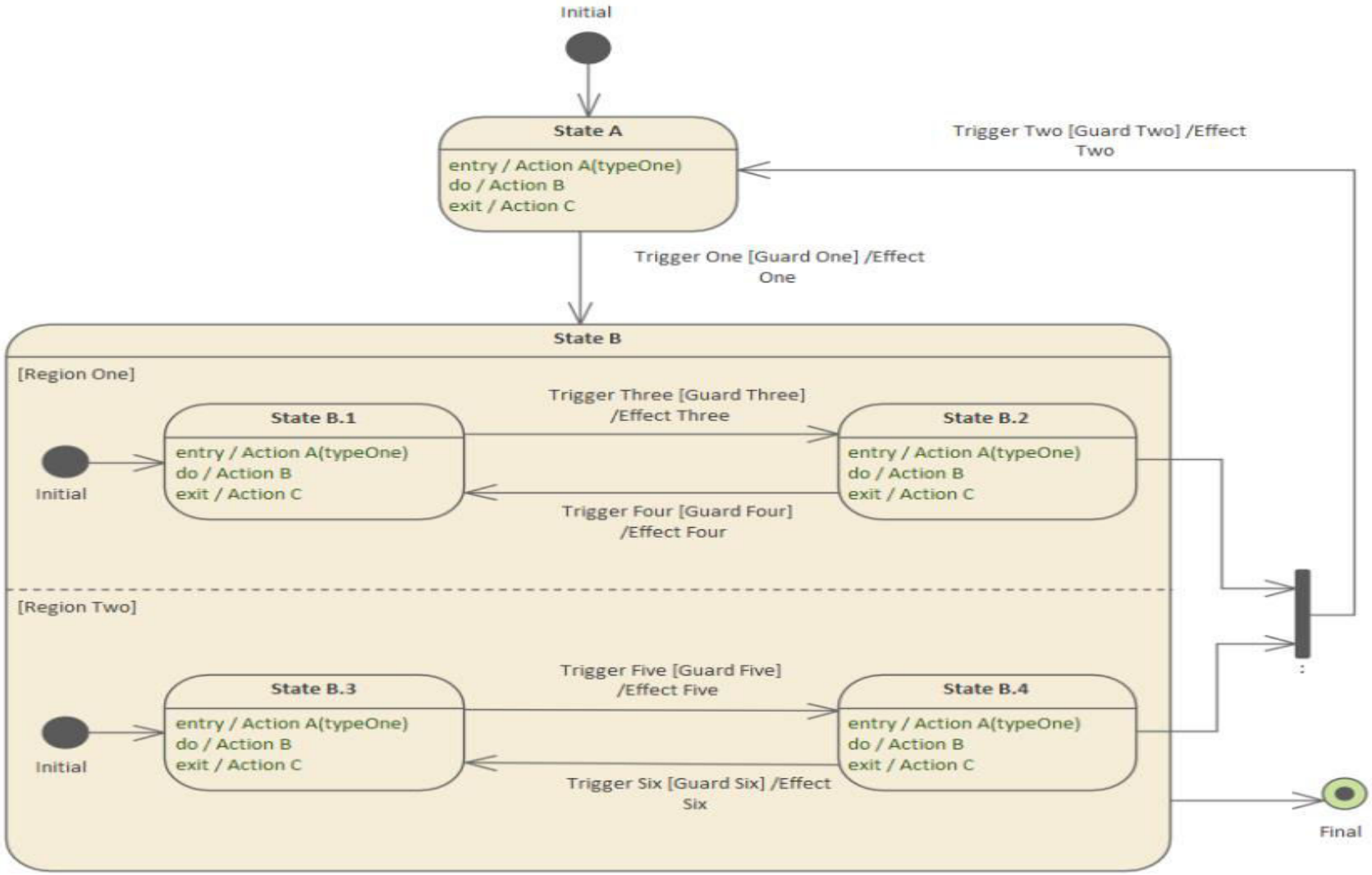
Simple composite state Serving Customer has two substates.

Composite States



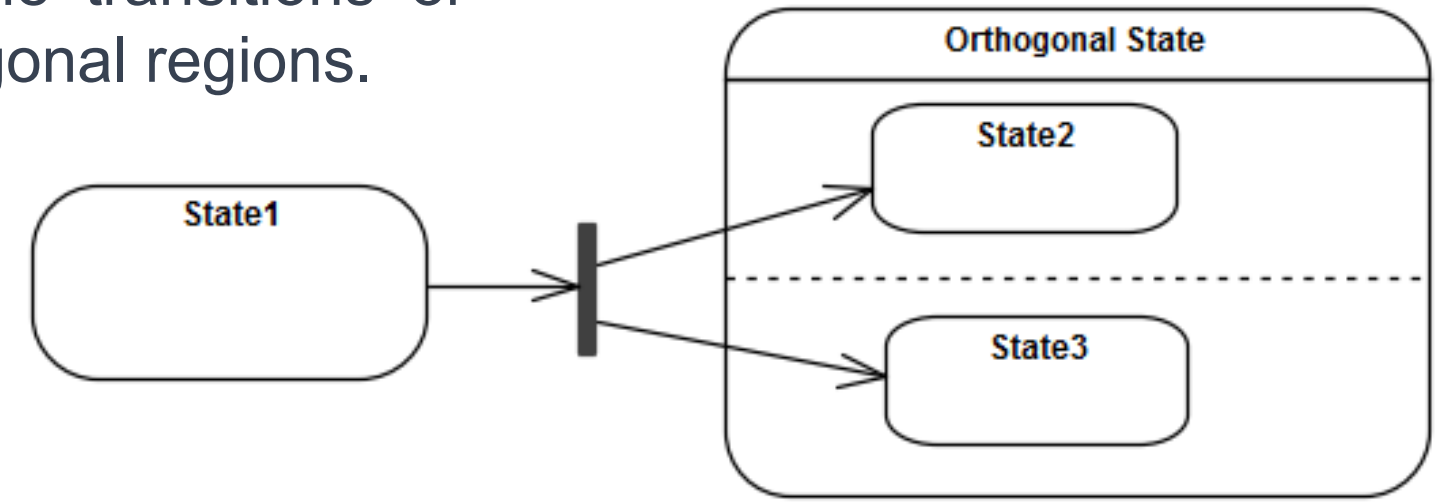
Object Oriented Analysis and Design

Composite States

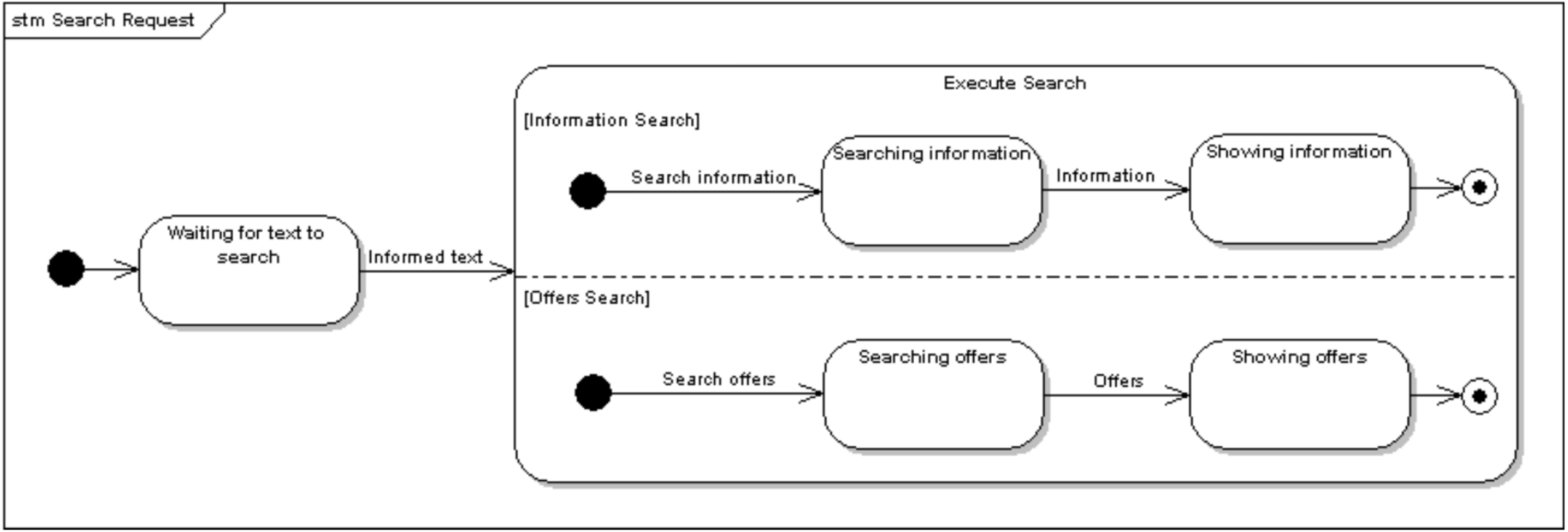


Composite State: Orthogonal States

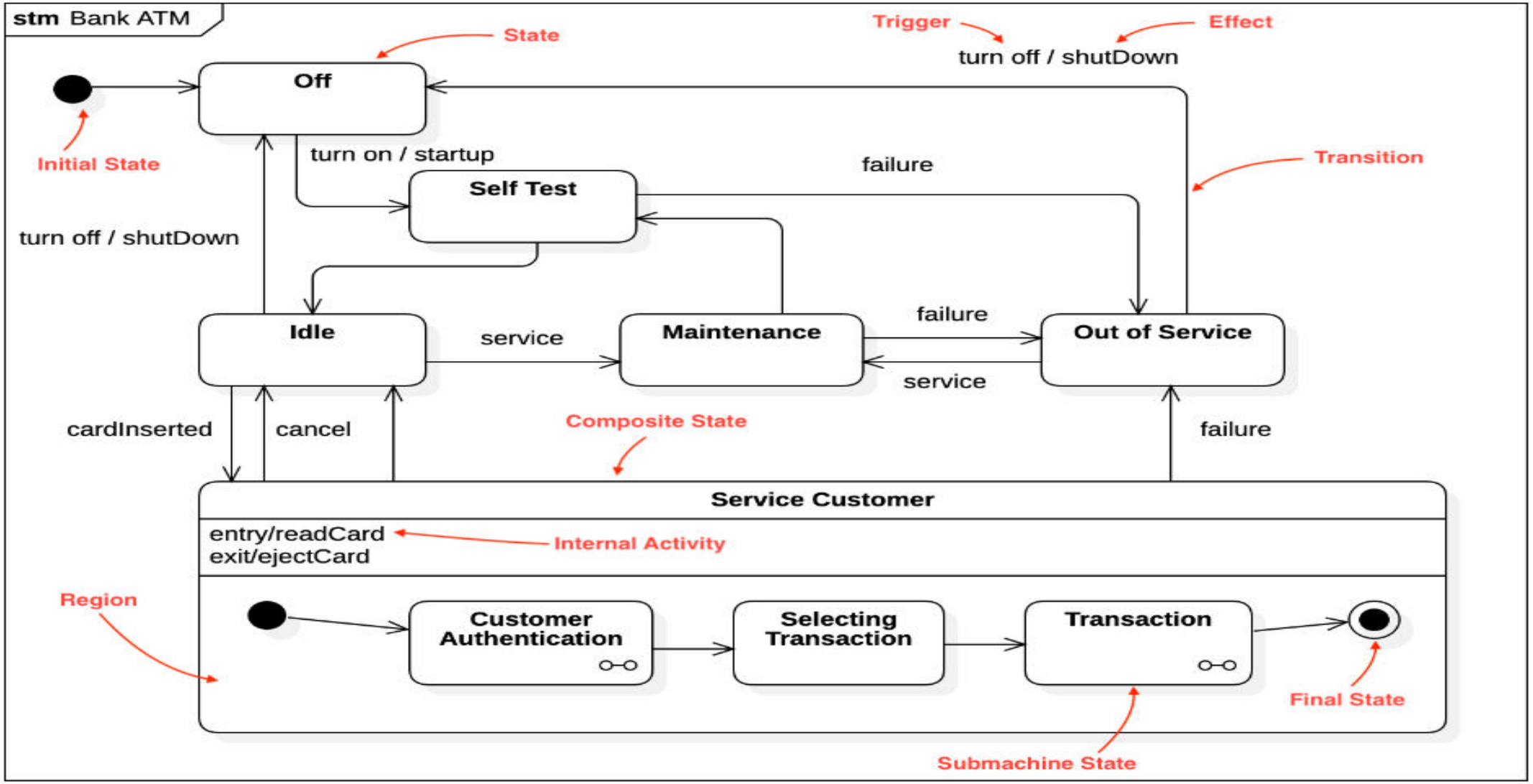
Orthogonal states in a state diagram represent multiple independent sets of substates that can exist concurrently. These substates operate independently of each other and do not affect the transitions or activities in the other orthogonal regions.



Composite State: Orthogonal States



Bank ATM


stm Bank ATM
State
Trigger
Effect
turn off / shutDown
Off
Initial State
turn on / startup
failure
Transition
Self Test
turn off / shutDown
Idle
service
Maintenance
failure
service
Out of Service
cardInserted
cancel
Composite State
failure
Service Customer
entry/readCard
exit/ejectCard
Internal Activity
Region
Customer Authentication
Selecting Transaction
Transaction
Final State
Submachine State

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

Object Oriented Analysis and Design

Advanced State Modeling

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgement:

Prof. Swati Shah

Department of Computer Science and Engineering

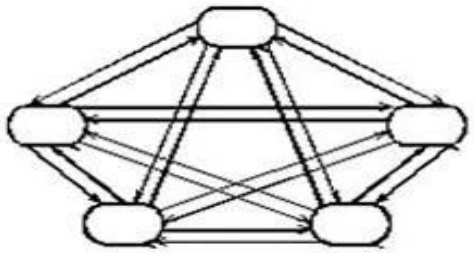
Advanced State Models

- Conventional state diagrams are **sufficient for describing simple systems but need additional power to handle large problems**.

- You can more richly model complex systems by using nest- ed state diagrams, nested states, signal generalization, and concurrency.

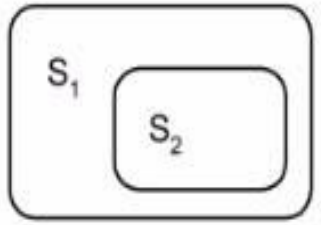
Problems with Flat State Models

- State diagrams have been often criticized because they allegedly are impractical for large problems.
- Unstructured state diagrams
- N independent boolean attributes that affect control. Representing such an object a single flat state diagram would require 2^n States. By partitioning the state into n independent state diagrams, however, only 2n states are required.
- State diagram in Figure in which n^2 transitions are needed to connect every state to every other state. It can be reduced as low as n transitions.

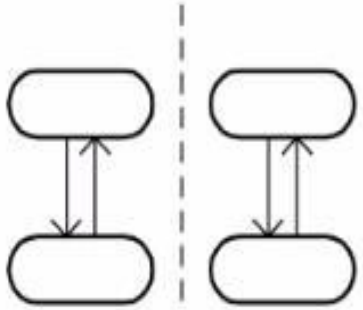


Features of Advanced state Diagram

- Two major features are introduced for controlling complexity and combinatorial explosion in state diagrams
  - **Nested** state diagrams
  - **Concurrent** state diagrams
- Many other features are also added
  - propagated transitions
  - broadcast messages
  - actions on state entry, exit
  - …



Nested State Diagrams



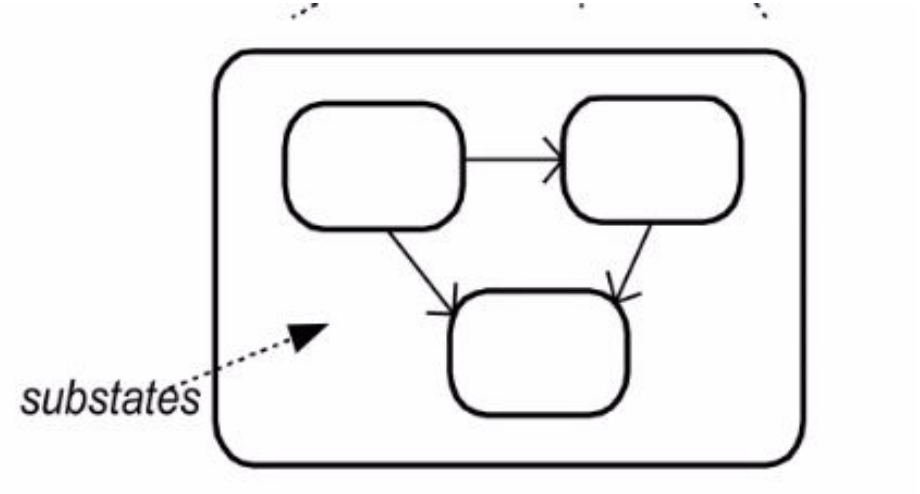
Concurrent State Diagrams

Nested State Diagram

- Activities in states are composite items denoting other lower-level state diagrams
- A lower-level state diagram corresponds to a sequence of lower-level states and events that are invisible in the higher-level diagram.

Super or Substate

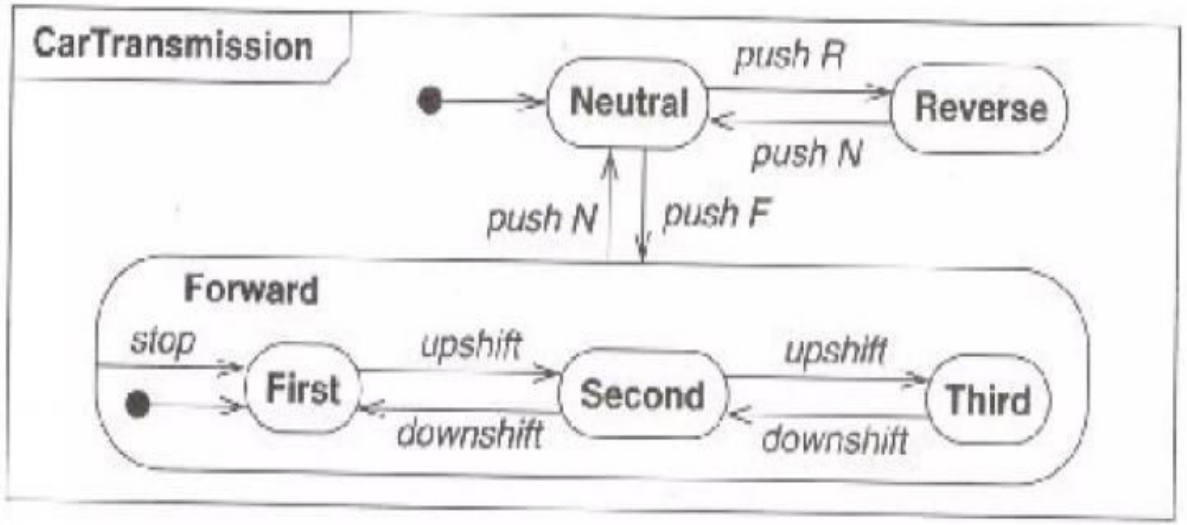
- When one state is complex, you can include substates in it.
  - drawn as nested rounded rectangles within the larger state



Nested state example

A state may be represented as nested substates.

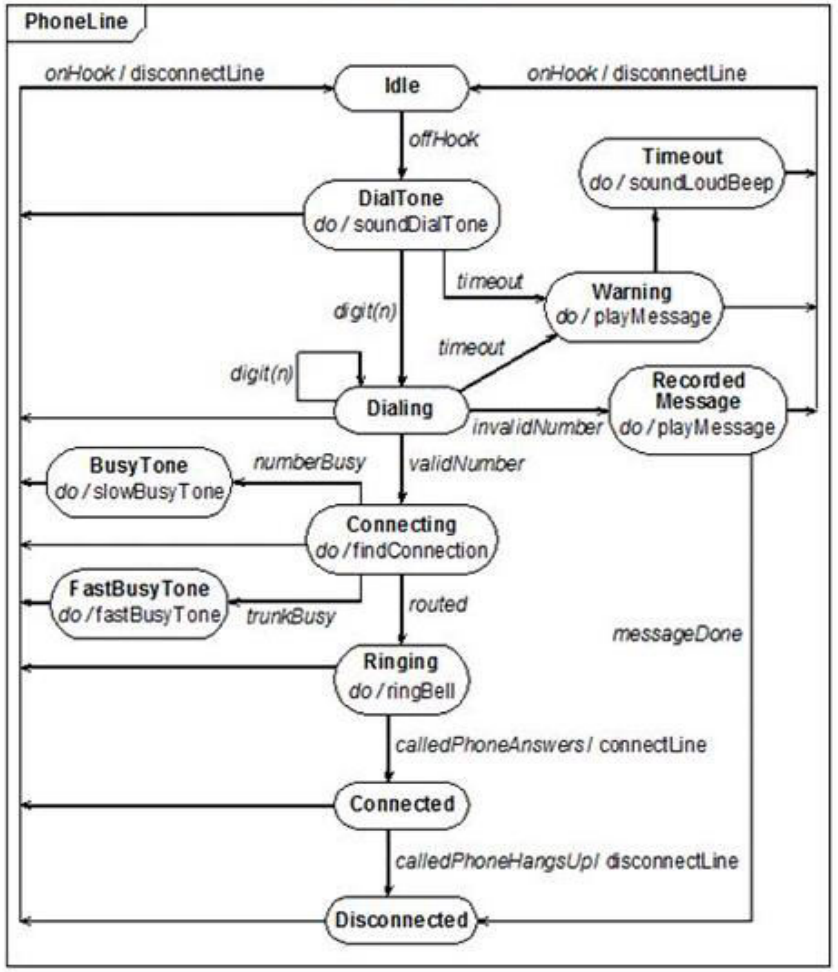
- In UML, substates are shown by nesting them in a superstate box.
- A substate inherits the transitions of its superstate.



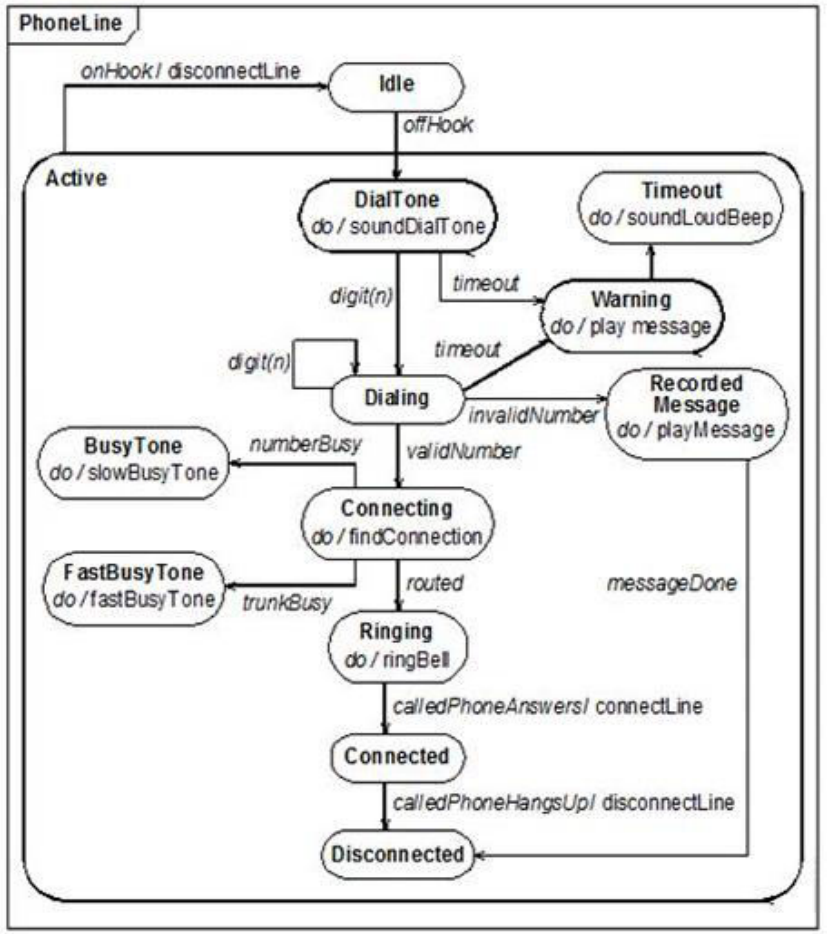
Object Oriented Analysis and Design

Simple state v/s Nested state

State Diagram for phone line with Activities

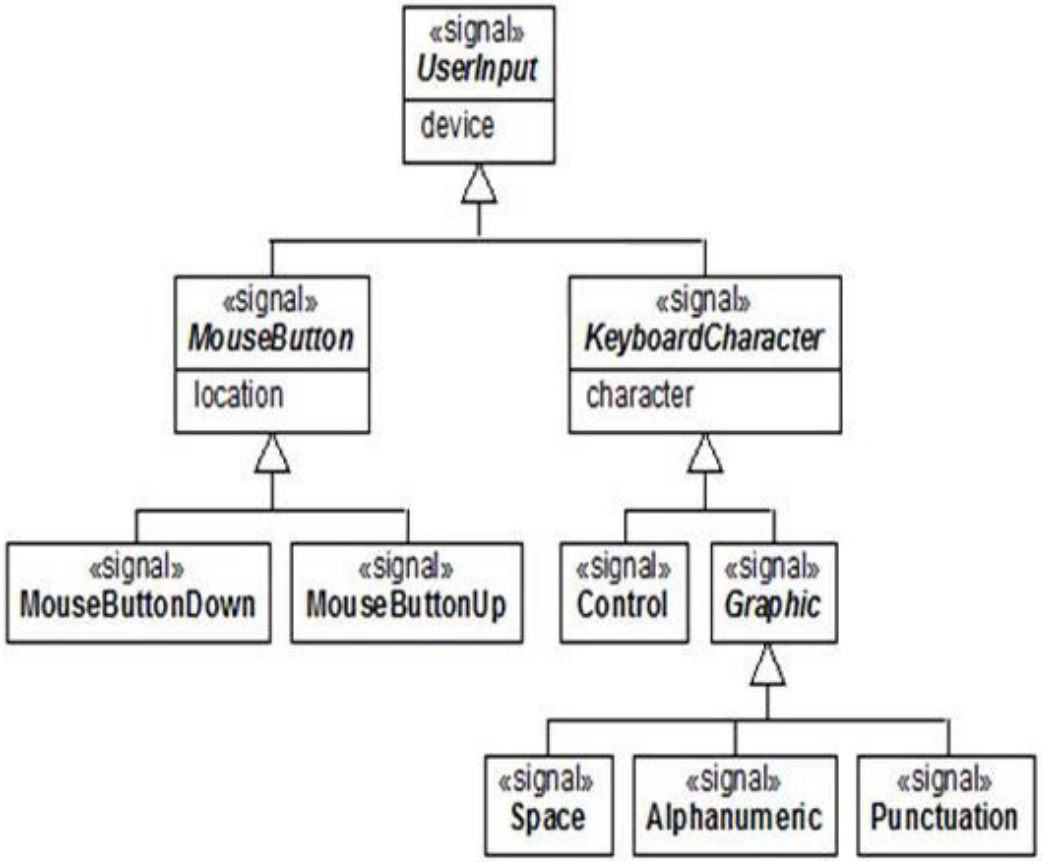


Nested states for a phone line

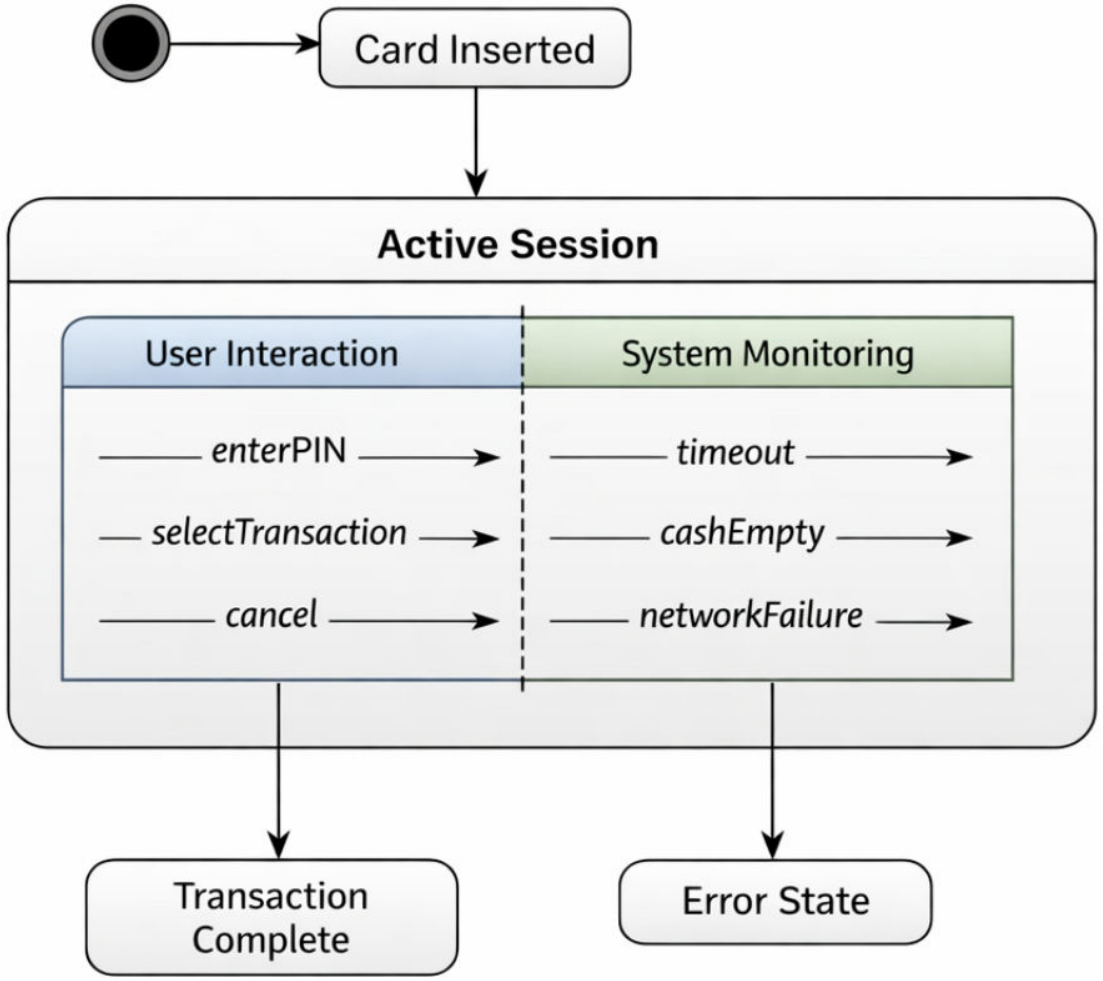


Signal Concurrency

- Organize signals into a generalization hierarchy with inheritance of signal attributes.



Signal Concurrency : Example



In an ATM system, while the ATM is in the Active Session state, user-generated signals such as *enterPIN* and *selectTransaction* and system-generated signals such as *timeout* or *cashEmpty* are handled concurrently using parallel regions. This demonstrates signal concurrency

Concurrency

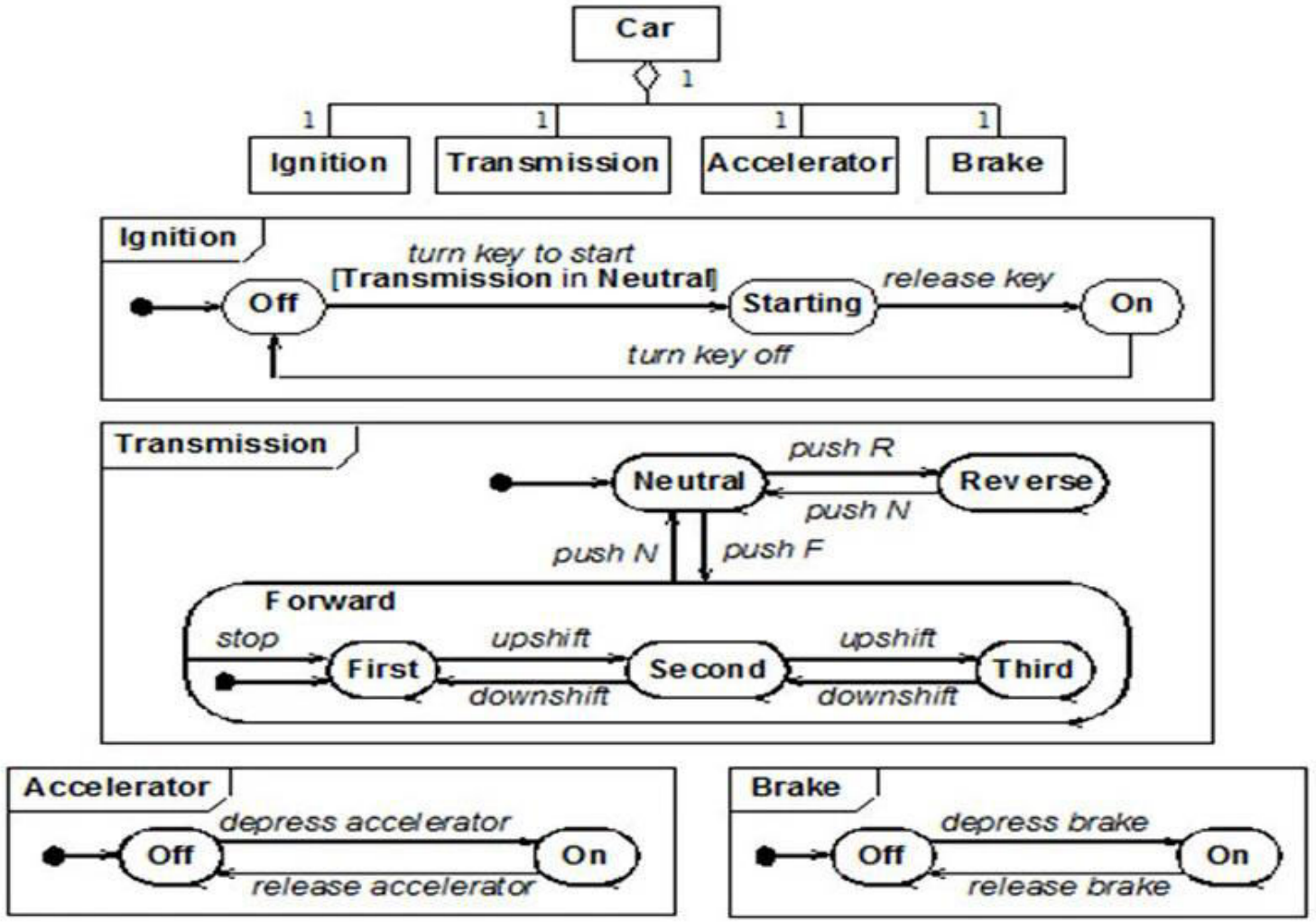
- State Models also supports concurrency among objects. It supports two types of concurrency-

    1. Aggregation Concurrency
    2. Concurrency within an object

Aggregation Concurrency

- A state diagram for an assembly is a collection of state diagram, one for each part. The aggregate state corresponds to the combined states of all the parts.
- Aggregation is "**and-relationship**".
- Aggregate state is one state from the first diagram, and a state from second diagram and a state from each other diagram. In the more interesting cases, the part states interact.

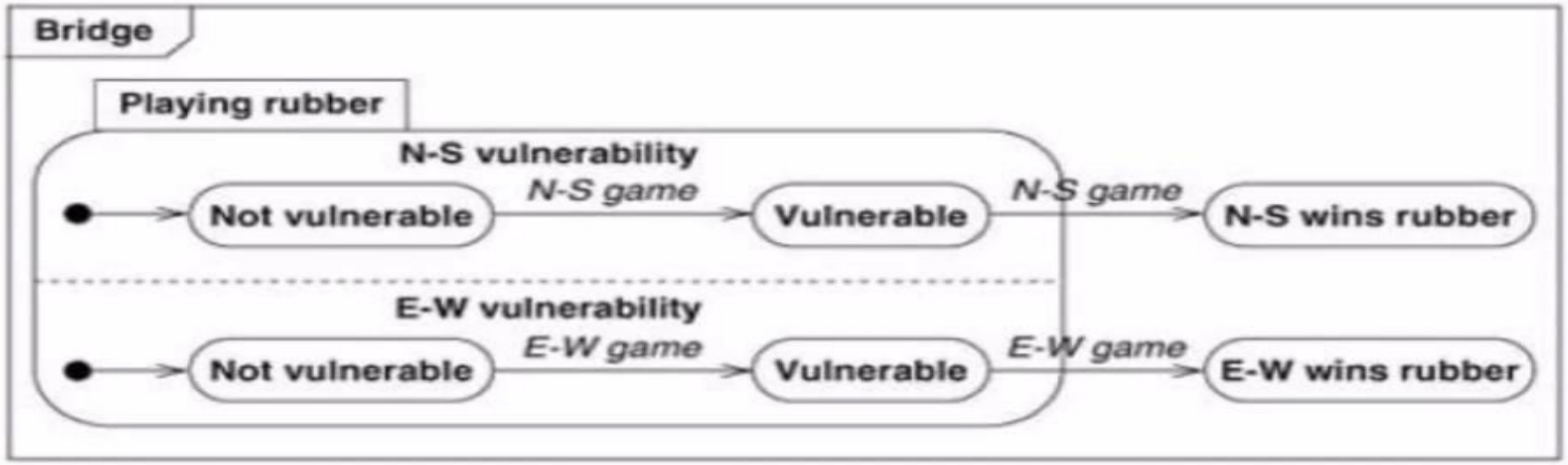
Aggregation Concurrency(Cont'd)



Concurrency within an object

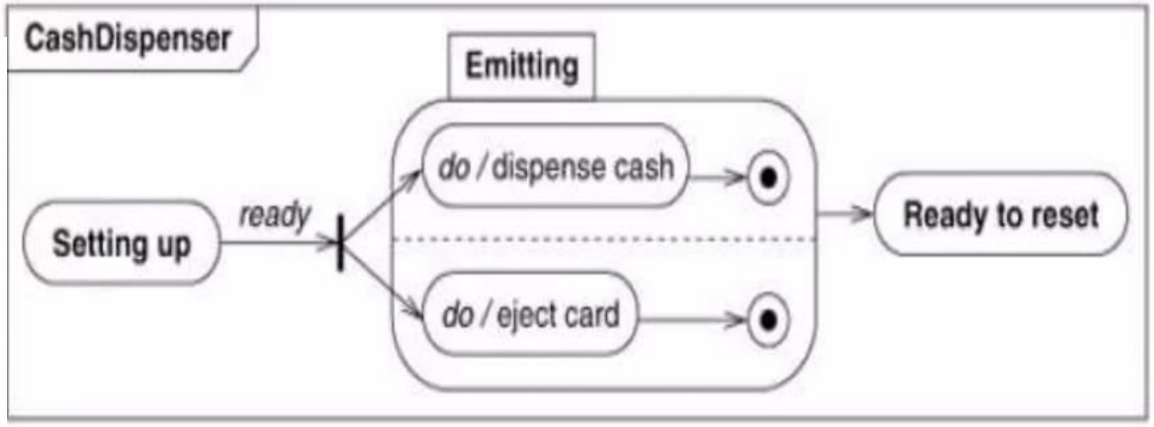
- **The state model implicitly supports concurrency among objects**. In general, objects are autonomous entities that can act and change state independent of one another.

- Objects need not be completely independent and may be subject to shared constraints that cause some correspondence among their state changes.



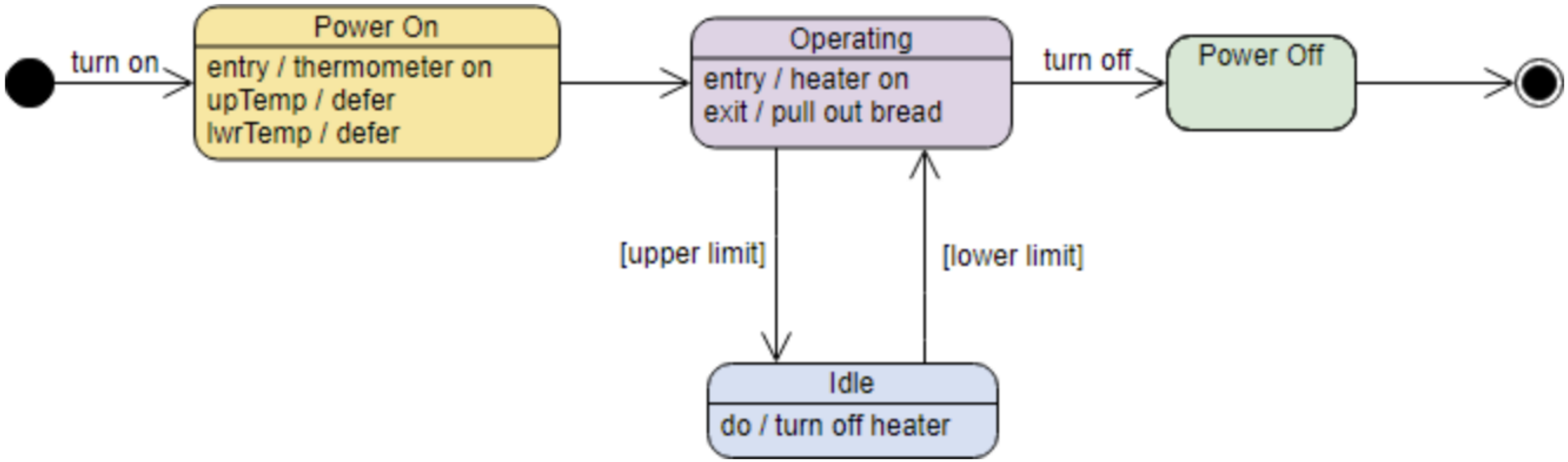
Concurrency within an object

- You can partition some objects into subsets of attributes or links, each of which has its own subdiagram.
- The state of the object comprises one state from each subdiagram.
- The UML shows concurrency within an object by partitioning the composite state diagram.



State Diagram : Examples

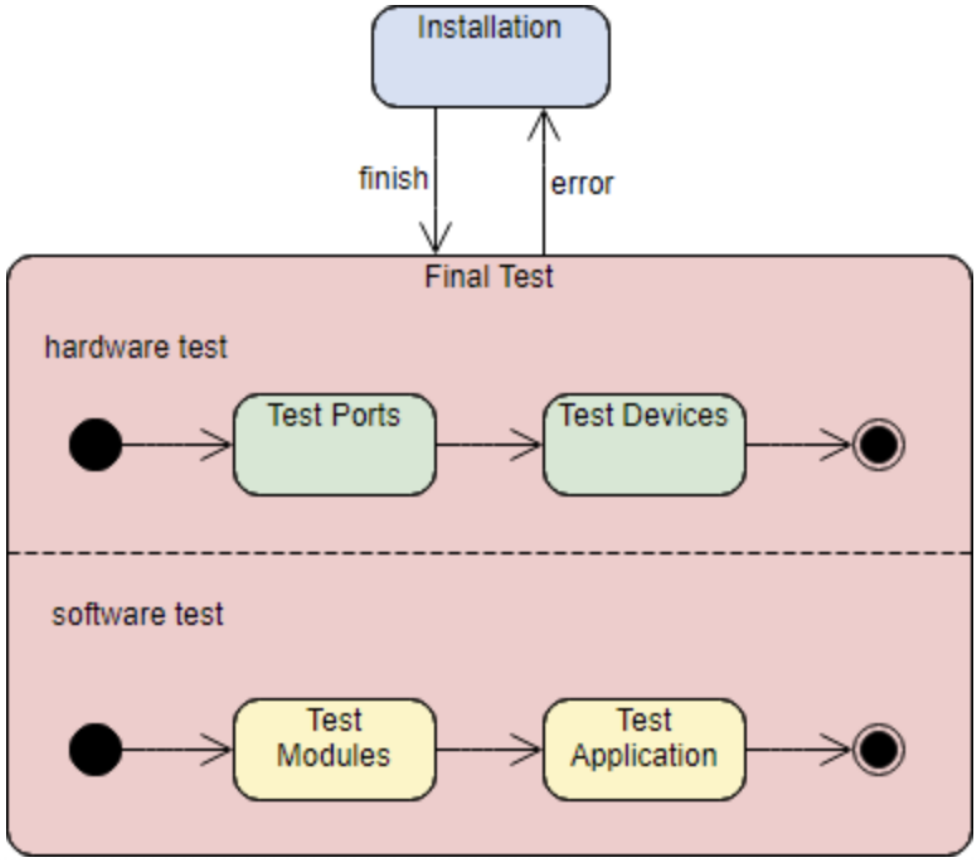
State Machine Diagram example: Toaster



State Diagram : Examples

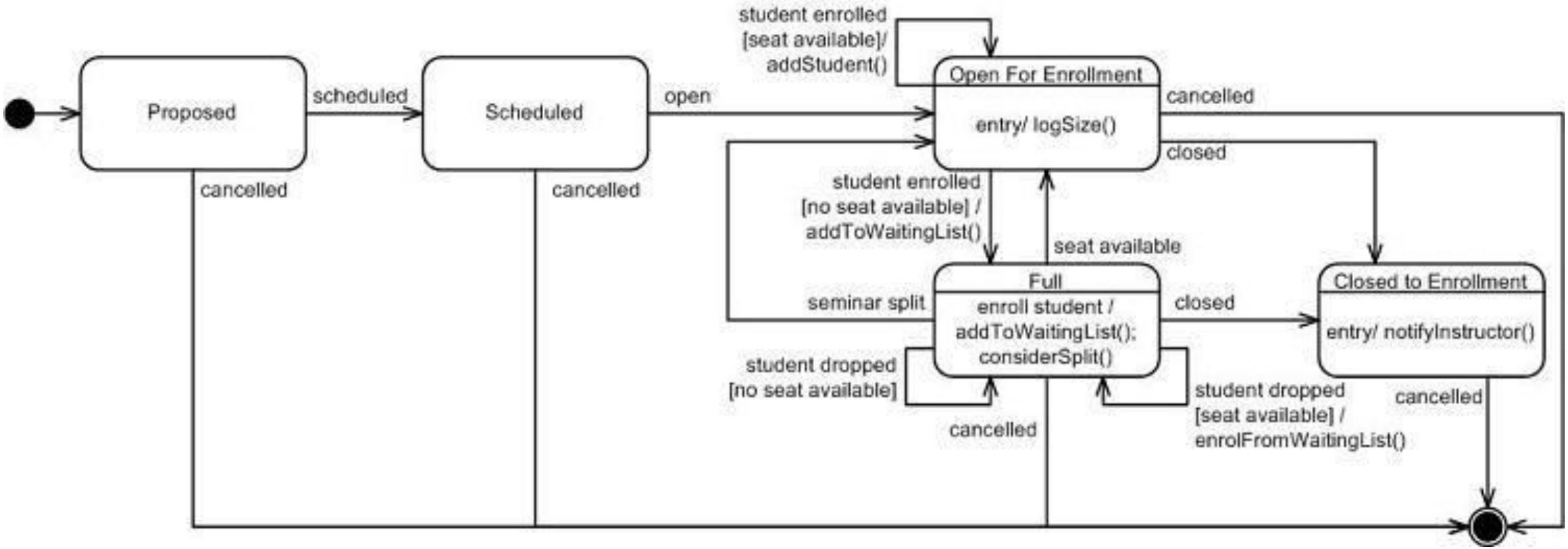
State Machine Diagram example: Computer Testing

This example represents two sets of concurrent substates by using two regions.



State Diagram : Examples

A seminar during registration.



THANK YOU

Prof Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design Using Java

UE23CS352B

Unit:2

Object-oriented Programming: JVM

Prof . Shridevi A Sawant

Department of Computer Science and Engineering

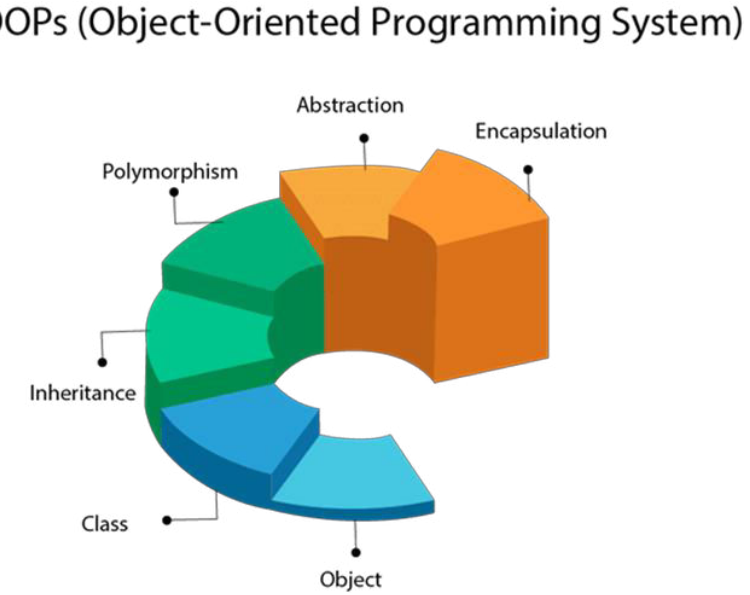
Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design Using Java

Introduction to Object Oriented Programming

Prof . Shridevi A Sawant

Department of Computer Science and Engineering



Acknowledgement:

Prof . Shilpa S

Department of Computer Science and Engineering

Agenda

- Introduction to Java
- Features
- JVM
- JRE
- Java First Program

Introduction to Java

- Java programming language was originally developed by Sun Microsystems, by James Gosling and released in 1995 as core component of Sun Microsystems' Java platform (Java 1.0)
- In 2010, Oracle Corporation acquired Sun Microsystems and became the steward of Java.

- The new J2 versions were renamed as Java SE, Java EE and Java ME respectively.

1. Java Platform, Standard Edition (Java SE)
2. Java Platform, Enterprise Edition (Java EE)
3. Java Platform, Micro Edition (Java ME)

- Java is guaranteed to be Write Once, Run Anywhere.

- Java was mainly developed to create software for consumer electronic devices that could be controlled by a remote

Introduction to Java

Why Use Java?

- Java works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc.)
- It is one of the most popular programming languages
- It is easy to learn and simple to use
- It is open-source and free
- It is secure, fast and powerful
- It has huge community support (tens of millions of developers)
- Java is an object-oriented language that gives a clear structure to programs and allows code to be reused, lowering development costs

Applications:

Mobile applications (specially Android apps), Desktop applications, Web applications
Web servers and application servers, Games, Database connection, And much, much more!

Introduction to Java: Features

1. Simple

There is no need for header files, pointer arithmetic, structures, unions, operator overloading, virtual base classes

2. Object-Oriented

Object-oriented design is a programming technique that focuses on the data (= objects) and on the interfaces to that object.

3. Distributed

Java has an extensive library of routines for coping with TCP/IP protocols like HTTP and FTP. Java applications can open and access objects across the Net via URLs with the same ease as when accessing a local file system.

4. Robust

Inbuilt exception handling features and memory management features.

Introduction to Java: Features

5. Secure

Java is intended to be used in networked/distributed environments. Toward that end, a lot of emphasis has been placed on security. Java enables the construction of virus-free, tamper-free systems.

6. Portable

Unlike C and C++, there are no "implementation-dependent" aspects of the specification. The sizes of the primitive data types are specified, as is the behavior of arithmetic on them.

7. Interpreted

The programmer writes code that will be executed by an interpreter, rather than compiled into object code loaded by the OS and executed by CPU directly. An interpreter executes the code line by line.

8. High-Performance

The bytecodes can be translated on the fly (at runtime) into machine code for the particular CPU the application is running on. The just-in-time compiler knows which classes have been loaded. It can use inlining when, based upon the currently loaded collection of classes, a particular function is never overridden, and it can undo that optimization later if necessary.

Introduction to Java: Features

Common terms of Java:

- **Java Virtual Machine(JVM):** This is generally referred to as JVM. There are three execution phases of a program. They are written, compile and run the program. In the Running phase of a program, JVM executes the bytecode generated by the compiler.

- **Java Development Kit(JDK):** It is a complete Java development kit that includes compiler, Java Runtime Environment (JRE), java debuggers, java docs, etc.

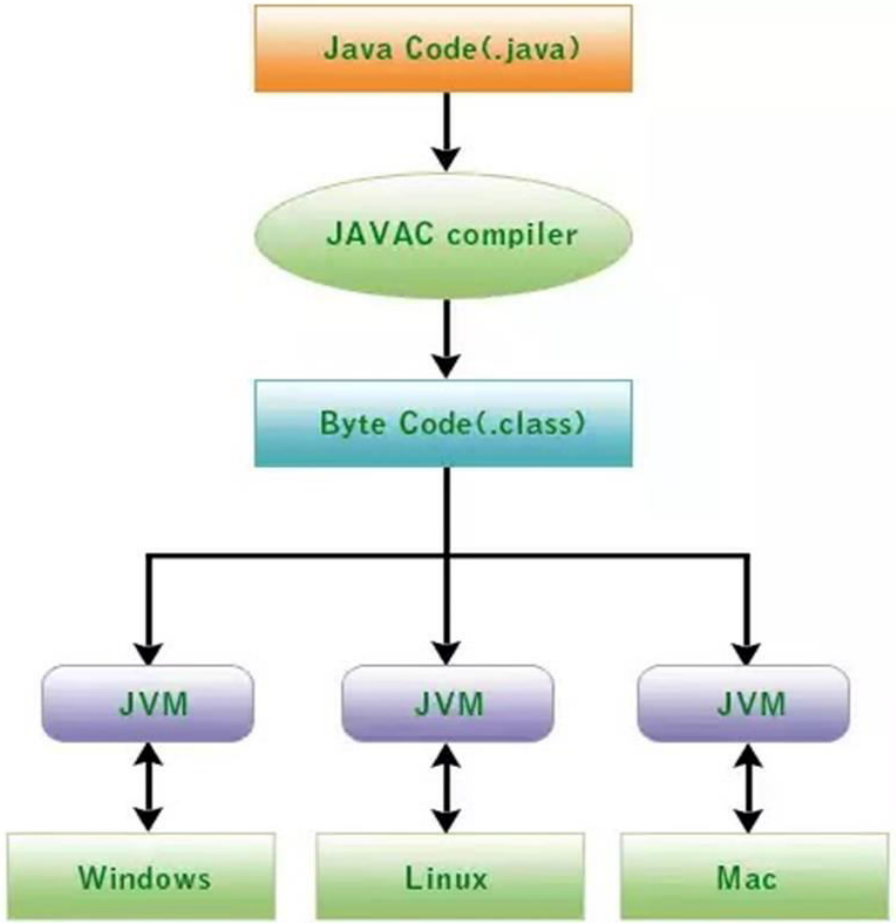
- **Java Runtime Environment (JRE):** JDK includes JRE. JRE installation on our computers allows the java program to run, however, we cannot compile it. JRE includes a browser, JVM, applet support, and plugins. For running the java program, a computer needs JRE.

JAVA Virtual Machine

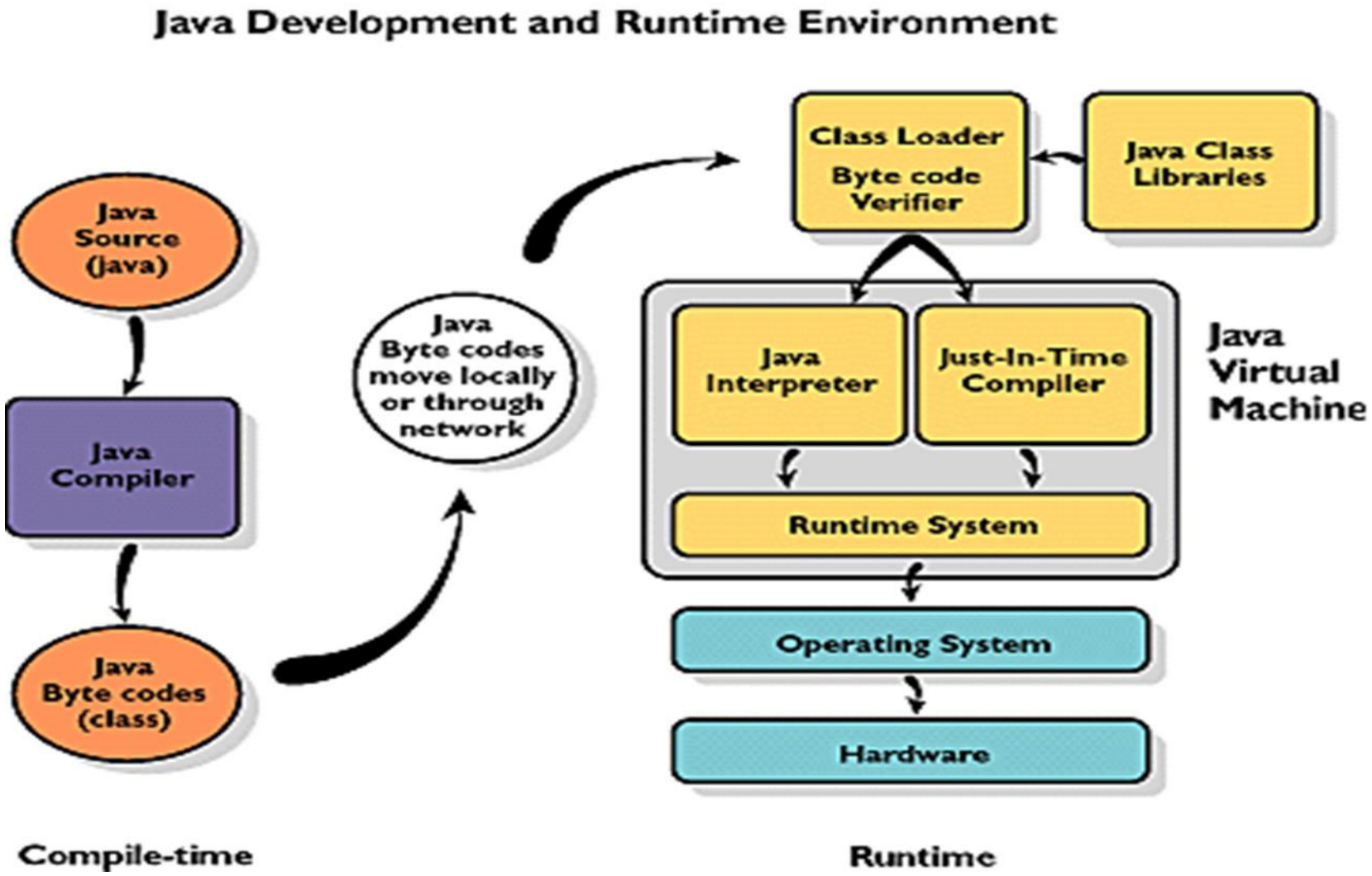
JVM is a platform-dependent execution environment that converts Java bytecode(.class file) into machine language and executes it.

It is:

- **A specification** where the working of Java Virtual Machine is specified. But implementation provider is independent in choosing the algorithm. Its implementation has been provided by Oracle and other companies.
- **An implementation** Its implementation is known as JRE (Java Runtime Environment).
- **Runtime Instance** Whenever you write Java command on the command prompt to run the Java class, an instance of JVM is created.



Java Runtime Environment(JRE)



Java First Program

Implementation of a Java application program involves the following steps.

1. Creating the program
2. Compiling the program
3. Running the program

Example:

```
  public class Test {
   public static void main(String[] args) {
      System.out.println("My Java First Program");
   }
}
```

Save file as -> Test.java
Compile -> javac Test.java
Run -> java Test

Output:

My Java First Program

PES UNIVERSITY

MCQ

1. **Java was mainly developed to create software for:**
   A. Scientific simulations
   B. Consumer electronic devices
   C. Operating systems
   D. Database servers

1. **Which Java component includes the compiler, debugger, and documentation tools?**
   A. JVM
   B. JRE
   C. JDK
   D. Bytecode

3. **The file generated after compiling a Java program has the extension:**
   A. .java
   B. .exe
   C. .class
   D. .obj

4. **The implementation of JVM is commonly known as:**
   A. JDK
   B. JRE
   C. Compiler
   D. Bytecode

5. Which command is used to compile Test.java?

A. java Test
B. javac Test
C. javac Test.java
D. compile Test.java

PES UNIVERSITY

MCQ SOLUTIONS

1. B
2. C
3. C
4. B
5. C

THANK YOU

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design Using Java

UE23CS352B

Unit:2

Abstraction, Encapsulation, Composition

Prof . Shridevi A Sawant

Department of Computer Science and Engineering

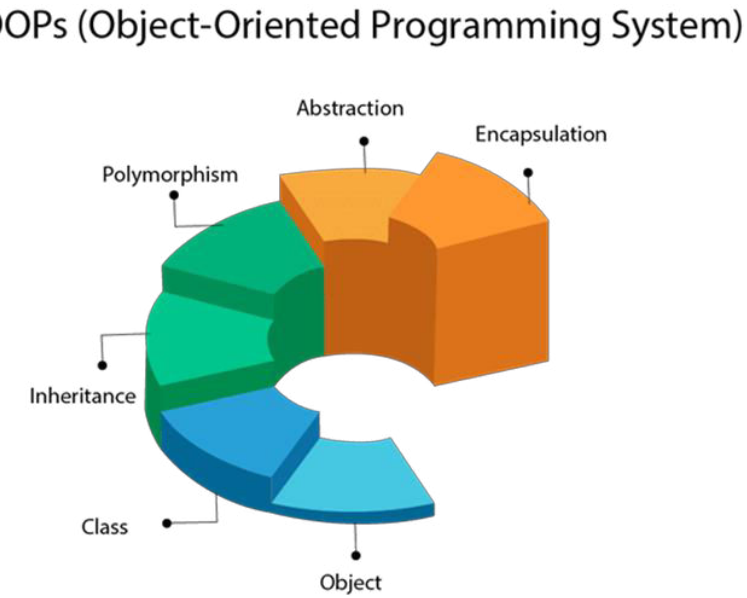
Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design Using Java

Abstraction, Encapsulation, Composition

Prof . Shridevi A Sawant

Department of Computer Science and Engineering



Acknowledgements:

Prof. Mahitha G

Prof . Shilpa S

Department of Computer Science and Engineering

Agenda

- Abstraction
- Encapsulation
- Composition

Abstraction

Data abstraction in Object-oriented programming is a process of providing functionality to the users by hiding its implementation details from them. In other words, the user will have just the knowledge of what an entity is doing instead of its internal working.

In Java, abstraction is achieved by interfaces and abstract classes. We can achieve 100% abstraction using interfaces.

Advantages of Java Abstraction:

- Reduce Complexity
- Avoid code duplication
- Eases the burden of maintenance
- Increase Security and Confidentiality

Abstraction

Abstraction defines an object in terms of its properties (attributes), behavior (methods), and interfaces (means of communicating with other objects).
Abstraction refers to the act of representing essential features without including the background details or explanations.
Since classes use the concept of data abstraction, they are known as **Abstract Data Types (ADT)**

Example:

Abstraction

Abstract class in Java:

- In Java, we can achieve Data Abstraction using Abstract classes and interfaces.
- Interfaces allow 100% abstraction (complete abstraction).
- An Abstract class is a class whose objects can’t be created.
- An Abstract class is created through the use of the abstract keyword. It is used to represent a concept.
- An abstract class can have abstract methods (methods without body) as well as non-abstract methods or concrete methods (methods with the body). A non-abstract class cannot have abstract methods.

Abstraction

Abstract class in Java:

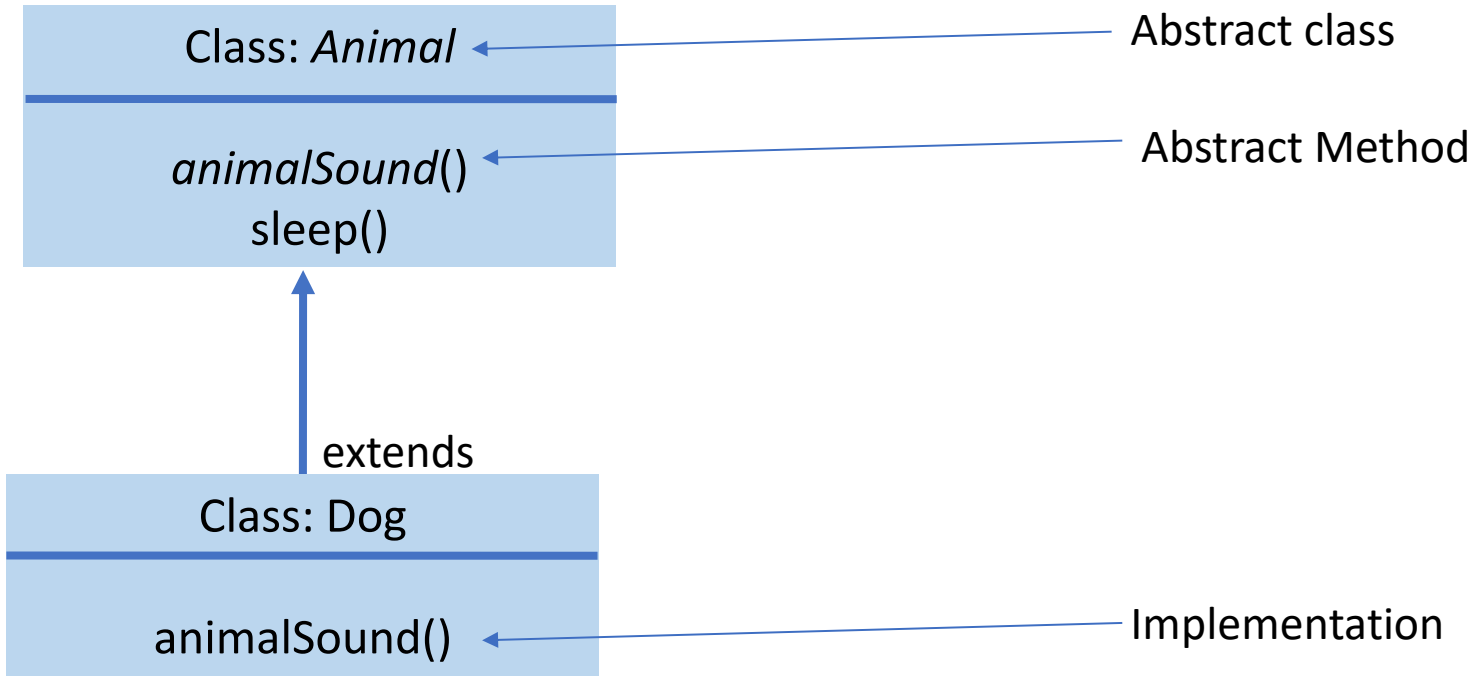
- The class has to be declared as abstract if it contains at least one abstract method.
- An abstract class does not allow you to create objects of its type. In this case, we can only use the objects of its subclass.
- Using an abstract class, we can achieve 0 to 100% abstraction.
- There is always a default constructor in an abstract class, it can also have a parameterized constructor.
- The abstract class can also contain final and static methods.

Abstraction : Example

```
abstract class Animal {   // Abstract class

    public abstract void animalSound();  // Abstract method

    public void sleep() {              // Regular method
        System.out.println("Zzz");
    }
}

class Dog extends Animal {              // Subclass

    @Override
    public void animalSound() {
        System.out.println("The dog says: bow bow");
    }
}

class Main {

    public static void main(String[] args) {
        Dog myDog = new Dog();          // Create a Dog object
        myDog.animalSound();
        myDog.sleep();
    }
}
```

Output:
The Dog says: bow bow
Zzz

Abstraction : Example



Encapsulation

- The wrapping up of data and functions into a single unit is known as encapsulation.
- The data is not accessible to the outside world, only those function which are wrapped in can access it.
- These functions provide the interface between the object's data and the program.
- This insulation of the data from direct access by the program is called data hiding or information hiding.

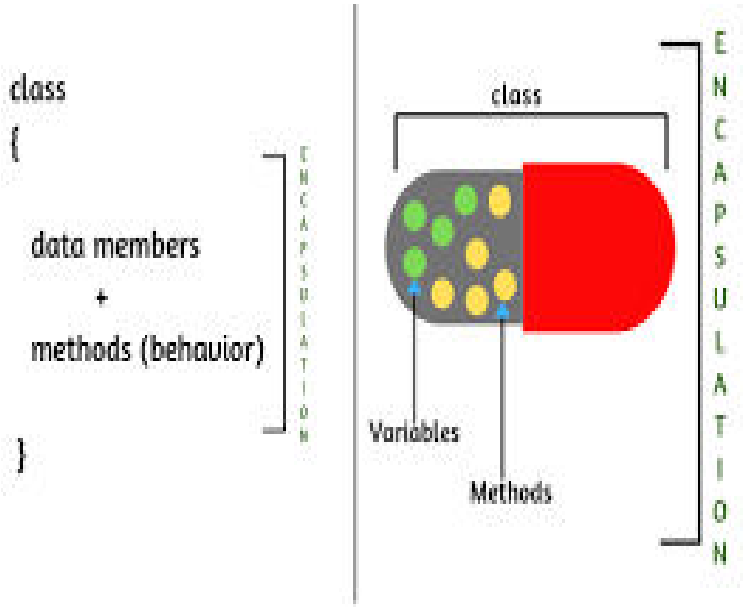


Fig: Encapsulation

Encapsulation

Advantages of Encapsulation

- **Data Hiding**: it is a way of restricting the access of our data members by hiding the implementation details.
- **Increased Flexibility**: We can make the variables of the class read-only or write-only depending on our requirements.
- **Reusability**: Encapsulation also improves the re-usability and is easy to change with new requirements.
- **Testing code is easy**: Encapsulated code is easy to test for unit testing.
- **Freedom to the programmer in implementing the details of the system**: This is one of the major advantages of encapsulation that it gives the programmer freedom in implementing the details of a system.

Encapsulation

Disadvantages of Encapsulation in Java

- Can lead to increased complexity, especially if not used properly.
- Can make it more difficult to understand how the system works.
- May limit the flexibility of the implementation.

Encapsulation: Example

```
class Person {

    private String name;   // private = restricted access

    public String getName() {    // Getter
        return name;
    }

    public void setName(String newName) {    // Setter
        this.name = newName;
    }
}

class Main {

    public static void main(String[] args) {
        Person myObj = new Person();

        myObj.name = "John";                // error
        System.out.println(myObj.name);     // error
    }
}
```

Output:

ERROR!

javac/tmp/mNvaXCz9rO/Main.java/tmp/mNvaXCz9rO/Main.java:15: error: name has private access in Person

myObj.name = "John";   //error

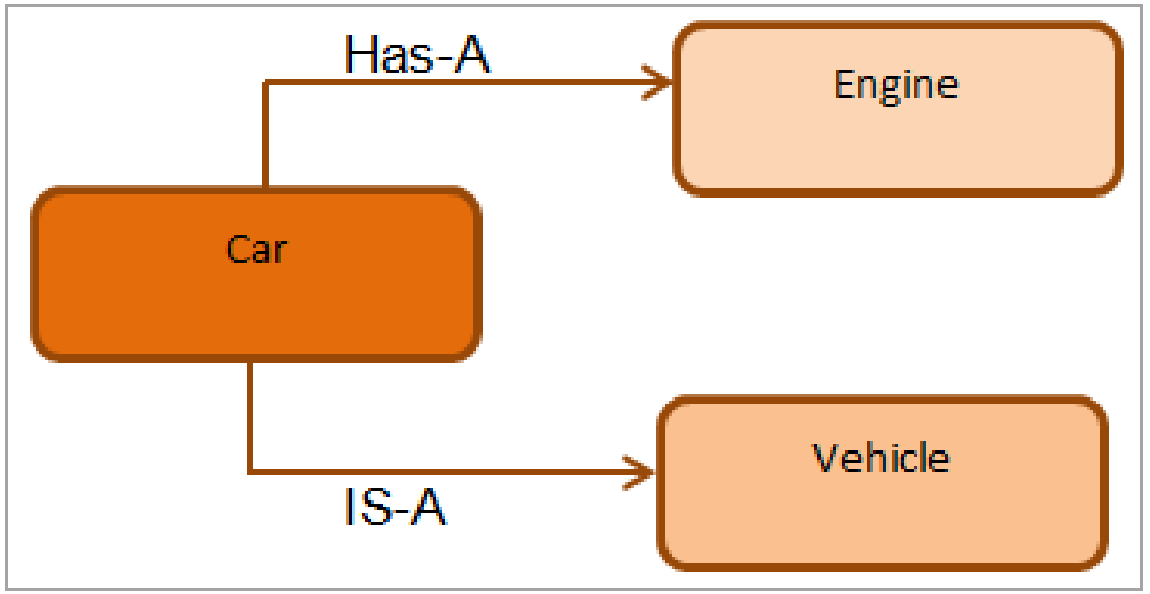
/tmp/mNvaXCz9rO/Main.java:16: error: name has private access in Person

System.out.println(myObj.name); // error

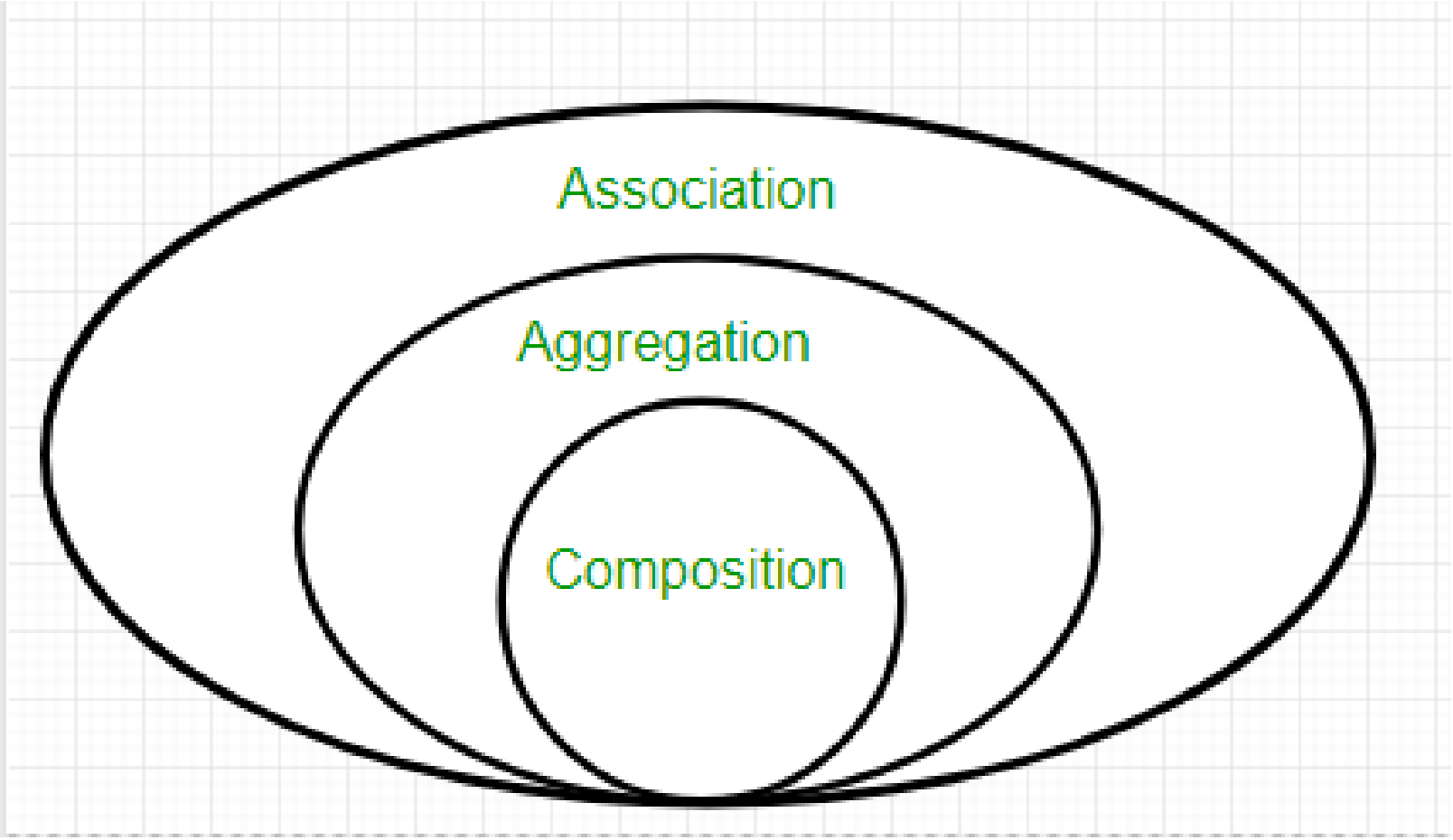
^2 errors

Composition

- The Composition is a way to design or implement the "has-a" relationship.
- Composition and Inheritance both are design techniques.
- The Inheritance is used to implement the "is-a" relationship. The "has-a" relationship is used to ensure the code reusability in our program.
- In Composition, we use an instance variable that refers to another object.



Composition



Composition

- The Composition represents a part-of relationship.
- Both entities are related to each other in the Composition.
- The Composition between two entities is done when an object contains a composed object, and the composed object cannot exist without another entity. For example, if a university HAS-A college-lists, then a university is a whole, and college-lists are parts of that university.
- Favor Composition over Inheritance.
- If a university is deleted, then all corresponding colleges for that university should be deleted.

Composition

Benefits of using Composition:

- Composition allows us to reuse the code.
- In Java, we can use multiple Inheritance by using the composition concept.
- The Composition provides better test-ability of a class.
- Composition allows us to easily replace the composed class implementation with a better and improved version.
- Composition allows us to dynamically change our program's behavior by changing the member objects at run time.

Composition : Example

```
class Engine {
    void start() {
        System.out.println("Engine started");
    }
}

class Car {
    private Engine engine = new Engine(); // Engine is created inside Car → strong ownership

    void drive() {
        engine.start();
        System.out.println("Car is moving");
    }
}
```

Composition : Example

```
public class Main {

    public static void main(String[] args) {
        Car car = new Car();
        car.drive();
    }

}
```

OUTPUT

Engine started
Car is moving

MCQ

1. **Data abstraction in object-oriented programming focuses on:**
   A. Showing internal implementation details
   B. Representing only background details
   C. Providing functionality while hiding implementation details
   D. Eliminating methods from classes
2. **In Java, 100% abstraction can be achieved using:**
   A. Abstract classes only
   B. Concrete classes
   C. Interfaces
   D. Final classes

MCQ

3. Encapsulation in object-oriented programming refers to:

A. Separating data and methods
B. Wrapping data and functions into a single unit
C. Exposing internal data to the user
D. Eliminating data members

4. In encapsulation, data members of a class are accessed:

A. Directly by the program
B. Only through wrapped functions (methods)
C. By any external class
D. Without any restriction

MCQ

5. Composition in object-oriented programming represents a:

A. Has-a relationship
B. Is-a relationship
C. Part-of relationship
D. Uses-a relationship

6. Which benefit of composition allows behavior to change at runtime?

A. Code reuse
B. Static binding
C. Dynamic replacement of member objects
D. Compile-time polymorphism

MCQ SOLUTION

1. C
2. C
3. B
4. B
5. A
6. C

THANK YOU

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design (UE23CS352B)

Prof Shridevi A Sawant

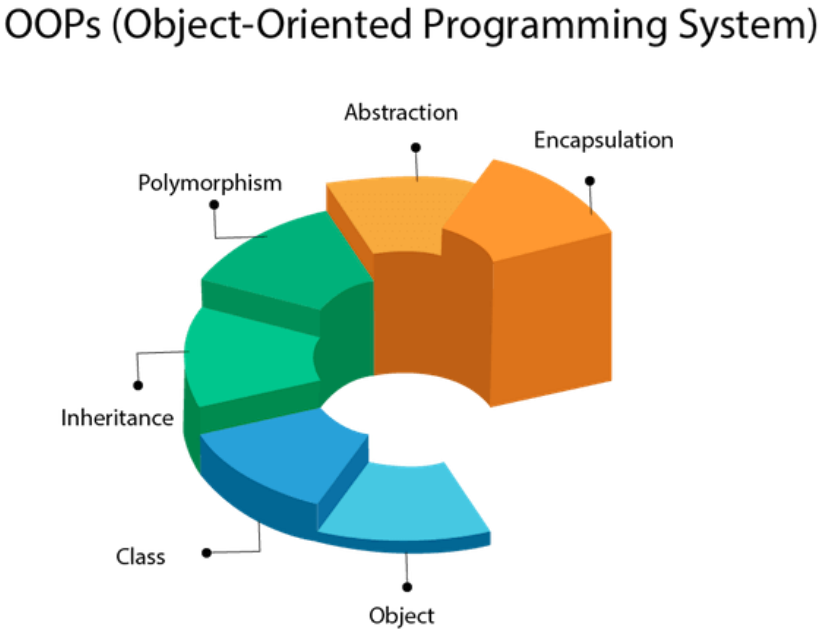
Department of Computer Science and Engineering

Object Oriented Analysis and Design using Java

Class, Attributes, Methods and Objects

Prof. Shridevi A Sawant

Department of Computer Science and Engineering



What is a Class in Java?

A class is a blueprint or template used to create objects.

It defines what an object will have (**attributes**) and what an object can do (**methods**).

Think of a class like a design of a house, and objects are the actual houses built from it.

Object Oriented Programming: Structure of a Class

```
class ClassName {
    // attributes (variables)
    // methods (functions)
}
```

Class : Attributes

- Attributes represent the properties or data of an object.
- They store information about the object.
- Hence they are called data members of the class.

```
class Student {
    int rollNo;
    String name;
}
```

rollNo → attribute
name → attribute

Class : Methods

- **Methods** define the **behavior or actions** an object can perform.
- It **uses attributes** to perform an action

```
class Student {
    int rollNo;
    String name;

    void display() {
        System.out.println(rollNo + " " + name);
    }
}
```

rollNo → attribute
name → attribute
display() → method

Class - Example

```
class Car {
    // Attributes
    String brand;
    int speed;

    // Method
    void drive() {
        System.out.println("Car is driving at " + speed + " km/h");
    }
}
```

Java: Object

What is an Object in Java?

- An **object** is a real-world entity **created from a class**.
- It represents a specific **instance of a class** and occupies memory.
- If a class is a blueprint, an object is the actual thing made from it.

Object: An instance of a class that contains **state (attributes)** and **behavior (methods)**.

- Attributes and methods can be accessed via dot(.) operator.
- Object (instance) of a class is created with '**new**' keyword.

Class and Objects

```
class Student {
    int rollNo;
    String name;

    void display() {
        System.out.println(rollNo + " " + name);
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();  // object

        s1.rollNo = 1;
        s1.name = "John";

        s1.display();
    }
}
```

An object has three things:

Identity – Unique reference (s1)

State – Values of attributes (rollNo, name)

Behavior – Methods it can perform (display())

Java Object : Key Characteristics

- ✓ Created using **new** keyword
- ✓ Has its own **memory**
- ✓ Can access class attributes and methods using dot operator
- ✓ Multiple objects can be created from one class

Class vs Object

Class	Object
Blueprint	Instance
Logical entity	Physical entity
No memory	Occupies memory

Complete Example

```
class Car {
    // Attributes
    String brand;
    int speed;

    // Method
    void drive() {
        System.out.println("Car is driving at " + speed + " km/h");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Car car = new Car();   // object

        car.brand = "Toyota";
        car.speed = 80;

        car.drive();
    }
}
```

Output:

Car is driving at 80 km/h

Types of Access Modifiers in Java

Access modifiers in Java define the visibility of classes, variables, methods, and constructors.

They control who can access what in a program.

Java has **four** access modifiers:

1. public
2. private
3. protected
4. *default* (no keyword)

Types of Access Modifiers in Java

Modifier	Same Class	Same Package	Subclass (diff pkg)	Everywhere
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
default	✓	✓	✗	✗
private	✓	✗	✗	✗

Types of Access Modifiers in Java

Why Access Modifiers are Important?

- Provide **security**
- Support **encapsulation**
- Control **data visibility**
- Improve **code maintainability**

PES UNIVERSITY
CELEBRATING 50 YEARS

MCQ

1. What is a class in Java?

A. A real-world entity
B. An instance of an object
C. A blueprint for creating objects
D. A method

2. An object is:

A. A blueprint of a class
B. A variable
C. An instance of a class
D. A package

MCQ

3. Which keyword is used to create an object in Java?

A. create

B. new

C. class

D. object

4. How many objects can be created from one class?

A. Only one

B. Two

C. Depends on memory

D. Any number

MCQ

5. Which access modifier allows access from anywhere?

A. private

B. protected

C. default

D. public

6. Which access modifier is used when no keyword is specified?

A. public

B. protected

C. private

D. default

PES UNIVERSITY
CELEBRATING 50 YEARS

MCQ SOLUTION

1.C
2.C
3.B
4.D
5.D
6.D

THANK YOU

Shridevi A Sawant
Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design
UE23CS352B

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

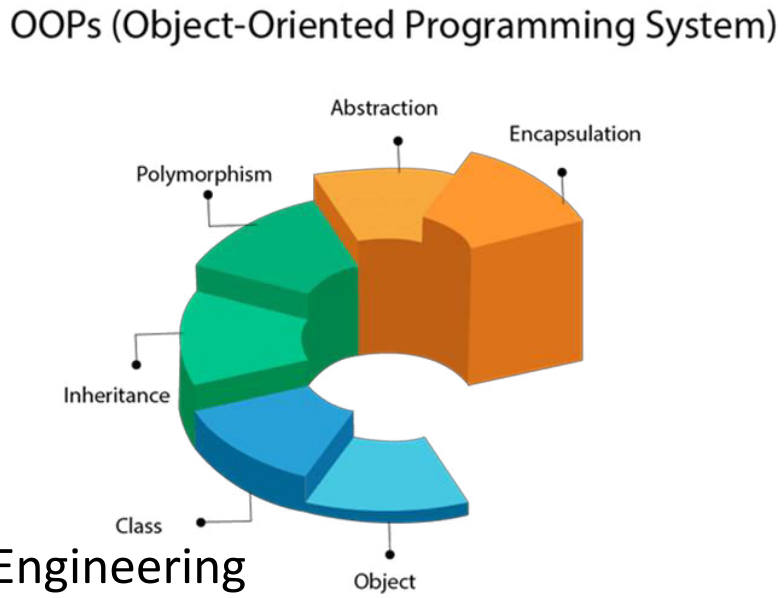
Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

Interfaces and Implementation

Prof . Shridevi A Sawant

Department of Computer Science and Engineering



Acknowledgements:

Prof. Mahitha G & Prof. Bhargavi M

Department of Computer Science and Engineering

Interface in Java

- An **interface** is a collection of abstract methods and constants that a class must implement.
- An interface in Java says what a class supporting this interface can do. It does not say how these will be implemented by the class supporting this interface. It says “what” and “not how”.
- It is used to achieve **100% abstraction** and **multiple inheritance** in Java.

- An interface in Java specifies the method signatures and has no default implementation. So, these methods are **abstract** and also **public**

- **Note :** In Java 8+ version, interfaces can have methods with default implementation.

Interface : Syntax

Interface in Java is defined with the keyword '**interface**'

```
interface InterfaceName {
    // abstract methods
    // constants
}
```

Interface : Example

A bicycle's behaviour, if specified as an interface, might appear as follows:

```
interface Bicycle {

    //  wheel revolutions per minute
    void changeCadence(int newValue);

    void changeGear(int newValue);

    void speedUp(int increment);

    void applyBrakes(int decrement);
}
```

Interface in Java

To implement the interface, the class would use the ***implements*** keyword in the class declaration:

```
class AAABicycle implements Bicycle {

   int cadence = 0;
   int speed = 0;
   int gear = 1;

  // The compiler will now require that methods
  // changeCadence, changeGear, speedUp, and applyBrakes
  // all be implemented. Compilation will fail if those
  // methods are missing from this class.

  public void changeCadence(int newValue) {
        cadence = newValue;
   }

  public  void changeGear(int newValue) {
        gear = newValue;
   }

    public void speedUp(int increment) {
        speed = speed + increment;
    }

   public void applyBrakes(int decrement) {
        speed = speed - decrement;
    }

   public void printStates() {
        System.out.println("cadence:" +
            cadence + " speed:" +
            speed + " gear:" + gear);
    }
}
```

Interface in Java

1. Used to achieve abstraction
2. Supports the functionality of Multiple inheritance
3. It can be used to achieve loose coupling

It is possible that there could be a number of implementations for the same interface.

Interface vs Class

Feature	Class	Interface
Keyword used	class	interface
Object creation	✓ Yes	✗ No
Variables	Instance variables allowed	Only constants (public static final)
Methods	Concrete + abstract	Abstract (default/static allowed from Java 8)
Method implementation	Can exist	Must be provided by implementing class
Inheritance	Extends **one** class	Class can implement **multiple** interfaces
Constructor	✓ Allowed	✗ Not allowed
Access modifiers for methods	Any	Always public
Multiple inheritance	✗ Not supported	✓ Supported
Purpose	Represents **what an object is**	Represents **what an object can do**

Interface in Java

- **Can we instantiate an interface directly?**

NO. you cannot have a constructor. There is no default constructor. You can not make one either.

- **Can we have data members in an interface?**

We can. But these will be for the whole class and will be immutable. In Java terminology, these will be static and final. So, the client of the class has a guarantee about these members. They exist in every class implementing the interface, can be accessed through the class or the object – no difference though – will never change

- **Can a class with all methods implemented also be abstract?**

It can be. If creating an object of that class does not make sense in the domain of application, the class can be made abstract.

Interface in Java

Can we specify that the method of an interface is private?

◦ Definitely NO. (In Java 9+, yes)

Can we specify that the method of an interface is protected?

◦ Interface should remain an interface for everybody in the world. The answer is a clear NO.

- **Can an interface extend an interface?** ◦

Definitely YES.

- **Can a class implement more than one interface?** ◦

  Yes, a class in Java can implement more than one interface using the implements keyword, separated by commas.

  - **Can a class override a few methods of the interface which it implements?**

  ◦ Then the class remains abstract – therefore cannot be instantiated.

MCQ

1. **What does an interface in Java primarily specify?**
   A. How a class implements methods
   B. What a class must do
   C. Memory allocation rules
   D. Object creation logic

2. **In Java, methods declared in an interface are by default:**
   A. private and abstract
   B. protected and abstract
   C. public and abstract
   D. static and final

MCQ

3. Which concept does an interface mainly help achieve?

A. Encapsulation
B. Abstraction
C. Polymorphism
D. Compilation

4. What does it mean if a class implements an interface but does not override all methods?

A. The class is concrete
B. The class is final
C. The class remains abstract
D. The code will not compile

MCQ

5. Can an interface be instantiated directly in Java?

A. Yes
B. No
C. Yes, using new keyword
D. Only using anonymous classes

6. Can an interface contain data members?

A. No
B. Yes, mutable
C. Yes, static and final
D. Yes, instance variables

MCQ SOLUTION

1.B
2.C
3.B
4.C
5.B
6.C

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design (UE23CS352B)

Prof Shridevi A Sawant

Department of Computer Science and Engineering

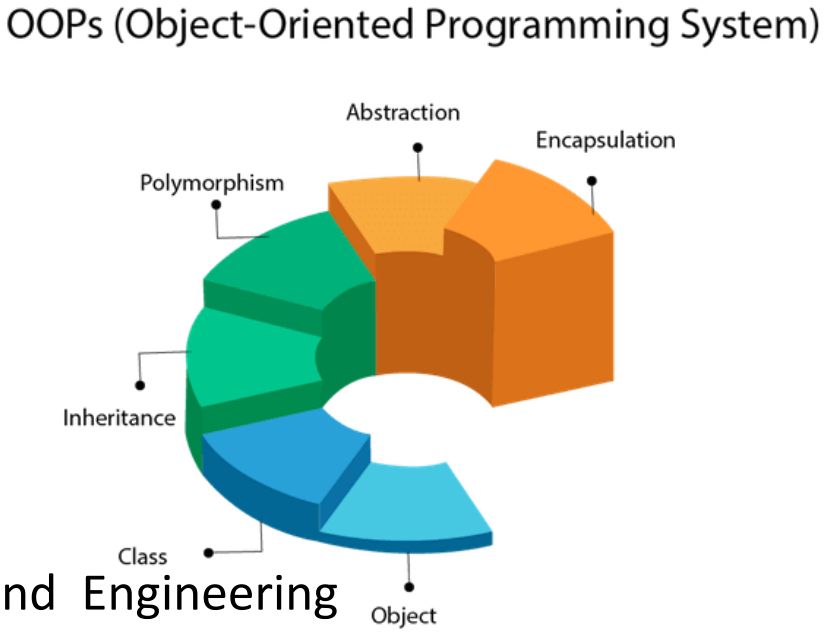
Object Oriented Analysis and Design

Constructors, Destructors and Garbage Collector

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements:
Prof. Mahitha G
Department of Computer Science and Engineering



Object Oriented Analysis and Design

Object Oriented Programming: Constructor

Constructor

- A constructor **initializes an object** when it is created.
- It has the **same name as its class** and is syntactically similar to a method.
- Constructors have **no explicit return type**.
- Use a constructor to give initial values to the instance variables defined by the class, or to perform any other **start-up procedures** required to create a fully formed object.
- All classes have constructors, whether you define one or not, because Java automatically provides a **default constructor** that initializes all member variables to zero or corresponding default value. However, once you define your own constructor, the default constructor is no longer added.
- **Each time a object is created using new operator, constructor is invoked to assign initial values to the data members of the class.**

Object Oriented Programming: Constructor

```
class Student {
    Student() {
        // initialization
    }
}
//Next we create an object of the above class.
Student obj = new Student();
```

Object Oriented Analysis and Design

Object Oriented Programming: Types of Constructors

Default constructor :

- A constructor that has no parameters. If we don't define a constructor for a class, then compiler creates a default constructor.
- Default constructor provides default values to the objects like 0, false, null etc depending on the data type of the instance variables.

Parameterized constructor:

- A constructor with parameters.
- To initialize the fields of a object with given values
- There are no return value statements in a constructor but constructors return the current class instance.

Copy Constructor: Use to create an exact copy of the existing object

Object Oriented Analysis and Design

Constructor - Example

```
class Rect {
     int a; int b;

    Rect() {
        System.out.println("this is constructor");
    }

    void disp() {
        System.out.println("disp");
    }
}
```

```
public class Demo {

    public static void main(String[] args) {

        Rect r = new Rect();   // constructor is called here

        // r.rect();  //  cannot explicitly invoke constructor

        r.disp();

        System.out.println(r.a);
        System.out.println(r.b);
     }
}
```

```
Output -
this is constructor
disp
0
0
```

Constructor - Example

If the constructor is made private, you cannot create the instance of that class from outside the class.

By default the access modifier is “default”

```
class A {

    private A() {          // private constructor
    }

    void msg() {
        System.out.println("Welcome to OOAD with Java class");
    }
}

public class Sample {

    public static void main(String[] args) {

        A obj = new A();   //  Compile Time Error
    }
}
```

Parameterized Constructor - Example

- Constructor **accepts parameters that are u**sed to **initialize object data**
- Uses ‘this’ to avoid name conflict

```
class Student {
    int id;
    String name;

   // Parameterized constructor
    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }

   void display() {
      System.out.println(id + " " + name);
    }
}
```

Copy Constructor - Example

- Copy constructor takes **object of the same class** as argument
- Prevents unwanted **reference sharing**

```
class Student {
   int id;
   String name;

   // Copy constructor
   Student(Student s) {
      this.id = s.id;
      this.name = new String(s.name);//reference not shared
   }
}
```

Object Oriented Programming: Garbage Collector

- Java Garbage Collection is the process to identify and remove the unused objects from the memory and free space.
- One of the best feature of java programming language is the **automatic garbage collection**, unlike other programming languages such as C where memory allocation and de-allocation is a manual process.
- **Garbage Collector** is the program running in the background that looks into all the objects in the memory and find out objects that are not referenced by any part of the program.
- All these unreferenced objects are deleted and space is reclaimed for allocation to other objects.

Object Oriented Programming: finalization

There are certain actions to be performed before an object is destroyed like:

- Closing all the database connections or files
- Releasing all the network resources
- Other Housekeeping tasks
- Recovering the heap space allocated during the lifetime of an object
- Release of release locks

Java provides a mechanism called finalization to do this through finalize( ) method.

Object Oriented Analysis and Design

Object Oriented Programming: finalize ( )

General form of finalize ( ) method

```
protected void finalize( )
{
        //finalization code here
        //specify those actions that must be performed before an object is destroyed.
}
```

Java run time calls this method whenever it is about to recycle an object of the class.

- Keyword protected is used to prevent access to finalize ( ) by the code defined outside the class hierarchy.
- Called just prior to garbage collection and not called when an object goes out of scope
- Execution is not guaranteed
- **Deprecated since Java 9**

MCQ

1. **What happens if a class does not define any constructor?**
   A. Compilation error occurs
   B. Object cannot be created
   C. Java provides a default constructor
   D. Constructor returns null

2. **Which of the following correctly describes a parameterized constructor?**
   A. Constructor without arguments
   B. Constructor used only for copying objects
   C. Constructor that initializes object with given values
   D. Constructor that returns an object

MCQ

3. What is the result if a constructor is declared private?

A. Object can be created anywhere
B. Object can be created only inside the same class
C. Compiler creates a default constructor
D. Constructor becomes static

4. Which of the following is NOT a valid characteristic of a constructor?

A. It initializes an object
B. It has the same name as the class
C. It can have parameters
D. It has an explicit return type

MCQ

5. What is Java Garbage Collection?

A. Manual removal of objects from memory
B. Process of removing unused objects from memory automatically
C. Allocation of memory to objects
D. Compilation of Java programs

6. Which of the following objects are eligible for garbage collection?

A. Objects not referenced by any part of the program
B. Objects referenced by active variables
C. Objects that are currently in use
D. Objects created using new keyword

PES UNIVERSITY
CELEBRATING 50 YEARS

MCQ SOLUTION

1.C
2.C
3.B
4.D
5.B
6.A

PES UNIVERSITY
CELEBRATING 50 YEARS

Reference Book

R3: “Java the Complete Reference”, Herbert Schildt ,McGraw-Hill ,11th Edition, 2018.

THANK YOU

Shridevi A Sawant
Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design - UE23CS352B

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Object Oriented Analysis and Design
UE23CS352

Parameter Passing –
Value Types and Reference Types

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements:
Prof. Mahitha G & Bhargavi M
Department of Computer Science and Engineering

Object Oriented Analysis and Design

Agenda

1. Introduction
2. Value types with Examples
3. Reference types with Examples

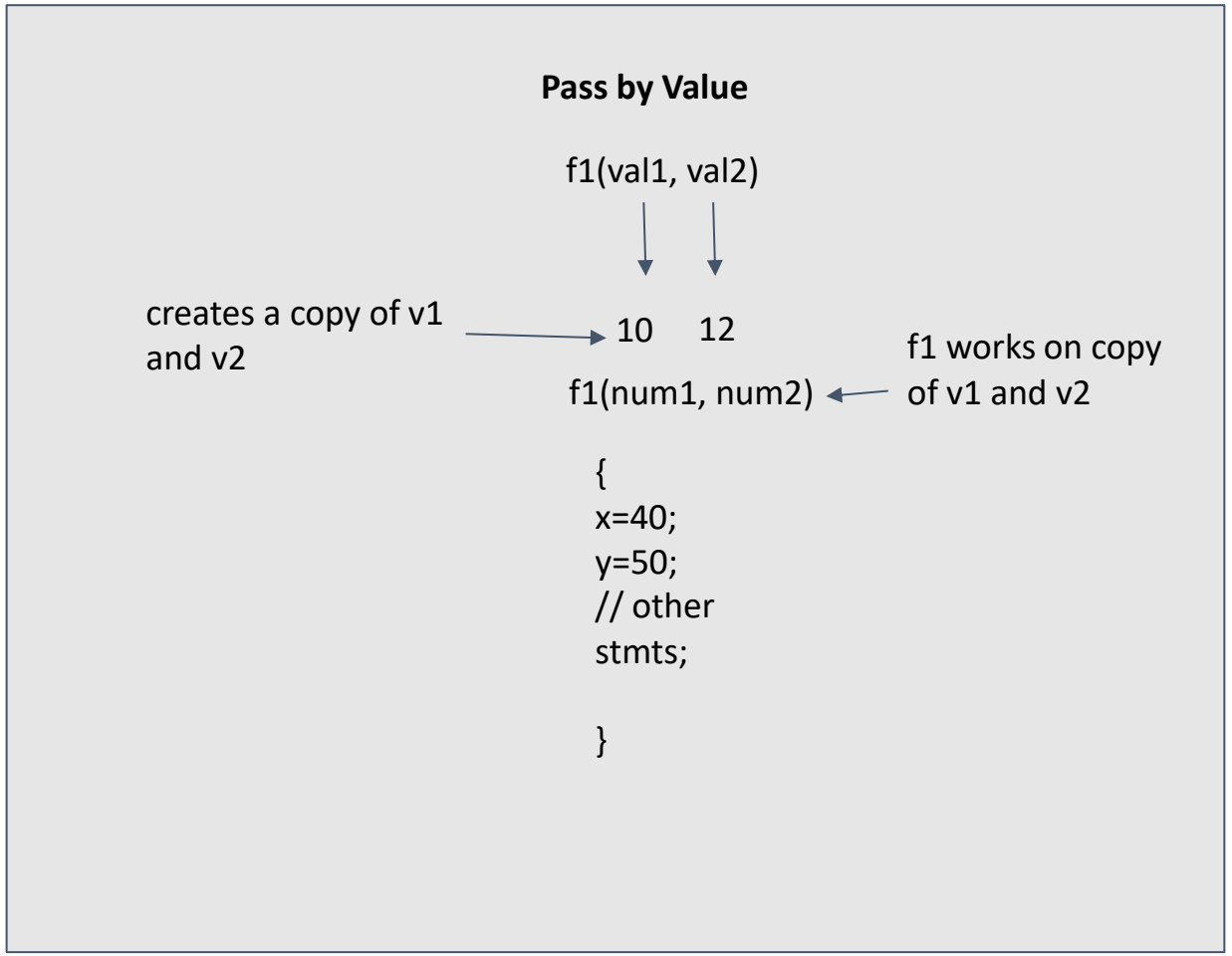
Parameter Passing

Introduction

- Argument is copied to the parameter when some data has to be passed between methods / functions.
- 2 Types of Parameters
  - **Formal Parameter**
  - **Actual Parameter**
- Parameter passing techniques
  - **Pass by Value**
  - **Pass by Reference**

Value types

- Changes made to formal parameter do not get transmitted back to the caller.
- Any modifications to the formal parameter variable inside the called function or method affect only the separate storage location and will not be reflected in the actual parameter in the calling environment.

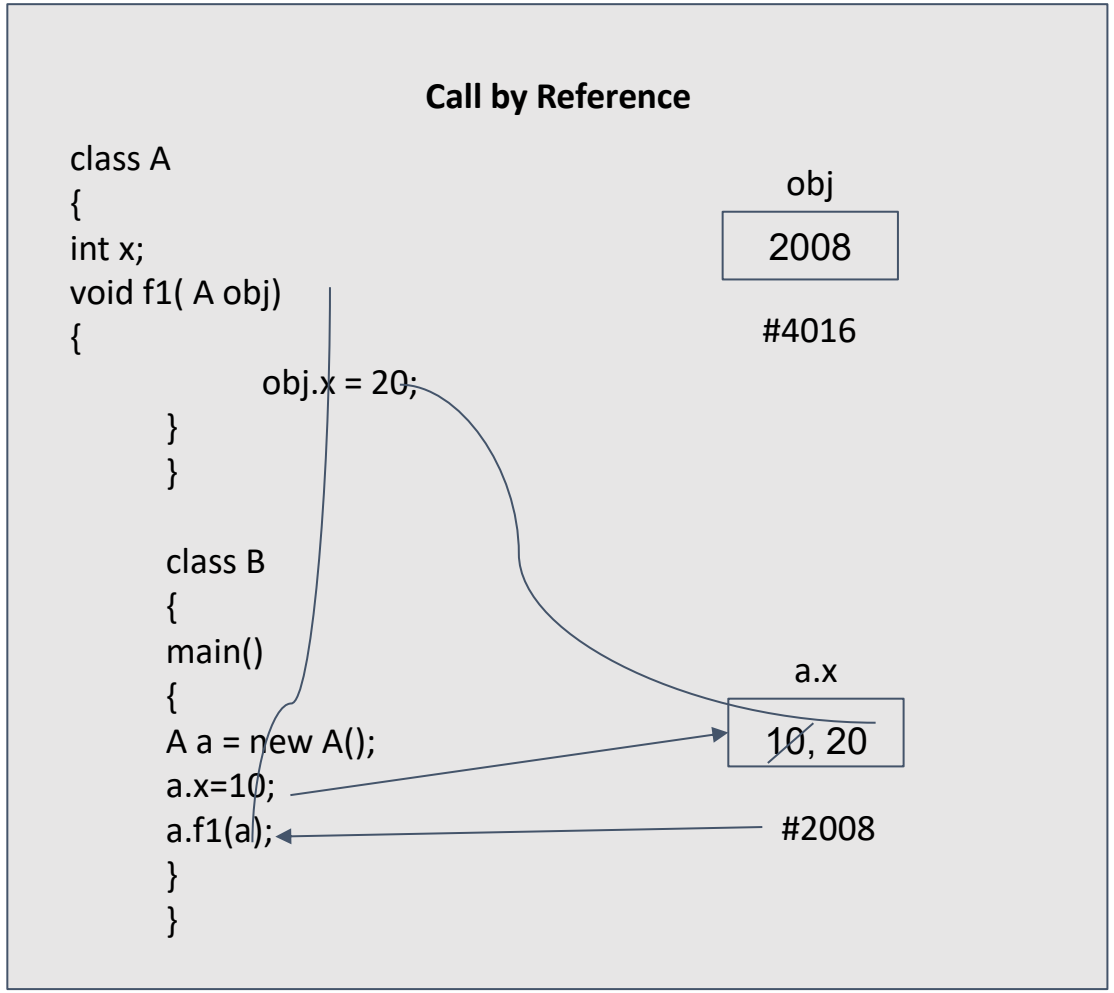


Object Oriented Analysis and Design

Parameter Passing

References- alias

- **Non-primitive type variables store references to objects**
- Changes made to formal parameter do get transmitted back to the caller through parameter passing.
- Any changes to the formal parameter are reflected in the actual parameter in the calling environment as formal parameter receives a reference (or pointer) to the actual data



Reference vs Object

Term	Meaning
Object	Actual data in heap memory
Reference	Memory address of the object
Variable	Stores the reference value

```
Student s = new Student();

(variable) s ──► (reference on stack) ──► Student object (on heap)
```

Object Oriented Analysis and Design

Parameter Passing

Reference types continued..

- **Trick 1: The changes are not reflected back if we change the object itself to refer some other location or object**

```
class Test
{   int x;      Test(int i) { x = i; };     Test()      { x = 0; }
}
class Main2
{
    public static void main(String[] args)
    {   Test t = new Test(5);
        System.out.println(t.x);
        change(t);
        System.out.println(t.x);
    }
    public static void change(Test t)
    {   t = new Test();     t.x = 10;       }
}
```

- **Trick 2: Changes are reflected back if we do not assign reference to a new location or object**

```
class Test
{   int x;      Test(int i) { x = i; };     Test()      { x = 0; }
}
class Main2
{
    public static void main(String[] args)
    {   Test t = new Test(5);
        System.out.println(t.x);
        change(t);
        System.out.println(t.x);
    }
    public static void change(Test t)
    {    t.x = 10;  }
}
```

1. **What happens to an argument when data is passed to a method in Java?**
   A.The argument is deleted
   B. The argument is copied to the parameter
   C. The argument becomes global
   D. The argument is ignored
2. **In pass by value, what happens when the formal parameter is modified?**
   A. Changes are reflected back to caller
   B. Caller variable is destroyed
   C. Changes affect only the formal parameter
   D. Reference is shared
3. **Which of the following data types in Java are passed as references?**
   A. Primitive types
   B. Value types
   C. Non-primitive types
   D. Constants
4. **When are changes to a reference type reflected back to the caller?**
   A. When reference is reassigned
   B. When object data is modified
   C. When method returns null
   D. When primitive data is used

MCQ SOLUTION

1.B
2.C
3.C
4.B

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Object Oriented Analysis and Design - UE23CS352B

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Object Oriented Analysis and Design

Overloading of Methods

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements:
Prof. Priya Badarinath
Department of Computer Science and Engineering

Object Oriented Analysis and Design

Agenda

1. Introduction
2. Coding examples

Object Oriented Analysis and Design

Method Overloading

Introduction

- A feature that allows a class to have more than one method having the same name, if their argument lists are different.
- Method overloading is also known - Compile Time polymorphism, Static polymorphism , Early Binding.
- **Three ways to overload :** The argument lists of the methods must differ in either of these:
  - Changing the number of parameters
  - Changing the data type of parameters
  - Changing the order of parameters of methods.

Note: Method overloading has no relation with return-type

Overloaded methods

Overloaded methods:

- Have the same name
- Have different parameter lists
  - Number of parameters
  - Type of parameters
  - Order of parameters

✓ Compiler decides which method to call (compile-time polymorphism) based on the method signature : **method name + parameter list (number, type, order)**

Method Overloading

Benefits of using Method Overloading

1. Method overloading increases the readability of the program.
2. This provides flexibility to programmers so that they can call the same method for different types of data.
3. This makes the code look clean.
4. This reduces the execution time because the binding is done in compilation time itself.
5. Method overloading minimises the complexity of the code.
6. The code can be used again, which saves memory.

Method Overloading : Example

```
class Calculator {
    // Method with two int parameters
    int add(int a, int b) {
        return a + b;
    }
    // Method with three int parameters
    int add(int a, int b, int c) {
        return a + b + c;
    }
    // Method with double parameters
    double add(double a, double b) {
        return a + b;
    }
    public static void main(String[] args) {
       Calculator calc = new Calculator();

        System.out.println(calc.add(10, 20));
        System.out.println(calc.add(10, 20, 30));
        System.out.println(calc.add(5.5, 2.5));
    }
}
```

Output :

30
60
8.0

Object Oriented Analysis and Design

Method Overloading

MCQ

1. **What is method overloading in Java?**
   A. Defining methods with same name and same parameters
   B. Defining methods with different names
   C. Defining multiple methods with the same name but different argument lists
   D. Defining methods in different classes
2. **Method overloading is also known as:**
   A. Runtime polymorphism
   B. Late binding
   C. Dynamic polymorphism
   D. Compile-time polymorphism
3. **Which of the following best explains early binding?**
   A. Method call resolved during execution
   B. Method call resolved at runtime
   C. Method call resolved during compilation
   D. Method call resolved by JVM dynamically
4. **Which of the following is a key advantage of method overloading?**
   A. Reduces code reuse
   B. Increases complexity
   C. Allows same method name for different data types
   D. Prevents compilation

Object Oriented Analysis and Design

Method Overloading

MCQ SOLUTION

1.C

2.D

3.C

4.C

THANK YOU

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design (UE23CS352)

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

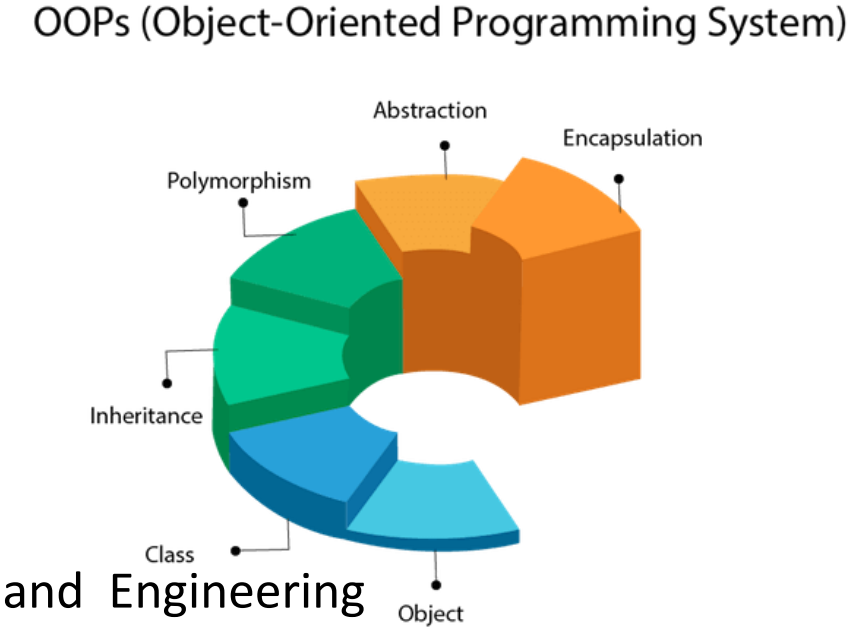
Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

Object Oriented Analysis and Design

Class Attributes and Methods

Prof. Shridevi A Sawant

Department of Computer Science and Engineering



Acknowledgements:

Prof. Mahitha G

Department of Computer Science and Engineering

Method Types

There are two types of methods:

Instance Methods

Instance method are methods which require an object of its class to be created before it can be called. To invoke a instance method, we have to create an Object of the class in which the method is defined.

Static Methods

Static methods are the methods in Java that can be called without creating an object of class. They are referenced by the class name itself or reference to the Object of that class.

By default, methods are said to be instance methods

Attribute Types

There are two types of attributes:

Instance Variables

Instance variables are attributes which require an object of its class to be created before it can be used. To access a instance variable, we have to create an object of the class in which the attribute is defined.

Static Variables

Static attributes are the variables that can be accessed without creating an object of class. They are referenced by the class name itself or reference to the object of that class.

By default, attributes are said to be instance attributes.

Instance attributes and methods : Example

```
class Example1 {

    public static void main(String[] args) {

        // create an instance of the class
        Sample obj1 = new Sample();

        // calling an instance method
        obj1.setName("Ramu");
        System.out.println(obj1.name);
    }
}

class Sample {

    String name = ""; // instance attribute

    public void setName(String s) {
        name = s;
    }
}
```

Static Keyword

The static keyword can be used for :

- Variable (also known as a class variable)
- Method (also known as a class method)
- Block
- Class

Class variables (or static fields)

- Variables that are common to all objects
- They are associated with the class, rather than with any object
- Every instance of the class shares a class variable, which is in one fixed location in memory
- Any object can change the value of a class variable, but class variables can also be manipulated without creating an instance of the class

Object Oriented Analysis and Design

Static Method and Attribute: Example

```
class Example2 {
      public static void main(String[] args) {

       // Accessing the static method setName and field by class name itself.
       Sample.setName("abhiram");
       System.out.println(Sample.name);

       // Accessing the static method setName using object
       Sample obj = new Sample();
       obj.setName("manish");
       System.out.println(obj.name);
   }
}

class Sample {
   public static String name = "";

   public static void setName(String s) {
      name = s;
   }
}
```

Static Attribute vs Instance Attribute

Feature	Static Attribute	Instance Attribute
Belongs to	Class	Object
Copies in memory	One	One per object
Accessed using	Class name	Object
Value shared	Yes	No
this keyword	❌ Not allowed	✅ Allowed

Object Oriented Analysis and Design

Static vs Instance Method

Feature	Static Method	Instance Method
Belongs to	Class	Object
Called using	Class name	Object
Object required	❌ No	✅ Yes
Access instance variables	❌ No	✅ Yes
Access static variables	✅ Yes	✅ Yes
Use of this	❌ Not allowed	✅ Allowed
Memory	One copy	One per object

Static Block

Java static block

Is used to initialize the static data member.

It is executed before the main method at the time of class loading and is executed exactly once.

Static block gets executed exactly once, when the class is first loaded

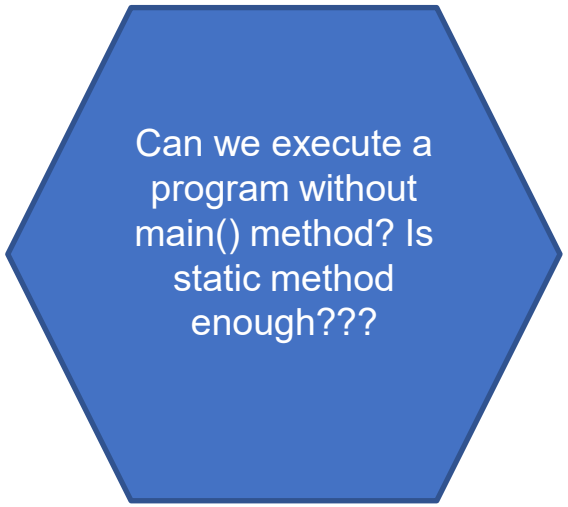
Example of static block

```
class A2{
  static{System.out.println("static block is invoked");}
  public static void main(String args[]){
   System.out.println("Hello main");
  }
}
```

Output:

static block is invoked

Hello main



Static Class

A class can be made static only if it is a **nested class**. We cannot declare a top-level class with a static modifier but can declare nested classes as static. Such types of classes are called Nested static classes. Nested static class doesn't need a reference of Outer class. In this case, a static class cannot access non-static members of the Outer class.

MCQ

1. **Which of the following correctly describes an instance method?**
   A. Can be called without creating an object
   B. Must be called using the class name
   C. Requires an object of the class to be created
   D. Is executed only once
2. **Which of the following is true about static methods?**
   A. They require object creation
   B. They can be called only inside main
   C. They can be called using the class name
   D. They are inherited automatically
3. **By default, methods in Java are considered as:**
   A. Instance methods
   B. Abstract methods
   C. Static methods
   D. Final methods

4. **Which of the following can use the static keyword?**
   A. Variables only
   B. Methods only
   C. Classes only
   D. Variables, methods, blocks, and nested classes
5. **When is a static block executed?**
   A. After main method execution
   B. Before object creation
   C. Before the main method during class loading
   D. When garbage collection occurs
6. **Which statement is true about class variables (static variables)?**
   A. Each object has its own copy
   B. They are shared by all instances of the class
   C. They are stored inside objects
   D. They must be initialized inside constructors

MCQ SOLUTION

1.C
2.C
3.A
4.D
5.C
6.B

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

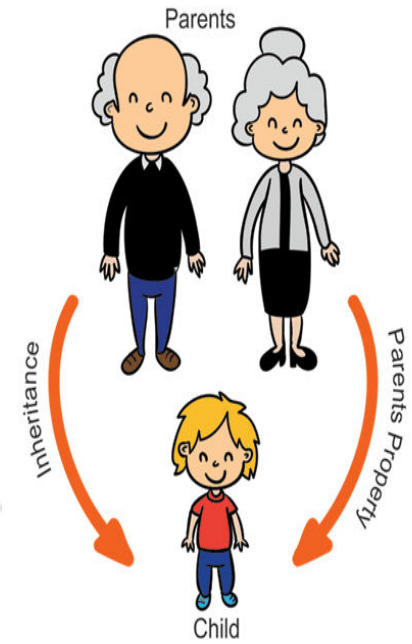
Object Oriented Analysis and Design
UE23CS352B

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

Object Oriented Analysis and Design

Inheritance



Prof. Shridevi A Sawant

Department of Computer Science and Engineering

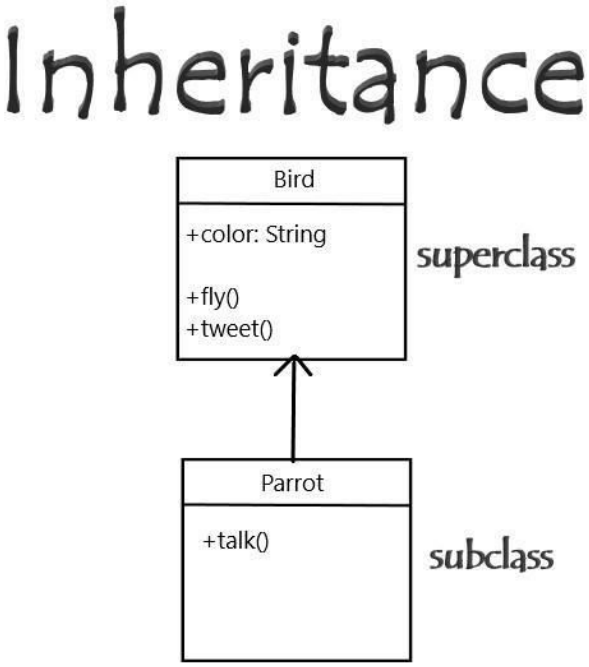
Acknowledgements:

Prof. Mahitha G

Department of Computer Science and Engineering

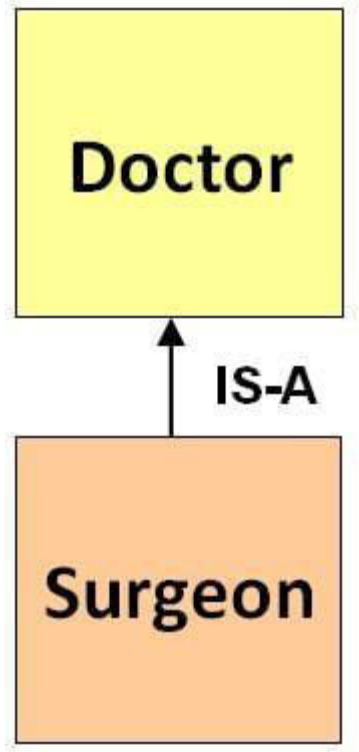
Inheritance

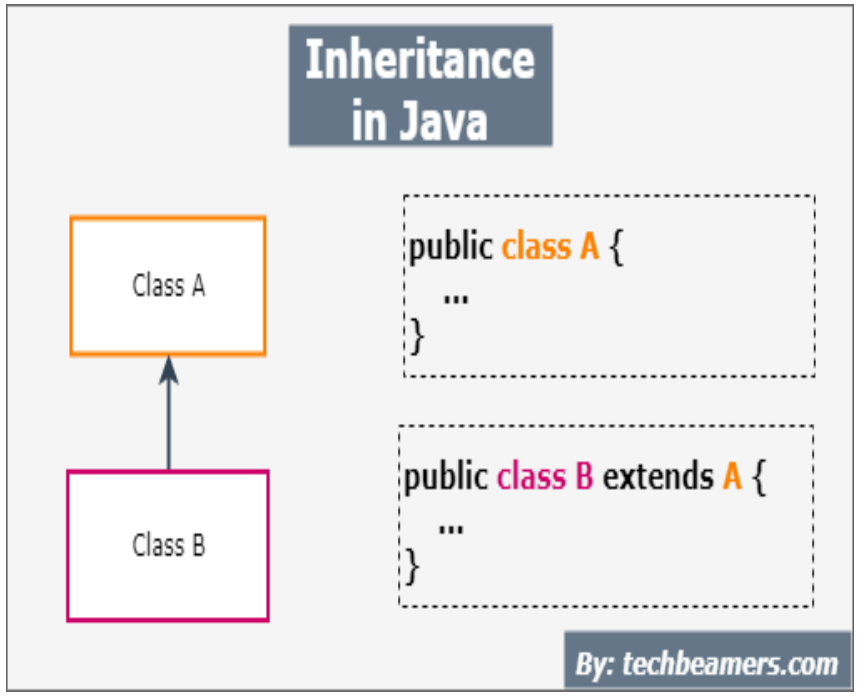
- Inheritance is an important pillar of OOP(Object-Oriented Programming). It is the mechanism in java by which one class is allowed to inherit the features(fields and methods) of another class.
- Inheritance represents **IS-A** relationship which is also known as a parent-child relationship
- Acquiring the properties (data and methods) from one class to other classes enables reusability of code.
- The class whose features are inherited is known as **superclass** (or a base class or a parent class), and the class to which its inherited to is called as **subclass** or child class

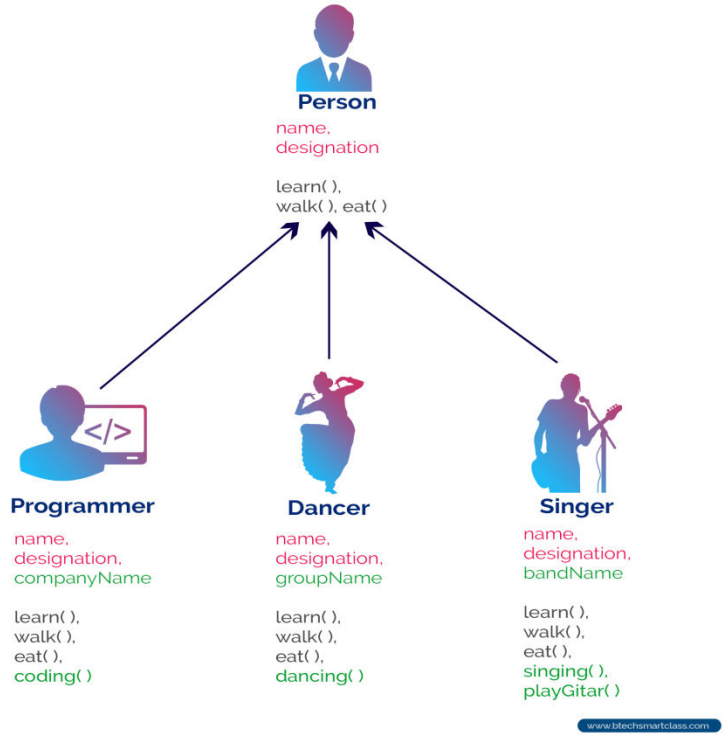


Inheritance

- The child class is a specific type of the parent class.







Inheritance

Syntax:

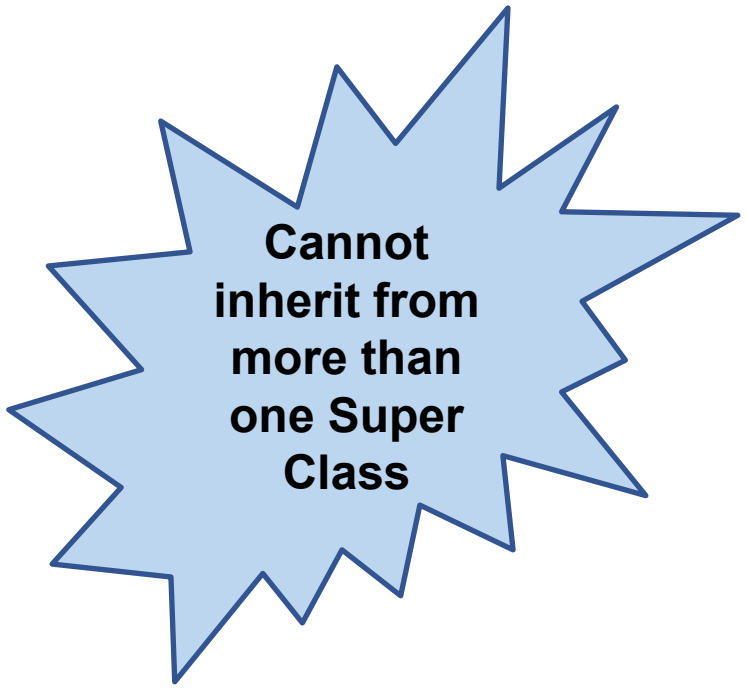
```
class subclass-name extends Superclass-name
{
   //methods and fields
}
```

Inheritance in Java

```
class Teacher {
   String designation = "Teacher";
   String collegeName = "PESU";
   void teach(){
System.out.println("Teaching");
   }
}
public class JavaTeacher extends Teacher{
   String mainSubject = "Java";
   public static void main(String args[]){
JavaTeacher obj = new JavaTeacher();
System.out.println(obj.collegeName);
System.out.println(obj.designation);
System.out.println(obj.mainSubject);
obj.teach();
   }
}
```

OUTPUT

```
PESU
Teacher
Java
Teaching
```



Advantages:

- Reusability and saves time
- Enhances Readability
- Overriding

With inheritance, we will be able to override the methods of the base class so that meaningful implementation of the base class method can be designed in the derived class.

Types of Inheritance

- Single Level Inheritance
- Multilevel Inheritance
- Hierarchical Inheritance
- Multiple Inheritance (Not in Java)
- Hybrid Inheritance (Partially in Java)

Types of Inheritance -Example

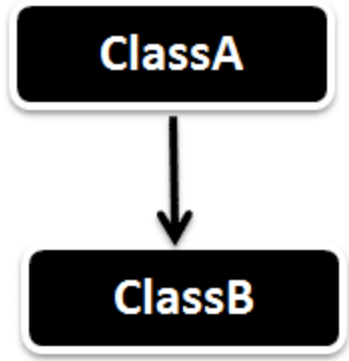
Single Level Inheritance:

```
class Employee {
   float salary = 40000;
}

class Programmer extends Employee {

   int bonus = 10000;

   public static void main(String[] args) {
      Programmer p = new Programmer();
      System.out.println("Programmer salary is: " + p.salary);
      System.out.println("Bonus of Programmer is: " + p.bonus);
   }
}
```

One child class inherits from one parent class.

OUTPUT

```
Programmer salary is: 40000.0
Bonus of Programmer is: 10000
```



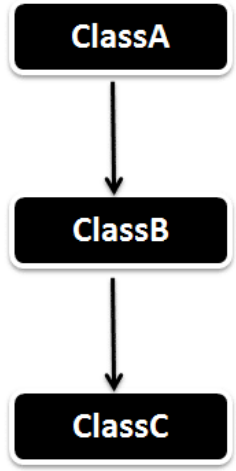
Types of Inheritance

Multilevel Inheritance

```
class Animal{
        void eat(){ System.out.println("eating...");}
}
class Dog extends Animal{
        void bark(){ System.out.println("barking...");}
}
class BabyDog extends Dog{
        void weep(){ System.out.println("weeping...");}
}
class TestInheritance2{
public static void main(String args[]){
BabyDog d=new BabyDog();
d.weep();
d.bark();
d.eat();
}
}
```

OUTPUT

```
weeping...
barking...
eating...
```



instanceof is a keyword used for checking if a reference variable contains a given type of object reference or not. It returns boolean value.

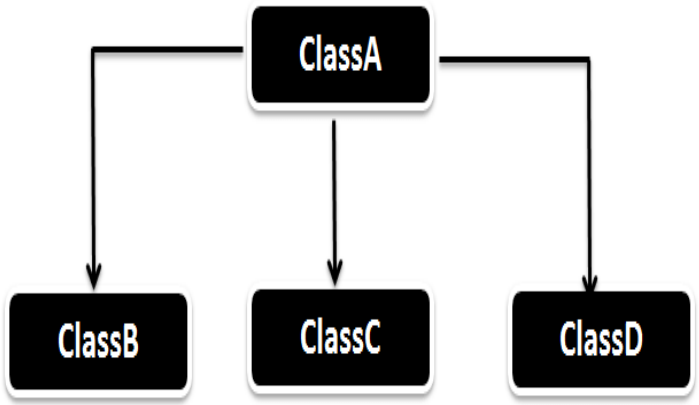
Types of Inheritance

Hierarchical Inheritance

```
class Animal{
        void eat(){ System.out.println("eating...");}
}
class Dog extends Animal{
        void bark(){ System.out.println("barking...");}
}
class Cat extends Animal{
        void meow(){ System.out.println("meowing...");}
}
class TestInheritance3{
public static void main(String args[]){
Cat c=new Cat();
c.meow();
c.eat();
//c.bark();  //C.T.Error
}
}
```

OUTPUT

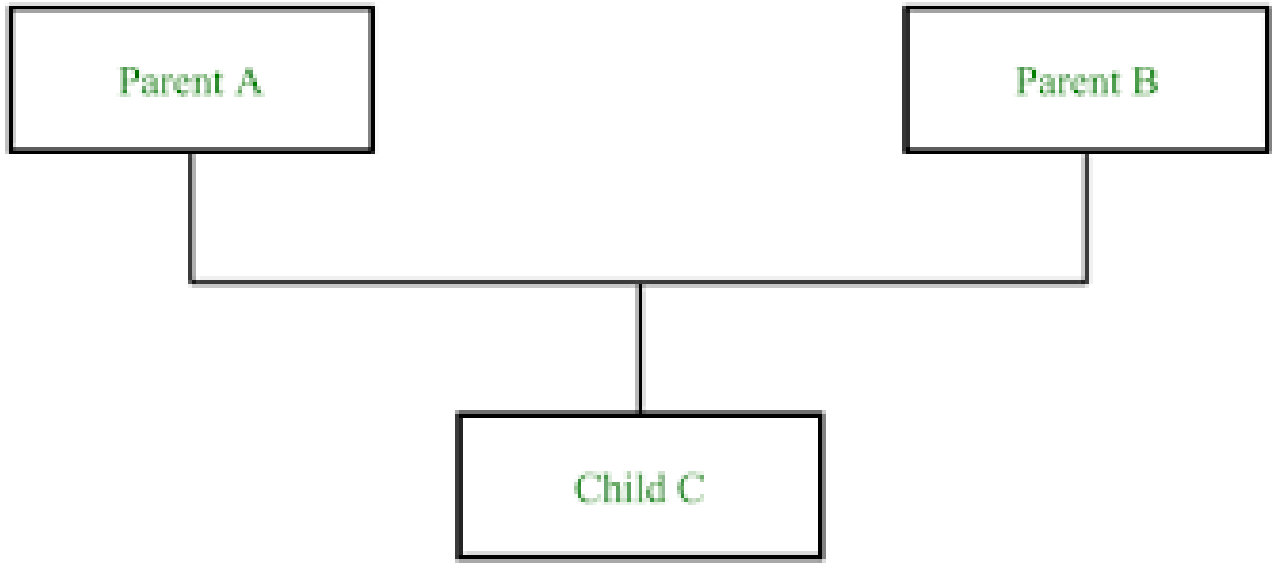
```
meowing...
eating...
```



Types of Inheritance

Multiple Inheritance

- One class inherits from more than one parent class.
- **Java does NOT support multiple inheritance using classes** (to avoid ambiguity problem).
- But Java achieves it using **interfaces.**



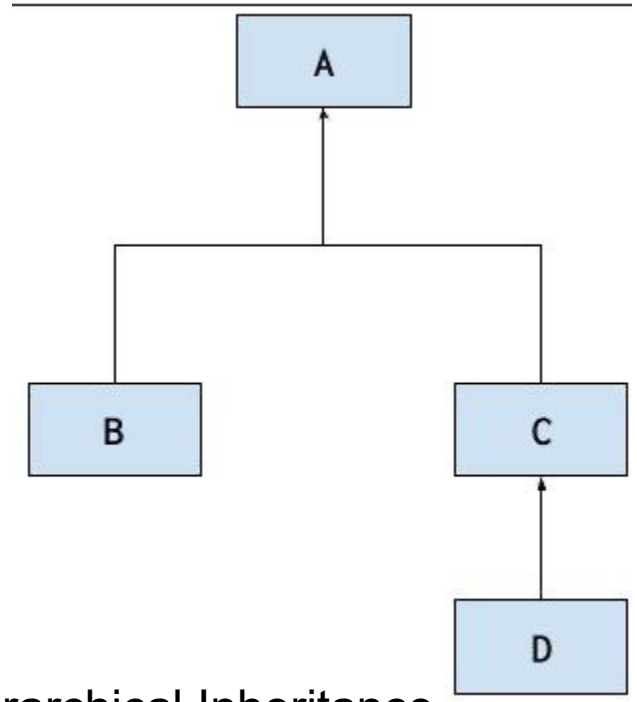
Multiple Inheritance via Interface

```
interface A {
    void show();
}

interface B {
    void display();
}

class C implements A, B {
    public void show() {}
    public void display() {}
}
```

Types of Inheritance

Hybrid Inheritance

- Two or more types of inheritance—such as single, multiple, multilevel, or hierarchical—are combined to create a complex class hierarchy
- It typically merges patterns like **hierarchical and multiple** inheritance or **single and multilevel** inheritance.
- Java does not support multiple inheritance via classes; must use interfaces to achieve hybrid structures.



A, B, C - > Hierarchical Inheritance

A, C, D → Multilevel Inheritance

Constructor and Inheritance

When the object of subclass is created, constructor of sub class by default invokes the **default constructor of super class** . Hence, in inheritance the objects are constructed top-down.
The superclass constructor can be called explicitly using the **super** keyword, but it should be first statement in a constructor.
The **super** keyword refers to the superclass, immediately above of the calling class in the hierarchy.

Method Over-riding

If subclass (child class) has the same method as declared in the parent class, it is known as **method overriding in Java**.

In other words, If a subclass provides the specific implementation of the method that has been declared by one of its parent class, it is known as method overriding.

Usage of Java Method Overriding

- Method overriding is used to provide the specific implementation of a method which is already provided by its superclass.
- Method overriding is used for **runtime polymorphism**

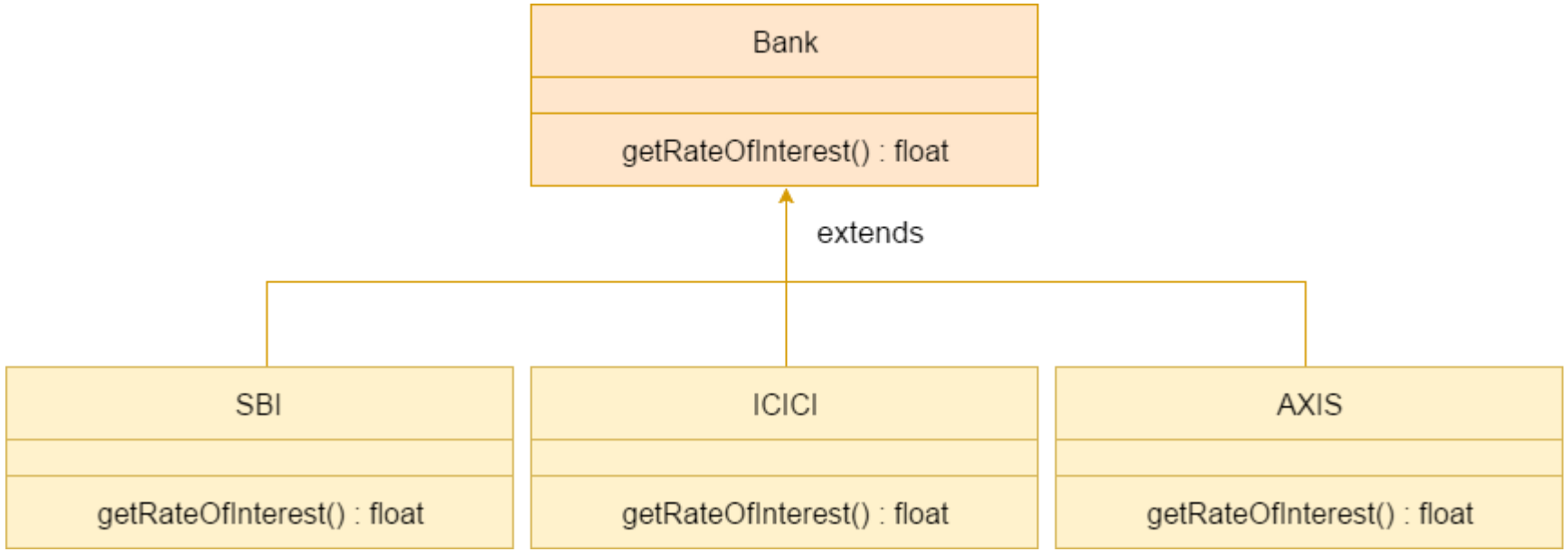
Rules for Java Method Overriding

1. The method must have the same name as in the parent class
2. The method must have the same parameter as in the parent class.
3. There must be an IS-A relationship (inheritance).

Method Over-riding

A real example of Java Method Overriding

Consider a scenario where Bank is a class that provides functionality to get the rate of interest. However, the rate of interest varies according to banks. For example, SBI, ICICI and AXIS banks could provide 8%, 7%, and 9% rate of interest.



Method Over-riding - Example program

An Example of Java Method Overriding

Hands-On

MCQ

1. **What does inheritance represent in Java?**
   A. HAS-A relationship
   B. USES-A relationship
   C. IS-A relationship
   D. PART-OF relationship
2. **The class whose properties are inherited is called:**
   A. Subclass
   B. Superclass
   C. Derived class
   D. Child class
3. **Which keyword is used in Java to achieve inheritance?**
   A. implements
   B. inherits
   C. extends
   D. instanceof

4. **What is method overriding in Java?**
   A. Defining a method with same name but different parameters
   B. Redefining a superclass method in the subclass
   C. Calling superclass methods
   D. Hiding method

5. **Method overriding supports which type of polymorphism?**
   A. Compile-time polymorphism
   B. Static polymorphism
   C. Early binding
   D. Runtime polymorphism

MCQ SOLUTION

1.C
2.B
3.C
4.B
5.D

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design
UE23CS352B

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Object Oriented Analysis and Design using Java

Single Rooted Hierarchy and Object class

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgement: Prof. Mahitha G

Department of Computer Science and Engineering

Single Rooted Hierarchy

- Same Base Class
- Common Interface
- It enables easy memory management
- Simplifies argument passing, allowing generic handling of different object types.

The singly-rooted hierarchy is common with most other object-oriented programming languages.

As Java was created from scratch, it has no backward compatibility issues with any existing language

Single Rooted Hierarchy

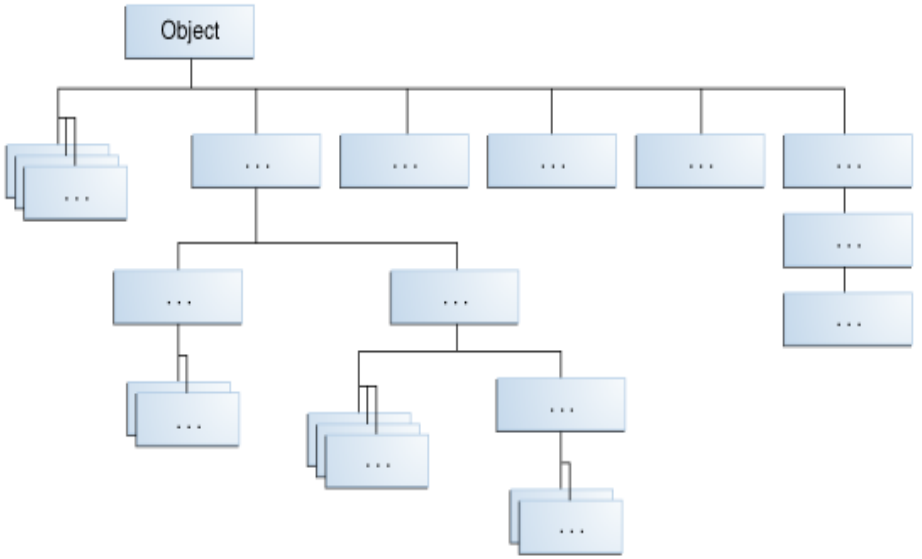
Implementation of Garbage Collector is became easy since required implementation is provided in the base class, enabling to send messages to every object.

All objects in Java are inherited from same base class called 'Object'.

In Java all objects have common interface to implement and it makes implementation of Garbage collector lot easier in Java.

Object class -Introduction

- **Object** class defined by Java is a super class of all other classes, in the absence of any other explicit superclass
- Class Object is the root of the class hierarchy. Every class has Object as a superclass. All objects, including arrays, implement the methods of this class
- A reference variable of type Object can refer to an object of any class
- This is defined in the **java.lang** package



Object class: Methods

Method	Purpose
Object clone( )	Creates a new object that is the same as the object being cloned.
boolean equals(Object *object*)	Determines whether one object is equal to another.
void finalize( )	Called before an unused object is recycled.
Class getClass( )	Obtains the class of an object at run time.
int hashCode( )	Returns the hash code associated with the invoking object.
void notify( )	Resumes execution of a thread waiting on the invoking object.
void notifyAll( )	Resumes execution of all threads waiting on the invoking object.
String toString( )	Returns a string that describes the object.
void wait( ) void wait(long *milliseconds*) void wait(long *milliseconds*, int *nanoseconds*)	Waits on another thread of execution.

Object class Methods

- **`public final Class<?> getClass()`**
- Returns the runtime class of this `Object`.
- **`public int hashCode()`**

Returns a hash code value for the object. This method is supported for the benefit of hash tables such as those provided by `HashMap`.

The general contract of `hashCode` is:

- Whenever it is invoked on the same object more than once during an execution of a Java application, the `hashCode` method must consistently return the same integer. This integer need not remain consistent from one execution of an application to another execution of the same application.

Object class Methods

public boolean equals(Object obj)

Indicates whether some other object is "equal to" this one.

The equals method implements an equivalence relation on non-null object references:

Purpose:

Default Behavior: Compares memory reference (same as ==).

Often overridden for content comparison.

Object class

```
class Box {
    int width, height, depth;

    Box(int l, int m, int n) {
        this.width = l;
        this.height = m;
        this.depth = n; }
    @Override
    public boolean equals(Object o) {
        Box b2 = (Box) o;   // important
        return this.width == b2.width
                && this.height == b2.height
                && this.depth == b2.depth;}}
public class P2Object {
    public static void main(String[] args) {
        Box obj1 = new Box(3, 2, 1);
        Box obj2 = new Box(3, 2, 1);
        System.out.println(obj1 == obj2);
        System.out.println(obj1.equals(obj2)); }}
```

== Operator	equals() Method
Compares memory address (reference)	Compares content (logical equality)

Output

false
true

Object class- Methods

public String toString()

Returns a string representation of the object. In general, the toString method returns a string that "textually represents" this object. The result should be a concise but informative representation that is easy for a person to read. It is recommended that all subclasses override this method.

The toString method for class Object returns a string consisting of the name of the class of which the object is an instance, the at-sign character `@', and the unsigned hexadecimal representation of the hash code of the object

Default return : *ClassName@hashcode*

Object class

```
class Box {
   int width;
   int height;
   int depth;
   Box(int l, int m, int n) {
      this.width = l;
      this.height = m;
      this.depth = n;
   }
   @Override
   public String toString() {
      return width + " " + height + " " + depth; }
}

public class BoxDemo {
   public static void main(String[] args) {
       Box newObj = new Box(3, 2, 1);
      System.out.println(newObj); }
}
```

Output:

3 2 1

MCQ

1. **What is meant by *single-rooted hierarchy* in Java?**
   A. Every class can inherit from multiple base classes
   B. All classes inherit from a single base class
   C. Interfaces act as the root of inheritance
   D. Only abstract classes can be inherited
2. **Which class acts as the root of the Java class hierarchy?**
   A. Base
   B. Root
   C. Object
   D. Class

MCQ

3. A reference variable of type Object can refer to:

A. Only Object instances
B. Only primitive wrapper classes
C. Any class object
D. Only abstract class objects

4. Which of the following methods returns the runtime class of an object?

A. getRuntime()
B. getType()
C. instanceof
D. getClass()

MCQ

5. What should x.equals(null) return for any non-null reference value x?

A. true
B. false
C. NullPointerException
D. Compiler error

6. What is the default output format of Object.toString()?

A. Object memory address
B. Class name only
C. Class name + @ + hexadecimal hash code
D. Hash code only

MCQ SOLUTION

1.B
2.C
3.C
4.D
5.B
6.C

THANK YOU

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design - UE23CS352B

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Object Oriented Analysis and Design

Abstract class

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements:
Prof. Mahitha G
Department of Computer Science and Engineering

Abstract class and Object class - Agenda

1. Introduction to Abstract class
2. Creation and Usage in Java
3. Coding examples – Demo

Abstract class

Introduction

- Provides **implementation reuse – provides default implementation**
- To create a super class that only defines a generalized form that will be shared by all of its subclasses, leaving it to each subclass to fill in the details.
- The super class determines the nature of methods that the subclasses must implement.
- Referred to as **subclasser responsibility** because they have no implementation specified in the superclass. No method body is present.

Abstract class

Creation and usage in Java

- Created using **abstract** keyword at the beginning of the class declaration.
- May contain abstract methods, i.e., methods without body
- If a class has **at least one abstract method**, then the **class must be declared abstract**
- **Cannot be instantiated**
- If a class extends abstract class then either it has to provide implementation of all abstract methods or declare this class as abstract class
- Can have both **static and non-static data members and methods** like any other java class
- **Can not be final** in Java because abstract classes are used only by extending
- A class **can extend only one abstract class** as Java does not support multiple inheritance

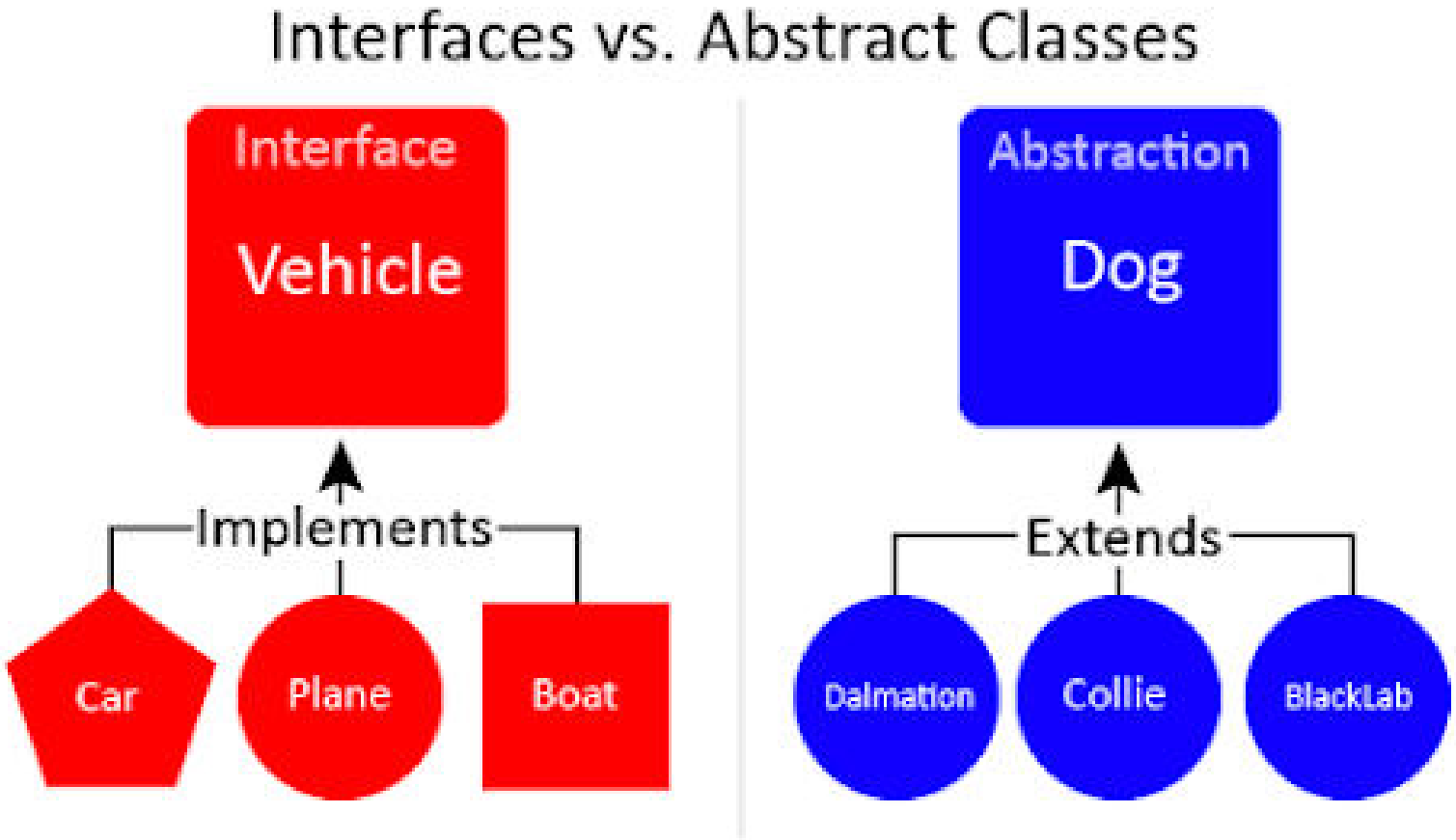
Object Oriented Analysis and Design

Abstract class: Example

```
abstract class Figure {

    double dim1;
    double dim2;

    Figure(double a, double b) {
        dim1 = a;
        dim2 = b;
    }

    abstract double findArea();
}
```

```
class Rectangle extends Figure {
        Rectangle(double height, double width){
         super(height, width);
         }
           double findArea(){
              return dim1 * dim2
           }
}

class Triangle  extends Figure {
        Triangle(double base, double height){
         super(base, height);
         }
           double findArea(){
              return 0.5 * dim1 * dim2
           }
```

Abstraction Vs Interface



Abstraction Vs Interface

Feature	Abstract Class	Interface
Keyword	abstract class	interface
Object Creation	Cannot create object	Cannot create object
Methods	Can have abstract + concrete methods	By default abstract methods (can have default & static methods from Java 8)
Variables	Can have instance variables	Only public static final (constants)
Constructors	✅ Allowed	❌ Not allowed
Access Modifiers	Can use any (private, protected, public)	Methods are public by default
Multiple Inheritance	❌ Not supported (for classes)	✅ A class can implement multiple interfaces
Purpose	Partial abstraction	Full abstraction (possible)

MCQ

1. **What is the primary purpose of an abstract class?**
   A. To support multiple inheritance
   B. To provide implementation reuse
   C. To define only method signatures
   D. To improve execution speed
2. **Why is an abstract class used as a superclass?**
   A. To avoid inheritance
   B. To define a generalized form shared by subclasses
   C. To enforce object creation
   D. To remove abstraction
3. **What is true about abstract methods?**
   A. They have a method body
   B. They are private
   C. They have no method body
   D. They must be static

MCQ

4. Can an abstract class be instantiated?

A. Yes
B. No
C. Only using inheritance
D. Only using constructors

5. What must a subclass do when it extends an abstract class?

A. Nothing
B. Implement all abstract methods or be abstract
C. Override only static methods
D. Implement interfaces

6. Why can an abstract class not be declared final?

A. Because it has abstract methods
B. Because it cannot be instantiated
C. Because it must be extended
D. Because it is slow

MCQ SOLUTION

1.B
2.B
3.C
4.B
5.B
6.C

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Object Oriented Analysis and Design - UE23CS352B

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Object Oriented Analysis and Design

Array, List and Stack

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements:
Prof. Mahitha G & Bhargavi M
Department of Computer Science and Engineering

Object Oriented Analysis and Design

Collections - Arrays, List, Stack

1. Introduction to Array class
2. Creation and Usage of Array in Java
3. Coding examples – Demo
4. Introduction to List class
5. Creation and Usage of List interface in Java
6. Coding examples – Demo
7. Introduction to Stack class
8. Creation and Usage of Stack in Java
9. Coding examples – Demo

Object Oriented Analysis and Design

Collections - Introduction

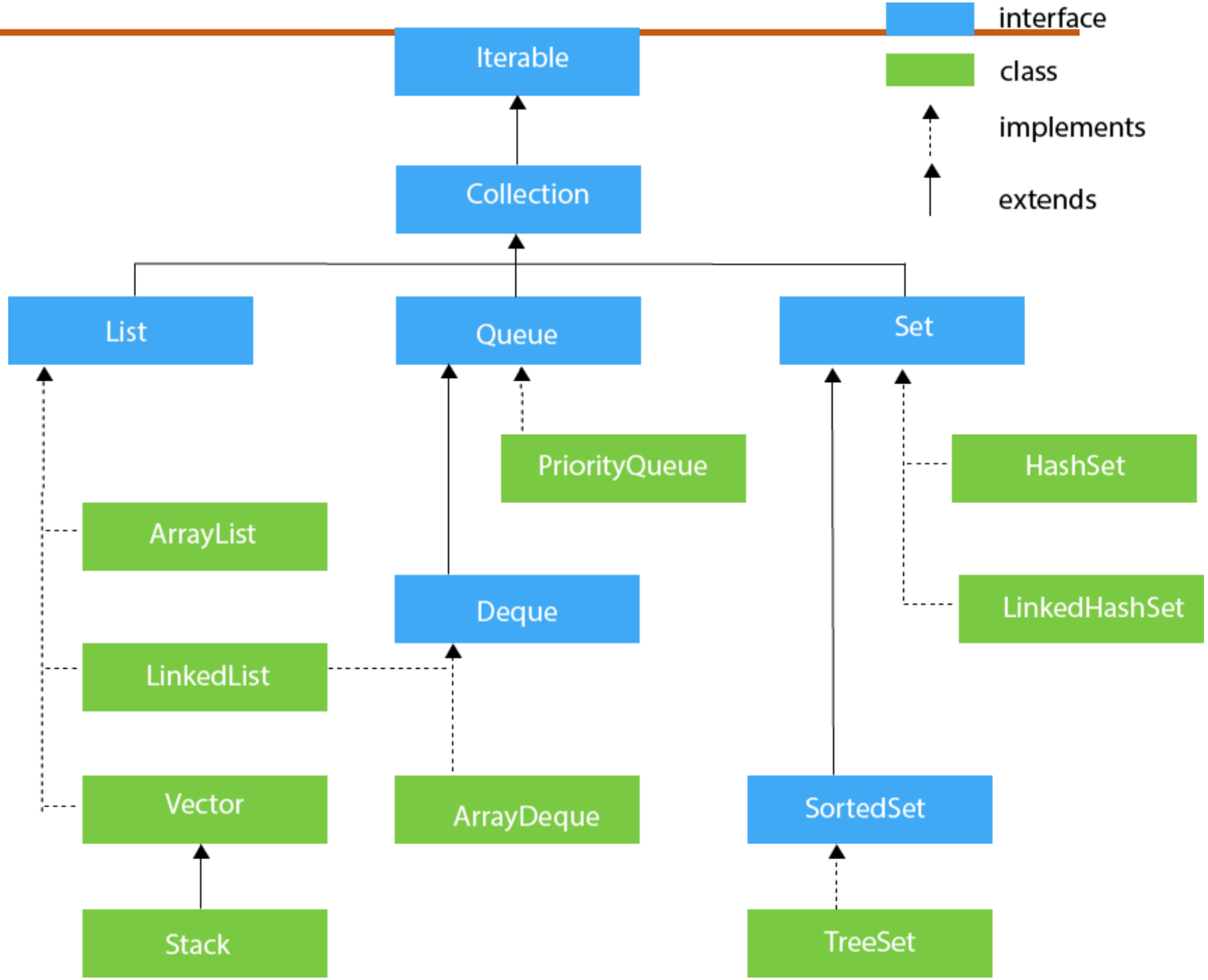
Collections framework. A *collection* is an object that represents a group of objects.

The primary advantages of a collections framework are that it:

- **Reduces programming effort** by providing data structures and algorithms so you don't have to write them yourself.
- **Increases performance** by providing high-performance implementations of data structures and algorithms. Because the various implementations of each interface are interchangeable, programs can be tuned by switching implementations.
- **Provides interoperability between unrelated APIs** by establishing a common language to pass collections back and forth.
- **Reduces the effort required to learn APIs** by requiring you to learn multiple ad hoc collection APIs.
- **Fosters software reuse** by providing a standard interface for collections and algorithms with which to manipulate them.

Object Oriented Analysis and Design

Collections - Introduction



Object Oriented Analysis and Design

Array - Introduction

- An Object of similar elements stored in contiguous memory allocation.
- Can be of any type
- Types: One Dimensional, Two Dimensional
- Declaration of 1D Array & Instantiation:
  - dataType[ ] arr; (or)
  - dataType [ ]arr; (or)
  - dataType arr[ ];

Instantiation:

arrayRefVar=**new** datatype[size];

- Declaration of 2D Array & Instantiation: :
  - dataType[ ][ ] arr; (or)
  - dataType [ ][ ]arr; (or)
  - dataType arr [ ][ ];

Instantiation:

int[][] arr=**new int**[3][3]; //3 rows and 3 columns

Array Representation

Representation of 1 Dimensional Array

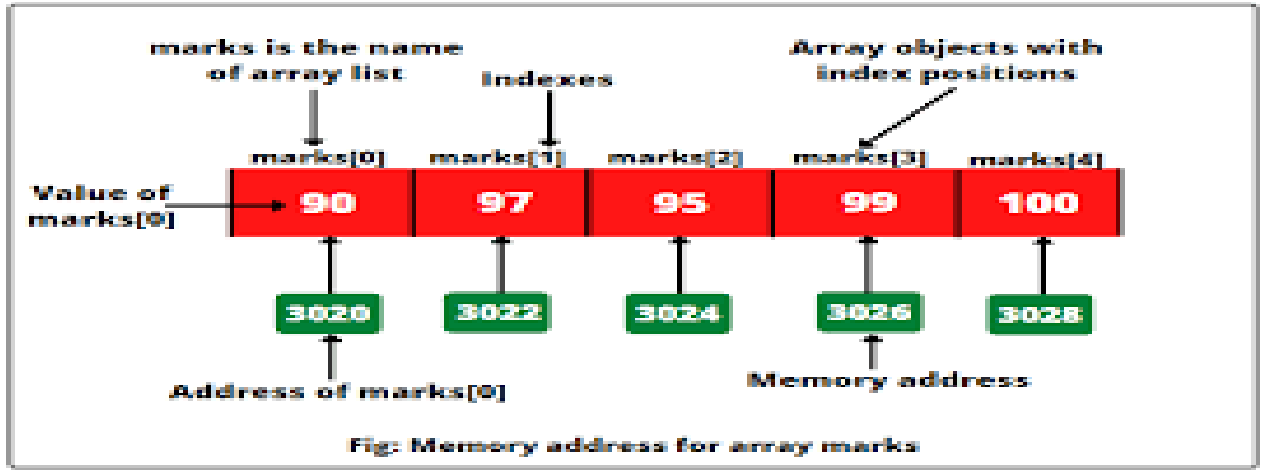


Fig: Memory address for array marks

Representation of 2 Dimensional Array

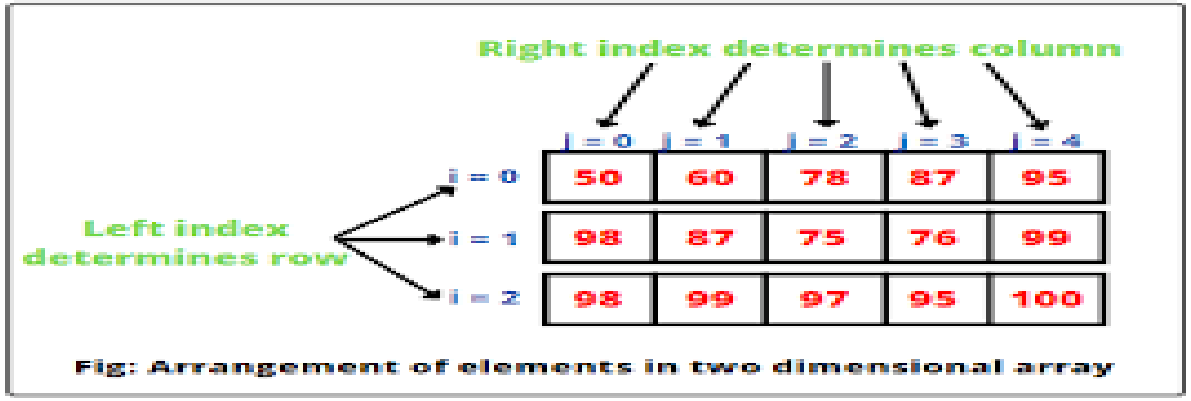


Fig: Arrangement of elements in two dimensional array

Foreach Loop

- Foreach loop prints the array elements one by one.
- It holds an array element in a variable, then executes the body of the loop.
- The syntax of the for-each loop is given below:
  - **for**(data_type variable:array){ …}.

Object Oriented Analysis and Design

Arrays Class in Java

- Arrays class in java.util package is a part of the Java Collection Framework.
- It provides static methods to dynamically create and access Java arrays.
- It consists of only static methods and the methods of Object class. The methods of this class can be used by the class name itself.
- The class hierarchy is as follows:
    - java.lang.Object
    - ↳ java.util.Arrays.

Usage :

- Syntax: Class declaration
- public class Arrays extends Object
- Syntax: In order to use Arrays : Arrays.<function name>;

Functions:

- binarySearch(), asList(),toString(),equals(),sort().

Object Oriented Analysis and Design

Introduction to List Interface in Java

- *List* in Java provides the facility to maintain the *ordered collection.*
- It contains the index-based methods to insert, update, delete and search the elements. It can have the duplicate elements also. We can also store the null elements in the list.
- List interface is found in the java.util package and inherits the Collection interface.
- The implementation classes of List interface are ArrayList, LinkedList, Stack and Vector.

Declarations: public interface List<E> extends Collection<E>

Few Methods :

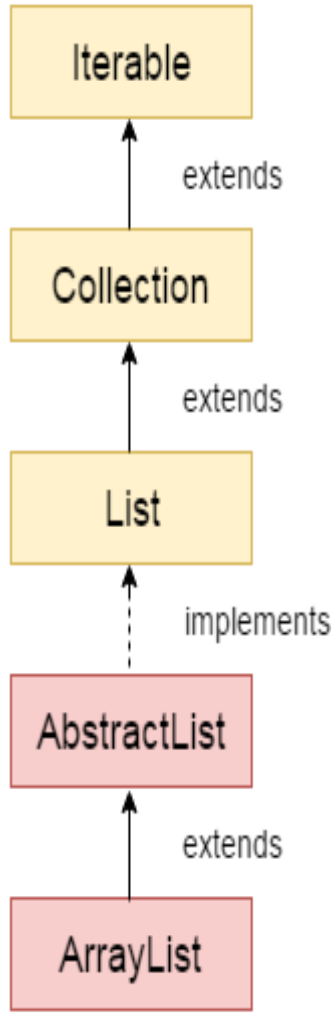
- size(), clear(), add(), Add(),addAll(), contains(), containsAll(), equals(), hashCode(), isEmpty(), indexOf(). etc...

Example : Programming Demo

Introduction to ArrayList Class in Java

- **ArrayList** class uses a ***dynamic array*** for storing the elements.
- No size limit.
- Can add or remove elements anytime.
- Found in java.util package.
- ArrayList in Java can have the duplicate elements also.
- It implements the List interface, all the methods of the List interface can be used.
- Java ArrayList class maintains insertion order.
- We can not create an array list of the primitive types, such as int, float, char, etc. It is required to use the required wrapper class in such cases.
  - **For example:**

1. ArrayList<int> al = ArrayList<int>(); // does not work
2. ArrayList<Integer> al = new ArrayList<Integer>(); // works fine



Introduction to ArrayList Class in Java

Declaration :

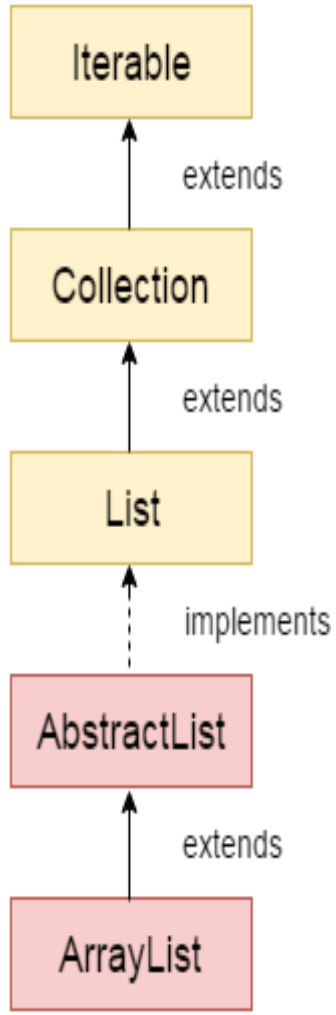
public class ArrayList<E> extends AbstractList<E> implements List<E>, RandomAccess, Cloneable, Serializable

Example for creating generic ArrayList:

ArrayList<String> list=new ArrayList<String>()

Few Methods()

add(), addAll(), clear(), clone(), contains(), remove(), removeAll(), replace(), replaceAll(), etc…



Object Oriented Analysis and Design

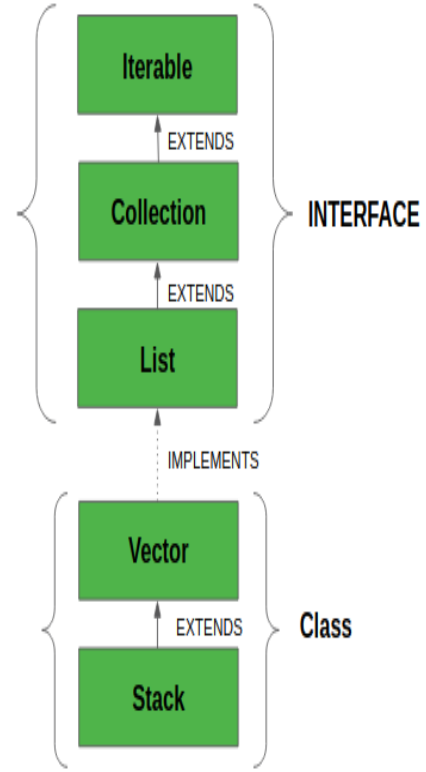
Introduction to Stack Class in Java

- Java Collection framework provides a Stack class that models and implements a **Stack data structure**.
- The class is based on the basic principle of last-in-first-out [LIFO].

Declaration:
public class Stack<E> extends Vector<E>

Implemented interfaces :

- **Serializable:** It is a marker interface that classes must implement if they are to be serialized and deserialized.
- **Cloneable:** This is an interface in Java which needs to be implemented by a class to allow its objects to be cloned.
- **Iterable<E>:** This interface represents a collection of objects which is iterable — meaning which can be iterated.
- **Collection<E>:** A Collection represents a group of objects known as its elements. The Collection interface is used to pass around collections of objects where maximum generality is desired.
- **List<E>:** The List interface provides a way to store the ordered collection. It is a child interface of Collection.
- **RandomAccess:** This is a marker interface used by List implementations to indicate that they support fast (generally constant time) random access.



Object Oriented Analysis and Design

Introduction to Stack Class in Java

- **creation of a stack class**
  - import **java.util.stack** package and use the Stack() constructor.

- **Few Methods:**
  - empty()
  - push(E item).
  - pop()
  - peek()
  - search(Object o)

PES UNIVERSITY
CELEBRATING 50 YEARS

DEMO

MCQ

1. **What is a collection in Java?**
   A. A primitive data type
   B. A class for storing numbers
   C. An object that represents a group of objects
   D. A memory allocation technique
2. **How does the Collections Framework improve performance?**
   A. By using contiguous memory
   B. By compiler optimizations
   C. By providing high-performance implementations
   D. By removing algorithms

MCQ

3. Which is a valid declaration of a one-dimensional array?

A. dataType arr{}
B. dataType[] arr
C. arr[dataType]
D. array dataType

4. Which class provides utility methods for arrays and belongs to java.util?

A. ArrayList
B. Collections
C. Arrays
D. Objects

MCQ

5. **What feature does the List interface provide?**
   A. Unordered collection
   B. Index-based access
   C. Fixed size
   D. No duplicates
6. **Which classes implement the List interface?**
   A. HashSet and TreeSet
   B. ArrayList, LinkedList, Stack, Vector
   C. Queue and Deque
   D. Map and HashMap

MCQ SOLUTION

1.C
2.C
3.B
4.C
5.B
6.B

THANK YOU

Prof Shridevi A Sawant
Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

OO Development Process, System Design and Frameworks

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements:

Prof. Mahitha G & Bhargavi M

Department of Computer Science and Engineering

Agenda

- OO Development Process
- OO Methodology
- Stages of OO methodology
- System Design
- Decisions – System Architecture
- Estimating performance
- Reuse Plan
- Libraries
- Patterns
- Frameworks

Object Oriented Development Process

- The essence is the **identification and organization of application concepts rather than their final representation in programming language.**
- Encourages software developers **to work and think in terms of application throughout the software life cycle**.
- Conceptual process independent of the programming language until final stage
- A way of thinking and it's greatest benefit comes from helping developers, customers express abstract concepts clearly and communicate them to each other.
- The OO concepts are used to express a design and also provide documentation

Object Oriented Methodology

- Emphasis how to go about developing a system or application
- **Encourages and facilitates re-use of software components**.
- It employs **international standard Unified Modeling Language (UML) from the Object Management Group (OMG).**
- A system can be developed on a component basis, which enables the effective re-use of existing components, it facilitates the sharing of its other system components
- Asks the analyst to determine what the objects of the system are?, What responsibilities and relationships an object has to do with the other objects? and how they behave over time?

Stages of Object Oriented Methodology

- **System conception :** Software development begins with business analysis or users conceiving an application and formulating tentative Requirements.
- **Analysis** : The analyst must work with the requestor to understand the problem, because problem statements are rarely complete or correct.

  The analysis model is a precise abstraction of **what the desired system must do**, **not how it will be done.** It should not contain implementation decisions.

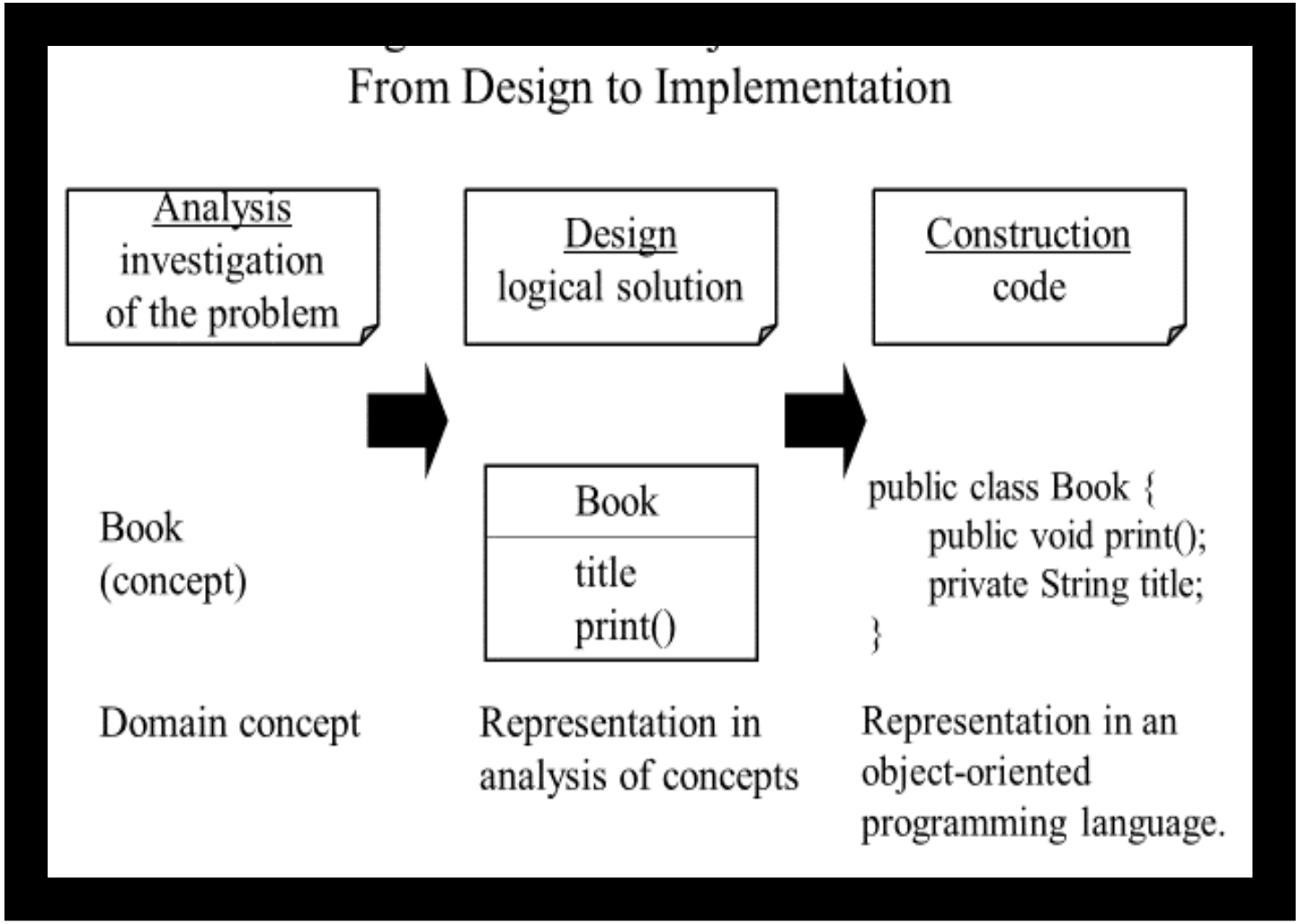
  Analyst must figure out the big picture of the entire application – for whom the application is to be developed, what problems the application will solve, when, where and why it will be needed.

  Analyst must also try and figure out the workflow of the application.

Stages of Object Oriented Methodology contd..

- **System design**: The development teams devise a **high – level strategy called the system architecture** for solving the application problem.
- **Class design** : The class designer adds details to the analysis model in accordance with the system design strategy. The focus of class design is the **data structures and algorithms** needed to implement each class.
- **Implementation** : Implementers **translate the classes and relationships** developed during class design into **particular programming language, database or hardware**. During implementation, it is important to follow **good software engineering practice** so that traceability to the design is apparent and so that the system remains **flexible and extensible.**

Sample


From Design to Implementation
Analysis
investigation
of the problem
Design
logical solution
Construction
code
Book
(concept)
Book
title
print()
public class Book {
public void print();
private String title;
}
Domain concept
Representation in
analysis of concepts
Representation in an
object-oriented
programming language.

System Design

- The first design stage for devising the basic approach to solve the problem.
- Developers make decisions about how the problem will be solved, first at a high level and then with more details – Overall **structure and style.**
- Need to apply **high level strategy – System Architecture** – for solving the problem and building a solution. It determines the **organization of the system into subsystems**. Also provides **the context for the detailed decisions** that are made in later stages.

Decisions - System Design

1. Estimating performance
2. Making a Reuse plan
3. Breaking system into sub-systems
4. Identifying Concurrency
5. Allocation of Sub Systems to hardware
6. Manage data storage
7. Handling global resources
8. Choosing a software control strategy
9. Handling boundary conditions
10. Set trade-off priorities
11. Select an architectural Style

System Design - Estimating performance

- Preparing a rough performance estimate – "**back of the envelope"** calculation
- Purpose is to **determine if the system is feasible** rather than achieving high accuracy
- Involves simplifying assumptions (i.e., assume factors).
- Don't worry about details – just approximate, estimate and guess
- Example: ATM Network
  - Bank has 40 branches and no. of terminals.
  - On a busy day, half the terminals are busy at once.
  - Suppose each customer takes one min to perform a session and most transactions involve a single transaction. i.e., 40 transactions a minute.
  - You can perform similar estimates for data storage.
  - Count the no. of customers, estimate amount of data for each one and multiply. i.e. 40 *20 =800.

System Design – Reuse Plan

- **Reuse –** Advantage of OO but it does not happen automatically.
- Two different aspects of reuse - **Using existing things and Creating reusable things.**
- Much easier to reuse existing things than to design new things.
- Most developers reuse existing things and a small fraction of developers create new things.
- Reusable things include **Models, Libraries, Frameworks and Patterns.**

Reuse Plan - Libraries

PES UNIVERSITY
CELEBRATING 50 YEARS

- Library is a collection of classes that are useful in many contexts.
- Classes must be carefully organized, so that users can find them.
- Classes must have accurate and thorough description to help users determine their relevance.
- Several qualities of good class libraries:

  Coherence: Organized about a few well focused themes;

  Completeness: Provide complete behavior for the chosen theme;

  Consistency: Consistent names and signatures for polymorphic operations across classes;

  Efficiency: Provide alternative implementations of algorithms that trade time and space;

  Extensibility: Must be able to define subclasses for library classes;

  Genericity: Should use parameterized class definitions where appropriate.

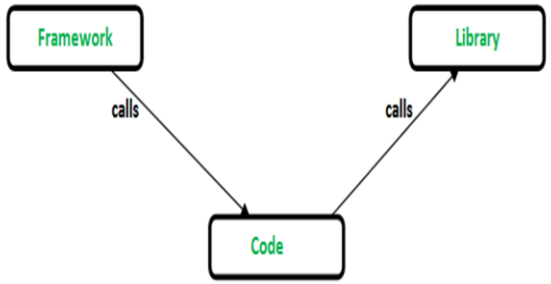
- Performs a set of specific and well-defined operations. Examples :
- Network protocols, compression, image manipulation, string utilities, regular expression etc.

Reuse Plan - Patterns

- A **pattern** is a best practice, a recipe, a time tested solution to a recurring problem that's proved to work and to be the best one among to choose.
- There are patterns for **analysis, architecture, design** and **implementation**.
- A *pattern* comes with **guidelines on when to use it,** as well as **trade-offs on its use.**
- Typically a small number of classes and relationships.
- Advantage is that pattern has been carefully considered by others and has already been applied to past problems successfully.
- **Software Architecture Patterns**: Layered Pattern, Client-Server Pattern
- **Design Patterns**: Gang of Four(GOF). Broken into three categories – **Creational Patterns** for the creation of objects, **Structural Patterns** to provide relationships between objects and **Behavioral Patterns** to help define how objects interact with each other.

Reuse Plan - Frameworks

- Frameworks provide a skeletal structure or ready-made architecture for our application.
- This skeletal structure that is provided must be elaborated to build complete application.
- This elaboration consists of specializing abstract classes with behavior specific to an individual application.
- Consists of more than just the classes involved and includes a paradigm for flow of control and shared variants.



Library Vs Framework

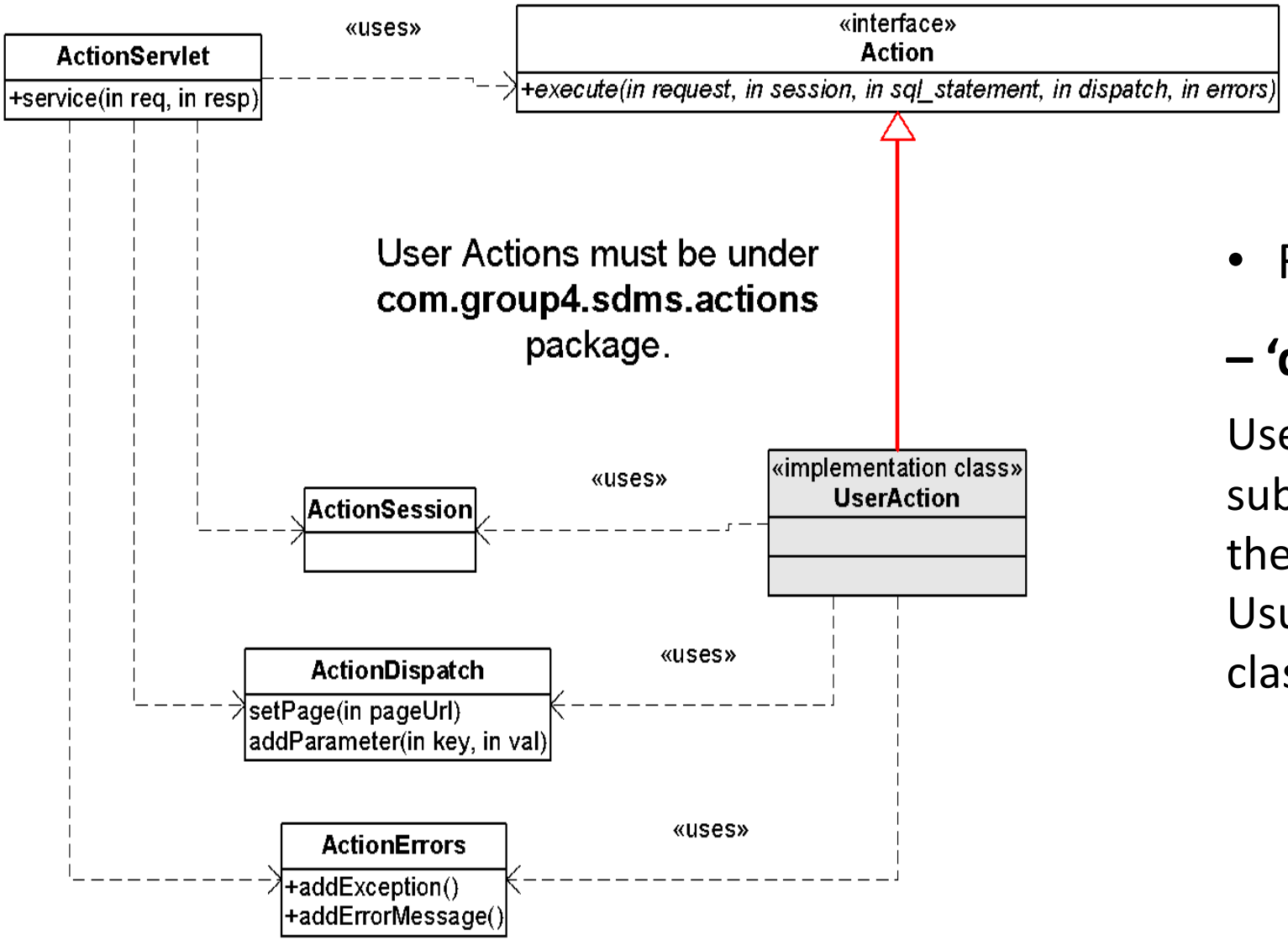
Frameworks in detail

- Pre-written code used by Java developers to develop Java applications or web applications.
- A set of cooperating classes that make up a reusable design for a specific class of software.
- It acts like a skeleton that helps the developer to develop an application by writing their own - The framework is in **control of the programmer**.
- Provides architectural guidance by **partitioning the design into abstract classes.**
- A developer will normally customize a framework to a specific application by “subclassing” and composing instances of framework classes.
- In libraries – you call the library functions. However, in frameworks – the framework handles the flow of control and calls your code whenever required.

Frameworks in Java

- **Collections:** Provides many interfaces (Set, List, Queue, Deque) and classes (ArrayList, Vector, LinkedList, PriorityQueue, HashSet, LinkedHashSet, TreeSet).
- **Swing:** The part of JFC (Java Foundation Classes) built on the top of AWT and written entirely in Java. The javax.swing API provides all the component classes like JButton, JTextField, JCheckbox, JMenu, etc.
- **AWT:** An abstract window toolkit that provides various component classes like Label, Button, TextField, etc., to show window components on the screen. All these classes are part of the Java.awt package.
- **Spring, Hibernate, Grails, Play, JavaServer Faces (JSF), Google Web Toolkit (GWT), Quarkus**

Framework Example



- Relies on the **'Hollywood Principle'**

– 'don't call us, we'll call you."

User-defined classes (for example new subclasses) will receive messages from the predefined framework classes. Usually handled by implementing super class abstract methods.

- Frameworks **emphasize design reuse over code reuse**

References

- Object - Oriented Modeling and Design With UML by RUMBAUGH and BLAHA, Chapter 1 and 14
- Applying UML and Patterns by Craig Larman, Chapter-34
- https://www.javatpoint.com/what-is-framework-in-java
- What is Object-Oriented Modeling (OOM)? - Definition from Techopedia
- Object oriented methodology (beingintelligent.com)
- 10 of the Most Popular Java Frameworks of 2020 (stackify.com)

MCQ

1. **What is the primary focus of the analysis model?**
   A. How the system will be implemented
   B. Performance optimization
   C. What the desired system must do
   D. Database design

2. **Which stage involves understanding incomplete or incorrect problem statements?**
   A. System design
   B. Implementation
   C. Analysis
   D. Class design

MCQ

3. **What is the main goal of system design?**
   A. Writing source code
   B. Creating test cases
   C. Devising a high-level system architecture
   D. Drawing UML class diagrams
4. **What is the focus of class design?**
   A. User requirements
   B. Data structures and algorithms
   C. Business workflow
   D. System feasibility

MCQ SOLUTION

1.C
2.C
3.C
4.B

THANK YOU

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

PES UNIVERSITY

UE23CS352B: Object Oriented Analysis and Design

Architectural Patterns

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements:

Prof. Mahitha G & Bhargavi M

Department of Computer Science and Engineering

Architectural Patterns - Agenda

- Introduction – Architectural Patterns
- Model View Controller [MVC]
- How MVC Works?
- MVC Architecture
- Modifying the MVC Design
- Advantages of MVC
- MVC in Java
- Implementation

Introduction to Architectural patterns

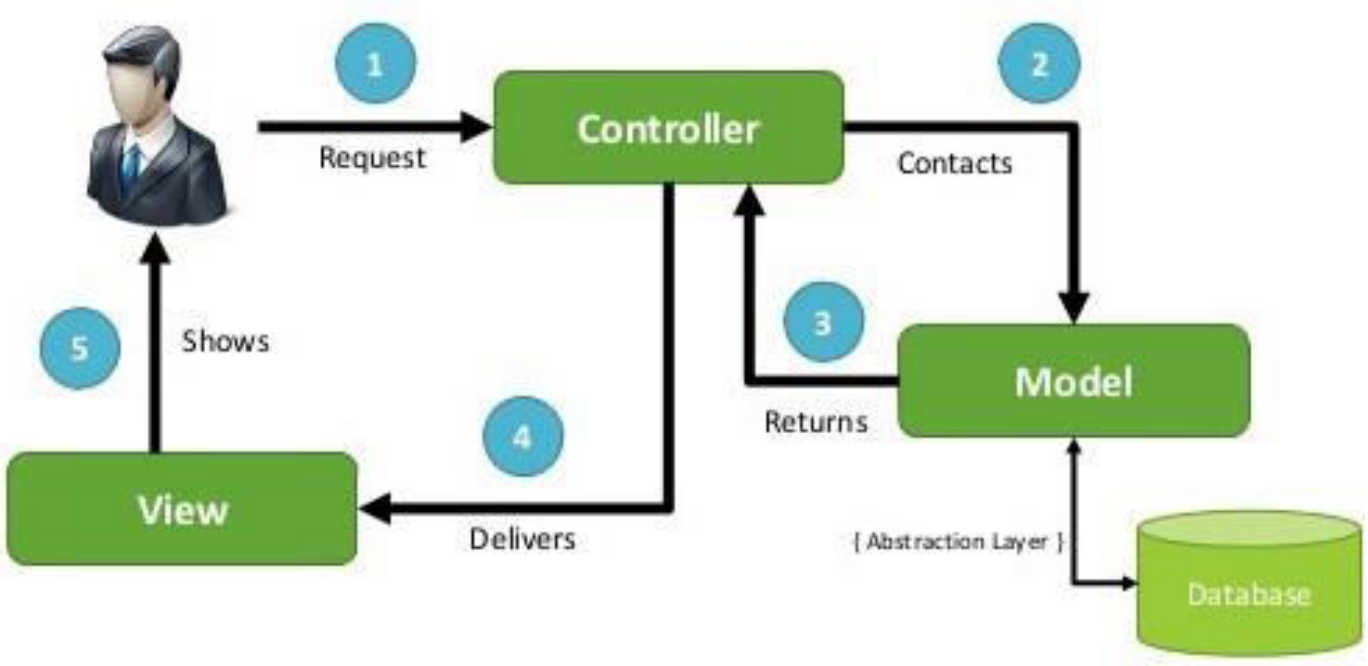
- An architectural pattern is a **general**, **reusable** solution to a commonly occurring problem in software architecture
- Helps define the basic characteristics and behavior of an application
- Some architecture patterns naturally lend themselves toward highly scalable applications, whereas other architecture patterns naturally lend themselves toward applications that are highly agile.
- Knowing the characteristics, strengths, and weaknesses of each architecture pattern is necessary in order to choose the one that meets the specific business needs and goals.
- Addresses various issues in software engineering, such as **computer hardware performance limitations, high availability and minimization of a business risk.**
- Layered, event driven, microkernel, microservices, space based and MVC, etc.

Model View Controller

- MVC - An architectural pattern in Software Engineering.
- First introduced by **Trygve Reenskaug, a Smalltalk developer at the Xerox Palo Alto Research Center in 1979 and** helps to decouple data access and business logic from the manner in which it is displayed to the user.
- A way of designing and building applications that separates application logic from presentation
- Division of application into three main logical components: **model**, **view**, and **controller.**
- A design pattern for computer software considered to distinguish between **the data model, processing control and the user interface.**
- It neatly separates the **graphical interface displayed to the user** from the **code that manages the user actions.**
- Completely separates the calculations and interface from each other

How MVC Works?

- Whenever the controller receives a request from the user (either directly or via the view), it puts the model to work. And when the model delivers the data requested in the right format, the controller forwards it to the view

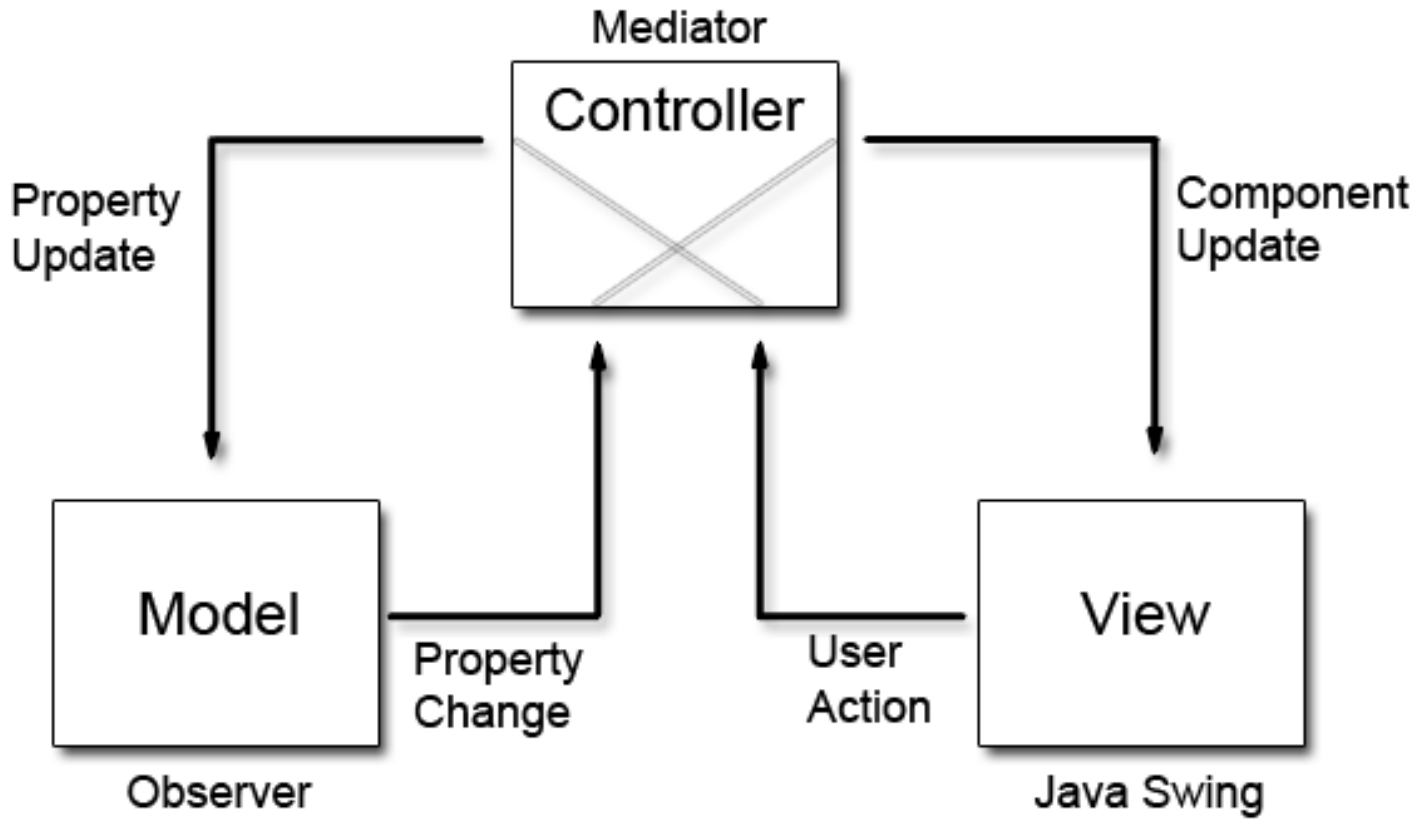


Model View Controller architecture

- Well-known design pattern in the web development field. It is a way to organize the code.
- Consists of **Data model, presentation information and control information.**
  - **Model:** The model represents data and the rules that govern access to and updates of this data. In enterprise software, a model often serves as a **software approximation of a real-world process**.
  - **View:** Renders the contents of a model. It specifies exactly how the model data should be presented. If the model data changes, the view must update its presentation as needed. This can be achieved by using a *push model*, in which the view registers itself with the model for change notifications, or a *pull model*, in which the view is responsible for calling the model when it needs to retrieve the most current data. Represents the presentation layer of application. It is used to visualize the data that the model contains.
  - **Controller:** The controller translates the user's interactions with the view into actions that the model will perform. In a stand-alone GUI client, user interactions could be button clicks or menu selections, whereas in an enterprise web application, they appear as GET and POST HTTP requests. Depending on the context, a controller may also select a new view -- for example, a web page of results -- to present back to the user
- The MVC pattern needs all these components to be separated as **different objects.**

Modifying the MVC Design

Program Demo: MVC Employee



Advantages of MVC

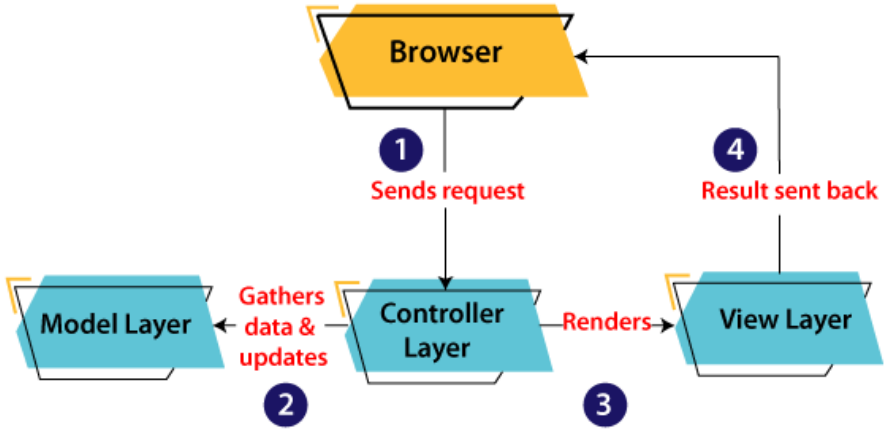
- Faster Development Process
- Model can be reused with multiple views
- Less dependency among components – loose coupling
- The modification does not affect the entire model
- Application becomes more understandable
- Not a single massive codebase

Model View Controller architecture in Java

- The Model contains the simple Java classes,

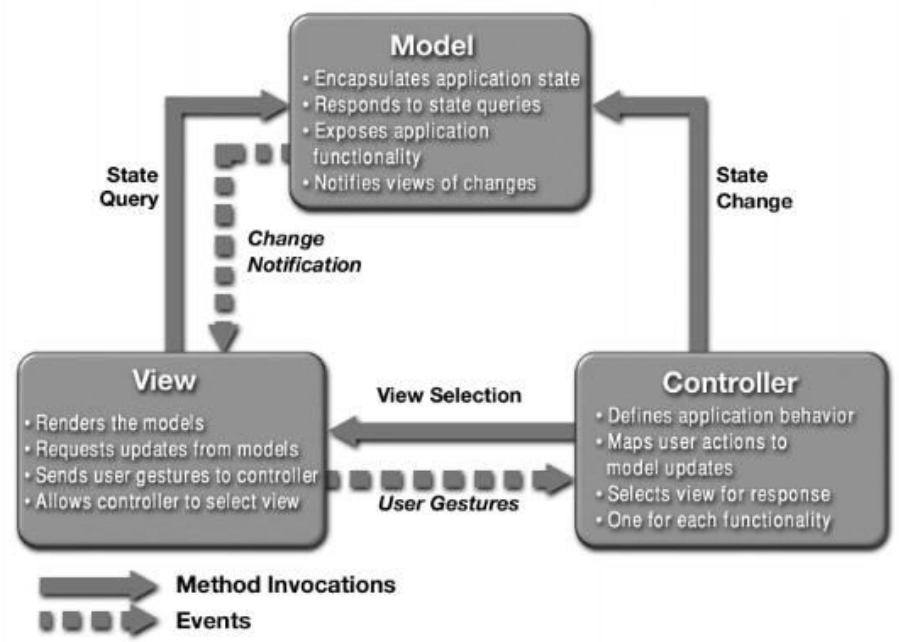
  The View used to display the data

  The Controller contains the servlets/ all entities.

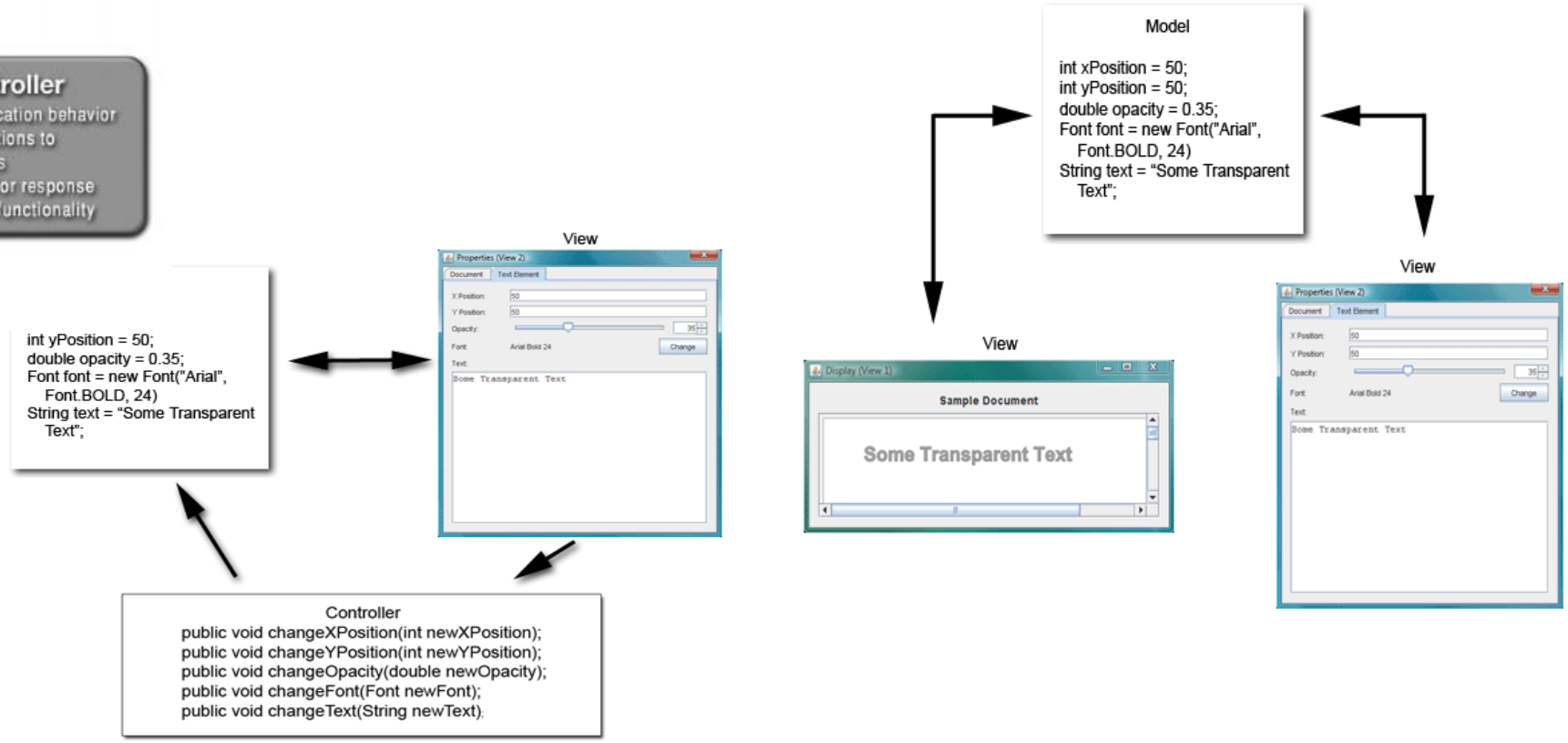


- Due to this separation the user requests are processed as follows:
  - A client (browser) sends a request to the controller on the server side, for a page.
  - The controller then calls the model. It gathers the requested data.
  - Then the controller transfers the data retrieved to the view layer.
  - Now the result is sent back to the browser (client) by the view

Model View Controller Implementation



- One of the consequences of this powerful design is that the many views can have the same underlying model.



Implementation of MVC in Java

- **Employee Class, will act as model layer:**

  It represents the business logic for application and also the state of application. The model object fetch and store the model state in the database. Using the model layer, rules are applied to the data that represents the concepts of application.

- **EmployeeView Class, will act as a view layer:**

  View represents the visualization of data received from the model. The view layer consists of output of application or user interface. It sends the requested data to the client, that is fetched from model layer by controller

- **EmployeeContoller Class, will act a controller layer:**

  The controller layer gets the user requests from the view layer and processes them, with the necessary validations. It acts as an interface between Model and View. The requests are then sent to model for data processing. Once they are processed, the data is sent back to the controller and then displayed on the view.

References

- Difference Between Architectural Style, Architectural Patterns and Design Patterns – GeeksforGeeks
- Software Architecture Patterns (oreilly.com)
- MVC Architecture in Java – Javatpoint
- Java SE Application Design With MVC (oracle.com)
- Advantages of using MVC model for effective web application development (brainvire.com)
- MVC Architecture in Java: How to implement MVC in Java? Edureka

PES UNIVERSITY
CELEBRATING 50 YEARS

MCQ

1. **What is the primary goal of the MVC pattern?**
   A. Improve execution speed
   B. Separate application logic from presentation
   C. Reduce memory usage
   D. Avoid inheritance
2. **What is the responsibility of the Model in MVC?**
   A. Handle user requests
   B. Render the user interface
   C. Represent data and business rules
   D. Control navigation

MCQ

3. **What happens when the controller receives a user request?**
   A. It directly updates the view
   B. It ignores the model
   C. It terminates the request
   D. It puts the model to work and forwards data to the view
4. **Which is an advantage of the MVC pattern?**
   A. Tight coupling
   B. Single massive codebase
   C. Reuse of model with multiple views
   D. Dependency on presentation layer
5. **In the MVC request flow, which component sends the final response back to the client (browser)?**
   A. Model
   B. Controller
   C. View
   D. Client itself

MCQ SOLUTION

1.B
2.C
3.D
4.C
5.C

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

UE23CS352B - Object Oriented Analysis and Design

GRASP

Prof. Shridevi A Sawant
Dept. of Computer Science and Engineering

Acknowledgements:
Dr. J. MANNAR MANNAN
Dept. of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

Agenda

- Introduction to GRASP
- Responsibility
- Types of Responsibility
- References
- GRASP Design Principles

Introduction - GRASP

- Stands for **General Responsibility Assignment Software Patterns**
- GRASP - Consists of **guidelines for assigning responsibility to classes and objects in object-oriented design**.
- **GRASP with object-oriented programming classifies problems and their solutions together as patterns. With these problems and solutions well defined, they can be applied in other similar instances**.
- First published by **Craig Larman in 1977, with** set of "**nine fundamental principles in object design and responsibility**"
- **GRASP is occasionally coupled with other design patterns such as SOLID**

Responsibility

- **A contract / obligation that a class / module / component** must accomplish
- Responsibility can be:

* Accomplished by a single object
* A group of object collaboratively accomplish a responsibility.

- Identify the objects and responsibilities from the problem domain, and also identify how objects interact with each other. Helps us in deciding which responsibility should be assigned to which object/class.
- **Define blueprint for those objects** – i.e., class with methods implementing those responsibilities

Responsibility contd..

- Design of software objects and larger scale components is in terms of **responsibilities, roles, and collaborations.** - RDD – Responsibility Driven Design

  RDD – **An abstraction of what they do**

  UML - **A contract or obligation of a classifier**

- Related to the obligations or behavior of an object in terms of its role
- **Assigned to classes of objects during object design**.

Example: I may declare that "a Sale is responsible for creating SalesLineItems"

- The **translation of responsibilities into classes and methods** is influenced by **the granularity of the responsibility**

Types of Responsibility

- **Doing:**

Doing something itself, such as creating an object or doing a calculation

Initiating action in other objects

Controlling and coordinating activities in other objects

- **Knowing:**

Knowing about private encapsulated data

Knowing about related objects

Knowing about things it can derive or calculate

GRASP Design principles

Different patterns/principles used are

- **Creator**
- **Information expert**
- **Low coupling**
- **Controller**
- **High cohesion**
- **Polymorphism**
- **Indirection**
- **Protected variations**
- **Pure fabrication**

Design principle (Larman: "pattern") – An attempt at a methodical way to do object design

Creator

- Who creates an Object? Or who should create a new instance of some class?
- Assign the responsibility of creating instances of a class to the class that has the most knowledge about when and how to create them
- "Container" object creates "contained" objects.
- Decide who can be creator based on the object's association and their interaction
- **Example:** Who creates the square object in monopoly game?

Creator

- **Identify Object Creation Scenarios**: Determine which classes are responsible for creating instances of other classes within the system.
- **Delegate Object Creation:** Assign responsibility for creating objects to classes that are best suited for the task.
- **Encapsulate Initialization Logic:** Ensure that the classes responsible for creating objects encapsulate the initialization logic within them.
- **Promote Loose Coupling:** Aim to minimize dependencies between classes by decoupling object creation logic from the classes that use the created objects.
- **Consider Dependency Inversion:** When designing classes that create objects, consider using dependency inversion principles to invert dependencies and promote abstraction.

Creator

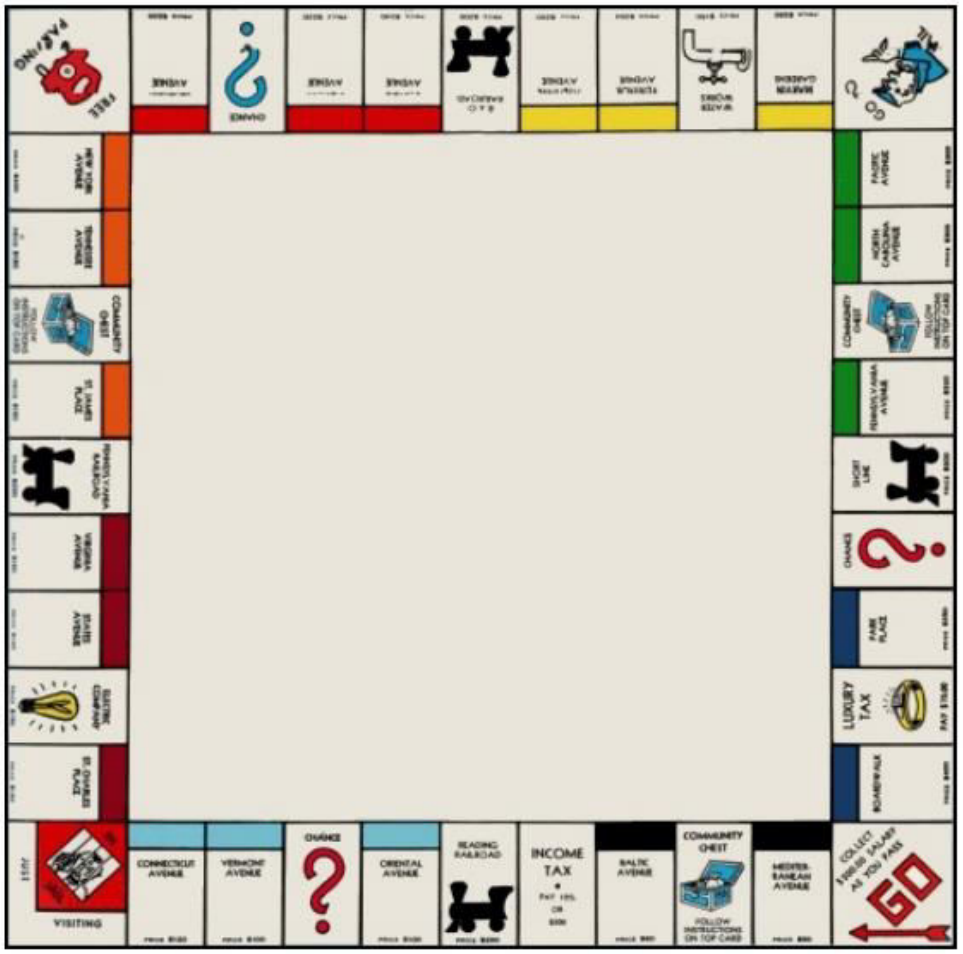
```
public class OrderService {

private final OrderRepository orderRepository;

public OrderService(OrderRepository orderRepository) {
        this.orderRepository = orderRepository;
}

public void createOrder(Customer customer, List<Product> products) {
        // Create a new order object
        Order order = new Order(customer, products);

        // Save the order to the repository
        orderRepository.saveOrder(order);
}
}
```

Creator

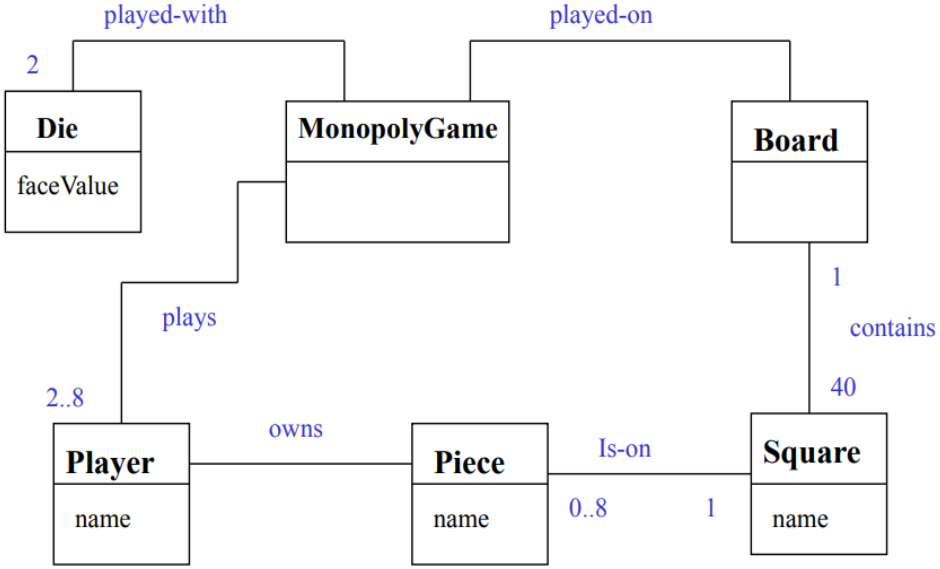
- In this example, the **OrderService** class is responsible for creating instances of **Order** objects.
- It encapsulates the initialization logic for creating orders and delegates the task of persisting orders to an **OrderRepository** class.
- By centralizing object creation logic within the **OrderService** class, we promote encapsulation and loose coupling in the system.

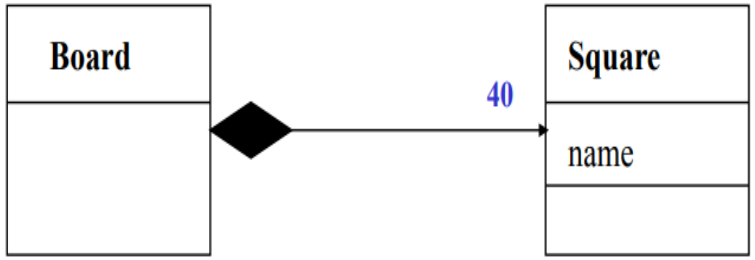
Object Oriented Analysis and Design

Monopoly game

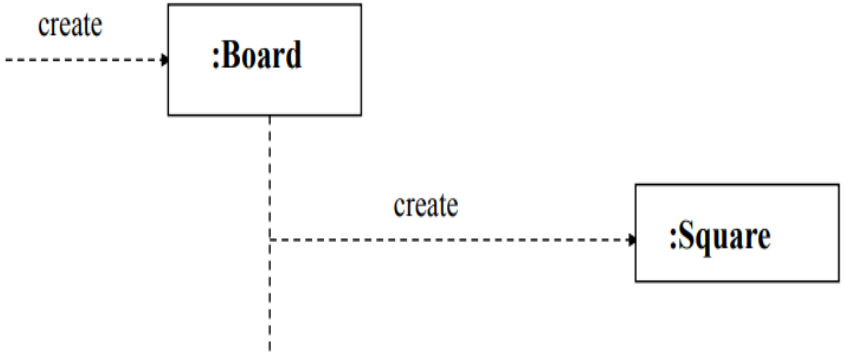


Creator – Monopoly example





Applying the Creator pattern in Static Model



Dynamic Model – illustrates Creator Pattern

Creator

Name:	**Creator**
Problem:	Who creates an A?
Solution: (this can be viewed as advice)	Assign class B the responsibility to create an instance of class A if one of these is true (the more the better): ■ B "contains" or compositely aggregates A. ■ B records A. ■ B closely uses A. ■ B has the initializing data for A.

- Start with domain model.
- Since a Sale contains SalesLineItem objects, it should be responsible according to the Creator pattern
  - Note: There is a related design pattern called **Factory pattern** for more complex creation situations

Information Expert

- Problem: What is a basic principle by which to assign responsibilities to objects?
- Solution: Assign a responsibility to the class that has the information needed to fulfill it.
- "**objects do things related to the information they have**"
- Information necessary may be spread across several classes => **objects interact via messages**
- **Definition:**

  Name: Information Expert

  Problem: What is the basic principle by which to assign responsibilities to object

  Solution: Assign a responsibility to the class that has the information needed to respond to it.
- Just because an object has information necessary doesn't mean it will have responsibility for action related to the information

Information Expert

- They have all the information needed to perform operations, or in some cases they collaborate with others to fulfill their responsibilities.
- **Example:** (from POS system) Who should be responsible for knowing the grand total of a sale?

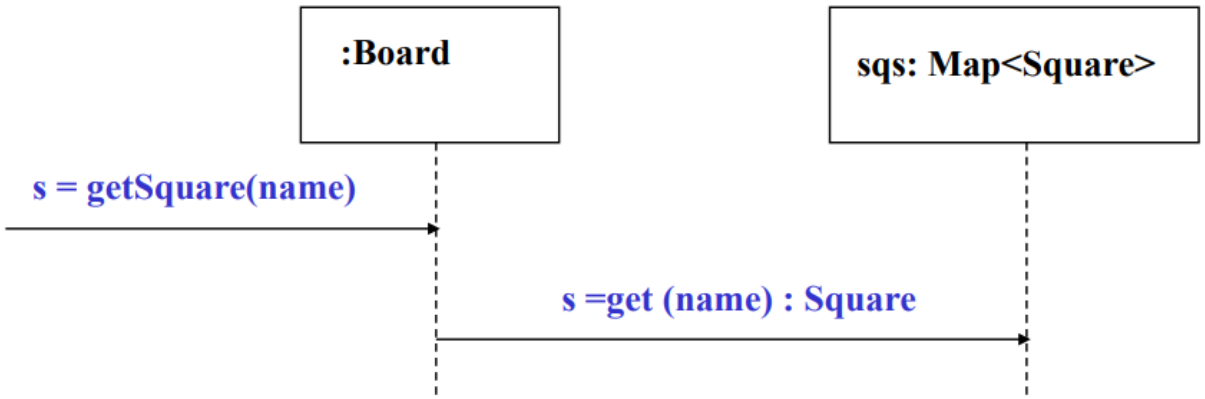
- Start with domain model and look for associations with Sale
- What information is necessary to calculate grand total and which objects have the information?

Information Expert - Summary

Design Class	Responsibility
Sale	knows sale total
SalesLineItem	knows line-item subtotal
ProductDescription	knows product price

Information Expert –Monopoly game

- The player Marker needs to find the square to which it is to move and the options pertaining to that square.
- The Board aggregates all of the Squares, so the Board has the Information needed to fulfill this responsibility.



Make board information expert

Information Expert –Shopping Cart

```
public class ShoppingCart {

    private List<Item> items = new ArrayList<>();

    public void addItem(Item item) {
        items.add(item);
    }

    public double calculateTotalPrice() {
        double totalPrice = 0;

        for (Item item : items) {
            totalPrice += item.getPrice();
        }

        return totalPrice;
    }
}
```

```
public class Item {

    private String name;
    private double price;

    public Item(String name, double price) {
        this.name = name;
        this.price = price;
    }

    public double getPrice() {
        return price;
    }
}
```

- In this example, the "**Information Expert**" principle is applied by assigning responsibility for calculating the total price of items in the shopping cart to the **ShoppingCart** class.
- Since the **ShoppingCart** class has access to the list of items and their prices, it is the most suitable class to perform this calculation.
- The **Item** class, on the other hand, encapsulates information about individual items, such as their name and price.
- By delegating responsibilities based on expertise and information, we promote high cohesion and encapsulation in the design.

Low Coupling

- **Coupling** – Coupling describes the degree of dependency between classes or modules. It indicates how closely connected different classes or modules are, and how much they rely on each other. There are two main types of coupling
- **Tight Coupling:** High dependency between classes. Changes in one class often require changes in the coupled class. Tight coupling occurs when a class is highly dependent on the specifics of another class. This often happens when one class creates an instance of another class and directly accesses its methods and fields.
- **Loose Coupling:** Low dependency between classes. Classes interact through well-defined interfaces, reducing the impact of changes. Loose coupling occurs when classes are less dependent on the specifics of other classes. This can be achieved by using interfaces, abstract classes, or dependency injection, allowing classes to interact without knowing the implementation details of each other.

Low Coupling

- **Definition:**

Name: Low Coupling
Problem: How to support low dependency, low change impact, increased reuse?
Solution: Assign a responsibility so coupling is low.

- **Minimizes the dependency** hence making system **maintainable, efficient and code reusable**
- **Example: Inheritance, Composition, aggregation, association**

Low coupling is an evaluative pattern that dictates how to assign responsibilities to support

- lower dependency between the classes
- change in one class having a lower impact on other classes
- higher reuse potential

Low Coupling contd..

- A class with high coupling leads to problems:
  - Changes in related classes force local changes
  - Harder to understand in isolation
  - Harder to reuse
- Low Coupling – How can we reduce the impact of change in depended elements on dependent elements? **Assign responsibilities so that coupling remains low.**
- Two **elements are coupled**, if – One element has aggregation/composition association with another element. – One element implements/extends other element

Low Coupling contd..

Assign a responsibility so that coupling remains low.

A class, for example, with high (or strong) coupling relies on many other classes. Such classes may be undesirable; some suffer from the following problems:

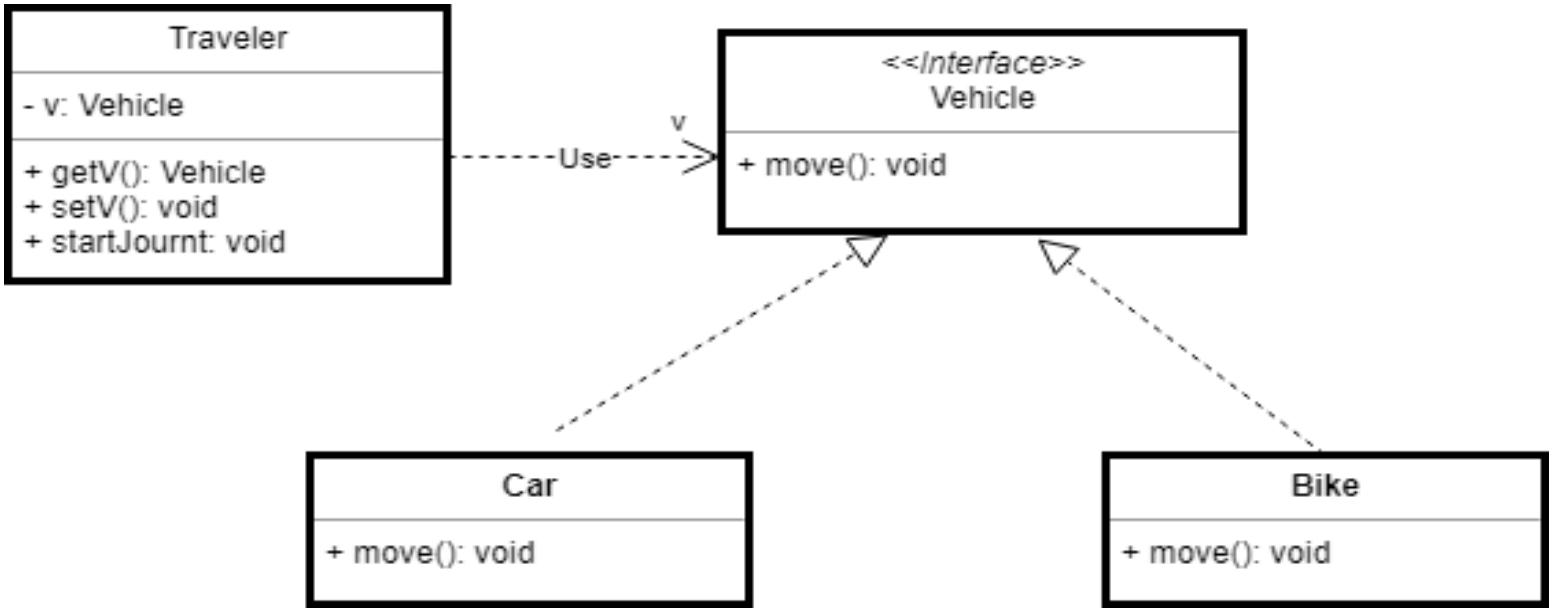
- Forced local changes because of changes in related classes.
- Harder to understand in isolation.
- Harder to reuse because its use requires the additional presence of the classes on which it is dependent.

Low Coupling contd..

Good to know below are some key points about low coupling

- Low dependencies between "artifacts" (classes, modules, components).
- There shouldn't be too much of dependency between the modules, even if there is a dependency it should be via the interfaces and should be minimal.
- Avoid tight-coupling for collaboration between two classes (if one class wants to call the logic of a second class, then first class needs an object of second class it means the first class creates an object of the second class).
- Strive for loosely coupled design between objects that interact.
- Inversion Of Control (IoC) / Dependency Injection (DI) - With DI objects are given their dependencies at creation time by some third party (i.e. Java EE CDI, Spring DI…) that coordinates each object in the system. Objects aren't expected to create or obtain their dependencies—dependencies are injected into the objects that need them. The key benefit of DI—loose coupling.

Low Coupling contd..



Low Coupling contd..

```
// Vehicle interface to allow loose coupling implementation
interface Vehicle {
    void move();
}

// Car class implements Vehicle interface
class Car implements Vehicle {
    @Override
    public void move() {
        System.out.println("Car is moving");
    }
}

// Bike class implements Vehicle interface
class Bike implements Vehicle {
    @Override
    public void move() {
        System.out.println("Bike is moving");
    }
}
```

Now create a Traveler class which holds the reference to the Vehicle interface.

```
class Traveler {
    private Vehicle v;

    public Vehicle getV() {
        return v;
    }
    public void setV(Vehicle v) {
        this.v = v;
    }
    public void startJourney() {
        v.move();
    }
}
```

Low Coupling contd..

Test class for loose coupling example - Traveler is an example of loose coupling

```
public static void main(String[] args) {
    Traveler traveler = new Traveler();
    traveler.setV(new Car()); // Inject Car dependency
    traveler.startJourney(); // start journey by Car
    traveler.setV(new Bike()); // Inject Bike dependency
    traveler.startJourney(); // Start journey by Bike
}
```

Low Coupling contd..

- **Example 2** (from POS system): Consider Payment, Register, Sale
- To create a Payment and associate it with a Sale, who is responsible?
- Register "records" a Payment. So, **Register is the creator**
- Register creates Payment *p* then sends *p* to a Sale => **coupling of Register class to Payment class**

Low Coupling contd..

- **Alternate approach:** Register requests Sale to create the Payment

Note:

- In both cases, Sale needs to know about a Payment
- However, a Register needs to know about a Payment in first but not in second
- **Second approach has lower coupling**

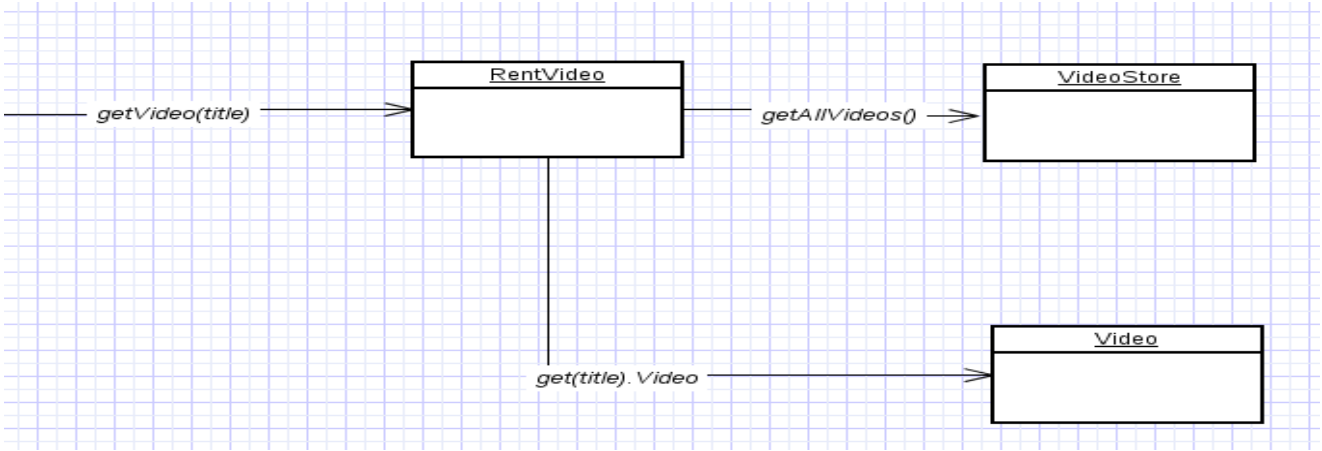
Low Coupling – More in terms of Java

- Low coupling is an evaluative principle, i.e., keep it in mind when evaluating designs
- Examples of coupling in Java (think "dependencies"):
  - TypeX has an attribute of TypeY
  - TypeX calls on services of a TypeY object
  - TypeX has a method that references an instance of TypeY
  - TypeX is a subclass of TypeY
  - TypeY is an interface and TypeX implements the interface

Note: subclassing => high coupling

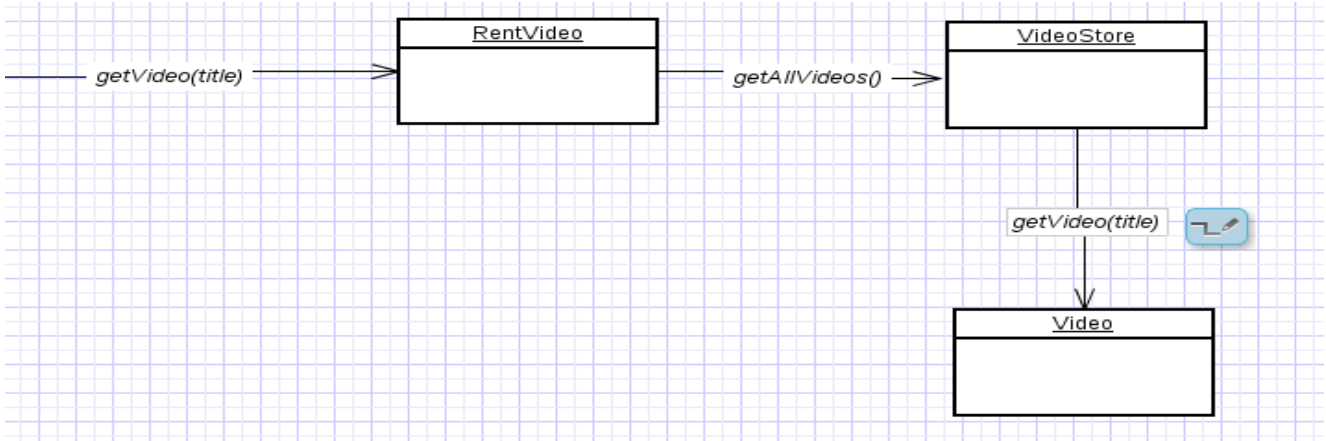
Example: Low Coupling

High Coupling



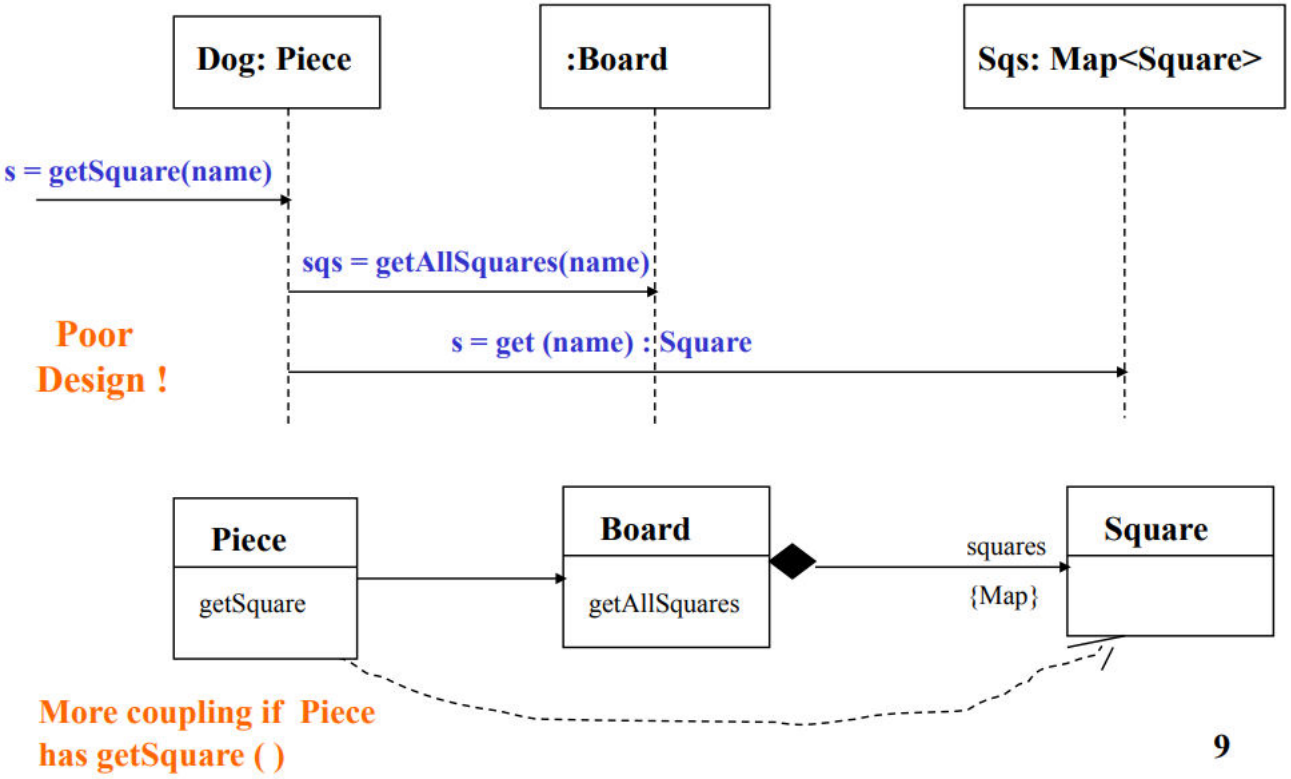
here class RentVideo knows about both VideoStore and Video objects. Rent is depending on both the classes.

Low Coupling



VideoStore and Video class are coupled, and Rent is coupled with VideoStore. **Thus, providing low coupling.**

Low Coupling – Monopoly example



Alternate Design: Bad solution

High cohesion

- **Cohesion** - A measure of how strongly related and focused the responsibilities of an element (class, subsystem, etc.)
- **Definition:**

Name: High Cohesion

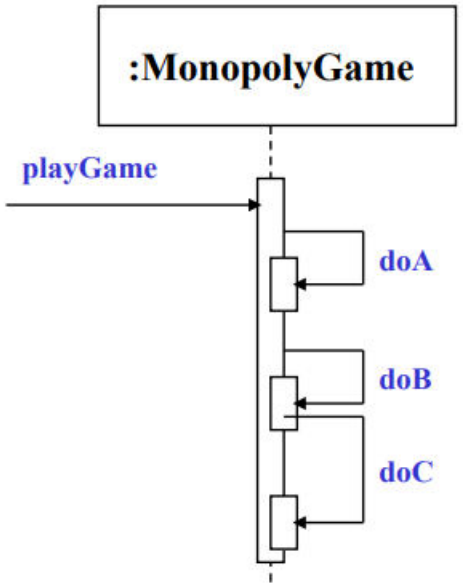
Problem: How to keep complexity manageable, understandable and support low coupling?

Solution: Assign the responsibility so that cohesion remains high.

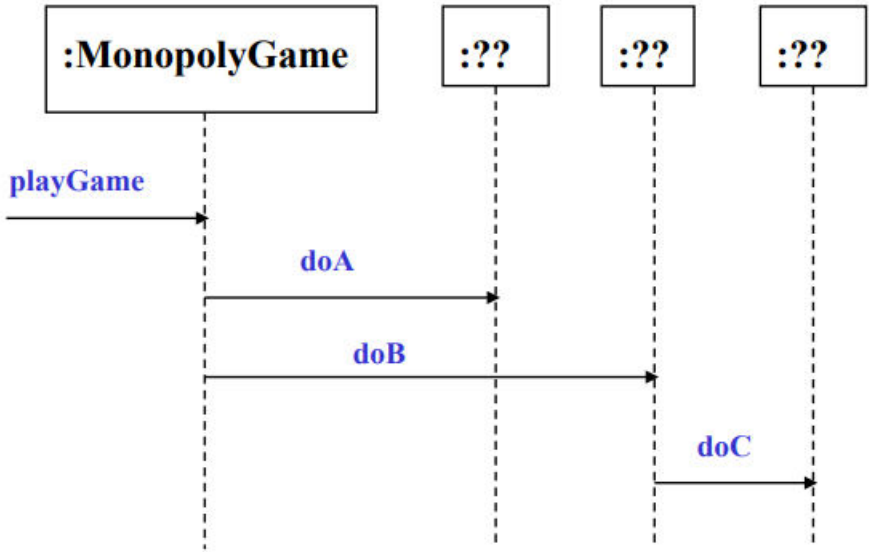
- Problems from low cohesion (does many unrelated things or does too much work):

Hard to understand/comprehend
Hard to reuse
Hard to maintain
Brittle – easily affected by change

High Cohesion - Monopoly example



Poor (low) Cohesion in the MonopolyGame object



Better Design

MCQ

1. GRASP stands for:

A) General Responsibility Assignment Software Patterns
B) General Rules for Application Software Programming
C) Generic Reusable Architecture System Principles
D) Global Responsibility Allocation Software Process

2. GRASP mainly provides guidelines for:

A) Database normalization
B) Assigning responsibilities to classes and objects
C) Writing algorithms
D) Network communication

MCQ

3. GRASP contains how many fundamental principles of object design?

A) Five
B) Seven
C) Nine
D) Twelve

4. In object-oriented design, responsibility refers to:

A) Data storage only
B) Database queries
C) Compilation rules
D) A contract or obligation of a class or component

MCQ

5. Loose coupling means:
A) High dependency between classes
B) No interaction between classes
C) Low dependency between classes
D) Only inheritance relationships

6. Cohesion measures:
A) Database connections
B) Relationship between modules
C) How strongly related responsibilities of a class are
D) Execution speed

MCQ SOLUTION

1.A
2.B
3.C
4.D
5.C
6.C

References

- Source Code Examples
- Ad-hoc, Inclusion, Parametric & Coercion Polymorphisms – GeeksforGeeks
- Larman - Monopoly Game.pdf - THE MONOPOLY GAME SYSTEM • The software version of the game will run as a simulation • One person will start the game and | Course Hero
- http://www.kamilgrzybek.com/design/grasp-explained/

THANK YOU

Prof. Shridevi A Sawant
Computer Science and Engineering

UE23CS352B - Object Oriented Analysis and Design

Prof. Shridevi A. Sawant
Dept. of Computer Science and Engineering

Acknowledgements:
Dr. J. MANNAR MANNAN
Dept. of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

Controller

- Deals with **how to delegate the request from the UI layer objects to domain layer objects.**
- Helps in **minimizing the dependency** between GUI components and the system operation classes
- When a request comes from UI layer object, Controller as a pattern helps us in determining which is the first object that should receive the message from the UI layer objects. – **Controller Object**
- Delegates the work to other class and coordinates all activities.
- **Benefits** - Can reuse the controller class. Can use to maintain the state of the use case. Can control the sequence of the activities

Controller contd..

- **Definition**

Problem: Who should be responsible for handling a system event? (Or, what object receives and coordinates a system operation?)

Solution: Assign the responsibility for receiving and/or handling a system event to one of following choices:

Object that represents overall system, device or subsystem (façade controller)

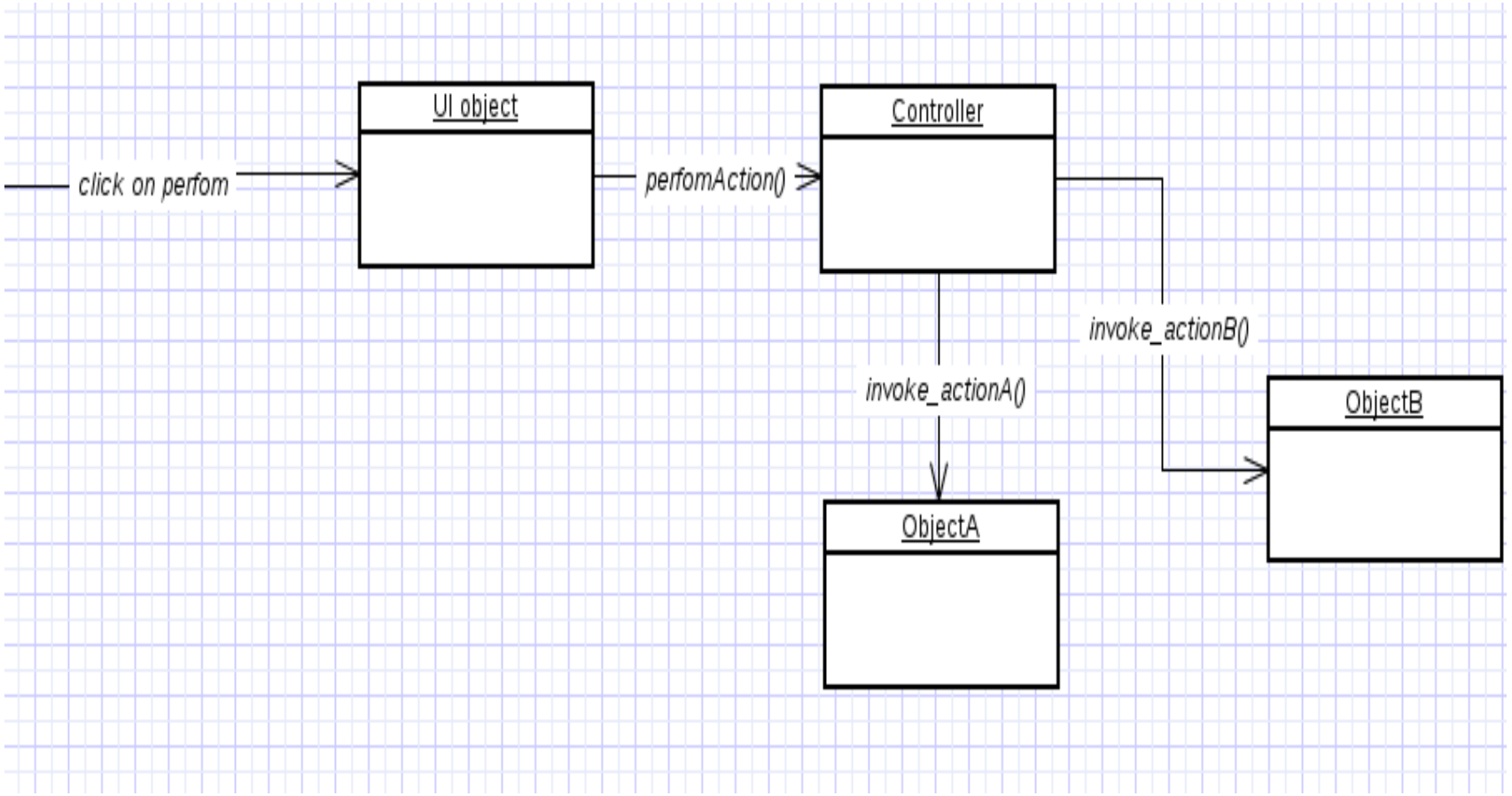
Object that represents a use case scenario within which the system event occurs (a <UseCase>Handler)

- **Input system event** – Event generated by an external actor associated with a system operation

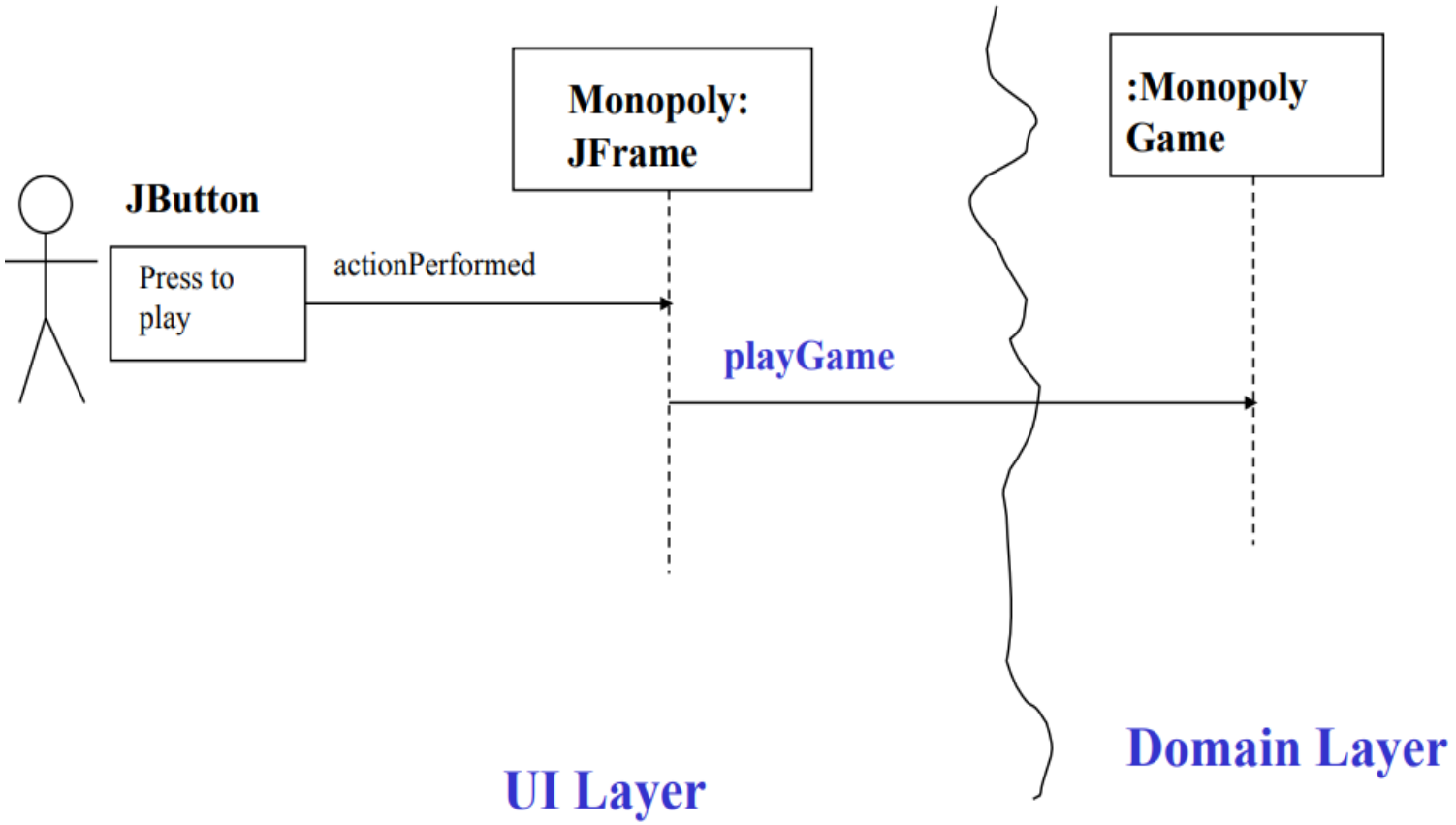
Controller contd..

- Normally controller coordinates activity but delegates work to other objects rather than doing work itself
- **Façade Controller :** A class that represents the overall system, device subsystems
- **Use case Controller:** A class that represent a use case, whereby performs handling particular system operations Often uses same controller class for all system events of one-use case so that one can maintain state information, e.g., events must occur in a certain order
- **Bloated Controller**
  - Single class receiving all system events and there are many of them
  - Controller performs many tasks rather than delegating them
  - Controller has many attributes and maintains significant information about system which should have been distributed among other objects

Example: Controller



Example: Monopoly example



Here's how to implement the "**Controller**" component in Java:

Identify System Boundaries: Determine the boundaries of the system, including user interfaces, external systems, or other modules that interact with the core functionality of the application. These boundaries represent entry points for system events and inputs.

Define Controller Classes: Create controller classes to handle incoming requests or events from system boundaries. Each controller class is responsible for a specific set of related actions or interactions within the system.

Separate Concerns: Ensure that controller classes focus on coordinating activities and delegating work to other parts of the system, rather than performing business logic or data manipulation directly.

Handle User Inputs: Implement methods or event handlers within controller classes to process user inputs from user interfaces or external systems.

Coordinate Interactions: Use controller classes to coordinate interactions between different components or modules within the system.

Promote Loose Coupling: Ensure that controller classes interact with other parts of the system through well-defined interfaces and abstractions, rather than direct dependencies.

Handle Exceptions: Implement error handling and exception management within controller classes to gracefully handle unexpected scenarios and communicate errors to users or other parts of the system.

Polymorphism

- Deals with How to act different depending on object's class
- How to handle related but varying elements based on type?
- Guides us in deciding which object is responsible for handling those varying elements.

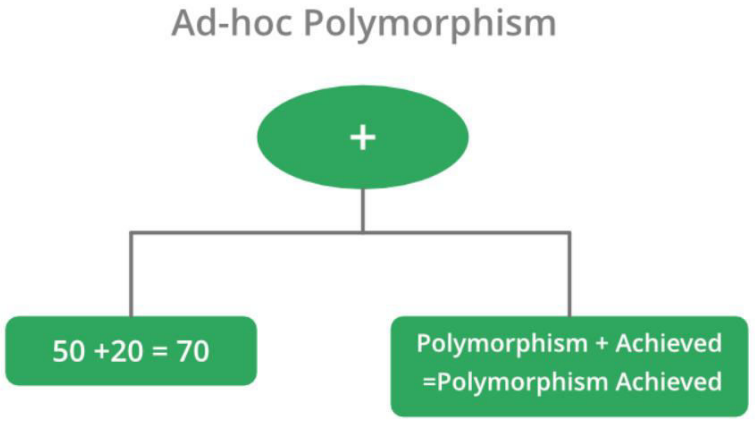
Types:

- Adhoc – Overloading
- Parametric – Early binding
- Inclusion – Sub typing
- Coercion - Casting

Polymorphism

Ad-hoc – Overloading

Ad-hoc Polymorphism, also called as Overloading Ad-hoc Polymorphism allows functions having **same name to act differently for different types**. For example: The + operator adds two integers and concatenates two strings.



Polymorphism

Parametric – Early binding

Parametric Polymorphism, also called as Early Binding Parametric Polymorphism opens a way to use the same piece of code for different types. Parametric polymorphism is implemented using Generics in Java.

> **Inclusion Polymorphism**, also called as Subtyping. Inclusion Polymorphism is the ability to use derived classes through base class references.

- It is also known as **Run-time polymorphism.** In Java, runtime polymorphism is achieved through dynamic method dispatch. The method to be invoked is determined at runtime based on the actual object type, not the reference type.

Polymorphism

Coercion Polymorphism, also called as **Casting Coersion. Polymorphism occurs when an object or primitive is cast into some other type.** It could be either Implicit or Explicit. Implicit casting happens as a responsibility of Compiler itself.
An object of one type is implicitly converted to another type to match an expected interface or function parameter.

In Java, coercion polymorphism occurs when the compiler automatically converts one type to another compatible type (widening), or when a developer explicitly casts a value to another type.

```
int x = 10;
double y = x;   // int → double (implicit widening)

// explicit cast - converting from a larger type to a smaller type
double d = 9.7;
int i = (int) d;
```

Polymorphism contd..

- **Definition:**

Name: Polymorphism

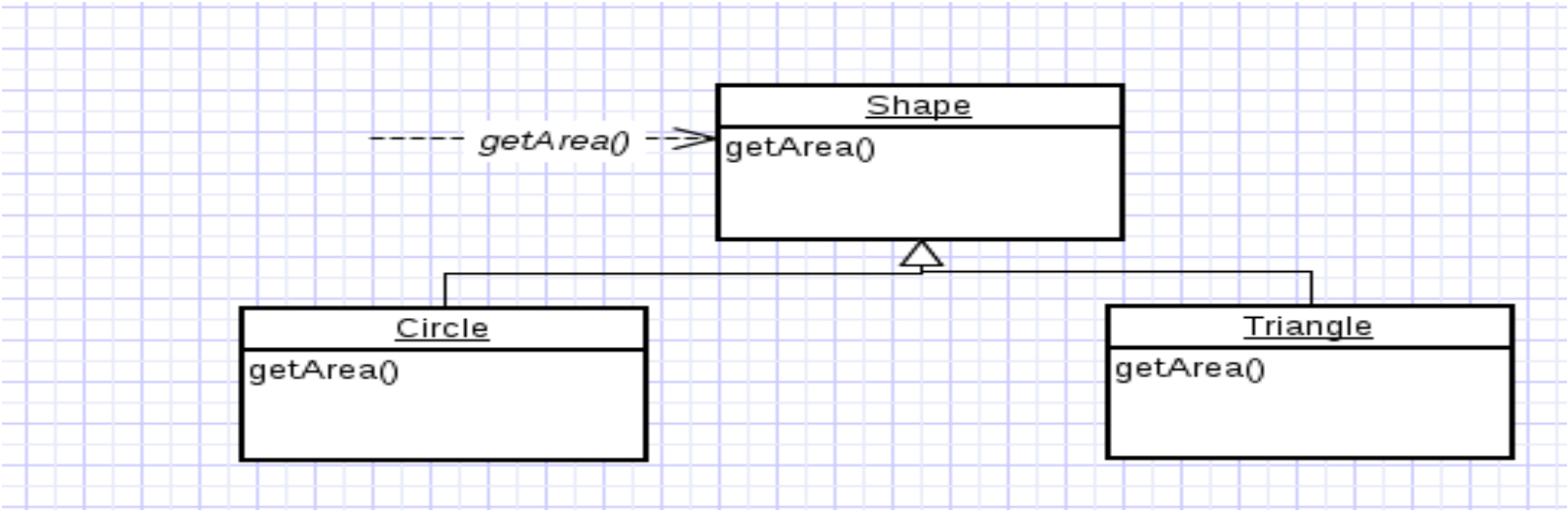
Problem: How to handle alternatives based on type.

Pluggable software components -- how can you replace one server component with another without affecting the client?

Solution: When related alternatives or behaviors vary by type (class), assign responsibility using polymorphic operations – to the types for which the behavior varies. In this context, polymorphism means giving the same name to similar or related services

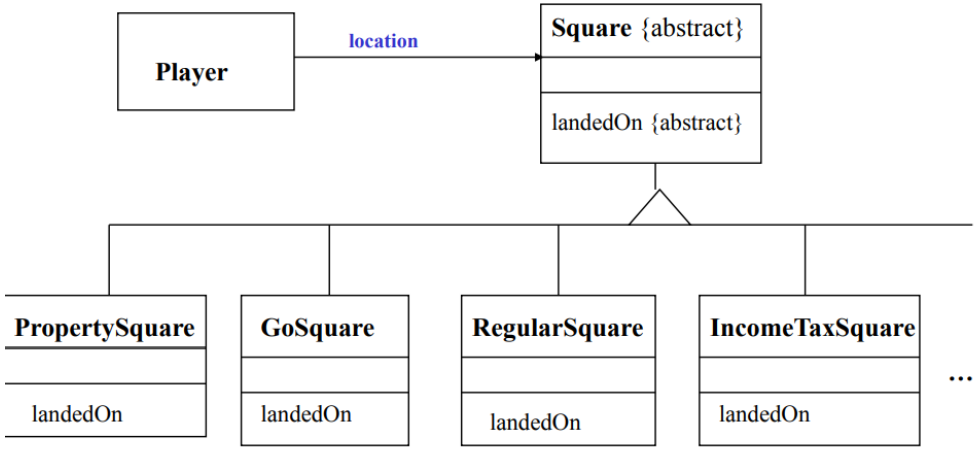
Polymorphism : Example

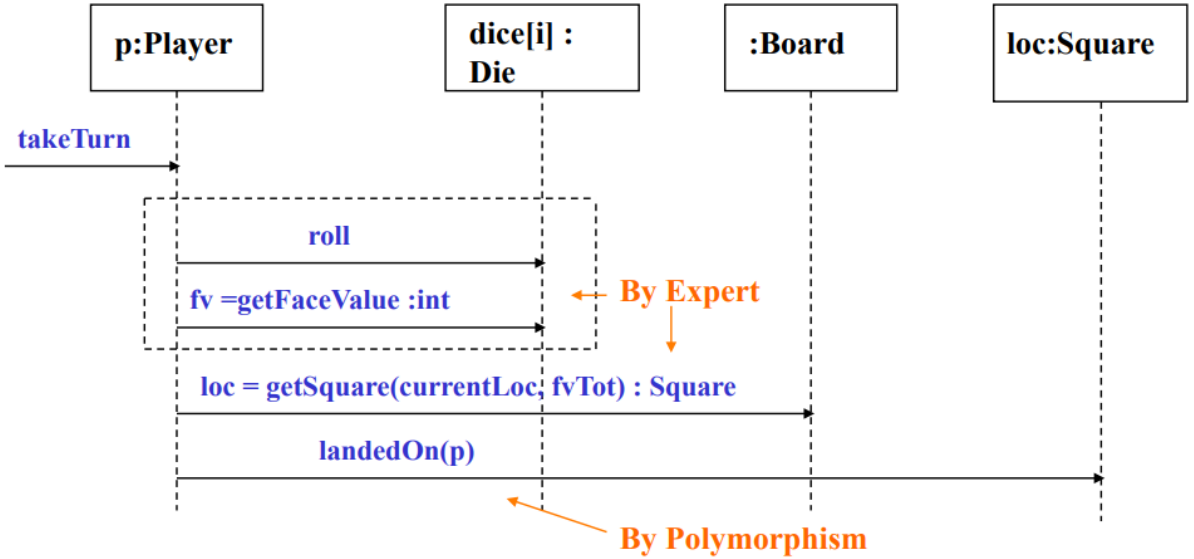
- The getArea() varies by the type of shape, so we assign that responsibility to the subclasses
- By sending message to the Shape object, a call will be made to the corresponding sub class object – Circle or Triangle



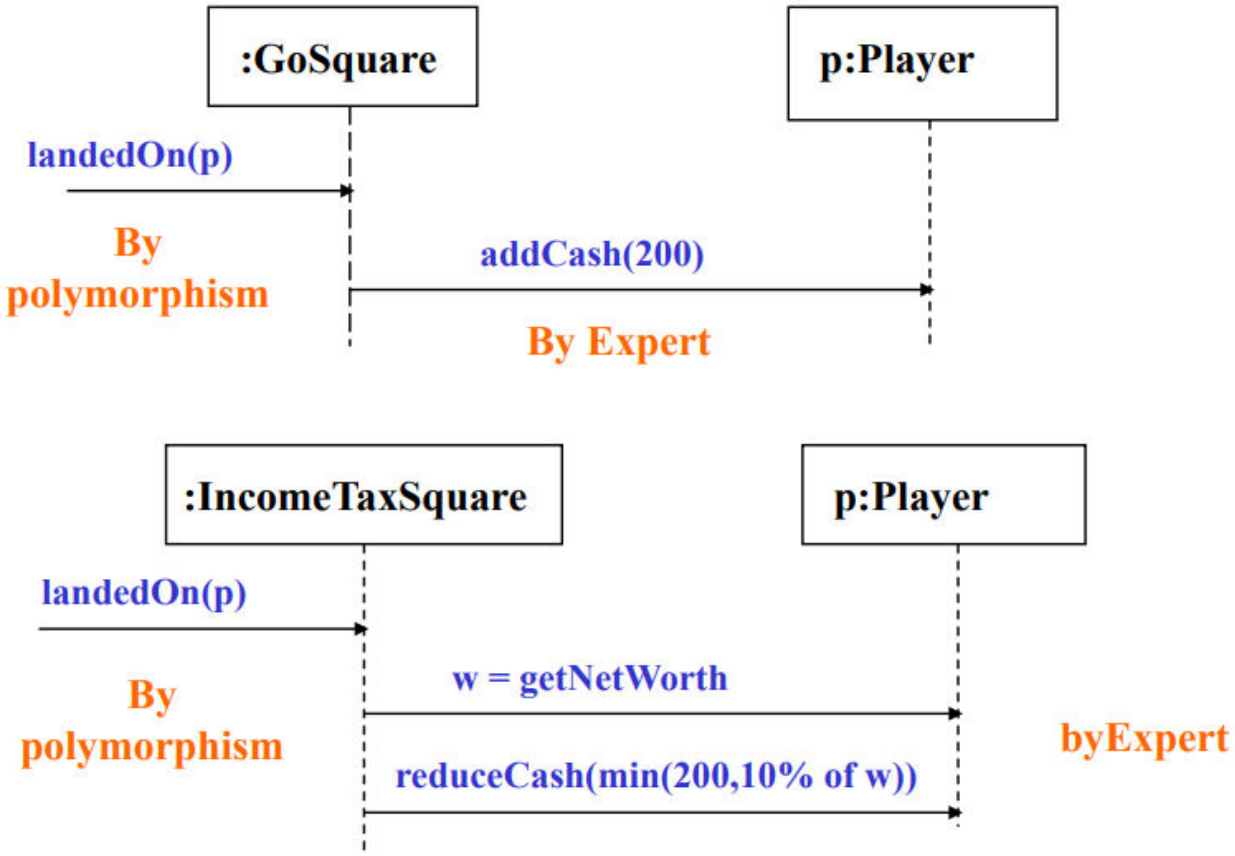
Object Oriented Analysis and Design

Polymorphism : Monopoly game





Polymorphism : Monopoly game



Polymorphic cases

MCQ

1. The Controller pattern is mainly responsible for:

A) Performing database operations
B) Delegating requests from UI to domain objects
C) Rendering user interfaces
D) Managing memory

2. The Controller pattern helps minimize dependency between:

A) Database and server
B) Packages and modules
C) Methods and variables
D) GUI components and system operation classes

3. In GRASP, a controller usually:

A) Performs all business logic
B) Stores all system data
C) Coordinates activities and delegates work
D) Replaces domain classes

MCQ

4. Polymorphism addresses the problem of:
A) Handling alternatives based on type
B) Memory Allocation
C) File storage
D) Data serialization

5. Giving the same name to related services is known as:
A) Inheritance
B) Encapsulation
C) Polymorphism
D) Aggregation

MCQ SOLUTION

1.B
2.D
3.C
4.A
5.C

References

- GRASP: Guiding Object-Oriented Design in Java | by Alex Klimenko | Medium
- Source Code Examples
- Ad-hoc, Inclusion, Parametric & Coercion Polymorphisms – GeeksforGeeks
- Larman - Monopoly Game.pdf - THE MONOPOLY GAME SYSTEM • The software version of the game will run as a simulation • One person will start the game and | Course Hero
- http://www.kamilgrzybek.com/design/grasp-explained/

THANK YOU

Prof. Shridevi A. Sawant
Computer Science and Engineering

PES UNIVERSITY
CELEBRATING 50 YEARS

UE23CS352B - Object Oriented Analysis and Design

Prof. Shridevi A Sawant
Dept. of Computer Science and Engineering

Acknowledgements:
Dr. J. MANNAR MANNAN
Dept. of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

Indirection

- **Definition:**

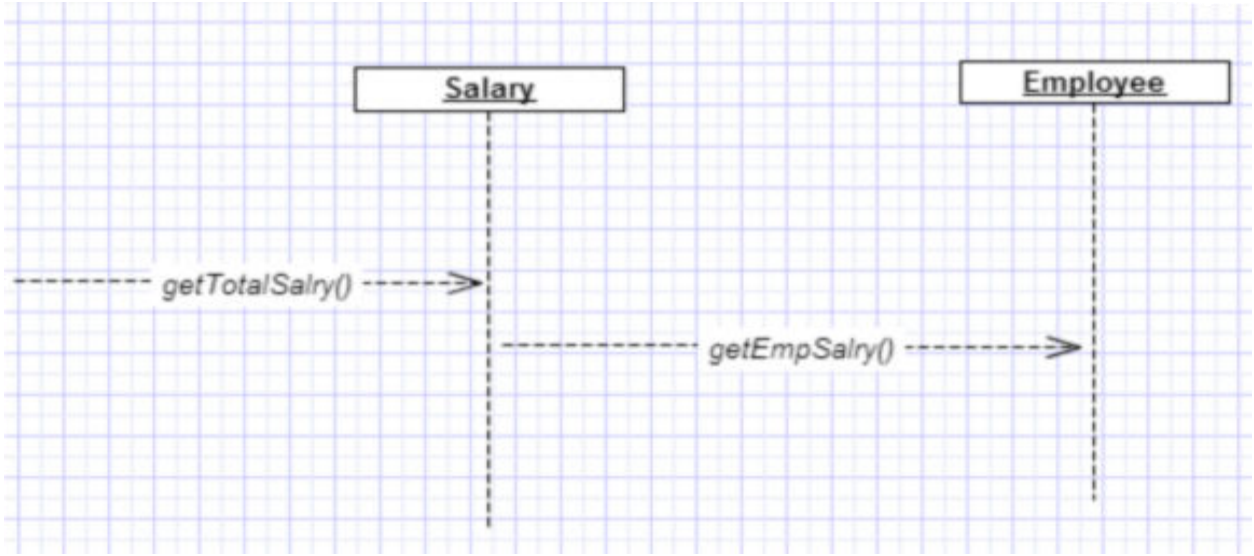
  Name: Indirection

  Problem**:** How to assign responsibilities in order to avoid direct coupling between two components and keep ability for reuse.

  Solution: Assign responsibility to intermediate class for providing linking between objects not linking directly

- Related Design patterns: Adapter, Bridge, Mediator.

Indirection



Salary serves as the intermediary between the caller and Employee to obtain salary details.

Indirection

- **Identify Mediating Responsibilities:** Determine which responsibilities involve mediating interactions between different parts of the system or coordinating activities across multiple components. These responsibilities may include routing requests, translating between interfaces, or managing dependencies.
- **Introduce Intermediate Objects:** Create intermediary classes or components to handle mediating responsibilities. These intermediate objects act as indirection points, decoupling the classes or modules that interact with each other.

Indirection

- **Encapsulate Mediating Logic:** Ensure that the intermediate objects encapsulate the logic for mediating interactions or delegating tasks. This helps to maintain encapsulation and prevents the proliferation of mediating logic throughout the codebase.
- **Promote Loose Coupling:** By introducing indirection, classes or modules become less dependent on each other's concrete implementations, leading to greater flexibility and easier maintenance. Changes to one part of the system are less likely to impact other parts, as long as the indirection layer remains stable.
- **Facilitate Evolution and Extension:** Indirection facilitates the evolution and extension of the system by providing a flexible point of interaction between components.

Pure Fabrication

- **Definition:**

Name: Pure Fabrication

Problem: What object should have responsibility when you do not want to violate High Cohesion and Low Coupling, or other goals, but solutions offered by Expert (for example) are not appropriate? Sometimes assigning responsibilities only to domain layer software classes leads to problems like poor cohesion or coupling, or low reuse potential.

Solution: Assign a highly cohesive set of responsibilities to an artificial or convenience class that does not represent a domain concept

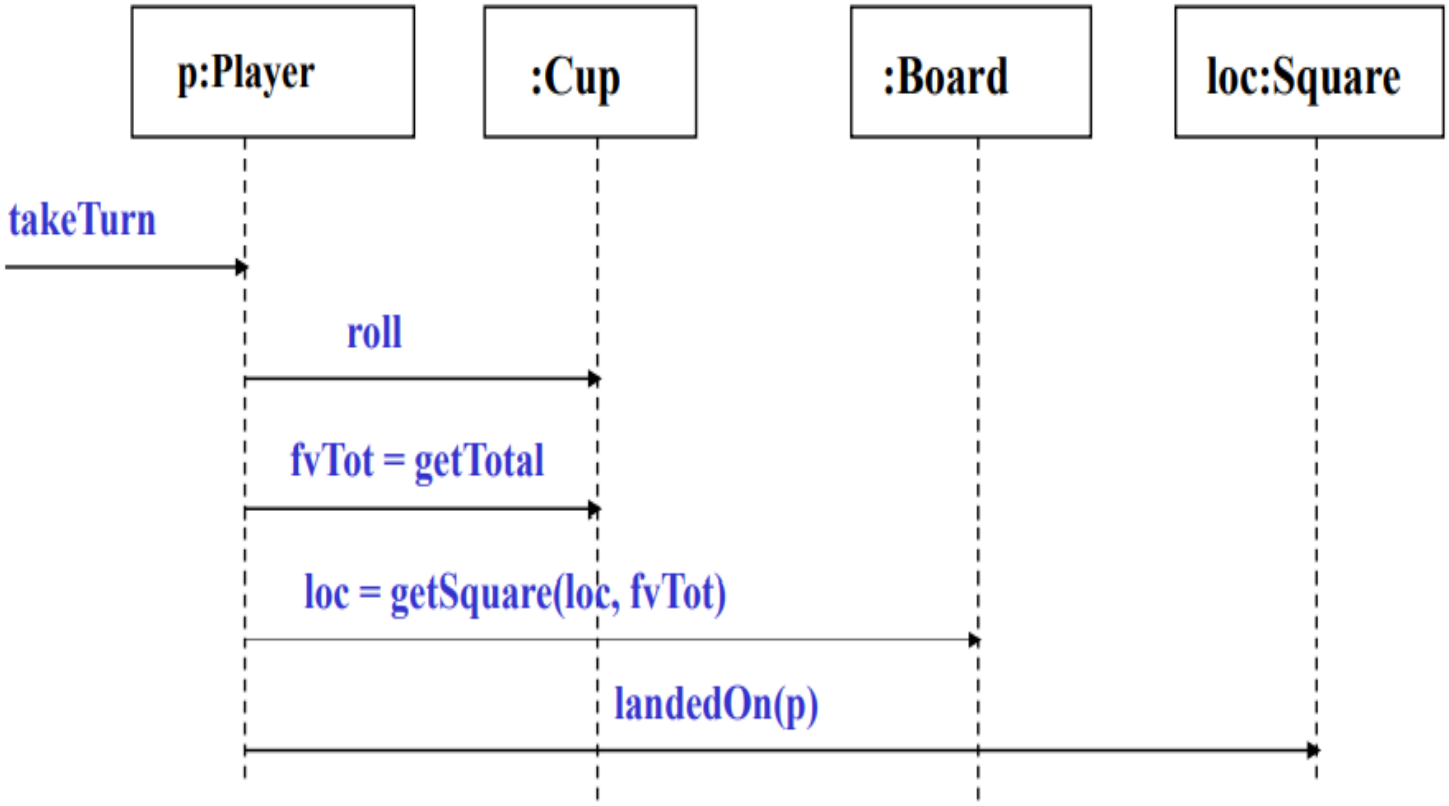
Pure Fabrication

- Here's how to understand "Pure Fabrication" in Java:
- **Identify Complex Responsibilities:** In Java application design, there may be certain responsibilities or tasks that do not naturally fit within the domain of any existing classes.
- **Introduce Fabrication Classes:** Once complex responsibilities are identified, the "Pure Fabrication" principle suggests introducing additional classes or objects solely for the purpose of encapsulating these responsibilities.
- **Encapsulation of Non-Natural Responsibilities:** The pure fabrication classes encapsulate the complex or non-natural responsibilities, isolating them from the core domain objects.

Pure Fabrication

- **Promote Modularity and Reusability:** By encapsulating complex responsibilities in pure fabrication classes, Java developers can promote modularity and reusability in their codebase.
- **Examples of Pure Fabrication:** In Java, pure fabrication classes may include utility classes, service classes, or helper classes that perform generic functions such as data validation, formatting, or error handling.

Pure Fabrication – Monopoly game



Use a Cup to hold the dice, roll them, and know their total. It can be reused in many different applications where dice are involved.

Controlled /Protected variation

- **Definition:**

Name: Protected Variation

Problem: How to design objects, subsystems, and systems so that the variations or instability in these elements does not have an undesirable impact on other elements?.

Solution: Identify points of predicted variation or instability; assign responsibilities to create a stable interface around them.

- Identify points of likely change (variation points) and create a stable interface around them so that changes don't affect the rest of the system.
- Example:- Open-Closed Principle similar to CV/PV pattern

- Protected variation pattern applying for both **variation and Evolution points**

Controlled /Protected variation contd.

Protected variation pattern applying for both **variation and Evolution point.**

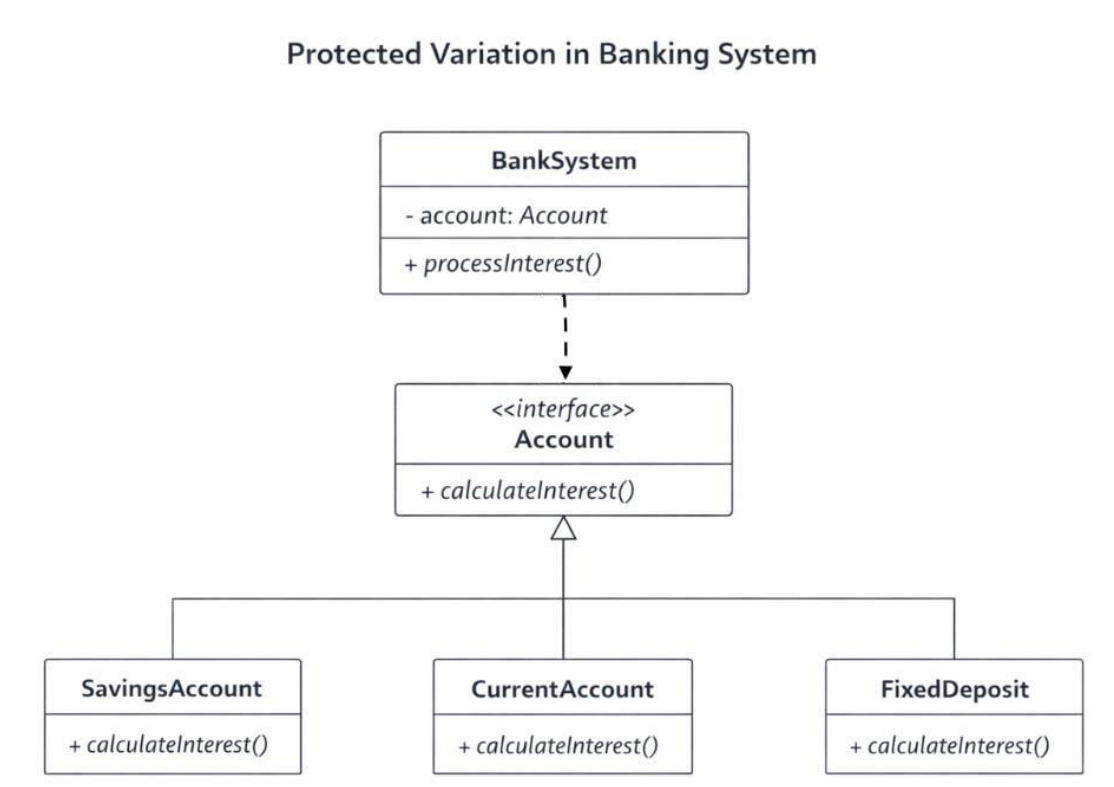
- **Variation point**

  Branching point on existing system or in requirements.

- **Evolution point**

  Supposed branching point, which might occur in future, but does not declare by existing requirements

Controlled /Protected variation contd.



✓ Variation Point: Interest logic
✓ Stable Interface: Account
✓ New Account Type → No modification of BankSystem
✓ Low Coupling
✓ High Cohesion
✓ Supports Open-Closed Principle

MCQ

1. The main purpose of the Indirection principle is to:

A) Increase system performance
B) Avoid direct coupling between components
C) Replace inheritance
D) Reduce the number of classes

2. Indirection works by introducing:

A) Additional databases
B) Static methods
C) Global variables
D) Intermediate classes or objects

MCQ

3. Pure Fabrication refers to:
A) Creating domain objects
B) Assigning responsibilities to artificial classes
C) Removing classes
D) Using inheritance for every design

4. Pure Fabrication is used when:
A) Assigning responsibility to domain classes causes design issues
B) Domain classes already solve the problem
C) There is no need for modularity
D) The system is very small

MCQ

5. Protected Variation works by:
A) Adding more methods
B) Creating stable interfaces around change points
C) Removing modules
D) Avoiding design principles

6. A variation point refers to:
A) Random change in code
B) Branching point in existing system requirements
C) Database migration
D) User interface redesign

PES UNIVERSITY
CELEBRATING 50 YEARS

MCQ SOLUTION

1.B
2.D
3.B
4.A
5.B
6.B

References

- Source Code Examples
- Ad-hoc, Inclusion, Parametric & Coercion Polymorphisms – GeeksforGeeks
- Larman - Monopoly Game.pdf - THE MONOPOLY GAME SYSTEM • The software version of the game will run as a simulation • One person will start the game and | Course Hero
- http://www.kamilgrzybek.com/design/grasp-explained/

THANK YOU

Prof. Shridevi A Sawant
Computer Science and Engineering

Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

PES UNIVERSITY

UE23CS352B: Object Oriented Analysis and Design

SOLID: Single Responsibility

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements:

Prof. Shilpa S

Dept. of Computer Science and Engineering

Object Oriented Analysis and Design

Agenda

- Important aspects of Bad Design
- Good Design
- Introduction – SOLID
- SRP
- Implementation of SRP
- References

Object Oriented Analysis and Design

Important Aspects of Bad Design

Rigidity

- the impact of a change is unpredictable
- every change causes a cascade of changes in dependent modules
- costs become unpredictable

Fragility

- the software tends to break in many places on every change
- the breakage occurs in areas with no conceptual relationship
- on every fix the software breaks in unexpected ways

Immobility

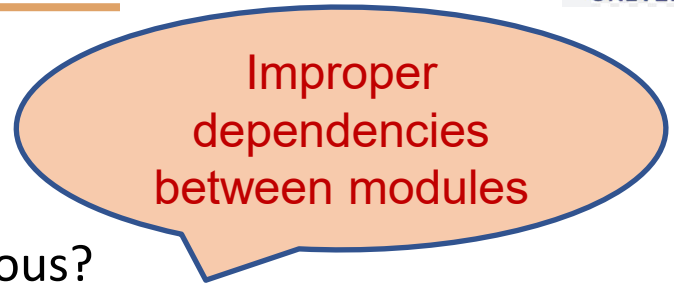
- it's almost impossible to reuse interesting parts of the software
- the useful modules have too many dependencies
- the cost of rewriting is less compared to the risk faced to separate those parts

Viscosity

- a hack is cheaper to implement than the solution within the design
- preserving design moves are difficult to think and to implement
- it's much easier to do the wrong thing rather than the right one

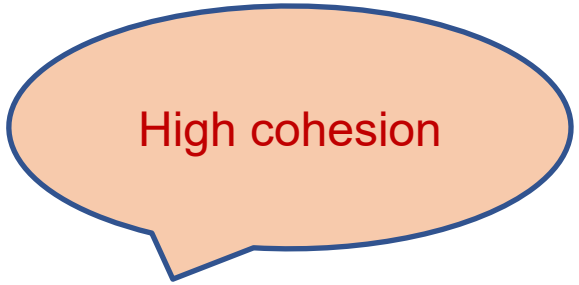
Object Oriented Analysis and Design

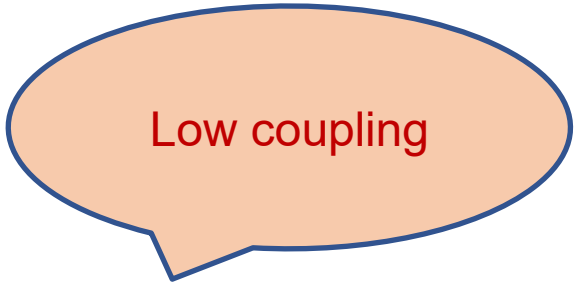
Good Design



- What's the reason why a design becomes rigid, fragile, immobile, and viscous?

- Characteristics of a good design



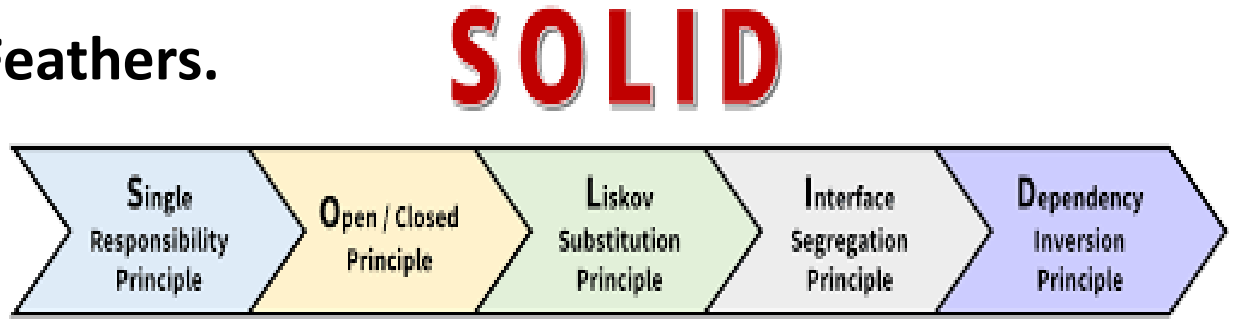


- How can we achieve a good design? - SOLID

Object Oriented Analysis and Design

Introduction to SOLID

- Is an acronym for 5 important design principles introduced by **Robert J. Martin**.
- Leads to more **flexible and stable** software architecture that's easier to maintain and extend, and **less likely to break**
- Acronym identified by **Michael Feathers.**



- SOLID principles are software design coding standards
- Helps us to obtain good software with low coupling and high cohesion
- Applied to reduce dependencies - changes in one part of software will not be impacting others.

Object Oriented Analysis and Design
Single Responsibility Principle(SRP)

- States that **every module or class should have responsibility over a single part of the functionality provided by the software**.
- A **class should have one, and only one, reason to change** - Robert C. Martin
  - Each class only does one thing.
  - Every class or module only has responsibility for one part of the software's functionality.
  - Ensures low coupling code.
  - Ensures easier coding to understand and maintain.
  - Can be applied to classes, software components, and microservices.
  - Code becomes easier to test and maintain, it makes software easier to implement, and it helps to avoid unanticipated side-effects of future changes.

Object Oriented Analysis and Design

Violation of SRP

- The code violates the Single Responsibility Principle, as the Book class has two responsibilities.
- First, it sets the data related to the books (title and author).
- Second, it searches for the book in the inventory.

Note: The setter methods change the Book object, which might cause problems when we want to search the same book in the inventory.

```
class Book {
   String title;
   String author;

   String getTitle() {
      return title;
   }

   void setTitle(String title) {
      this.title = title;
   }

   String getAuthor() {
      return author;
   }

   void setAuthor(String author) {
      this.author = author;
   }

   void searchBook() {...}
}
```

Solution

- Identify things that are changing for different reasons
- Group together things that change for the same reason
- Decouple the responsibilities.
- In the refactored code, the Book class will only be responsible for getting and setting the data of the Book object.
- Create another class called InventoryView that will be responsible for checking the inventory.
- Move the searchBook() method to the new class.

Object Oriented Analysis and Design

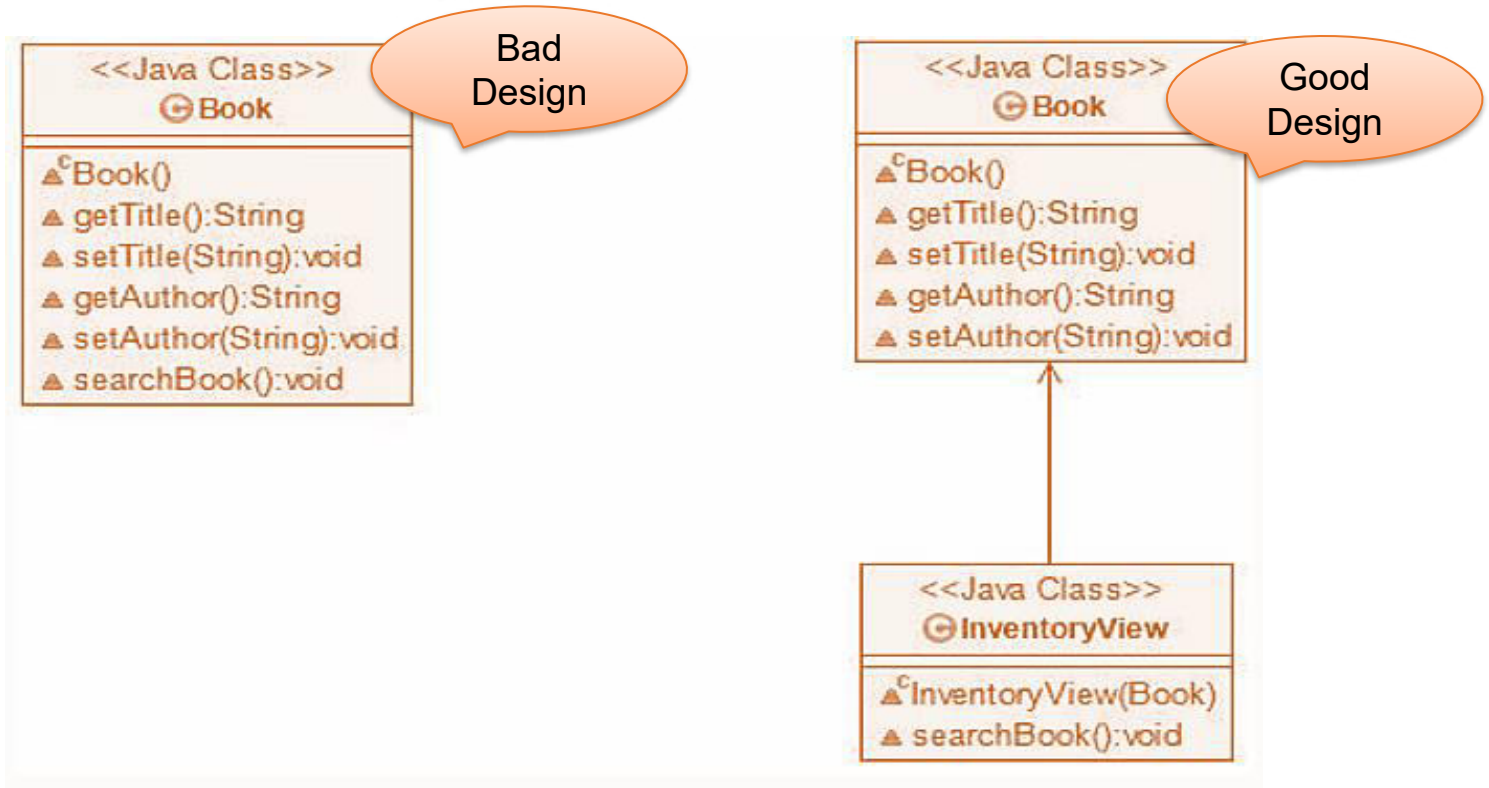
Good Design using SRP

```
class Book {
    String title;
    String author;

    String getTitle() {
        return title;
    }

    void setTitle(String title) {
        this.title = title;
    }

    String getAuthor() {
        return author;
    }

    void setAuthor(String author) {
        this.author = author;
    }
}
```

```
class InventoryView {
    Book book;

    InventoryView(Book book) {
        this.book = book;
    }

    void searchBook() {...}
}
```

Object Oriented Analysis and Design

UML Class Diagram



The Book class should only represent book data, not perform search operations.

SRP: One class → One responsibility → One reason to change.

Object Oriented Analysis and Design

Advantages

- **Testing** – A class with one responsibility will have fewer test cases.
- **Lower coupling** – Less functionality in a single class will have fewer dependencies.
- **Organization** – Smaller, well-organized classes are easier to search than monolithic ones.

Object Oriented Analysis and Design

SRP - Implementation

DEMO

MCQ

1. Who introduced the SOLID principles?

A) Michael Feathers
B) Robert J. Martin
C) Erich Gamma
D) James Gosling

2. According to SRP, a class should have how many reasons to change?

A) One and only one
B) At least two
C) Multiple
D) None

MCQ

3. Which is listed as an advantage of SRP?

A) More dependencies
B) Increased rigidity
C) Fewer test cases
D) Larger monolithic classes

4. SOLID principles help to:

A) Increase coupling
B) Create rigid design
C) Increase viscosity
D) Reduce dependencies

PES UNIVERSITY

MCQ SOLUTION

1. B
2. A
3. C
4. D

Object Oriented Analysis and Design

References

- Introduction To SOLID Principles (c-sharpcorner.com)
- A Solid Guide to SOLID Principles | Baeldung
- SOLID Principle in Programming: Understand With Real Life Examples - GeeksforGeeks

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

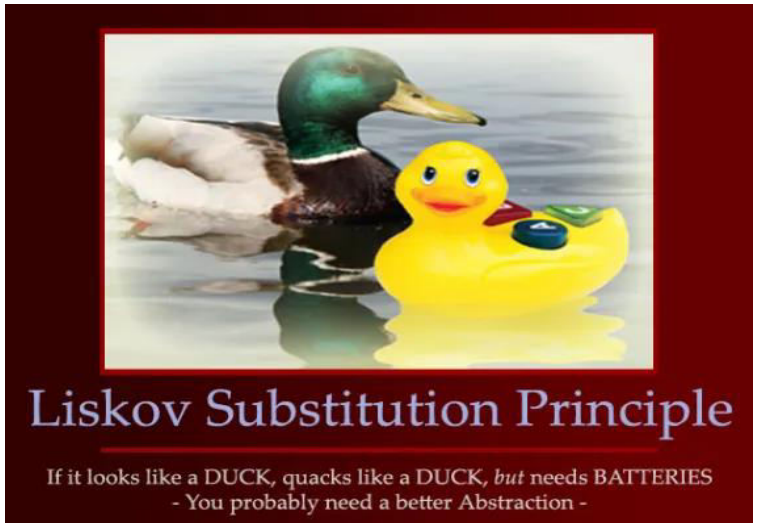
SOLID: Liskov Substitution Principle (LSP)

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements:
Prof. Sindhu R Pai
Dept. of Computer Science and Engineering

Object Oriented Analysis and Design
Liskov Substitution Principle (LSP)

- Introduced by Barbara Liskov in 1987
- **"Let S be a subtype of T, then for each object o1 of type S there is an object o2 of type T such that for all programs P defined in terms of T, the behaviour of P is unchanged when o1 is substituted for o2".**
- **Derived types must be completely substitutable for their base types**
- An object of a superclass should be replaceable by objects of its subclasses without causing issues in the application.
- A child class should never change the characteristics of its parent class
- Derived classes should never do less than their base class.
- Applies to inheritance(is – a relationship) hierarchies and avoids overuse/misuse



Object Oriented Analysis and Design

Example code for violation of LSP

- Consider the same book store example. The book store asks us to add a new delivery functionality to the application. The store also sells fancy hardcovers. It wants to deliver that to their high street shops
- Create a BookDelivery class that informs customers about the number of locations where they can collect their order. Create a new HardcoverDelivery subclass that extends BookDelivery and overrides the getDeliveryLocations() method with its own functionality

```
class BookDelivery {
    String titles;
    int userID;
    void getDeliveryLocations()
{...}
}
```

```
class HardcoverDelivery extends BookDelivery {
    @Override
    void getDeliveryLocations() {...}
}
```

Object Oriented Analysis and Design

Example code for violation of LSP

- Later, the store asks us to create delivery functionalities for audiobooks.
- Now, we extend the existing BookDelivery class with an AudiobookDelivery subclass

```
class AudiobookDelivery extends BookDelivery {
    @Override
    void getDeliveryLocations() {/* can't be implemented */}
    // audiobooks can’t be delivered to physical locations.
}
```

- We have to change some characteristics of the getDeliveryLocations() method. That would violate the Liskov Substitution Principle.
- After the modification, we couldn’t replace the BookDelivery superclass with the AudiobookDelivery subclass without breaking the application

Solution

- To solve the problem, we need to fix the inheritance hierarchy.
- Let's introduce an extra layer of abstraction that better differentiates book delivery types.
- The new OfflineDelivery and OnlineDelivery classes split up the BookDelivery superclass.
- Move the getDeliveryLocations() method to OfflineDelivery and create a new getSoftwareOptions() method for the OnlineDelivery class (as this is more suitable for online deliveries).

Object Oriented Analysis and Design

Good Design

```
class BookDelivery {

    String title;
    int userID;
}

class OfflineDelivery extends BookDelivery {

    void getDeliveryLocations() {...}
}

class OnlineDelivery extends BookDelivery {

    void getSoftwareOptions() {...}
}
```

Object Oriented Analysis and Design

Good Design

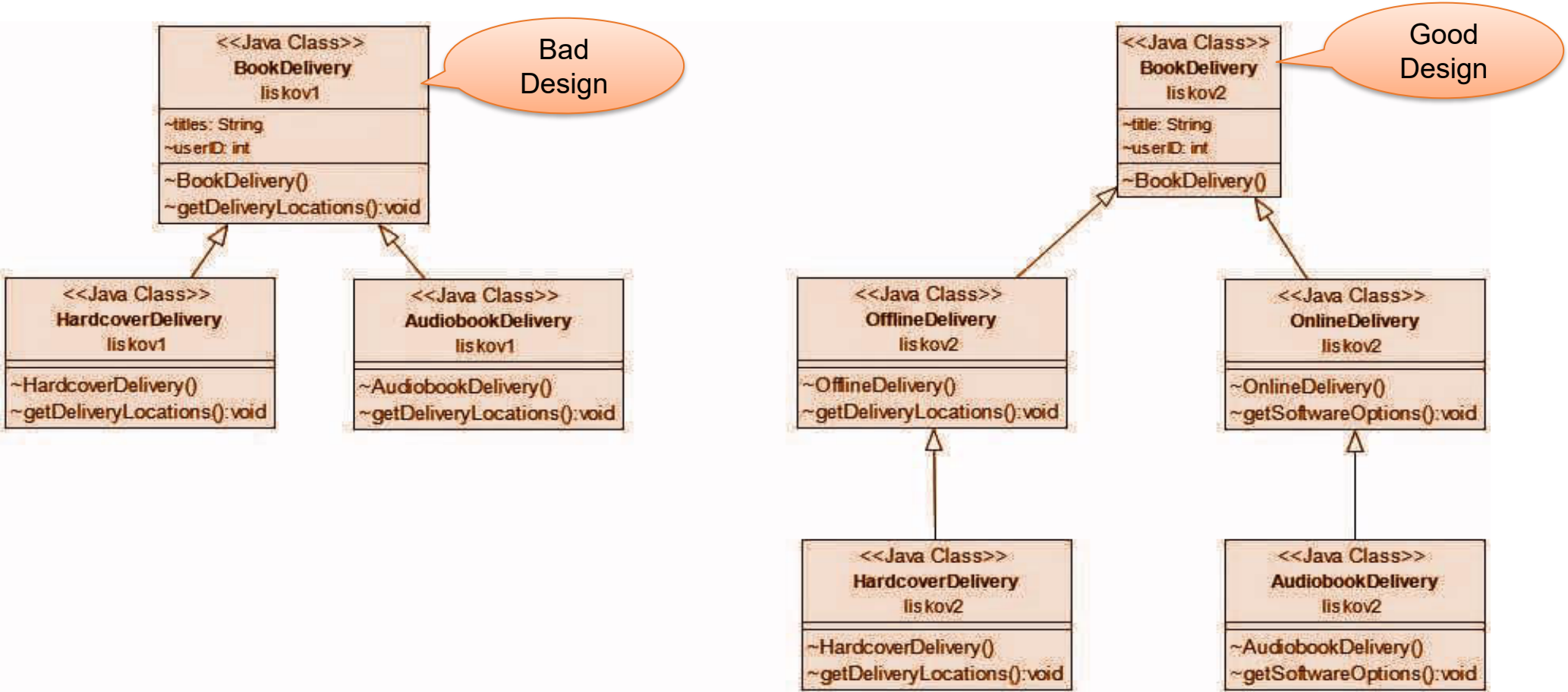
- In the refactored code, HardcoverDelivery will be the child class of OfflineDelivery and it will override the getDeliveryLocations() method with its own functionality.
- AudiobookDelivery will be the child class of OnlineDelivery, now it doesn't have to deal with the getDeliveryLocations() method.
- Instead, it can override the getSoftwareOptions() method of its parent with its own implementation (for instance, by listing and embedding available audio players).

```
class HardcoverDelivery extends OfflineDelivery {
    @Override
    void getDeliveryLocations() {...}
}

class AudiobookDelivery extends OnlineDelivery {
    @Override
    void getSoftwareOptions() {...}
}
```

Object Oriented Analysis and Design

UML Class Diagram

- An extra layer is added to the inheritance hierarchy.
- New architecture is more complex, it provides us with a more flexible design.



MCQ

1. LSP primarily applies to which type of relationship?
A) Has–a relationship
B) Uses–a relationship
C) Is–a relationship
D) Depends–on relationship

2. According to LSP, derived types must be:
A) More complex than base types
B) Completely substitutable for their base types
C) Independent of base types
D) Modified frequently

MCQ

3. LSP ensures that substituting a subclass object should:

A) Improve performance
B) Change the program structure
C) Leave program behavior unchanged
D) Remove base class methods

4. According to LSP, a child class should:

A) Redefine all parent behavior
B) Never change the characteristics of its parent class
C) Remove inherited methods
D) Reduce functionality

MCQ

5. LSP is concerned with preserving:

A) Method names
B) Class size
C) Behavioral consistency
D) Code comments

6. LSP ensures that an object of a superclass should be replaceable by:

A) Any unrelated class
B) Only abstract classes
C) Objects of its subclasses
D) Static methods

PES UNIVERSITY

MCQ SOLUTION

1.C
2.B
3.C
4.B
5.C
6.C

Object Oriented Analysis and Design

References

- Liskov's Substitution Principle | SOLID Design Principles (ep 1 part 1) – YouTube
- SOLID Design Principles Explained: Interface Segregation with Code Examples (stackify.com)
  - Interface Segregation Principle explained with example in Java – JavaBrahman
  - Interface Segregation Principle in Java with Example (javaguides.net)
  - (221) Interface Segregation Principle - YouTube

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

SOLID: Open – Closed Principle

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements:

Prof. Sindhu R Pai

Dept. of Computer Science and Engineering

SOLID: Open-Closed Principle (OCP)

- States that **classes, modules, microservices, and other code units should be open for extension but closed for modification.**

- **Bertrand Meyer** originated the term OCP
  - A software entity must be easily extensible with new features without having to modify its existing code in use.
  - We should be able to **extend the existing code using OOP features like inheritance via subclasses and interfaces.**
  - Never modify classes, interfaces, and other code units that already exist, as it can lead to unexpected behavior.
  - While adding a new feature extend the code rather than modifying it, so that the risk of failure is minimized

Violation of OCP

- The store wants to hand out cookbooks at a discount price
- Based on the Single Responsibility Principle, Create two separate classes:

CookbookDiscount - to hold the details of the discount

DiscountManager - to apply the discount to the price

```
class CookbookDiscount {
    String getCookbookDiscount() {
        String discount = "30% between Dec 1 and 24";
        return discount;
    }
}

class DiscountManager {
    void processCookbookDiscount(CookbookDiscount
discount) {...}
}
```

Violation of OCP continued

The code works fine until the store management informs us that their cookbook discount sales were so successful that they want to extend it

To process the new type of discount, we need to add the new functionality to the DiscountManager class, too:

```
class BiographyDiscount
{
    String getBiographyDiscount()
    {
    String discount = "50% on the
subject's birthday.";
    return discount;
    }
}
```

```
class DiscountManager {
    void processCookbookDiscount(CookbookDiscount
discount) {...}
    void
processBiographyDiscount(BiographyDiscount
discount) {...}
}
```

Solution

- Refactor the code by adding an extra layer of abstraction that represents all types of discounts.
- Create a new interface called BookDiscount that, the CookbookDiscount and BiographyDiscount classes will implement.

```
public interface BookDiscount{
    String getBookDiscount();
}
```

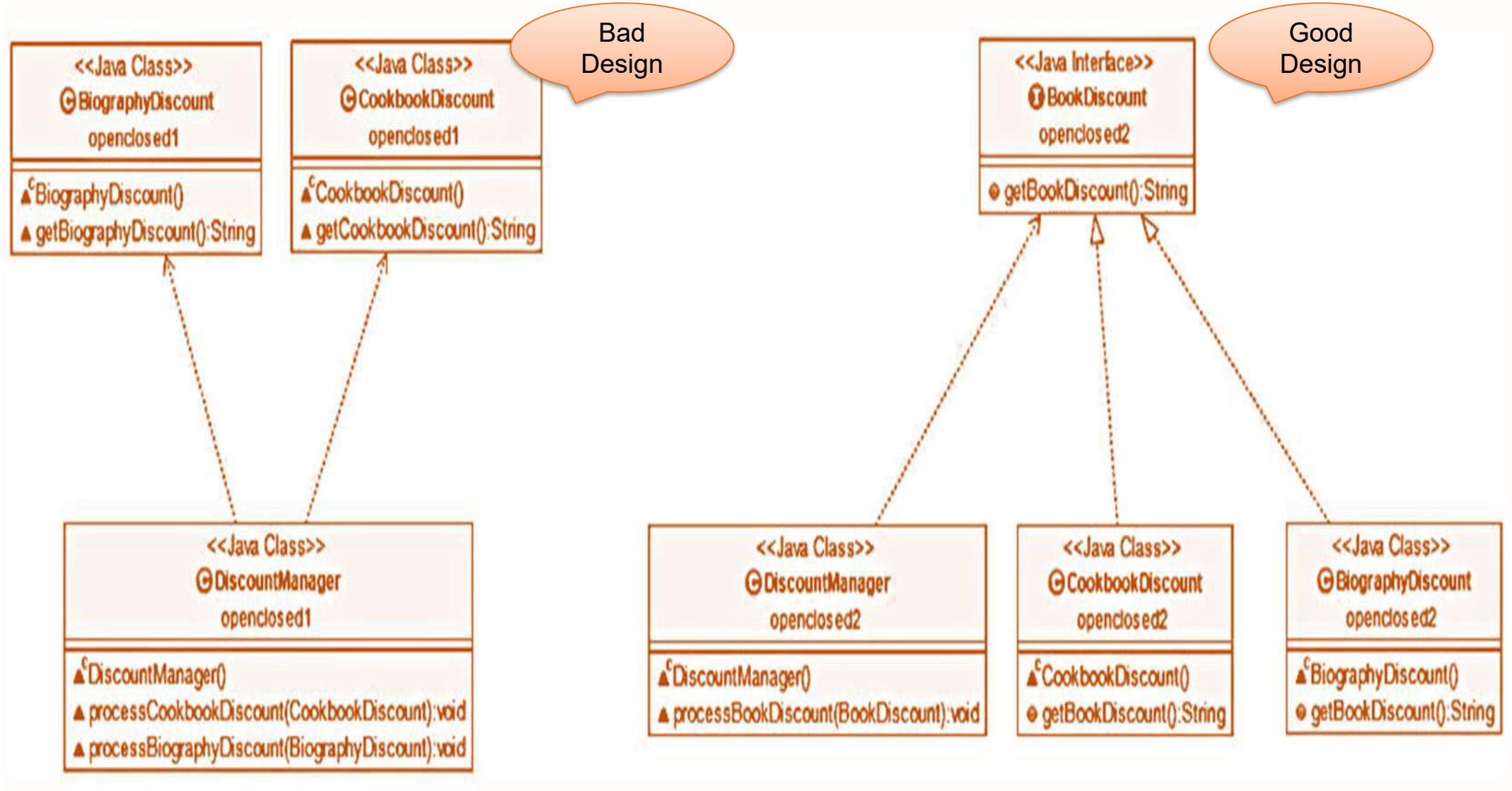
```
class CookbookDiscount implements BookDiscount {
    @Override
    public String getBookDiscount() {
        String discount = "30% between Dec 1 and
24";
        return discount;
    }
}
class BiographyDiscount implements BookDiscount {
    @Override
    public String getBookDiscount() {
        String discount = "50% on the subject's
birthday";
        return discount;
    }
}
```

Solution continued

Now, DiscountManager can refer to the BookDiscount interface instead of the concrete classes. When the processBookDiscount() method is called, pass both CookbookDiscount and BiographyDiscount as an argument, as both are the implementation of the BookDiscount interface.

```
class DiscountManager
{
    void processBookDiscount(BookDiscount discount)
{...}
}
```

UML Class Diagram



Why OCP?

- If not followed

❑ End up testing the entire functionality

❑ QA Team needs to test the entire flow

❑ Costly process for the organization

❑ Breaks the SRP as well

❑ Maintenance overhead increase on the classes

MCQ

1. Which OOP features are suggested to extend existing code?

A) Encapsulation and Polymorphism
B) Inheritance and Interfaces
C) Abstraction only
D) Composition only

2. According to OCP, software entities should be:

A) Open for modification and closed for extension
B) Closed for both modification and extension
C) Open for extension but closed for modification
D) Open for modification only

MCQ

3. If OCP is not followed, what happens to testing?

A) Only new features are tested
B) No testing required
C) Entire functionality must be tested
D) Testing effort decreases

4. Violating OCP may also:

A) Strengthen SRP
B) Break SRP
C) Reduce cohesion
D) Eliminate abstraction

MCQ SOLUTION

1.B
2.C
3.C
4.B

Object Oriented Analysis and Design

References

- Introduction To SOLID Principles (c-sharpcorner.com)
- A Solid Guide to SOLID Principles | Baeldung
- SOLID Principle in Programming: Understand With Real Life Examples - GeeksforGeeks

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

SOLID: Interface Segregation Principle (ISP)

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements:

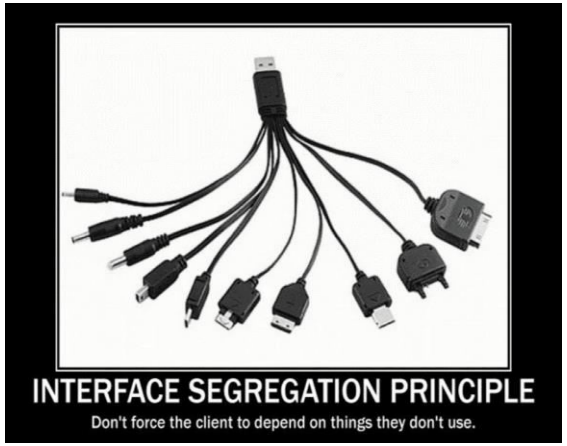
Prof. Sindhu R Pai

Dept. of Computer Science and Engineering

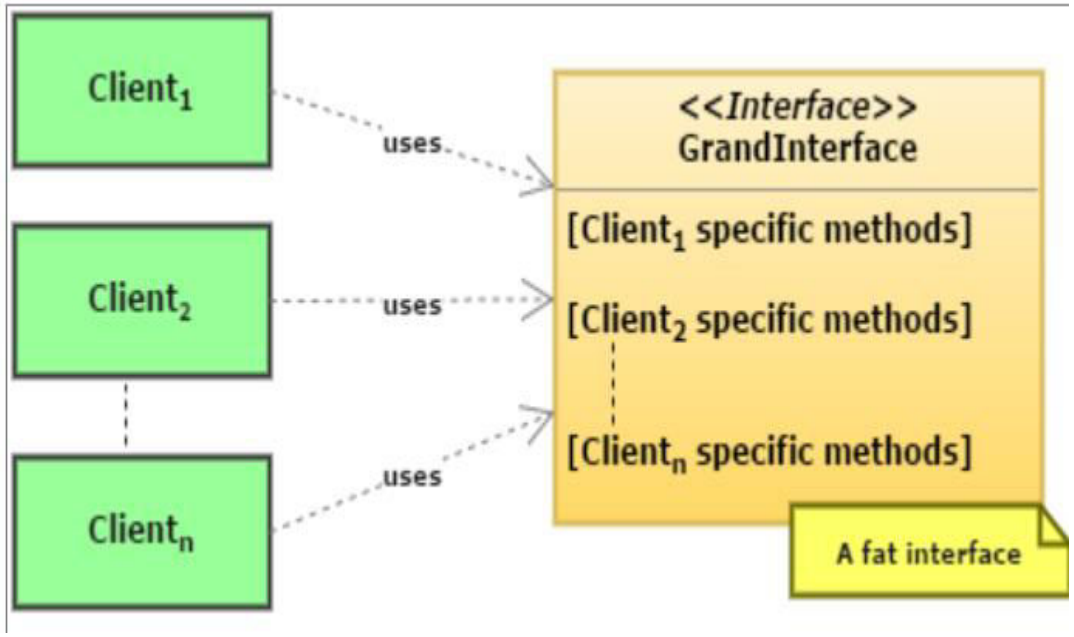
Object Oriented Analysis and Design
Interface Segregation Principle (ISP)

States that "Make fine grained interfaces that are client-specific. Clients should not be forced to implement interfaces they do not use".

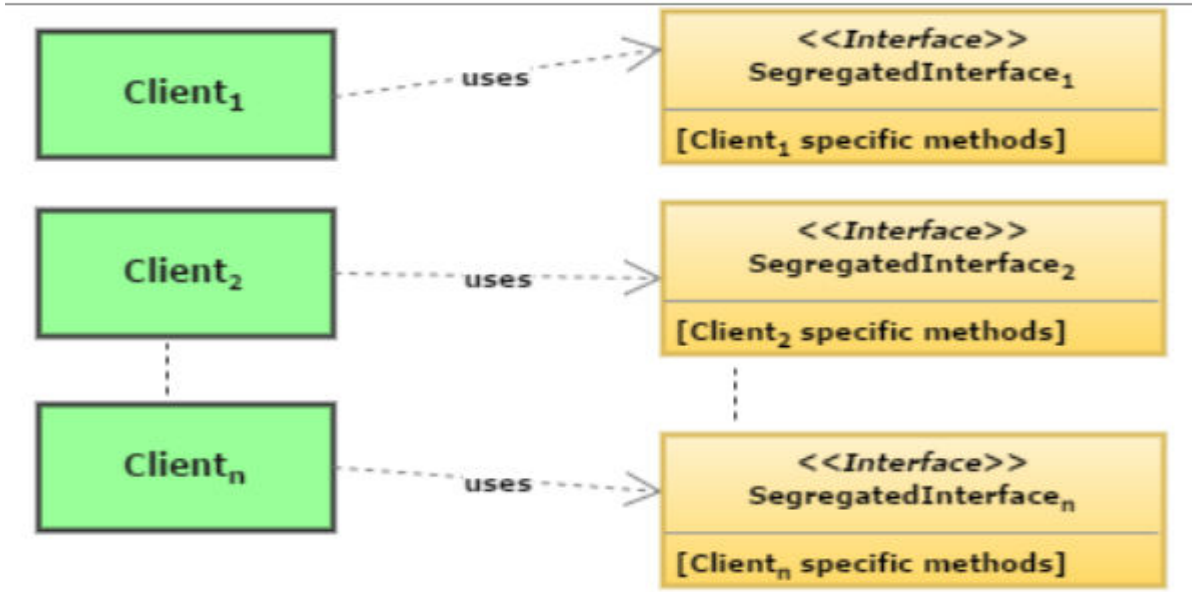
- Break the fat interfaces which has too many methods.
- The principle states that many client-specific interfaces are better than one general-purpose interface.
- Clients should not be forced to implement a function they do not need.
- Instead, start by building a new interface and then let your class implement multiple interfaces as needed.
- Create smaller interfaces that you can implement more flexibly
- Failure to comply with this principle means that in our implementations we will have dependencies on methods that we do not need but that we are obliged to define.



Object Oriented Analysis and Design
Interface Segregation Principle (ISP)



Before Applying ISP: Fat interface



After Applying ISP: Lean interface/role interface

Note: Interface should not be bloated with methods that implementing classes don't require.

Object Oriented Analysis and Design

Example code

```
interface IUser{
    boolean login(String name, String password);
    boolean register(String name, String password, String email);
    void logError(String error);
    boolean sendmail(String emailContent);
}
```

Code violating ISP

Code following ISP

```
interface IUser{
    boolean login(String name, String password);
    boolean register(String name, String password, String email);
}

interface ILogger{
    void logError(String error);
}

interface IMail{
    boolean sendmail(String emailContent);
}
```

Object Oriented Analysis and Design

Example code for violation of ISP

- Adding some user actions to the online bookstore so that customers can interact with the content before making a purchase by creating an interface called BookAction with three methods:

```
public interface BookAction {
    void seeReviews();
    void searchSecondhand();
    void listenSample();
}
```

Object Oriented Analysis and Design

Example code for violation of ISP continued

- Create two classes: HardcoverUI and an AudiobookUI that implement the BookAction interface with their own functionalities
- Both classes depend on methods they don't use, so it violates the Interface Segregation Principle

```
class HardcoverUI implements BookAction {
    @Override
    public void seeReviews() {…}

    @Override
    public void searchSecondhand() {…}

    @Override
    public void listenSample() {...}
}
```

```
class AudiobookUI implements BookAction {
    @Override
    public void seeReviews() {…}

    @Override
    public void searchSecondhand() {…}

    @Override
    public void listenSample() {...}
}
```

Object Oriented Analysis and Design

Solution

- Extend the code with two more specific sub-interfaces: HardcoverAction and AudioAction.

```
public interface BookAction {
    void seeReviews();
}

public interface HardcoverAction extends BookAction {
    void searchSecondhand();
}

public interface AudioAction extends BookAction {
    void listenSample();
}
```

Object Oriented Analysis and Design

Solution

- The HardcoverUI class can implement the HardcoverAction interface and the AudiobookUI class can implement the AudioAction interface.

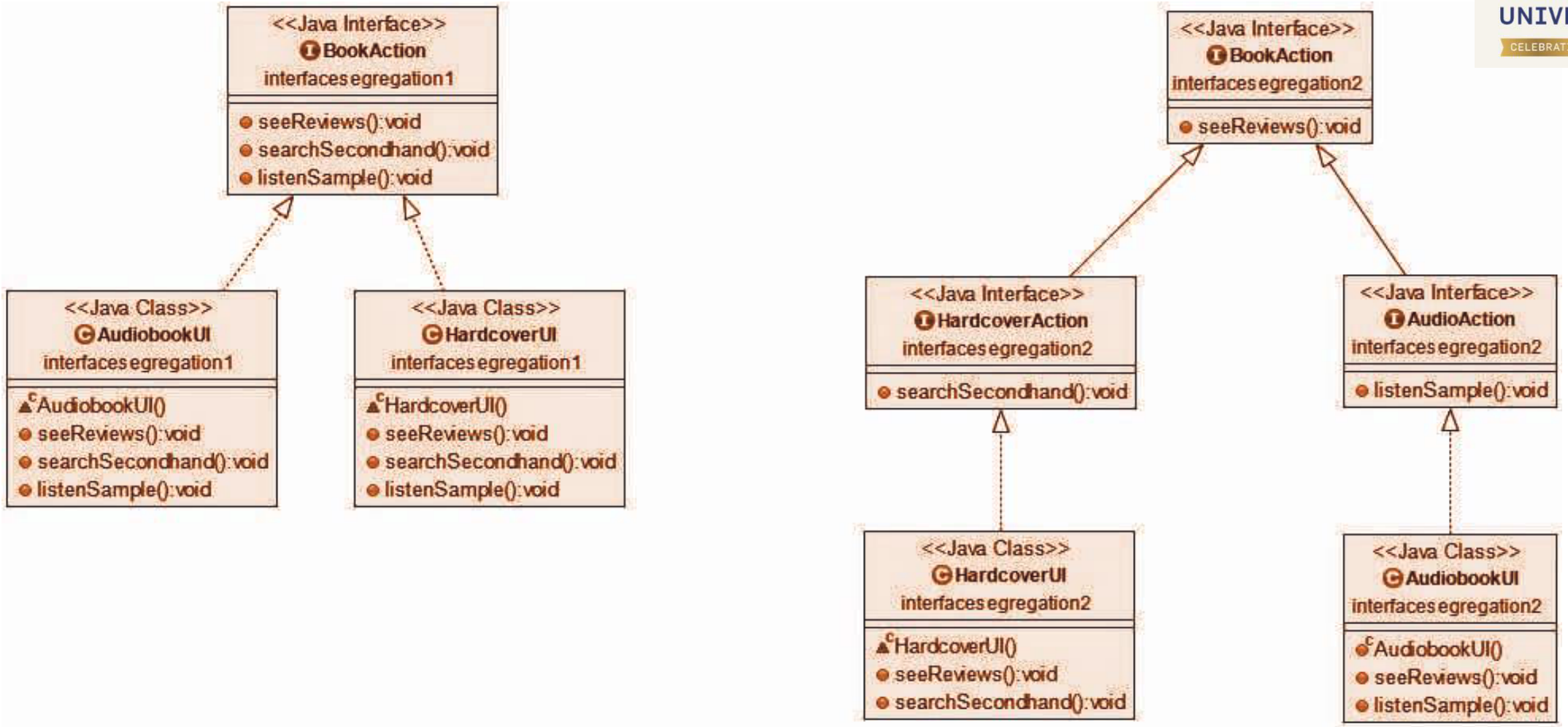
- Both the classes can implement the seeReviews() method of the BookAction super-interface.

```
class HardcoverUI implements HardcoverAction {
    @Override
    public void seeReviews() {…}

    @Override
    public void searchSecondhand() {...}
}
```

```
class AudiobookUI implements AudioAction
{
    @Override
    public void seeReviews() {…}

    @Override
    public void listenSample() {...}
}
```

Object Oriented Analysis and Design

UML Class Diagram



Object Oriented Analysis and Design

Case Study : Xerox

- First used and formulated by Robert C. Martin while consulting for Xerox
- Xerox had created a new printer system that could perform a variety of tasks such as stapling and faxing along with the regular printing task
- The software for this system was created from the ground up
- Modifications and Deployment to the system became more complex

Problem: One large Job class having all the responsibilities of the entire system

Solution: One large Job class is segregated to multiple interfaces depending on the requirement

MCQ

1. The Interface Segregation Principle states that:

A) Clients should depend on large general-purpose interfaces
B) Interfaces should contain as many methods as possible
C) Clients should not be forced to implement interfaces they do not use
D) All classes must implement a single interface

2. According to ISP, implementing classes should:

A) Implement all available methods
B) Implement only necessary methods
C) Avoid interfaces
D) Use inheritance instead

MCQ

3. ISP suggests making interfaces:

A) Large and reusable
B) Fine-grained and specific
C) Generic and broad
D) Static

4. Failure to follow ISP results in:

A) Reduced dependencies
B) Dependencies on unused methods
C) Increased abstraction
D) Better flexibility

MCQ SOLUTION

1.C
2.B
3.B
4.B

Object Oriented Analysis and Design

References

- SOLID Design Principles Explained: Interface Segregation with Code Examples (stackify.com)
  - Interface Segregation Principle explained with example in Java – JavaBrahman
  - Interface Segregation Principle in Java with Example (javaguides.net)
  - (221) Interface Segregation Principle - YouTube

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

SOLID - Dependency Inversion Principle (DIP)

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements:
Prof. Sindhu R Pai
Dept. of Computer Science and Engineering

Object Oriented Analysis and Design
Dependency Inversion Principle (DIP)

Program to the interface, not the implementation

- **High level modules should not depend on low level modules, both should depend upon abstractions.**
- **Abstractions should not depend upon details , details should depend upon abstractions.**
- The goal is to avoid tightly coupled code, as it easily breaks the application.
- Decouple high-level and low-level classes. The interaction between them must be thought of as an **abstract interaction**
- According to Robert C Martin, Dependency Inversion Principle is a **specific combination of the Open-Closed and Liskov Substitution Principles.**

Object Oriented Analysis and Design

Example code violates DIP

- Suppose a book store asked us to build a new feature that enables customers to put their favorite books on a shelf.
- To implement this functionality, we create a lower-level Book class and a higher-level Shelf class. The Book class will allow users to see reviews and read a sample of each book they store on their shelves. The Shelf class will let them add a book to their shelf and customize the shelf.

```
class Book {
    void seeReviews() {...}
    void readSample() {...}
}
```

```
class Shelf {
    Book book;
    void addBook(Book book) {...}
    void customizeShelf() {...}
}
```

Object Oriented Analysis and Design

Example code violates DIP

- When the store asks us to enable customers to add DVDs to their shelves,

First create DVD.

```
class DVD {
    void seeReviews() {...}
    void watchSample() {...}
}
```

Now, we should modify the Shelf class so that it can accept DVDs, too.

- However, this would clearly break the Open-Closed Principle.

The solution is to create an abstraction layer for the lower-level classes

(Book and DVD).

Object Oriented Analysis and Design

Solution

- Introduce the Product interface and both classes will implement.

```
public interface Product {
    void seeReviews();
    void getSample();
}
```

```
class Book implements Product {
    @Override
    public void seeReviews() {…}

    @Override
    public void getSample() {...}

}
```

```
class DVD implements Product {
    @Override
    public void seeReviews() {…}

    @Override
    public void getSample() {...}

}
```

Object Oriented Analysis and Design

Solution

Now, Shelf can reference the Product interface instead of its implementations (Book and DVD).

The refactored code also allows us to later introduce new product types (for instance, Magazine) that customers can put on their shelves, too.

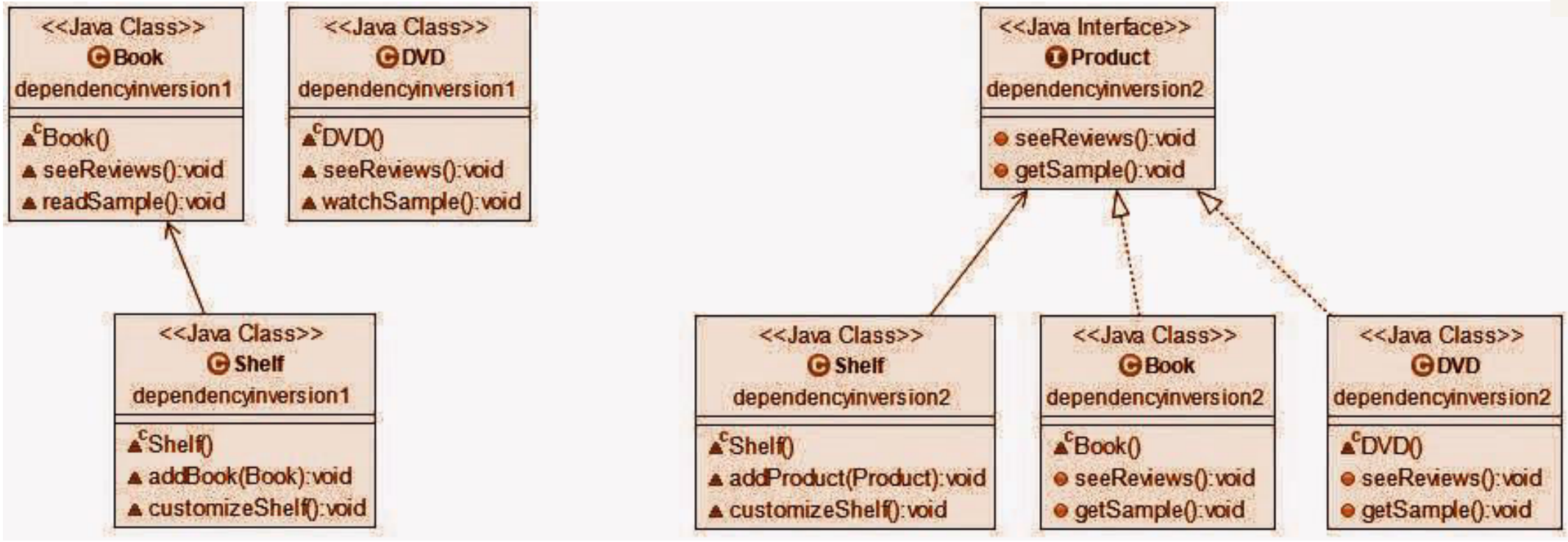
```
class Shelf {
    Product p;
    void getSample();
    void addProduct(Product product) {...}
    void customizeShelf() {...}
}
```

Note: Code follows the **Liskov Substitution Principle**, as the Product type can be substituted with both of its subtypes (Book and DVD) without breaking the program.

At the same time, we have also implemented the **Dependency Inversion** Principle, as high-level classes don't depend on low-level classes

Object Oriented Analysis and Design

UML Class Diagram



Object Oriented Analysis and Design

Intention of Usage – DIP and it's Implementation

MCQ

1. According to DIP, high-level modules should:
A) Depend on low-level modules
B) Avoid abstraction
C) Depend on abstractions
D) Directly control low-level modules

2. In DIP, abstractions should:
A) Depend on details
B) Be avoided
C) Not depend upon details
D) Be concrete

MCQ

3. The primary goal of DIP is to avoid:

A) Inheritance
B) Polymorphism
C) Tightly coupled code
D) Interfaces

4. The interaction between high-level and low-level modules should be:

A) Direct and concrete
B) Static
C) Abstract
D) Hardcoded

Object Oriented Analysis and Design

MCQ SOLUTION

1.C
2.C
3.C
4.C

THANK YOU

Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

UE23CS352B - Object Oriented Analysis and Design

Dr. R. Nandhi Kesavan
Computer Science and Engineering

Scenario-based exercise to apply each SOLID principle

PES UNIVERSITY
CELEBRATING 50 YEARS

Online Food ordering

- A food delivery platform allows users to **order food online** from multiple restaurants. It includes the following key features:

- ❑ **User Management** (Register/Login, Address Management)
- ❑ **Restaurant Management** (Menu, Pricing, Availability)
- ❑ **Order Processing** (Order Placement, Payment, Delivery Tracking)

- Now, let's **apply each SOLID principle** to design the system.

Single Responsibility Principle (SRP)

Problem:

A single Order class is responsible for multiple tasks:

- Managing order details (items, price, restaurant)
- Processing payments
- Sending order confirmation notifications

Single Responsibility Principle (SRP)

Solution:

```
class Order {
    private List<Item> items;
    private double totalPrice;

    public void addItem(String name,
int price) {
        /* Add item logic */
    }

    public double calculateTotal() {
        /* Calculate total price */
        return totalPrice;
    }
}
```

```
class PaymentProcessor {
    public void processPayment(Order order,
PaymentMethod method) {
        /* Payment logic */
    }
}

class NotificationService {
    public void sendOrderConfirmation(User
user, Order order) {
        /* Send email/SMS */
    }
}
```

Open-Closed Principle (OCP)

Problem:

The **PaymentProcessor** class has a **processPayment()** method that only supports **Credit Card** payments. If we need to add **PayPal, UPI, or Wallets**, we have to modify the existing class, violating OCP.

Solution:

Use **Polymorphism**—create an interface for payments, and extend it for new payment methods **without modifying existing code**.

Open-Closed Principle (OCP)

```
interface PaymentMethod {
    void pay(double amount);
}

class CreditCardPayment implements PaymentMethod {
    public void pay(double amount) {
        /* Credit Card logic */
    }
}

class PayPalPayment implements PaymentMethod {
    public void pay(double amount) {
        /* PayPal logic */
    }
}

class PaymentProcessor {
    public void processPayment(PaymentMethod method, double amount)
{
        method.pay(amount);
    }
}
```

Liskov Substitution Principle (LSP)

Problem:

We have a DeliveryPerson class that has a deliverOrder() method. Now, we introduce **DroneDelivery** for automated delivery.

If DroneDelivery extends DeliveryPerson, it **inherits methods like assignVehicle() that drones don't need**, leading to incorrect behavior.

Solution:

Use a **common interface** instead of forcing inheritance.

Liskov Substitution Principle (LSP)

```
interface DeliveryService {
    void deliverOrder(Order order);
}

class DeliveryPerson implements DeliveryService {
    public void deliverOrder(Order order) {
        /* Assign person and vehicle */
    }
}

class DroneDelivery implements DeliveryService {
    public void deliverOrder(Order order) {
        /* Autonomous drone delivery */
    }
}
```

Interface Segregation Principle (ISP)

PES UNIVERSITY
CELEBRATING 50 YEARS

Problem:

A RestaurantService interface has methods:

1. addMenuItem()
2. updateStock()
3. generateMonthlyReport()

A **small home-based restaurant** only wants to manage menu items but is **forced to implement** generateMonthlyReport(), which it doesn't need.

Solution:

- Split into smaller interfaces.

Interface Segregation Principle (ISP)

```
interface MenuManagement {
    void addMenuItem(Item item);
    void updateStock(Item item, int quantity);
}

interface Reporting {
    void generateMonthlyReport();
}

class SmallRestaurant implements MenuManagement {
    public void addMenuItem(Item item) {
        /* Add item logic */
    }

    public void updateStock(Item item, int quantity) {
        /* Stock logic */
    }
}
```

Dependency Inversion Principle (DIP)

Problem:

- OrderService directly depends on MySQLDatabase to store orders. If we switch to **MongoDB**, we need to change multiple parts of the system.

Solution:

- Depend on **abstractions** instead of concrete classes.

Dependency Inversion Principle (DIP)

```
interface Database {
    void saveOrder(Order order);
}

class MySQLDatabase implements Database {
    public void saveOrder(Order order) {
        /* MySQL logic */
    }
}

class MongoDBDatabase implements Database {
    public void saveOrder(Order order) {
        /* MongoDB logic */
    }
}
```

Dependency Inversion Principle (DIP)

```
class OrderService {
    private Database database;

    public OrderService(Database database) {
        this.database = database;
    }

    public void placeOrder(Order order) {
        database.saveOrder(order);
    }
}
```

THANK YOU

Dr. R. Nandhi Kesavan
Computer Science and Engineering

Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

Introduction To Design Patterns

Acknowledgements:

Dr. Sudeepa Roy Dey and Dr Geetha D

Dept. of Computer Science and Engineering

Agenda

- The Beginning of Design Patterns
- Design Challenges
- What is a design pattern?
- What is the advantage of knowing/using design patterns?

The Beginning of Patterns

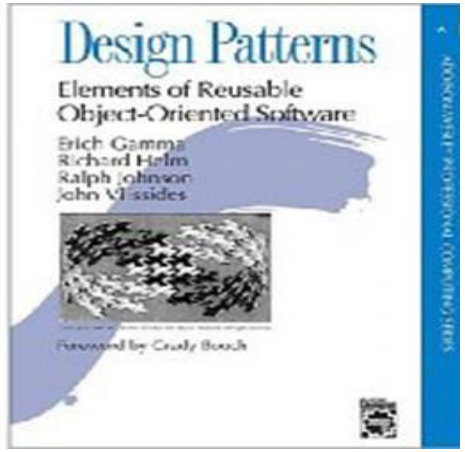
Christopher Alexander, architect

- A Pattern Language--Towns, Buildings, Construction
- Timeless Way of Building (1979)
- "Each pattern describes a *problem* which occurs over and over again in our environment, and then describes the core of the *solution* to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice."

"Gang of Four" (GoF) Book

Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley Publishing Company, 1994 Written by this "gang of four"

- Dr. Erich Gamma, then Software Engineer, Taligent, Inc.
- Dr. Richard Helm, then Senior Technology Consultant, DMR Group
- Dr. Ralph Johnson, then and now at University of Illinois, Computer Science Department
- Dr. John Vlissides, then a researcher at IBM



"Gang of Four" (GoF) Book

PES UNIVERSITY

Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley

This book defined 23 patterns in three categories

- Creational patterns deal with the process of object creation
- Structural patterns, deal primarily with the static composition and structure of classes and objects
- Behavioral patterns, which deal primarily with dynamic interaction among classes and objects

Design challenges

- Designing software for reuse is hard. One must find:

i. a good problem decomposition, and the right software

ii. a design with flexibility, modularity and elegance

- Designs often emerge from trial and error
- Successful designs do exist

i. two designs they are almost never identical

ii. they exhibit some recurring characteristics

Can designs be described, codified or standardized?

- this would short circuit the trial and error phase
- produce "better" software faster

Architecture vs Design Patterns

Architecture

- High-level framework for structuring an application
  - "client-server based on remote procedure calls"
  - "abstraction layering"
  - "distributed object-oriented system based on CORBA"
- Defines the system in terms of computational components & their interactions

Design Patterns

- Lower level than architectures (Sometimes, called *micro-architecture*)
- Reusable collaborations that solve subproblems within an application
  - how can I decouple subsystem X from subsystem Y?

Why Design Patterns?

- Design patterns support *object-oriented reuse* at a high level of abstraction
- Design patterns provide a "framework" that guides and constrains object-oriented implementation

What are Design Patterns?

Design patterns are used to represent some of the best practices adapted by experienced object-oriented software developers. They are like pre-made blueprints that you can customize to solve a recurring design problem in your code.

- Reusable solutions to the problems: In software engineering, a design pattern is general reusable solution to commonly occurring problem in software design.
- Interaction between the objects :Object-oriented design patterns typically show relationships and interactions between classes or objects, without specifying final application classes or objects that are involved.
- Template, not a solution :It is not a finished design that can be transferred directly into code.

- Language independent

Pattern has four Essential Elements

- Pattern Name- is a handle we can use to describe a design problem, its solutions, and consequences in a word or two
- The Problem – describes when to apply the pattern. It explains the problem and its context. It might describe specific design problems such as how to represent algorithms as objects. It might describe class or object structures that are symptomatic of an inflexible design. Sometimes the problem will include a list of conditions that must be met before it makes sense to apply the pattern.
- The Solution - describes the elements that make up the design, their relationships, responsibilities, and collaborations.
- Consequences - are the results and trade-offs of applying the pattern.

Other Elements

- **Applicability -** What are the situations in which the design pattern can be applied?
- **Structure -** A graphical representation of the classes in the pattern using a notation based on the Object Modeling Technique (OMT)
- **Participants -** The classes and/or objects participating in the design pattern and their responsibilities.
- **Collaborations -** How the participants collaborate to carry out their responsibilities.
- **Consequences -** How does the pattern support its objectives? What are the trade-offs and results of using the pattern? What aspect of system structure does it let you vary independently?
- **Implementation -** What pitfalls, hints, or techniques should you be aware of when implementing the pattern? Are there language-specific issues?
- **Related Pattern -** What design patterns are closely related to this one? What are the important differences? With which other patterns should this one be used?

What Does a Pattern consists of?

Most patterns are described very formally so people can reproduce them in many contexts. Here are the sections that are usually present in a pattern description:

- **Pattern Name and Classification:** The pattern's name conveys the essence of the pattern succinctly. A good name is vital, because it will become part of your design vocabulary
- **Intent :**of the pattern briefly describes both the problem and the solution.
- **Motivation:** further explains the problem and the solution the pattern makes possible.
- **Structure of classes:** shows each part of the pattern and how they are related.
- **Code example:** in one of the popular programming languages makes it easier to grasp the idea behind the pattern.

Why we should learn Patterns?

❑ **Patterns are a common design vocabulary**

- Allows engineers to abstract a problem and talk about that abstraction in isolation from its implementation
- Embodies a culture; domain-specific patterns increase design speed

❑ **Patterns capture design expertise and allow that expertise to be communicated**

- promotes design reuse and avoid mistakes

❑ **Improve documentation (less is needed) and understandability (patterns are described well once)**

Pros of Design Patterns

Pros

- Add consistency to designs by solving similar problems the same way, independent of language
- Add clarity to design and design communication by enabling a common vocabulary
- Improve time to solution by providing templates which serve as foundations for good design
- Improve reuse through composition

Cons of Design Patterns

Cons

- Some patterns come with negative consequences (i.e. object proliferation, performance hits, additional layers)
- Consequences are subjective depending on concrete scenarios
- Patterns are subject to different interpretations, misinterpretations, and philosophies
- Patterns can be overused and abused--□ Anti-Patterns

Selection of a Design Pattern

As there are 23 patterns in the catalog it will be difficult to choose a design pattern. It might become hard to find the design pattern that solves our problem, especially if the catalog is new and unfamiliar to you. Below is a list of approaches we can use to choose the appropriate design pattern:

- **Consider how design patterns solve design problems**: Considering how design patterns help you find appropriate objects, determine object granularity, specify object interfaces and several other ways in which design patterns solve the problems will let you choose the appropriate design pattern.
- **Scan intent sections:** Looking at the intent section of each design pattern's specification lets us choose the appropriate design pattern.
- **Study how patterns interrelate**: The relationships between the patterns will direct us to choose the right patterns or group of patterns.

Selection of a Design Pattern

- **Study patterns of like purpose**: Each design pattern specification will conclude with a comparison of that pattern with other related patterns. This will give you an insight into the similarities and differences between patterns of like purpose.
- **Examine a cause of redesign:** Look at your problem and identify if there are any causes of redesign. Then look at the catalog of patterns that will help you avoid the causes of redesign
- **Consider what should be variable in your design**: Consider what you want to be able to change without redesign. The focus here is on encapsulating the concept that varies.

Usage of Design Pattern-How to use a selected Design Pattern

Below steps will show you how to use a design pattern after selecting one:

- **Read the pattern once through for a overview:**

Give importance to the applicability and consequences sections(in format) to ensure that the pattern is right for your problem.

- **Study the structure, participants and collaborations sections:**

Make sure to understand the classes and objects in the pattern and how they relate to one another.

- **Look at the sample code section to see a concrete example of the pattern in action:**

Studying the code helps us to implement the pattern.

Usage of a Design Pattern-How to use a Design Pattern

- **Define the classes:**

Declare their interfaces, inheritance relationships and instance variables, which represent the data and object references. Identify the affected classes by the pattern and modify them accordingly.

- **Define application-specific names for the operations in the pattern:**

Use the responsibilities and collaborations as a guide to name the operations. Be consistent with the naming conventions.

- **Implement the operations to carry out the responsibilities and collaborations in the pattern:**

The implementation section provides hints to guide us in the implementation. The examples in the sample code section can also help as well.

PES UNIVERSITY

MCQ

1.-Design patterns mainly represent:
A) Programming languages
B) Best practices used by experienced developers
C) Database queries
D) Compiler techniques

2. The famous “Gang of Four” book defined how many design patterns?
A) 15
B) 18
C) 23
D) 30

PES UNIVERSITY

MCQ

3. Creational patterns are concerned with:

A) Program execution
B) Object creation
C) Network communication
D) UI design

4. Structural patterns mainly focus on:

A) Object creation
B) Data storage
C) Compilation
D) Static composition of classes and objects

PES UNIVERSITY

MCQ

5. Behavioral patterns focus on:
A) Dynamic interaction among classes and objects
B) Memory allocation
C) Hardware design
D) Database schemas

6. One disadvantage of design patterns is:
A) They cannot be reused
B) They prevent modular design
C) They eliminate abstraction
D) They may introduce extra complexity or layers

PES UNIVERSITY

MCQ SOLUTION

1.B
2.C
3.B
4.D
5.A
6.D

References

- https://sourcemaking.com/design_patterns/
- https://refactoring.guru/design-patterns/java
- Design Patterns: Elements of Reusable Object-Oriented SoftwareErich Gamma, Ralph Johnson, Richard Helm · 1995

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

Creational Patterns – Singleton

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements:

Prof. Nivedita Kasturi

Dept. of Computer Science and Engineering

Agenda

- ❑ Introduction to creational design patterns
- ❑ Types of creational Design Patterns.
- ❑ Singleton-definition
  - ✓ Motivation
  - ✓ Intent
  - ✓ Problem(use case)
  - ✓ Solution (without singleton and with singleton)
  - ✓ Implementation
  - ✓ Applicability
  - ✓ Consequence

Introduction to Creational Design Pattern

- Creational design patterns provide various object creation mechanisms, which increase flexibility and reuse of existing code.
- The basic form of object creation could result in design problems or added complexity to the design.
- Creational design patterns solve this problem by somehow controlling this object creation.

Recurring themes:

- Encapsulate knowledge about which concrete classes the system uses (so we can change them easily later)
- Hide how instances of these classes are created and put together (so we can change it easily later)

Introduction to Creational Design Pattern

Everyone knows an object is created by using *new* keyword in java.
For example:

```
StudentRecord s1=new StudentRecord();
```

The *new* operator is often considered harmful as it scatters objects all over the application. Over time it can become challenging to change an implementation because classes become tightly coupled.

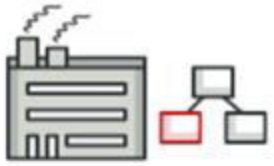
Hard-Coded code is not a good programming approach. Here, we are creating the instance by using the new keyword. Sometimes, the nature of the object must be changed according to the nature of the program. In such cases, we must get the help of creational design patterns to provide more general and flexible approach.

Object Oriented Analysis and Design

Types of Creational Design pattern

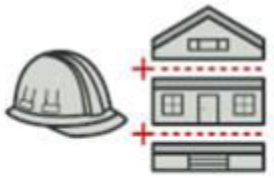
Singleton

Lets you ensure that a class has only one instance, while providing a global access point to this instance.



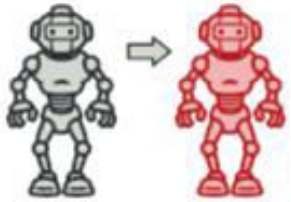
Factory Method

Provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.



Builder

Lets you construct complex objects step by step. The pattern allows you to produce different types and representations of an object using the same construction code.



Prototype

Lets you copy existing objects without making your code dependent on their classes.

Composition over inheritance: "Favor 'object composition' over 'class inheritance'." (Gang of Four 1995:20)

Types of Creational Design pattern: Scope

		Purpose		
		Creational	**Structural**	**Behavioral**
Scope	**Class**	Factory Method (107)	Adapter (class) (139)	Interpreter (243) Template Method (325)
	Object	Abstract Factory (87) Builder (97) Prototype (117) Singleton (127)	Adapter (object) (139) Bridge (151) Composite (163) Decorator (175) Facade (185) Flyweight (195) Proxy (207)	Chain of Responsibility (223) Command (233) Iterator (257) Mediator (273) Memento (283) Observer (293) State (305) Strategy (315) Visitor (331)

Singleton: Class, Object Structural

Motivation

The Singleton is part of the Creational Design Pattern Family .Singleton Design Pattern aims to keep a check on initialization of objects of a particular class by **ensuring that only one instance of the object exists throughout the Java Virtual Machine.**

A Singleton class also provides one unique global access point to the object so that each subsequent call to the access point returns only that particular object.

A real-life example will be, **Imagine you work for a big company that has cloud storage💾 system for storing shared resources of files, images and documents we create shared storage as creating separate cloud storage💾 for every user👨‍💻 may be costly**.

Definition:

The singleton pattern is a design pattern that restricts the instantiation of a class to one object.

PES UNIVERSITY

Why implement Singleton Design Pattern?

1. We can be sure that a class has only a single instance.
2. We gain a global access point to that instance.
3. The Singleton Object is initialized only when it's requested for the first time.

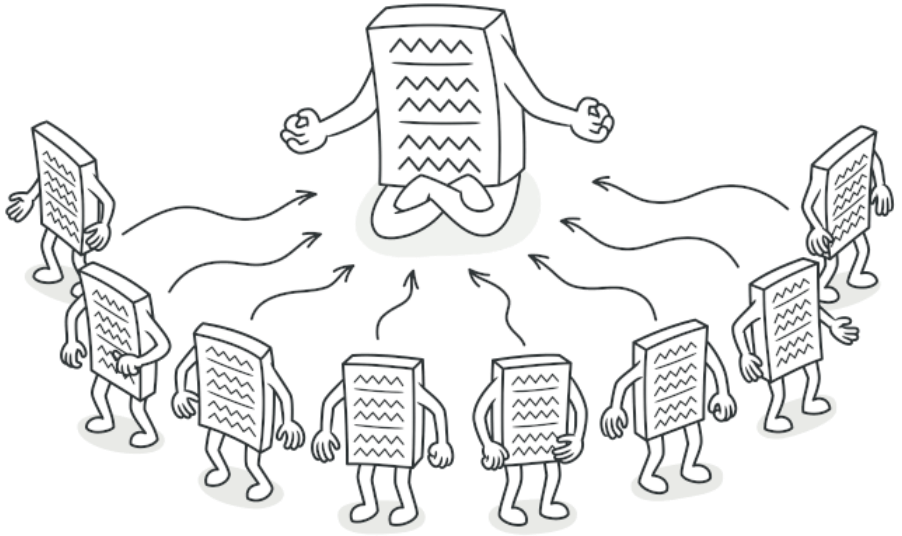
Singleton: Class, Object Structural

Intent

There are only two points in the definition of a singleton design pattern,

1) There should be only one instance allowed for a class and
2) We should allow global point of access to that single instance.

Ensure a class only has one instance, and provide a global point of access to it.



Real-World Analogy

The government is an excellent example of the Singleton pattern.

A country can have only one official government.

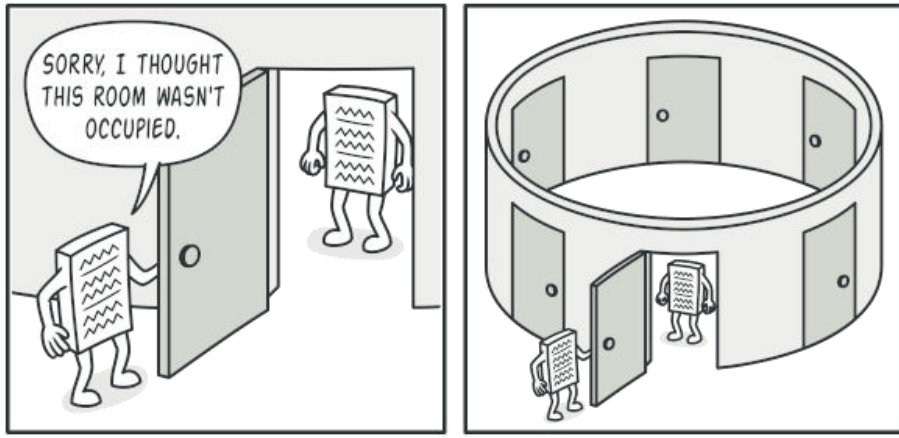
Regardless of the personal identities of the individuals who form governments, the title, “The Government of X”, is a global point of access that identifies the group of people in charge.

Singleton: Class, Object Structural

Problem

The Singleton pattern solves two problems at the same time, violating the Single Responsibility Principle:

1. Ensure that a class has just a single instance. Why would anyone want to control how many instances a class has? The most common reason for this is to control access to some shared resource—for example, a database or a file.



Clients may not even realize that they're working with the same object all the time.

Here's how it works: imagine that you created an object, but after a while decided to create a new one. Instead of receiving a fresh object, you'll get the one you already created.

Note that this behavior is impossible to implement with a regular constructor since a constructor call must always return a new object by design.

Singleton: Class, Object Structural

Problem

2.Provide a global access point to that instance. Remember those global variables that you (all right, me) used to store some essential objects? While they're very handy, they're also very unsafe since any code can potentially overwrite the contents of those variables and crash the app.

Just like a global variable, the Singleton pattern lets you access some object from anywhere in the program. However, it also protects that instance from being overwritten by other code.

There's another side to this problem: you don't want the code that solves problem #1 to be scattered all over your program. It's much better to have it within one class, especially if the rest of your code already depends on it.

Singleton: Class, Object Structural

Solution

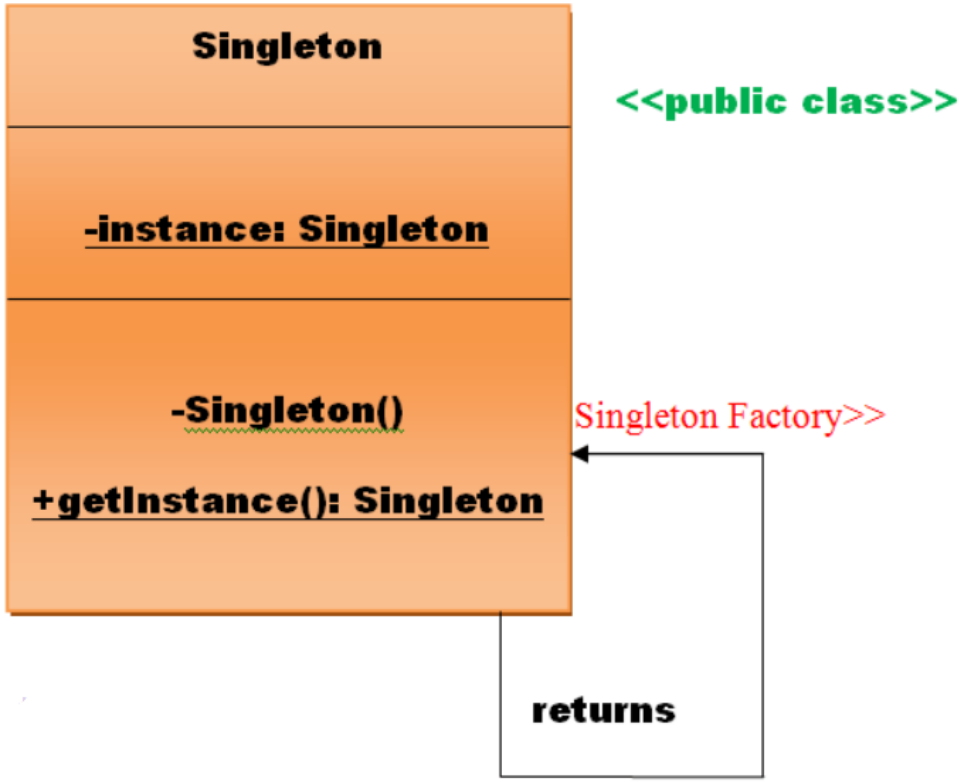
All implementations of the Singleton have these two steps in common:

- Make the default constructor private, to prevent other objects from using the new operator with the Singleton class.
- Create a static creation method that acts as a constructor. Under the hood, this method calls the private constructor to create an object and saves it in a static field. All following calls to this method return the cached object.

If your code has access to the Singleton class, then it's able to call the Singleton's static method. So whenever that method is called, the same object is always returned.

Singleton: Implementation

UML class diagram for the Singleton Pattern



Singleton pattern is used for logging, drivers objects, caching, and thread pool.

Singleton design pattern is also used in other design patterns like Abstract Factory, Builder, Prototype, Facade, etc.

Singleton design pattern is used in core Java classes also (for example, java.lang.Runtime, java.awt.Desktop).

Singleton: Implementation example

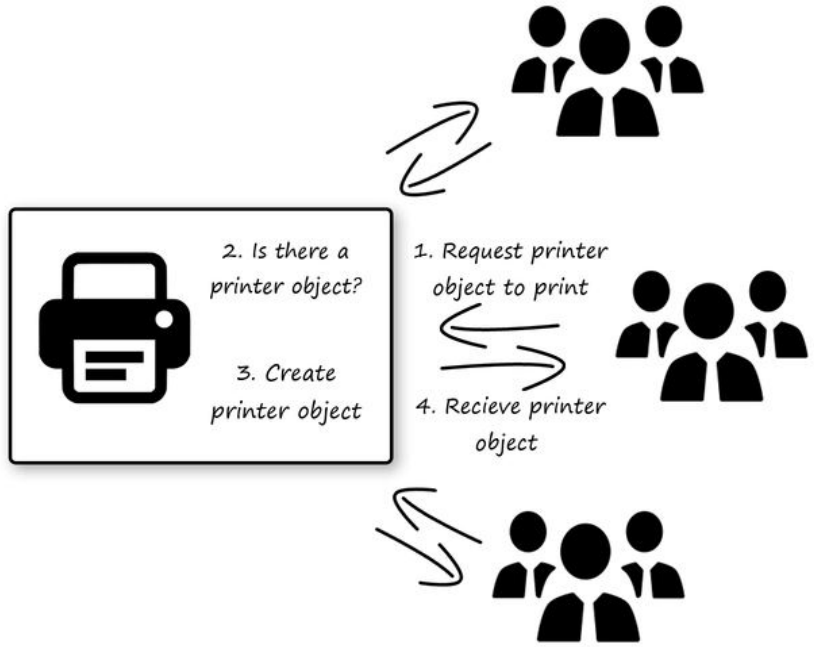
PES UNIVERSITY

❑ **Problem Statement**:

How can a shared office space with many employees, share the same printer. This makes sense, since it is often the reality for corporations and companies with employees sharing office facilities.

Design Solution

- The concept of the singleton design pattern is based around a private constructor and a public static initialization method.
- The private constructor ensures that the class can only be initialized by itself, which makes the public static initialization method the only way of getting an instance of the class.



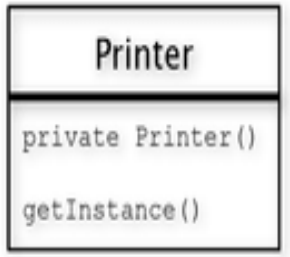
Design Solution

- However, we do not want the method to start a new instance for each time it is called. Therefore, we save the instance of the class in a static field inside its own class.
- Now that we have an instance of the class ready to work with inside the class, we null check on it when we call the static initialization method. If the class has not previously been initialized, then we initialize it, otherwise we return our static field instance.
- This means that there will always only be one instance of the class available, hence the name *singleton* pattern.

Class Diagram (UML)

- In terms of class diagram, the singleton pattern is not worth talking about. As you might have figured out by now, the singleton pattern is based around two methods inside the same class, which makes the class diagram simple to understand.

- Thereby said, that if you understand the concept of the singleton pattern from the previous section, you should be ready to dive into the code.



Code (Java)

Printer

```
public class Printer {
    private static Printer printer;
    private int nrOfPages;
    private Printer() {
    }
    public static Printer getInstance() {
        return printer == null ?
                printer = new Printer() :
                printer;
    }
    public void print(String text){
        System.out.println(text +
                "n" + "Pages printed today " + ++nrOfPages +
                "n" + "---------");
    }
}
```

Employee

```
public class Employee {
    private final String name;
    private final String role;
    private final String assignment;
    public Employee(String name, String role, String assignment) {
        this.name = name;
        this.role = role;
        this.assignment = assignment;
    }
    public void printCurrentAssignment(){
        Printer printer = Printer.getInstance();
        printer.print("Employee: " + name + "n" +
                "Role: " + role + "n" +
                "Assignment: " + assignment + "n");
    }
}
```

Code (Java)

How To Use The Singleton Pattern

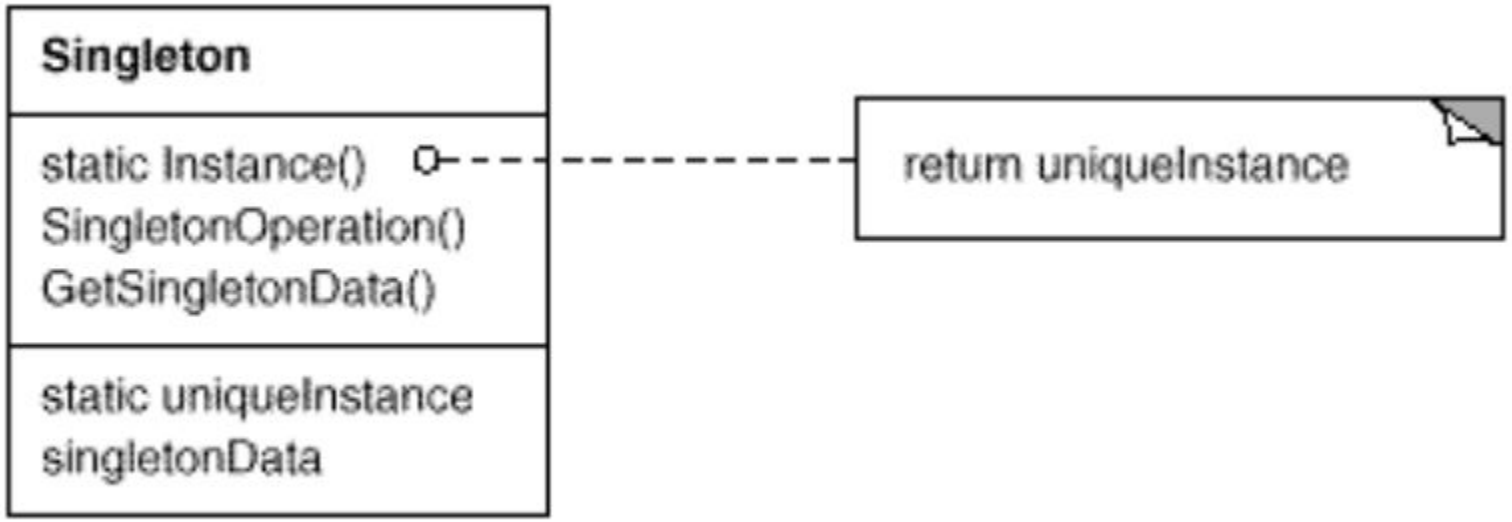
```
public class Main {
    public static void main(String[] args) {

        Employee graham = new Employee("Graham",   "CEO", "Making excutive decisions");
        Employee sara = new Employee("Sara","Consultant",  "Consuting the company");
        Employee tim = new Employee("Tim","Salesmen", "Selling the company's products");
        Employee emma = new Employee("Emma", "Developer", "Developing the latest mobile app.");

        graham.printCurrentAssignment();
        sara.printCurrentAssignment();
        tim.printCurrentAssignment();
        emma.printCurrentAssignment();
    }
}
```

Applicability

Use the Singleton pattern when

- there must be exactly one instance of a class, and it must be accessible to clients from a well-known access point.
- when the sole instance should be extensible by subclassing, and clients should be able to use an extended instance without modifying their code.

Structure

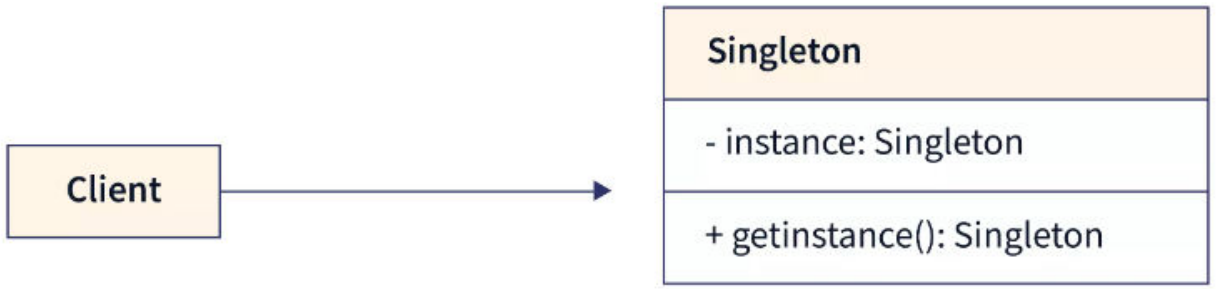

Singleton
static Instance()
SingletonOperation()
GetSingletonData()
static uniqueInstance
singletonData
return uniqueInstance

Participants

❑ Singleton design pattern has two core participants: Singleton and Client.

❑ defines an Instance operation that lets clients access its unique instance. Instance is a class operation (that is, a class method and a static member function).

❑ may be responsible for creating its own unique instance.



Collaborations

Clients access a Singleton instance solely through Singleton's Instance operation.

Consequence

The Singleton pattern has several benefits:

1. Controlled access to sole instance. Because the Singleton class encapsulates its sole instance, it can have strict control over how and when clients access it.

1. Reduced name space. The Singleton pattern is an improvement over global variables. It avoids polluting the name space with global variables that store sole instances.

1. Permits refinement of operations and representation. The Singleton class may be subclassed, and it's easy to configure an application with an instance of this extended class. You can configure the application with an instance of the class you need at run-time.

Consequence

The Singleton pattern has several benefits:

4. Permits a variable number of instances. The pattern makes it easy to change your mind and allow more than one instance of the Singleton class. Moreover, you can use the same approach to control the number of instances that the application uses. Only the operation that grants access to the Singleton instance needs to change

5. More flexible than class operations. Another way to package a singleton's functionality is to use class operations (that is, static member functions or class methods). But both of these language techniques make it hard to change a design to allow more than one instance of a class. Moreover, static member functions in class are compile time, so subclasses can't override them polymorphically.

Implementation

Java Singleton Pattern Implementation

1. Add a private static field to the class for storing the singleton instance.
2. Declare a public static creation method for getting the singleton instance.
3. Implement “lazy initialization” inside the static method. It should create a new object on its first call and put it into the static field. The method should always return that instance on all subsequent calls.
4. Make the constructor of the class private. The static method of the class will still be able to call the constructor, but not the other objects.
5. Go over the client code and replace all direct calls to the singleton’s constructor with calls to its static creation method.

Implementation

various design options for implementing Singleton:

Method 1: lazy instantiation

```
// Classical Java implementation of singleton design pattern
class Singleton
{
private static Singleton obj;

// private constructor to force use of
// getInstance() to create Singleton object
private Singleton() {}

public static Singleton getInstance()
{
        if (obj==null)
        obj = new Singleton();
        return obj;
}
}
```

- Here we have declared getInstance() static so that we can call it without instantiating the class.
- The first time getInstance() is called it creates a new singleton object and after that it just returns the same object.
- Note that Singleton obj is not created until we need it and call getInstance() method. This is called lazy instantiation.

Implementation

The main problem with above method is that it is not thread safe. Consider the following execution sequence.

Thread one

```
public static Singleton getInstance(){
if(obj==null)

obj=new Singleton();
return obj;
}
```

Thread two

```
public static Singleton getInstance(){
if(obj==null)


obj=new Singleton();
return obj;
}
```

This execution sequence creates two objects for singleton. Therefore this classic implementation is not thread safe.

Implementation

Method 2: make getInstance() synchronized

```
// Thread Synchronized Java implementation of
// singleton design pattern
class Singleton
{
    private static Singleton obj;

    private Singleton() {}

    // Only one thread can execute this at a time
    public static synchronized Singleton getInstance()
    {
        if (obj==null)
            obj = new Singleton();
        return obj;
    }
}
```

- Here using synchronized makes sure that only one thread at a time can execute getInstance().
- The main disadvantage of this is method is that using synchronized every time while creating the singleton object is expensive and may decrease the performance of your program.
- However if performance of getInstance() is not critical for your application this method provides a clean and simple solution.

Implementation

Method 3: Eager Instantiation

```
// Static initializer based Java implementation of
// singleton design pattern
class Singleton
{
    private static Singleton obj = new Singleton();

    private Singleton() {}

    public static Singleton getInstance()
    {
        return obj;
    }
}
```

- Here we have created instance of singleton in static initializer.
- JVM executes static initializer when the class is loaded and hence this is guaranteed to be thread safe.
- Use this method only when your singleton class is light and is used throughout the execution of your program.

Implementation

Method 4 (Best): Use "Double Checked Locking"

- By considering method 2,once an object is created , the synchronization is no longer useful because obj will not be null and any sequence of operations will lead to consistent results.
- So we will only acquire lock on the getInstance() once, when the obj is null.
- This way we only synchronize just what we want.

```
// Double Checked Locking based Java implementation of
// singleton design pattern
class Singleton
{
    private static volatile Singleton obj  = null;


    private Singleton() {}


    public static Singleton getInstance()
    {
        if (obj == null)
        {
            // To make thread safe
            synchronized (Singleton.class)
            {
                // check again as multiple threads
                // can reach above step
                if (obj==null)
                    obj = new Singleton();
            }
        }
        return obj;
    }
}
```

Implementation

- We have declared the obj volatile which ensures that multiple threads offer the obj variable correctly when it is being initialized to Singleton instance.

- This method drastically reduces the overhead of calling the synchronized method every time.

Pros and Cons

PROS

1. You can be sure that a class has only a single instance.
2. You gain a global access point to that instance.
3. The singleton object is initialized only when it's requested for the first time.

CONS

1. Violates the Single Responsibility Principle. The pattern solves two problems at the time.
2. The Singleton pattern can mask bad design, for instance, when the components of the program know too much about each other.
3. The pattern requires special treatment in a multithreaded environment so that multiple threads won't create a singleton object several times.
4. It may be difficult to unit test the client code of the Singleton because many test frameworks rely on inheritance when producing mock objects.

Relations with Other Patterns

- A **Facade** class can often be transformed into a **Singleton** since a single facade object is sufficient in most cases.

- **Flyweight** would resemble **Singleton** if you somehow managed to reduce all shared states of the objects to just one flyweight object. But there are two fundamental differences between these patterns:
  - There should be only one Singleton instance, whereas a Flyweight class can have multiple instances with different intrinsic states.
  - The Singleton object can be mutable. Flyweight objects are immutable.

- **Abstract Factories**, **Builders** and **Prototypes** can all be implemented as **Singletons**.
  - Facade objects are often Singletons because only one Facade object is required.

Demo

PES UNIVERSITY

DEMO

PES UNIVERSITY

MCQs

Q1. Which of the following is the primary purpose of the Singleton design pattern?

A. To allow multiple instances of a class

B. To restrict a class to only one instance and provide global access

C. To improve code readability

D. To separate interface from implementation

Q2. Which keyword is essential in implementing Double-Checked Locking correctly?

A. static

B. final

C. volatile

D. transient

MCQs

Q3. What is the role of a private constructor in Singleton?

A. To allow subclassing

B. To prevent object creation from outside the class

C. To improve performance

D. To enable inheritance

Q4. What is a major drawback of using Singleton excessively?

A. Increased memory usage

B. Difficulty in unit testing due to tight coupling

C. Slower execution speed

D. Lack of encapsulation

MCQs - Answers

Q1. B
Q2. C
Q3. B
Q4. B

References

Text Reference

- Design Patterns: Elements of Reusable Object-Oriented Software, GOF

Web Reference

https://integu.net/singleton-pattern/
https://refactoring.guru/design-patterns/singleton
https://www.geeksforgeeks.org/singleton-design-pattern/
https://refactoring.guru/design-patterns/singleton/java/example
https://sourcemaking.com/design_patterns/singleton

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design

UE23CS352B

Prof Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

OO Design - Creational Design Patterns

Factory

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements:
Prof. Nivedita Kasturi
Dept. of Computer Science and Engineering

Agenda

- ❑ Factory-definition
- ✓ Motivation
- ✓ Intent
- ✓ Implementation
- ✓ Applicability
- ✓ Structure-Consequence
- ✓ Issues

PES UNIVERSITY

Introduction

The Factory Design Pattern or Factory Method Design Pattern is one of the most used design patterns in Java.

According to GoF, this pattern **"defines an interface for creating an object, but let subclasses decide which class to instantiate.** The Factory method lets a class defer instantiation to subclasses".

This pattern delegates the responsibility of initializing a class from the client to a particular factory class by creating a type of virtual constructor.

To achieve this, we rely on a factory which provides us with the objects, hiding the actual implementation details. The created objects are accessed using a common interface.

Also Known As Virtual Constructor

Factory: Class, Object Structural

PES UNIVERSITY

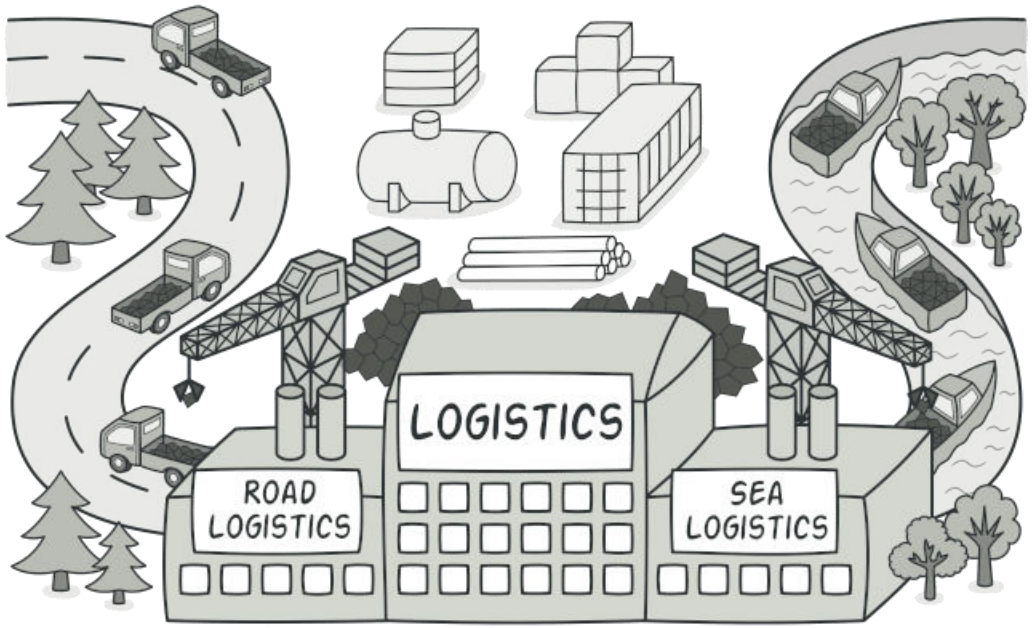
Motivation

- To deal with the problem of creating objects without having to specify the exact class of the object that will be created.
- Creating an object often requires complex processes not appropriate to include within a composing object.
- The object's creation may lead to a significant duplication of code, may require information not accessible to the composing object, may not provide a sufficient level of abstraction, or may otherwise not be part of the composing object's concerns.
- The factory method design pattern handles these problems by defining a separate method for creating the objects, which subclasses can then override to specify the derived type of product that will be created.

Factory: Class, Object Structural

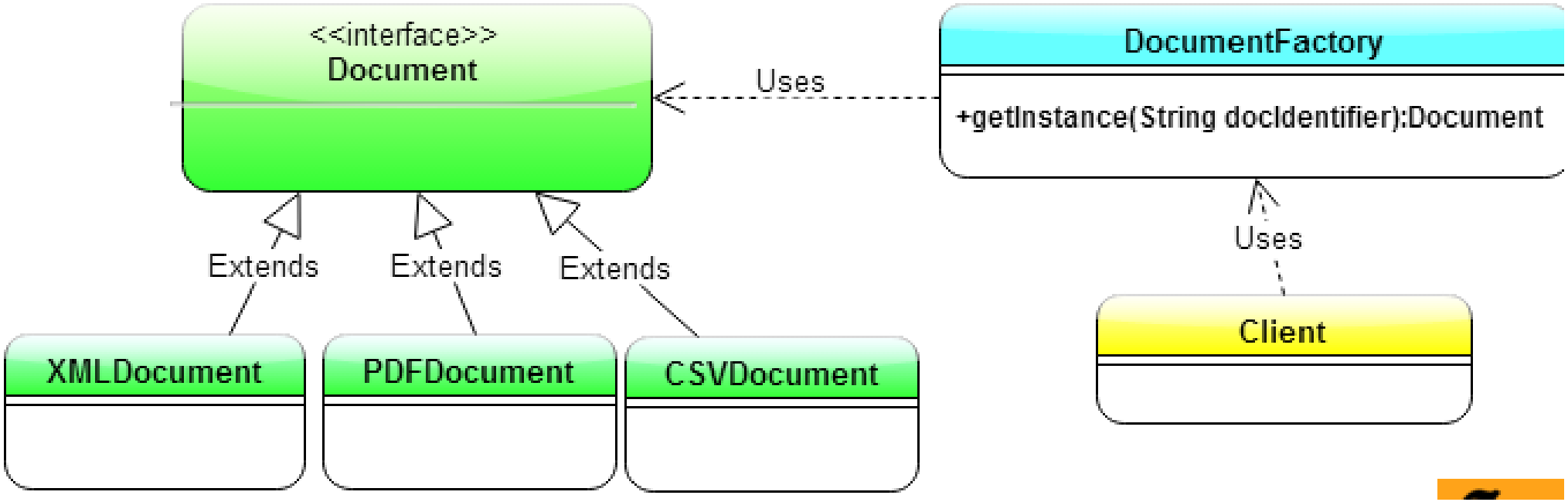
Intent

Factory Method is a creational design pattern that provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.



Factory: Implementation

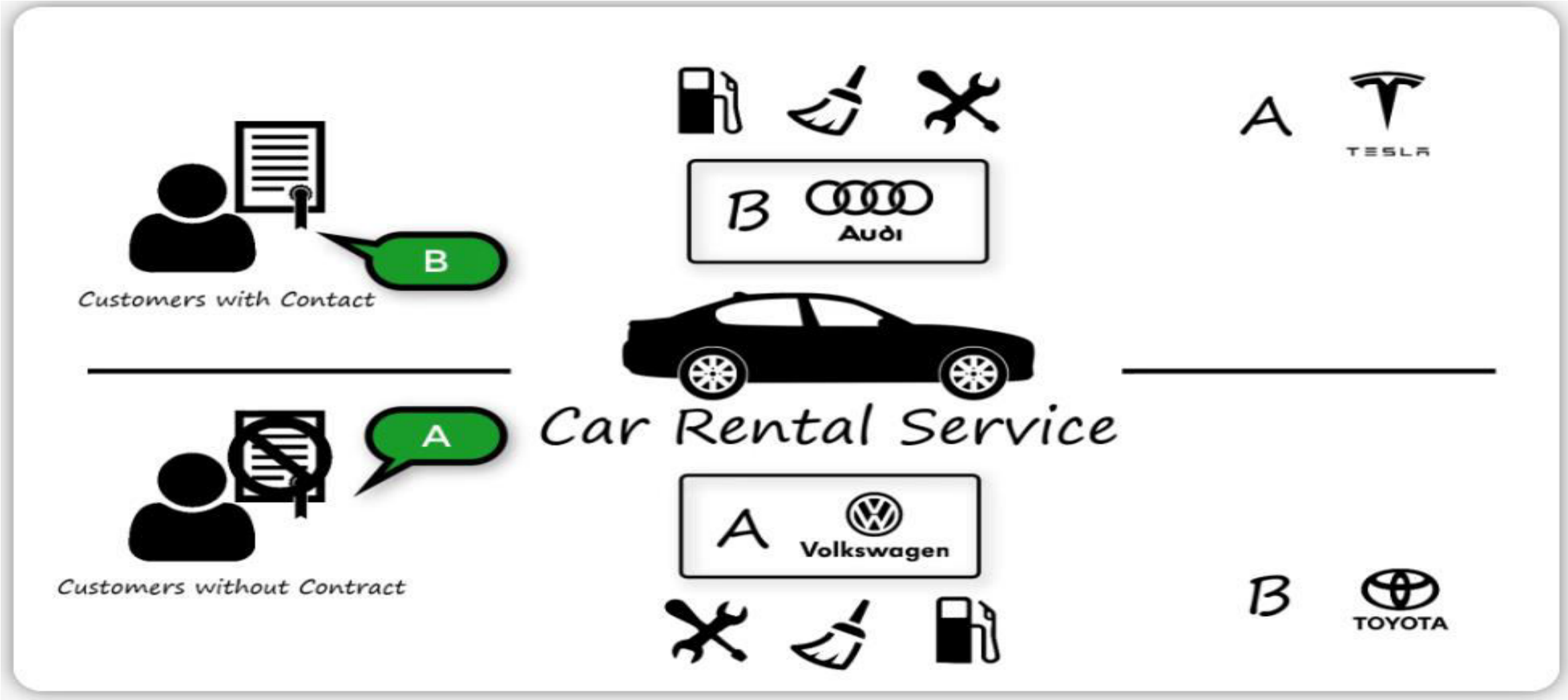
UML class diagram for the Factory Pattern



Object Oriented Analysis and Design

Factory: Implementation example-1

Problem Statement:



Solution

Open Creation

As a beginning, we start by checking whether our customer has a company contract. If so, we retrieve their requested grade of car and runs through the options to see if we have a match.

For this scenario, we have already a Car super-class, which can be extended by all car brands sub-classes (Tesla, Audi, Volkswagen, and Toyota). This allows us to utilize the common methods of the Car super-class to perform a mechanical service check, clean up the car, and put fuel on it.

All of the above mentioned steps have all be accomplished inside the main-method of our system, as can be seen from the code sample below.

Solution

Although this code sample solves the objective, it has multiple problems.

Firstly, it is not desirable to have all the car selection and preparation logics at the same level of abstraction as where the customer is. We want that kind of logic hidden away from the customer so that they only see the ready and shiny car. In code practice we also want this logic hidden away in its own class, to avoid modifications, while working on other parts of the main-method.

Secondly, we also want each contract type to be handled individually. Let's say that the rental service one day shut down their company contract agreement. In this case, we want to easily be able to remove the logics related to the company contracts, without jeopardizing changes anywhere else in the system. With these issues in mind, let's see what the factory pattern can do for us.

Solution

1. Creation of the customer- and car object should not be associated

2. Each contract type should be handled individually

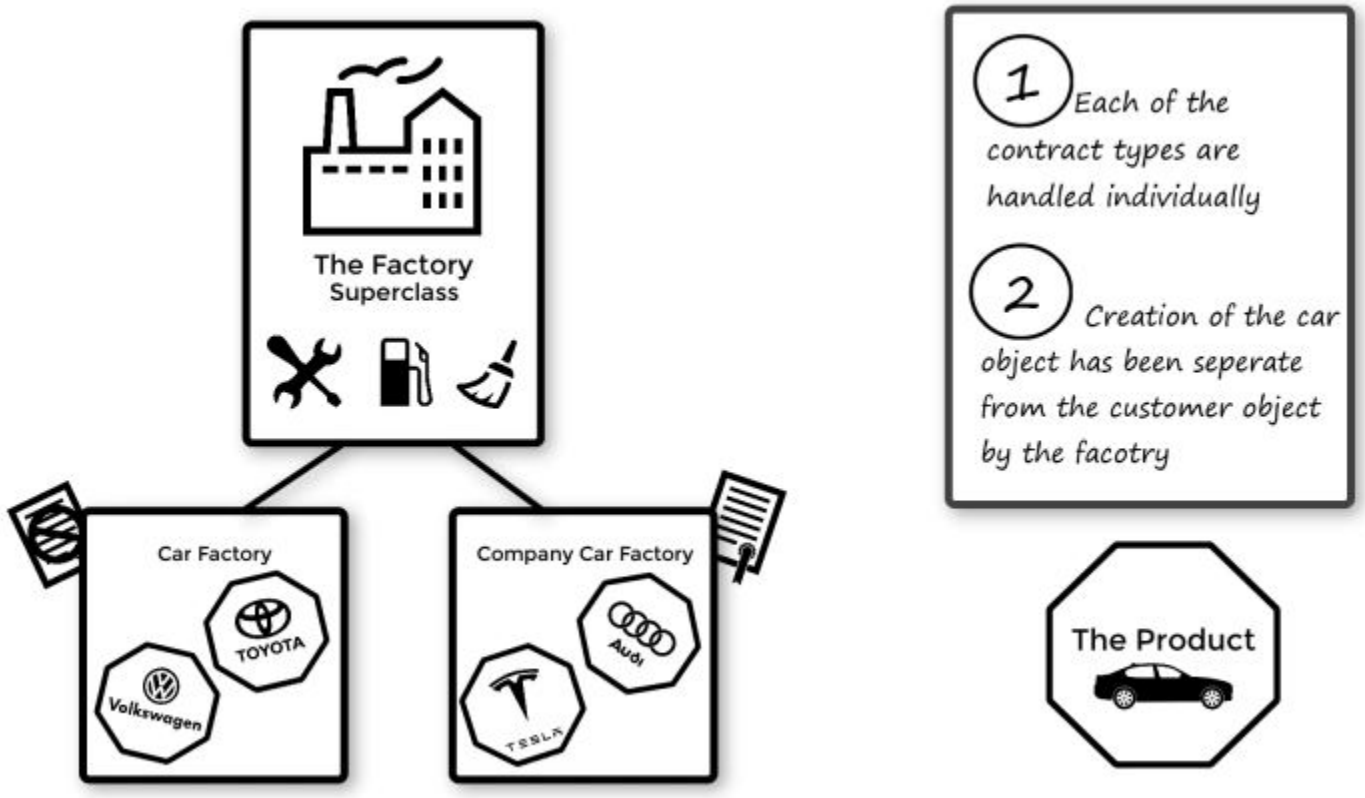
Solution

```
public static void main(String[] args) {

    Customer customerOne = new Customer("B", true);
    Car carOne;

    if (customerOne.hasCompanyContract()) {

        switch (customerOne.getGradeRequest()) {
            case "A":
                carOne = new Tesla();
                break;

            case "B":
                carOne = new Audi();
                break;

            default:
                System.out.println("The requested car was
unfortunately not available.");
                carOne = null;
        }
```

```
    } else {

        switch (customerOne.getGradeRequest()) {
            case "A":
                carOne = new Volkswagen();
                break;

            case "B":
                carOne = new Toyota();
                break;

            default:
                System.out.println("The requested car was
unfortunately not available.");
                carOne = null;
        }
    }

    carOne.clean();
    carOne.mechanicCheck();
    carOne.fuelCar();
    carOne.startEngine();
}
```

Solution

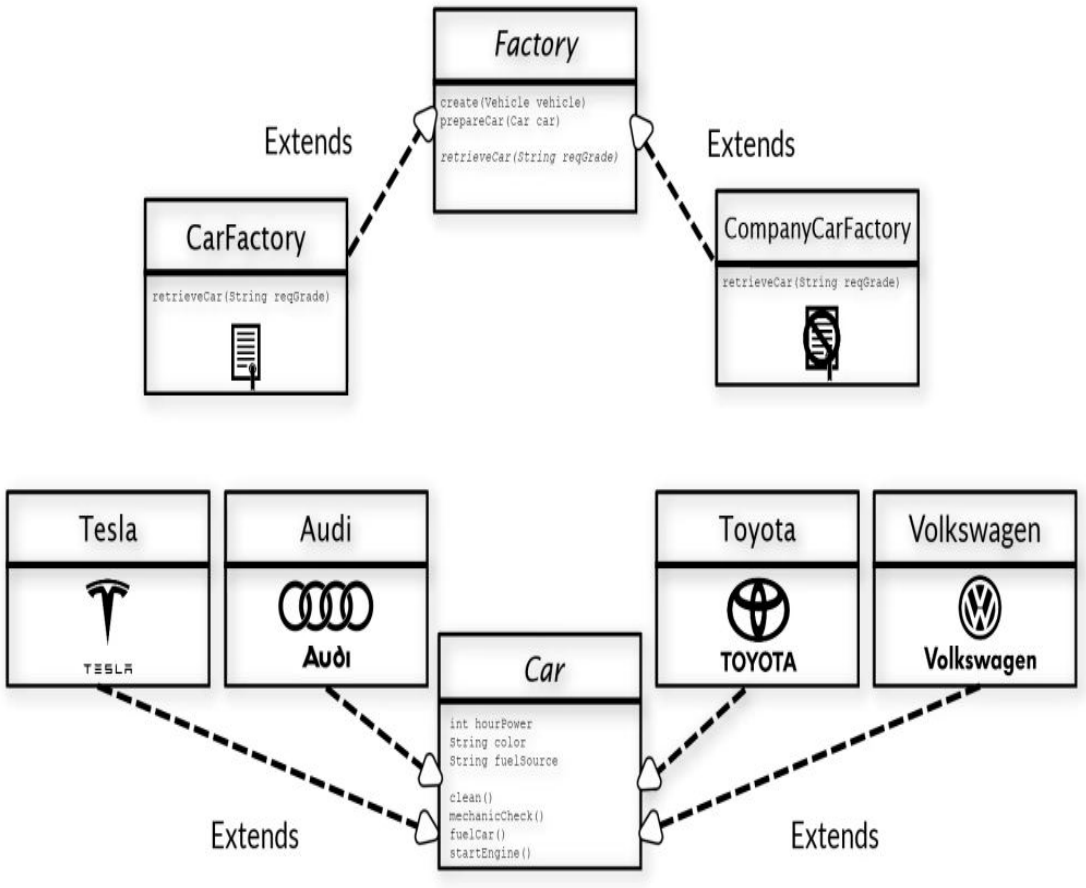
Concept Of The Factory Pattern



The core concept of the factory pattern is to make a class (the factory), which defines the rules of how to create a specific type of object (the product). The pattern can always be recognized by the create-method.

Solution

Class Diagram (UML)



- The Factory super-class contains the **create-method,** which is close to obligatory for any factory pattern. Within this method the initial step is to retrieve the requested car. However, since the super-class does not know anything about the selection logic, *it simply states the retrieveCar-method as abstract.*
- Thereby it requires any class, which extends it, must override the retrieveCar-method and implement its own selection logic.
- After having retrieved the car, we come back to the create-method. The remaining part of the method then goes into preparation procedures. This procedure is the same for all cars and we can, therefore, state the steps within the factory super-class.

Solution

PES UNIVERSITY

DEMO

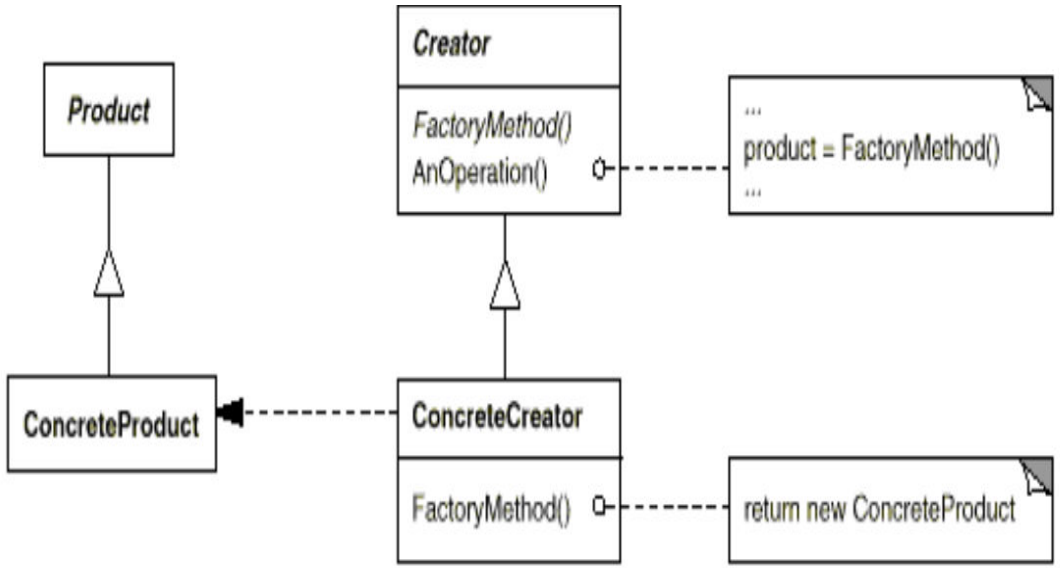
Applicability

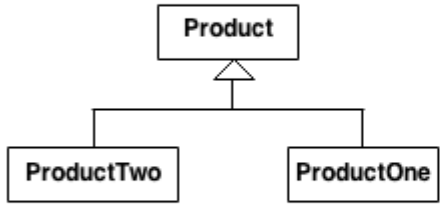
Use the Factory pattern when

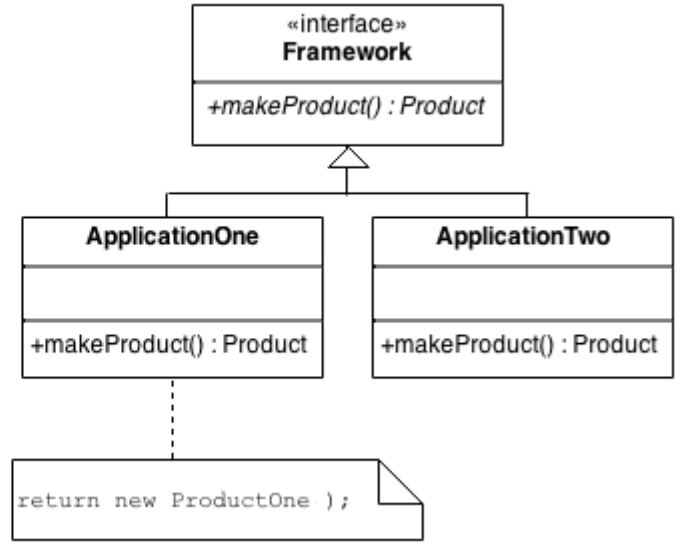
- a class can't anticipate the class of objects it must create.
- a class wants its subclasses to specify the objects it creates.
- classes delegate responsibility to one of several helper subclasses, and you want to localize the knowledge of which helper subclass is the delegate.

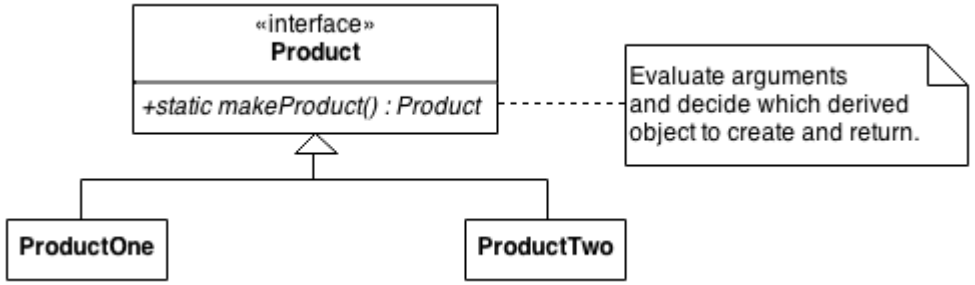
Object Oriented Analysis and Design

Structure









Participants

- Product -defines the interface of objects the factory method creates.
- ConcreteProduct - implements the Product interface.
- Creator - declares the factory method, which returns an object of type Product.Creator may also define a default implementation of the factory method that returns a default ConcreteProduct object.

  - may call the factory method to create a Product object.

- ConcreteCreator -overrides the factory method to return an instance of a ConcreteProduct.

Collaboration

Creator relies on its subclasses to define the factory method so that it returns an instance of the appropriate ConcreteProduct.

Consequence

1. Provides hooks for subclasses. Creating objects inside a class with a factory method is always more flexible than creating an object directly. Factory Method gives subclasses a hook for providing an extended version of an object.

2. Connects parallel class hierarchies. In the examples we've considered so far, the factory method is only called by Creators. But this doesn't have to be the case; clients can find factory methods useful, especially in the case of parallel class hierarchies.

Parallel class hierarchies result when a class delegates some of its responsibilities to a separate class. Consider graphical figures that can be manipulated interactively; that is, they can be stretched, moved, or rotated using the mouse. Implementing such interactions isn't always easy. It often requires storing and updating information that records the state of the manipulation at a given time.

Pros and Cons

Pros and Cons

- ✓ You avoid tight coupling between the creator and the concrete products.
- ✓ *Single Responsibility Principle*. You can move the product creation code into one place in the program, making the code easier to support.
- ✓ *Open/Closed Principle*. You can introduce new types of products into the program without breaking existing client code.
- ✗ The code may become more complicated since you need to introduce a lot of new subclasses to implement the pattern. The best case scenario is when you're introducing the pattern into an existing hierarchy of creator classes.

Relationship with other pattern

⇄ Relations with Other Patterns

- Many designs start by using **Factory Method** (less complicated and more customizable via subclasses) and evolve toward **Abstract Factory**, **Prototype**, or **Builder** (more flexible, but more complicated).
- **Abstract Factory** classes are often based on a set of **Factory Methods**, but you can also use **Prototype** to compose the methods on these classes.
- You can use **Factory Method** along with **Iterator** to let collection subclasses return different types of iterators that are compatible with the collections.
- **Prototype** isn't based on inheritance, so it doesn't have its drawbacks. On the other hand, *Prototype* requires a complicated initialization of the cloned object. **Factory Method** is based on inheritance but doesn't require an initialization step.
- **Factory Method** is a specialization of **Template Method**. At the same time, a *Factory Method* may serve as a step in a large *Template Method*.

Issues to consider when using the Factory pattern

1.**Two major varieties**. The two main variations of the Factory Method pattern are (1) the case when the Creator class is an abstract class and does not provide an implementation for the factory method it declares, and (2) the case when the Creator is a concrete class and provides a default implementation for the factory method. It's also possible to have an abstract class that defines a default implementation, but this is less common. The first case requires subclasses to define an implementation, because there's no reasonable default.

2.**Parameterized factory methods**. Another variation on the pattern lets the factory method create multiple kinds of products. The factory method takes a parameter that identifies the kind of object to create. All objects the factory method creates will share the Product interface.

3.**Language-specific** variants and issues. Different languages lend themselves to other interesting variations and caveats. Smalltalk programs often use a method that returns the class of the object to be instantiated. A Creator factory method can use this value to create a product, and a ConcreteCreator may store or even compute this value. The result is an even later binding for the type of ConcreteProduct to be instantiated.

Demo

DEMO -

PES UNIVERSITY

MCQ

1. What is the main purpose of the Factory Method pattern?
A. To hide database queries
B. To define an interface for object creation while allowing subclasses to decide the class to instantiate
C. To increase coupling
D. To remove inheritance

2. Factory Method is also known as:
A. Static Constructor
B. Virtual Constructor
C. Smart Constructor
D. Dynamic Builder

MCQ

3. In Factory pattern, the client accesses objects using:

A. Direct constructor calls
B. Global objects
C . Static classes
D. A common interface

4. Factory pattern mainly helps to:

A. Increase complexity
B. Hide object creation logic
C. Remove abstraction
D. Avoid inheritance

PES UNIVERSITY

MCQ

5. Factory Method improves flexibility by:
A. Removing classes
B. Delegating object creation to subclasses
C. Using static variables
D. Avoiding interfaces

6. The factory method usually returns:
A. Objects of Product interface
B. Primitive data types
C. Only concrete classes
D. Database connections

PES UNIVERSITY

MCQ SOLUTION

1.B
2.B
3.D
4.B
5.B
6.A

References

PES UNIVERSITY

Text Reference

- Design Patterns: Elements of Reusable Object-Oriented Software, GOF

Web Reference

https://www.tutorialspoint.com/design_pattern/factory_pattern.htm
https://www.javatpoint.com/factory-method-design-pattern
https://refactoring.guru/design-patterns/factory-method
https://www.geeksforgeeks.org/factory-method-design-pattern-in-java/
https://www.digitalocean.com/community/tutorials/factory-design-pattern-in-java
https://www.baeldung.com/java-factory-pattern
https://sourcemaking.com/design_patterns/factory_method
https://www.scaler.com/topics/factory-design-pattern-in-java/

THANK YOU

Prof Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

OO Design – Creational Design Patterns

Builder

Prof Shridevi A Sawant
Department of Computer Science and Engineering

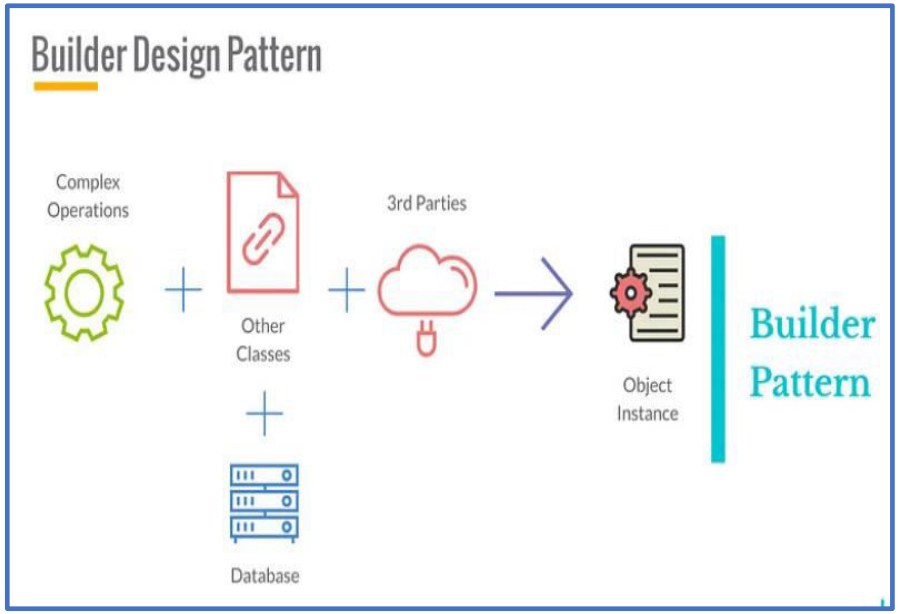
Acknowledgements:
Prof. Nivedita Kasturi
Dept. of Computer Science and Engineering

Agenda

- ❑ Builder-definition
- ✓ Motivation
- ✓ Intent
- ✓ Implementation
- ✓ Applicability
- ✓ Structure-Consequence
- ✓ Issues

Introduction

- **Builder Design Pattern** as it was intended implies that a sequence of complex operations is needed in order to **produce an object instance**.
- The idea is that we delegate the construction of an object to a specialized class, which is aware of the complex process and logic required to build that specific instance.
- This helps us create Single Responsibility classes for complex object creation while at the same time ensuring separation of object creation from business logic.
- Based on the nature of the application, Builder implementations might lead to code re-usability, reducing the code base and improving SOLID compliance of our code.



Builder

Definition:

The Builder Design Pattern is another creational pattern designed to deal with the construction of comparatively complex objects.

When the complexity of creating object increases, the Builder pattern can separate out the instantiation process by using another object (a builder) to construct the object.

This builder can then be used to create many other similar representations using a simple step-by-step approach.

Why Builder Pattern?

The Builder design pattern solves problems like:

- How can a class (the same construction process) create different representations of a complex object?
- How can a class that includes creating a complex object be simplified?

The Builder design pattern describes how to solve such problems:

- Encapsulate creating and assembling the parts of a complex object in a separate Builder object.
- A class delegates object creation to a Builder object instead of creating the objects directly.

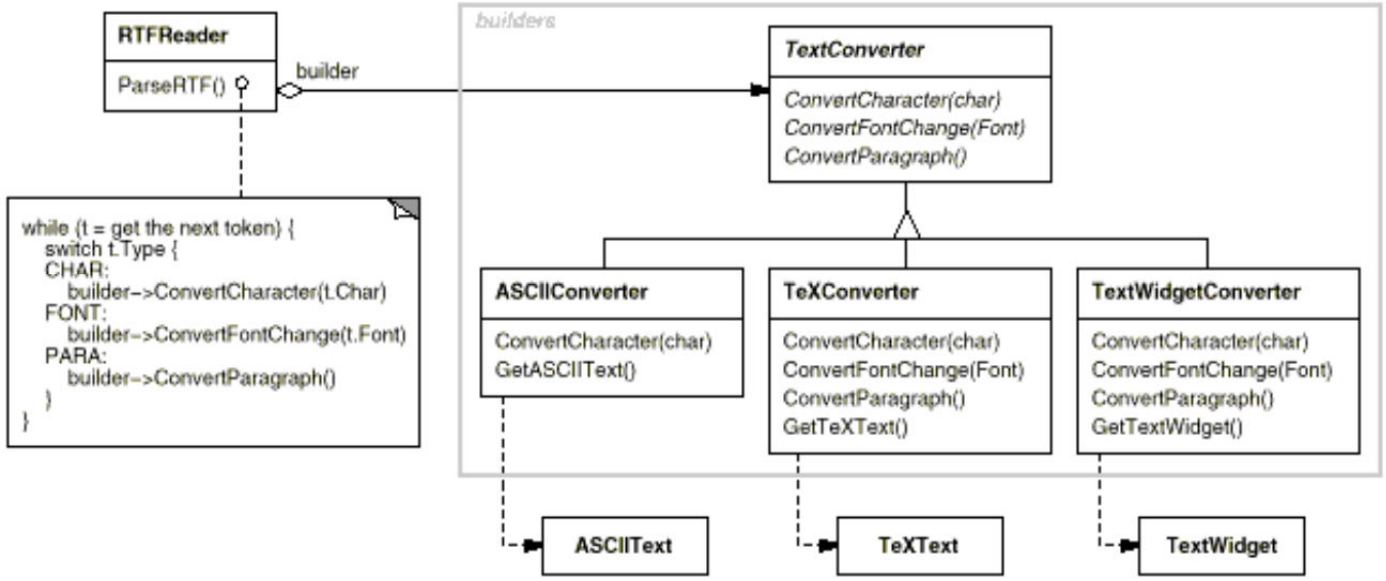
A class (the same construction process) can delegate to different Builder objects to create different representations of a complex object.

Builder : Class, Object Structural

Motivation

- A reader for the RTF (Rich Text Format) document exchange format should be able to convert RTF to many text formats. The reader might convert RTF documents into plain ASCII text or into a text widget that can be edited interactively.
- A **solution** is to configure the RTFReader
- The **problem**, however, is that the number of possible conversions is open-ended. So it should be easy to add a new conversion without modifying the reader.
- class with a TextConverter object that converts RTF to another textual representation.

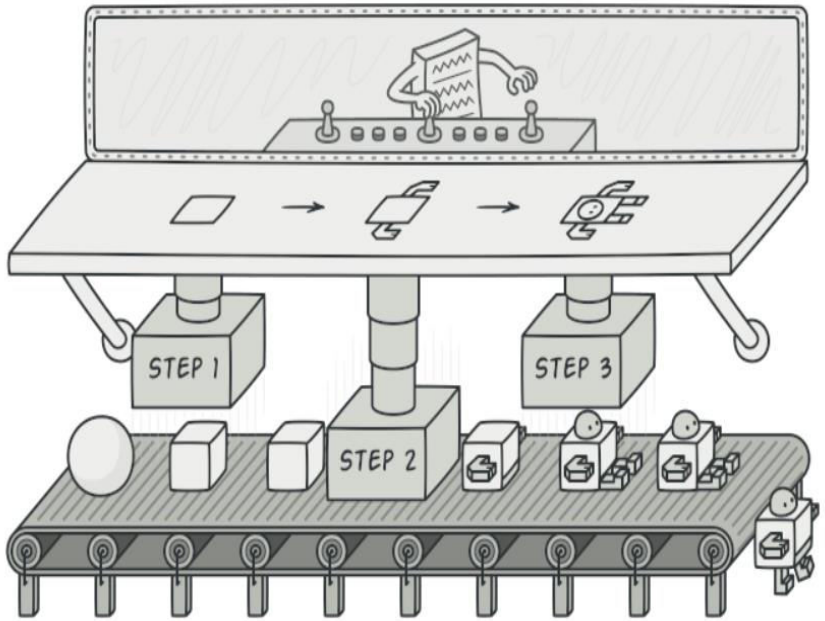
- The Builder pattern captures all these relationships. **Each converter class is called a builder** in the pattern, and the **reader is called the director.**



Builder : Class, Object Structural

Intent

Separate the construction of a complex object from its representation so that the same construction process can create different representations

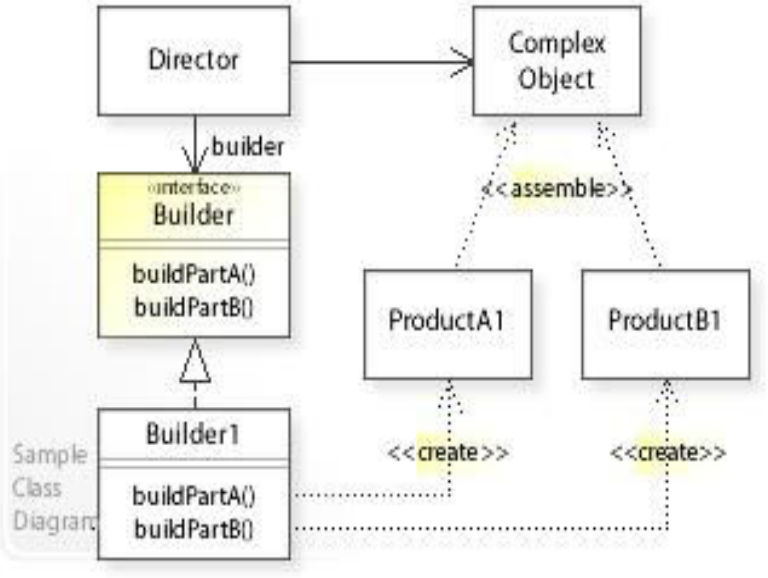


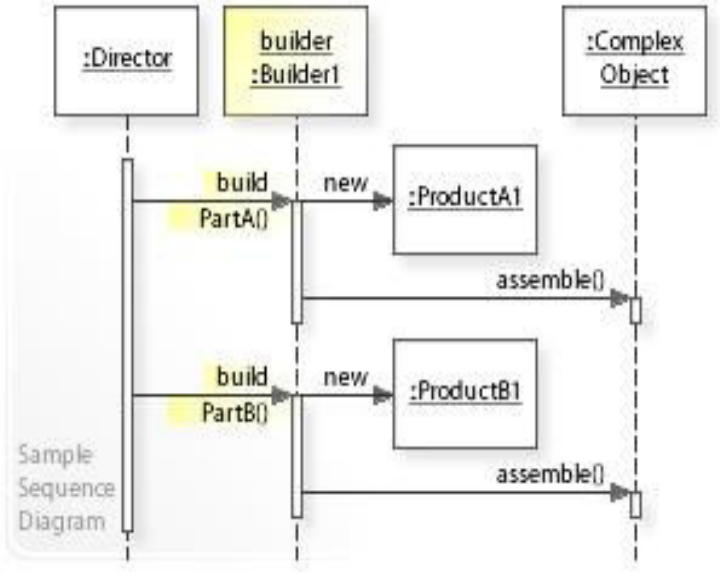
Think of a car factory

Boss tells workers (or robots) to build each part of a car
Workers build each part and add them to the car being constructed

Builder : Implementation

UML class diagram for the Builder Pattern



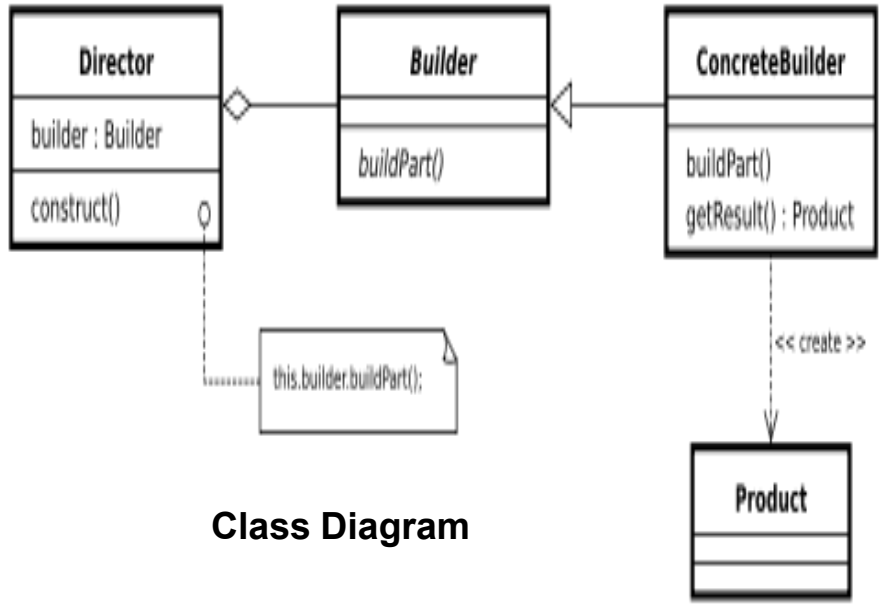


Builder

Abstract interface for creating objects (product).

ConcreteBuilder

Provides implementation for Builder. It is an object able to construct other objects. Constructs and assembles parts to build the objects.

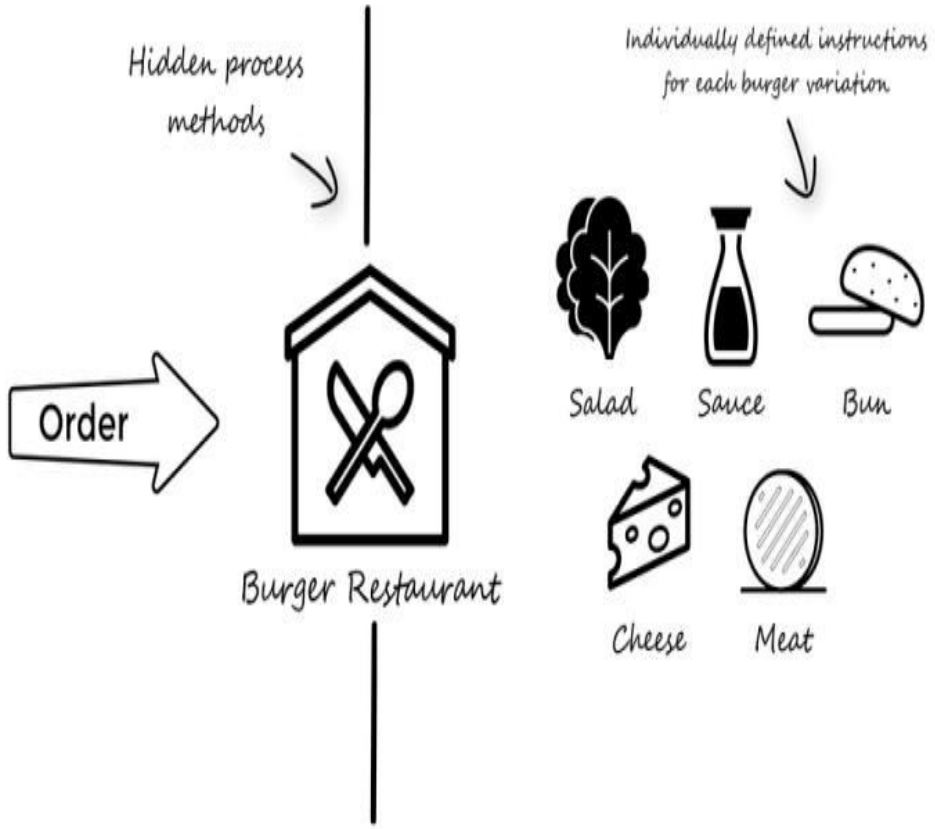


Class Diagram

Builder : Implementation example-1

Problem Statement: Case study

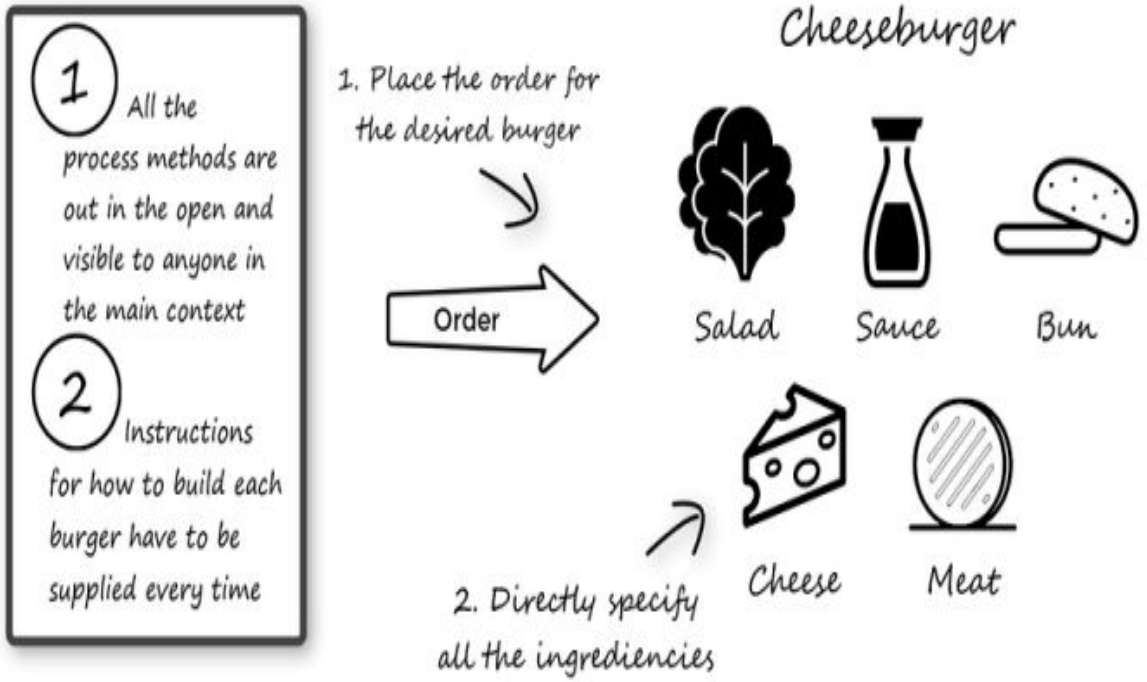
- We will set the objective of making a burger restaurant, which can make different variations of burgers.
- Secondly it would also be preferable to have defined instructions for how to build each of the different variations of burgers (e.g. cheeseburger), so that we do not have to provide all the ingredients each time when we are making a burger.
- Before, how the builder design pattern will be able to solve these issues, we will start by seeing how this system could be build based on construction of the burger object in the main context.



Object Oriented Analysis and Design

Builder : Implementation example-1

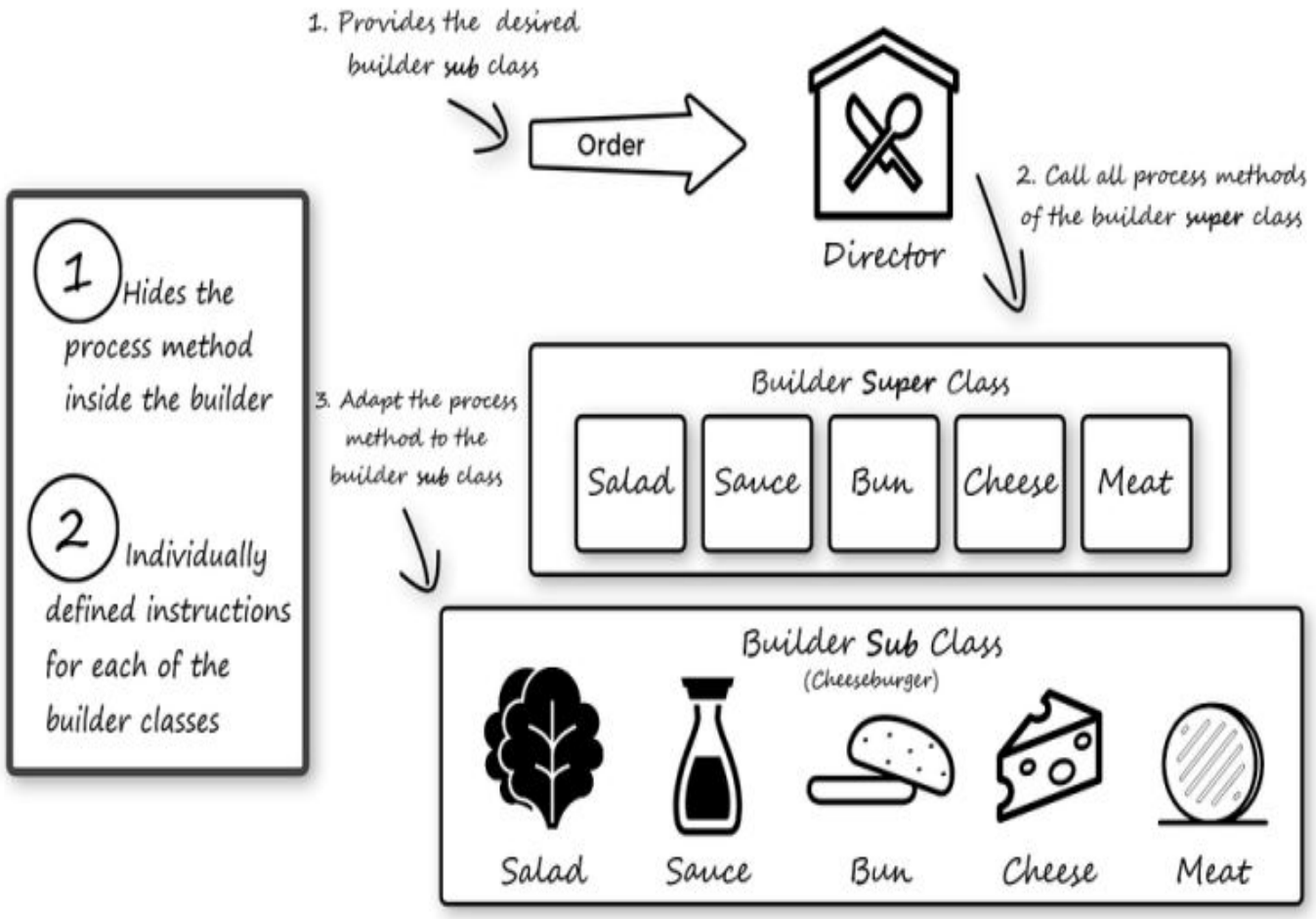
Solution: Construction In The Main Context




```
public class Main {
    public static void main(String[] args) {
        Burger cheeseBurger = new Burger();
        cheeseBurger.setBun("White Bread");
        cheeseBurger.setMeat("Beef");
        cheeseBurger.setSalad("Iceberg");
        cheeseBurger.setCheese("American Chesse");
        cheeseBurger.setSauce("Secret Sauce");
        cheeseBurger.print();
    }
```

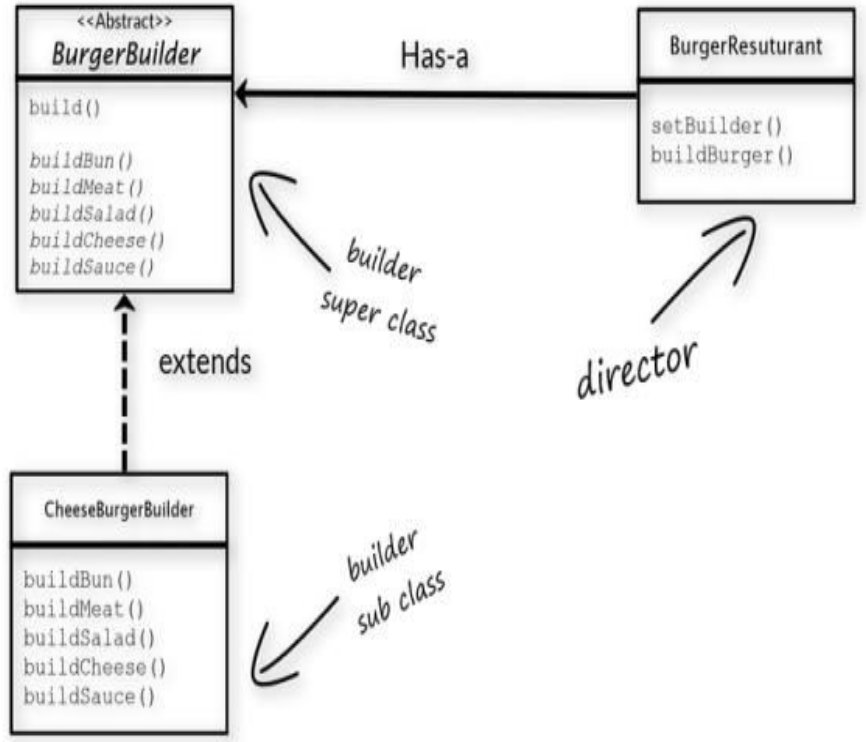
Solution: Concept Of The Builder Pattern

- When talking about the builder design pattern, it is important to understand the concept of the Director and the Builder.
- The director's job is to invoke the building process of the builder. The builder's job is to manage the different building procedures associated with each of the different variations of objects, in this case the burgers.
- The builder pattern consists of two classes, a sub- and super class.



Design Solution :Class Diagram (UML)

- Builder pattern consists of two main class types: the builder and the director
- With the context of program we will be using the builder pattern, it means we actually only want to be interacting with the director.
- The director will ensure that we build the correct burger object based on which builder we provide it with.
- Inside the director, we set its builder object field (setBuilder-method) and afterwards ask it to build the object based on the provided builder (build-method).

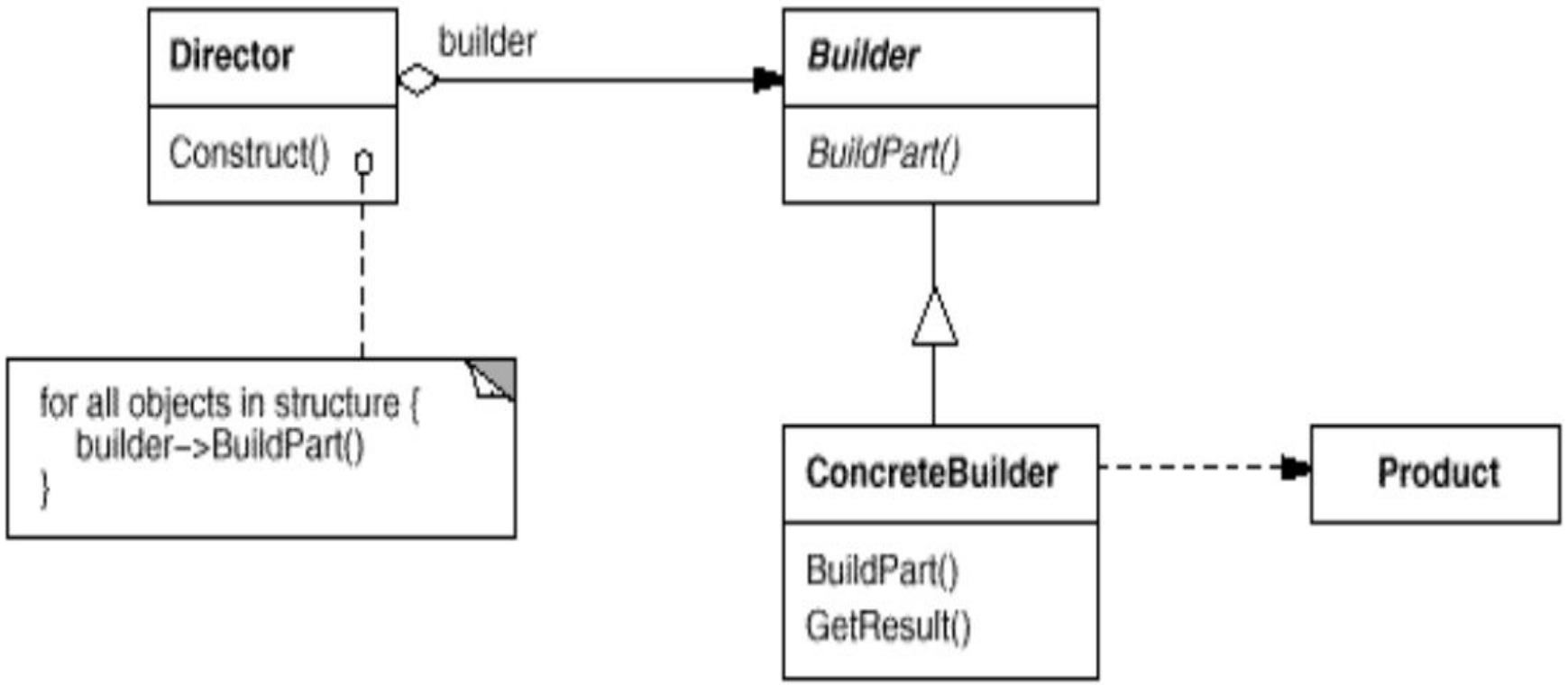


Applicability

Use the Builder pattern when

- The algorithm for creating a complex object should be independent of the parts that make up the object and how they're assembled.
- The construction process must allow different representations for the object that's constructed.
- You want to get rid of a “telescoping constructor”. Say you have a constructor with ten optional parameters. Calling such a beast is very inconvenient; therefore, you overload the constructor and create several shorter versions with fewer parameters.
- You want your code to be able to create different representations of some product (for example, stone and wooden houses). The Builder pattern can be applied when construction of various representations of the product involves similar steps that differ only in the details.

Structure


Director
builder
Builder
Construct()
BuildPart()
for all objects in structure {
builder->BuildPart()
}
ConcreteBuilder
Product
BuildPart()
GetResult()

Participants

Builder (BurgerBuilder)
o specifies an abstract interface for creating parts of a Product object.

ConcreteBuilder (CheeseBurgerBuilder, VegBurgerBuilder)
o constructs and assembles parts of the product by implementing the Builder interface.
o defines and keeps track of the representation it creates.
o provides an interface for retrieving the product (e.g., getBurger()). ·

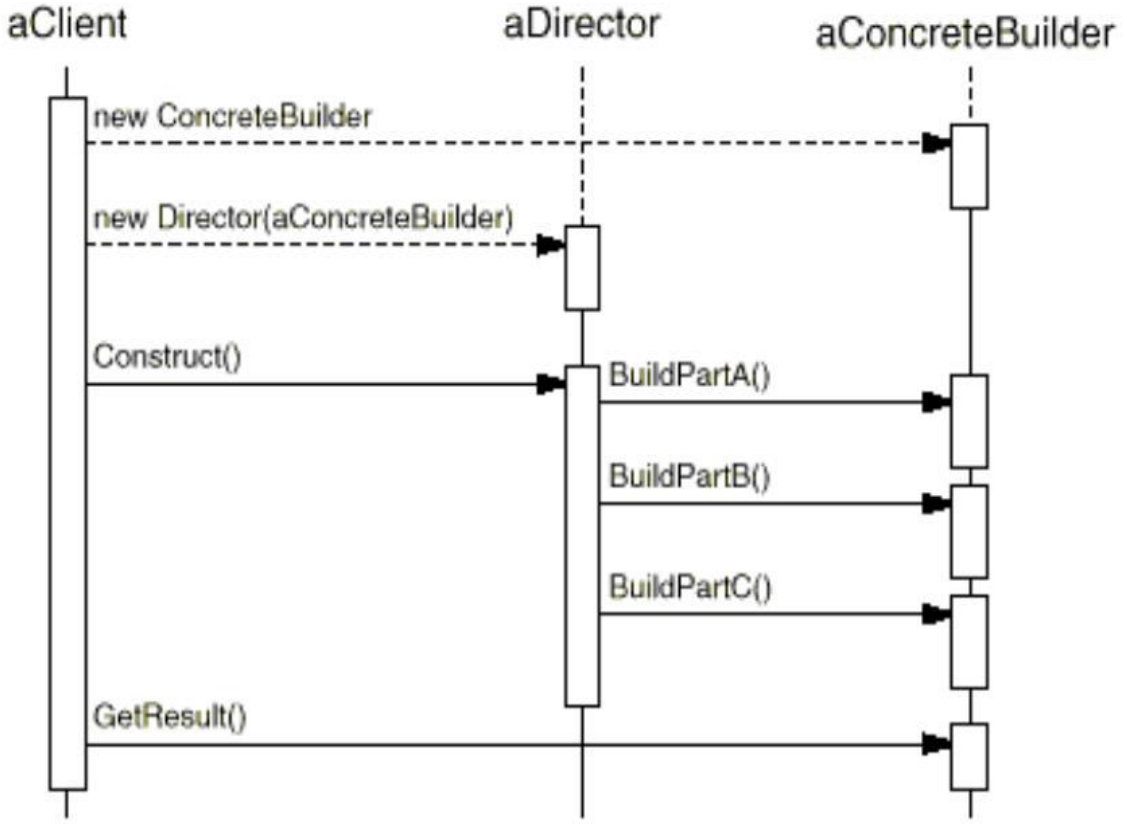
Director (Chef)
o constructs an object using the Builder interface.

Product (Burger)
o represents the complex object under construction. ConcreteBuilder builds the product's internal representation and defines the process by which it's assembled.
o includes classes that define the constituent parts, including interfaces for assembling the parts into the final result.

Collaboration

- Client creates Director object and configures it with a Builder
- Director notifies Builder to build each part of the product
- Builder handles requests from Director and adds parts to the product
- Client retrieves product from the Builder

The following interaction diagram illustrates how Builder and Director cooperate with a client.



Consequence

- Lets you vary a product's internal representation by using different Builders
- Isolates code for construction and representation
- Gives finer-grain control over the construction process

Issues to consider when using the Builder pattern

- Assembly and construction interface: generality
- Is an abstract class for all Products necessary?
- Usually products don't have a common interface
- Usually there's an abstract Builder class that defines an operation for each component that a director may ask it to create.
- These operations do nothing by default (empty, static methods)
- The ConcreteBuilder overrides operations selectively

PES UNIVERSITY

How to Implement

1. Make sure that you can clearly define the common construction steps for building all available product representations.

1. Declare these steps in the base builder interface.

1. Create a concrete builder class for each of the product representations and implement their construction steps.

1. Think about creating a director class. It may encapsulate various ways to construct a product using the same builder object.

1. The client code creates both the builder and the director objects. Before construction starts, the client must pass a builder object to the director.

1. The construction result can be obtained directly from the director only if all products follow the same interface. Otherwise, the client should fetch the result from the builder.

Pro and Cons

Pros

- You can construct objects step-by-step, defer construction steps or run steps recursively.
- You can reuse the same construction code when building various representations of products.
- *Single Responsibility Principle*. You can isolate complex construction code from the business logic of the product.

Cons

- The overall complexity of the code increases since the pattern requires creating multiple new classes.

Relations with Other Patterns

- Many designs start by using **Factory Method** (less complicated and more customizable via subclasses) and evolve toward **Abstract Factory**, **Prototype**, or **Builder** (more flexible, but more complicated).
- **Builder** focuses on constructing complex objects step by step. **Abstract Factory** specializes in creating families of related objects. *Abstract Factory* returns the product immediately, whereas *Builder* lets you run some additional construction steps before fetching the product.
- You can use **Builder** when creating complex **Composite** trees because you can program its construction steps to work recursively.
- You can combine **Builder** with **Bridge**: the director class plays the role of the abstraction, while different builders act as implementations.
- **Abstract Factories**, **Builders** and **Prototypes** can all be implemented as **Singletons**.

PES UNIVERSITY

DEMO

PES UNIVERSITY

MCQ

1. In the Builder pattern, who controls the building process?

A. Product
B. Director
C. Client
D. Constructor

2. The Builder interface is responsible for:

A. Running the application
B. Creating parts of a product
C. Database connectivity
D. User interface rendering

3. ConcreteBuilder mainly:

A. Implements building steps
B. Stores client data
C. Executes main method
D. Replaces Director

PES UNIVERSITY

MCQ

4. Which component represents the complex object under construction?

A. Director
B. Builder
C. Product
D. Client

5. One benefit of the Builder pattern is:

A. Removes abstraction
B. Provides finer control over construction
C. Eliminates interfaces
D. Avoids object creation

6. Builder pattern helps avoid:

A. Polymorphism
B. Telescoping constructors
C. Inheritance
D. Interfaces

PES UNIVERSITY

MCQ SOLUTION

1.B
2.B
3.A
4.C
5.B
6.B

References

Text Reference

- Design Patterns: Elements of Reusable Object-Oriented Software, GOF

Web Reference

https://www.baeldung.com/creational-design-patterns
https://www.javatpoint.com/builder-design-pattern
https://sourcemaking.com/design_patterns/builder
https://integu.net/builder-pattern/
https://www.c-sharpcorner.com/article/builder-design-pattern-with-java/

THANK YOU

Prof Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design

UE23CS352B

Prof Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

OO Design – Creational Design Patterns

Prototype

Prof Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements:
Prof. Nivedita Kasturi
Dept. of Computer Science and Engineering

Agenda

- ❑ Prototype-definition
- ✓ Motivation
- ✓ Intent
- ✓ Implementation
- ✓ Applicability
- ✓ Structure-Consequence
- ✓ Issues

Prototype : Class, Object Structural

Motivation

The Prototype Pattern specify the kind of objects to create using a prototypical instance, and create new objects by copying this prototype.

Use the Prototype Pattern when a client needs to create a set of objects that are alike or differ from each other only in terms of their state and creating an instance of a such object (e.g., using the “new” keyword) is either expensive or complicated.

The Prototype Pattern allows you to make new instances by copying existing instances.

– In Java this typically means using the clone() method or de-serialization when you need deep copies

Key aspect of this pattern:

– Client code can make new instances without knowing which specific class is being instantiated

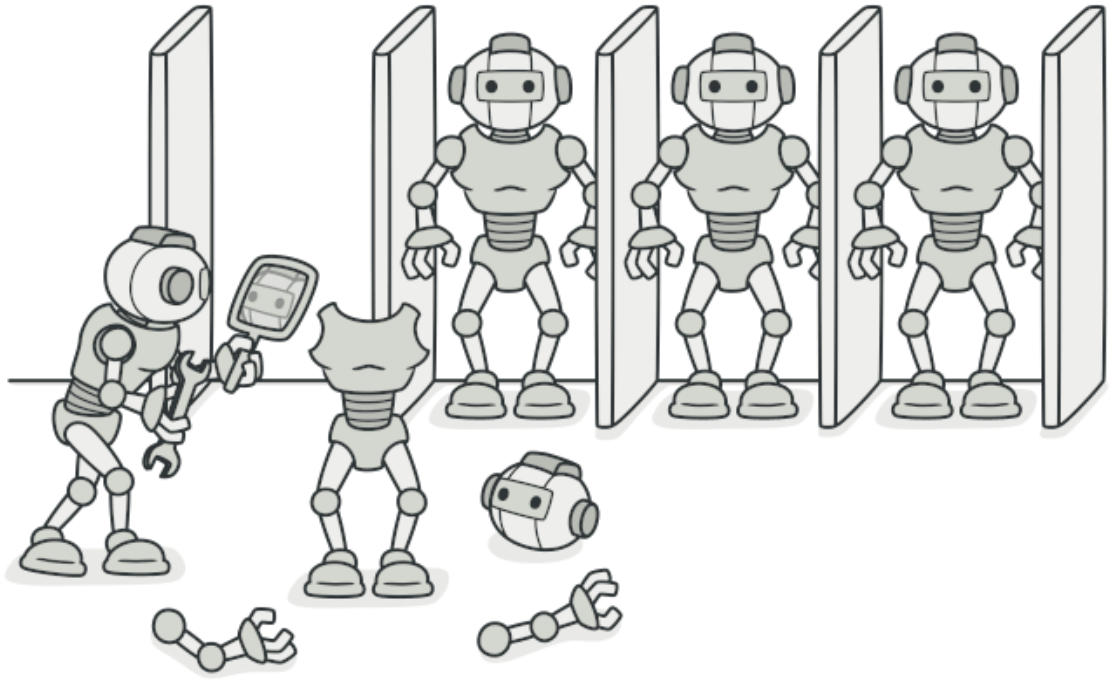
Prototype : Class, Object Structural

Intent

Prototype is a creational design pattern that lets you copy existing objects without making your code dependent on their classes.

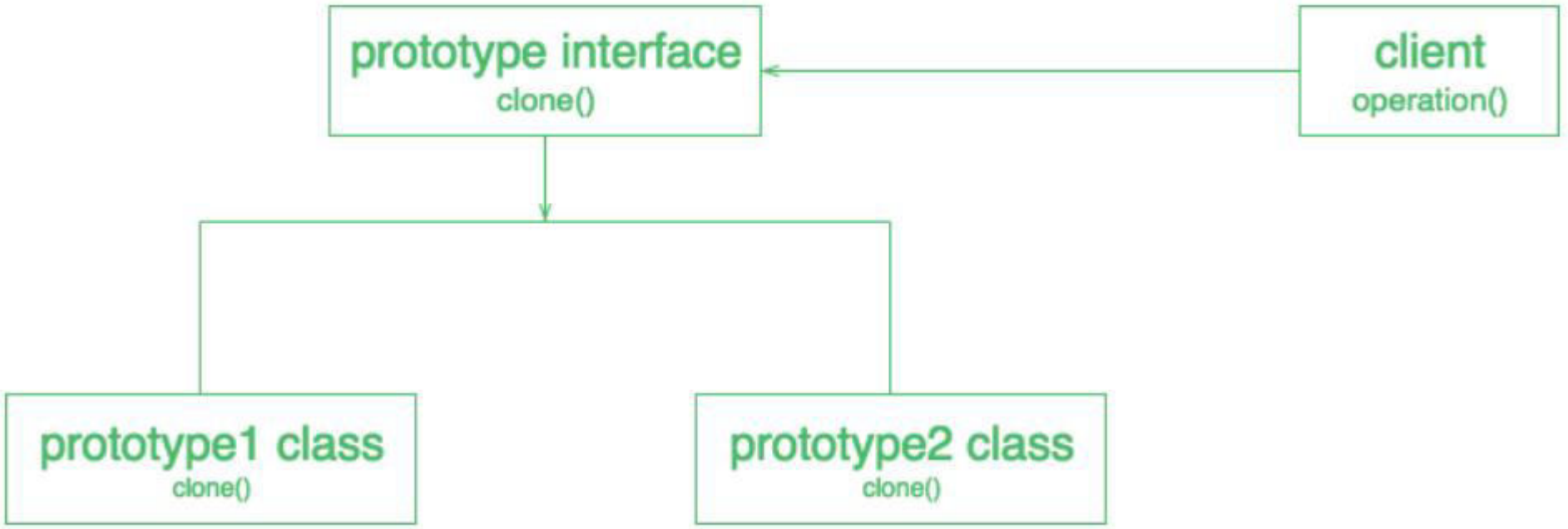
Usage examples: The Prototype pattern is available in Java out of the box with a Cloneable interface.
Any class can implement this interface to become cloneable.

java.lang.Object#clone() (class should implement the java.lang.Cloneable interface)
Identification: The prototype can be easily recognized by a clone or copy methods, etc.



Prototype : Implementation

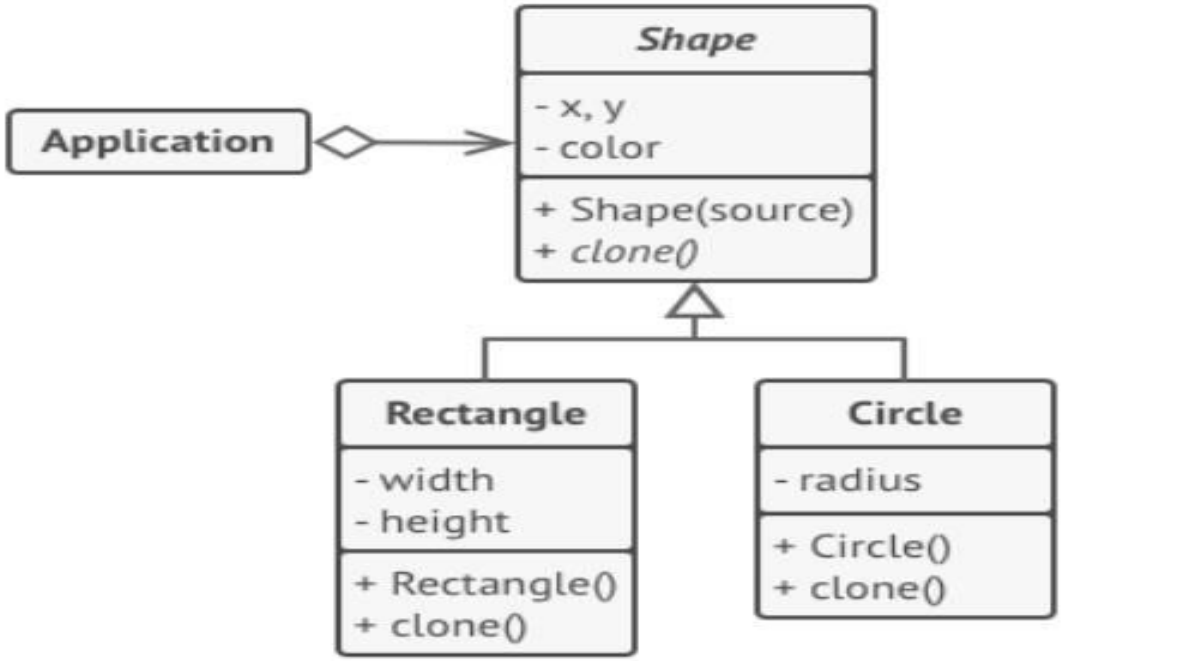
UML class diagram for the Prototype Pattern



Prototype : Implementation example-1

Problem Statement:

In this example, the **Prototype** pattern lets you produce exact copies of geometric objects, without coupling the code to their classes.



Cloning a set of objects that belong to a class hierarchy.

All shape classes follow the same interface, which provides a cloning method. A subclass may call the parent's cloning method before copying its own field values to the resulting object.

Solution

Let's take a look at how the Prototype can be implemented without the standard Cloneable interface.

DEMO

Note: Its example for prototype with different shape objects

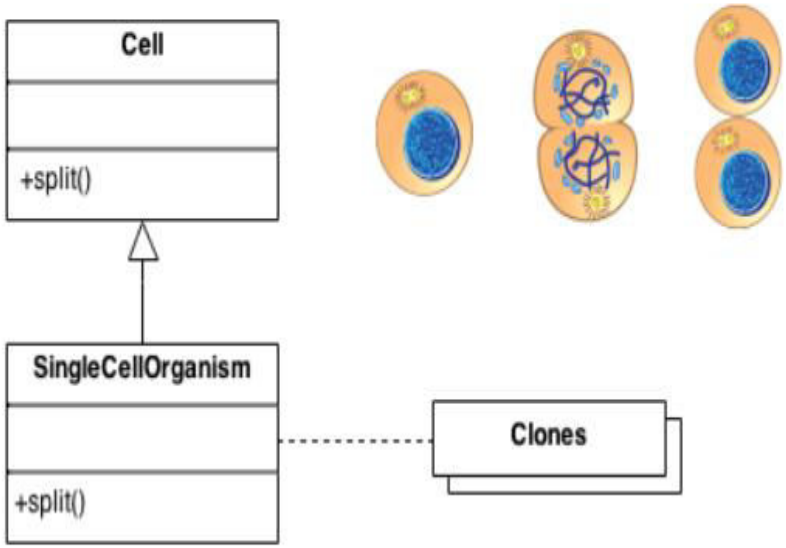
Prototype registry

You could implement a centralized prototype registry (or factory), which would contain a set of pre-defined prototype objects. This way you could retrieve new objects from the factory by passing its name or other parameters. The factory would search for an appropriate prototype, clone it and return you a copy.

Prototype : Scenario example-2

- The Prototype pattern specifies the kind of objects to create using a prototypical instance.
- Prototypes of new products are often built prior to full production.
- The mitotic division of a cell - resulting in two identical cells - is an example of a prototype that plays an active role in copying itself and thus, demonstrates the Prototype pattern.
- When a cell splits, two cells of identical genotype result. In other words, the cell clones itself.

Prototype : Scenario example-2



✓ Check list

- Add a clone() method to the existing "product" hierarchy.
- Design a "registry" that maintains a cache of prototypical objects. The registry could be encapsulated in a new Factory class, or in the base class of the "product" hierarchy.
- Design a factory method that: may (or may not) accept arguments, finds the correct prototype object, calls clone() on that object, and returns the result.
- The client replaces all references to the new operator with calls to the factory method.

Applicability

Use the Prototype pattern when

Use the Prototype pattern when a system should be independent of how its products are created, composed, and represented; and

- when the classes to instantiate are specified at run-time, for example, by dynamic loading; or
- to avoid building a class hierarchy of factories that parallels the class hierarchy of products; or
- when instances of a class can have one of only a few different combinations of state. It may be more convenient to install a corresponding number of prototypes and clone them rather than instantiating the class manually, each time with the appropriate state.

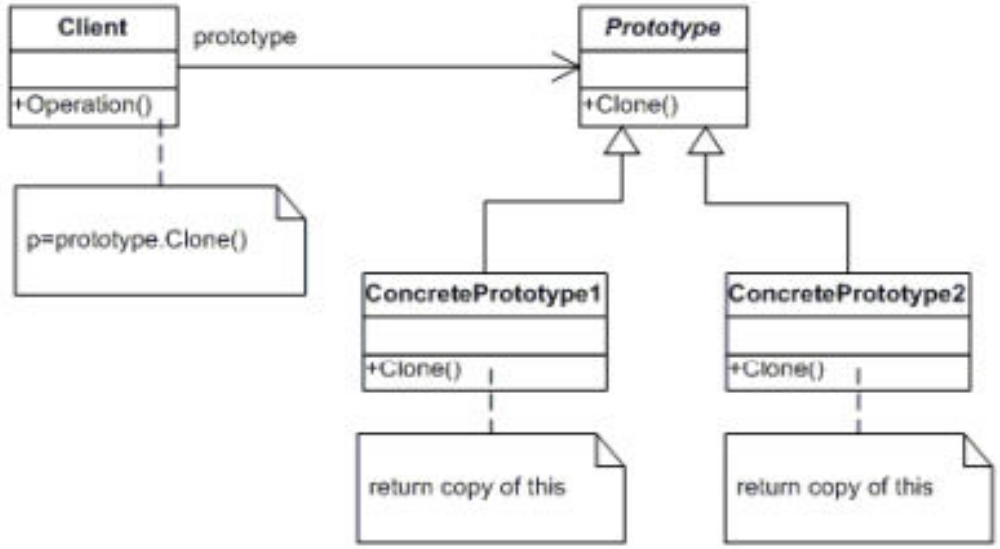
Structure

Participants

Prototype: declares an interface for cloning itself.

ConcretePrototype: implements an operation for cloning itself.

Client: creates a new object by asking a prototype to clone itself and then making required modifications.



Collaboration

A client asks a prototype to clone itself.

Consequence

Additional benefits of the Prototype pattern are listed below.

- ❑ Adding and removing products at run-time.
- ❑ Specifying new objects by varying values
- ❑ Specifying new objects by varying structure.
- ❑ Reduced subclassing.
- ❑ Configuring an application with classes dynamically.

Issues to consider when using the Prototype pattern

Consider the following issues when implementing prototypes:

- **Using a prototype manager**. : When the number of prototypes in a system isn't fixed (that is, they can be created and destroyed dynamically), keep a registry of available prototypes. Clients won't manage prototypes themselves but will store and retrieve them from the registry. A client will ask the registry for a prototype before cloning it. We call this registry a prototype manager.

- **Implementing the Clone operation**. The hardest part of the Prototype pattern is implementing the Clone operation correctly. It's particularly tricky when object structures contain circular references.

- **Initializing clones**. While some clients are perfectly happy with the clone as is, others will want to initialize some or all of its internal state to values of their choosing. You generally can't pass these values in the Clone operation, because their number will vary between classes of prototypes. Some prototypes might need multiple initialization parameters; others won't need any. Passing parameters in the Clone operation precludes a uniform cloning interface.

Pros and Cons

⚖ Pros and Cons

- ✓ You can clone objects without coupling to their concrete classes.
- ✓ You can get rid of repeated initialization code in favor of cloning pre-built prototypes.
- ✓ You can produce complex objects more conveniently.
- ✓ You get an alternative to inheritance when dealing with configuration presets for complex objects.
- ✗ Cloning complex objects that have circular references might be very tricky.

PES UNIVERSITY

MCQ

1. The Prototype pattern creates objects by:
A. Using constructors repeatedly
B. Copying existing objects
C. Using static methods
D. Using interfaces only

2. The main intent of the Prototype pattern is to:
A. Hide inheritance
B. Copy existing objects without depending on their classes
C. Increase object creation complexity
D. Eliminate interfaces

PES UNIVERSITY

MCQ

3. Prototype pattern is useful when:
A. Objects are simple
B. Object creation is expensive or complex
C. Only one instance is needed
D. Classes are final

4. A prototype registry is used to:
A. Store and retrieve prototype objects
B. Delete objects
C. Compile classes
D. Execute programs

PES UNIVERSITY

MCQ

5. In Prototype pattern, the client:

A. Directly creates objects using new
B. Requests an object to clone itself
C. Avoids object creation
D. Uses static classes

6. One benefit of Prototype pattern is:

A. Reduced subclassing
B. Increased subclassing
C. Removal of interfaces
D. Elimination of objects

PES UNIVERSITY

MCQ SOLUTION

1.B
2.B
3.B
4.A
5.B
6.A

References

Text Reference

- Design Patterns: Elements of Reusable Object-Oriented Software, GOF

Web Reference

https://sourcemaking.com/design_patterns/prototype
https://www.geeksforgeeks.org/prototype-design-pattern/

THANK YOU

Prof Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design

UE23CS35B

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B : Object Oriented Analysis and Design

Structural Patterns – Adapter

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements:

Dr. Sudeepa Roy Dey

Dept. of Computer Science and Engineering

Agenda

- ❑ Introduction to Structural design patterns
- ❑ Types of Structural Design Patterns.
- ❑ Adapter-definition
  - ✓ Motivation
  - ✓ Intent
  - ✓ Implementation
  - ✓ Applicability
  - ✓ Structure-Consequence
  - ✓ Issues

Introduction to Structural Design Pattern

- Structural design patterns are concerned with how classes and objects can be composed, to form larger structures.
- The structural design patterns simplifies the structure by identifying the relationships.
- These patterns focus on, how the classes inherit from each other and how they are composed from other classes.
- Structural *class* patterns use inheritance to compose interfaces or implementations.
- Structural design patterns explain how to assemble objects and classes into larger structures, while keeping these structures flexible and efficient.

Types of Structural Design pattern: Scope

		Purpose		
		Creational	**Structural**	**Behavioral**
Scope	**Class**	Factory Method (107)	Adapter (class) (139)	Interpreter (243) Template Method (325)
	Object	Abstract Factory (87) Builder (97) Prototype (117) Singleton (127)	Adapter (object) (139) Bridge (151) Composite (163) Decorator (175) Facade (185) Flyweight (195) Proxy (207)	Chain of Responsibility (223) Command (233) Iterator (257) Mediator (273) Memento (283) Observer (293) State (305) Strategy (315) Visitor (331)

Adapter: Class, Object Structural

Motivation

- The adapter pattern is adapting between classes and objects.
- Like any adapter in the real world it is used to be an interface, a bridge between two objects.
  Ex: In real world we have adapters for power supplies, adapters for camera memory cards, and so on.
- Sometimes a toolkit class that's designed for reuse isn't reusable only because its interface doesn't match the domain-specific interface an application requires.

Advantage of Adapter Pattern

- It allows two or more previously incompatible objects to interact.
- It allows reusability of existing functionality.

Adapter: Class, Object Structural

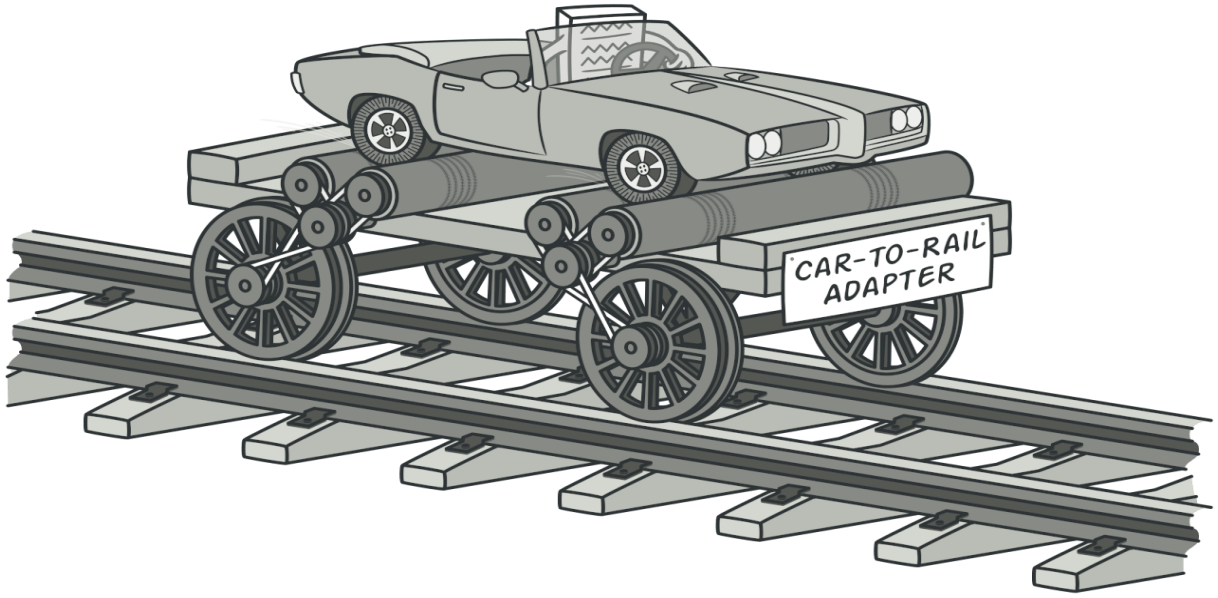
Intent

- Adapter lets classes work together that couldn' t otherwise because of incompatible interfaces.

- An Adapter Pattern says that "convert the interface of a class into another interface that a client wants". In other words, to provide the interface according to client requirement while using the services of a class with a different interface.

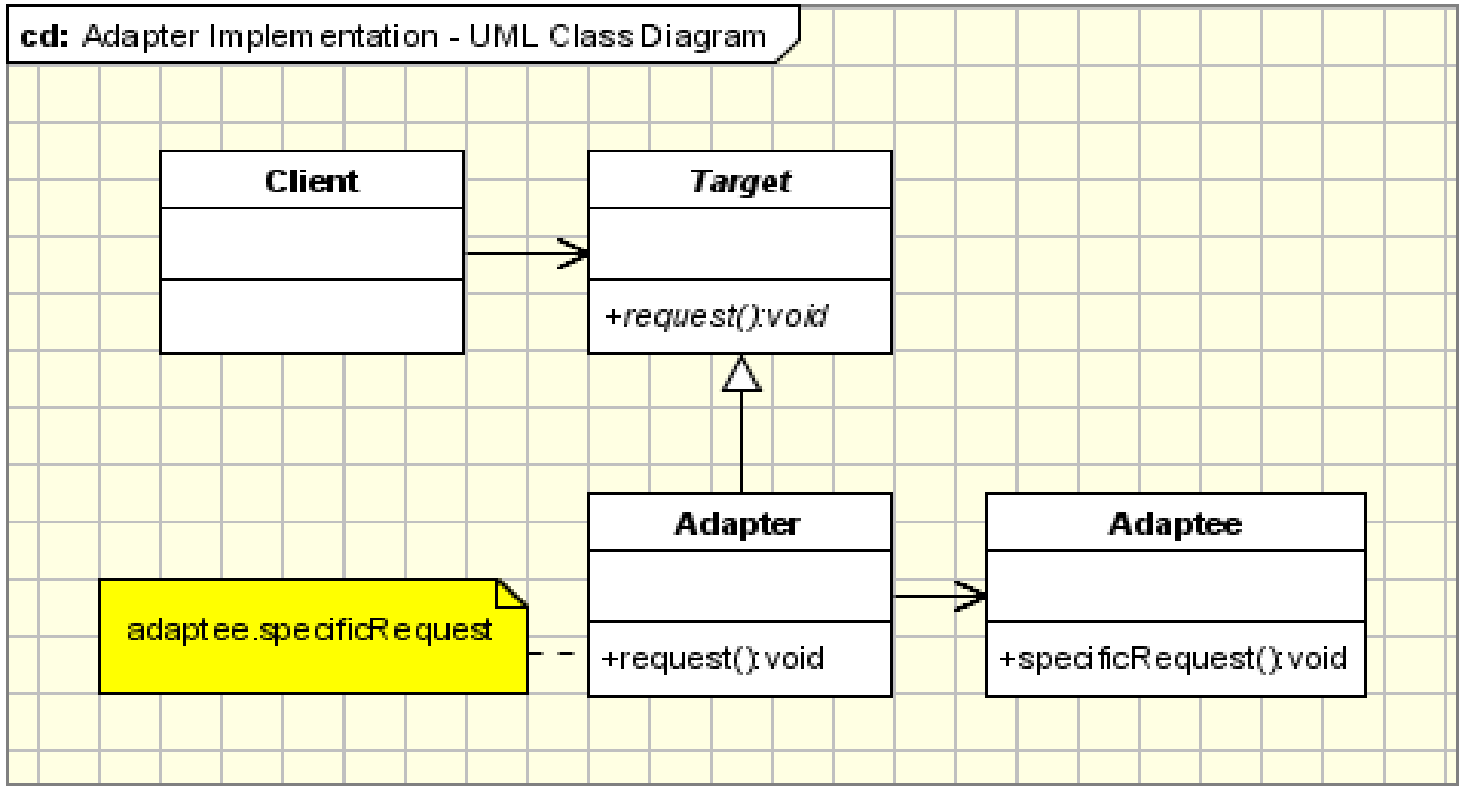
- The Adapter Pattern is also known as Wrapper.

Adapter is a structural design pattern that allows classes with incompatible interfaces to collaborate.



Adapter: Implementation

UML class diagram for the Adapter Pattern



The classes/objects participating in adapter pattern:

1. Target - defines the domain-specific interface that Client uses.
2. Adapter - adapts the interface Adaptee to the Target interface.
3. Adaptee - defines an existing interface that needs adapting.
4. Client - collaborates with objects conforming to the Target interface.

Adapter class: This class is a wrapper class which implements the desired target interface and modifies the specific request available from the Adaptee class.

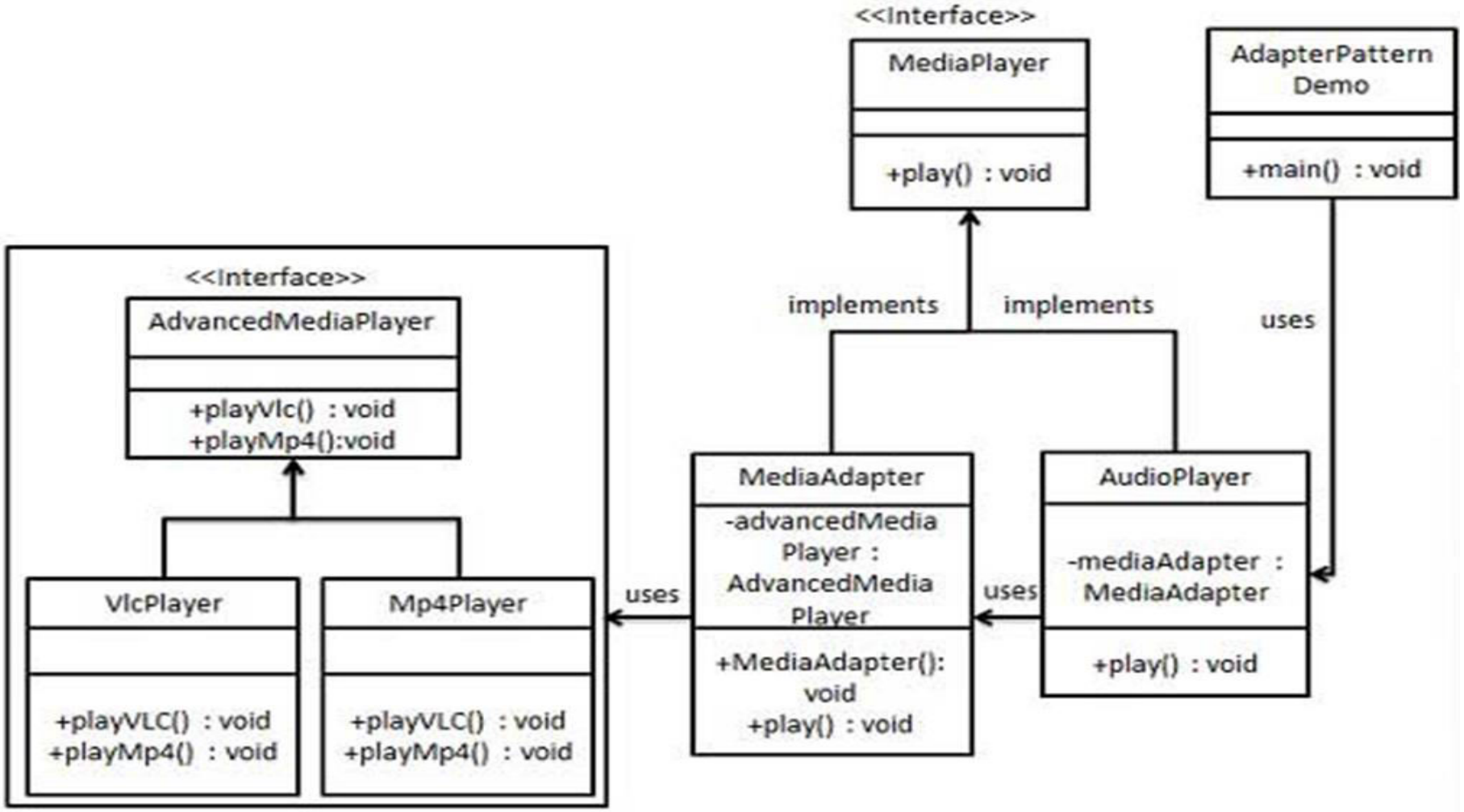
Adapter: Implementation example-1

Problem Statement: We have a MediaPlayer interface and a concrete class AudioPlayer implementing the MediaPlayer interface. AudioPlayer can play mp3 format audio files by default. We are having another interface AdvancedMediaPlayer and concrete classes implementing the AdvancedMediaPlayer interface. These classes can play vlc and mp4 format files. We want to make AudioPlayer to play other formats as well.

Solution: To attain this, we have created an adapter class MediaAdapter which implements the MediaPlayer interface and uses AdvancedMediaPlayer objects to play the required format. AudioPlayer uses the adapter class MediaAdapter passing it the desired audio type without knowing the actual class which can play the desired format. AdapterPatternDemo, our demo class will use AudioPlayer class to play various formats.

Demo….>refer to AdapterPatternDemo.java

Design Solution



Design Solution

Adapter Pattern

Modules :

Module Type	Module Example
Adapter (concrete of AbstractA)	*MediaAdapter*
Adaptee (concrete of AbstractA)	*AudioPlayer*
AbstractB	*AdvanceMediaPlayer*
ConcreteB	*VLCplayer, MP4player*
AbstactA	*MediaPlayer*
Demo	*AdapterPatternDemo*

Adapter: Scenario example-2

Suppose you have a Bird class with fly() and makeSound()methods. And also a ToyDuck class with squeak() method. Let's assume that you are short on ToyDuck objects and you would like to use Bird objects in their place. Birds have some similar functionality but implement a different interface, so we can't use them directly.

Solution: So, we will use adapter pattern. Here our client would be ToyDuck and adaptee would be Bird.(refer to demo)

The adapter pattern we have implemented above is called Object Adapter Pattern because the adapter holds an instance of adaptee.
What a Object adapter??? Next slides we will explain in detail

Adapter: Scenario example-2

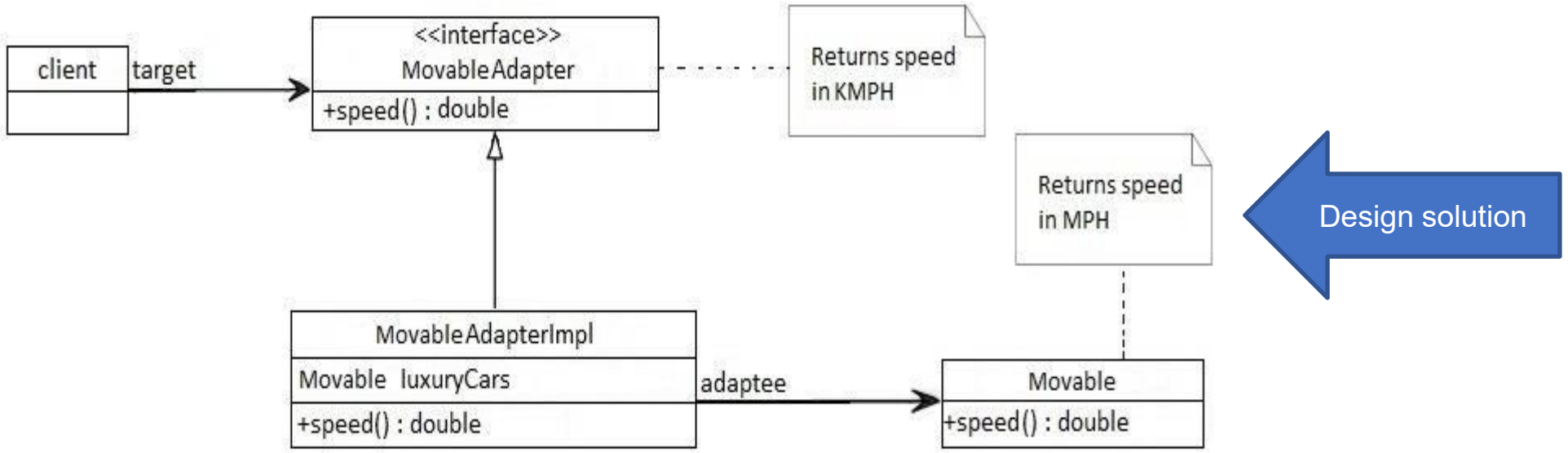
Explain: Here we have created a BirdAdapter class implementing inteface ToyDuck to convert Bird 's makeSound() method to makeSound() 's squeak() ToyDuck . This way, we can still call the squeak() of Bird objects through BirdAdapter

Suppose we have a bird that can makeSound(), and we have a plastic toy duck that can squeak(). Now suppose our client changes the requirement and he wants the toyDuck to makeSound then ?

Simple solution is that we will just change the implementation class to the new adapter class and tell the client to pass the instance of the bird(which wants to squeak()) to that class.
Before : ToyDuck toyDuck = new PlasticToyDuck();
After : ToyDuck toyDuck = new BirdAdapter(sparrow);
You can see that by changing just one line the toyDuck can now do Chirp Chirp !!

Adapter: Scenario example-3

Consider a scenario in which there is an app that's developed in the US which returns the top speed of luxury cars in miles per hour (MPH). Now we need to use the same app for our client in the UK that wants the same results but in kilometers per hour (km/h).
To deal with this problem, we'll create an adapter which will convert the values and give us the desired results:



(Students can Try coding with the given design solution)

Applicability

Use the Adapter pattern when

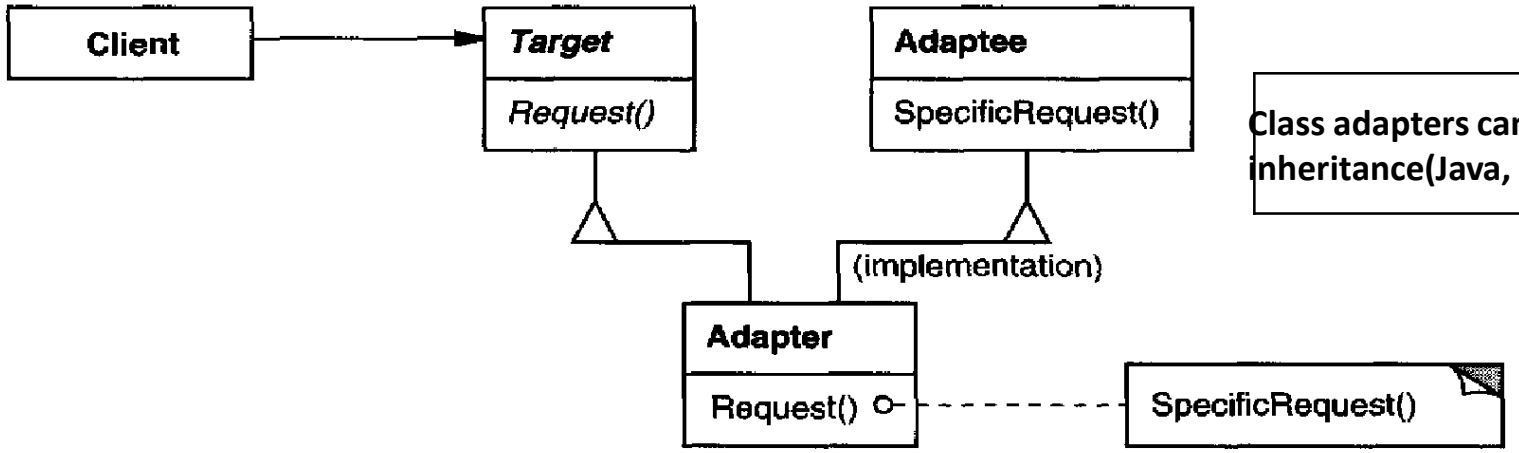
- you want to use an existing class, and its interface does not match the one you need.
- you want to create a reusable class that cooperates with unrelated or unforeseen classes, that is, classes that don't necessarily have compatible interfaces.
- (object adapter only) you need to use several existing subclasses, but it's unpractical to adapt their interface by subclassing every one. An object adapter can adapt the interface of its parent class.

Software Examples of Adapter Patterns: Wrappers used to adopt 3rd parties libraries and frameworks - most of the applications using third party libraries use adapters as a middle layer between the application and the 3rd party library to decouple the application from the library.

Object Oriented Analysis and Design

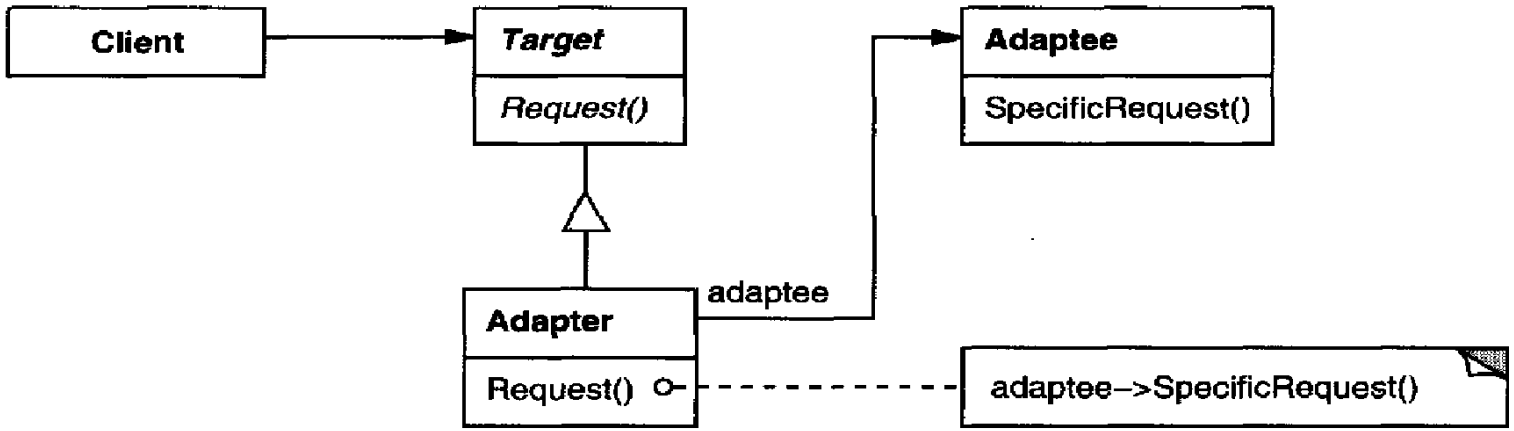
Structure

A class adapter uses multiple inheritance to adapt one interface to another:



Class adapters can be implemented in languages supporting multiple inheritance(Java, C# or PHP does not support multiple inheritance)

An object adapter relies on object composition: Based on delegation



Consequence

Class and object adapters have different trade-offs.

A class adapter

- adapts Adaptee to Target by committing to a concrete Adaptee class. As a consequence, a class adapter won't work when we want to adapt a class and all its subclasses.
- let's Adapter override some of Adaptee’s behavior, since Adapter is a subclass of Adaptee.
- introduces only one object, and no additional pointer indirection is needed to get to the adaptee.

An object adapter

- **let's a single Adapter work with many Adaptees**—that is, the Adaptee itself and all of its subclasses (if any). The Adapter can also add functionality to all Adaptees at once.
- makes it harder to override Adaptee behavior. It will require subclassing Adaptee and making Adapter refer to the subclass rather than the Adaptee itself.

Difference: Class and Object structure pattern

- Objects Adapters uses composition, the Adaptee delegates the calls to Adaptee (opossed to class adapters which extends the Adaptee).
- The main advantage is that the object Adapter adapts not only the Adaptee but all its subclasses. All it's subclasses with one "small" restriction: all the subclasses which don't add new methods, because the used mechanism is delegation. So for any new method the Adapter must be changed or extended to expose the new methods as well.
- The main disadvantage is that it requires to write all the code for delegating all the necessary requests to the Adaptee.

- Class adapter uses inheritance instead of composition. It means that instead of delegating the calls to the Adaptee, it subclasses it. In conclusion it must subclass both the Target and the Adaptee.
- There are advantages and disadvantages: It adapts the specific Adaptee class. The class it extends. If that one is subclassed it can not be adapted by the existing adapter.
- It doesn't require all the code required for delegation, which must be written for an Object Adapter.
- If the Target is represented by an interface instead of a class then we can talk about "class" adapters, because we can implement as many interfaces as we want.

Issues to consider when using the Adapter pattern

- *How Much the Adapter Should Do?*

It should do how much it has to in order to adapt. It's very simple, if the Target and Adaptee are similar then the adapter has just to delegate the requests from the Target to the Adaptee. If Target and Adaptee are not similar, then the adapter might have to convert the data structures between those and to implement the operations required by the Target but not implemented by the Adaptee.

- *Using two-way adapters to provide transparency.*

A potential problem with adapters is that they aren't transparent to all clients. An adapted object no longer conforms to the Adaptee interface, so it can't be used as is wherever an Adaptee object can. Two-way adapters can provide such transparency. Specifically, they're useful when two different clients need to view an object differently.

DEMO

MCQ

1. What is the primary purpose of the Adapter design pattern?

A. To optimize memory usage
B. To convert incompatible interfaces into a compatible one
C. To create objects dynamically
D. To enforce strict inheritance

2. What does the Adapter pattern allow incompatible classes to do?

A. Replace each other completely
B. Collaborate despite incompatible interfaces
C. Avoid inheritance
D. Eliminate abstraction

MCQ

3. What is the role of the Target in the Adapter pattern?
A. Converts data formats
B. Defines the interface expected by the client
C. Stores internal data
D. Executes system commands

4. What does the Adaptee represent in the Adapter pattern?
A. A new interface created by the client
B. An existing interface that needs adapting
C. A database system
D. A collection of objects

MCQ SOLUTION

1.B
2.B
3.B
4.B

References

Text Reference

- Design Patterns: Elements of Reusable Object-Oriented Software, GOF

Web Reference

- https://www.javatpoint.com/adapter-pattern
- https://www.oodesign.com/adapter-pattern.html
- https://www.geeksforgeeks.org/adapter-pattern/
- https://www.tutorialspoint.com/design_pattern/adapter_pattern.htm
- https://itzone.com.vn/en/article/adapter-design-pattern-in-java

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE22CS35B : Object Oriented Analysis and Design with Java

Structural Patterns : Facade pattern

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements:
Prof. Sindhu R Pai, Saranya Devi B
Dept. of Computer Science and Engineering

Agenda

- What is façade?
- Why Façade?
- Pictorial Representation
- Applicability
- Advantages
- Known Uses
- Implementation
- References

What is facade?

Meaning:

- The principal front of a building, that faces on to a street or open space.

"the house has a half-timbered façade"

- A deceptive outward appearance.

"her flawless public façade masked private despair"

Facade in Software

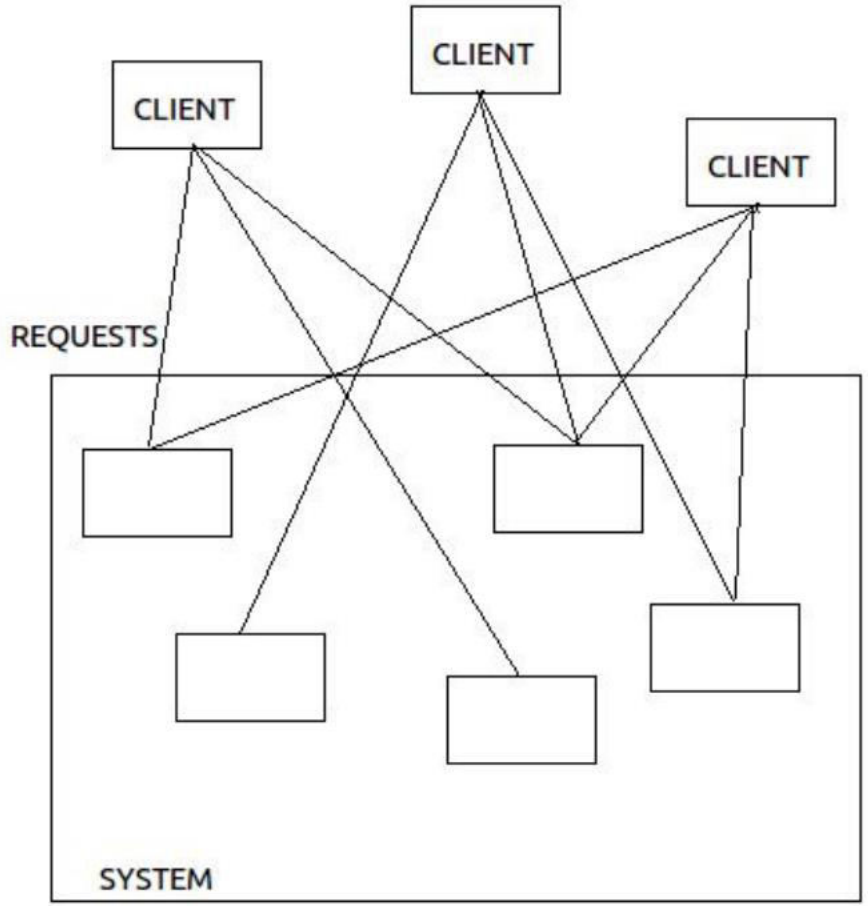
- An object that provides a **simplified interface to a larger body** of code, a library, a framework, or any other complex set of classes.
- The main intent of Façade is to provide a **unified interface to a set of interfaces in a subsystem**. Façade defines a **higher-level interface** that makes the subsystem easier to use.
- Example: fopen is an interface to open and read system commands.
- A structural design pattern, which **wraps a complicated subsystem with a simpler interface.**
- If the Facade is the only access point for the subsystem, it will limit the features and flexibility that **"power users**" may need

Why Facade?

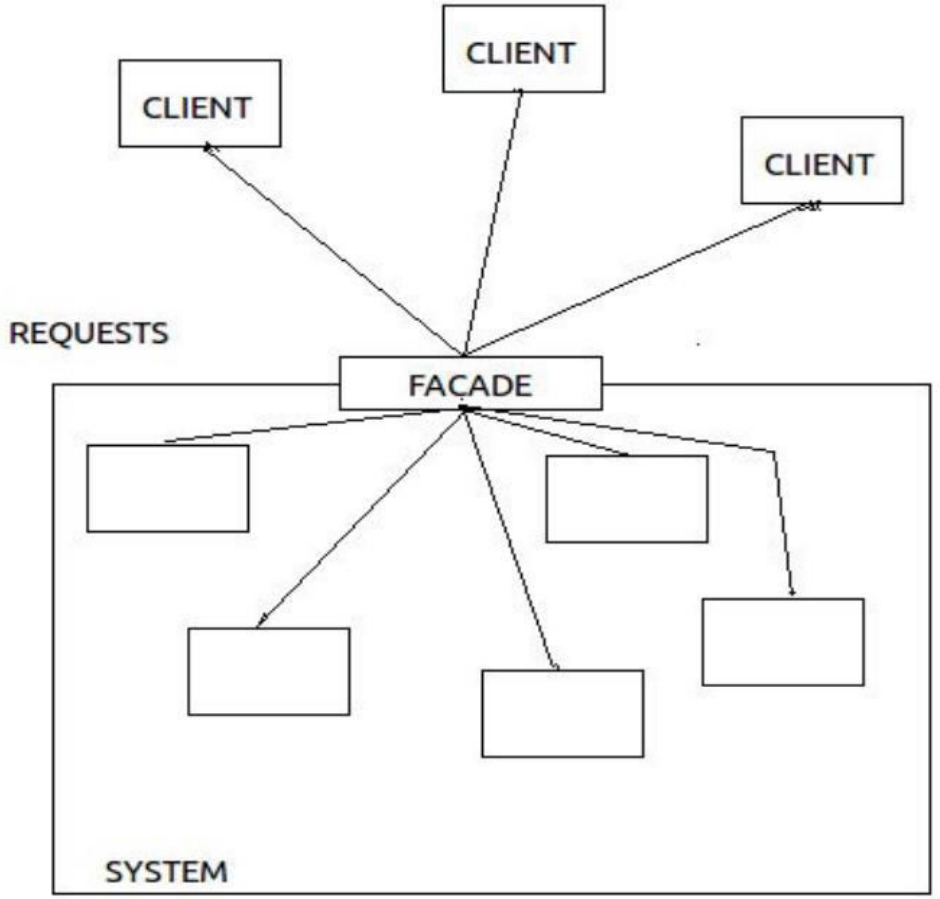
- Structuring a system into subsystems **helps reduce complexity**.

- A common design goal is to **minimize the communication and dependencies between subsystems**. One way to achieve this goal is to introduce a façade object that provides a single, simplified interface to the more general facilities of a subsystem.

Pictorial representation



Without facade



With facade

Applicability

- To provide a **simple interface to a complex subsystem**. A façade can provide **a simple default view of the subsystem** that is good enough for most clients.
- Introduce a façade to **decouple the subsystem from clients and other subsystems**, thereby promoting **subsystem independence and portability**.
- Use a façade to **define an entry point to each subsystem** level.
- To **simplify the dependencies between subsystems** by making them communicate with each other solely through their façades.

Advantages

- It **shields clients from subsystem components**, thereby reducing the number of objects that clients deal with and making the subsystem easier to use.
- It **promotes weak coupling between the subsystem and its clients**. Often the components in a subsystem are strongly coupled.
- This can **eliminate complex or circular dependencies**. This can be an important consequence when the client and the subsystem are implemented independently.
- Reducing compilation dependencies with façades **can limit the recompilation needed for a small change** in an important subsystem.

Known uses

• In the **ET++ application framework**, an application that can have built-in browsing tools for inspecting its objects at run-time. These browsing tools are implemented in a separate subsystem that includes a façade class called "ProgrammingEnvironment." This façade defines operations such as InspectObject and InspectClass for accessing the browsers.

•The **Choices operating system** uses façade to compose many frameworks into one. The key abstractions in Choices are processes, storage, and address spaces. For each of these abstractions there is a corresponding subsystem, implemented as a framework, that supports porting Choices to a variety of different hardware platforms.

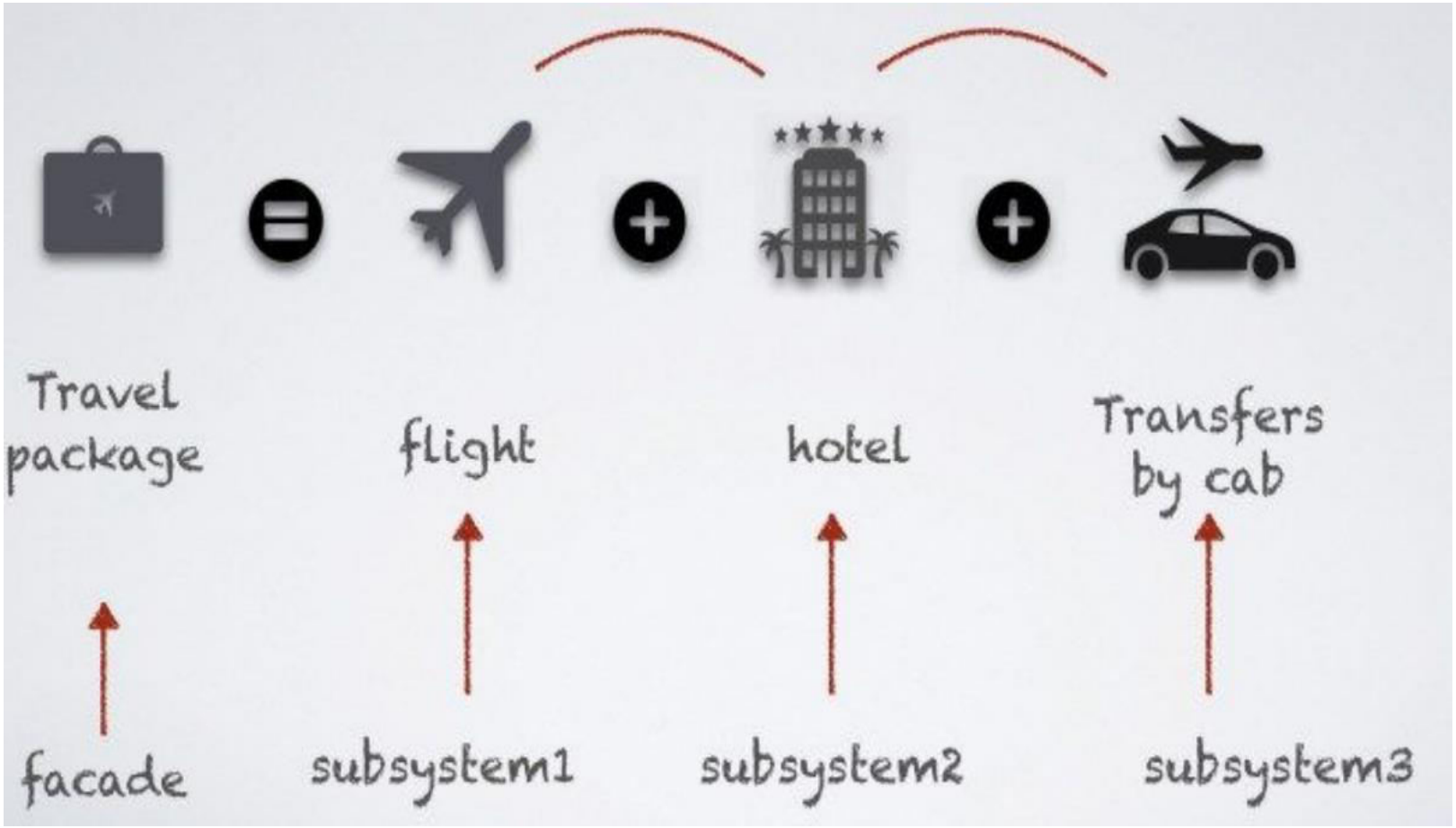
Implementation

• **Reduce client-subsystem coupling**. This can be done by simply **creating a "Façade Abstract Class" and concrete subclasses for the implementation of the subsystem**. Now the client class can communicate with the subsystem through the "**Abstract Façade Class**“.

An alternative approach is to configure a façade object with different subsystem objects

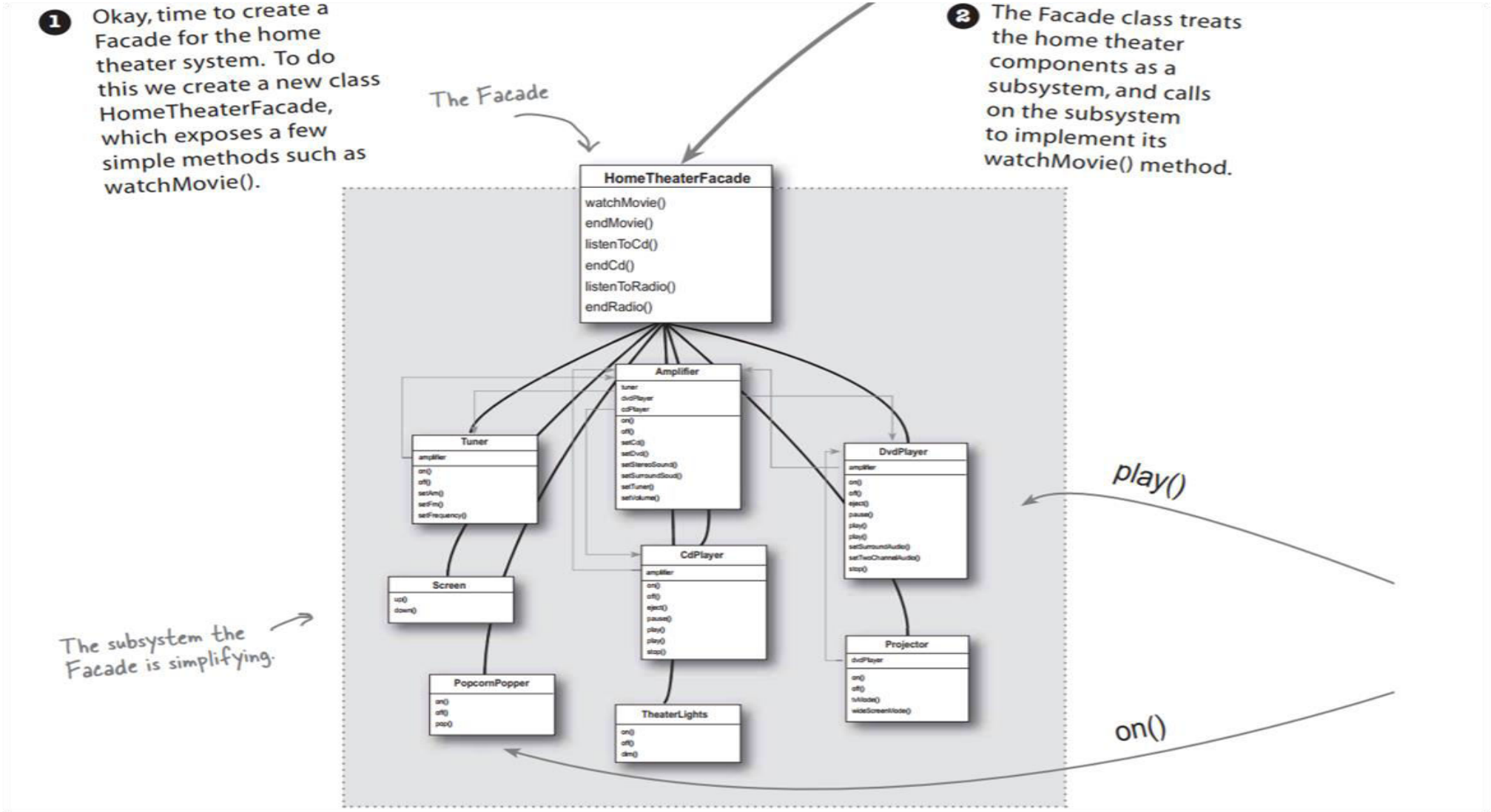
• Public versus Private Interfaces. Classes and Subsystems are similar. Both encapsulate something. Both have private and public interfaces. The **public interface consists of classes accessible by all Clients** whereas the private interface is only for subsystem extenders. **Façade is part of the public interface**

Example -1: Booking a package



Booking a package

Example -2: Designing Hometheater



MCQ

1. What is the primary intent of the Facade pattern?

A. To provide multiple interfaces to a subsystem
B. To simplify algorithms
C. To provide a unified interface to a set of interfaces
D. To enforce strict coupling

2. Why is the Facade pattern used in system design?

A. To increase dependencies between subsystems
B. To complicate communication
C. To minimize communication and dependencies between subsystems
D. To eliminate all classes

MCQ

3. What is one key advantage of the Facade pattern?
A. It increases system complexity
B. It reduces the number of objects clients interact with
C. It removes abstraction
D. It enforces tight coupling

4. What does the Facade pattern promote between subsystem and client?
A. Strong coupling
B. No interaction
C. Weak coupling
D. Direct dependency

MCQ SOLUTION

1.C
2.C
3.B
4.C

References

- Facade Pattern from Head First Design Patterns (javaguides.net)
- Facade Design Pattern (sourcemaking.com)
- https://refactoring.guru/design-patterns/java

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

Unit 4
Structural Patterns - Proxy

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements:
Prof. Spurthi N Anjan and Saranya devi B
Dept. of Computer Science and Engineering

Proxy

- In proxy design pattern, a proxy object provide a surrogate or placeholder for another object to control access to it.
- A proxy controls access to the original object, allowing you to perform something either before or after the request gets through to the original object.

Proxy design pattern - Intent

- Proxy is heavily used to implement lazy loading related use cases where we do not want to create full object until it is actually needed.
- Using the proxy pattern, a class represents the functionality of another class.

Proxy design pattern - Intent

In the Proxy Design Pattern, a client does not directly talk to the original object, it delegates calls to the proxy object which calls the methods of the original object. Moreover, the important point is that the client does not know about the proxy. The proxy acts as an original object for the client.

There are three main variations of the Proxy Pattern:

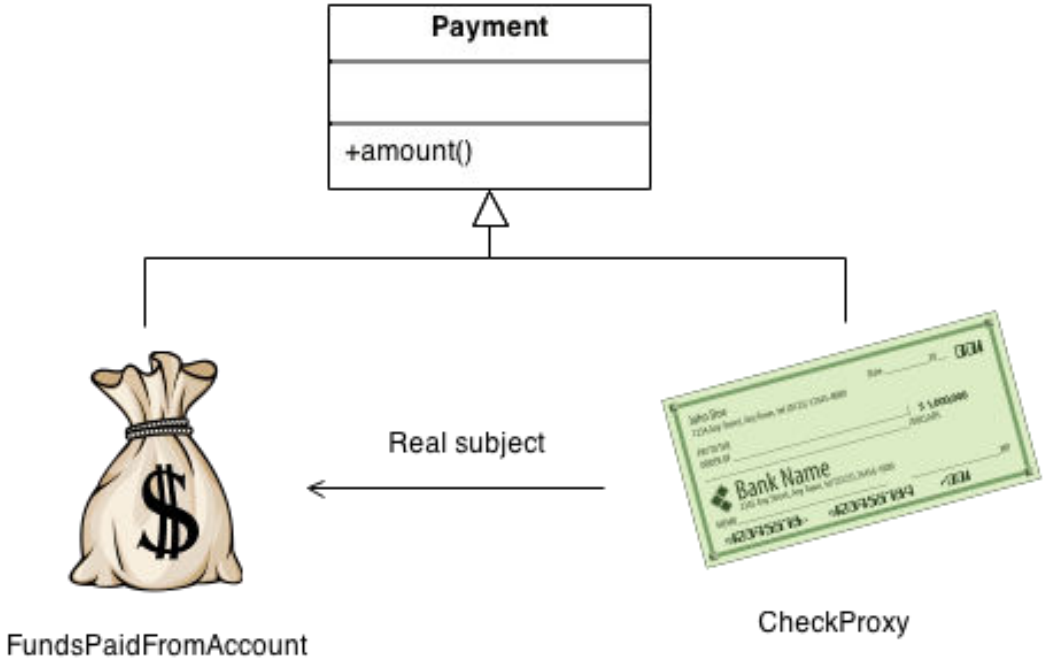
- A remote proxy provides a local representative for an object in a different address space.
- A virtual proxy creates expensive objects on demand.
- A protection proxy controls access to the original object. Protection proxies are useful when objects should have different access rights.

Design participants

- **Subject –** is an interface which expose the functionality available to be used by the clients.
- **Real Subject –** is a class implementing Subject and it is concrete implementation which needs to be hidden behind a proxy.
- **Proxy –** hides the real object by extending it and clients communicate to real object via this proxy object. Usually, frameworks create this proxy object when client request for real object.

Example

A check or bank draft is a proxy for funds in an account. A check can be used in place of cash for making purchases and ultimately controls access to cash in the issuer's account.



Real-world example

- In corporate networks, internet access is guarded behind a network proxy. All network requests goes through proxy which first check the requests for allowed websites and posted data to network. If request looks suspicious, proxy block the request – else request pass through.

When to use proxy design pattern- Applicability

The Proxy Pattern can be used in scenarios like, when data can be cached to improve responsiveness, the Real Subject is in a remote location and requires wrapping and unwrapping of requests and making remote calls. The Real Subject should be protected from unauthorized access, or they are expensive to create. A few of the different types of Proxy Pattern implementation are discussed below:

- **Cache Proxy(**Caching request results)

  The Cache Proxy **improves the responsiveness by caching relevant data** (that does not change frequently or is expensive to create) at the Proxy object level, when a client sends a request, the Proxy checks if the requested data is present with it if the data is found, it is returned to the client without sending the request to the Real Object.

- **Remote Proxy(**Local execution of a remote service)

  The Remote Proxy provides a **proxy object at a local level which represents an object present at another location**.

When to use proxy design pattern

- **Protection Proxy(**Access control)

  As its name suggests, the Protection Proxy is used to provide some **security for Real Subject at the proxy level**

- **Virtual Proxy(**Lazy initialization)

  Virtual Proxy is used in a scenario when the **Real Object is a complex object or expensive to create**. This is when you have a heavyweight service object that wastes system resources by being always up, even though you only need it from time to time.

Advantages of Proxy Design Pattern

- A few of the advantages of the Proxy Design Pattern are as follows:

The proxy works even if the service object isn't ready or is not available.

Open/Closed Principle - new proxies can be introduced without changing the service or clients.

Disadvantages of Proxy Design Pattern

- A few of the disadvantages of the Proxy Design Pattern are as follows:

The Proxy Design Pattern introduces an additional layer between the client and the Real Subject, this contributes to the code complexity.

The response from the service might get delayed.

Additional request forwarding is introduced between the client and the Real Subject.

Usage examples:

1.Image viewer program that lists and displays high resolution photos. The program has to show a list of all photos however it does not need to display the actual photo until the user selects an image item from a list.

2.The same for document editor that can embed graphical objects in a document. It isn't necessary to load all pictures when the document is opened, because not all of these objects will be visible at the same time.

3.The protective proxy acts as an authorization layer to verify if the actual user has access to appropriate content. An example can be thought about the proxy server which provides restrictive internet access in office. Only the websites and contents which are valid will be allowed and the remaining ones will be blocked.

4.Maybe used also for adding a thread-safe feature to an existing class without changing the existing class's code.

Example

Let's take a scenario where the real image contains a huge size data which clients needs to access. To save our resources and memory the implementation will be as below:

– Create an interface which will be accessed by the client. All its methods will be implemented by the ProxyImage class and RealImage class.

– RealImage runs on the different system and contains the image information is accessed from the database.

– The ProxyImage which is running on a different system can represent the RealImage in the new system. Using the proxy we can avoid multiple loading of the image.

Example

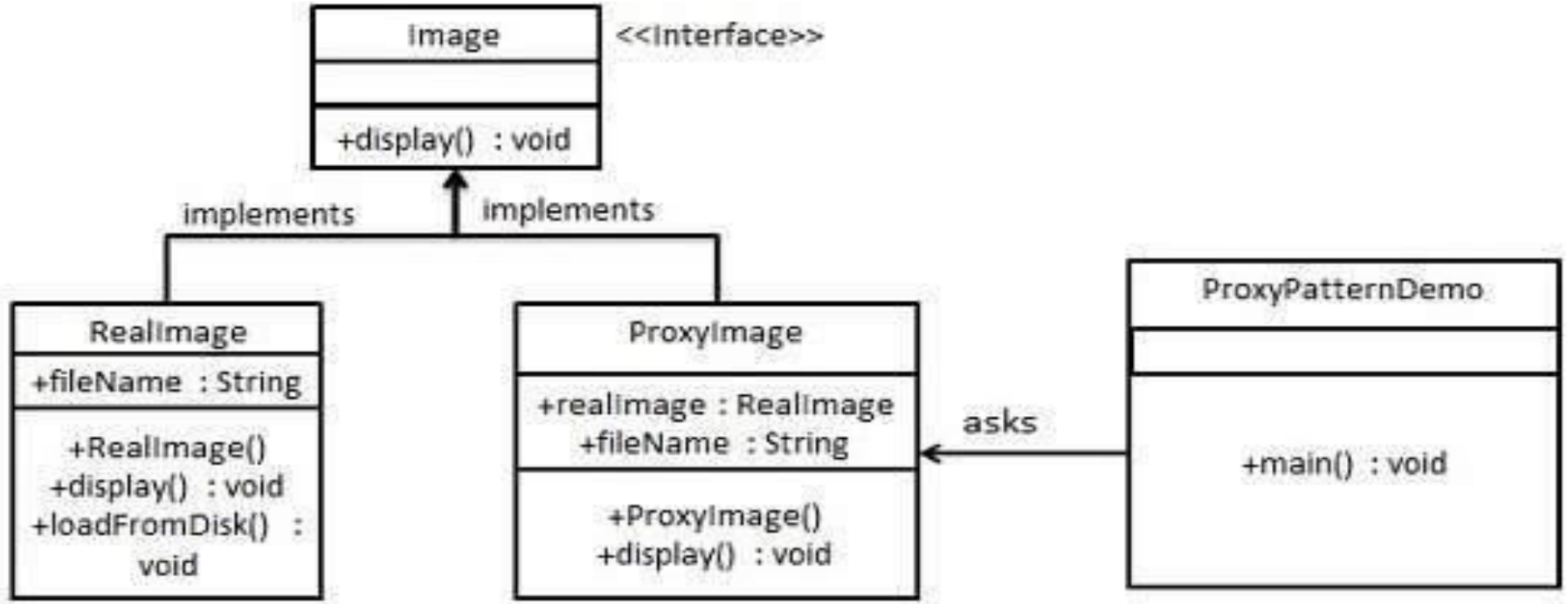
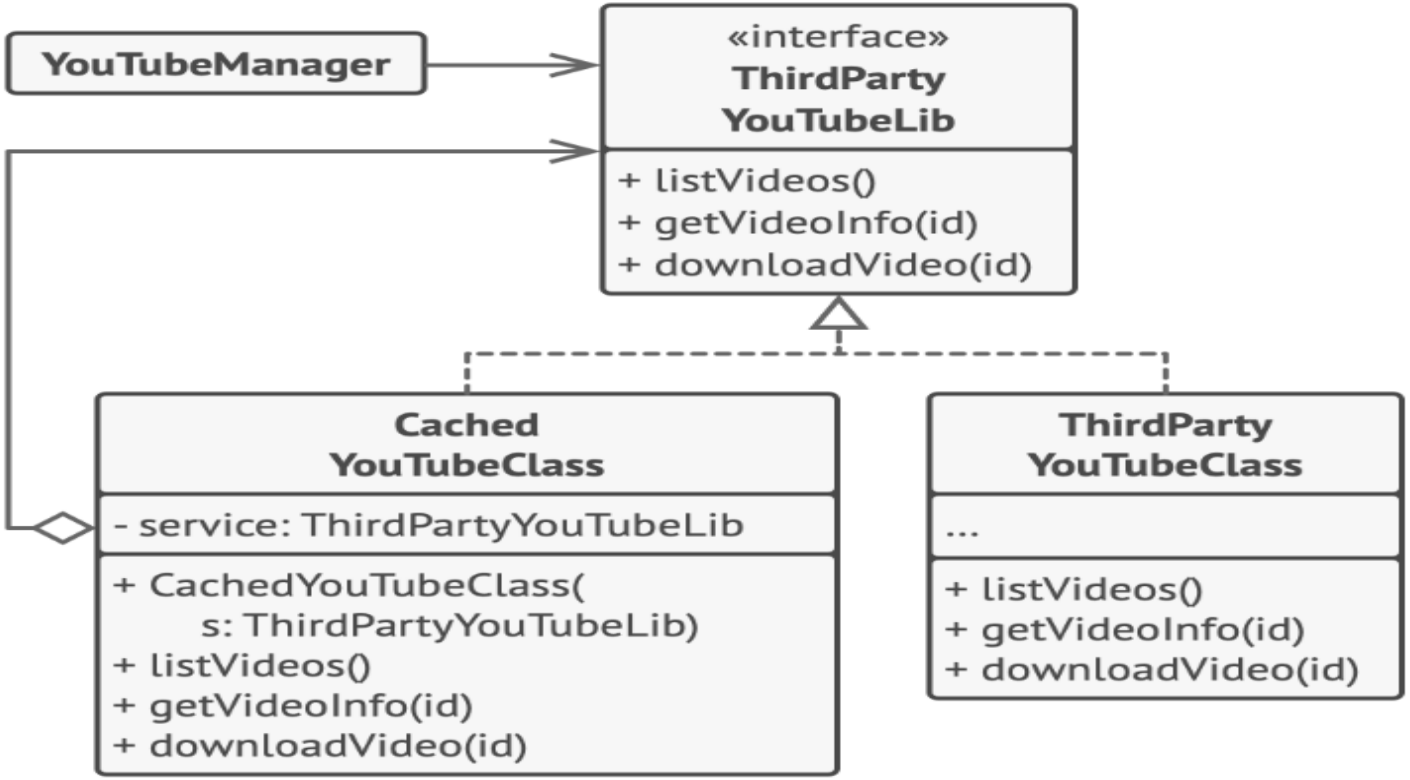


Image
<<Interface>>
+display() : void
implements
implements
RealImage
+fileName : String
+RealImage()
+display() : void
+loadFromDisk() : void
ProxyImage
+realImage : RealImage
+fileName : String
+ProxyImage()
+display() : void
asks
ProxyPatternDemo
+main() : void

Example

This example illustrates how the **Proxy** pattern can help to introduce lazy initialization and caching to a 3rd-party YouTube integration library.



Caching results of a service with a proxy.

Example

The library provides us with the video downloading class. However, it's very inefficient. If the client application requests the same video multiple times, the library just downloads it over and over, instead of caching and reusing the first downloaded file.

The proxy class implements the same interface as the original downloader and delegates it all the work. However, it keeps track of the downloaded files and returns the cached result when the app requests the same video multiple times.

MCQ

1. What is the primary role of a Proxy object?

A. To replace the original object permanently
B. To optimize memory allocation
C. To act as a placeholder or surrogate for another object
D. To eliminate object creation

2. What is one key use of the Proxy pattern?

A. Static binding
B. Lazy loading of objects
C. Eliminating inheritance
D. Database indexing

MCQ

3. What is the role of the Subject in the Proxy pattern?
A. Stores data
B. Defines the interface used by clients
C. Handles memory
D. Creates objects dynamically

4. What does a Virtual Proxy do?
A. Controls user permissions
B. Represents remote objects
C. Creates expensive objects only when needed
D. Caches data permanently

MCQ

5. What is the purpose of a Protection Proxy?

A. Improve performance
B. Handle remote communication
C. Control access rights to an object
D. Store cached data

6. In which scenario is Proxy commonly used?

A. Simple arithmetic calculations
B. Loading all resources at startup
C. Sorting algorithms
D. Controlling internet access in networks

MCQ SOLUTION

1.C
2.B
3.B
4.C
5.C
6.D

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

Unit 4
Structural Patterns - Flyweight

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements:
Prof. Saranya Devi B
Dept. of Computer Science and Engineering

Agenda

- ❑ Flyweight-definition
- ✓ Intent
- ✓ Motivation
- ✓ Structure
- ✓ Applicability
- ✓ Consequence
- ✓ Implementation
- ✓ Issues

Flyweight design pattern - Introduction

- The flyweight pattern is for sharing objects, where each instance does not contain its own state but stores it externally.
- This allows efficient sharing of objects to save space when there are many instances but a few different types.

Flyweight design pattern - Intent

- Use sharing to support large number of fine-grained objects efficiently.
- Flyweight is a structural design pattern that lets you fit more objects into the available amount of RAM by sharing common parts of state between multiple objects instead of keeping all of the data in each object.

Flyweight Pattern- Motivation

- An application uses many objects.
- A flyweight is a shared object that can be used in multiple contexts simultaneously.
- It acts as an individual object in each context.
- The key concept is the distinction between intrinsic and extrinsic state.

Problem

Placing the same information in many different places-

1. Mistakes can be made in copying the information.
2. If the data changes, all occurrences of the information must be located and changed. This makes maintenance of the document difficult.
3. Documents can become quite large if the same information is repeated over and over again.

Issues –

1. RAM overhead associated with each Instance.
2. CPU overhead associated with Memory management.

Solution

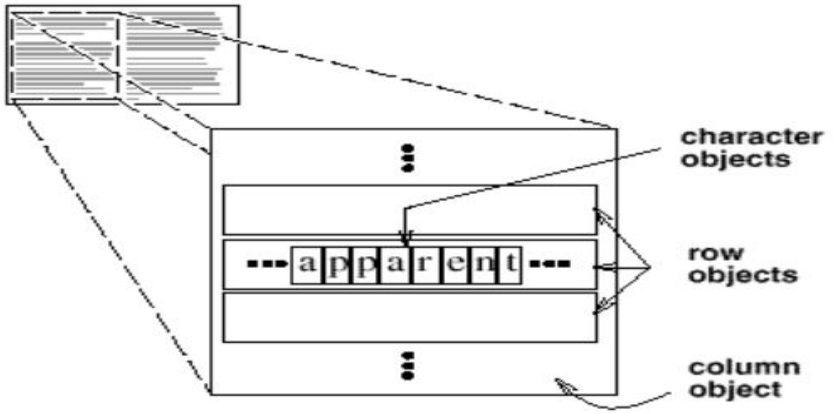
Create Flyweight Objects for Intrinsic state

- **Intrinsic State** – Information independent of the object's context, sharable(e.g state might include name, postal abbreviation, time zone etc.). This is included in the flyweight and instead of storing the same information n times for n objects, it is stored only once.
- **Extrinsic state** – Information dependent of the object's context, unshareable, stateless, having no stored values, but values that can be calculated on the spot(e.g. state might need access to region). This is excluded from the Flyweight.

Object Oriented Analysis and Design

Example: OO Document Editor

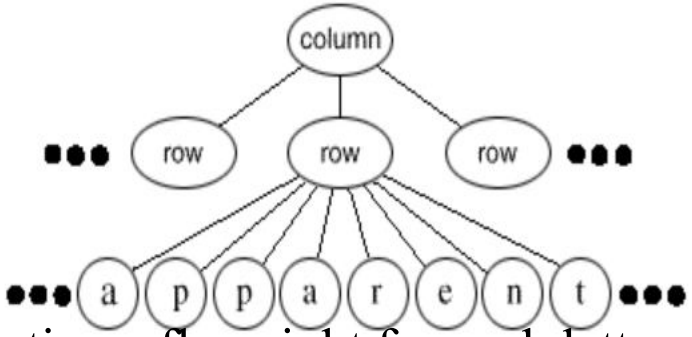
- Using an object for each character in the document:
- Promotes flexibility at the finest levels in the application.
- Character and embedded elements can be treated uniformly with respect to how they are drawn and formatted.
- The application could be extended to support new character sets without disturbing other functionalities.
- The application's object structure could mimic the document's physical structure.



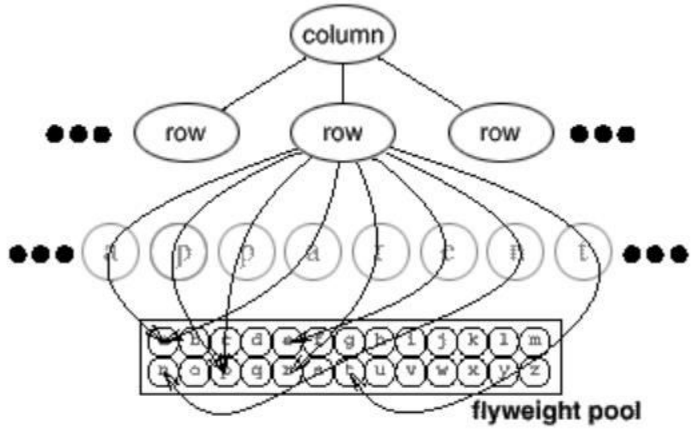
Object Oriented Analysis and Design

Example: OO Document Editor

Logically



physically

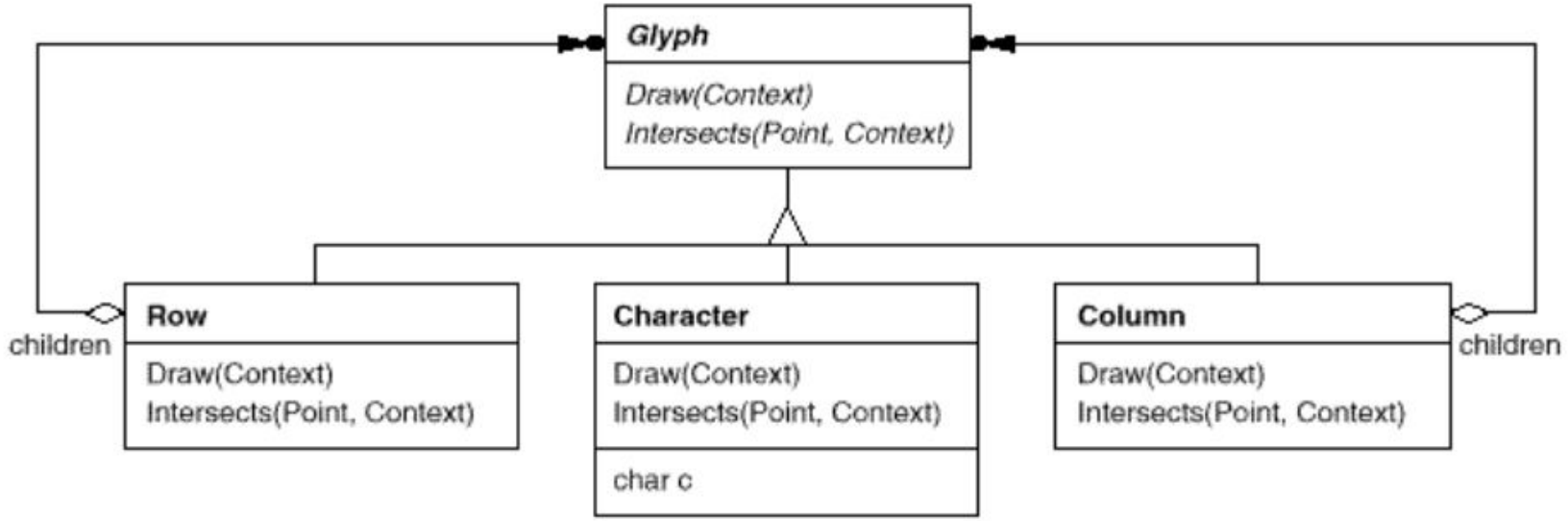


Creating a flyweight for each letter of the alphabet:
Intrinsic state: A character code
Extrinsic state: Coordinate position in the document

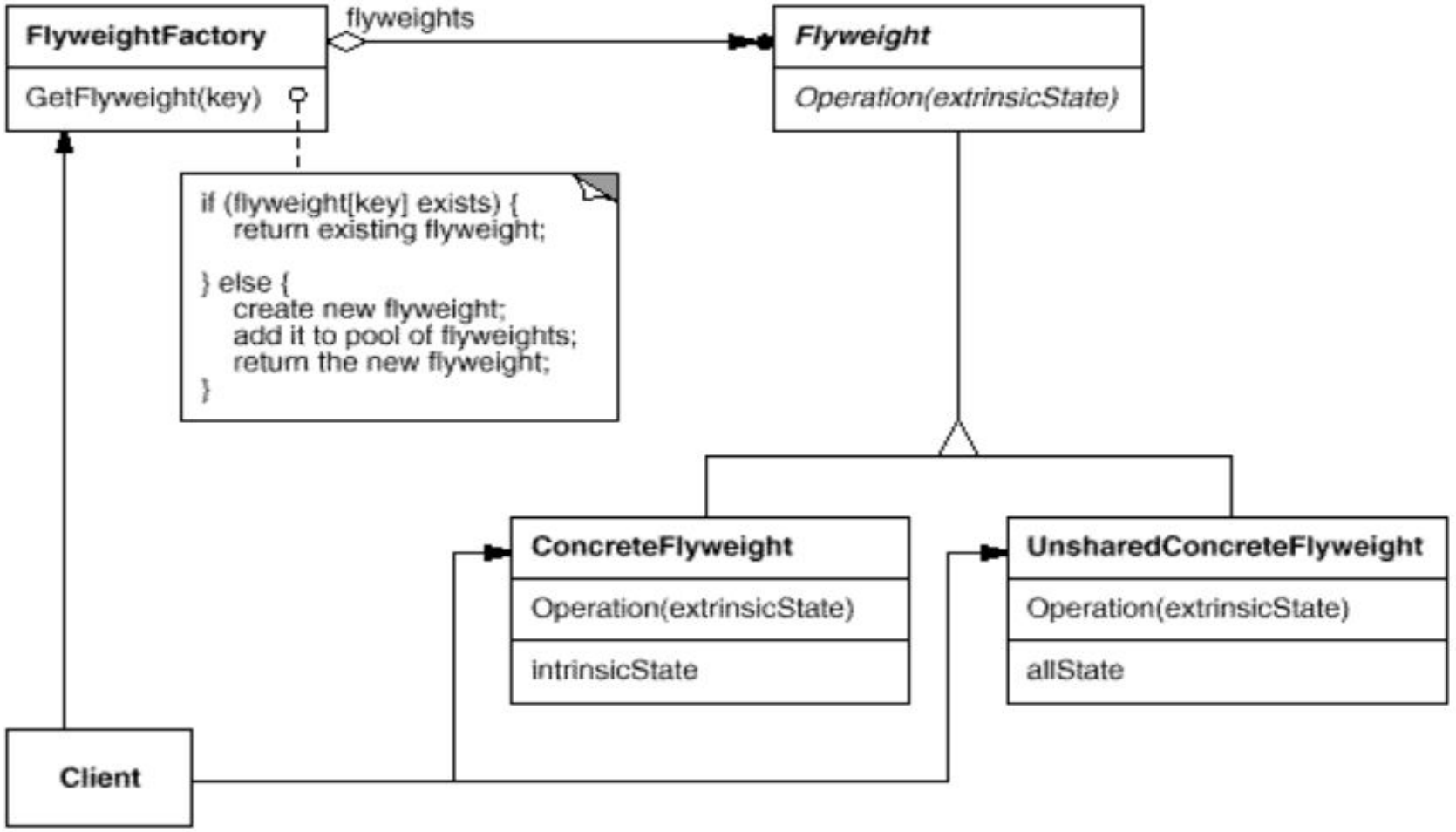
Typographic style(font, color)

Is determined from the text layout algorithms and formatting commands in effect wherever the character appears.

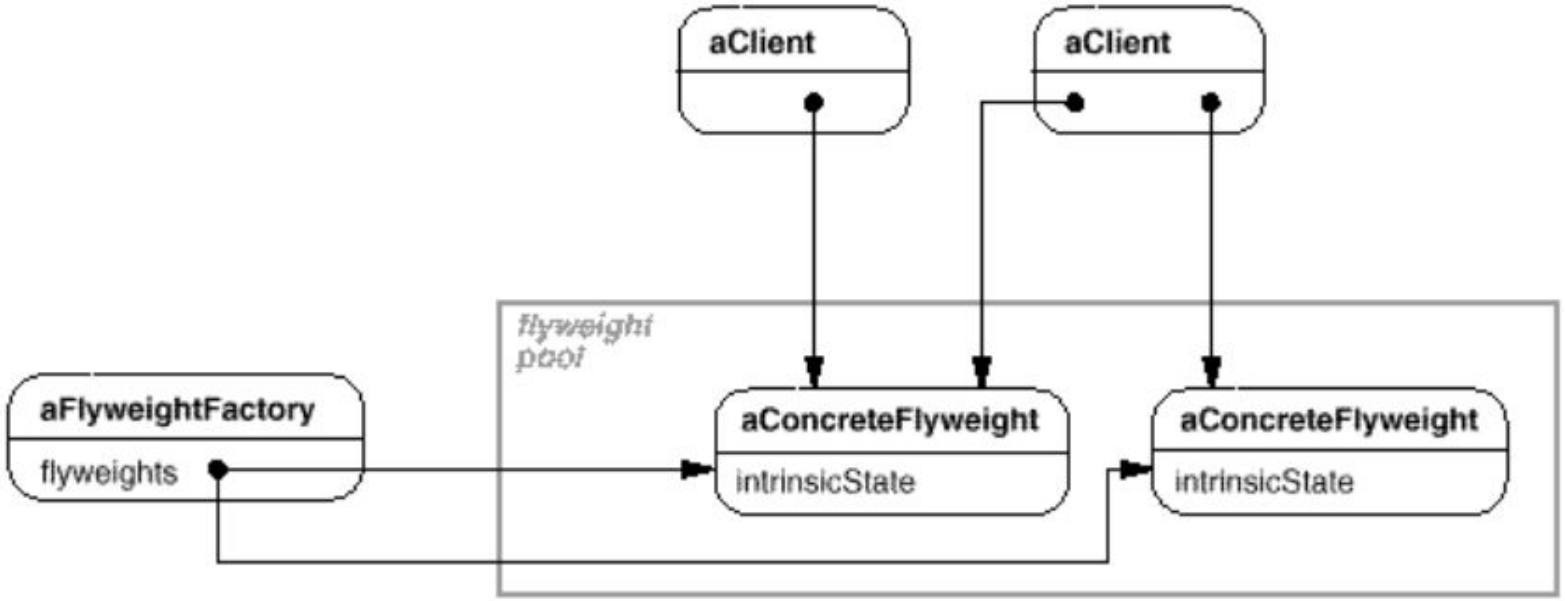
Class Structure



Structure


FlyweightFactory
GetFlyweight(key)
flyweights
Flyweight
Operation(extrinsicState)
if (flyweight[key] exists) {
return existing flyweight;
} else {
create new flyweight;
add it to pool of flyweights;
return the new flyweight;
}
ConcreteFlyweight
Operation(extrinsicState)
intrinsicState
UnsharedConcreteFlyweight
Operation(extrinsicState)
allState
Client

Sharing of flyweights



Flyweight Participants

- **Flyweight (Glyph)**
  - declares an interface through which flyweights can receive and act on extrinsic state.
- **ConcreteFlyweight (CharacterGlyph)**
  - implements the Flyweight interface and adds storage for intrinsic state, if any. A ConcreteFlyweight object must be sharable. Any state it stores must be intrinsic; that is, it must be independent of the ConcreteFlyweight object's context.
- **UnsharedConcreteFlyweight (Row, Column)**
  - not all Flyweight subclasses need to be shared. The Flyweight interface *enables* sharing; it doesn't enforce it. It's common for UnsharedConcreteFlyweight objects to have ConcreteFlyweight objects as children at some level in the flyweight object structure (as the Row and Column classes have).
- **FlyweightFactory (GlyphFactory)**
  - creates and manages flyweight objects.
  - ensures that flyweights are shared properly. When a client requests a flyweight, the FlyweightFactory object supplies an existing instance or creates one, if none exists.
- **Client**
  - maintains a reference to flyweight(s).
  - computes or stores the extrinsic state of flyweight(s).

Applicability

- An application uses a large number of objects.
- Storage costs are high because of the sheer quantity of objects.
- Most object state can be made extrinsic.
- Many groups of objects may be replaced by relatively few shared objects once extrinsic state is removed.
- The application doesn't depend on object identity. Since flyweight objects may be shared, identity tests will return true for conceptually distinct objects.

Consequences

- Disadvantage: Flyweights may introduce run-time costs associated with transferring, finding, and/or computing extrinsic state, especially if it was formerly stored as intrinsic state.
- Advantage: Storage savings are a function of several factors:
  - The reduction in the total number of instances that comes from sharing.
  - The amount of intrinsic state per object
  - Whether extrinsic state is computed or stored.

Implementation

- Removing extrinsic state - The pattern's applicability is determined largely by how easy it is to identify extrinsic state and remove it from shared objects. Removing extrinsic state won't help reduce storage costs if there are as many different kinds of extrinsic state as there are objects before sharing.

- Managing shared objects - Because objects are shared, clients shouldn't instantiate them directly. FlyweightFactory lets clients locate a particular flyweight. FlyweightFactory objects often use an associative store to let clients look up flyweights of interest.

Examples:

- Suppose we have a **pen** which can exist with/without **refill**. A refill can be of any color thus a pen can be used to create drawings having N number of colors.

Here Pen can be flyweight object with refill as extrinsic attribute. All other attributes such as pen body, pointer etc. can be intrinsic attributes which will be common to all pens. A pen will be distinguished by its refill color only, nothing else.

All application modules which need to access a red pen – can use the same instance of red pen (shared object). Only when a different color pen is needed, application module will ask for another pen from flyweight factory.

- In programming, we can see **java.lang.String** constants as flyweight objects. All strings are stored in string pool and if we need a string with certain content then runtime return the reference to already existing string constant from the pool – if available.
- In browsers, we

- use an image in multiple places in a webpage. Browsers will load the image only one time, and for other times browsers will reuse the image from cache. Now image is same but used in multiple places. It's URL is intrinsic attribute because it's fixed and shareable. Images position coordinates, height and width are extrinsic attributes which vary according to place (context) where they have to be rendered.

MCQ

1. What is the primary goal of the Flyweight pattern?

A. Reduce memory usage by sharing objects
B. Increase object creation
C. Eliminate abstraction
D. Improve inheritance

2. What is intrinsic state in Flyweight?

A. Context-dependent and unshared state
B. State stored externally
C. Context-independent and sharable state
D. Temporary state

MCQ

3. What is extrinsic state in Flyweight?

A. Shared and stored inside object
B. Independent of context
C. Dependent on context and not shared
D. Stored permanently

4. What is the role of FlyweightFactory?

A. Deletes objects
B. Creates and manages shared flyweight objects
C. Executes business logic
D. Handles user input

MCQ

5. What must be true for Flyweight objects?
A. They must store all state internally
B. They must not be shared
C. They must be sharable
D. They must be immutable

6. What is an example of Flyweight usage?
A. Database indexing
B. String constant pool
C. Sorting algorithm
D. File compression

MCQ SOLUTION

1.A
2.C
3.C
4.B
5.C
6.B

References

- https://www.cs.unc.edu/~stotts/GOF/hires/pat4ffso.htm
- https://refactoring.guru/design-patterns/flyweight
- https://howtodoinjava.com/design-patterns/structural/flyweight-design-pattern/
- Design Patterns: Elements of Reusable Object-Oriented SoftwareErich Gamma, Ralph Johnson, Richard Helm · 1995

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

OO Behavioral Design Patterns with Sample implementation in Java

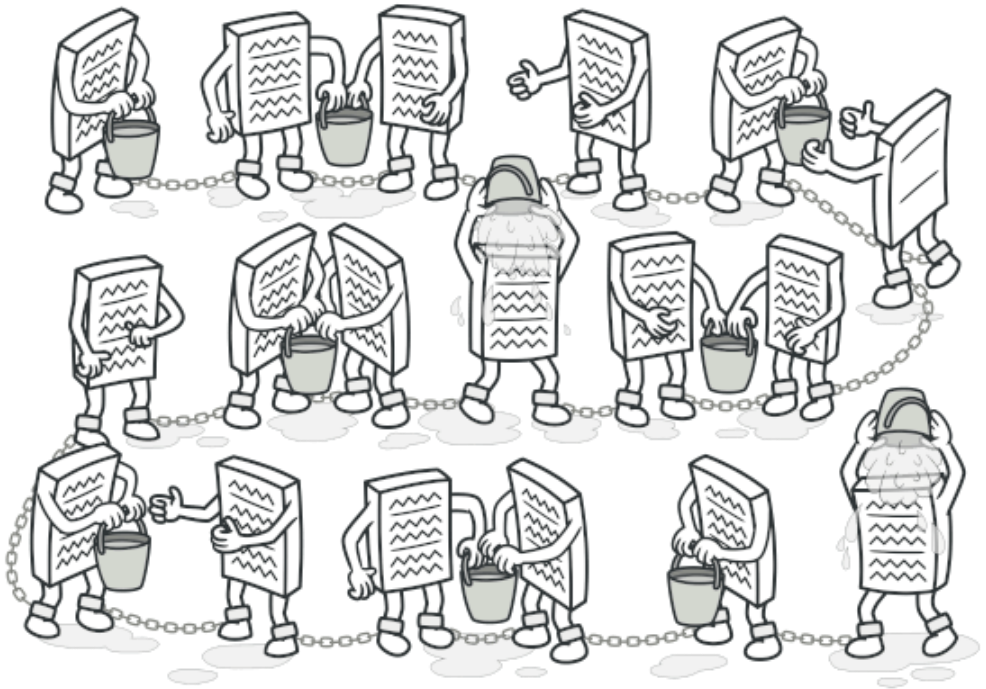
Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements:
Prof. Vinay Joshi
Dept. of Computer Science and Engineering

PES UNIVERSITY

Unit-4

Behavioral Patterns – Chain of Responsibility Pattern



Behavioral patterns

Behavioral design patterns are design patterns that identify common communication patterns between objects and realize these patterns.

These patterns increase flexibility in carrying out this communication.

Behavioral patterns influence how state and behavior flow through a system.

By optimizing how state and behavior are transferred and modified, you can simplify, optimize, and increase the maintainability of an application

List of common behavioral design patterns

1. **Chain of responsibility:** Command objects are handled or passed on to other objects by logic-containing processing objects
2. **Command:** Command objects encapsulate an action and its parameters
3. **Interpreter:** Implement a specialized computer language to rapidly solve a specific set of problems
4. **Iterator:** Iterators are used to access the elements of an aggregate object sequentially without exposing its underlying representation

Mediator: Provides a unified interface to a set of interfaces in a subsystem

Memento: Provides the ability to restore an object to its previous state (rollback)

Observer: also known as Publish/Subscribe or Event Listener. Objects register to observe an event that may be raised by another object

State: A clean way for an object to partially change its type at runtime

Strategy: Algorithms can be selected on the fly

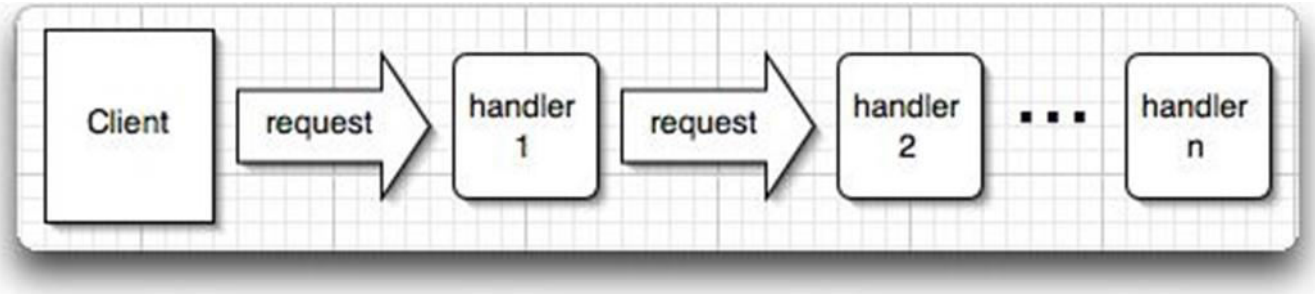
Template Method: Describes the program skeleton of a program

Visitor: A way to separate an algorithm from an object

Chain of Responsibility Pattern

The Chain of Responsibility pattern establishes a chain within a system, so that a message can either be handled at the level where it is first received, or be directed to an object that can handle it.

it is used to manage algorithms, relationships and responsibilities between objects



Intent

Avoid coupling sender-of-request to its receiver by giving more than one object a chance to handle request.

Chain receiving objects and pass request along until an object handles it.

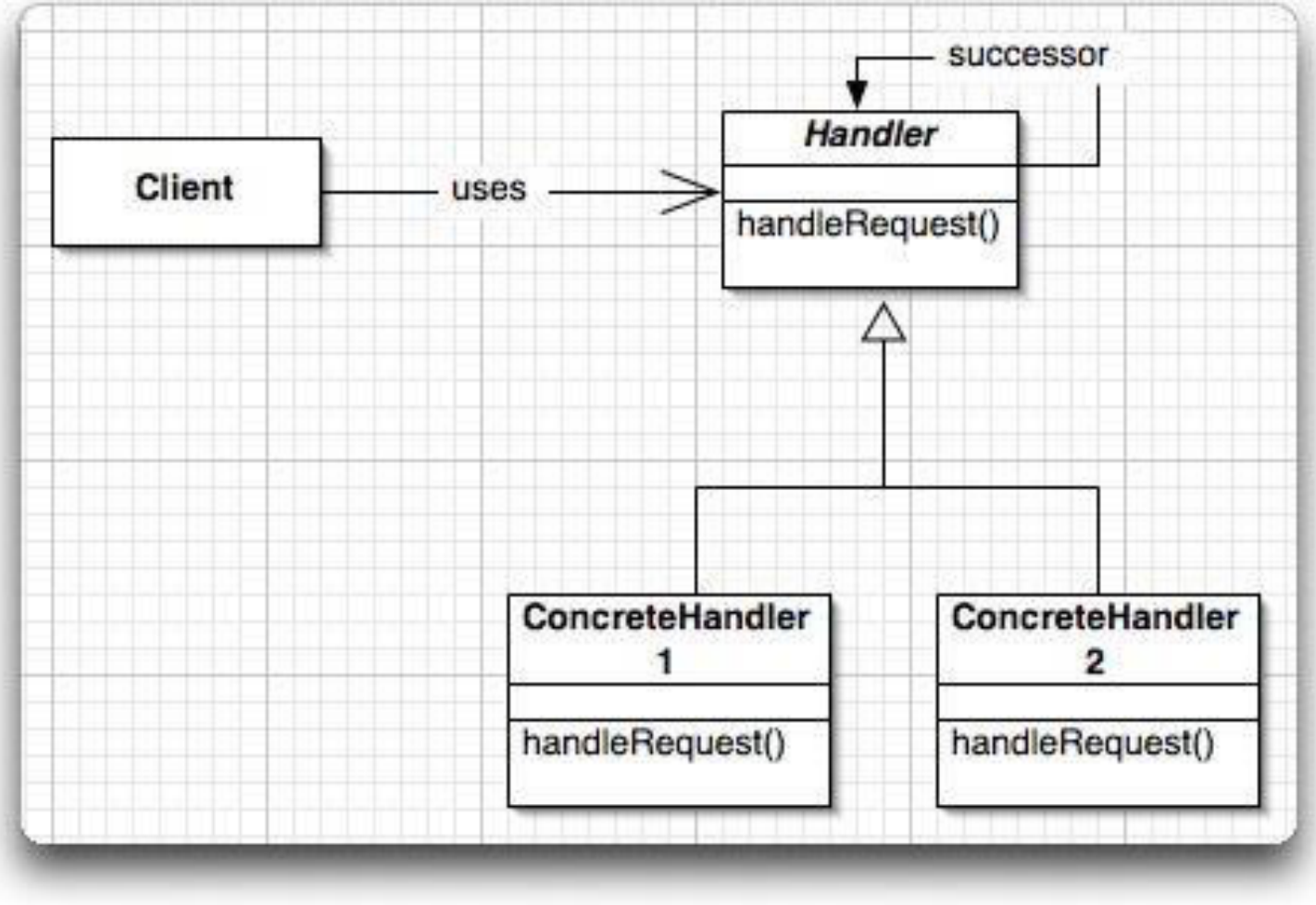
Motivation

The Chain of Responsibility is intended to promote loose coupling between the sender of a request and its receiver by giving more than one object an opportunity to handle the request.

The receiving objects are chained and pass the request along the chain until one of the objects handles it.

The set of potential request handler objects and the order in which these objects form the chain can be decided dynamically at runtime by the client depending on the current state of the application.

Structure


successor
Handler
handleRequest()
Client
uses
ConcreteHandler 1
handleRequest()
ConcreteHandler 2
handleRequest()

Participants

Handler:

- This can be an interface which will primarily receive the request and dispatches the request to a chain of handlers. It has reference to only the first handler in the chain and does not know anything about the rest of the handlers.

ConcreteHandler:

- handles requests it is responsible for
- can access its successor
- if it does not handle the request, forwards the request to its successor

Client:

- initiates the request to a ConcreteHandlerObject on the chain

Object Oriented Analysis and Design

Example – Scenario 1

Scenario

An enterprise has been getting more email than they can handle. The enterprise gets 4 types of e-mail.

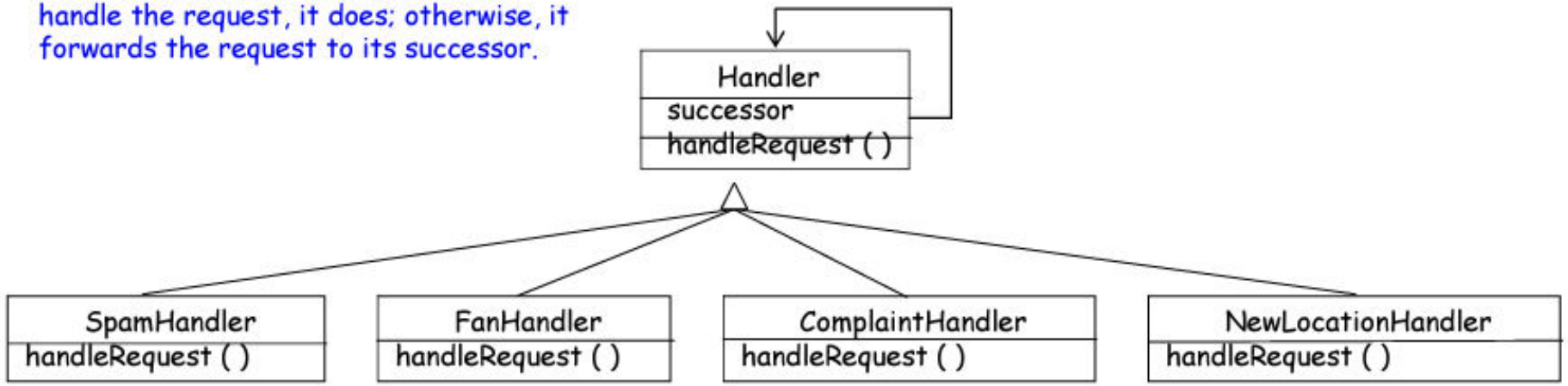
They are:

- Fan mail with compliments,
- Complaints,
- Requests for new features,
- Spam.

Your task:

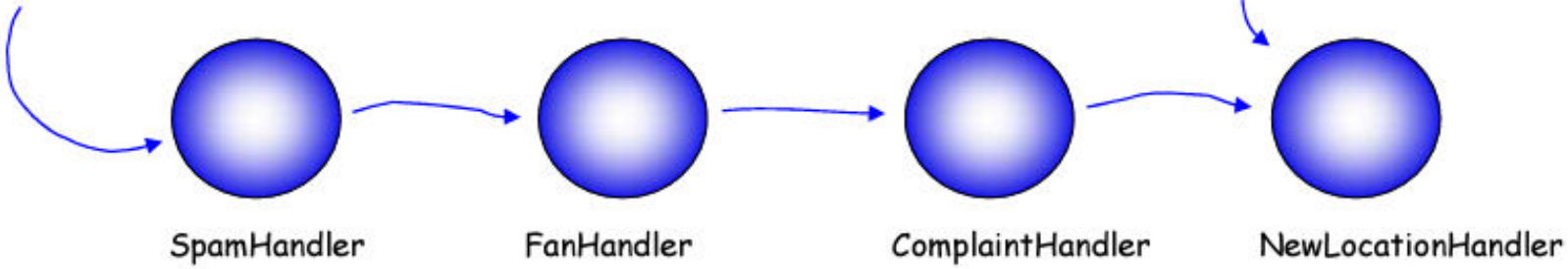
- The enterprise has already written the AI detectors that can tell whether an email is fan, complaint, request or spam,
- You need to create a design that can use the detectors to handle incoming email.

Each object in the chain acts as a handler and has a successor object. If it can handle the request, it does; otherwise, it forwards the request to its successor.



Each email is passed to the first handler

Email is not handled if it falls off the end of the chain -- although, you can always implement a catch-all handler



Consequences

Reduced Coupling

- Objects are free from knowing what object handles the request

Added Flexibility in assigning responsibilities to objects

- Can change chain at runtime
- Can subclass for special handlers

Receipt is NOT guaranteed

- Request could fall off the chain
- Request could be dropped with bad chain

Applicability

Use Chain of Responsibility

when

- More than one object may handle a request and the handler isn't known a priori.
- You want to issue a request to one of several objects without specifying the receiver explicitly
- The Set of objects than can handle a request should be specified dynamically

Implementation – Example Scenario 2

Let us consider the transaction approval process in a company. Suppose we want to approve the transactions based on certain conditions?

For instance, in the transaction approval application user keys in transaction details into the application and this transaction need to be processed by any one of the higher level employee in the company.

All transactions lesser than 1,00,000 amount can be approved by manager, lesser than 10,00,000 can be approved by vice president and lesser than 25,00,000 can be approved by CEO.

Implementation – Example Scenario 2

The transaction approval behavior can be implemented with a simple if else conditions, by checking if amount is less than 1,00,000 or else if amount is less than 10,00,000 and so on. But this approach is static, means we can not change these conditions dynamically. Chain of responsibility design pattern can be used in this situation.

In the transaction processing application, approval process acts like a chain of responsibilities.

First, manager acts on the transaction, if the amount is higher than his approval limit then the transaction is transferred to vice president and so on.

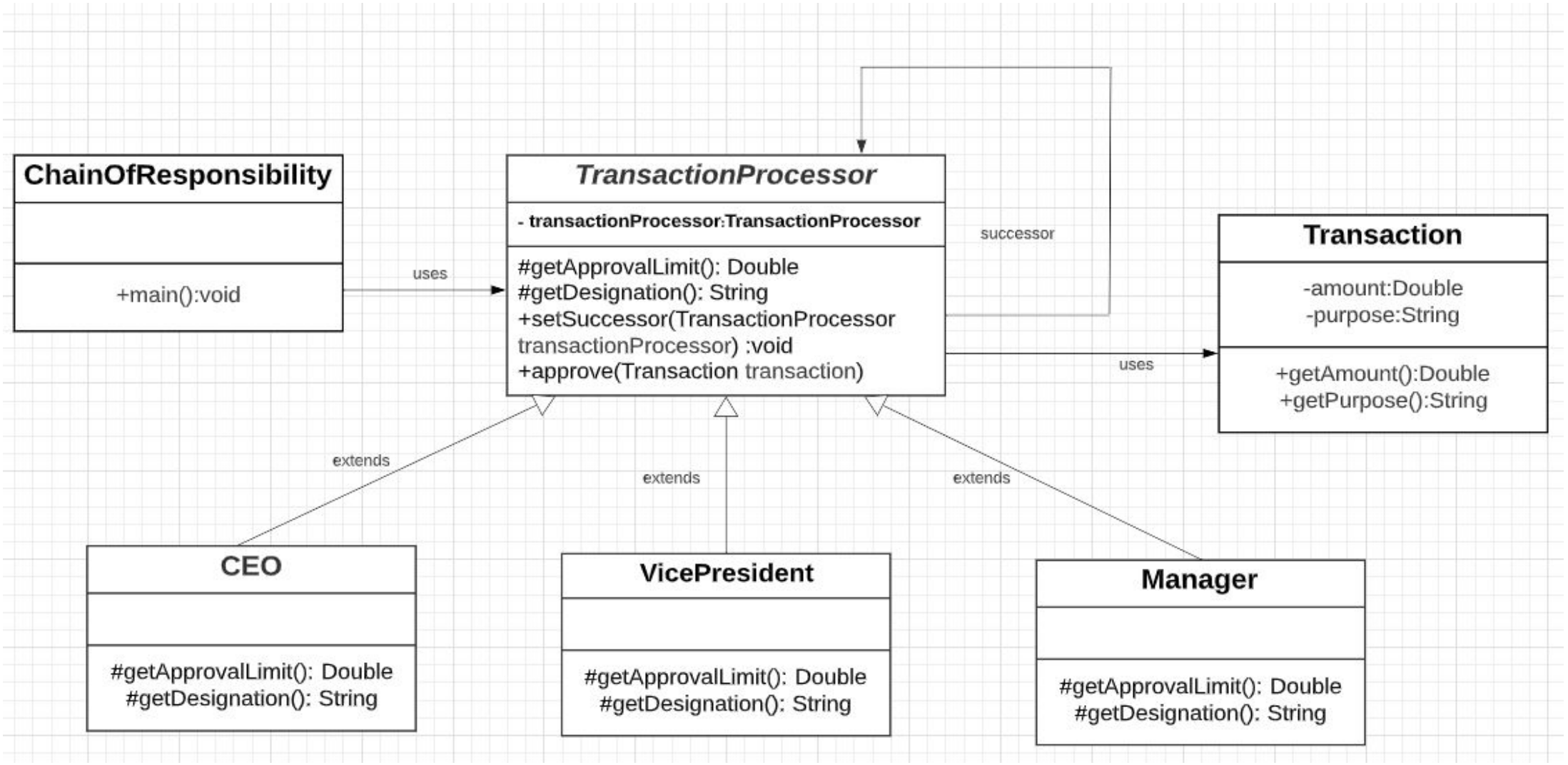
Object Oriented Analysis and Design

Implementation – Example Scenario 2

We need to design processing and command objects. Based on the above example, processing objects are employees like Manager, Vice President because they process the transaction by approving and command object is the transaction. Once processing objects are created then we need to chain them together like manager -> vice president -> CEO and pass the transaction at the beginning of the chain. Transaction continues to flow in the chain until it reaches the employee, who's limit allows to approve it.

Object Oriented Analysis and Design

UML Class Diagram



Implementation – Example Scenario 2

First create an abstract class for processing the transactions.

If the transaction can be processed then it should be passed to its successor so this class store

its next successor object.

If transaction is with in the limit then it prints the message.

If it is out of range then passes it to next processor by calling ‘approve’ method,

before calling approve check for null if processor request is last in the chain.

Object Oriented Analysis and Design

Implementation – Example Scenario 2

```
abstract class TransactionProcessor
{
    private TransactionProcessor transactionProcessor;

    abstract protected Double getApprovalLimit();
    abstract protected String getDesignation();

    public void setSuccessor(TransactionProcessor
transactionProcessor)
    {
        this.transactionProcessor = transactionProcessor;
    }

    public void approve(Transaction transaction)
    {
        if (transaction.getAmount() <= 0.0)
        {
            System.out.println("Invalid Amount. Amount should be
> 0");
            return;
        }
```

```
 if (transaction.getAmount() <= getApprovalLimit())
        {
            System.out.println("Transaction for amount " +
transaction.getAmount() +
                               " approved by " + getDesignation());
        }
        else
        {
            if (transactionProcessor == null)
            {
                System.out.println("Invalid Amount. Amount should not
exceed 25lacs!");
                return;
            }

            transactionProcessor.approve(transaction);
        }
    }
}
```

Implementation – Example Scenario 2

Now we need to create concrete class for each approver (Manager, Vice President, CEO) and this class will extend TransactionProcessor.

Each processing class sets its limit and designation.

```
class Manager extends TransactionProcessor
{
  @Override
    protected Double getApprovalLimit()
    {
        return 100000.0;
    }

    @Override
    protected String getDesignation()
    {
        return "manager";
    }
}
```

Implementation – Example Scenario 2

```
class VicePresident extends TransactionProcessor
{

    @Override
    protected Double getApprovalLimit()
    {
        return 1000000.0;
    }

    @Override
    protected String getDesignation()
    {
        return "Vice President";
    }
}
```

Implementation – Example Scenario 2

```
class CEO extends TransactionProcessor{

    @Override
    protected Double getApprovalLimit() {
        return 2500000.0;
    }

    @Override
    protected String getDesignation() {
        return "CEO";
    }
}
```

Object Oriented Analysis and Design

Implementation – Example Scenario 2

```
class Transaction
{
    private Double amount;
    private String purpose;

    Transaction(Double amount, String purpose)
    {
        this.amount = amount;
        this.purpose = purpose;
    }

    public Double getAmount()
    {
        return amount;
    }

    public String getPurpose()
    {
        return purpose;
    }
}
```

Object Oriented Analysis and Design

Implementation – Example Scenario 2

Now let us glue all these classes together to form chain of responsibility pattern using ChainOfResponsibility class.

- Create processing objects. We have three processing objects in the chain.
- Chain together all the processing objects. Order of the objects is important because manager handover transaction to vice president.
- Because the chain starts from the manager so transactions are pushed using the manager instance.

```
public class ChainOfResponsibility {
public static void main(String[] args) {
        Manager manager = new Manager();
        VicePresident vicePresident = new VicePresident();
        CEO ceo = new CEO();
        manager.setSuccessor(vicePresident);
        vicePresident.setSuccessor(ceo);
        manager.approve(new Transaction(2600000.0, "general"));
        manager.approve(new Transaction(50000.0, "general"));
        manager.approve(new Transaction(120000.0, "general"));
        manager.approve(new Transaction(1500000.0, "general"));
        manager.approve(new Transaction(0.0, "general"));
}
}
```

Creating Chain

manager class returns its approval limit amount. TransactionProcessor class approve method checks whether the given amount is <= approval limit. If it returns false it propagates the request to the successor VicePresident class to get its approval limit and so on. In case the request can not be serviced then null object will be return for the amount exceeding 25lacs.

Implementation – Example Scenario 2

Output

```
Invalid Amount. Amount should not exceed 25lacs!
Transaction for amount 50000.0 approved by manager
Transaction for amount 120000.0 approved by vice president
Transaction for amount 1500000.0 approved by CEO
Invalid Amount. Amount should be > 0
```

Benefits, Uses and Drawbacks

Benefits:

- Decouples the sender of the request and its receivers,
- Simplifies your object because it doesn't have to know the chain's structure and keep direct reference to its members,
- Allows you to add or remove responsibilities dynamically by changing the members or the order of the chain.

Uses:

- Commonly used in Windows systems to handle events like mouse clicks and keyboard events.
- In java it is used in handling chain of Exceptions

Drawbacks:

- Execution of the request isn't guaranteed; it may fall off the end of the chain if no object handles it (this can be an advantage or a disadvantage),
- Can be hard to observe the runtime characteristics and debug.

PES UNIVERSITY

MCQ

1. What is the main idea behind the Chain of Responsibility pattern?
A. Objects communicate using inheritance only
B. Requests are handled by multiple objects sequentially in a chain
C. Requests are always handled by the first object
D. Objects directly call each other

2. How are requests processed in the Chain of Responsibility pattern?
A. All handlers process the request simultaneously
B. Requests are broadcast to all handlers
C. Requests are passed along a chain until handled
D. Only the client processes the request

PES UNIVERSITY

MCQ

3. What is the role of the Handler in the Chain of Responsibility pattern?
A. Stores application data
B. Receives requests and dispatches them to handlers
C. Creates objects dynamically
D. Stores database connections

4. What is the responsibility of a ConcreteHandler?
A. Handles all requests in the system
B. Only stores requests
C. Handles the request it is responsible for and forwards others
D. Creates the chain dynamically

PES UNIVERSITY

MCQ

5. When should the Chain of Responsibility pattern be used?

A. When only one object must handle the request
B. When multiple objects may handle a request
C. When objects cannot communicate
D. When there is only one handler

6. What is a drawback of the Chain of Responsibility pattern?

A. It increases memory usage drastically
B. Execution of request is not guaranteed
C. It cannot be implemented in Java
D. It does not support object-oriented design

PES UNIVERSITY

MCQ SOLUTION

1.B
2.C
3.B
4.C
5.B
6.B

References

PES UNIVERSITY

Text Reference

Design Patterns: Elements of Reusable Object-Oriented Software, GOF

Web Reference

https://www.cs.uah.edu/~rcoleman/CS307/DesignPatterns/DP17-ChainOfResponsibility.html
https://refactoring.guru/design-patterns
https://thetechstack.net/chain-of-responsibility-design-pattern/

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

OO Behavioral Design Patterns with Sample implementation in Java

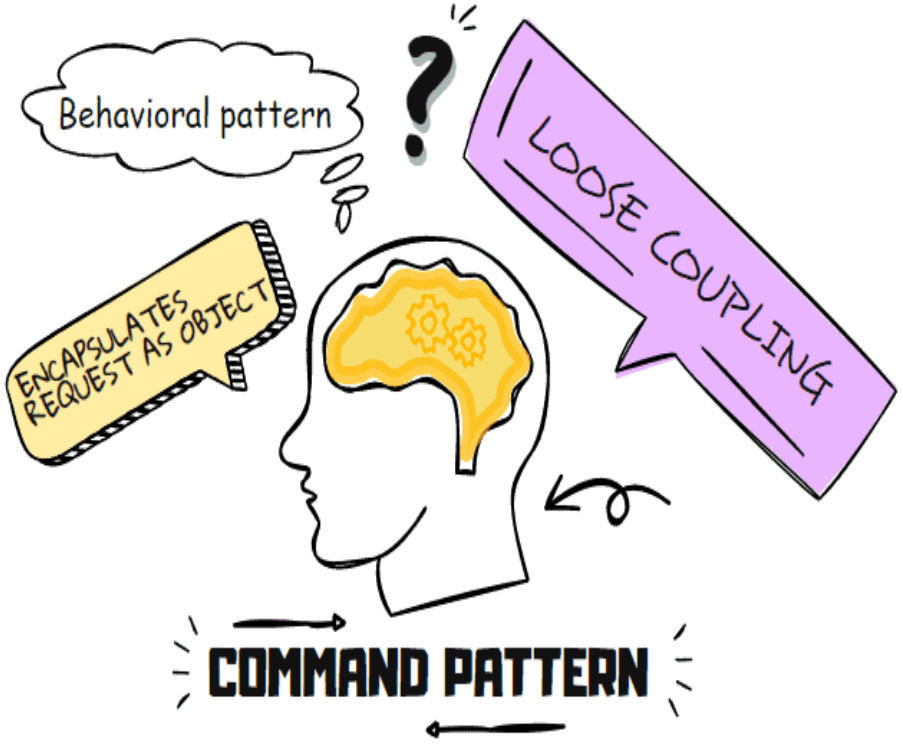
Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements:
Prof. Vinay Joshi
Dept. of Computer Science and Engineering

PES UNIVERSITY

Unit-4

Behavioral Patterns – Command Pattern



Command Pattern

Command pattern is a data-driven design pattern and falls under behavioral patterns.

A request is wrapped under an object as command and passed to an invoker object.

The Invoker then searches for the appropriate object to handle the Command and passes it to the corresponding object that can execute it.

The object, called Command, contains all the information needed to perform an action or trigger an event, such as the method name, the method owner, and the arguments for method parameters.

Intent

- encapsulate a request in an object
- allows the parameterization of clients with different requests
- allows saving the requests in a queue

Motivation

Sometimes it's necessary to issue requests to objects without knowing anything about the operation being requested or the receiver of the request.

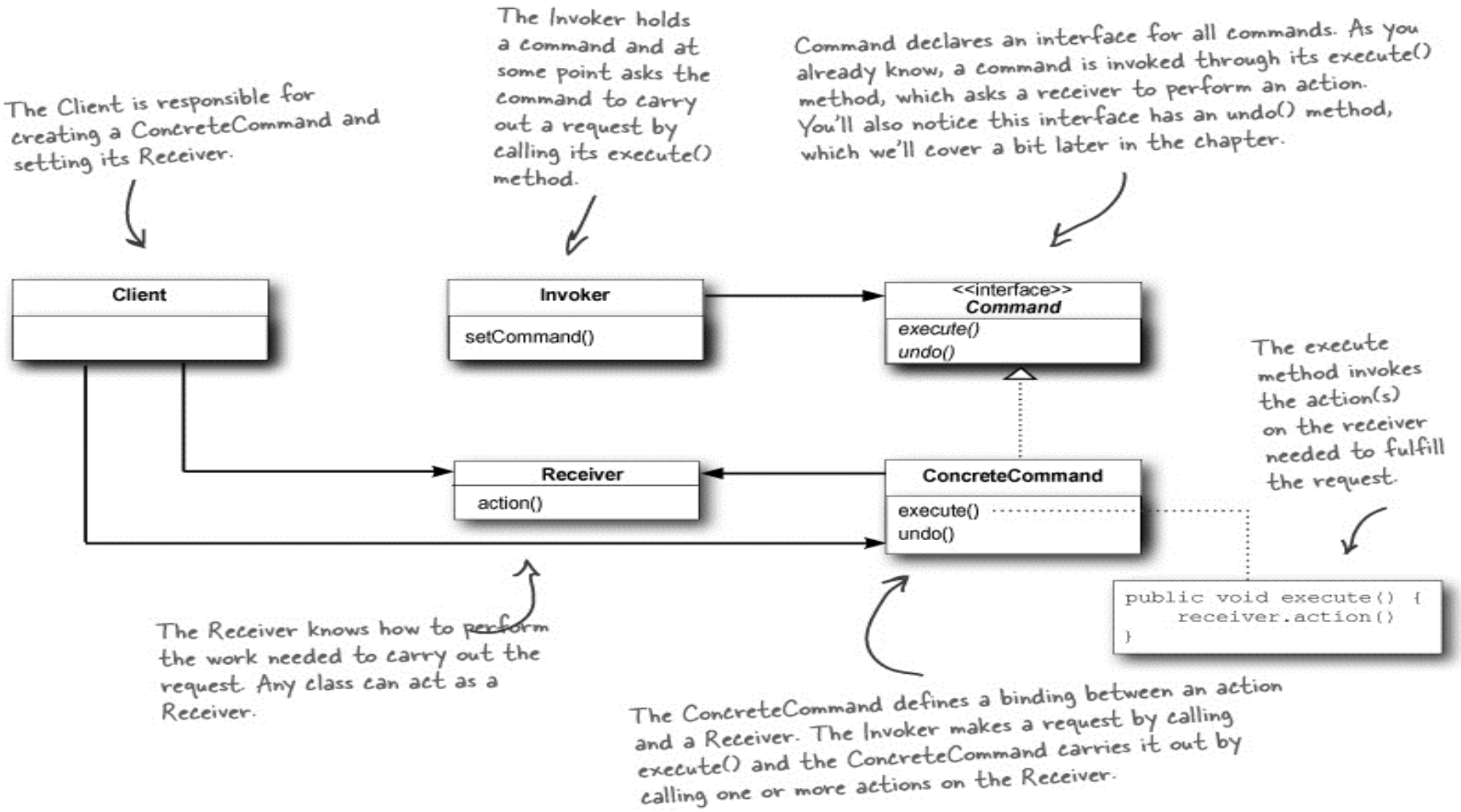
For example, user interface toolkits include objects like buttons and menus that carry out a request in response to user input.

But the toolkit can't implement the request explicitly in the button or menu, because only applications that use the toolkit know what should be done on which object. As toolkit designers we have no way of knowing the receiver of the request or the operations that will carry it out.

The Command pattern lets toolkit objects make requests of unspecified application objects by turning the request itself into an object.

Object Oriented Analysis and Design

Structure



Participants

Command: Declares an interface for executing an operation.

ConcreteCommand

- Defines a binding between a Receiver object and an action.
- Implements execute() by invoking a corresponding operation on Receiver.

Client (Application): Creates a Command object and sets its Receiver.

Invoker: Asks the Command to carry out a request.

Receiver: Knows how to perform the operation associated with a request. Can be any class.

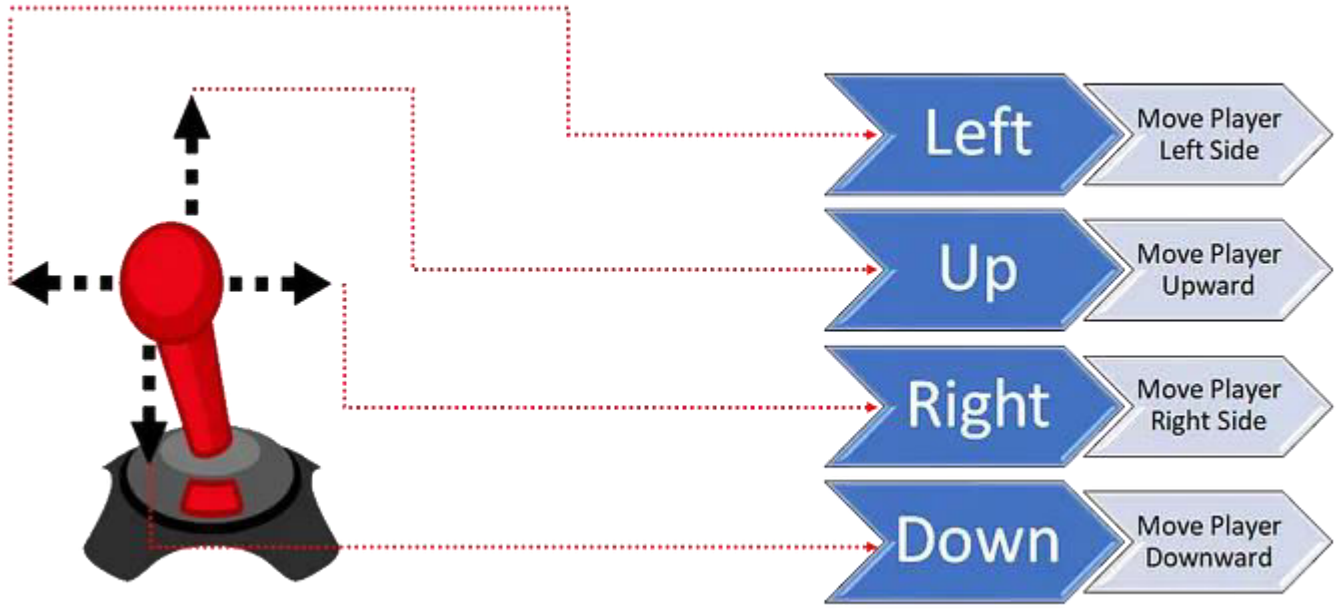
Collaborations

The Client creates a ConcreteCommand object and specifies its receiver.
An Invoker object stores the ConcreteCommand object.
The invoker issues a request by calling execute( ) on the command. When commands are undoable, ConcreteCommand stores state for undoing prior to invoking execute( ).
The ConcreteCommand object invokes operations on its receiver to carry out the request.

Example – Scenario 1

Command pattern helps to control the movement of the video game player using a joystick.

Based on the inputs being received for joysticks, we can invoke the commands which will move the player.



Consequences

Command decouples the object that invokes the operation from the one that knows how to perform it.

Commands are first-class objects. They can be manipulated and extended like any other object.

Its easy to add new Commands, because you don't have to change existing classes.

Commands can be assembled into a composite command.

Command Pattern has the following Limitations

- Must create a concrete command for each request
- May end up with large number of objects if the classes were not managed carefully

Applicability

parameterize objects by actions they perform(a way to implement callbacks)

specify, queue, and execute requests at different times (Partitioning complex requests)

Application that needs to support an undo command (single or multi-level)

Need to support change log for recovery purposes

Transaction based applications (Extendable)

Applications that learn about the receiver dynamically

Implementation – Example Scenario 2

Let us think of placing orders for buying and selling stocks. To implement the solution create an interface Order which acts as a command. Next create Stock class which acts as a request.

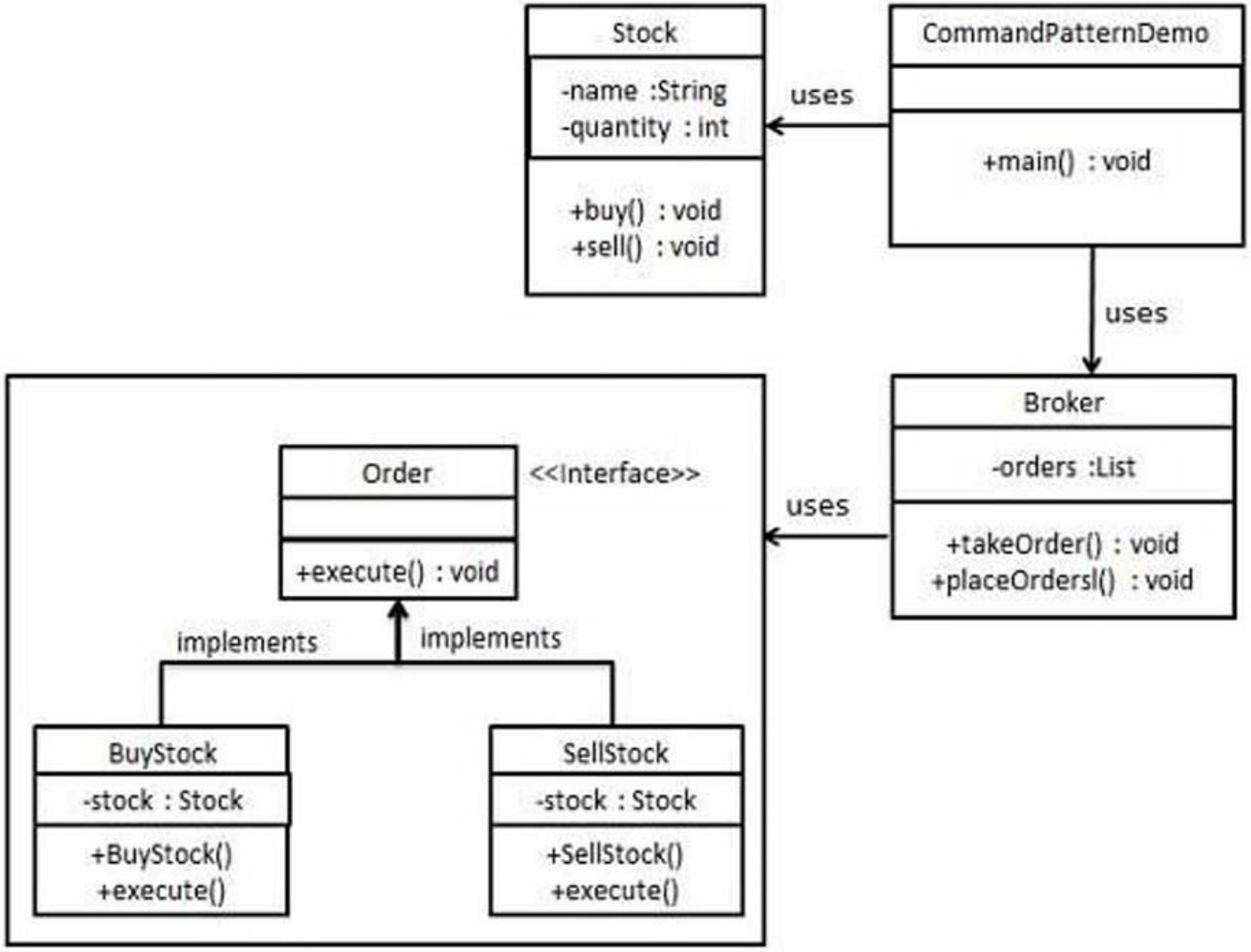
Then create the concrete command classes BuyStock and SellStock by implementing Order interface. These two classes do the actual command processing.

Create a class Broker which acts as an invoker object. It can take and place orders. Broker object uses command pattern to identify which object will execute which command based on the type of command.

Finally create a client CommandPatternDemo class for placing orders to buy and sell the stocks to demonstrate command pattern using broker.

Object Oriented Analysis and Design

UML Class Diagram



Command: Order

ConcreteCommand: BuyStock, SellStock

Client : CommandPatternDemo

Invoker: Broker

Receiver: Stock

Step 1 Create a command interface.
Order.java

```
public interface Order
{
   void execute();
}
```

Step 2 Create a request class or Receiver
Stock.java

```
public class Stock
{
   private String name = "ABC";
   private int quantity = 10;
   public void buy() {
      System.out.println("Stock [ Name: "+name+",  Quantity: " + quantity +" ] bought");
   }
   public void sell() {
      System.out.println("Stock [ Name: "+name+",  Quantity: " + quantity +" ] sold");
   }
  }
```

Step 3 Create concrete classes implementing the Order interface.

BuyStock.java

```
public class BuyStock implements Order
{
   private Stock abcStock;
   public BuyStock(Stock abcStock) {
       this.abcStock = abcStock;
   }
   public void execute() {
      abcStock.buy();
   }
}
```

SellStock.java

```
public class SellStock implements Order
{
    private Stock abcStock;
    public SellStock(Stock abcStock){
        this.abcStock = abcStock;
    }
    public void execute() {
        abcStock.sell();
    }
}
```

Step 4 Create command invoker class.
Broker.java

```
import java.util.ArrayList;
import java.util.List;
public class Broker
{
    private List<Order> orderList = new ArrayList<Order>();
    public void takeOrder(Order order){
       orderList.add(order);
    }
    public void placeOrders(){
       for (Order order : orderList){
          order.execute();
       }
       orderList.clear();
    }
}
```

Step 5 Client class. Use the Broker class to take and execute commands.
CommandPatternDemo.java

```
public class CommandPatternDemo
{
    public static void main(String[] args) {
        Stock abcStock = new Stock();
        BuyStock buyStockOrder = new BuyStock(abcStock);
        SellStock sellStockOrder = new SellStock(abcStock);

        Broker broker = new Broker();
        broker.takeOrder(buyStockOrder);
        broker.takeOrder(sellStockOrder);
        broker.placeOrders();
    }
}
```

PES UNIVERSITY

Output

```
Stock [ Name: ABC,  Quantity: 10 ] bought
Stock [ Name: ABC,  Quantity: 10 ] sold
```

Benefits and Drawbacks

Benefits:

- It decouples the classes that invoke the operation from the object that knows how to execute the operation
- It allows you to create a sequence of commands by providing a queue system
- Extensions to add a new command is easy and can be done without changing the existing code
- You can also define a rollback system with the Command pattern, for example, in the Wizard example, we could write a rollback method

Drawbacks:

- There are a high number of classes and objects working together to achieve a goal. Application developers need to be careful developing these classes correctly.
- Every individual command is a ConcreteCommand class that increases the volume of classes for implementation and maintenance.

PES UNIVERSITY

MCQ

1. **What is the primary intent of the Command Pattern?**

A. To reduce memory usage

B. To encapsulate object creation

C. To encapsulate a request as an object

D. To enforce inheritance

2. **Which component invokes the command execution?**

A. Invoker

B. Receiver

C. Client

D. ConcreteCommand

MCQ

3. **In the Command Pattern, which object knows how to perform the actual operation?**

A. Invoker

B. Client

C. Receiver

D. Command

4. **Which participant binds the receiver with an action?**

A. Invoker

B. ConcreteCommand

C. Client

D. Command

MCQ

5. **What does the execute() method typically do in a ConcreteCommand?**

A. Creates a new object

B. Calls another command

C. Invokes operation on the receiver

D. Stores commands in queue

6. **Which of the following is a key benefit of the Command Pattern?**

A. Reduces number of classes

B. Eliminates need for interfaces

C. Decouples sender and receiver

D. Avoids object creation

PES UNIVERSITY

MCQ SOLUTION

1.C
2.A
3.C
4.B
5.C
6.C

References

PES UNIVERSITY

Text Reference

Design Patterns: Elements of Reusable Object-Oriented Software, GOF

Web Reference

https://dzone.com/articles/design-patterns-command
https://refactoring.guru/design-patterns

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

Unit-4
Behavioural Patterns – Interpreter

Agenda

- What is an Interpreter design pattern?
- Intent and Motivation
- Applicability
- Design Solution
  - Structure
  - Example Implementation
- Advantages and Disadvantages

Interpreter design pattern

The Interpreter pattern is a **behavioral design pattern** that defines a way to interpret a domain-specific language or grammar. It allows you to define the grammar of a language and provide a way to evaluate sentences in that language.

Intend

1. Provide a way to interpret and execute expressions in that language.
2. Break down complex expressions into smaller components that can be easily interpreted.
3. Create parsers and compilers for the defined language.
4. Build scripting languages and other domain-specific languages.
5. Provide a flexible and extensible way to define and interpret expressions in the language.
6. Support the implementation of rules that govern the behavior of the language.
7. Facilitate the creation of abstract syntax trees to represent expressions in the language.
8. Allow for the creation of interpreters that can handle different types of expressions and operators.

Motivation

1. **Define a domain-specific language:** It allows developers to define a language that is specific to a particular domain or problem. This language can be tailored to meet the needs of a specific application, making it easier to write code that is expressive and concise.
2. **Interpret and execute expressions dynamically:** It provides a way to interpret and execute expressions at runtime. This is useful when dealing with complex expressions that are difficult or impossible to evaluate statically.
3. **Separate concerns:** It separates the concerns of defining a language and interpreting expressions in that language. This makes it easier to modify the language and add new expressions without affecting the interpreter code.

Motivation

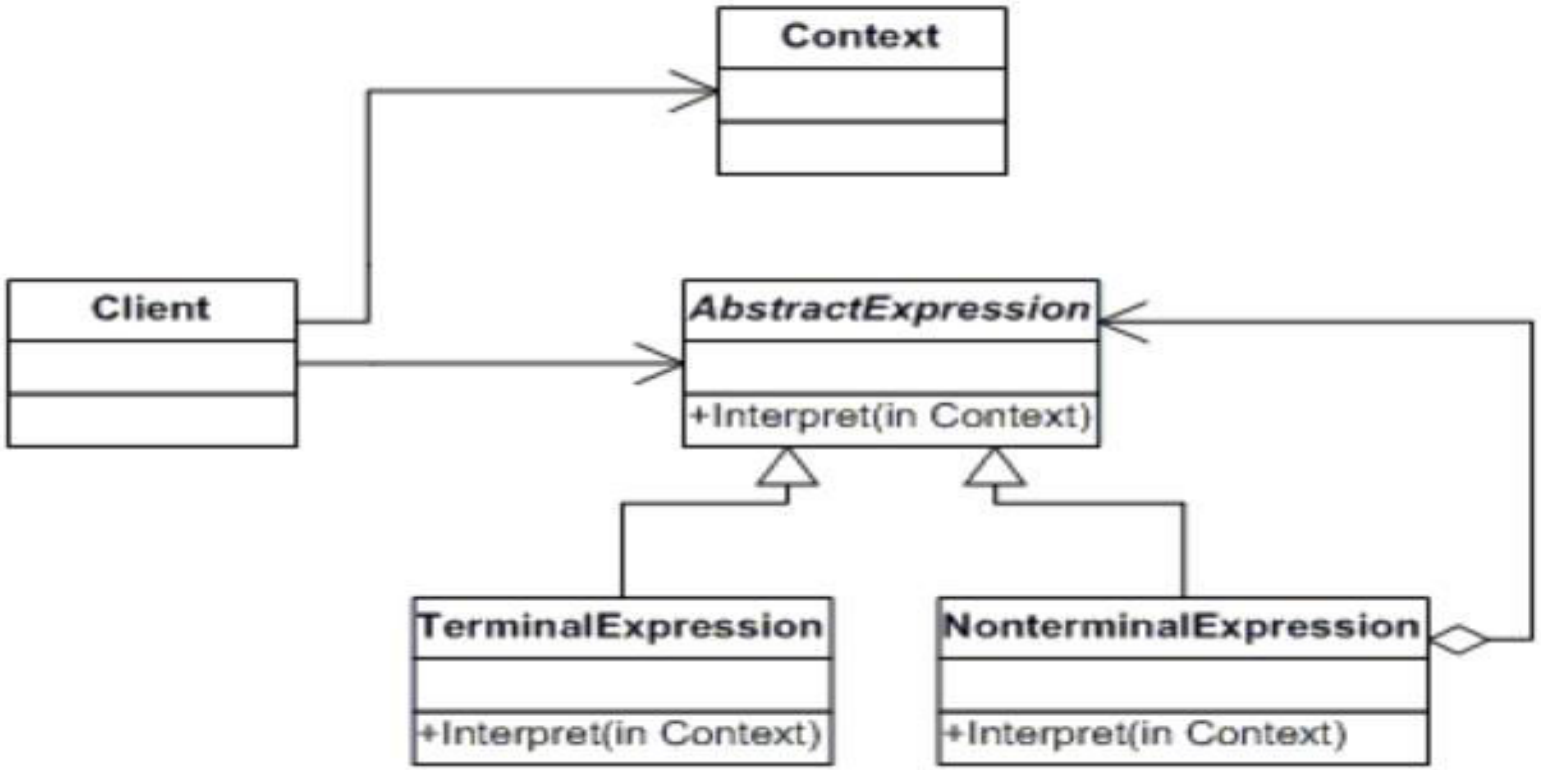
4. **Build parsers and compilers:**

The Interpreter pattern can be used to build parsers and compilers for the defined language. This allows the language to be used in a wide range of applications and contexts.

5. Facilitate communication between different components:

The Interpreter pattern can be used to facilitate communication between different components in a system. By defining a common language, different components can exchange messages and work together more easily.

Design Solution – Structure



Design Solution –Participants

AbstractExpression (Expression)

It declares an interface for executing an operation

TerminalExpression(ThousandExpression,HundredExpression,TenExpression, OneExpression)

It implements an Interpret operation associated with terminal symbols in the grammar.

An instance is required for every terminal symbol in the sentence.

NonterminalExpression (not used)

One such class is required for every rule R ::= R1R2...Rn in the grammar maintains instance variables of type AbstractExpression for each of the symbols R1 through Rn. Implements an Interpret operation for nonterminal symbols in the grammar. Interpret typically calls itself recursively on the variables representing R1 through Rn.

Design Solution –Participants

Context (Context)

It contains information that is global to the interpreter

Client (InterpreterApp)

It builds (or is given) an abstract syntax tree representing a particular sentence in the language that the grammar defines. The abstract syntax tree is assembled from instances of the NonterminalExpression and TerminalExpression classes

It invokes the Interpret operation

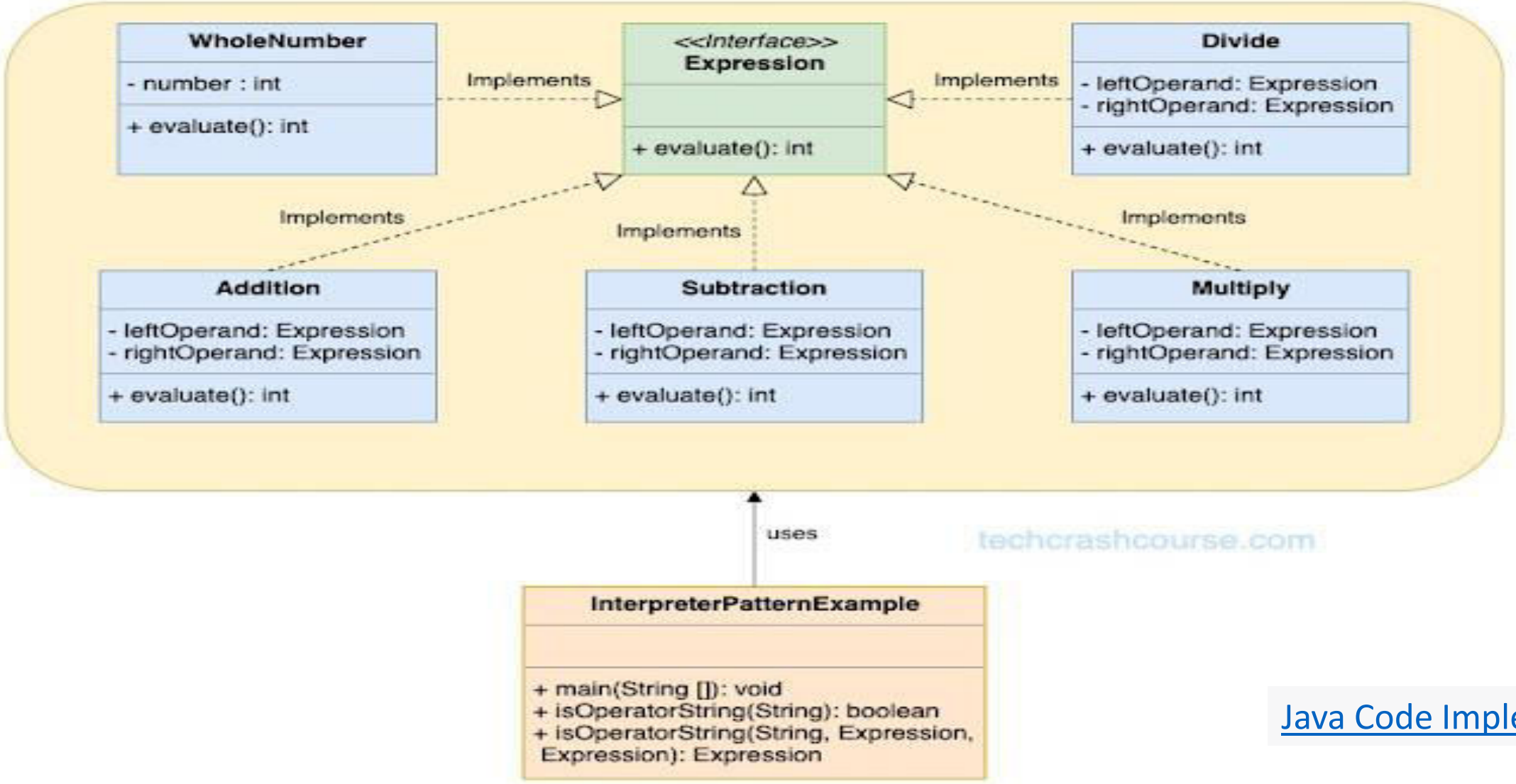
Example -Implementation

Let's take an example of an algebraic expression 2 + 3. To model this expression we need ConstantExpression that represents the constant operands(2 and 3) and then AddExpression that takes two ConstantExpression expressions operands and performs addition operation. All Expression classes must implement a common interface 'Expression'.

Above expression can Modelled as :
AddExpression(ConstantExpression(2), ConstantExpression(3))

Similarly, (2 + 6) + (1 + 9) can be modelled as:
AddExpression(
AddExpression(ConstantExpression(2), ConstantExpression(6)),
AddExpression(ConstantExpression(1), ConstantExpression(9))
)

Example -Implementation



Java Code Implementation

Applicability

- The Interpreter pattern is often used in situations where you need to interpret a domain-specific language or grammar, such as in a compiler or an interpreter for a scripting language.

- It can also be used to implement rule-based systems, where rules are expressed in a domain-specific language.

Applications

- **Regular Expressions:** Regular expressions are a powerful tool for matching patterns in strings. An interpreter can be used to parse and execute regular expressions, making it easier to build complex matching patterns.
- **SQL Queries:** SQL queries can be complex and difficult to write. An interpreter can be used to parse and execute SQL queries, allowing developers to build complex queries without having to understand the underlying implementation details.
- **Mathematical Expressions:** Mathematical expressions can be difficult to parse and evaluate. An interpreter can be used to parse and evaluate mathematical expressions, allowing you to build complex calculations and formulas.
- **Programming Languages:** Interpreters are commonly used in programming languages to execute code. They can be used to interpret scripts, run-time code, or compile code on-the-fly.

Advantages

- **Flexibility:** The Interpreter pattern provides a flexible way to implement complex grammars or expressions. It allows you to define your own language and parse sentences in that language.
- **Extensibility:** The Interpreter pattern makes it easy to add new functionality or behavior to your application. You can simply add new expressions or modify existing ones to change the behavior of your program.
- **Separation of Concerns:** The Interpreter pattern separates the parsing and execution logic, making it easier to maintain and extend your code.

Disadvantages

- **Complexity:** The Interpreter pattern can add complexity to your code, especially when dealing with complex grammars or expressions.
- **Performance:** The Interpreter pattern can be slower than other approaches, such as compiling or using a parser generator, since it requires interpreting the expressions at run-time.
- **Maintenance:** The Interpreter pattern can make your code more difficult to maintain, especially if the language or expressions change frequently.

Consequence

- The Interpreter pattern is a powerful tool that can make your code more flexible and extensible.
- However, it also comes with some trade-offs in terms of complexity, performance, and maintenance. It's important to consider these factors when deciding whether to use the Interpreter pattern in your application.

PES UNIVERSITY

MCQ

1. What is the primary purpose of the Interpreter design pattern?

A. To manage memory allocation
B. To interpret a domain-specific language or grammar
C. To create objects dynamically
D. To optimize database queries

2. Why is the Interpreter pattern useful for domain-specific languages?

A. It replaces compilers entirely
B. It allows developers to define a language specific to a problem domain
C. It removes the need for algorithms
D. It automatically optimizes performance

MCQ

3. What is the role of the Context in the Interpreter pattern?
A. Stores grammar rules
B. Executes the final program
C. Contains information global to the interpreter
D. Compiles expressions into machine code

4. In the Interpreter pattern, what does the Client typically do?
A. Executes the grammar rules directly
B. Builds an abstract syntax tree and invokes interpret
C. Creates memory buffers
D. Handles database connections

PES UNIVERSITY

MCQ

5. Which of the following is a typical application of the Interpreter pattern?

A. Regular expressions
B. Image compression
C. File encryption
D. Hardware drivers

6. What is a key advantage of the Interpreter pattern?

A. Reduces runtime execution completely
B. Provides flexibility to implement complex grammars
C. Eliminates the need for classes
D. Guarantees faster execution

PES UNIVERSITY

MCQ SOLUTION

1.B
2.B
3.C
4.B
5.A
6.B

References

- “Design Patterns: Elements of Reusable Object-Oriented Software” by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides.
- “Head First Design Patterns: A Brain-Friendly Guide” by Eric Freeman and Elisabeth Robson.
- “Design Patterns Explained: A New Perspective on Object-Oriented Design” by Alan Shalloway and James R. Trott.

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

UE23CS352B: Object Oriented Analysis and Design

Unit-4
Behavioral Patterns – Iterator

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements:
Prof. Vinay Joshi
Dept. of Computer Science and Engineering

Agenda

- What is an Iterator design pattern?
- Intent and Motivation
- Applicability
- Design Solution
  - Structure
  - Participants
  - Example Implementation
- Concequences
- Advantages and Disadvantages

Iterator design pattern

- The Iterator design pattern is a **behavioral pattern** that provides a way to access the elements of an aggregate object sequentially, without exposing its underlying implementation.

- In other words, it provides a uniform way to access the elements of a collection, such as a list, set, or map, without having to know how the collection is implemented.

Intend

- The intent of using an iterator is to provide a standardized way of accessing and manipulating collections of data in a simple, efficient, and flexible manner.
- It is useful for implementing algorithms that require sequential access to data, such as sorting, searching, and filtering. By using an iterator, you can easily apply these algorithms to any collection of data, regardless of its specific implementation or data structure

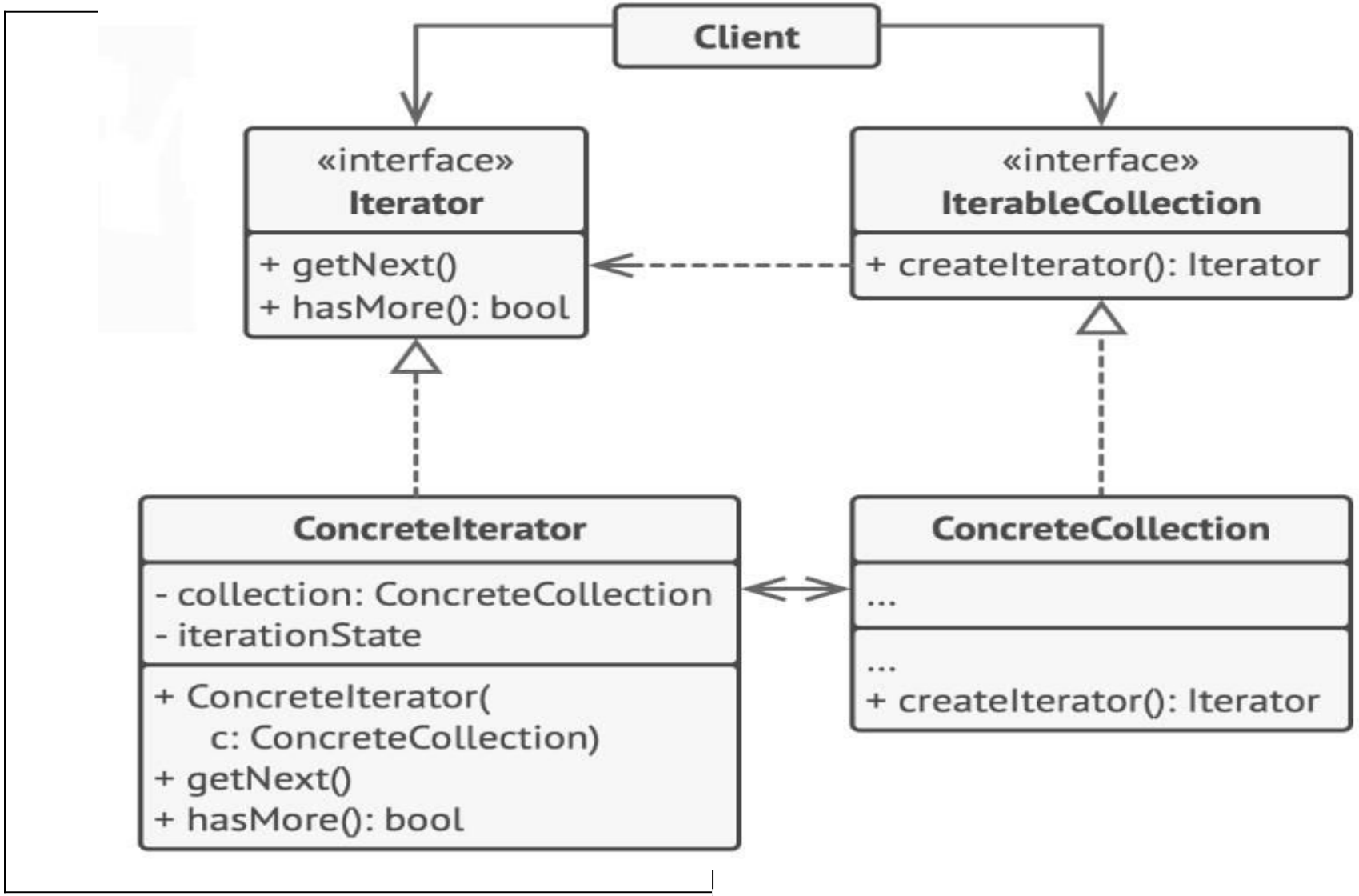
Motivation

- Encapsulation
- Abstraction
- Separation of concerns
- Lazy evaluation
- Multiple traversal support

Lazy evaluation

The Iterator pattern supports lazy evaluation of collections, where the elements of the collection are computed or retrieved only when they are needed. This can be especially useful for working with large datasets or data streams.

Design Solution – Structure



Design Solution –Participants

- **Iterator (AbstractIterator)**

Defines an interface for accessing and traversing elements.

- **ConcreteIterator (Iterator)**

Implements the Iterator interface.
Keeps track of the current position in the traversal of the aggregate.

- **Aggregate (AbstractCollection)**

Defines an interface for creating an Iterator object

- **ConcreteAggregate (Collection)**

Implements the Iterator creation interface to return an instance of the proper ConcreteIterator

How to Implement

1. **Declare the iterator interface.** At the very least, it must have a method for fetching the next element from a collection. But for the sake of convenience you can add a couple of other methods, such as fetching the previous element, tracking the current position, and checking the end of the iteration.
2. **Declare the collection interface and describe a method for fetching iterators.** The return type should be equal to that of the iterator interface. You may declare similar methods if you plan to have several distinct groups of iterators.
3. **Implement concrete iterator classes for the collections that you want to be traversable with iterators.** An iterator object must be linked with a single collection instance. Usually, this link is established via the iterator's constructor.
4. **Implement the collection interface in your collection classes.** The main idea is to provide the client with a shortcut for creating iterators, tailored for a particular collection class. The collection object must pass itself to the iterator's constructor to establish a link between them.
5. Go over the client code to **replace all of the collection traversal code with the use of iterators.** The client fetches a new iterator object each time it needs to iterate over the collection elements.

Consequence

Consequences of using the Iterator design pattern are:

1. **Improved code flexibility:** The Iterator pattern allows clients to iterate over collections in different ways without changing the underlying collection's structure. This improves code flexibility and makes it easier to add new iteration methods in the future.
2. **Better encapsulation:** By providing a separate object to handle iteration, the internal details of the collection are hidden from the client code. This improves encapsulation and helps to prevent code from becoming tightly coupled.
3. **Simplified client code:** The Iterator pattern simplifies the client code by providing a standardized way to access collection elements. This reduces the complexity of the client code and makes it easier to read and maintain.
4. **Performance overhead:** The Iterator pattern may introduce a performance overhead, as the iteration process involves additional method calls and object creations. However, this overhead is typically negligible for small collections.

Overall, the Iterator design pattern is a useful tool for improving the flexibility, encapsulation, and maintainability of code that involves iterating over collections.

Applicability

- Accessing elements in a collection
- Simplifying complex data structures
- Implementing lazy evaluation
- Supporting multiple traversals
- Providing a standard interface

Accessing elements in a collection

The Iterator pattern is used to provide a simple and efficient way to access and iterate over the elements in a collection, such as an array or a list.

Advantages

- *Single Responsibility Principle*. You can clean up the client code and the collections by extracting bulky traversal algorithms into separate classes.

- *Open/Closed Principle*. You can implement new types of collections and iterators and pass them to existing code without breaking anything.

- You can iterate over the same collection in parallel because each iterator object contains its own iteration state.

- For the same reason, you can delay an iteration and continue it when needed.

Disadvantages

- Applying the pattern can be an overkill if your app only works with simple collections.

- Using an iterator may be less efficient than going through elements of some specialized collections directly.

PES UNIVERSITY

MCQ

1. What is the main purpose of the Iterator design pattern?

A. To create objects dynamically
B. To provide sequential access to elements of an aggregate object without exposing its implementation
C. To manage memory allocation
D. To optimize database queries

2. What does the Iterator pattern provide for collections such as lists or sets?

A. Direct memory management
B. A uniform way to access collection elements
C. Automatic sorting of collections
D. Encryption of collection data

PES UNIVERSITY

MCQ

3. What is the role of the ConcreteIterator?
A. Stores database records
B. Implements the iterator interface and tracks traversal position
C. Creates collections dynamically
D. Executes system commands

4. What does the Aggregate component define in the Iterator pattern?
A. Interface for creating an iterator object
B. Methods for storing data
C. Methods for encryption
D. Rules for compiling programs

PES UNIVERSITY

MCQ

5. Which concept does the Iterator pattern support when dealing with large datasets?

A. Lazy evaluation
B. Static memory allocation
C. Binary compilation
D. Network caching

6. What advantage does the Iterator pattern provide regarding encapsulation?

A. It removes encapsulation completely
B. It hides internal details of the collection from client code
C. It exposes internal structure of collections
D. It forces direct data access

PES UNIVERSITY

MCQ SOLUTION

1.B
2.B
3.B
4.A
5.A
6.B

References

- “Design Patterns: Elements of Reusable Object-Oriented Software” by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides.
- “Head First Design Patterns: A Brain-Friendly Guide” by Eric Freeman and Elisabeth Robson.
- “Design Patterns Explained: A New Perspective on Object-Oriented Design” by Alan Shalloway and James R. Trott.

PES UNIVERSITY

CELEBRATING 50 YEARS

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

shridevias@pes.edu

Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

PES UNIVERSITY

UE22CS352B: Object Oriented Analysis and Design

Anti-Patterns - Architecture and Design Anti-Patterns

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements:
Prof. Priya Badarinath
Dept. of Computer Science and Engineering

Definitions

- Pattern: Good ideas
- Antipattern: Bad ideas
- Refactoring: Better ideas

What is an Antipattern?

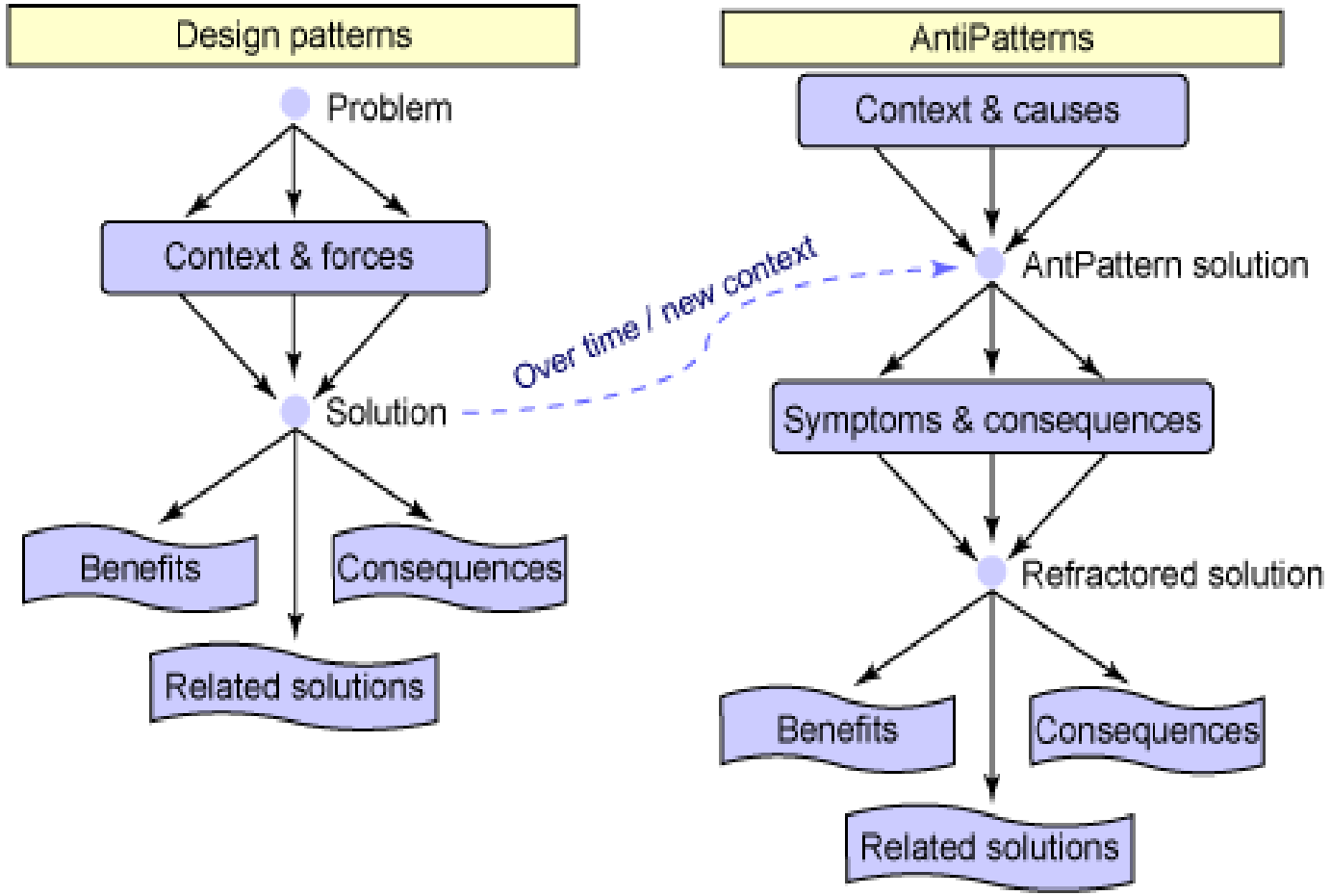
- An AntiPattern is a literary form that describes a commonly occurring solution to a problem that generates decidedly negative consequences.
- The AntiPattern may be the result of a manager or developer not knowing any better, not having sufficient knowledge or experience in solving a particular type of problem, or having applied a perfectly good pattern in the wrong context.

What are AntiPatterns?

- “Negative Solutions,” or solutions that present more problems than they address.
- Natural extensions to design patterns.
- Bridge the gap between architectural concepts and real-world implementations.
- Provide Knowledge to prevent and recover from common Mistakes.
- Presents a detailed plan for reversing the underlying causes and implementing productive solutions.
- Effectively describe the measures that can be taken at several levels to improve the developing of applications, the designing of software systems, and the effective management of software projects.

Antipatterns Vs. Design Patterns

- **Design Pattern-** a general repeatable solution to a commonly occurring problem
- **Antipattern-** Such a solution which is recognized as a poor way to solve the problem, and a refactored solution



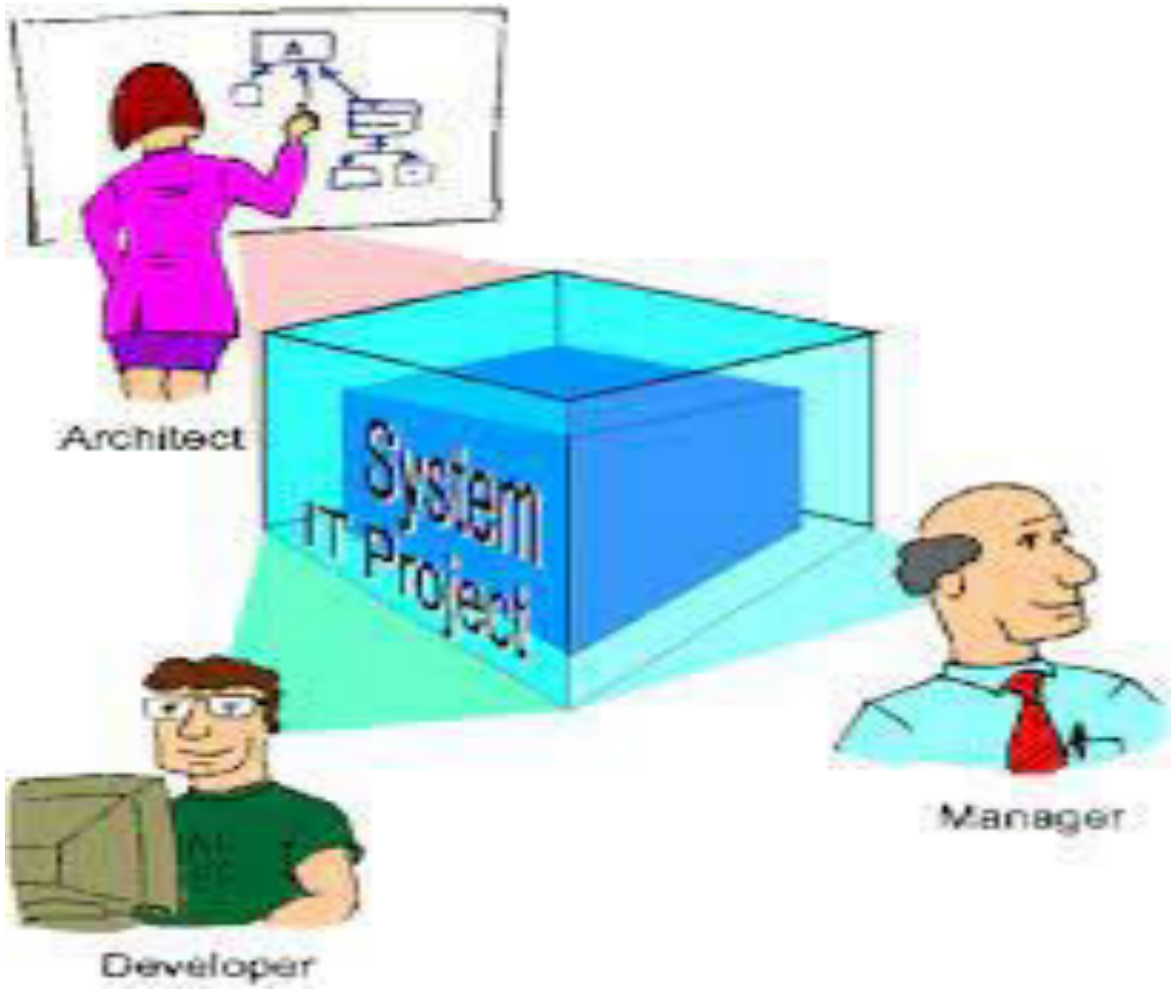
Why study Antipatterns?

- Antipatterns provide easily identifiable templates for common problems, as well as a path of action to rectify these problems.
- Antipatterns provide real world experience in recognizing recurring problems in the software industry providing a detailed remedy for the most common ones.
- Antipatterns provide a common vocabulary for identifying problems and discussing solutions.
- Antipatterns provide stress release in the form of shared misery.
- Antipatterns ensure common problems are not continually repeated within an organization

Viewpoints

AntiPatterns from three major viewpoints:

- The software developer,
- The software architect,
- The software manager



Principal AntiPattern viewpoints

Viewpoints

Development AntiPatterns describe situations encountered by the programmer when solving programming problems.

Architectural AntiPatterns focus on common problems in system structure, their consequences, and solutions. Many of the most serious unresolved problems in software systems occur from this perspective.

Management AntiPatterns describe common problems and solutions due to the software organization. It affects people in all software roles, and their solutions directly affect the technical success of the project.

Reference Model

A reference model for terminology common to all three viewpoints. The reference model is based upon three topics that introduce the key concepts of AntiPatterns:

- Root causes

- Provide fundamental context for the AntiPattern

- Primal forces

- are the key motivators for decision making

- Software design-level model (SDLM)

- Define architectural scale

Reference Model – Root Causes

Root Causes

- Root causes are mistakes in software development that result in failed projects, cost overruns, schedule slips, and unfulfilled business needs.
- The root causes are based upon the "seven deadly sins," a popular analogy that has been used successfully to identify ineffective practices.

Haste: Tight deadlines often lead to neglecting important activities

Apathy: The attitude of not caring about solving known problems.

Narrow-Mindedness: Refusal of developers to learn proven solutions.

Root Causes (Contd.,)

Sloth: adaptation of the most simple “solution”.

Avarice: Greed in creating a system can result in very complex, and difficult to maintain software.

Ignorance: The lack of motivation to understand things.

Pride: The Failure to reuse existing software packages because they were not invented by a specific company

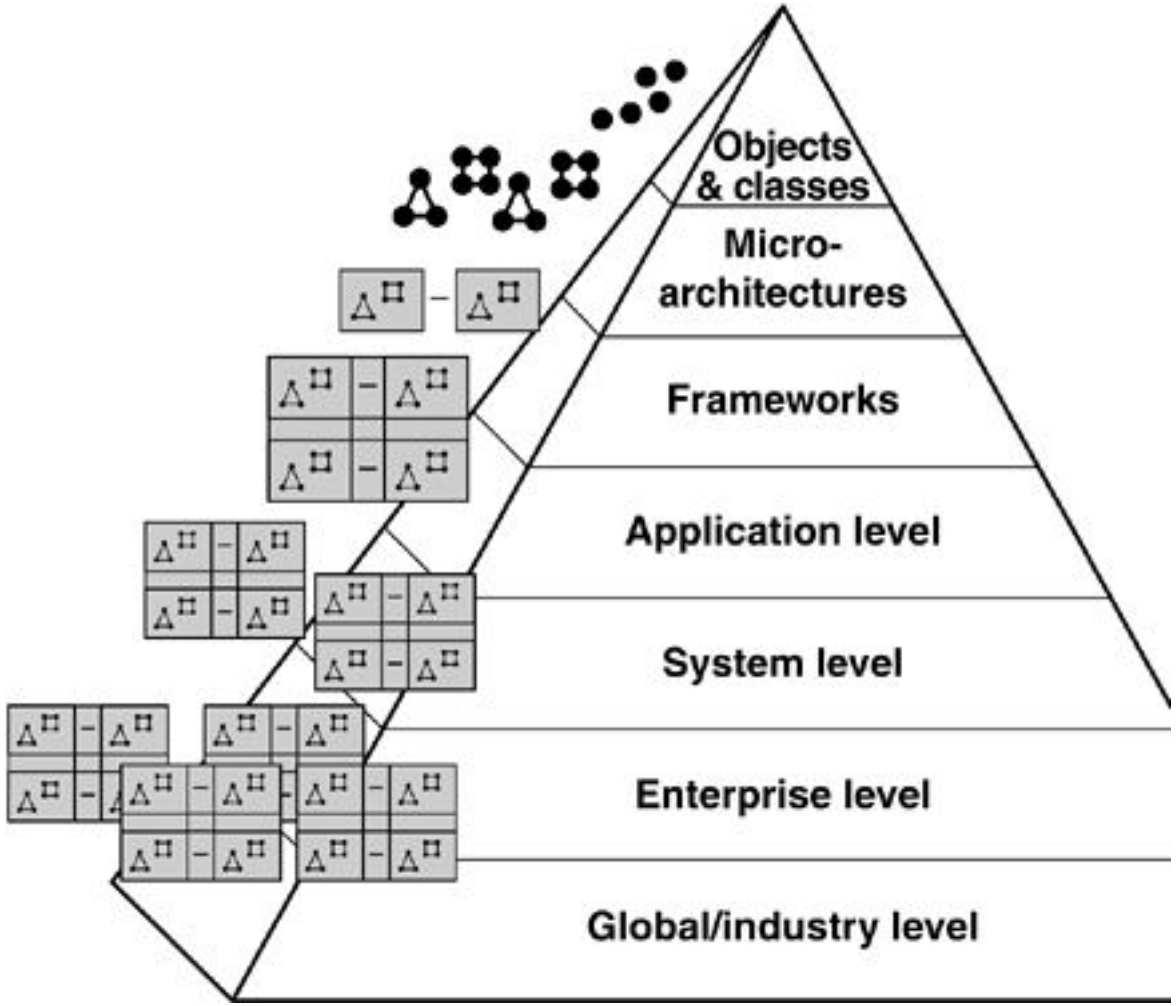
Primal Forces

Forces are concerns or issues that exist within a decision–making context. In a design solution, forces that are successfully addressed (or resolved) lead to benefits, and forces that are unresolved lead to consequences.

The primal forces include:

- Management of Functionality - Meeting the requirements
- Management of Performance - Meeting required speed and operation
- Management of Complexity - Defining abstractions
- Management of Change - Controlling the evolution of software
- Management of IT Resources - Managing people and IT artifacts
- Management of Technology Transfer - Controlling technology evolution

Software Design – Level Model



Software Design – Level Model (Contd.,)

- **The global level** contains the design issues that are globally applicable across all systems. This level is concerned with coordination across all organizations, which participate in cross−organizational communications and information sharing.
- **The enterprise level** is focused upon coordination and communication across a single organization. The organization can be distributed across many locations and heterogeneous hardware and software systems.
- **The system level** deals with communications and coordination across applications and sets of applications.
- **The application level** is focused upon the organization of applications developed to meet a set of user requirements. The macro−component levels are focused on the organization and development of application frameworks.

Software Design – Level Model (Contd.,)

- **The micro−component level** is centered on the development of software components that solve recurring software problems. Each solution is relatively self−contained and often solves just part of an even larger problem.
- **The object level** is concerned with the development of reusable objects and classes. The object level is more concerned with code reuse than design reuse. Each of the levels is discussed in detail along with an overview of the patterns documented at each level.

PES UNIVERSITY

MCQ

1. What is an AntiPattern?

A. A reusable design solution
B. A coding standard
C. A solution that produces negative consequences
D. A testing methodology

2. What is the relationship between design patterns and antipatterns?

A. Both are good solutions
B. Antipatterns are poor solutions compared to design patterns
C. Design patterns are always incorrect
D. They are unrelated concepts

PES UNIVERSITY

MCQ

3. What do Development AntiPatterns describe?
A. Hardware failures
B. Programming-level issues faced by developers
C. Network communication
D. Database indexing

4. What are root causes in AntiPatterns?
A. Final solutions
B. Performance metrics
C. Mistakes leading to project failures
D. Testing tools

PES UNIVERSITY

MCQ SOLUTION

1.C
2.B
3.B
4.C

References

- https://sourcemaking.com/antipatterns/
- Antipatterns-Refactoring-Software-Architectures-and-Proj.pdf

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Object Oriented Analysis and Design

UE23CS352B

Prof. Shridevi A Sawant

Department of Computer Science and Engineering

Acknowledgements: Significant portions of the information in the slide sets presented through the course in the class, are extracted from the prescribed text books, information from the Internet. Since these are only intended for presentation for teaching within PESU, there was no explicit permission solicited. We would like to sincerely thank and acknowledge that the credit/rights remain with the original authors/creators only

PES UNIVERSITY

UE23CS352B: Object Oriented Analysis and Design

Anti-Patterns - Architecture and Design Anti-Patterns

Prof. Shridevi A Sawant
Department of Computer Science and Engineering

Acknowledgements:
Dr. Geetha D and Swati Shah
Dept. of Computer Science and Engineering

Project Management AntiPatterns

In the modern engineering profession, more than half of the job involves human communication and resolving people issues. The management AntiPatterns identify some of the key scenarios in which these issues are destructive to software processes.

The areas where managers play vital role,

1. Software process management
2. Resource management (human & IT infrastructure)
3. External relationship management (e.g., customers, development partners)

Project Management AntiPatterns – Analysis Paralysis

PES UNIVERSITY

Symptoms and Consequences:

- There are multiple project restarts and much model rework, due to personnel changes or changes in project direction.
- Design and implementation issues are continually reintroduced in the analysis phase.
- Cost of analysis exceeds expectation without a predictable end point.
- The analysis phase no longer involves user interaction. Much of the analysis performed is speculative.
- The complexity of the analysis models results in intricate implementations, making the system difficult to develop, document, and test.
- Design and implementation decisions such as those used in the Gang of Four design patterns are made in the analysis phase.

Project Management AntiPatterns – Analysis Paralysis

PES UNIVERSITY

Typical Causes:

- The management process assumes a waterfall progression of phases. In reality, virtually all systems are built incrementally even if not acknowledged in the formal process.
- Management has more confidence in their ability to analyze and decompose the problem than to design and implement.
- Management insists on completing all analysis before the design phase begins.
- Goals in the analysis phase are not well defined.
- Planning or leadership lapses when moving past the analysis phase.
- Management is unwilling to make firm decisions about when parts of the domain are sufficiently described.
- The project vision and focus on the goal/deliverable to customer is diffused. Analysis goes beyond providing meaningful value.

Project Management AntiPatterns – Analysis Paralysis

Refactored Solution:

- Key to the success of object-oriented development is incremental development. Whereas a waterfall process assumes a priori knowledge of the problem, incremental development processes assume that details of the problem and its solution will be learned in the course of the development process.

- There are two kinds of increments: internal and external. An internal increment builds software that is essential to the infrastructure of the implementation. For example, a third-tier database and data-access layer would comprise an internal increment. Internal increments build a common infrastructure that is utilized by multiple use cases. In general, internal increments minimize rework. An external increment comprises user-visible functionality.

Software Architecture Antipatterns

- Architecture AntiPatterns focus on the system-level and enterprise-level structure of applications and components.
- Although the engineering discipline of software architecture is relatively immature, what has been determined repeatedly by software research and experience is the overarching importance of architecture in software development.

The following AntiPatterns focus on some common problems and mistakes in the creation, implementation, and management of architecture.

Software Architecture Antipatterns – Vendor Lock-In

Vendor Lock-In

- A software project adopts a product technology and becomes completely dependent upon the vendor's implementation. When upgrades are done, software changes and interoperability problems occur, and continuous maintenance is required to keep the system running.

- In addition, expected new product features are often delayed, causing schedule slips and an inability to complete desired application software features.

Software Architecture Antipatterns – Vendor Lock-In

Vendor Lock-In – Symptoms and Consequences

- Commercial product upgrades drive the application software maintenance cycle.
- Promised product features are delayed or never delivered, subsequently, causing failure to deliver application updates.
- The product varies significantly from the advertised open systems standard.
- If a product upgrade is missed entirely, a product repurchase and reintegration is often necessary.

Software Architecture Antipatterns – Vendor Lock-In

Vendor Lock-In – Typical Causes

- The product varies from published open system standards because there is no effective conformance process for the standard.
- **The product is selected based entirely upon marketing and sales information, and not upon more detailed technical inspection.**
- There is no technical approach for isolating application software from direct dependency upon the product.
- Application programming requires in-depth product knowledge.
- The complexity and generality of the product technology greatly exceeds that of the application needs; direct dependence upon the product results in failure to manage the complexity of the application system architecture.

Software Architecture Antipatterns – Vendor Lock-In

Vendor Lock-In – Refactored Solution

The refactored solution to the Vendor Lock-In AntiPattern is called *isolation layer.* An isolation layer separates software packages and technology. This solution is applicable when one or more of the following conditions apply:

- *Isolation of application software from lower-level infrastructure.* This infrastructure may include middleware, operating systems, security mechanisms, or other low-level mechanisms.
- *Changes to the underlying infrastructure are anticipated within the life cycle of the affected software;* for example, new product releases or planned migration to new infrastructure.
- *A more convenient programming interface is useful or necessary.* The level of abstraction provided by the infrastructure is either too primitive or too flexible for the intended applications and systems.
- *There is a need for consistent handling of the infrastructure across many systems.* Some heavyweight conventions for default handling of infrastructure interfaces must be instituted.
- Multiple infrastructures must be supported, either during the life cycle or concurrently.

Software Development Antipatterns

Good software structure is essential for system extension and maintenance. Software development is a chaotic activity, therefore the implemented structure of systems tends to stray from the planned structure as determined by architecture, analysis, and design.

Software Development Antipatterns – The Blob

The Blob

Procedural-style design leads to one object with a lion's share of the responsibilities, while most other objects only hold data or execute simple processes. The solution includes refactoring the design to distribute responsibilities more uniformly and isolating the effect of changes.

Background:

The Blob is found in designs where one class monopolizes the processing, and other classes primarily encapsulate data. This AntiPattern is characterized by a class diagram composed of a single complex controller class surrounded by simple data classes. The key problem here is that the majority of the responsibilities are allocated to a single class

Software Development Antipatterns – The Blob

Symptoms and Consequences

- Single class with a large number of attributes, operations, or both. A class with 60 or more attributes and operations usually indicates the presence of the Blob.
- A disparate collection of unrelated attributes and operations encapsulated in a single class. An overall lack of cohesiveness of the attributes and operations is typical of the Blob.
- The Blob Class is typically too complex for reuse and testing. It may be inefficient, or introduce excessive complexity to reuse the Blob for subsets of its functionality.
- The Blob Class may be expensive to load into memory, using excessive resources, even for simple operations.

Software Development Antipatterns – The Blob

Typical Causes

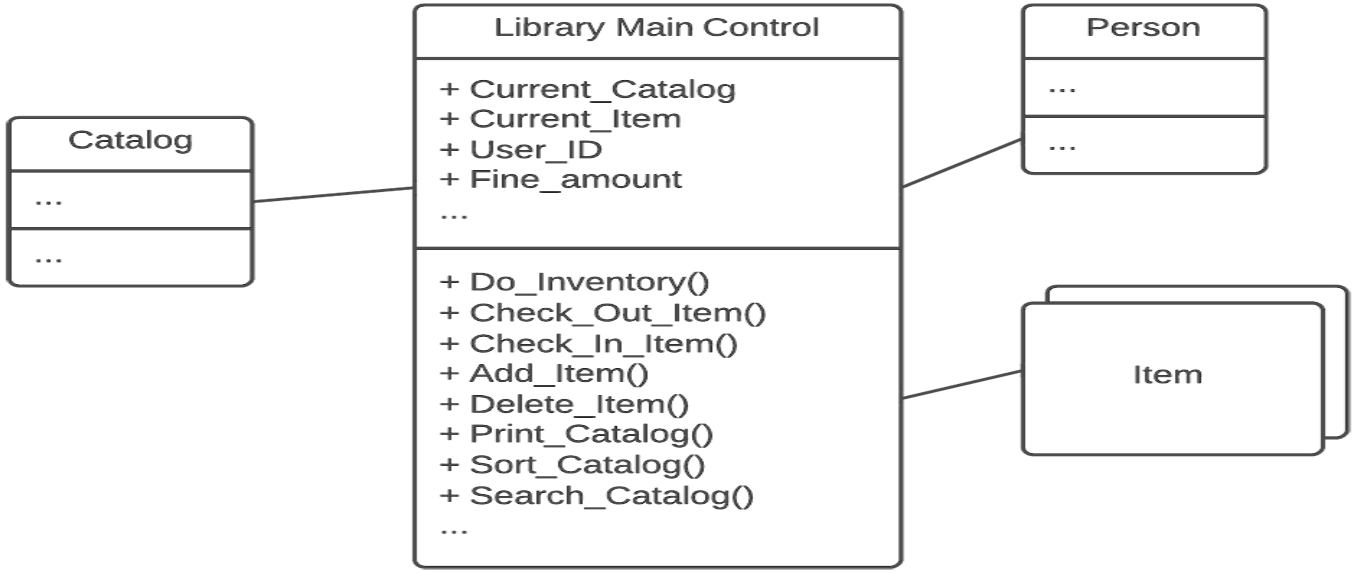
- Lack of an object-oriented architecture.
- Lack of (any) architecture.
- Lack of architecture enforcement.
- Too limited intervention.
- Specified disaster.

Software Development Antipatterns – The Blob

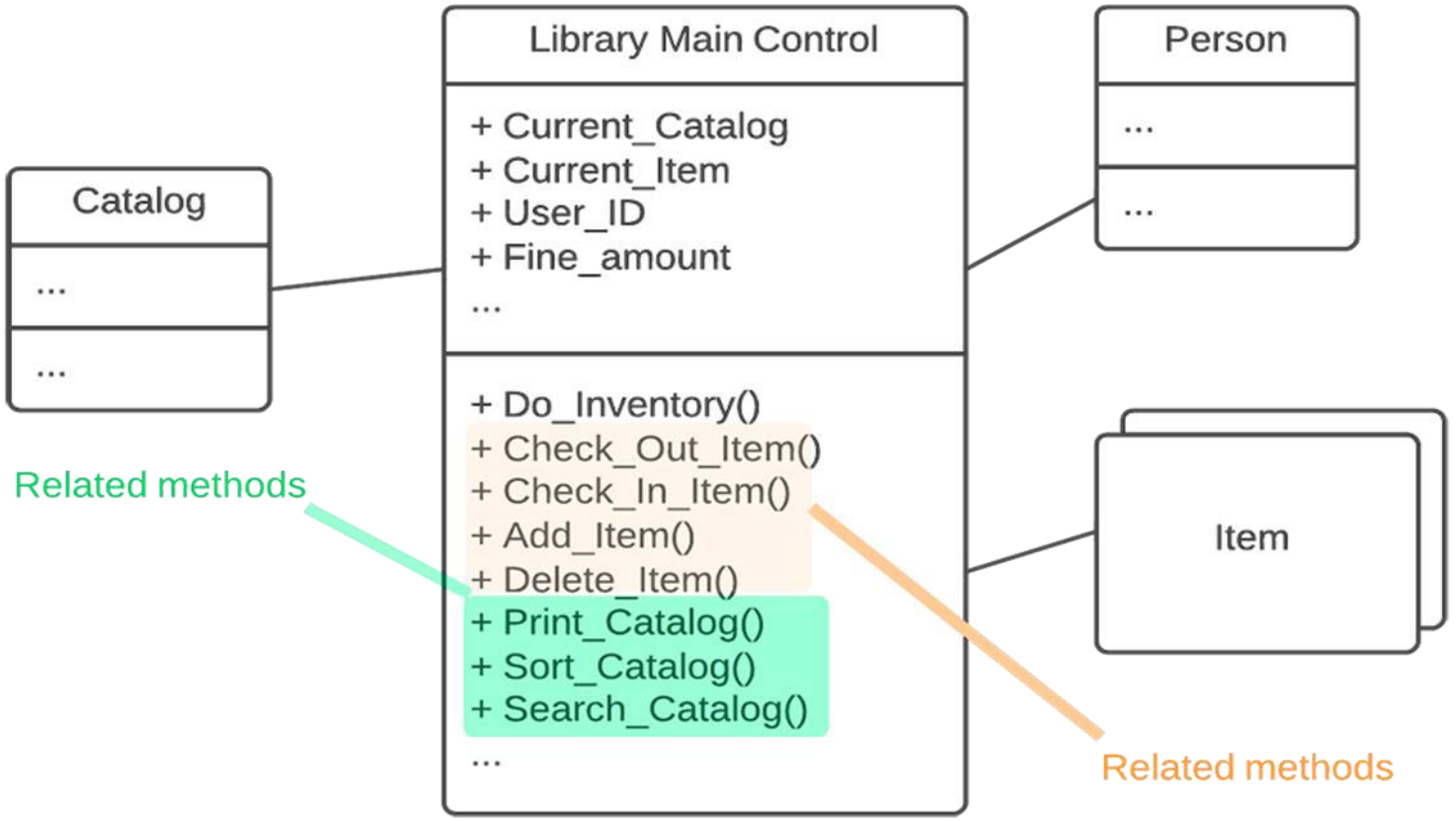
Refactored Solution

Step 1:

- Identify or categorize related attributes and operations according to contracts. These contracts should be cohesive in that they all directly relate to a common focus, behavior, or function within the overall system.



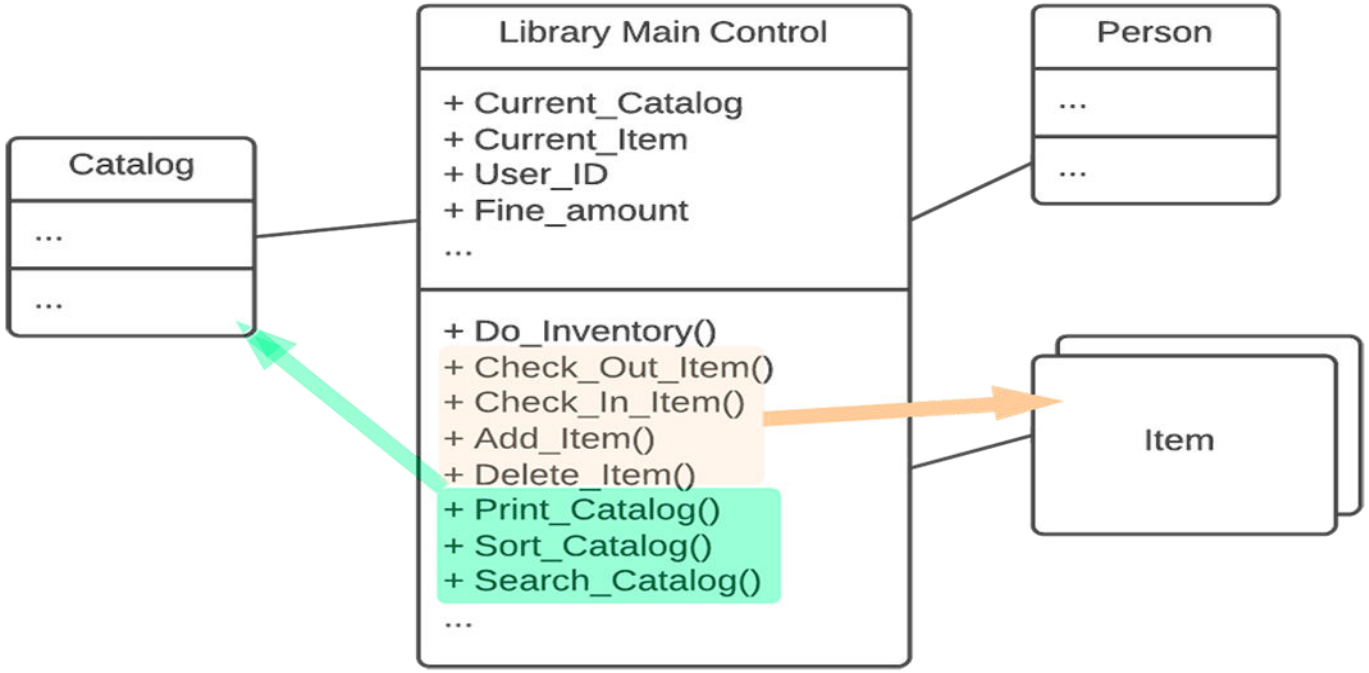
Software Development Antipatterns – The Blob



Software Development Antipatterns – The Blob

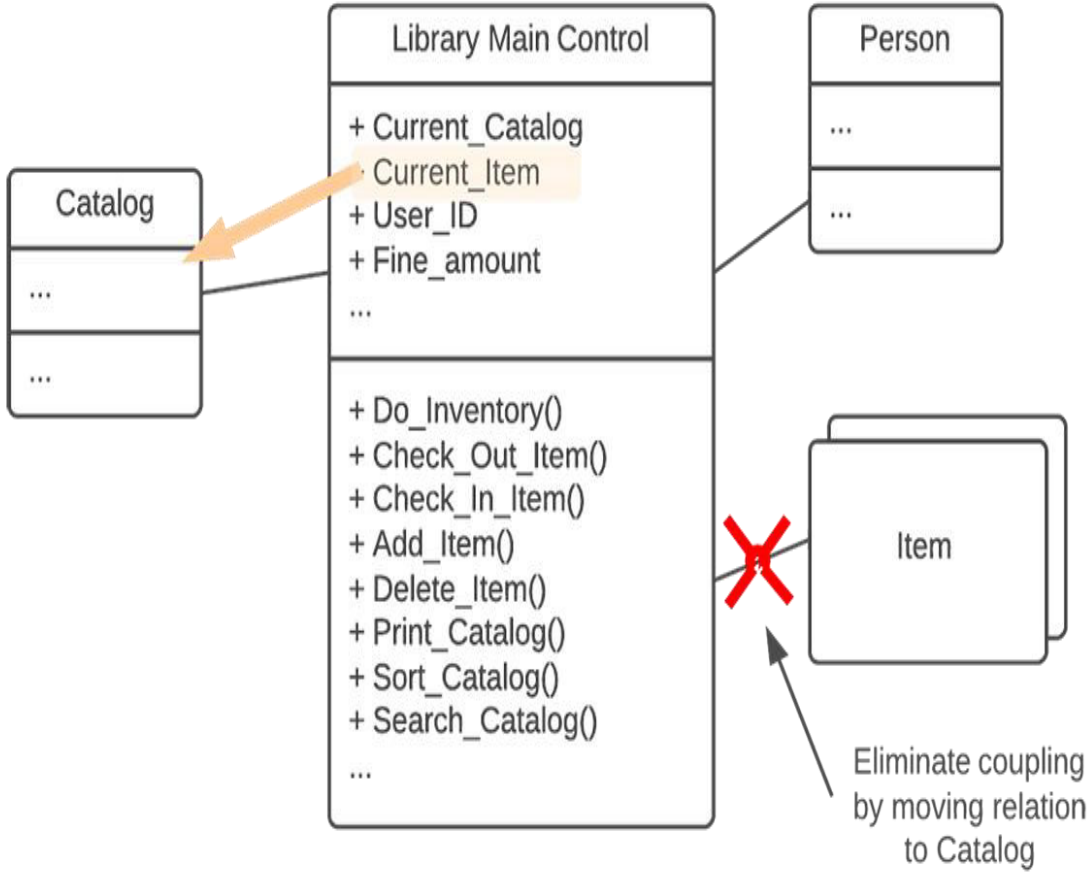
Step 2:

- Look for "natural homes" for these contract-based collections of functionality and then migrate them there. In this example, we gather operations related to catalogs and migrate them from the LIBRARY class and move them to the CATALOG class.



Software Development Antipatterns – The Blob

- The third step is to remove all "far-coupled," or redundant, indirect associations. In the example, the ITEM class is initially far-coupled to the LIBRARY class in that each item really belongs to a CATALOG, which in turn belongs to a LIBRARY.
- Next, where appropriate, we migrate associates to derived classes to a common base class. In the example, once the far-coupling has been removed between the LIBRARY and ITEM classes, we need to migrate ITEMs to CATALOGs,
- Finally, we remove all transient associations, replacing them as appropriate with type specifiers to attributes and operations arguments.



PES UNIVERSITY

MCQ

1. What do Project Management AntiPatterns primarily deal with?

A. Human communication and management issues
B. Database optimization
C. Memory allocation
D. Algorithm design

2. What is a major cause of Analysis Paralysis?

A. Incremental development
B. Strong design phase
C. Insistence on completing all analysis before design
D. Agile methodology

PES UNIVERSITY

MCQ

3. What is Vendor Lock-In?
A. Use of open standards
B. Dependency on a specific vendor’s technology
C. Using multiple vendors
D. Avoiding external tools

4. What problem occurs due to Vendor Lock-In?
A. Reduced maintenance
B. Faster updates
C. Interoperability issues and maintenance problems
D. Simpler systems

PES UNIVERSITY

MCQ

5. What is the Blob AntiPattern?
A. Many small classes with clear responsibilities
B. One class handling most responsibilities
C. Distributed system design
D. Modular architecture

6. What is a key issue with the Blob AntiPattern?
A. Easy testing
B. Efficient reuse
C. Low complexity
D. Difficult reuse and high complexity

PES UNIVERSITY

MCQ SOLUTION

1.A
2.C
3.B
4.C
5.B
6.D

References

- https://sourcemaking.com/antipatterns/
- Antipatterns-Refactoring-Software-Architectures-and-Proj.pdf

THANK YOU

Prof. Shridevi A Sawant

Department of Computer Science and Engineering